

**LAPORAN AKHIR PRAKTIKUM
APLIKASI WEB DAN MOBILE
PERTEMUAN 1 – 15**



DISUSUN OLEH :

Maessa Andrea Vallenia

23051430044

**PROGRAM STUDI TEKNIK INDUSTRI
FAKULTAS TEKNIK
UNIVERSITAS NEGERI YOGYAKARTA
2026**

LEMBAR PENGESAHAN
LAPORAN AKHIR PRAKTIKUM APLIKASI WEB DAN MOBILE
LAPORAN AKHIR PRAKTIKUM APLIKASI WEB DAN MOBILE
PERTEMUAN 1 – 15
Maessa Andrea Vallenia
NIM. 23051430044

Laporan ini Disusun Guna Memenuhi Salah Satu Persyaratan
Menempuh Mata Kuliah Aplikasi Web dan Mobile
Program Studi Teknik Industri
Fakultas Teknik UNY

Menyetujui/Mengesahkan :

Dosen Pengampu,

Mahasiswa,

Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT
NIP. 198103172005011001

Maessa Andrea Vallenia
NIM. 23051430044

DAFTAR ISI

MODUL 1 PENGENALAN PEMROGRAMAN WEB & HTML DASAR.....	4
MODUL 2 CSS DASAR & STYLING	24
MODUL 3 LAYOUT CSS LANJUTAN (FLEXBOX & GRID) + BOOTSTRAP	47
MODUL 4 JAVASCRIPT DASAR (VARIABEL, TIPE DATA, OPERATOR, CONTROL FLOW).....	73
MODUL 5 ARRAY, OBJECT DI JAVASCRIPT	127
MODUL 6 DOM MANIPULATION & EVENT	170
MODUL 7 FORM VALIDATION & LOCAL STORAGE	217
MODUL 8 ASYNCHRONOUS JAVASCRIPT (FETCH API, JSON, REST API SEDERHANA	262
MODUL 9 PENGENALAN REACT.JS (COMPONENT, JSX, PROPS)	303
MODUL 10 STATE, HOOKS (USESTATE, USEEFFECT), DAN CONDITIONAL RENDERING.....	328
MODUL 11 REACT ROUTING & MULTI-PAGE APP + INTEGRASI API	359
MODUL 12 PROYEK WEB: DASHBOARD MONITORING PRODUKSI (REACT+CHART.JS).....	398
MODUL 14 PENGENALAN REACT NATIVE & SETUP EXPO	446
MODUL 15 COMPONENT, NAVIGATION, DAN STYLING DI REACT NATIVE + PROYEK MOBILE.....	486

MODUL 1

PENGENALAN PEMROGRAMAN WEB & HTML DASAR

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 1 (Pertemuan 1)
Topik Utama	Pengenalan Pemrograman Web & HTML Dasar
Sub-Topik	Struktur dokumen HTML, perancangan halaman web dasar, dan pembuatan formulir input
Durasi Praktikum	340 Menit
Tanggal Praktikum	13 Februari 2026
Nama Mahasiswa	Maessa Andrea Vallenia
NIM	23051430044
Kelas	A

B. TUJUAN PEMBELAJARAN

Tuliskan tujuan pembelajaran yang ingin dicapai pada modul ini, sesuai dengan yang tertera di Manual Praktikum.

1	Memahami peran penting web application dalam sistem informasi industri.
2	Mampu menjelaskan struktur dasar HTML dan cara kerjanya di browser.
3	Mampu membuat halaman web statis dengan elemen teks, gambar, dan tabel.
4	Mampu menyusun formulir input data sederhana (misal: data produksi).

C. ALAT DAN BAHAN

Sebutkan seluruh perangkat keras, perangkat lunak, dan sumber referensi yang digunakan selama praktikum.

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	Ekstensi VS Code	Live Server	Menjalankan HTML dan memantau

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
			perubahan file
4	Ekstensi VS Code	<i>CodeSnap</i>	Dokumentasi kode program

D. DASAR TEORI

Tuliskan ringkasan konsep/teori yang menjadi landasan pada modul ini. Ditulis dengan bahasa sendiri, bukan salinan mentah dari modul. Minimal 3 paragraf.

D.1 Konsep Utama

HTML (*HyperText Markup Language*) merupakan bahasa markup fundamental yang digunakan untuk membangun struktur sebuah halaman web. HTML tidak berfungsi sebagai bahasa pemrograman, melainkan sebagai bahasa penanda yang memberi arti dan susunan pada konten agar dapat dipahami dan ditampilkan oleh browser. Melalui HTML, pengembang dapat mendefinisikan elemen-elemen seperti judul, paragraf, gambar, tabel, hingga formulir input. Setiap elemen HTML direpresentasikan oleh tag yang ditulis di dalam tanda kurung siku (< >) dan umumnya terdiri dari pasangan tag pembuka dan tag penutup. Struktur berpasangan ini penting agar browser dapat mengenali batasan suatu elemen secara jelas dan menghindari kesalahan tampilan.

Dokumen HTML yang baik memiliki struktur hierarkis yang teratur. Elemen <html> berfungsi sebagai *root element* yang membungkus seluruh isi dokumen, menandakan bahwa seluruh konten di dalamnya merupakan bagian dari satu halaman web. Di dalam elemen tersebut terdapat dua bagian utama, yaitu <head> dan <body>. Bagian <head> berisi informasi deskriptif tentang dokumen, seperti judul halaman, metadata, pengaturan karakter, dan referensi ke file eksternal seperti CSS atau JavaScript. Informasi pada bagian ini tidak ditampilkan langsung kepada pengguna, namun memiliki peran penting dalam pengelolaan halaman, kompatibilitas browser, serta optimasi mesin pencari. Sebaliknya, bagian <body> memuat seluruh konten yang dapat dilihat dan diinteraksikan oleh pengguna, sehingga menjadi pusat pengalaman pengguna (*user experience*) pada halaman web.

Penerapan HTML yang benar tidak hanya bergantung pada kelengkapan elemen, tetapi juga pada penggunaan elemen yang sesuai dengan fungsinya, yang dikenal sebagai markup semantik. Markup semantik bertujuan untuk memberikan makna yang jelas terhadap setiap bagian konten, baik bagi browser, mesin pencari, maupun pengembang lain yang membaca kode tersebut. Penggunaan elemen seperti <h1> hingga <h6> untuk judul, <p> untuk paragraf, <form> untuk input data, serta <table> untuk data terstruktur membantu menciptakan dokumen yang lebih mudah dipahami dan dipelihara. Dalam konteks industri, markup semantik sangat penting karena meningkatkan aksesibilitas, memperbaiki performa SEO, dan mempermudah pengembangan lanjutan atau integrasi dengan sistem lain.

Selain elemen semantik, HTML juga menyediakan elemen non-semantik seperti <div> dan yang berfungsi sebagai wadah. Elemen <div> bersifat *block-level* dan umumnya digunakan untuk membagi halaman ke dalam bagian-bagian besar seperti *header*, konten utama, atau *footer*. Sementara itu, elemen bersifat *inline* dan digunakan untuk

menandai bagian kecil dari teks tanpa mengganggu alur paragraf. Perbedaan karakteristik ini sangat berpengaruh dalam pengaturan *layout* dan *styling*, terutama ketika dikombinasikan dengan CSS. Pemahaman yang baik terhadap penggunaan `<div>` dan `<span>` memungkinkan pengembang membangun struktur halaman yang fleksibel, rapi, dan mudah dikontrol tampilannya.

Dalam praktik pengembangan web modern, HTML menjadi fondasi utama yang akan dikombinasikan dengan CSS dan JavaScript. HTML bertanggung jawab atas struktur, CSS mengatur tampilan visual, dan JavaScript menambahkan interaktivitas. Oleh karena itu, kesalahan pada struktur HTML dapat berdampak langsung pada kesulitan *styling* maupun fungsi interaktif halaman. Dengan menerapkan struktur HTML yang valid, semantik, dan terorganisir, pengembang dapat menghasilkan halaman web yang tidak hanya berfungsi dengan baik, tetapi juga efisien, mudah dikembangkan, dan sesuai dengan standar profesional di lingkungan industri.

D.2 Keterkaitan dengan Teknik Industri

HTML (*HyperText Markup Language*) merupakan kerangka dasar dalam pembuatan halaman web yang berfungsi untuk menyusun struktur dan tata letak informasi. Dalam konteks Teknik Industri, HTML memiliki peran penting sebagai media digital untuk mendokumentasikan, mencatat, dan menyajikan data operasional secara terstruktur. Banyak aktivitas di bidang Teknik Industri, seperti pengendalian kualitas, pemeliharaan mesin, dan pencatatan produksi, membutuhkan sistem pencatatan yang rapi, konsisten, dan mudah diakses. HTML menjadi fondasi awal untuk membangun sistem tersebut dalam bentuk halaman web.

Sebagai contoh, pada kegiatan pengendalian kualitas, seorang engineer industri sering kali membutuhkan formulir inspeksi untuk mencatat ID produk, jenis cacat, jumlah cacat, serta keterangan tambahan. Materi HTML pada Pertemuan 1 memungkinkan pembuatan formulir digital menggunakan elemen `<form>`, `<input>`, `<select>`, dan `<textarea>`. Dengan struktur HTML yang baik, formulir inspeksi yang sebelumnya dilakukan secara manual di kertas dapat dialihkan ke bentuk digital. Hal ini tidak hanya mengurangi kesalahan pencatatan, tetapi juga mempercepat proses pengumpulan data dan memudahkan penyimpanan arsip inspeksi.

Selain itu, HTML juga relevan dalam penyusunan laporan operasional dan pemantauan kondisi mesin produksi. Melalui penggunaan elemen seperti `<table>`, `<h1>` hingga `<h3>`, serta `<p>`, data seperti jadwal pemeliharaan, status mesin, dan kapasitas produksi dapat ditampilkan secara terstruktur dan mudah dibaca. Misalnya, status mesin “Operasional” atau “Tidak Operasional” dapat ditampilkan sebagai teks terpisah dalam struktur HTML yang jelas, sehingga memudahkan pengambilan keputusan oleh *supervisor* atau manajer produksi. Struktur ini menjadi dasar sebelum dikembangkan lebih lanjut dengan tampilan visual dan sistem otomatis.

Secara keseluruhan, materi HTML pada Pertemuan 1 menjadi landasan penting bagi mahasiswa Teknik Industri untuk memahami bagaimana proses bisnis dan operasional dapat diterjemahkan ke dalam sistem informasi berbasis web. HTML membantu membentuk pola pikir sistematis dalam menyusun data, alur informasi, dan dokumentasi kerja. Kemampuan ini sangat relevan di dunia industri modern, di mana hampir seluruh aktivitas produksi, kualitas, dan pemeliharaan telah terintegrasi dengan sistem digital berbasis web atau aplikasi internal perusahaan.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

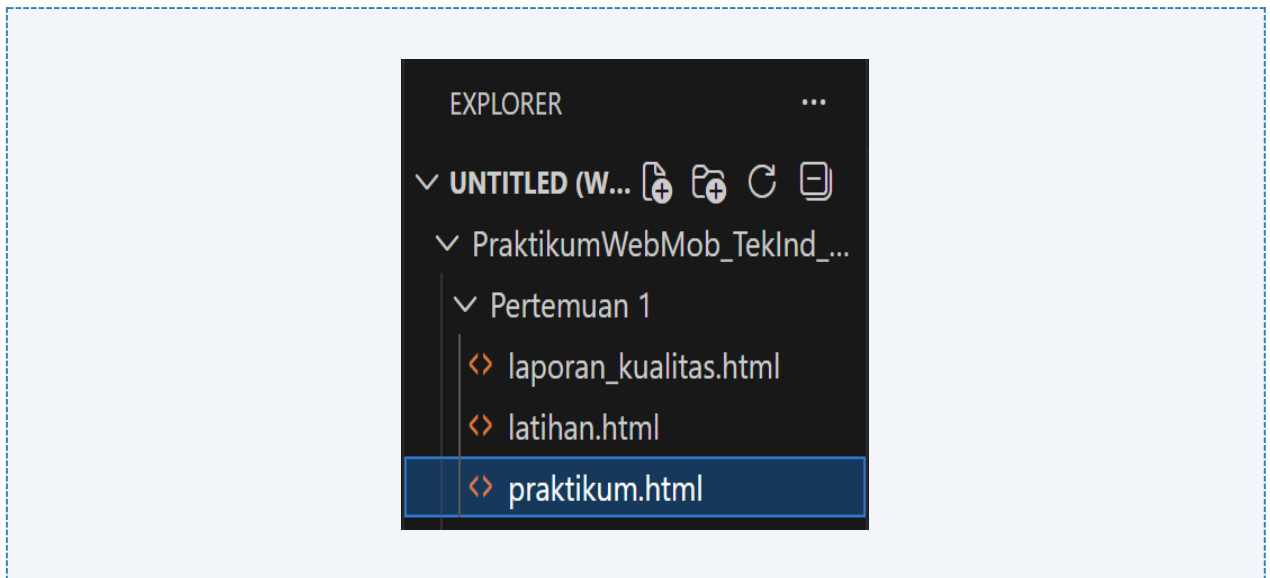
Dokumentasikan setiap langkah kerja yang Anda lakukan, beserta hasil yang diperoleh (screenshot, kode, atau output). Jawab sesuai urutan langkah di Manual Praktikum.

E.1 Langkah 1 – Membuat File HTML

Deskripsi singkat apa yang dilakukan pada langkah ini:

- 1) Buka VS Code.
- 2) Buat folder baru di desktop bernama `PraktikumWebMob\_TekInd\_2026`.
- 3) Di dalam VS Code, pilih File > Open Folder dan pilih folder tersebut.
- 4) Buat file baru bernama `praktikum.html`.
- 5) Ketik `!` lalu tekan Tab (atau Enter). VS Code akan otomatis membuat kerangka HTML dasar.

Screenshot Hasil:



Gambar E.1 – Membuat File HTML

E.2 Langkah 2 – Menyusun Profil Lini Produksi

```
<!DOCTYPE html>
<html lang="id">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Data Mesin Produksi - Lini A</title>
</head>

<body>
  <h1>Profil Mesin CNC-01</h1>
  <hr>

  <h2>Spesifikasi Teknis</h2>
  <ul>
    <li><strong>Merk:</strong> Yamazaki Mazak</li>
    <li><strong>Kapasitas:</strong> 500 unit/hari</li>
    <li><strong>Status:</strong> <span style="color: green;">Operasional</span></li>
  </ul>
  <h2>Jadwal Pemeliharaan</h2>
```

```

<table border="1" cellpadding="10">
  <thead>
    <tr>
      <th>Tanggal</th>
      <th>Jenis Perawatan</th>
      <th>Penanggung Jawab</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>12 Jan 2024</td>
      <td>Penggantian Oli</td>
      <td>Teknisi A</td>
    </tr>
    <tr>
      <td>15 Feb 2024</td>
      <td>Kalibrasi Presisi</td>
      <td>Teknisi B</td>
    </tr>
  </tbody>
</table>
</body>
</html>

```

Screenshot Hasil:



Gambar E.2 – Menyusun Profil Lini Produksi

E.3 Langkah 3 – Membuat Form Input Data Produksi

```

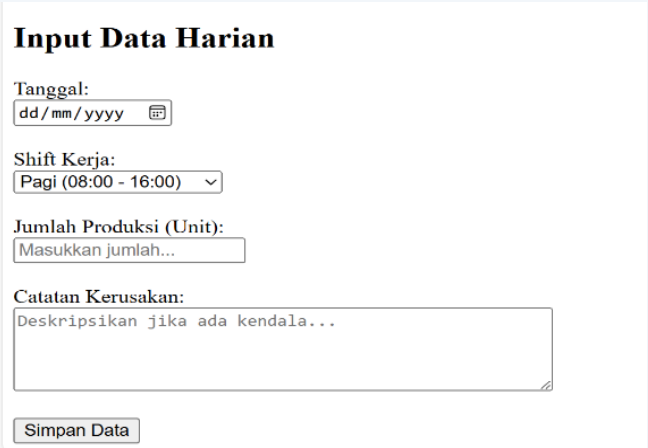
<hr>

<h2>Input Data Harian</h2>
<form action="">
  <label for="tanggal">Tanggal:</label><br>
  <input type="date" id="tanggal" name="tanggal" required><br><br>
  <label for="shift">Shift Kerja:</label><br>
  <select id="shift" name="shift">
    <option value="Pagi">Pagi (08:00 - 16:00)</option>

```

```
<option value="Siang">Siang (16:00 - 24:00)</option>
<option value="Malam">Malam (00:00 - 08:00)</option>
</select><br><br>
<label for="jumlah">Jumlah Produksi (Unit):</label><br>
<input type="number" id="jumlah" name="jumlah" min="0"
placeholder="Masukkan jumlah..."><br><br>
<label for="catatan">Catatan Kerusakan:</label><br>
<textarea id="catatan" name="catatan" rows="4" cols="50"
placeholder="Deskripsikan jika ada
kendala..."></textarea><br><br>
<button type="submit">Simpan Data</button>
</form>
</body>
</html>
```

Screenshot Hasil:

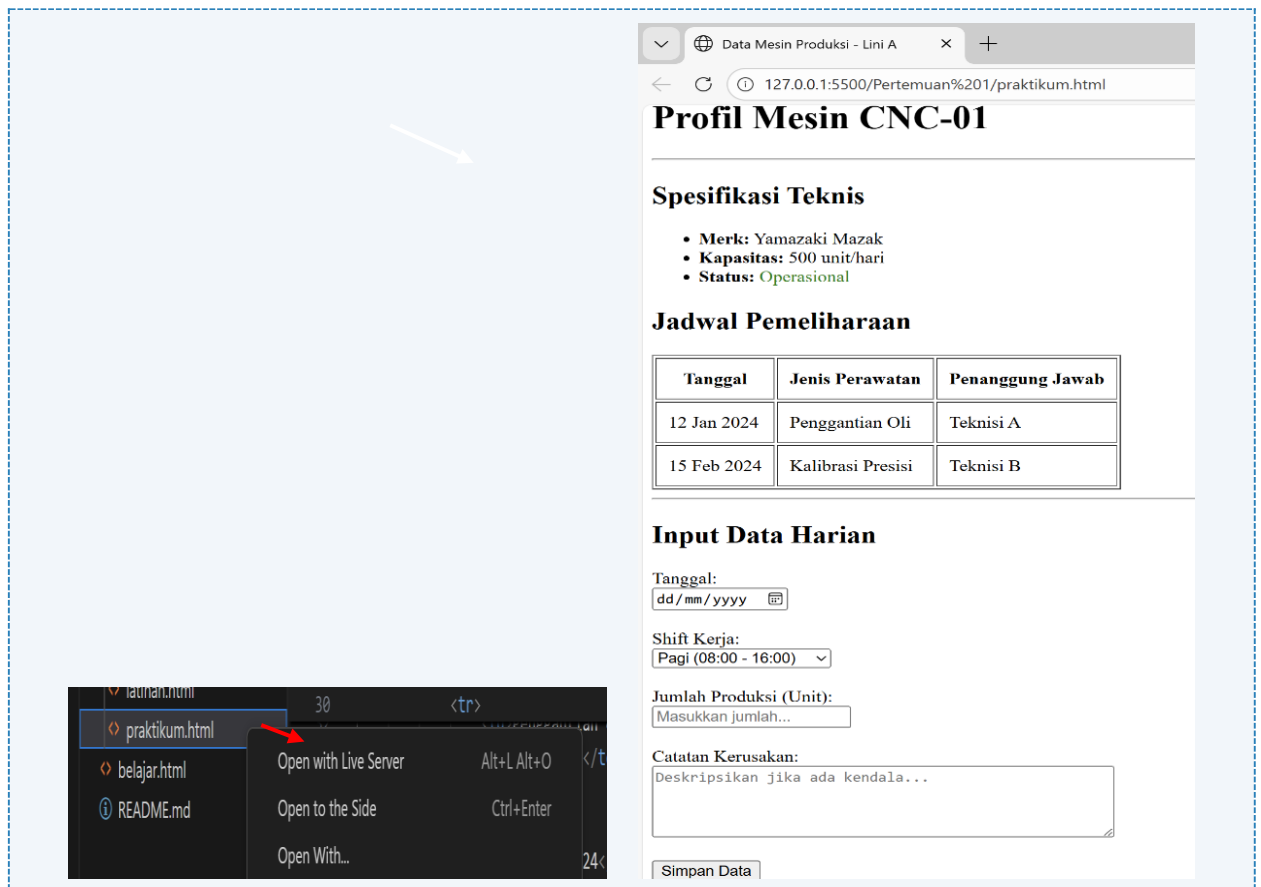


Gambar E.3 – Membuat Form Input Data Produksi

E.4 Langkah 4 – Menjalankan File

- 1) Klik kanan pada area kode `praktikum.html`.
- 2) Pilih "Open with Live Server".
- 3) Browser akan terbuka otomatis menampilkan halaman yang Anda buat.

screenshot Hasil:



Gambar E.4 – Menjalankan Live Server dan Hasil File

F. LATIHAN DAN TUGAS MANDIRI

Jawab dan dokumentasikan seluruh latihan/tugas yang ada di bagian "Latihan/Tugas Mandiri" pada modul Anda.

F.1 Latihan 1 – Pengayaan Tabel

Pertanyaan / Instruksi Latihan:

Tambahkan satu kolom baru di tabel "Jadwal Pemeliharaan" bernama "Status" dan isi datanya secara manual (Selesai/Pending).

Jawaban / Kode Solusi:

```
<section>
  <h2>Jadwal Pemeliharaan</h2>
  <table border="1" cellpadding="10">
    <thead>
      <tr>
        <th>Tanggal</th>
        <th>Jenis Perawatan</th>
        <th>Penanggung Jawab</th>
        <!-- Tambah kolom status (latihan 1) -->
        <th>Status</th>
      </tr>
    </thead>
    <tbody>
      <tr>
```

```

        <td>12 Jan 2024</td>
        <td>Penggantian Oli</td>
        <td>Teknisi A</td>
        <td>Pending</td>
    </tr>
    <tr>
        <td>15 Feb 2024</td>
        <td>Kalibrasi Presisi</td>
        <td>Teknisi B</td>
        <td>Selesai</td>
    </tr>
</tbody>
</table>
</section>

```

Jadwal Pemeliharaan

Tanggal	Jenis Perawatan	Penanggung Jawab	Status
12 Jan 2024	Penggantian Oli	Teknisi A	Pending
15 Feb 2024	Kalibrasi Presisi	Teknisi B	Selesai

Gambar F.1 – Hasil Pengayaan Tabel

F.2 Latihan 2 – Validasi Dasar

Pertanyaan / Instruksi Latihan:

Pada input "Jumlah Produksi", tambahkan atribut `max` dengan nilai 600 (maksimal kapasitas mesin).

Jawaban / Kode Solusi:

```

<label for="jumlah">Jumlah Produksi (Unit):</label><br>
    <!-- Tambah atribut "max" = 600 pada input "jumlah produksi"
    (latihan 2) -->
    <input type="number" id="jumlah" name="jumlah" min="0" max="600"
    placeholder="Masukkan jumlah..."><br><br>

```

Jumlah Produksi (Unit):

Masukkan

Gambar F.2 – Hasil Validasi Dasar

F.3 Latihan 3 – Semantic HTML

Pertanyaan / Instruksi Latihan:

Bungkus judul "Profil Mesin" dan spesifikasinya menggunakan tag ``<header>`` dan ``<section>`` agar kode lebih rapi dan semantik.

Jawaban / Kode Solusi:

```
<body>
  <!-- Pisah header dan section agar kode html lebih semantic (latihan 3)-
->
  <header>
    <h1>Profil Mesin CNC-01</h1>
  </header>
  <!-- Pisah header dan section agar kode html lebih semantic (latihan 3)-
->
  <main>
    <section>
      <h2>Spesifikasi Teknis</h2>
      <ul>
        <li><strong>Merk:</strong> Yamazaki Mazak</li>
        <li><strong>Kapasitas:</strong> 500 unit/hari</li>
        <li><strong>Status:</strong> <span style="color:
green;">Operasional</span></li>
      </ul>
    </section>
    <!-- Pisah header dan section agar kode html lebih semantic (latihan
3)-->
    <section>
      <h2>Jadwal Pemeliharaan</h2>
      <table border="1" cellpadding="10">
        <thead>
          <tr>
            <th>Tanggal</th>
            <th>Jenis Perawatan</th>
            <th>Penanggung Jawab</th>
            <!-- Tambah kolom status (latihan 1) -->
            <th>Status</th>
          </tr>
        </thead>
        <tbody>
          <tr>
            <td>12 Jan 2024</td>
            <td>Penggantian Oli</td>
            <td>Teknisi A</td>
            <td>Pending</td>
          </tr>
          <tr>
            <td>15 Feb 2024</td>
            <td>Kalibrasi Presisi</td>
            <td>Teknisi B</td>
            <td>Selesai</td>
          </tr>
        </tbody>
      </table>
    </section>
    <!-- Pisah header dan section agar kode html lebih semantic (latihan
3)-->
    <section>
      <h2>Input Data Harian</h2>
      <form action="">
```



```
        <label for="tanggal">Tanggal:</label><br>
        <input type="date" id="tanggal" name="tanggal"
required><br><br>
        <label for="shift">Shift Kerja:</label><br>
        <select id="shift" name="shift">
            <option value="Pagi">Pagi (08:00 - 16:00)</option>
            <option value="Siang">Siang (16:00 - 24:00)</option>
            <option value="Malam">Malam (00:00 - 08:00)</option>
        </select><br><br>
        <label for="jumlah">Jumlah Produksi (Unit):</label><br>
        <!-- Tambah atribut "max" = 600 pada input "jumlah produksi"
(latihan 2) -->
        <input type="number" id="jumlah" name="jumlah" min="0"
max="600" placeholder="Masukkan jumlah..."><br><br>
        <label for="catatan">Catatan Kerusakan:</label><br>
        <textarea id="catatan" name="catatan" rows="4" cols="50"
placeholder="Deskripsikan jika ada
kendala..."></textarea><br><br>
        <button type="submit">Simpan Data</button>
    </form>
</section>
</main>
</body>
```

Spesifikasi Teknis

- **Merk:** Yamazaki Mazak
- **Kapasitas:** 500 unit/hari
- **Status:** Operasional

Jadwal Pemeliharaan

Tanggal	Jenis Perawatan	Penanggung Jawab	Status
12 Jan 2024	Penggantian Oli	Teknisi A	Pending
15 Feb 2024	Kalibrasi Presisi	Teknisi B	Selesai

Input Data Harian

Tanggal:

dd/mm/yyyy 

Shift Kerja:

Pagi (08:00 - 16:00) 

Jumlah Produksi (Unit):

Masukkan

Gambar F.3 – Hasil Semantic HTML

F.4 Tugas Proyek Mini – Relevansi Industri

Deskripsi proyek mini:

Buatlah halaman HTML baru bernama `laporan\_kualitas.html`. Halaman ini berisi formulir "Laporan Inspeksi Kualitas" dengan field:

- ID Produk (Text)
- Nama Inspektur (Text)
- Jenis Cacat (Radio Button: Geometri, Permukaan, Fungsional)
- Jumlah Cacat (Number)
- Foto Produk (Input type file - placeholder saja)
- Tombol Submit

Penjelasan pendekatan/solusi yang digunakan:

Penjelasan Langkah Pengerjaan:

1. Menyiapkan alat dan aplikasi praktikum
Praktikum diawali dengan menyiapkan perangkat yang dibutuhkan, yaitu laptop atau komputer, aplikasi *Visual Studio Code* sebagai *code editor*, serta browser Google Chrome untuk menampilkan hasil halaman HTML. Selain itu, ekstensi *Live Server* digunakan untuk menjalankan file HTML secara langsung. Untuk keperluan dokumentasi kode program pada laporan, digunakan pula ekstensi *CodeSnap* yang berfungsi mengambil tangkapan layar kode agar lebih jelas dan rapi saat disajikan dalam laporan praktikum.
2. Membuat file HTML baru
Selanjutnya dibuat sebuah file HTML baru dengan nama laporan_kualitas.html. File dibuat menggunakan *Visual Studio Code* dan disimpan dengan ekstensi .html agar dapat dikenali sebagai dokumen HTML oleh browser.
3. Menuliskan deklarasi dan struktur dasar HTML
Pada bagian awal file dituliskan deklarasi `<!DOCTYPE html>` sebagai penanda bahwa dokumen menggunakan standar HTML. Setelah itu dibuat struktur utama dokumen yang terdiri dari tag `<html>`, `<head>`, dan `<body>`. Pada bagian `<head>` ditambahkan pengaturan karakter dan judul halaman “Laporan Inspeksi Kualitas”.
4. Menyusun tampilan *header* halaman
Di dalam tag `<body>`, dibuat bagian `<header>` yang berisi judul halaman menggunakan heading, deskripsi singkat fungsi formulir, serta identitas penyusun. Untuk memperjelas tampilan, ditambahkan garis horizontal sebagai pemisah antara bagian header dan isi halaman.
5. Membuat formulir laporan inspeksi kualitas
Setelah bagian *header*, dibuat sebuah formulir menggunakan tag `<form>`. Formulir ini berfungsi sebagai wadah utama untuk seluruh input data laporan inspeksi kualitas yang akan diisikan oleh pengguna.
6. Menambahkan field ID Produk
Field ID Produk dibuat menggunakan input bertipe teks. Field ini digunakan untuk memasukkan kode atau identitas produk yang diperiksa. Label dan *placeholder* ditambahkan agar pengguna memahami data yang harus diisi.
7. Menambahkan field Nama Inspektur
Selanjutnya ditambahkan field Nama Inspektur menggunakan input bertipe teks. Field ini berfungsi untuk mencatat nama petugas yang melakukan inspeksi kualitas pada produk.
8. Membuat pilihan Jenis Cacat
Untuk menentukan jenis cacat produk, digunakan input bertipe radio button. Tiga pilihan disediakan, yaitu cacat Geometri, Permukaan, dan Fungsional. Seluruh radio button menggunakan nama yang sama agar pengguna hanya dapat memilih satu jenis cacat.
9. Menambahkan field Jumlah Cacat
Field Jumlah Cacat dibuat menggunakan input bertipe *number*. Penggunaan tipe number bertujuan agar data yang dimasukkan berupa angka. Nilai minimum ditetapkan untuk mencegah input jumlah cacat bernilai negatif.
10. Menambahkan field Foto Produk sebagai *placeholder*
Field Foto Produk dibuat menggunakan input bertipe file. Field ini berfungsi sebagai placeholder untuk memilih file gambar produk. Pada tahap praktikum ini, input file hanya digunakan sebagai simulasi tampilan dan belum dilengkapi dengan proses penyimpanan atau pengolahan data.
11. Menambahkan tombol submit dan reset

Pada bagian akhir formulir, ditambahkan dua buah tombol, yaitu tombol Submit dan Reset. Tombol Submit digunakan untuk mengirimkan data laporan inspeksi kualitas, sedangkan tombol Reset berfungsi untuk mengosongkan seluruh field input pada formulir sehingga pengguna dapat mengisi ulang data dari awal tanpa harus me-*refresh* halaman.

12. Menyimpan dan menjalankan file HTML

Setelah seluruh kode selesai ditulis, file disimpan dan dijalankan menggunakan browser melalui fitur *Live Server* atau dengan membuka langsung file HTML. Halaman kemudian ditampilkan dan formulir dapat digunakan untuk simulasi pengisian laporan inspeksi kualitas.

Kode Utama:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Laporan Inspeksi Kualitas</title>
</head>
<body>
  <header>
    <h1>Laporan Inspeksi Kualitas</h1>
    <p>Formulir pencatatan hasil inspeksi produk</p>
    <p>Disusun oleh : Maessa Andrea Vallenia (23051430044)</p>
  </header>
  <section>
    <!-- Formulir Laporan Inspeksi Kualitas -->
    <form action="">

      <!-- ID Produk -->
      <label for="id_produk">ID Produk:</label><br>
      <input type="text" id="id_produk" name="id_produk"
placeholder="Masukkan ID produk"><br><br>

      <!-- Nama Inspektur -->
      <label for="nama_inspektur">Nama Inspektur:</label><br>
      <input type="text" id="nama_inspektur" name="nama_inspektur"
placeholder="Masukkan nama inspektur"><br><br>

      <!-- Jenis Cacat -->
      <label>Jenis Cacat:</label><br>
      <!-- Radio Geometri -->
      <input type="radio" id="geometri" name="jenis_cacat"
value="Geometri">
      <label for="geometri">Geometri</label><br>
      <!-- Radio Permukaan -->
      <input type="radio" id="permukaan" name="jenis_cacat"
value="Permukaan">
      <label for="permukaan">Permukaan</label><br>
      <!-- Radio Fungsional -->
      <input type="radio" id="fungsional" name="jenis_cacat"
value="Fungsional">
      <label for="fungsional">Fungsional</label><br><br>

      <!-- Jumlah Cacat -->
      <label for="jumlah_cacat">Jumlah Cacat:</label><br>
```

```
        <input type="number" id="jumlah_cacat" name="jumlah_cacat"
min="0" placeholder="Masukkan jumlah cacat"><br><br>

        <!-- Foto Produk (Placeholder) -->
        <label for="foto_produk">Foto Produk:</label><br>
        <input type="file" id="foto_produk" name="foto_produk"><br><br>

        <!-- Tombol Submit -->
        <button type="submit">Kirim Laporan</button>
        <!-- Tombol Reset -->
        <button type="reset">Reset Formulir</button>
    </form>
</section>
</body>
</html>
```

✓ Laporan Inspeksi Kualitas × +

← ↻ ⓘ 127.0.0.1:5500/Pertemuan%201/laporan_kualitas.html

Laporan Inspeksi Kualitas

Formulir pencatatan hasil inspeksi produk

Disusun oleh : Maessa Andrea Vallenia (23051430044)

ID Produk:

Nama Inspektur:

Jenis Cacat:

☐ Geometri

☐ Permukaan

☐ Fungsional

Jumlah Cacat:

Foto Produk:

 No file chosen

Gambar F.4 – Hasil Proyek Mini Relevansi Industri (Laporan Kualitas)

G. PEMBAHASAN

Analisis hasil praktikum secara kritis. Jawab pertanyaan berikut dengan lengkap dan mendalam (bukan hanya "ya/tidak").

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum yang diperoleh pada modul HTML ini menunjukkan tingkat kesesuaian yang tinggi dengan teori dan konsep dasar HTML yang dipelajari pada Pertemuan 1. Secara struktur, dokumen HTML telah disusun sesuai standar penulisan HTML, yaitu diawali dengan deklarasi `<!DOCTYPE html>` yang berfungsi memberi tahu browser bahwa dokumen menggunakan HTML5. Selanjutnya, seluruh isi halaman dibungkus di dalam elemen root `<html>`, sebagaimana dijelaskan dalam teori bahwa elemen ini wajib ada sebagai pembungkus utama dokumen.

Bagian `<head>` digunakan untuk menyimpan informasi deskriptif seperti metadata karakter, pengaturan *viewport*, serta judul halaman menggunakan elemen `<title>`. Hal ini sesuai dengan teori bahwa konten di dalam `<head>` tidak ditampilkan langsung ke pengguna, tetapi berperan penting dalam kompatibilitas browser, pengaturan tampilan awal, dan identitas halaman web. Hasilnya, judul halaman tampil dengan benar pada tab browser dan halaman dapat dimuat tanpa peringatan kesalahan.

Pada bagian `<body>`, seluruh elemen konten ditempatkan sesuai fungsinya. Elemen heading (`<h1>`) digunakan untuk menampilkan judul utama laporan inspeksi kualitas, sedangkan elemen paragraf (`<p>`) digunakan untuk memberikan penjelasan pendukung. Penggunaan elemen ini sejalan dengan konsep markup semantik, di mana setiap elemen HTML digunakan sesuai makna dan perannya, bukan hanya untuk tujuan tampilan visual.

Implementasi elemen `<form>` beserta elemen input di dalamnya juga menunjukkan kesesuaian dengan teori. Elemen `<input>`, `<textarea>`, dan `<button>` digunakan untuk menerima data dari pengguna secara terstruktur. Berdasarkan hasil pengujian di browser, form dapat diisi dengan baik dan setiap field berfungsi sesuai tujuan perancangannya. Hal ini membuktikan bahwa teori mengenai form HTML sebagai sarana interaksi pengguna telah diterapkan secara nyata dalam praktikum.

Selain itu, penggunaan elemen `<div>` sebagai *container* utama menunjukkan pemahaman terhadap konsep pengelompokan konten. Elemen `<div>` digunakan untuk membungkus seluruh isi halaman agar struktur lebih rapi dan siap dikembangkan pada tahap styling menggunakan CSS. Secara keseluruhan, hasil praktikum tidak hanya sesuai dengan teori HTML, tetapi juga menunjukkan penerapan konsep secara konsisten dan fungsional.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Tag <code><title></code> di beberapa file masih menggunakan teks lama "Dokumen" atau <code><title>Data Mesin Produksi - Lini A</title></code> , sehingga tidak sesuai dengan isi halaman.	Mengganti tag <code><title></code> sesuai konten halaman. Contoh: <code><title>Laporan Inspeksi Kualitas</title></code> .
Tag penutup <code></html></code> sempat terhapus sehingga halaman tidak tampil dengan benar di browser	Menambahkan kembali tag penutup <code></html></code> di bagian akhir dokumen agar struktur HTML kembali lengkap dan valid

G.3 Kaitan dengan Penerapan di Industri

Materi HTML yang dipelajari pada modul ini memiliki keterkaitan yang sangat kuat dengan penerapan di dunia Teknik Industri. HTML dapat digunakan sebagai fondasi dalam pembuatan berbagai sistem informasi operasional, terutama yang berkaitan dengan pencatatan, pelaporan, dan pengendalian proses kerja. Dalam konteks industri modern, banyak aktivitas yang sebelumnya dilakukan secara manual kini beralih ke sistem digital berbasis web.

Sebagai contoh, formulir laporan inspeksi kualitas yang dibuat dalam praktikum ini dapat diterapkan secara langsung di lingkungan pabrik. Operator produksi atau petugas *quality control* dapat mengisi laporan inspeksi produk melalui halaman web, sehingga data tersimpan secara terstruktur dan konsisten. Hal ini mendukung prinsip Teknik Industri dalam meningkatkan efisiensi kerja, mengurangi kesalahan pencatatan, serta mempercepat proses pengolahan data.

Selain itu, struktur HTML yang baik memungkinkan integrasi dengan teknologi lain seperti CSS dan JavaScript untuk meningkatkan tampilan dan fungsionalitas sistem. Dalam praktik industri, halaman HTML dapat dikembangkan menjadi *dashboard* pemantauan kualitas, sistem pelaporan kerusakan mesin, atau checklist pemeliharaan rutin. Semua sistem tersebut membutuhkan struktur HTML yang rapi dan valid sebagai dasar pengembangannya.

Dari sudut pandang Teknik Industri, kemampuan memahami dan menyusun HTML juga mendukung kolaborasi lintas disiplin. Seorang mahasiswa Teknik Industri yang memahami HTML dapat berkomunikasi lebih efektif dengan tim IT atau pengembang sistem dalam merancang aplikasi internal perusahaan. Dengan demikian, keterampilan HTML bukan hanya kemampuan teknis semata, tetapi juga menjadi bagian dari kompetensi pendukung dalam menghadapi kebutuhan industri yang semakin terdigitalisasi.

H. KESIMPULAN

Tuliskan 3–5 poin kesimpulan yang merangkum hal-hal yang dipelajari dan dicapai pada praktikum modul ini. Kesimpulan harus spesifik, bukan bersifat umum.

1	Praktikum modul HTML dasar berhasil memberikan pemahaman yang konkret mengenai struktur dokumen HTML, mulai dari penggunaan <code><!DOCTYPE html></code> , elemen root <code><html></code> , hingga pembagian dokumen ke dalam bagian <code><head></code> dan <code><body></code> . Mahasiswa tidak hanya memahami teori, tetapi juga mampu menerapkannya secara langsung dalam pembuatan halaman web yang valid dan dapat ditampilkan dengan baik di browser.
2	Mahasiswa mampu menggunakan berbagai elemen HTML dasar secara tepat sesuai fungsinya, seperti <code><h1></code> untuk judul, <code><p></code> untuk teks penjelas, <code><table></code> untuk penyajian data terstruktur, serta <code><form></code> dan elemen input untuk pengumpulan data. Penerapan elemen-elemen ini menunjukkan pemahaman terhadap konsep markup semantik dan pentingnya struktur logis dalam penyusunan halaman web.
3	Melalui pembuatan formulir laporan inspeksi kualitas, mahasiswa berhasil mempraktikkan penggunaan form HTML sebagai media input data operasional. Formulir yang dibuat mampu menampung data seperti identitas produk, jenis cacat, jumlah cacat, serta dokumentasi pendukung, sehingga mencerminkan kebutuhan nyata dalam proses pengendalian kualitas di lingkungan industri.
4	Praktikum ini juga melatih ketelitian dan kemampuan pemecahan masalah mahasiswa dalam menghadapi kendala teknis, seperti kesalahan struktur HTML, penghapusan tag penutup, dan pengaturan tata letak elemen. Dengan melakukan pengecekan ulang kode dan menyesuaikan penulisan elemen, mahasiswa memahami pentingnya penulisan kode yang rapi, konsisten, dan sesuai standar.
5	Secara keseluruhan, modul ini menunjukkan bahwa HTML merupakan fondasi penting dalam pengembangan sistem informasi berbasis web di bidang Teknik Industri. Kemampuan menyusun halaman web statis yang terstruktur dan fungsional menjadi bekal awal bagi mahasiswa untuk mengembangkan aplikasi pendukung proses produksi, pelaporan, dan pengambilan keputusan berbasis data di dunia industri.

I. DAFTAR PUSTAKA

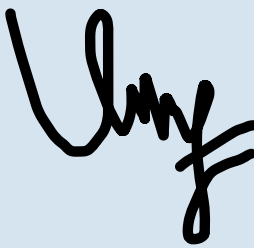
Tuliskan semua sumber referensi yang digunakan dalam format APA 7th Edition. Minimal 3 referensi yang relevan (termasuk Manual Praktikum dan sumber online resmi seperti MDN, Reactjs.org, dll).

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Sari, I. P., Jannah, A., Meuraxa, A. M., Syahfitri, A., & Omar, R. (2022). Perancangan sistem informasi penginputan database mahasiswa berbasis web. <i>Hello World Jurnal Ilmu Komputer</i> , 1(2), 106-110.

[3]	Muthohir, M. (2021). Mudah membuat web bagi pemula (mengenal HTML, HTML5, CSS dan JavaScript). <i>Penerbit Yayasan Prima Agus Teknik</i> , 1-73.
[4]	Sinlae Goreng, IM, Setiyansyah, F., & Ihsan, M. (2024). Pengenalan pemrograman web: Pembuatan aplikasi web sederhana dengan PHP dan MySQL. <i>Jurnal Siber Multi Disiplin</i> .

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Diperiksa oleh Asisten / Dosen
 _____ Maessa Andrea Vallenia NIM: 23051430044	 _____ Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____

MODUL 2

CSS DASAR & STYLING

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 2 (Pertemuan 2)
Topik Utama	CSS Dasar & Styling
Sub-Topik	Penerapan CSS dasar, pengaturan tampilan halaman web, dan styling formulir
Durasi Praktikum	340 Menit
Tanggal Praktikum	20 Februari 2026
Nama Mahasiswa	Maessa Andrea Vallenia
NIM	23051430044
Kelas	A

B. TUJUAN PEMBELAJARAN

Tuliskan tujuan pembelajaran yang ingin dicapai pada modul ini, sesuai dengan yang tertera di Manual Praktikum.

1	Memahami cara kerja CSS untuk mempercantik antarmuka pengguna (UI).
2	Mampu memanipulasi warna, font, dan ukuran elemen.
3	Menguasai konsep Box Model (Margin, Border, Padding) untuk tata letak.
4	Menerapkan CSS pada formulir agar terlihat profesional dan mudah digunakan.

C. ALAT DAN BAHAN

Sebutkan seluruh perangkat keras, perangkat lunak, dan sumber referensi yang digunakan selama praktikum.

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	Ekstensi VS Code	Live Server	Menjalankan HTML dan memantau

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
			perubahan file
4	Ekstensi VS Code	CodeSnap	Dokumentasi kode program

D. DASAR TEORI

Cascading Style Sheets atau **CSS** merupakan bahasa yang digunakan untuk mengatur tampilan visual elemen yang dibangun menggunakan **HTML**. **HTML** berfungsi sebagai kerangka dasar halaman web dengan mendefinisikan struktur menggunakan tag seperti `<html>`, `<head>`, dan `<body>`, sedangkan **CSS** berperan untuk menentukan bagaimana elemen-elemen tersebut ditampilkan di layar pengguna. Pemisahan fungsi antara **HTML** sebagai struktur dan **CSS** sebagai pengatur tampilan merupakan prinsip penting dalam pengembangan web modern karena menghasilkan kode yang lebih terorganisir, mudah dibaca, dan mudah dikembangkan.

Peran **CSS** menjadi sangat penting dalam perancangan *User Interface* (UI), khususnya pada sistem yang digunakan oleh manusia secara langsung. Elemen **HTML** seperti `<h1>`, `<p>`, `<form>`, `<input>`, dan `<button>` secara default memiliki tampilan standar dari browser. Tanpa penerapan **CSS**, halaman web cenderung sulit dibaca, tidak konsisten, dan berpotensi menimbulkan kesalahan pengguna. Dengan **CSS**, properti seperti *color*, *font-size*, *background-color*, dan *width* dapat diatur sehingga tampilan halaman menjadi lebih jelas, kontras, dan nyaman digunakan.

CSS dapat diterapkan pada dokumen **HTML** melalui tiga metode utama, yaitu *inline*, *internal*, dan *external*. *Inline CSS* ditulis langsung pada elemen **HTML**, misalnya `<h1 style="color:red">`, namun metode ini tidak disarankan karena membuat kode menjadi tidak rapi. *Internal CSS* ditulis menggunakan tag `<style>` di dalam bagian `<head>`, sedangkan *external CSS* ditulis pada file terpisah dengan ekstensi `.css` dan dihubungkan ke **HTML** menggunakan tag `<link>`. Pada modul ini digunakan metode *external CSS* karena merupakan standar yang umum diterapkan di dunia industri.

Dalam **CSS** dikenal konsep *selector* yang berfungsi untuk menentukan elemen **HTML** mana yang akan diberikan gaya. *Selector* dapat berupa *selector* elemen seperti `body`, *selector class* seperti `.container`, maupun *selector id* seperti `#header`. Pemahaman *selector* sangat penting agar styling dapat diterapkan secara tepat sasaran. Kesalahan dalam penggunaan *selector* dapat menyebabkan tampilan tidak sesuai dengan yang diharapkan atau bahkan menimbulkan konflik antar aturan **CSS**.

Konsep penting lainnya dalam **CSS** adalah **Box Model**. Setiap elemen **HTML**, seperti `<div>`, `<form>`, atau `<table>`, dianggap sebagai sebuah kotak yang terdiri dari empat bagian utama, yaitu *content*, *padding*, *border*, dan *margin*. *Content* merupakan isi utama elemen, *padding* adalah jarak antara isi dan batas elemen, *border* merupakan garis tepi elemen, sedangkan *margin* adalah jarak antar elemen. Pengaturan *box model* yang tepat sangat berpengaruh terhadap tata letak dan kerapian halaman web.

Penerapan **CSS** pada elemen formulir seperti `<form>`, `<input>`, `<select>`, `<textarea>`, dan `<button>` bertujuan untuk meningkatkan kemudahan penggunaan dan mengurangi kesalahan input. Dengan **CSS**, ukuran input dapat diseragamkan, jarak antar field dapat diatur menggunakan *margin* dan *padding*, serta tombol dapat diberi efek visual seperti `:hover` untuk memberikan umpan balik kepada pengguna. Hal ini sangat penting pada sistem pelaporan atau inspeksi yang membutuhkan ketelitian dan kenyamanan saat pengisian data.

Berdasarkan modul ini, dapat disimpulkan bahwa CSS tidak hanya berfungsi sebagai alat untuk mempercantik tampilan halaman web, tetapi juga sebagai komponen penting dalam perancangan sistem yang efektif dan efisien. Penguasaan konsep CSS, selector, serta box model menjadi dasar yang kuat untuk membangun sistem berbasis web yang profesional, mudah digunakan, dan sesuai dengan kebutuhan di lingkungan industri.

D.1 Konsep Utama

Konsep utama dari CSS (Cascading Style Sheets) adalah sebagai bahasa yang digunakan untuk mengatur tampilan visual elemen HTML. HTML berfungsi sebagai kerangka dasar halaman web dengan menentukan struktur menggunakan tag seperti `<html>`, `<head>`, dan `<body>`, sedangkan CSS bertugas mengatur bagaimana elemen-elemen tersebut ditampilkan. Pemisahan fungsi antara HTML sebagai struktur dan CSS sebagai pengatur tampilan membuat kode lebih terorganisir, mudah dibaca, dan mudah dikembangkan. CSS sangat penting dalam perancangan User Interface (UI), karena elemen HTML secara default memiliki tampilan standar dari browser yang cenderung kurang konsisten dan kurang nyaman digunakan. Dengan CSS, properti seperti warna, ukuran font, latar belakang, dan lebar elemen dapat diatur sehingga halaman web menjadi lebih jelas, kontras, dan nyaman bagi pengguna. Penerapan CSS dapat dilakukan melalui metode inline, internal, dan external, dengan external CSS menjadi standar yang umum digunakan di industri karena lebih rapi dan mudah dikelola. Selain itu, CSS mengenal konsep selector untuk menentukan elemen mana yang akan diberikan gaya, baik berupa selector elemen, class, maupun ID, sehingga styling dapat diterapkan secara tepat. Konsep Box Model juga menjadi penting, di mana setiap elemen dianggap sebagai kotak yang terdiri dari content, padding, border, dan margin, yang pengaturannya berpengaruh pada tata letak dan kerapian halaman. Dalam konteks formulir, CSS membantu menyeragamkan ukuran input, mengatur jarak antar field, serta memberikan efek visual pada tombol, sehingga meningkatkan kenyamanan dan mengurangi kesalahan input. Dengan demikian, CSS bukan hanya mempercantik tampilan halaman web, tetapi juga menjadi komponen penting dalam membangun sistem berbasis web yang profesional, efisien, dan mudah digunakan.

D.2 Keterkaitan dengan Teknik Industri

Materi pada modul ini memiliki keterkaitan yang kuat dengan bidang Teknik Industri, khususnya dalam perancangan dan pengembangan sistem kerja berbasis digital. Dalam Teknik Industri, banyak sistem digunakan langsung oleh operator, pengawas, maupun manajer, seperti sistem pelaporan kualitas, inspeksi produksi, input data persediaan, dan pengendalian proses. Sistem-sistem tersebut umumnya berbasis web dan dibangun menggunakan struktur `<strong>HTML</strong>` dengan tampilan yang diatur menggunakan `<strong>CSS</strong>`. Oleh karena itu, pemahaman tentang CSS menjadi penting agar sistem yang dirancang tidak hanya berfungsi secara teknis, tetapi juga mudah digunakan dan minim kesalahan manusia (*human error*).

Penerapan konsep *User Interface* (UI) dalam CSS berperan langsung dalam meningkatkan efektivitas kerja di lingkungan industri. Sebagai contoh, pada formulir inspeksi kualitas berbasis web yang menggunakan elemen `<form>`, `<input>`, dan `<button>`, pengaturan warna (color), ukuran font (*font-size*), dan kontras latar (*background-color*) dapat membantu operator membaca informasi dengan cepat dan akurat. Tanpa styling yang baik, tampilan formulir yang terlalu padat atau tidak jelas dapat menyebabkan kesalahan pengisian data, yang berdampak pada kualitas laporan dan pengambilan keputusan manajerial.

Konsep **Box Model** yang terdiri dari *content*, *padding*, *border*, dan *margin* juga sangat relevan dalam konteks Teknik Industri, khususnya dalam perancangan tata letak sistem informasi. Misalnya, pada *dashboard* produksi atau laporan inspeksi, pengaturan margin digunakan untuk memberi jarak antar komponen agar informasi tidak saling bertumpuk, sedangkan padding digunakan untuk meningkatkan keterbacaan isi elemen seperti tabel dan field input. Pengaturan box model yang tepat mencerminkan prinsip ergonomi kerja, di mana sistem dirancang agar nyaman dan efisien bagi penggunaanya.

Penggunaan `CSS selector` seperti *selector elemen* (*input*, *form*), *selector class* (*.form-control*), dan *selector id* (*#laporan*) juga mencerminkan prinsip standarisasi dalam Teknik Industri. Dengan selector class, tampilan berbagai elemen input dapat dibuat konsisten tanpa menuliskan ulang aturan CSS. Hal ini sejalan dengan konsep efisiensi proses dan pengurangan pemborosan (`waste`) dalam perancangan sistem kerja, karena perubahan desain dapat dilakukan secara terpusat dan sistematis.

Selain itu, pemahaman tentang prioritas penerapan CSS (*inline*, *internal*, dan *external*) relevan dengan praktik pengembangan sistem industri berskala besar. Penggunaan *external* CSS melalui file `.css` memungkinkan pemeliharaan sistem yang lebih mudah, terutama ketika sistem digunakan oleh banyak departemen atau dikembangkan secara berkelanjutan. Prinsip ini sejalan dengan pendekatan perbaikan berkelanjutan (`continuous improvement`) yang menjadi dasar dalam Teknik Industri.

Dengan demikian, materi modul ini tidak hanya membekali mahasiswa dengan kemampuan teknis dalam mendesain tampilan web, tetapi juga melatih pola pikir sistematis yang dibutuhkan dalam Teknik Industri. CSS menjadi alat pendukung dalam menciptakan sistem kerja digital yang ergonomis, efisien, dan andal, sehingga dapat mendukung pengambilan keputusan yang berbasis data serta meningkatkan produktivitas di lingkungan industri.

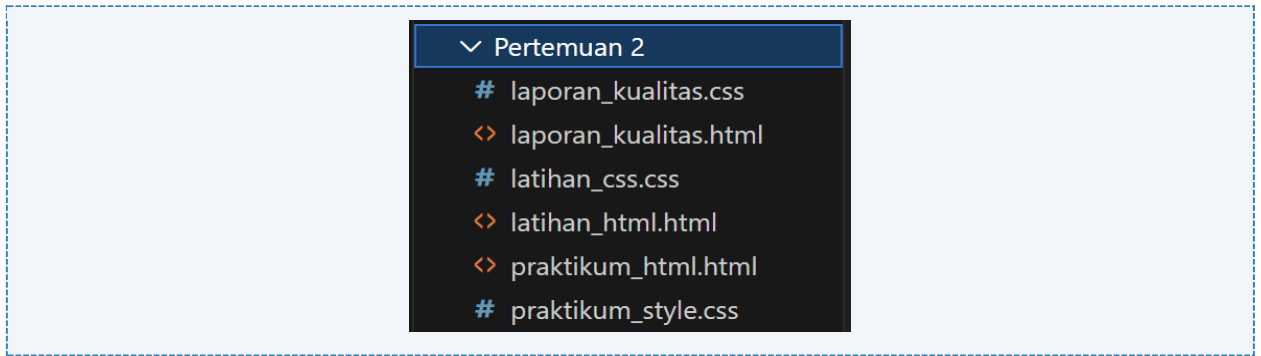
E. LANGKAH KERJA DAN HASIL PRAKTIKUM

Dokumentasikan setiap langkah kerja yang Anda lakukan, beserta hasil yang diperoleh (screenshot, kode, atau output). Jawab sesuai urutan langkah di Manual Praktikum.

E.1 Langkah 1 – Membuat File CSS Eksternal

Deskripsi singkat apa yang dilakukan pada langkah ini:

```
1) Buka folder `PraktikumWebMob_TekInd_2026`.
2) Buat file baru bernama `praktikum_style.css`.
3) Buka `praktikum_html.html`, dan hubungkan ke file CSS dengan menambahkan
   kode ini di dalam ``:
   <link rel="stylesheet" href="style.css">
```

Screenshot Hasil:

Gambar E.1 – Membuat File CSS Eksternal & Menambahkan Kode

E.2 Langkah 2 – Styling Body dan Tipografi

```
/* Import font dari Google Fonts */
@import
url('https://fonts.googleapis.com/css2?family=Roboto:wght@400;700&display=
swap');

body {
  font-family: 'Roboto', sans-serif;
  background-color: #f4f4f4; /* Warna abu-abu muda untuk background */
  color: #333;
  margin: 0;
  padding: 20px;
}

h1 {
  color: #2c3e50; /* Biru tua gelap */
  text-align: center;
  border-bottom: 2px solid #2c3e50;
  padding-bottom: 10px;
}

h2 {
  color: #34495e;
  margin-top: 30px;
}
```


Screenshot Hasil:

Profil Mesin CNC-01

Spesifikasi Teknis

- Merk: Yamazaki Mazak
- Kapasitas: 500 unit/hari
- Status: Operasional

Jadwal Pemeliharaan

Tanggal	Jenis Perawatan	Penanggung Jawab
12 Jan 2024	Penggantian Oli	Teknisi A
15 Feb 2024	Kalibrasi Presisi	Teknisi B

Input Data Harian

Tanggal:

Shift Kerja:

Jumlah Produksi (Unit):

Catatan Kerusakan:

Simpan Data

Gambar E.2 – Styling Body dan Tipografi

E.3 Langkah 3 – Styling Tabel Data

```

table {
  width: 100%;
  border-collapse: collapse; /* Menggabungkan border ganda menjadi satu */
  margin-top: 20px;
  box-shadow: 0 2px 5px rgba(0,0,0,0.1); /* Efek bayangan halus */
}

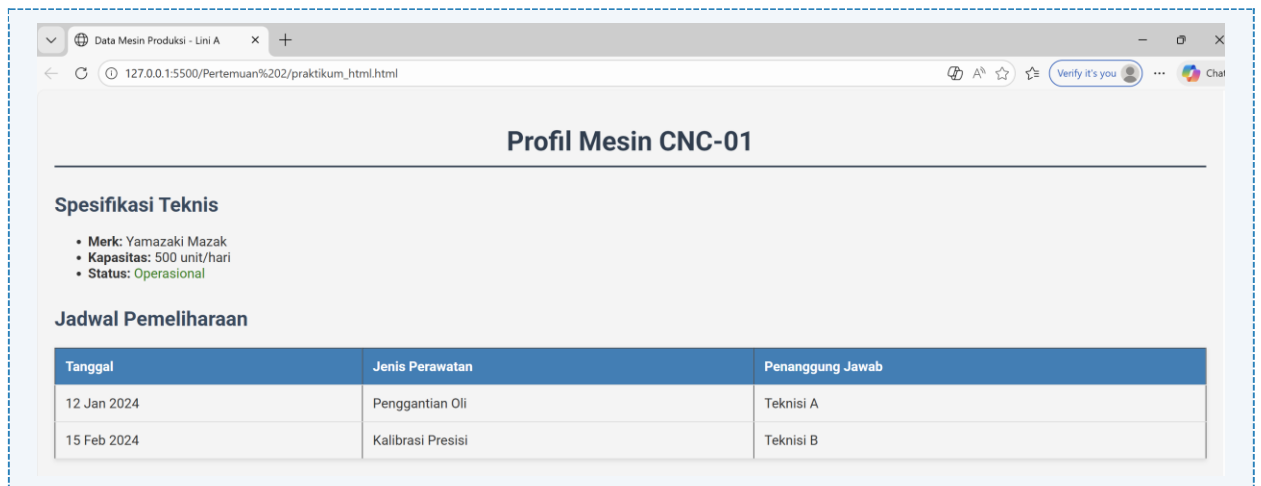
th, td {
  padding: 12px;
  text-align: left;
  border-bottom: 1px solid #ddd;
}

th {
  background-color: #2980b9; /* Biru profesional */
  color: white;
}

tr:hover {
  background-color: #f1f1f1; /* Highlight baris saat mouse diarahkan */
}

```

Screenshot Hasil:



Gambar E.3 – Styling Tabel Data

E.4 Langkah 4 – Styling Formulir (Fokus pada UX)

```

form {
  background-color: white;
  padding: 20px;
  border-radius: 8px; /* Sudut melengkung */
  box-shadow: 0 2px 5px rgba(0,0,0,0.1);
  max-width: 600px;
  margin: 0 auto; /* Formulir berada di tengah */
}

label {
  display: block;
  margin-bottom: 5px;
  font-weight: bold;
}

input[type="text"],
input[type="number"],
input[type="date"],
select,
textarea {
  width: 100%; /* Lebar penuh */
  padding: 10px;
  margin-bottom: 15px;
  border: 1px solid #ccc;
  border-radius: 4px;
  box-sizing: border-box; /* Padding tidak menambah lebar total */
}

/* Styling khusus tombol */
button {
  background-color: #27ae60; /* Hijau */
  color: white;
  padding: 12px 20px;
  border: none;
  border-radius: 4px;
  cursor: pointer;
  width: 100%;
  font-size: 16px;
}

button:hover {

```

```
background-color: #219150; /* Hijau lebih gelap saat hover */  
}
```

Screenshot Hasil:

The screenshot shows a web form titled "Input Data Harian" (Daily Data Input). The form is contained within a light gray box. It has the following fields:

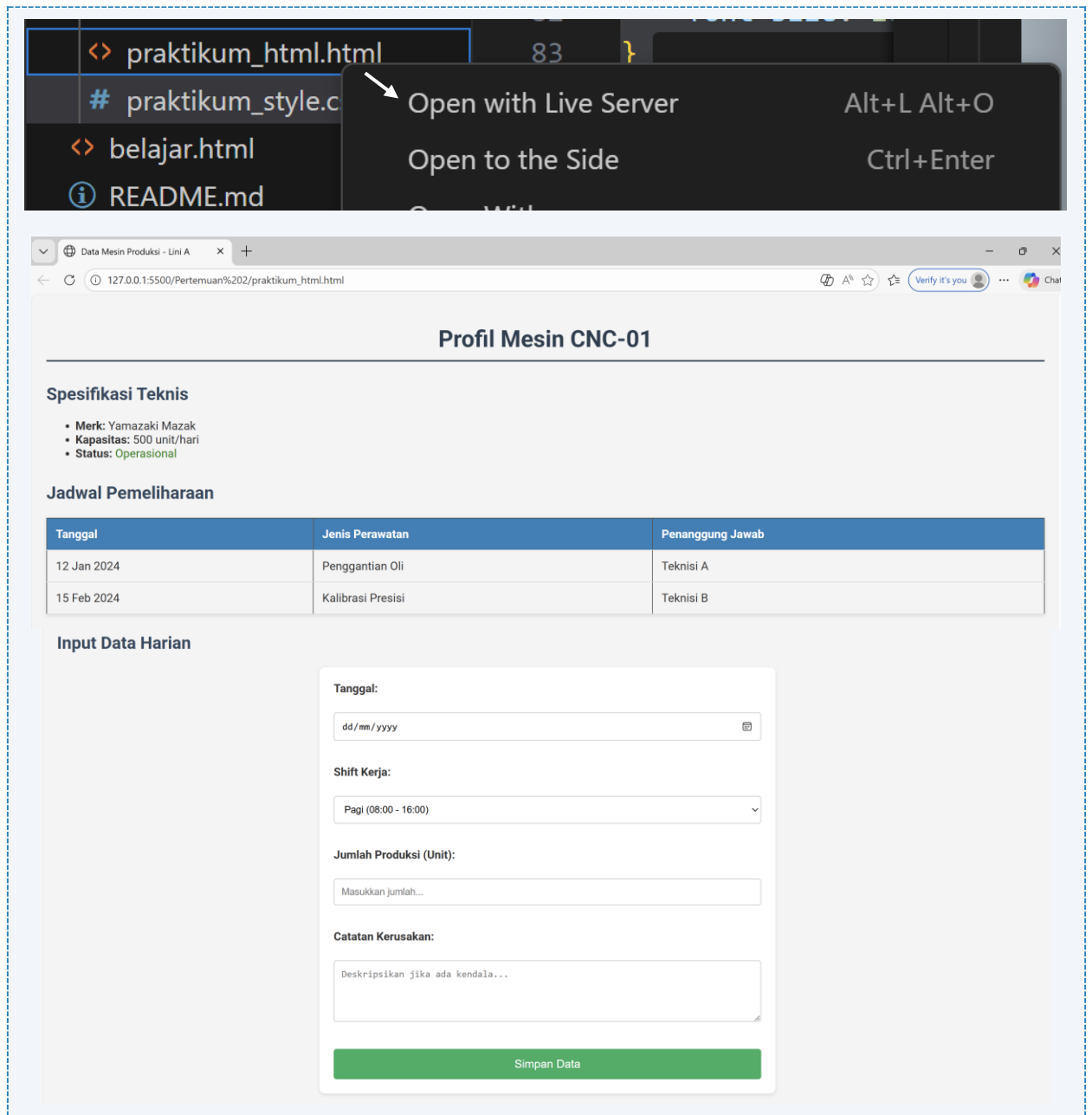
- Tanggal:** A date input field with a placeholder "dd/mm/yyyy" and a calendar icon on the right.
- Shift Kerja:** A dropdown menu currently showing "Pagi (08:00 - 16:00)".
- Jumlah Produksi (Unit):** A text input field with a placeholder "Masukkan jumlah..."
- Catatan Kerusakan:** A larger text area with a placeholder "Deskripsikan jika ada kendala..." and a small icon at the bottom right.
- Simpan Data:** A green button at the bottom of the form.

Gambar E.4 – Styling Formulir (Fokus pada UX)

E.5 Langkah 5 – Melihat Hasil

```
Menyimpan semua file. Browser (Live Server aktif) otomatis refresh.
```

Screenshot Hasil:



Gambar E.5 – Menjalankan Live Server dan Hasil File

F. LATIHAN DAN TUGAS MANDIRI

Jawab dan dokumentasikan seluruh latihan/tugas yang ada di bagian "Latihan/Tugas Mandiri" pada modul Anda.

F.1 Latihan 1 – Class Selector

Pertanyaan / Instruksi Latihan:

Berikan kelas `status-ok` pada teks status operasional di HTML, dan berikan warna hijau tebal di CSS. Berikan kelas `status-down` dan berikan warna merah.

Jawaban / Kode Solusi:

Kode jawaban latihan 1 (HTML)

```

<h2>Spesifikasi Teknis</h2>
<ul>
  <li><strong>Merk:</strong> Yamazaki Mazak</li>
  <li><strong>Kapasitas:</strong> 500 unit/hari</li>
  <!-- Tambah class status ok di text operasional -->
  <li><strong>Status:</strong> <span class="status-ok">Operasional</span></li>
</ul>
<ul>
  <li><strong>Merk:</strong> Okuma</li>
  <li><strong>Kapasitas:</strong> 400 unit/hari</li>
  <!-- Tambah class status down di text tidak operasional -->
  <li><strong>Status:</strong> <span class="status-down">Tidak Operasional</span></li>
</ul>

```

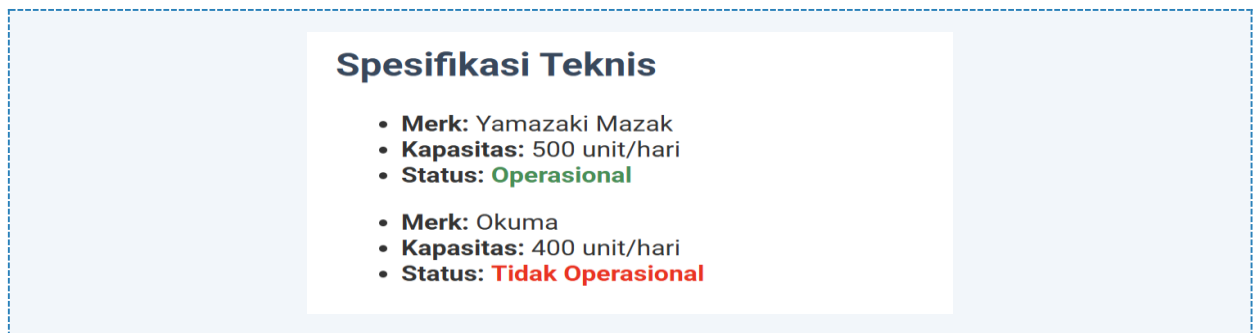
Kode jawaban latihan 1 (CSS)

```

button:hover {
  background-color: #219150; /* Hijau lebih gelap saat hover */
}
/* Tambah Style status ok dan status down (latihan 1) */
.status-ok {
  color: #219150;          /* Hijau tebal sesuai latihan 1 */
  font-weight: bold;
}

.status-down {
  color: red;              /* Merah tebal sesuai latihan 1 */
  font-weight: bold;
}

```



Gambar F.1 – Hasil Class Selector

F.2 Latihan 2 – Layout Container

Pertanyaan / Instruksi Latihan:

Buat sebuah `

` dengan id `container` di HTML yang membungkus seluruh konten. Di CSS, berikan `max-width: 800px`, `margin: 0 auto`, dan `background: white` agar konten terlihat seperti kertas di tengah layar.

Jawaban / Kode Solusi:

Kode jawaban latihan 2 (HTML)

```

<body>
  <!-- Tambah `<div>` dengan id `container` -->
  <div id="container">
    <h1>Profil Mesin CNC-01</h1>
    <hr>

    <h2>Spesifikasi Teknis</h2>
    <ul>
      <li><strong>Merk:</strong> Yamazaki Mazak</li>
      <li><strong>Kapasitas:</strong> 500 unit/hari</li>
      <!-- Tambah class status ok di text operasional -->
      <li><strong>Status:</strong> <span class="status-ok">Operasional</span></li>
    </ul>
    <ul>
      <li><strong>Merk:</strong> Okuma</li>
      <li><strong>Kapasitas:</strong> 400 unit/hari</li>
      <!-- Tambah class status down di text tidak operasional -->
      <li><strong>Status:</strong> <span class="status-down">Tidak
Operasional</span></li>
    </ul>
    <h2>Jadwal Pemeliharaan</h2>
    <table border="1" cellpadding="10">
      <thead>
        <tr>
          <th>Tanggal</th>
          <th>Jenis Perawatan</th>
          <th>Penanggung Jawab</th>
        </tr>
      </thead>
      <tbody>
        <tr>
          <td>12 Jan 2024</td>
          <td>Penggantian Oli</td>
          <td>Teknisi A</td>
        </tr>
        <tr>
          <td>15 Feb 2024</td>
          <td>Kalibrasi Presisi</td>
          <td>Teknisi B</td>
        </tr>
      </tbody>
    </table>
    <hr>
    <h2>Input Data Harian</h2>
    <form action="">
      <label for="tanggal">Tanggal:</label><br>
      <input type="date" id="tanggal" name="tanggal" required><br><br>
      <label for="shift">Shift Kerja:</label><br>
      <select id="shift" name="shift">
        <option value="Pagi">Pagi (08:00 - 16:00)</option>
        <option value="Siang">Siang (16:00 - 24:00)</option>
        <option value="Malam">Malam (00:00 - 08:00)</option>
      </select><br><br>
      <label for="jumlah">Jumlah Produksi (Unit):</label><br>
      <input type="number" id="jumlah" name="jumlah" min="0"
placeholder="Masukkan jumlah..."><br><br>
      <label for="catatan">Catatan Kerusakan:</label><br>
      <textarea id="catatan" name="catatan" rows="4" cols="50"
placeholder="Deskripsikan jika ada
kendala..."></textarea><br><br>

```

```
        <button type="submit">Simpan Data</button>
    </form>
</div>
```

Kode jawaban latihan 2 (CSS)

```
/* Tambah Style container sesuai latihan 2 */
#container {
    max-width: 800px;
    margin: 0 auto;          /* Tengah horizontal */
    background: white;
    padding: 20px;
    border-radius: 15px;
    box-shadow: 0 0 10px rgba(0,0,0,0.1);
}
```

Profil Mesin CNC-01

Spesifikasi Teknis

- Merk: Yamazaki Mazak
- Kapasitas: 500 unit/hari
- Status: **Operasional**
- Merk: Okuma
- Kapasitas: 400 unit/hari
- Status: **Tidak Operasional**

Jadwal Pemeliharaan

Tanggal	Jenis Perawatan	Penanggung Jawab
12 Jan 2024	Penggantian Oli	Teknisi A
15 Feb 2024	Kalibrasi Presisi	Teknisi B

Input Data Harian

Tanggal:

dd/mm/yyyy

Shift Kerja:

Pagi (08:00 - 16:00)

Jumlah Produksi (Unit):

Masukkan jumlah...

Catatan Kerusakan:

Deskripsikan jika ada kendala...

Simpan Data

Gambar F.2 – Hasil Layout Container

F.3 Tugas Proyek Mini – Industri

Deskripsi proyek mini:

Terapkan styling CSS yang sama ke file `laporan\_kualitas.html` yang Anda buat di pertemuan 1.

- a. Tambahkan aturan CSS agar tombol "Reset" (jika ada) berwarna merah `#c0392b`).
- b. Pastikan form responsif (tetap rapi saat di-zoom).

Penjelasan pendekatan/solusi yang digunakan:

Langkah Penyelesaian Proyek Mini Pertemuan 2

1. Menyiapkan alat dan aplikasi praktikum

Praktikum mini proyek pertemuan kedua diawali dengan menyiapkan perangkat yang diperlukan, yaitu laptop atau komputer sebagai perangkat utama. Aplikasi *Visual Studio Code* digunakan sebagai *code editor* untuk menuliskan dan mengelola file HTML serta CSS. Browser Google Chrome dimanfaatkan untuk menampilkan dan menguji hasil tampilan halaman web. Selain itu, ekstensi *Live Server* digunakan untuk menjalankan file HTML secara lokal dan menampilkan perubahan secara real-time. Untuk keperluan dokumentasi laporan praktikum, digunakan ekstensi *CodeSnap* yang berfungsi mengambil tangkapan layar kode program agar dokumentasi kode terlihat lebih jelas, rapi, dan mudah dipahami.

2. Menyiapkan File Proyek

Langkah kedua yang dilakukan adalah menyiapkan file HTML hasil proyek mini pada Pertemuan 1, yaitu laporan_kualitas.html. File ini berisi formulir Laporan Inspeksi Kualitas yang terdiri dari input ID produk, nama inspektur, jenis cacat, jumlah cacat, unggahan foto produk, serta tombol submit dan reset. File ini menjadi dasar untuk penerapan *styling* CSS pada Pertemuan 2.

3. Membuat dan Menghubungkan File CSS Eksternal

Selanjutnya dibuat file CSS eksternal bernama laporan_kualitas.css. File CSS ini kemudian dihubungkan ke file HTML melalui tag `<link>` yang diletakkan di dalam bagian `<head>`. Penggunaan CSS eksternal dipilih agar kode lebih rapi, mudah dipelihara, dan sesuai dengan praktik standar pengembangan web.

4. Mengatur Styling Dasar Halaman (Body dan Tipografi)

Pada tahap ini dilakukan pengaturan tampilan dasar halaman dengan mengatur *font* menggunakan *Google Fonts (Roboto)*, warna latar belakang halaman, warna teks, margin, dan padding. Pengaturan ini bertujuan untuk meningkatkan keterbacaan serta memberikan tampilan yang lebih profesional dan konsisten dengan standar antarmuka sistem industri.

5. Menata Header dan Informasi Pendukung

Bagian header yang berisi judul laporan, deskripsi singkat, dan identitas penyusun diberi styling agar tampil terpusat dan rapi. Penambahan garis pemisah (*horizontal rule*) digunakan untuk membedakan bagian header dengan formulir utama sehingga struktur halaman menjadi lebih jelas secara visual.

6. Styling Formulir Laporan Inspeksi

Formulir laporan inspeksi kualitas diberi latar belakang putih, padding, sudut membulat, serta efek bayangan (*box-shadow*). Formulir juga dibatasi dengan *max-width* dan diletakkan di tengah halaman. Tujuan langkah ini adalah agar formulir mudah dibaca, nyaman digunakan, dan menyerupai tampilan dokumen kerja profesional di lingkungan industri.

7. Pengaturan Input agar Responsif

Seluruh elemen input seperti *text field*, *number*, *file*, *select*, dan *textarea* diatur dengan lebar 100% serta menggunakan properti *box-sizing: border-box*. Dengan pengaturan ini, ukuran elemen tetap stabil meskipun halaman diperbesar (*zoom*) atau diakses pada resolusi layar yang berbeda. Hal ini memastikan formulir tetap rapi dan responsif.

8. Styling Tombol Submit dan Reset

Tombol Submit diberi warna hijau sesuai panduan praktikum sebagai simbol aksi utama pengiriman data. Tombol Reset diberi warna merah dengan kode `#c0392b` untuk menandakan fungsi penghapusan data. Selain itu, ditambahkan efek *hover* pada kedua tombol agar pengguna mendapatkan umpan balik visual saat berinteraksi dengan sistem.

9. Penggunaan *Container* untuk Tata Letak

Seluruh konten halaman dibungkus menggunakan elemen `<div class="container">`. *Container* ini diatur dengan lebar maksimum dan latar belakang putih sehingga konten terlihat seperti dokumen kerja yang berada di tengah layar. Langkah ini sesuai dengan Latihan 2 pada Pertemuan 2 mengenai pengaturan layout menggunakan *container*.

10. Pengujian Menggunakan *Live Server*

Setelah seluruh *styling* diterapkan, file HTML dijalankan menggunakan ekstensi *Live Server* pada *Visual Studio Code*. Pengujian dilakukan untuk memastikan tampilan sesuai, formulir tetap responsif saat di-*zoom*, dan seluruh komponen CSS berfungsi dengan baik tanpa *error*.

Kode Utama:

Kode jawaban Proyek Mini (HTML)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link rel="stylesheet" href="laporan_kualitas.css">
  <title>Laporan Inspeksi Kualitas</title>
</head>

<body>
  <div class="container">
    <header>
      <h1>Laporan Inspeksi Kualitas</h1>
      <p>Formulir pencatatan hasil inspeksi produk</p>
      <p>Disusun oleh : Maessa Andrea Vallenia (23051430044)</p>
    </header>
    <section>
      <!-- Formulir Laporan Inspeksi Kualitas -->
      <form action="">

        <!-- ID Produk -->
        <label for="id_produk">ID Produk:</label><br>
        <input type="text" id="id_produk" name="id_produk"
placeholder="Masukkan ID produk"><br><br>
```

```

        <!-- Nama Inspektur -->
        <label for="nama_inspektur">Nama Inspektur:</label><br>
        <input type="text" id="nama_inspektur" name="nama_
inspektur" placeholder="Masukkan nama inspektur"><br><br>

        <!-- Jenis Cacat -->
        <label>Jenis Cacat:</label><br>
        <!-- Radio Geometri -->
        <input type="radio" id="geometri" name="jenis_
cacat" value="Geometri">
        <label for="geometri">Geometri</label><br>
        <!-- Radio Permukaan -->
        <input type="radio" id="permukaan" name="jenis_
cacat" value="Permukaan">
        <label for="permukaan">Permukaan</label><br>
        <!-- Radio Fungsional -->
        <input type="radio" id="fungsional" name="jenis_
cacat" value="Fungsional">
        <label for="fungsional">Fungsional</label><br><br>

        <!-- Jumlah Cacat -->
        <label for="jumlah_cacat">Jumlah Cacat:</label><br>
        <input type="number" id="jumlah_cacat" name="jumlah_cacat"
min="0" placeholder="Masukkan jumlah cacat"><br><br>

        <!-- Foto Produk (Placeholder) -->
        <label for="foto_produk">Foto Produk:</label><br>
        <input type="file" id="foto_produk" name="foto_produk"><br>
<br>

        <!-- Tombol Submit & Reset -->
        <button type="submit">Kirim Laporan</button>
        <button type="reset">Reset data</button>
    </form>
</section>
</div>
</body>
</html>

```

Kode jawaban Proyek Mini (CSS)

```

/* Import Font dari Google Fonts */
@import
url('https://fonts.googleapis.com/css2?family=Roboto:wght@400;700&display=swap');

body {
    font-family: 'Roboto', sans-serif;
    background-color: #f4f4f4; /* Warna abu-abu muda untuk background */
    color: #333;
    margin: 10px;
    padding: 20px;
}

/* Header styling */
h1 {
    color: #2c3e50;
    text-align: center;
    margin-bottom: 5px;
}

p {
    text-align: center;
}

```

```
    margin: 2px 0;
    font-size: 14px;
    color: #555;
}

hr {
    margin: 15px 0 20px 0;
    border: 0;
    border-top: 2px solid #2980b9; /* Biru profesional */
}

/* Formulir styling */
form {
    background-color: white;
    padding: 20px;
    border-radius: 8px;
    box-shadow: 0 2px 5px rgba(0, 0, 0, 0.1);
    max-width: 600px;
    margin: 0 auto; /* Formulir berada di tengah */
}

label {
    display: block;
    margin-bottom: 5px;
    font-weight: bold;
}

input[type="text"],
input[type="number"],
input[type="file"],
select,
textarea {
    width: 100%; /* Lebar penuh */
    padding: 10px;
    margin-bottom: 15px;
    border: 1px solid #ccc;
    border-radius: 4px;
    box-sizing: border-box; /* Padding tidak menambah lebar total */
}

/* Styling khusus tombol */
button {
    padding: 12px 20px;
    border: none;
    border-radius: 4px;
    cursor: pointer;
    font-size: 16px;
    width: 100%;
    margin-bottom: 10px;
    color: white;
}

/* Tombol submit (hijau resmi latihan 2) */
button[type="submit"] {
    background-color: #27ae60;
}

button[type="submit"]:hover {
    background-color: #219150;
}

/* Tombol reset (merah resmi latihan 2) */
```

```
button[type="reset"] {  
  background-color: #c0392b;  
}  
  
button[type="reset"]:hover {  
  background-color: #e74c3c;  
}  
  
/* Tambah style container sesuai latihan 2 */  
.container {  
  max-width: 800px;  
  margin: auto;  
  background: white;  
  padding: 20px;  
  border-radius: 15px;  
}
```

Laporan Inspeksi Kualitas
Formulir pencatatan hasil inspeksi produk
Disusun oleh : Maessa Andrea Vallenia (23051430044)

ID Produk:

Nama Inspektur:

Jenis Cacat:
☐ Geometri
☐ Permukaan
☐ Fungsional

Jumlah Cacat:

Foto Produk:
 No file chosen

Gambar F.3 – Hasil Proyek Mini Industri (Laporan Kualitas)

G. PEMBAHASAN

Hasil praktikum menunjukkan bahwa penerapan CSS pada halaman web yang dibuat berhasil memberikan tampilan yang rapi, konsisten, dan mudah digunakan. Penggunaan external CSS melalui file terpisah memungkinkan pemisahan yang jelas antara struktur HTML dan styling, sehingga kode menjadi lebih terorganisir, mudah dibaca, dan mudah dipelihara. Elemen-elemen formulir seperti `<input>`, `<select>`, `<textarea>`, dan `<button>` berhasil distandarisasi ukurannya, dengan jarak antar field yang cukup, berkat pengaturan padding dan margin yang konsisten. Border-radius dan box-shadow pada form dan container menambah kesan modern dan profesional, sementara properti box-sizing: border-box memastikan bahwa padding dan border tidak mempengaruhi total lebar elemen, menjaga tata letak tetap proporsional.

Penerapan selector CSS juga terlihat efektif. Selector elemen seperti `body`, `h1`, `p`, dan `form` berhasil mengatur tampilan dasar, sedangkan selector atribut seperti `input[type="text"]` atau `button[type="submit"]` memberikan styling spesifik yang sesuai kebutuhan, tanpa menambah banyak class atau ID. Hal ini memperlihatkan bahwa elemen yang berbeda dapat diberikan gaya secara tepat sasaran. Pseudo-class `:hover` pada tombol submit dan reset berfungsi dengan baik, memberikan umpan balik visual saat pengguna berinteraksi, sehingga meningkatkan pengalaman pengguna (UI/UX) dan mengurangi risiko kesalahan input.

Secara keseluruhan, hasil praktikum membuktikan bahwa CSS dapat meningkatkan kualitas antarmuka halaman web secara signifikan, membuatnya lebih nyaman, menarik, dan interaktif. Namun, secara kritis, beberapa aspek masih bisa ditingkatkan. Layout saat ini efektif di tampilan desktop, tetapi perlu diuji pada berbagai ukuran layar untuk memastikan responsivitas. Selain itu, meskipun selector atribut digunakan dengan tepat, penggunaan tambahan class atau ID bisa meningkatkan skalabilitas dan mempermudah pengelolaan styling ketika proyek berkembang menjadi lebih kompleks. Dengan demikian, praktikum ini tidak hanya menunjukkan keberhasilan penerapan konsep CSS, tetapi juga membuka peluang untuk pengembangan lebih lanjut yang mempertimbangkan fleksibilitas, aksesibilitas, dan adaptasi pada berbagai perangkat.

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum yang diperoleh telah sesuai dengan teori dan konsep CSS yang dipelajari pada modul, khususnya terkait penerapan *external CSS*, *selector*, *box model*, dan *styling form*. Hal ini terlihat dari penggunaan `<link rel="stylesheet" href="laporan_kualitas.css">` pada bagian `<head>` dokumen HTML, yang menunjukkan bahwa styling dilakukan secara terpisah dari struktur HTML. Pendekatan ini sesuai dengan standar industri karena memisahkan antara struktur (HTML) dan tampilan (CSS), sehingga kode menjadi lebih rapi, mudah dipelihara, dan *scalable*.

Penerapan CSS *selector* juga sesuai dengan teori. Selector elemen seperti `body`, `h1`, `p`, dan `form` digunakan untuk mengatur tampilan dasar halaman, sementara selector atribut seperti `input[type="text"]`, `input[type="number"]`, `input[type="file"]`, serta `button[type="submit"]` dan `button[type="reset"]` digunakan untuk styling yang lebih spesifik. Hal ini menunjukkan pemahaman bahwa selector CSS dapat digunakan untuk menargetkan elemen tertentu tanpa harus menambahkan banyak *class* atau *id*, sebagaimana dijelaskan dalam materi selector CSS.

Konsep Box Model diterapkan secara konsisten pada berbagai elemen. Properti `padding` digunakan untuk memberi ruang antara konten dan batas elemen, seperti pada `form` dan `.container`, sehingga tampilan tidak terlalu rapat. Properti `margin: 0 auto` pada `form` digunakan untuk memposisikan formulir di tengah halaman, sedangkan *`border-radius`* dan *`box-shadow`* diterapkan untuk memberikan kesan antarmuka modern dan profesional. Penggunaan *`box-sizing: border-box`* memastikan bahwa `padding` tidak menambah lebar total elemen, sesuai dengan teori box model yang dibahas pada modul.

Selain itu, penerapan *`pseudo-class`* `:hover` pada tombol `<button>` telah sesuai dengan konsep interaksi pengguna (UI/UX). Ketika pengguna mengarahkan kursor ke tombol `submit` atau `reset`, warna tombol berubah, memberikan umpan balik visual yang jelas. Hal ini membuktikan bahwa teori *`pseudo-class`* dinamis telah diterapkan dengan benar dan berfungsi sebagaimana mestinya pada output halaman web.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Warna hijau pada status Operasional (<code>.status-ok</code>) tidak muncul pada halaman web meskipun <code>class</code> sudah ditambahkan pada elemen HTML.	Setelah dilakukan pengecekan, diketahui bahwa komentar pada file CSS ditulis menggunakan sintaks HTML (<code><!-- --></code>). Komentar tersebut menyebabkan browser mengabaikan sebagian aturan CSS. Solusi yang diterapkan adalah mengganti komentar menjadi format komentar CSS yang benar, yaitu <code>/* ... */</code> , sehingga aturan <code>.status-ok { color: #219150; font-weight: bold; }</code> dapat terbaca dan dijalankan dengan baik.

G.3 Kaitan dengan Penerapan di Industri

Materi CSS yang dipelajari pada modul ini memiliki keterkaitan yang sangat kuat dengan penerapan di dunia Teknik Industri, khususnya dalam perancangan sistem informasi operasional berbasis web. Dalam konteks industri manufaktur, insinyur industri sering terlibat dalam pengembangan sistem pelaporan produksi, inspeksi kualitas, dan monitoring kondisi mesin. CSS berperan penting dalam memastikan antarmuka sistem tersebut mudah dibaca, konsisten, dan tidak membingungkan operator lapangan yang menggunakannya setiap hari.

Penerapan konsep *Box Model* (*margin*, *border*, dan *padding*) pada proyek mini laporan inspeksi kualitas merupakan contoh nyata bagaimana tata letak memengaruhi efektivitas kerja. Formulir yang diberi *padding* dan *margin* yang tepat akan mengurangi kepadatan visual, sehingga operator dapat mengisi data dengan lebih cepat dan minim kesalahan. Dalam lingkungan produksi yang menuntut kecepatan dan ketelitian, desain antarmuka yang buruk dapat meningkatkan risiko *human error*, seperti salah memasukkan jumlah cacat atau jenis cacat produk.

Penggunaan selector CSS seperti selector kelas (*.container*, *.status-ok*, *.status-down*) juga mencerminkan praktik standar industri dalam pengembangan sistem. Dengan selector kelas, satu aturan visual dapat diterapkan secara konsisten pada banyak elemen. Dalam sistem industri berskala besar, misalnya dashboard pemantauan mesin, pendekatan ini memudahkan pemeliharaan sistem karena perubahan tampilan dapat dilakukan tanpa harus mengubah struktur HTML secara menyeluruh.

Selain itu, penerapan warna sebagai indikator status (hijau untuk operasional dan merah untuk tidak operasional) merupakan praktik umum dalam sistem industri seperti SCADA, *dashboard* OEE, dan sistem *maintenance*. Warna digunakan sebagai sinyal visual cepat agar pengguna dapat langsung memahami kondisi sistem tanpa membaca teks secara detail. Pada proyek mini ini, penerapan warna melalui CSS menunjukkan bagaimana prinsip visual management dalam Teknik Industri dapat diterapkan dalam pengembangan aplikasi web sederhana.

Terakhir, kemampuan membuat formulir HTML yang distyling menggunakan CSS menjadi fondasi awal untuk pengembangan sistem digital yang lebih kompleks, seperti sistem pelaporan kualitas berbasis web, *e-form* audit internal, atau aplikasi pencatatan kerusakan mesin. Dengan dasar ini, mahasiswa Teknik Industri tidak hanya memahami konsep teknis pemrograman web, tetapi juga mampu mengintegrasikannya dengan kebutuhan sistem kerja industri yang menuntut efisiensi, akurasi data, dan kemudahan penggunaan.

H. KESIMPULAN

Tuliskan 3–5 poin kesimpulan yang merangkum hal-hal yang dipelajari dan dicapai pada praktikum modul ini. Kesimpulan harus spesifik, bukan bersifat umum.

1	Penerapan CSS eksternal berhasil memisahkan tampilan dan struktur HTML. File laporan_kualitas.css digunakan untuk mengatur seluruh <i>styling</i> halaman, sedangkan laporan_kualitas.html tetap fokus pada struktur konten. Hal ini membuat kode lebih rapi, mudah dipelihara, dan mengikuti praktik standar industri. Seluruh perubahan tampilan, termasuk warna, <i>font</i> , <i>margin</i> , dan <i>padding</i> , diterapkan melalui file CSS eksternal sehingga mempermudah perbaikan atau pembaruan desain di masa depan.
2	Penggunaan selector CSS dan <i>pseudo-class</i> bekerja sesuai teori. Selector elemen seperti body, h1, p, dan form mengatur tampilan umum halaman, sedangkan selector spesifik seperti .status-ok, .status-down, button[type="submit"], dan button[type="reset"] berhasil menampilkan warna hijau untuk status operasional, merah untuk status tidak operasional, serta efek hover pada tombol submit dan reset. Hal ini membuktikan penguasaan selector, <i>pseudo-class</i> , dan penggunaan atribut CSS sesuai modul.
3	Box Model diterapkan secara konsisten untuk tata letak dan estetika. Elemen .container dan form menggunakan padding, margin: 0 auto, border-radius, dan <i>box-shadow</i> , sehingga konten dan formulir tampil rapi, tidak menumpuk, dan berada di tengah layar. Properti <i>box-sizing: border-box</i> diterapkan pada input dan textarea sehingga ukuran elemen tetap proporsional walaupun padding ditambah, memastikan <i>layout</i> responsif dan ergonomis untuk pengguna.
4	Formulir dan elemen input dibuat responsif dan <i>user-friendly</i> . Semua input (<i>text</i> , <i>number</i> , <i>file</i> , <i>select</i> , <i>textarea</i>) memiliki lebar penuh (<i>width</i> : 100%) dan jarak antar elemen diatur menggunakan margin dan padding. Tombol submit diberi warna hijau dan tombol reset merah (#c0392b) sesuai standar visual management, sehingga pengguna dapat dengan cepat mengenali fungsi tombol. Pengaturan ini menjaga kenyamanan, mengurangi risiko human error saat pengisian data laporan kualitas, dan menjamin konsistensi tampilan di berbagai ukuran layar.
5	Penerapan praktikum relevan dengan praktik industri. Tata letak halaman, indikator warna, <i>styling</i> formulir, dan efek hover mencontohkan prinsip visual management, ergonomi, dan standarisasi dalam sistem industri, khususnya pada laporan inspeksi kualitas berbasis web. Hasil praktikum menunjukkan bagaimana CSS tidak hanya mempercantik tampilan, tetapi juga mendukung efektivitas kerja operator dan akurasi pengisian data, sesuai tujuan Teknik Industri untuk meningkatkan produktivitas, mengurangi kesalahan manusia, dan memudahkan pengambilan keputusan berbasis data.

I. DAFTAR PUSTAKA

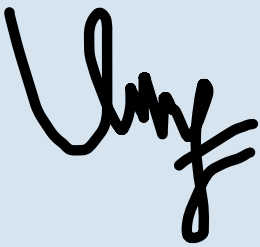
Tuliskan semua sumber referensi yang digunakan dalam format APA 7th Edition. Minimal 3 referensi yang relevan (termasuk Manual Praktikum dan sumber online resmi seperti MDN, Reactjs.org, dll).

No.	Referensi (Format APA 7th Edition)
-----	------------------------------------

[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Sholikhan, M. (2022). Html, Css Dan Javascript. <i>Penerbit Yayasan Prima Agus Teknik</i> , 1-343.
[3]	Sinlae, F., Bowono, P., Christian, H. R., & Elfan, A. (2024). Pelatihan Dalam Desain Website Menggunakan HTML dan CSS. <i>Jurnal Ilmu Multidisiplin</i> , 3(2), 190-198.
[4]	Kusumawardani, D. M., Astiti, S., Fathoni, M. Y., Sunardi, D., & Fernandez, S. (2023). <i>Web Dasar Menggunakan HTML, CSS, JS, PHP dan Studi Kasus</i> . PT. Sonpedia Publishing Indonesia.

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Diperiksa oleh Asisten / Dosen
 Maessa Andrea Vallenia NIM: 23051430044	 Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____

MODUL 3

LAYOUT CSS LANJUTAN (FLEXBOX & GRID) + BOOTSTRAP

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 3 (Pertemuan 3)
Topik Utama	CSS Flexbox, CSS Grid, dan Framework CSS (Bootstrap 5) untuk prototyping cepat
Sub-Topik	Flexbox (row, column, alignment), Grid (rows, columns, area), layout responsif, komponen Bootstrap 5, prototyping antarmuka web
Durasi Praktikum	340 Menit
Tanggal Praktikum	27 Februari 2026
Nama Mahasiswa	Maessa Andrea Vallenia
NIM	23051430044
Kelas	A

B. TUJUAN PEMBELAJARAN

Tuliskan tujuan pembelajaran yang ingin dicapai pada modul ini, sesuai dengan yang tertera di Manual Praktikum.

1	Memahami konsep layout modern menggunakan Flexbox dan Grid.
2	Mengenal dan menggunakan Framework CSS (Bootstrap) untuk mempercepat pengembangan UI.
3	Mampu membuat layout dashboard responsif (Sidebar + Main Content).
4	Menerapkan komponen Bootstrap (Card, Navbar, Button) pada konteks industri.

C. ALAT DAN BAHAN

Sebutkan seluruh perangkat keras, perangkat lunak, dan sumber referensi yang digunakan selama praktikum.

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	<i>Code Editor</i>

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
2	Google Chrome / Browser Modern	Versi terbaru	<i>Testing & Debugging</i>
3	Live Server	Versi 5.7.9	Menjalankan HTML dan memantau perubahan file
4	Bootstrap 5 (CDN)	Versi 5.3.8	Framework CSS untuk layout responsif dan komponen UI
5	CodeSnap	Versi 1.3.4	Dokumentasi kode program
6	Git dan GitHub	Versi 2.53.0	Version control system (VCS) berbasis Git.
7	Koneksi Internet	Stabil	Mengakses Library dan Framework.

D. DASAR TEORI

1. CSS Grid Layout adalah sistem tata letak dua dimensi yang memungkinkan kita mengatur elemen dalam baris dan kolom secara fleksibel. Berbeda dengan Flexbox yang fokus pada satu arah saja (horizontal atau vertikal), CSS Grid dapat mengatur elemen secara bersamaan di kedua dimensi, sehingga memudahkan pembuatan layout kompleks dengan presisi tinggi. Dalam praktiknya, kita menggunakan properti seperti `display: grid`, `grid-template-columns`, dan `grid-template-rows` untuk menentukan ukuran, jumlah, dan posisi setiap elemen di dalam container.
2. Konsep dasar dari CSS Grid mencakup beberapa elemen penting, yaitu Grid Container sebagai elemen induk, Grid Items sebagai elemen anak, serta Grid Lines, Grid Cells, dan Grid Areas yang menentukan garis, sel, dan area tempat item ditempatkan. Dengan memahami konsep ini, seorang pengembang bisa menata layout halaman web dengan lebih rapi dan terstruktur, serta mempermudah pengaturan jarak antar elemen melalui properti `gap`, `row-gap`, dan `column-gap`.
3. Selain itu, CSS Grid memiliki fitur seperti `grid-template-areas`, `repeat()`, dan `minmax()` yang membuat layout menjadi lebih mudah dibaca dan responsif. Misalnya, fungsi `repeat(auto-fit, minmax(250px, 1fr))` memungkinkan kolom menyesuaikan ukuran secara otomatis sesuai lebar layar, sehingga

tampilan halaman tetap rapi di berbagai perangkat. Properti ini membantu desainer membuat halaman web yang fleksibel tanpa perlu menulis banyak media query.

4. Flexbox CSS merupakan sistem layout berbasis satu arah yang fokus pada pengaturan elemen secara horizontal atau vertikal. Flexbox memudahkan penataan elemen secara sejajar, pengaturan jarak antar item, dan pembuatan layout responsif. Properti penting dalam Flexbox antara lain flex-direction, justify-content, align-items, flex-wrap, dan gap, yang memungkinkan posisi dan distribusi elemen dapat diatur dengan cepat tanpa perhitungan margin manual.
5. Dengan Flexbox, elemen anak yang berada dalam flex container dapat tumbuh, menyusut, atau menyesuaikan ukuran awal menggunakan properti flex-grow, flex-shrink, dan flex-basis. Selain itu, properti order memungkinkan pengaturan urutan tampilan elemen berbeda dari urutan HTML. Hal ini membuat Flexbox sangat berguna untuk membuat layout yang fleksibel dan mudah diubah tanpa merombak struktur HTML dasar.
6. Bootstrap adalah framework front-end berbasis HTML, CSS, dan JavaScript yang dirancang untuk mempercepat proses pengembangan web responsif. Bootstrap menyediakan sistem grid siap pakai, komponen UI, dan utilitas CSS yang mempermudah pembuatan layout modern. Dengan memanfaatkan Bootstrap, pengembang dapat membuat halaman web mobile-first, memastikan tampilan konsisten di berbagai ukuran layar, serta memanfaatkan komponen interaktif seperti navbar, tombol, carousel, dan badge dengan cepat. Kekurangan Bootstrap adalah gaya visual defaultnya cenderung seragam dan ukuran file bisa membengkak jika terlalu banyak komponen digunakan.

D.1 Konsep Utama

CSS Grid Layout adalah sistem tata letak modern yang memungkinkan pengaturan elemen dalam dua dimensi sekaligus, yaitu baris dan kolom. Dengan CSS Grid, pengembang dapat menempatkan elemen secara presisi, menggabungkan beberapa cell menjadi satu area, dan mengatur jarak antar elemen dengan mudah menggunakan properti seperti grid-template-columns, grid-template-rows, grid-area, dan gap. Konsep dasar CSS Grid seperti Grid Container, Grid Items, Grid Lines, dan Grid Cells membuat layout menjadi terstruktur dan mudah dimodifikasi, sehingga sangat cocok untuk membuat halaman kompleks seperti dashboard atau formulir dengan berbagai kolom dan baris.

Sementara itu, Flexbox adalah sistem tata letak satu dimensi yang fokus pada pengaturan elemen secara horizontal atau vertikal dalam container. Flexbox memudahkan penyusunan elemen, pengaturan jarak, serta penyesuaian posisi antar item menggunakan properti justify-content, align-items, flex-wrap, serta pengaturan ukuran proporsional elemen melalui flex-grow, flex-shrink, dan flex-basis. Flexbox sangat efektif untuk layout yang responsif, misalnya navbar, menu, atau daftar konten yang menyesuaikan ukuran layar, karena elemen dapat tumbuh, menyusut, atau berubah urutan dengan mudah tanpa merombak struktur HTML.

Selain itu, Bootstrap merupakan framework front-end berbasis HTML, CSS, dan JavaScript yang mempermudah pembuatan website responsif dan mobile-first. Bootstrap menyediakan sistem grid siap pakai, komponen UI, dan utilitas kelas yang memudahkan pengembang menata layout, tombol, tabel, navigasi, dan elemen interaktif lainnya dengan cepat. Dengan kombinasi CSS Grid, Flexbox, dan Bootstrap, pengembang dapat membuat halaman web yang rapi, konsisten, responsif, dan mudah diubah, sehingga mempercepat proses pengembangan serta meningkatkan pengalaman pengguna di berbagai perangkat.

D.2 Keterkaitan dengan Teknik Industri

Materi modul tentang CSS Grid, Flexbox, dan Bootstrap sangat relevan dengan dunia Teknik Industri karena banyak proses industri modern memanfaatkan sistem informasi dan dashboard digital untuk pengelolaan operasional. Teknik Industri tidak hanya mempelajari produksi dan manajemen sumber daya, tetapi juga bagaimana informasi terkait proses, persediaan, dan kinerja dapat diakses dan dianalisis secara cepat. Dengan pemahaman layout web menggunakan CSS Grid dan Flexbox, seorang engineer industri bisa merancang dashboard yang menampilkan data stok bahan baku, jadwal produksi, atau alur kerja secara terstruktur dan mudah dibaca.

Sebagai contoh nyata, bayangkan sebuah perusahaan manufaktur yang memiliki gudang bahan baku. Dengan CSS Grid, data stok masuk, stok keluar, dan sisa bahan baku dapat ditampilkan dalam tabel multi-kolom di dashboard, sehingga manajer gudang dapat memantau persediaan secara real-time. Flexbox memungkinkan elemen-elemen seperti grafik produksi, status mesin, dan notifikasi penjadwalan ditempatkan secara fleksibel dalam satu baris atau kolom, menyesuaikan ukuran layar komputer atau tablet tanpa merusak tampilan. Dengan menggunakan Bootstrap, pengembang dapat membuat dashboard interaktif yang mobile-friendly, sehingga staf lapangan bisa mengakses informasi penting menggunakan smartphone saat melakukan inspeksi atau pengecekan gudang.

Skenario lain adalah pada perancangan layout pabrik atau workstation secara digital. Dengan konsep Grid, seorang insinyur industri dapat membuat mockup visual layout mesin, jalur produksi, dan area penyimpanan untuk menganalisis efisiensi aliran material. Flexbox bisa digunakan untuk menyesuaikan ukuran elemen visual agar proporsional dengan ruang kerja yang berbeda, sedangkan Bootstrap mempermudah pembuatan prototipe dashboard monitoring operasional dengan komponen siap pakai, seperti tabel, grafik, dan badge status. Dengan demikian, pemahaman modul ini membantu teknik industri mengintegrasikan aspek digital dan visualisasi data dalam pengambilan keputusan yang lebih cepat dan efisien.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

Dokumentasikan setiap langkah kerja yang Anda lakukan, beserta hasil yang diperoleh (screenshot, kode, atau output). Jawab sesuai urutan langkah di Manual Praktikum.

E.1 Langkah 1 – Integrasi Bootstrap 5

Buka file `index.html` dari pertemuan sebelumnya. Hapus link ke `style.css` sementara (atau biarkan di bawah agar bisa melakukan custom override). Tambahkan CDN Bootstrap di dalam ``.

```

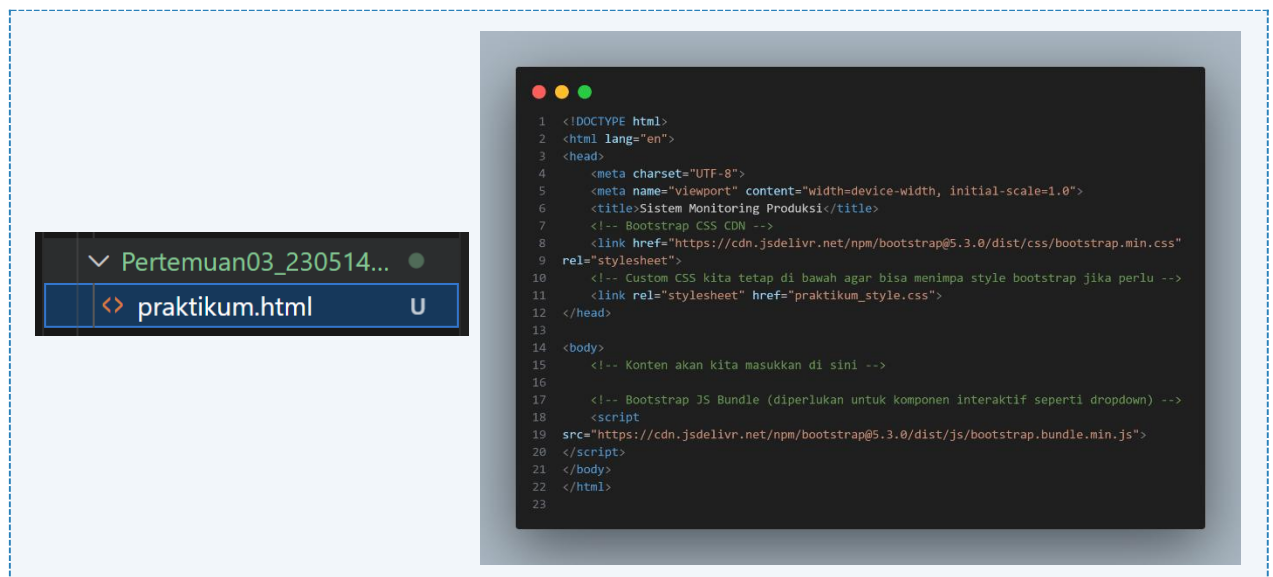
/* ===== KODE / OUTPUT LANGKAH 1 ===== */
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Sistem Monitoring Produksi</title>
  <!-- Bootstrap CSS CDN -->
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css
"
rel="stylesheet">
  <!-- Custom CSS kita tetap di bawah agar bisa menimpa style bootstrap
jika perlu -->
  <link rel="stylesheet" href="praktikum_style.css">
</head>

<body>
  <!-- Konten akan kita masukkan di sini -->

  <!-- Bootstrap JS Bundle (diperlukan untuk komponen interaktif seperti
dropdown) -->
  <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.mi
n.js">
  </script>
</body>
</html>

```

Screenshot Hasil:



Gambar E.1 – Menambahkan CDN Bootstrap di dalam `<head>

E.2 Langkah 2 – Membuat Navigasi Bar (Navbar)

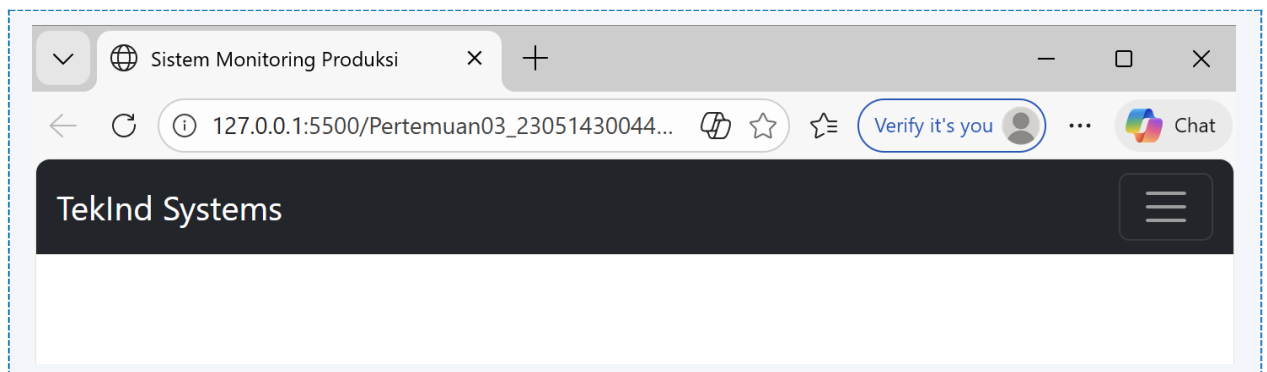
Hapus konten lama di dalam `<body>`, ganti dengan Navbar industri

```

/* ===== KODE / OUTPUT LANGKAH 2 ===== */
<body>
  <!-- Konten akan kita masukkan di sini -->
  <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <div class="container-fluid">
      <a class="navbar-brand" href="">TekInd Systems</a>
      <button class="navbar-toggler" type="button" data-bs-
toggle="collapse" data-bs-target="#navbarNav">
        <span class="navbar-toggler-icon"></span>
      </button>
      <div class="collapse navbar-collapse" id="navbarNav">
        <ul class="navbar-nav">
          <li class="nav-item">
            <a class="nav-link active" href="">Dashboard</a>
          </li>
          <li class="nav-item">
            <a class="nav-link" href="">Input Produksi</a>
          </li>
          <li class="nav-item">
            <a class="nav-link" href="">Inventori</a>
          </li>
        </ul>
      </div>
    </div>
  </nav>

```

Screenshot Hasil:



Gambar E.2 – Hasil Membuat Navigasi Bar (Navbar)

E.3 Langkah 3 – Layout Grid Dashboard (Sistem Grid)

Di bawah navbar, kita buat sistem Grid 12 kolom khas Bootstrap. Kita akan buat dashboard dengan 3 kartu di atas (KPI Utama) dan tabel di bawah.

```

/* ===== KODE / OUTPUT LANGKAH 3 ===== */
<div class="container mt-4">
  <!-- Row 1: KPI Cards (Menggunakan Grid System) -->
  <div class="row mb-4">
    <!-- Kolom lebar 4 (dari 12) -->
    <div class="col-md-4">
      <div class="card text-white bg-primary mb-3">

```



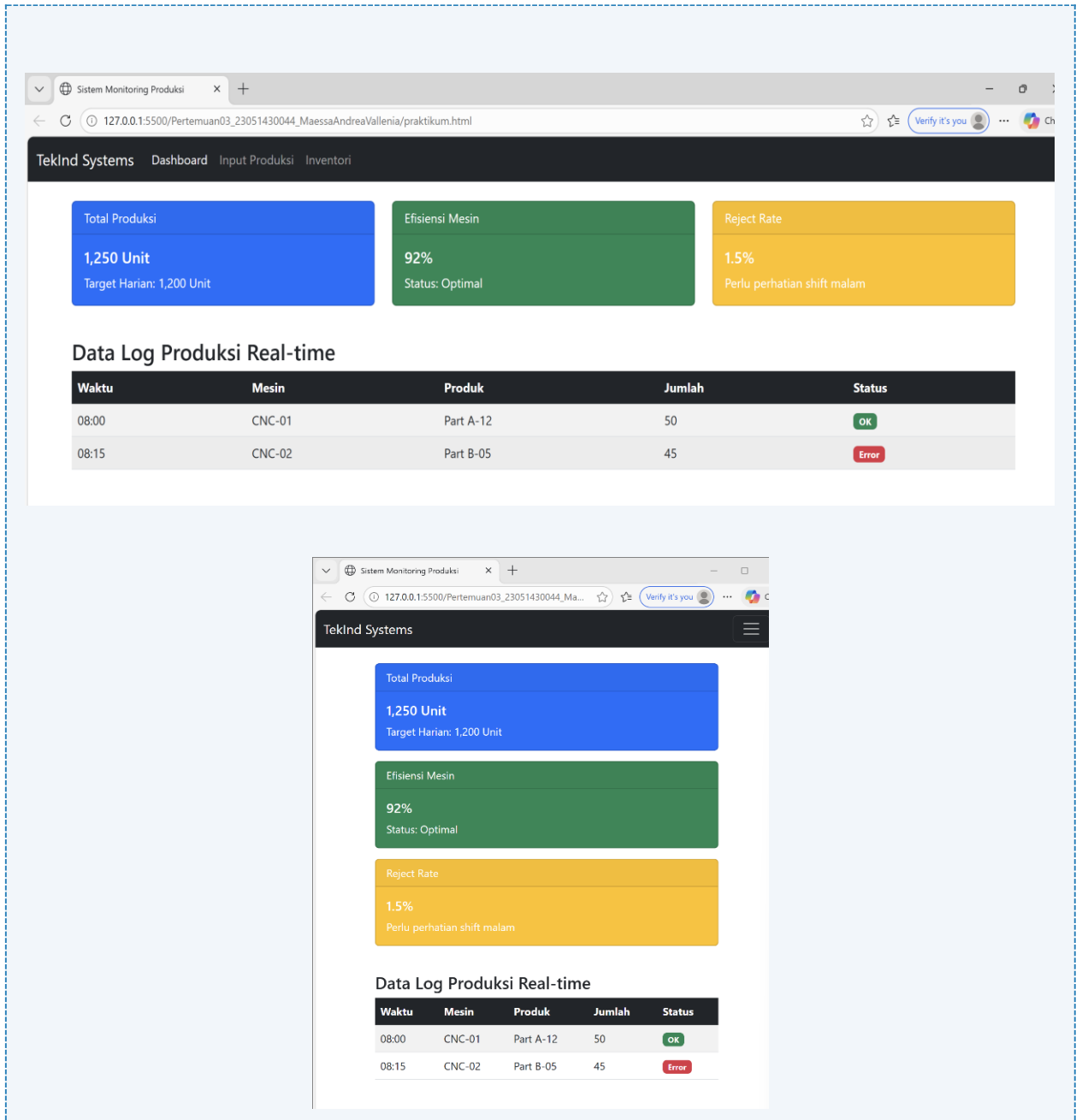
```

        <div class="card-header">Total Produksi</div>
        <div class="card-body">
            <h5 class="card-title">1,250 Unit</h5>
            <p class="card-text">Target Harian: 1,200 Unit</p>
        </div>
    </div>
</div>
<!-- Kolom lebar 4 -->
<div class="col-md-4">
    <div class="card text-white bg-success mb-3">
        <div class="card-header">Efisiensi Mesin</div>
        <div class="card-body">
            <h5 class="card-title">92%</h5>
            <p class="card-text">Status: Optimal</p>
        </div>
    </div>
</div>
<!-- Kolom lebar 4 -->
<div class="col-md-4">
    <div class="card text-white bg-warning mb-3">
        <div class="card-header">Reject Rate</div>
        <div class="card-body">
            <h5 class="card-title">1.5%</h5>
            <p class="card-text">Perlu perhatian shift malam</p>
        </div>
    </div>
</div>
</div>
<!-- Row 2: Tabel Data -->
<div class="row">
    <div class="col-12">
        <h3>Data Log Produksi Real-time</h3>
        <table class="table table-striped table-hover">
            <thead class="table-dark">
                <tr>
                    <th>Waktu</th>
                    <th>Mesin</th>
                    <th>Produk</th>
                    <th>Jumlah</th>
                    <th>Status</th>
                </tr>
            </thead>
            <tbody>
                <tr>
                    <td>08:00</td>
                    <td>CNC-01</td>
                    <td>Part A-12</td>
                    <td>50</td>
                    <td><span class="badge bg-success">OK</span></td>
                </tr>
            </tbody>
        </table>
    </div>
</div>

```

```
        </tr>
        <tr>
            <td>08:15</td>
            <td>CNC-02</td>
            <td>Part B-05</td>
            <td>45</td>
            <td><span class="badge bg-
danger">Error</span></td>
        </tr>
    </tbody>
</table>
</div>
</div>
</div>
```

Screenshot Hasil:



Gambar E.3 – Hasil Layout Grid Dashboard (Sistem Grid)

E.4 Langkah 4 – Styling Form dengan Bootstrap Classes

Kita ubah form manual tadi menjadi form rapi Bootstrap. Tambahkan di bawah tabel atau di halaman terpisah.

```
/* ===== KODE / OUTPUT LANGKAH 4 ===== */
<div class="container mt-5">
  <h2>Form Input Produksi Manual</h2>
  <form>
    <div class="mb-3">
      <label for="kodeMesin" class="form-label">Kode Mesin</label>
      <input type="text" class="form-control" id="kodeMesin"
placeholder="Contoh: CNC-01">

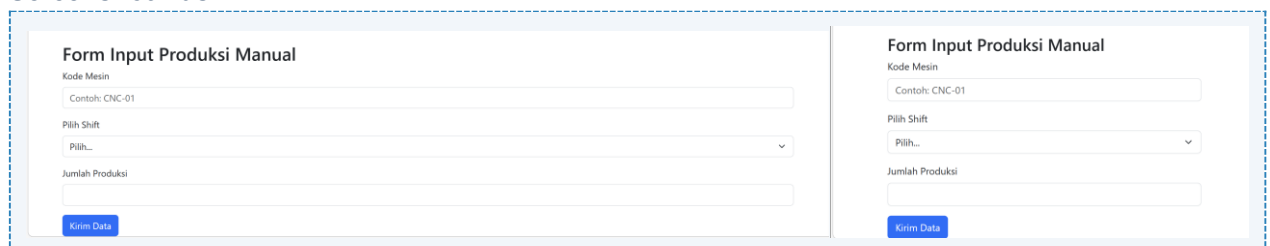
```

```

</div>
<div class="mb-3">
  <label for="shiftSelect" class="form-label">Pilih
Shift</label>
  <select class="form-select" id="shiftSelect">
    <option selected>Pilih...</option>
    <option value="1">Shift 1 (Pagi)</option>
    <option value="2">Shift 2 (Siang)</option>
    <option value="3">Shift 3 (Malam)</option>
  </select>
</div>
<div class="mb-3">
  <label for="jumlahProduksi" class="form-label">Jumlah
Produksi</label>
  <input type="number" class="form-control"
id="jumlahProduksi">
</div>
  <button type="submit" class="btn btn-primary">Kirim Data</button>
</form>
</div>

```

Screenshot Hasil:



Gambar E.4 – Hasil Styling Form dengan Bootstrap Classes

E.5 Langkah 5 – Hasil seluruh langkah

Hasil dari seluruh langkah langkah.

```

/* ===== KODE / OUTPUT LANGKAH 5 ===== */
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Sistem Monitoring Produksi</title>
  <!-- Bootstrap CSS CDN -->
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
  <!-- Custom CSS kita tetap di bawah agar bisa menimpa style bootstrap jika
perlu -->
  <link rel="stylesheet" href="praktikum_style.css">
</head>

```

```

<body>
  <!-- Konten akan kita masukkan di sini -->
  <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <div class="container-fluid">
      <a class="navbar-brand" href="">TekInd Systems</a>
      <button class="navbar-toggler" type="button" data-bs-toggle="collapse"
data-bs-target="#navbarNav">
        <span class="navbar-toggler-icon"></span>
      </button>
      <div class="collapse navbar-collapse" id="navbarNav">
        <ul class="navbar-nav">
          <li class="nav-item">
            <a class="nav-link active" href="">Dashboard</a>
          </li>
          <li class="nav-item">
            <a class="nav-link" href="">Input Produksi</a>
          </li>
          <li class="nav-item">
            <a class="nav-link" href="">Inventori</a>
          </li>
        </ul>
      </div>
    </div>
  </nav>
  <div class="container mt-4">
    <!-- Row 1: KPI Cards (Menggunakan Grid System) -->
    <div class="row mb-4">
      <!-- Kolom lebar 4 (dari 12) -->
      <div class="col-md-4">
        <div class="card text-white bg-primary mb-3">
          <div class="card-header">Total Produksi</div>
          <div class="card-body">
            <h5 class="card-title">1,250 Unit</h5>
            <p class="card-text">Target Harian: 1,200 Unit</p>
          </div>
        </div>
      </div>
      <!-- Kolom lebar 4 -->
      <div class="col-md-4">
        <div class="card text-white bg-success mb-3">
          <div class="card-header">Efisiensi Mesin</div>
          <div class="card-body">
            <h5 class="card-title">92%</h5>
            <p class="card-text">Status: Optimal</p>
          </div>
        </div>
      </div>
    </div>
    <!-- Kolom lebar 4 -->

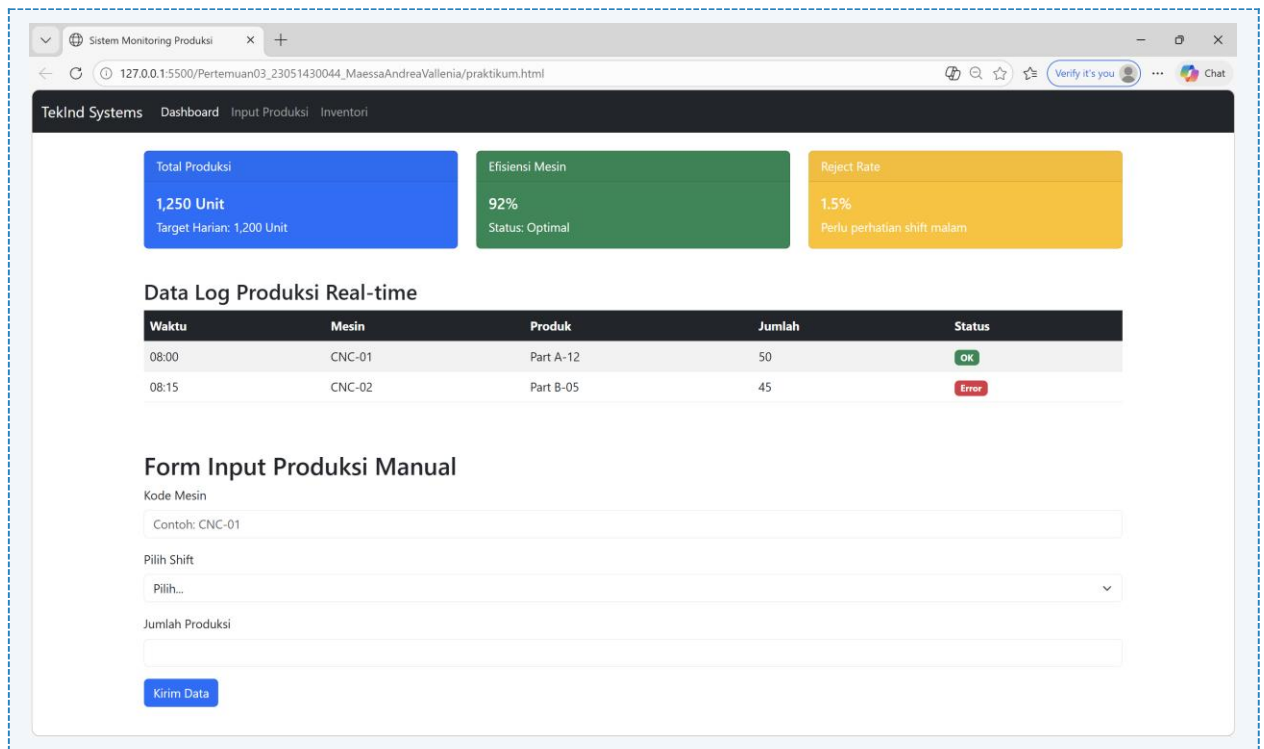
```

```

<div class="col-md-4">
  <div class="card text-white bg-warning mb-3">
    <div class="card-header">Reject Rate</div>
    <div class="card-body">
      <h5 class="card-title">1.5%</h5>
      <p class="card-text">Perlu perhatian shift malam</p>
    </div>
  </div>
</div>
<!-- Row 2: Tabel Data -->
<div class="row">
  <div class="col-12">
    <h3>Data Log Produksi Real-time</h3>
    <table class="table table-striped table-hover">
      <thead class="table-dark">
        <tr>
          <th>Waktu</th>
          <th>Mesin</th>
          <th>Produk</th>
          <th>Jumlah</th>
          <th>Status</th>
        </tr>
      </thead>
      <tbody>
        <tr>
          <td>08:00</td>
          <td>CNC-01</td>
          <td>Part A-12</td>
          <td>50</td>
          <td><span class="badge bg-success">OK</span></td>
        </tr>
        <tr>
          <td>08:15</td>
          <td>CNC-02</td>
          <td>Part B-05</td>
          <td>45</td>
          <td><span class="badge bg-danger">Error</span></td>
        </tr>
      </tbody>
    </table>
  </div>
</div>
<div class="container mt-5">
  <h2>Form Input Produksi Manual</h2>
  <form>
    <div class="mb-3">
      <label for="kodeMesin" class="form-label">Kode Mesin</label>

```

```
        <input type="text" class="form-control" id="kodeMesin"
placeholder="Contoh: CNC-01">
    </div>
    <div class="mb-3">
        <label for="shiftSelect" class="form-label">Pilih Shift</label>
        <select class="form-select" id="shiftSelect">
            <option selected>Pilih...</option>
            <option value="1">Shift 1 (Pagi)</option>
            <option value="2">Shift 2 (Siang)</option>
            <option value="3">Shift 3 (Malam)</option>
        </select>
    </div>
    <div class="mb-3">
        <label for="jumlahProduksi" class="form-label">Jumlah
Produksi</label>
        <input type="number" class="form-control" id="jumlahProduksi">
    </div>
    <button type="submit" class="btn btn-primary">Kirim Data</button>
</form>
</div>
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js">
</script>
</body>
</html>
```

Screenshot Hasil:

Gambar E.5 – Hasil seluruh langkah

F. LATIHAN DAN TUGAS MANDIRI

Jawab dan dokumentasikan seluruh latihan/tugas yang ada di bagian "Latihan/Tugas Mandiri" pada modul Anda.

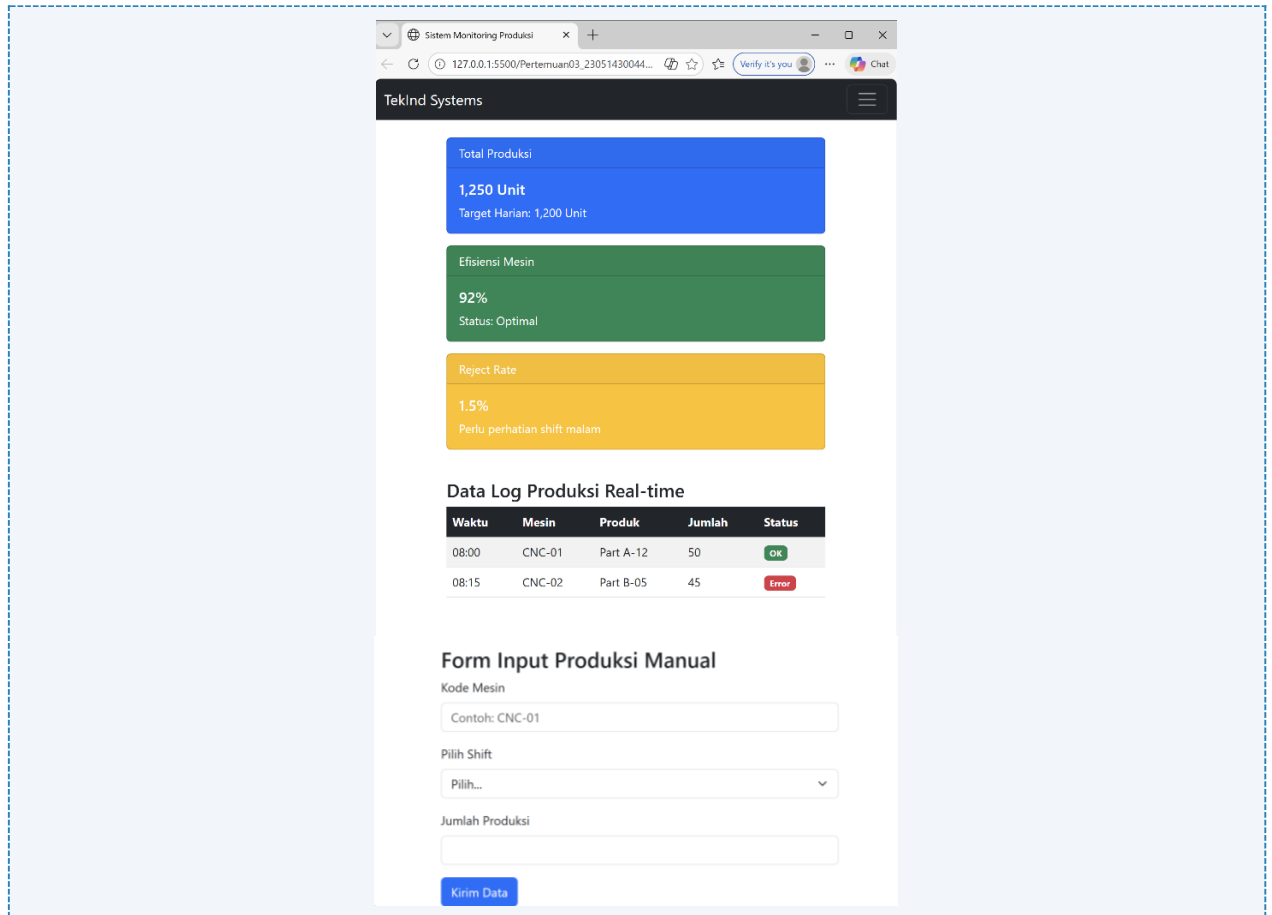
F.1 Latihan 1 – Grid Responsif

Pertanyaan / Instruksi Latihan:

Coba ubah ukuran layar browser Anda. Perhatikan bagaimana kartu KPI yang tadinya sejajar (3 kolom) akan menumpuk ke bawah (1 kolom) di layar HP. Ini adalah kekuatan Grid System (`col-md-4` artinya 4 kolom di layar medium ke atas, otomatis 12 kolom/1 baris di layar kecil).

Jawaban / Kode Solusi:

```
<!-- Row 1: KPI Cards (Gunakan Grid System) -->
<div class="row mb-4">
  <!-- Kolom lebar 4 (dari 12) -->
  <div class="col-md-4">
    <div class="card text-white bg-primary mb-3">
      <div class="card-header">Total Produksi</div>
      <div class="card-body">
        <h5 class="card-title">1,250 Unit</h5>
        <p class="card-text">Target Harian: 1,200 Unit</p>
      </div>
    </div>
  </div>
</div>
```

Gambar F.1 – Hasil Grid Responsif

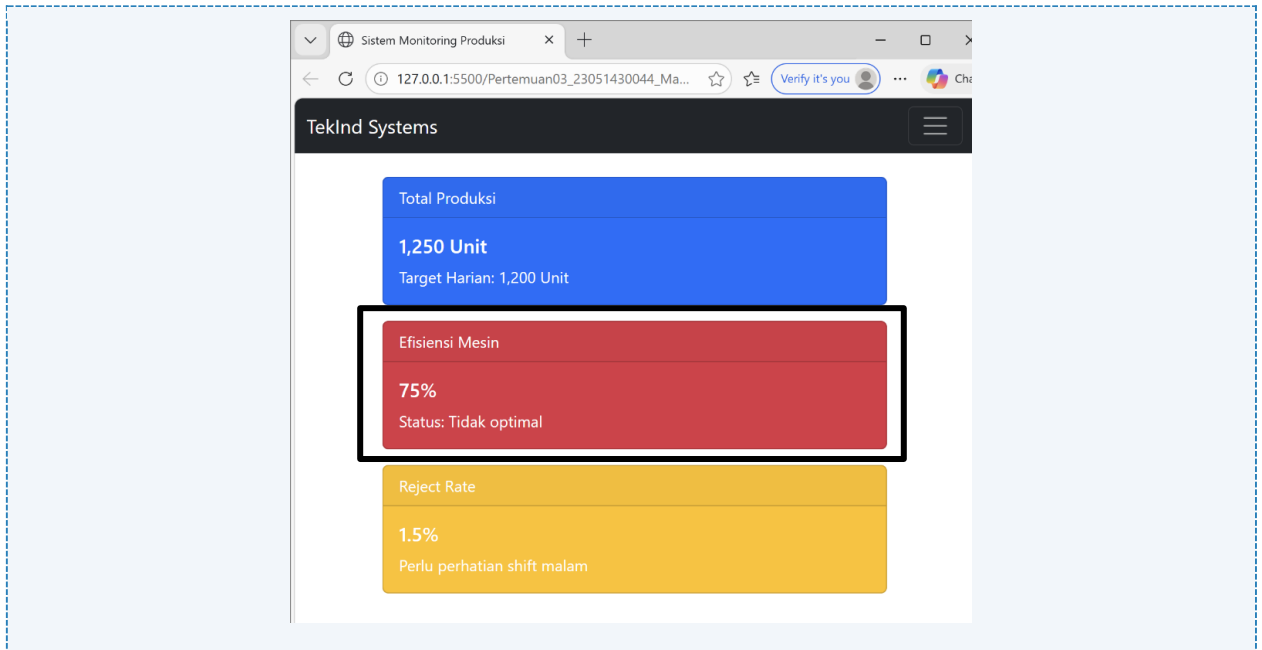
F.2 Latihan 2 – Modifikasi Card

Pertanyaan / Instruksi Latihan:

Ubah kartu "Efisiensi Mesin" menjadi warna merah (`bg-danger`) jika angkanya diubah menjadi di bawah 80%.

Jawaban / Kode Solusi:

```
<!-- Mengubah dari bg-success menjadi ('bg-danger') untuk status efisiensi mesin dibawah 80% -->
<div class="card text-white bg-danger mb-3">
  <div class="card-header">Efisiensi Mesin</div>
  <div class="card-body">
    <h5 class="card-title">75%</h5>
    <p class="card-text">Status: Tidak optimal</p>
  </div>
</div>
```



Gambar F.2 – Hasil Modifikasi Card

F.3 Tugas Proyek Mini – Dashboard Layout

Deskripsi proyek mini:

Buatlah struktur dashboard baru bernama `dashboard\_inventori.html`.

- Gunakan Bootstrap Grid.
- Bagian atas: Judul "Gudang Bahan Baku".
- Bagian kiri (4 kolom): Form Stok Masuk (Input nama barang, qty, supplier).
- Bagian kanan (8 kolom): Tabel Stok Saat Ini (Gunakan tabel Bootstrap).

Penjelasan pendekatan/solusi yang digunakan:

- Proyek mini dashboard dimulai dengan memahami kebutuhan tampilan. Dashboard ini dirancang untuk menampilkan Form Stok Masuk di sisi kiri dan Tabel Stok Saat Ini di sisi kanan, dengan judul utama di bagian atas. Penulis menentukan jumlah kolom dan baris menggunakan sistem grid Bootstrap agar layout rapi, proporsional, dan mudah dibaca oleh pengguna.
- File baru bernama dashboard_inventori.html dibuat menggunakan Visual Studio Code. Struktur dasar HTML disiapkan, meliputi tag `<html>`, `<head>`, dan `<body>`. Di bagian `<head>`, ditambahkan meta tag `charset="UTF-8"` dan `viewport` untuk tampilan responsif. Bootstrap CSS dimasukkan melalui CDN agar sistem grid dan komponen siap pakai bisa digunakan, dan `<title>` ditambahkan untuk memberi nama halaman.
- Pada `<body>`, dibuat judul utama menggunakan `<h1>` dan subjudul seperti identitas mahasiswa menggunakan `<h6>`. Kelas Bootstrap `text-center` digunakan agar judul tampil di tengah halaman, memberikan kesan profesional dan rapi.
- Halaman dibagi menjadi dua kolom utama menggunakan `container` dan `row` Bootstrap:
 - Kolom kiri (4 kolom) untuk Form Stok Masuk
 - Kolom kanan (8 kolom) untuk Tabel Stok Saat Ini
 Pemisahan ini memastikan layout tetap proporsional dan responsif.
- Form dibuat di kolom kiri dengan `<form>` dan elemen input:
 - Input teks untuk Nama Barang
 - Input number untuk Qty

c. Select dropdown untuk Supplier

Tombol Simpan/Kirim ditambahkan dengan kelas btn btn-success w-100 agar terlihat jelas dan memenuhi lebar form. Label dan placeholder setiap input ditambahkan agar mudah dipahami pengguna.

6. Di kolom kanan, tabel dibuat menggunakan <table> dengan kelas table table-striped table-hover. Kolom tabel meliputi No, Nama Barang, Supplier, Stock Awal, Stock Masuk, Stock Keluar, Stock Akhir, dan Status. Status stok diberi badge Bootstrap (bg-success, bg-warning, bg-danger) untuk memudahkan visualisasi kondisi stok.
7. Layout diperiksa di berbagai ukuran layar untuk memastikan form dan tabel tidak tumpang tindih, kolom tetap proporsional, dan elemen mudah dibaca. Kelas Bootstrap tambahan seperti mt-3, mb-4, dan text-center digunakan untuk menambah spasi dan keseimbangan visual.
8. Jika diperlukan, JavaScript sederhana bisa ditambahkan untuk validasi form atau perhitungan stok. Namun, fokus proyek ini adalah layout dan struktur visual, sehingga fungsionalitas dasar sudah cukup.
9. File HTML dijalankan di browser melalui Live Server. Form diuji agar input dapat diketik, tombol berfungsi, dan tabel menampilkan data serta badge status dengan benar. Screenshot dashboard disiapkan sebagai bukti pengerjaan.

Jawaban / Kode Solusi:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Gudang bahan baku</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css
" rel="stylesheet">
</head>

<body>
  <nav class="navbar navbar-expand-lg" style="background-color: #b1d6c6;"
data-bs-theme="light">
    <div class="container">
      <h1 class="mx-auto mb-0 fw-bold text-center">Gudang Bahan
Baku</h1>
    </div>
  </nav>
  <nav class="navbar navbar-expand-lg" style="background-color: #b1d6c6;"
data-bs-theme="light">
    <div class="container">
      <h6 class="mx-auto mb-0 fw-semibold text-center"> Maessa Andrea
Vallenia_23051430044</h6>
    </div>
  </nav>
  <div class="container mt-3">
```

```

<div class="row">
  <!-- KIRI: FORM (4 KOLOM) -->
  <div class="col-md-4 mb-4">
    <h2 class="text-center">Form Stock Masuk</h2>
    <form>
      <div class="mb-3">
        <label class="form-label">Nama Barang</label>
        <input type="text" class="form-control"
placeholder="Contoh: Tepung Terigu">
      </div>
      <div class="mb-3">
        <label class="form-label">Qty</label>
        <input type="number" class="form-control"
placeholder="Jumlah masuk">
      </div>
      <div class="mb-3">
        <label class="form-label">Supplier</label>
        <select class="form-select">
          <option selected>Pilih supplier</option>
          <option>PT Indofood</option>
          <option>PT Bogasari</option>
          <option>PT Wilmar</option>
        </select>
      </div>
      <button class="btn btn-success w-100">Simpan</button>
    </form>
  </div>
  <!-- KANAN: TABEL (8 KOLOM) -->
  <div class="col-md-8">
    <h2 class="text-center">Tabel Stock Saat Ini</h2>
    <table class="table table-striped table-hover">
      <thead class="table-success text-center">
        <tr>
          <th scope="col">No</th>
          <th scope="col">Nama Barang</th>
          <th scope="col">Supplier</th>
          <th scope="col">Stock Awal</th>
          <th scope="col">Stock Masuk</th>
          <th scope="col">Stock Keluar</th>
          <th scope="col">Stock Akhir</th>
          <th scope="col">Status</th>
        </tr>
      </thead>
      <tbody>
        <tr>
          <th scope="row" class="align-middle text-
center">1</th>
          <td class="align-middle text-center">Tepung
Terigu Fortune</td>

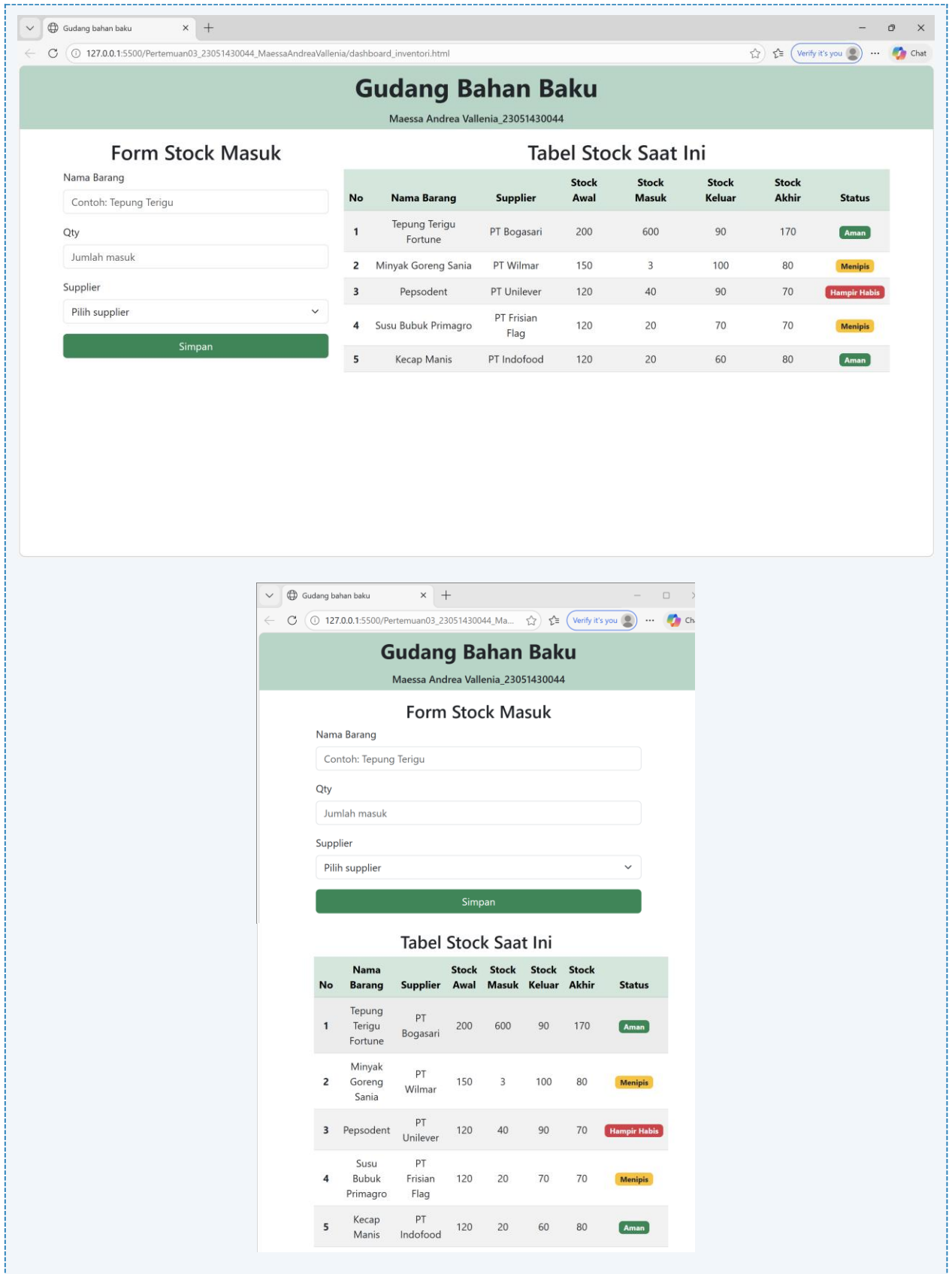
```

```
  |
```

```

                <td class="align-middle text-center"><span
class="badge bg-warning text-dark">Menipis</span></td>
            </tr>
            <tr>
                <th scope="row" class="align-middle text-
center">5</th>
                <td class="align-middle text-center">Kecap
Manis</td>
                <td class="align-middle text-center">PT
Indofood</td>
                <td class="align-middle text-center">120</td>
                <td class="align-middle text-center">20</td>
                <td class="align-middle text-center">60</td>
                <td class="align-middle text-center">80</td>
                <td class="align-middle text-center"><span
class="badge bg-success">Aman</span></td>
            </tr>
        </tbody>
    </table>
</div>
</div>
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.mi
n.js">
    </script>
</body>

</html>
```



Gambar F.3 – Hasil Dashboard Layout

G. PEMBAHASAN

Praktikum dimulai dengan penyiapan lingkungan pengembangan, yaitu laptop/PC, Visual Studio Code, dan browser Google Chrome, dilengkapi ekstensi Live Server untuk menampilkan halaman HTML secara real-time serta CodeSnap untuk dokumentasi kode. Mahasiswa menulis file HTML dari awal dengan deklarasi `<!DOCTYPE html>` dan struktur dasar `<html>`, `<head>`, dan `<body>`, memastikan karakter UTF-8 dan judul halaman tersetting dengan benar. Pada bagian header halaman, penggunaan heading `<h1>` dan `<h6>`, serta elemen `<hr>` sebagai pemisah visual, terbukti mempermudah identifikasi judul, deskripsi formulir, dan identitas penyusun. Selanjutnya, pembuatan form laporan inspeksi kualitas dilakukan dengan input teks, number, radio button, dan file, masing-masing dilengkapi label dan placeholder yang jelas, sehingga simulasi pengisian data dapat dilakukan tanpa error. Penerapan atribut seperti `min` pada input number dan nama sama pada radio button menegaskan validasi dasar input sesuai standar HTML.

Pada proyek mini dashboard layout, mahasiswa menerapkan sistem grid Bootstrap untuk membagi halaman menjadi dua kolom utama: form Stok Masuk di kiri (4 kolom) dan Tabel Stok Saat Ini di kanan (8 kolom). Setiap elemen form seperti input teks, number, dan dropdown telah diberi label yang jelas, sedangkan tombol submit menggunakan kelas `btn btn-success w-100` agar visual menonjol dan responsif. Tabel stok menggunakan kelas `table table-striped table-hover` dengan kolom No, Nama Barang, Supplier, Stock Awal, Stock Masuk, Stock Keluar, Stock Akhir, dan Status, termasuk badge dengan warna berbeda (`bg-success`, `bg-warning`, `bg-danger`) untuk menandai kondisi stok. Layout diuji pada berbagai ukuran layar dan terbukti responsif; tidak ada tumpang tindih antara form dan tabel, serta spasi antar elemen konsisten berkat kelas utilitas seperti `mt-3` dan `mb-4`.

Praktikum juga menekankan penggunaan GitHub dan internet secara praktis. Internet digunakan untuk mengakses CDN Bootstrap, mencari referensi teknis, dan menampilkan hasil praktikum secara real-time, sehingga mahasiswa belajar integrasi library eksternal. GitHub berperan sebagai version control dan media backup proyek, memungkinkan tracking perubahan kode dari tahap awal hingga finalisasi dashboard. Dengan kombinasi HTML, CSS, Bootstrap, dan praktik GitHub, mahasiswa tidak hanya menguasai penulisan kode dasar, tetapi juga memahami konsep layout responsif, validasi input, dan manajemen proyek berbasis web, sehingga pengalaman praktikum menjadi lebih komprehensif dan sesuai dengan praktik pengembangan web modern.

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum menunjukkan bahwa struktur file HTML, penggunaan tag, dan atribut sesuai dengan teori HTML dasar yang dipelajari. Misalnya, deklarasi `<!DOCTYPE html>` dan struktur `<html>`, `<head>`, `<body>` telah diterapkan dengan benar, sehingga browser dapat mengenali dokumen sebagai halaman HTML standar. Penggunaan heading `<h1>` dan `<h6>` di bagian header sesuai konsep hierarki heading yang menjelaskan informasi dari yang paling penting hingga detail tambahan. Selain itu, penempatan form di dalam tag `<form>` dengan input teks, number, radio button, dan file juga konsisten dengan teori form HTML, termasuk penggunaan label dan placeholder untuk memberikan petunjuk kepada pengguna, serta validasi sederhana seperti atribut `min` pada input number dan penggunaan nama yang sama pada radio button agar hanya satu pilihan yang bisa dipilih.

Dalam proyek mini dashboard layout, penerapan Bootstrap Grid terbukti sesuai teori CSS dan framework front-end. Kolom kiri form menggunakan `col-md-4` dan kolom kanan tabel menggunakan `col-md-8`, sesuai prinsip grid 12 kolom Bootstrap. Layout terbagi proporsional dan responsif pada berbagai ukuran layar, konsisten dengan konsep teori responsivitas. Tabel stok menggunakan kelas `table table-striped table-hover`, dan penggunaan badge (`bg-success`, `bg-warning`, `bg-danger`) untuk status stok sesuai teori visualisasi data agar informasi dapat langsung dikenali. Elemen form dan tabel juga sejajar dan memiliki spasi yang konsisten, menunjukkan penerapan teori CSS spacing dan alignment dengan benar.

Secara keseluruhan, hasil output praktikum sesuai dengan konsep teori yang dipelajari. Form dapat diisi tanpa error, tabel menampilkan data dengan format yang rapi dan interaktif, serta layout dashboard responsif di berbagai ukuran layar. Integrasi dengan ekstensi Live Server juga memungkinkan perubahan kode langsung terlihat di browser, sesuai teori pengembangan web modern yang menekankan feedback real-time. Dengan demikian, hasil praktikum membuktikan bahwa pemahaman terhadap teori HTML, CSS, Bootstrap, dan prinsip responsivitas berhasil diterapkan secara tepat dalam proyek mini dashboard layout.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Layout halaman tidak responsif saat ukuran layar diperkecil.	Memeriksa penggunaan sistem grid dan memastikan pemakaian kelas <code>col-md-</code> atau <code>col-lg-</code> agar layout tetap proporsional pada berbagai ukuran layar.

G.3 Kaitan dengan Penerapan di Industri

Praktikum CSS Grid, Flexbox, dan Bootstrap sangat relevan dengan industri modern yang membutuhkan dashboard dan sistem monitoring produksi berbasis web. Di dunia industri, informasi terkait produksi, stok bahan baku, efisiensi mesin, dan kualitas produk harus dapat diakses secara real-time. Dengan pemahaman layout responsif menggunakan CSS Grid dan Flexbox, engineer industri dapat merancang antarmuka web yang menampilkan data tersebut secara terstruktur, mudah dibaca, dan dapat diakses di berbagai perangkat, mulai dari komputer hingga tablet atau smartphone.

Selain itu, penggunaan Bootstrap mempermudah pembuatan prototipe dashboard interaktif yang cepat dan konsisten. Di industri manufaktur, dashboard ini bisa menampilkan indikator kinerja utama (KPI) seperti total produksi, efisiensi mesin, dan reject rate secara visual menggunakan kartu (card), tabel, dan badge status. Dengan layout responsif, staf produksi atau manajer gudang dapat memantau kondisi operasional dan stok bahan baku kapan saja tanpa khawatir tampilan halaman menjadi berantakan saat diakses dari perangkat berbeda.

Lebih jauh, keterampilan ini mendukung digitalisasi proses industri dan perencanaan strategis. Misalnya, mockup layout pabrik, jalur produksi, dan workstation dapat divisualisasikan menggunakan konsep Grid untuk analisis aliran material, sementara Flexbox memungkinkan elemen-elemen penting seperti grafik atau notifikasi ditempatkan secara fleksibel. Dengan dashboard berbasis web yang responsif dan mudah diubah, pengambilan keputusan menjadi lebih cepat, pengelolaan persediaan lebih efisien, dan koordinasi antar departemen lebih efektif, sehingga meningkatkan produktivitas dan mengurangi risiko kesalahan operasional di industri.

H. KESIMPULAN

Tuliskan 3–5 poin kesimpulan yang merangkum hal-hal yang dipelajari dan dicapai pada praktikum modul ini. Kesimpulan harus spesifik, bukan bersifat umum.

1	Penerapan CSS Grid dan Flexbox memungkinkan pembuatan layout halaman web yang terstruktur, rapi, dan responsif, baik untuk dashboard maupun form input data produksi.
2	Bootstrap 5 mempermudah pengembangan UI dengan menyediakan sistem grid siap pakai, komponen visual (card, badge, tabel, tombol), dan utilitas responsif, sehingga proses pembuatan halaman web menjadi lebih cepat.
3	Dashboard yang dibuat terbukti responsif, dimana elemen seperti kartu KPI, tabel stok, dan form input tetap proporsional dan tidak tumpang tindih pada berbagai ukuran layar, dari desktop hingga perangkat mobile.
4	Integrasi dengan Live Server dan GitHub memudahkan dokumentasi, testing, dan version control proyek, sehingga setiap perubahan kode dapat dipantau dan disimpan secara real-time.
5	Praktikum ini relevan dengan industri, karena keterampilan membuat dashboard web responsif dapat diterapkan untuk monitoring produksi, pengelolaan stok, dan visualisasi data operasional di perusahaan manufaktur atau logistik.

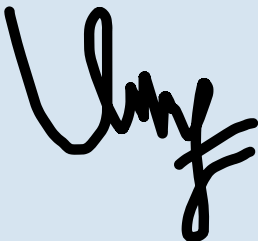
I. DAFTAR PUSTAKA

Tuliskan semua sumber referensi yang digunakan dalam format APA 7th Edition. Minimal 3 referensi yang relevan (termasuk Manual Praktikum dan sumber online resmi seperti MDN, Reactjs.org, dll).

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Rahmat, A. B., Sophan, M. K., & Permana, K. E. (2024). <i>Grid dan Flexbox: Panduan Praktis untuk Layout Web yang Efektif</i> . Thalibul Ilmi Publishing & Education.
[3]	https://cssreference.io/ .
[4]	https://www.hostinger.com/id/tutorial/apa-itu-bootstrap .

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Diperiksa oleh Asisten / Dosen
 _____ Maessa Andrea Vallenia NIM: 23051430044	 _____ Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____

MODUL 4

JAVASCRIPT DASAR (VARIABEL, TIPE DATA, OPERATOR, CONTROL FLOW)

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 4 (Pertemuan 4)
Topik Utama	Logika pemrograman, variabel, tipe data, operator aritmatika/logika, dan percabangan.
Sub-Topik	JavaScript Dasar, Variabel (let dan const), Tipe Data (String, Number, Boolean), Operator Aritmatika dan Logika, Control Flow (if/else, switch)
Durasi Praktikum	340 Menit
Tanggal Praktikum	6 Maret 2026
Nama Mahasiswa	Maessa Andrea Vallenia
NIM	23051430044
Kelas	A

B. TUJUAN PEMBELAJARAN

Tuliskan tujuan pembelajaran yang ingin dicapai pada modul ini, sesuai dengan yang tertera di Manual Praktikum.

1	Memahami peran JavaScript sebagai "otak" dari halaman web.
2	Mampu mendeklarasikan variabel dan memahami aturan penamaannya.
3	Menguasai operator aritmatika untuk perhitungan sederhana.
4	Mampu menggunakan Control Flow (`if/else`, `switch`) untuk pengambilan keputusan otomatis.

C. ALAT DAN BAHAN

Sebutkan seluruh perangkat keras, perangkat lunak, dan sumber referensi yang digunakan selama praktikum.

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	<i>Code Editor</i>

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
2	Google Chrome / Browser Modern	Versi terbaru	<i>Testing & Debugging</i>
3	Live Server	Versi 5.7.9	Menjalankan HTML dan memantau perubahan file
4	JavaScript (Built-in Browser)	ES6 (Default Browser Modern)	Bahasa pemrograman untuk logika dan perhitungan
5	CodeSnap	Versi 1.3.4	Dokumentasi kode program
6	Git dan GitHub	Versi 2.53.0	Version control system (VCS) berbasis Git
7	Koneksi Internet	Stabil	Mengakses Library dan Framework
8	Quokka.js (Opsional)	Versi 1.0.755	Melihat hasil JavaScript secara langsung di VS Code

D. DASAR TEORI

1. JavaScript merupakan bahasa pemrograman yang digunakan untuk membuat halaman web menjadi interaktif dan dinamis. Jika HTML berfungsi sebagai struktur dan CSS sebagai tampilan visual, maka JavaScript berperan sebagai pemroses logika dan pengolah data di dalam sistem. Dengan JavaScript, sebuah halaman web tidak hanya menampilkan informasi, tetapi juga dapat melakukan perhitungan otomatis, memvalidasi input pengguna, serta mengambil keputusan berdasarkan kondisi tertentu. Dalam konteks teknik industri, JavaScript dapat digunakan untuk menghitung total produksi, menentukan status mesin, atau menghitung efisiensi kerja secara otomatis melalui sistem berbasis web.

2. Variabel dalam JavaScript adalah tempat untuk menyimpan data di dalam program. Variabel dapat dianalogikan sebagai wadah yang memiliki nama dan berisi suatu nilai. Dalam JavaScript modern, variabel dibuat menggunakan kata kunci `let` dan `const`. Variabel dengan `let` digunakan untuk nilai yang dapat berubah, sedangkan `const` digunakan untuk nilai yang tetap dan tidak boleh diubah setelah dideklarasikan. Penamaan variabel memiliki aturan tertentu, seperti tidak boleh diawali angka dan disarankan menggunakan format camelCase (contoh: `totalProduksi`). Penggunaan variabel yang tepat sangat penting karena menjadi dasar dalam proses perhitungan dan pengambilan keputusan di dalam program.
3. Tipe Data dalam JavaScript adalah jenis nilai yang disimpan di dalam variabel. JavaScript memiliki beberapa tipe data dasar, yaitu String (teks), Number (angka), dan Boolean (`true` atau `false`). Selain itu terdapat juga tipe data lain seperti Object dan Array untuk penyimpanan data yang lebih kompleks. JavaScript bersifat dynamic typing, artinya programmer tidak perlu mendefinisikan tipe data secara eksplisit saat membuat variabel karena sistem akan mengenalinya secara otomatis. Untuk mengecek tipe data suatu variabel, dapat digunakan perintah `typeof`. Pemahaman tipe data sangat penting agar tidak terjadi kesalahan logika, seperti penggabungan string yang tidak disengaja saat melakukan operasi penjumlahan.
4. Operator dalam JavaScript digunakan untuk melakukan operasi terhadap variabel atau nilai. Operator aritmatika seperti `+`, `-`, `*`, `/`, dan `%` digunakan untuk perhitungan matematis. Operator perbandingan seperti `==`, `===`, `>`, `<`, `>=`, dan `<=` digunakan untuk membandingkan dua nilai dan menghasilkan nilai Boolean. Dalam praktik modern, penggunaan `===` lebih dianjurkan karena membandingkan nilai sekaligus tipe datanya sehingga lebih aman. Selain itu terdapat operator logika seperti `&&` (dan), `||` (atau), dan `!` (tidak) yang digunakan untuk menggabungkan beberapa kondisi dalam satu pernyataan. Operator menjadi dasar dalam proses pengambilan keputusan di dalam program.
5. Control Flow (percabangan) atau alur kontrol adalah mekanisme yang menentukan bagaimana program mengambil keputusan dan menjalankan instruksi tertentu berdasarkan kondisi yang diberikan. Secara default, JavaScript menjalankan kode secara berurutan dari atas ke bawah (alur sekuensial). Namun dalam praktiknya, program sering kali membutuhkan percabangan menggunakan struktur `if`, `if-else`, `else if`, dan `switch`. Struktur ini memungkinkan program memilih tindakan tertentu, misalnya menentukan apakah mesin perlu maintenance atau tidak berdasarkan jam operasionalnya. Dengan memahami control flow, mahasiswa dapat membangun sistem yang mampu merespons berbagai situasi secara otomatis dan lebih efisien.

D.1 Konsep Utama

Konsep utama pada Pertemuan 4 berfokus pada pemahaman dasar JavaScript sebagai bahasa pemrograman yang berfungsi mengatur logika dan pengolahan data dalam halaman web. JavaScript memungkinkan sebuah sistem tidak hanya menampilkan informasi, tetapi juga melakukan perhitungan otomatis dan pengambilan keputusan berdasarkan kondisi

tertentu. Dengan memahami dasar logika pemrograman, mahasiswa dapat mulai membangun sistem yang lebih dinamis dan responsif.

Selain itu, konsep penting lainnya adalah penggunaan variabel dan tipe data. Variabel digunakan sebagai tempat penyimpanan data yang akan diproses dalam program, dengan penggunaan `let` untuk nilai yang dapat berubah dan `const` untuk nilai yang bersifat tetap. Tipe data seperti String, Number, dan Boolean menjadi dasar dalam pengolahan informasi sehingga program dapat membedakan antara teks, angka, dan nilai logika.

Konsep berikutnya adalah penggunaan operator untuk melakukan perhitungan dan evaluasi kondisi. Operator aritmatika digunakan untuk proses matematis, operator perbandingan untuk membandingkan nilai, dan operator logika untuk menggabungkan beberapa kondisi. Semua konsep tersebut kemudian diterapkan dalam control flow seperti `if/else` dan `switch`, yang memungkinkan program mengambil keputusan secara otomatis sesuai dengan situasi yang diberikan. Pemahaman konsep ini menjadi fondasi dalam membangun sistem berbasis web yang mampu melakukan perhitungan dan pengambilan keputusan secara efisien.

D.2 Keterkaitan dengan Teknik Industri

Materi JavaScript dasar pada Pertemuan 4 memiliki keterkaitan yang erat dengan bidang Teknik Industri, khususnya dalam pengolahan data, analisis proses, dan pengambilan keputusan berbasis sistem. Dalam dunia industri, banyak proses kerja yang membutuhkan perhitungan otomatis, seperti perhitungan target produksi, jam kerja, lembur, efisiensi mesin, hingga biaya produksi per unit. Konsep variabel dan tipe data digunakan untuk menyimpan data operasional, sedangkan operator aritmatika dimanfaatkan untuk melakukan perhitungan kuantitatif secara cepat dan akurat.

Selain itu, konsep control flow seperti `if/else` dan `switch` sangat relevan dalam pengambilan keputusan operasional. Contohnya, sistem dapat menentukan apakah mesin perlu maintenance berdasarkan jam operasional, atau apakah produksi sudah memenuhi target harian. Logika ini mencerminkan proses pengambilan keputusan yang biasa dilakukan oleh seorang insinyur industri dalam mengelola sistem produksi.

Pemahaman dasar JavaScript juga mendukung pengembangan sistem informasi industri berbasis web, seperti dashboard monitoring produksi atau sistem perhitungan biaya otomatis. Dengan menguasai konsep logika pemrograman, mahasiswa Teknik Industri dapat merancang solusi digital yang membantu meningkatkan efisiensi, produktivitas, dan akurasi dalam pengelolaan proses bisnis dan manufaktur.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

Dokumentasikan setiap langkah kerja yang Anda lakukan, beserta hasil yang diperoleh (screenshot, kode, atau output). Jawab sesuai urutan langkah di Manual Praktikum.

E.1 Langkah 1 – Hello World & Variabel di Console

Buat file baru `script.js` dan hubungkan ke `index.html` (taruh di atas `</body>`):

`<script src="script.js"></script>`.

```
/* ===== KODE / OUTPUT LANGKAH 1 ===== */
// Komentar single line

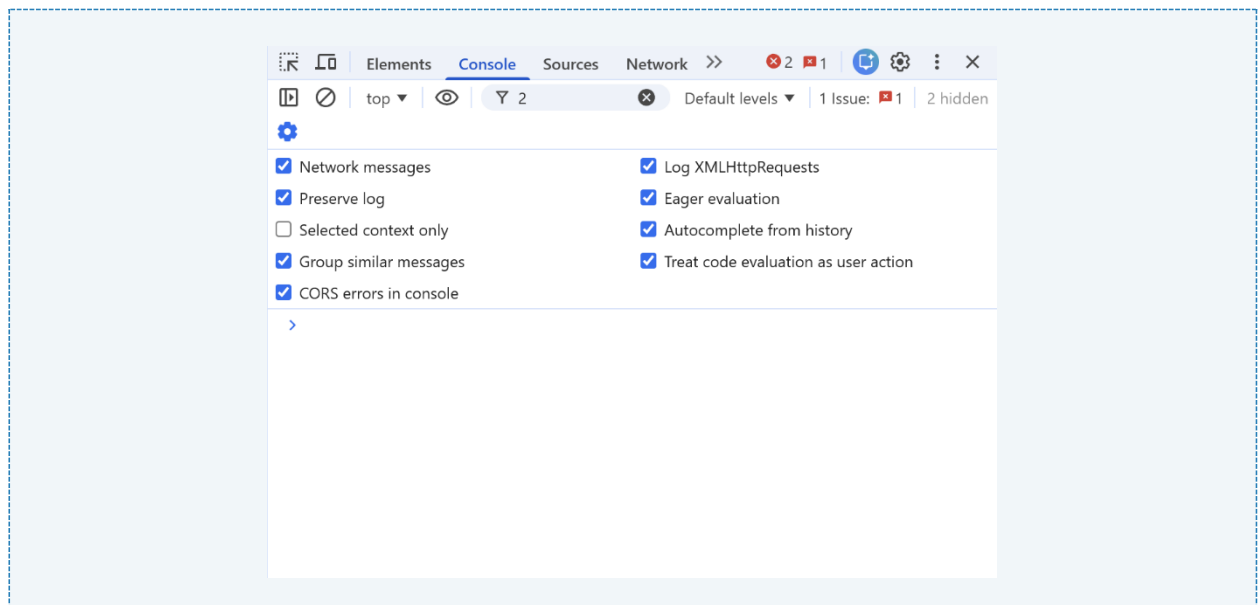
// 1. Variabel & Tipe Data
let namaMesin = "CNC-Mazak-01"; // String
let targetHarian = 500; // Number
// Menampilkan ke console browser (Tekan F12 -> Console)
console.log("Mesin: " + namaMesin);
console.log("Target: " + targetHarian);

// 2. Operator Aritmatika
let produksiPagi = 200;
let produksiSiang = 150;
let totalProduksi = produksiPagi + produksiSiang;

console.log("Total saat ini: " + totalProduksi);

// Hitung sisa kekurangan target
let kekurangan = targetHarian - totalProduksi;
console.log("Kekurangan target: " + kekurangan);
```

Screenshot Hasil:



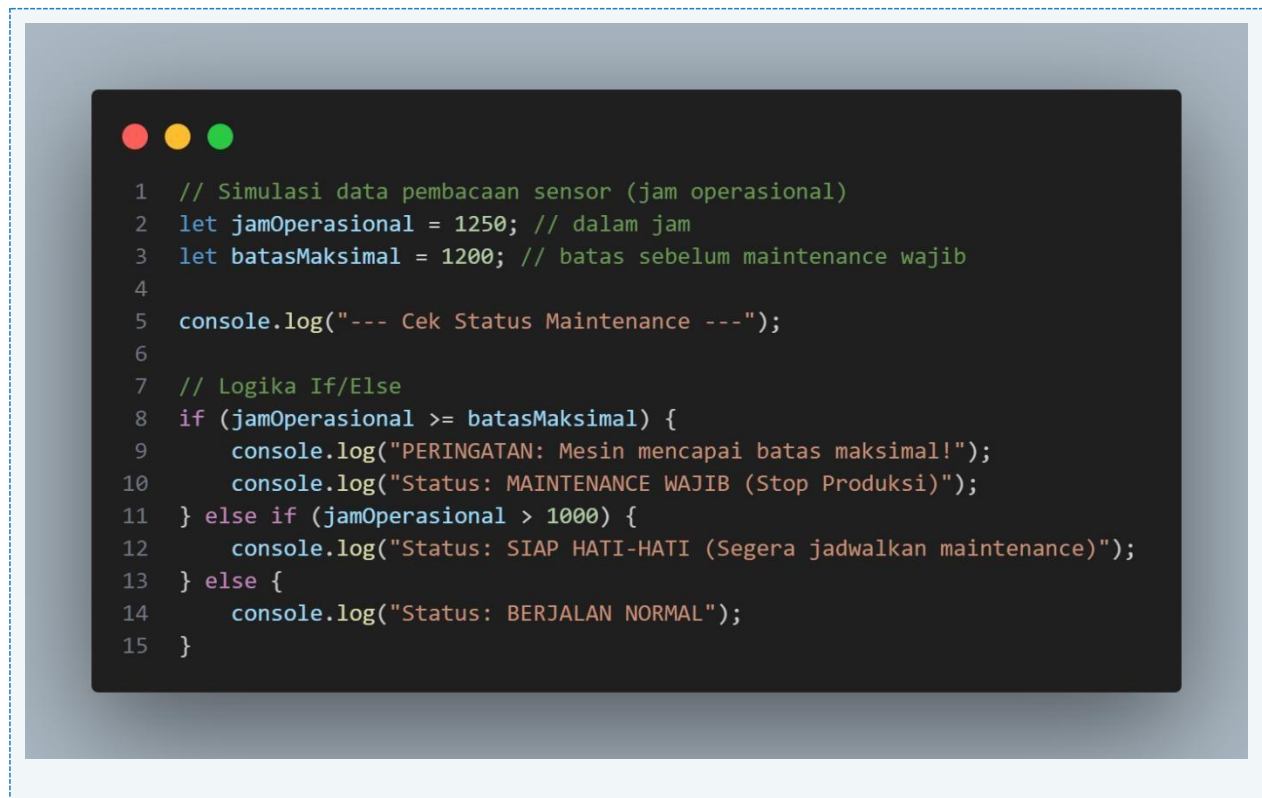
Gambar E.1 – Hasil Coding Hello World & Variabel di Console

E.2 Langkah 2 – Control Flow (Pengambilan Keputusan)

Membuat logika untuk menentukan apakah mesin perlu dimatikan untuk maintenance atau tidak. Menambahkan kode berikut di `script.js`:

```
/* ===== KODE / OUTPUT LANGKAH 2 ===== */  
// Simulasi data pembacaan sensor (jam operasional)  
let jamOperasional = 1250; // dalam jam  
let batasMaksimal = 1200; // batas sebelum maintenance wajib  
  
console.log("--- Cek Status Maintenance ---");  
  
// Logika If/Else  
if (jamOperasional >= batasMaksimal) {  
    console.log("PERINGATAN: Mesin mencapai batas maksimal!");  
    console.log("Status: MAINTENANCE WAJIB (Stop Produksi)");  
} else if (jamOperasional > 1000) {  
    console.log("Status: SIAP HATI-HATI (Segera jadwalkan maintenance)");  
} else {  
    console.log("Status: BERJALAN NORMAL");  
}
```

Screenshot Hasil:



Gambar E.2 – Hasil Coding Control Flow (Pengambilan Keputusan)

E.3 Langkah 3 – Studi Kasus Industri (Perhitungan OEE Sederhana)

Membuat fungsi sederhana untuk menghitung efisiensi berdasarkan Availability.

```
/* ===== KODE / OUTPUT LANGKAH 3 ===== */
// Data Input
let jamKerjaPlanned = 8; // Jam
let jamKerjaAktual = 6.5; // Jam (Ada 1.5 jam breakdown)

// Perhitungan
let availability = (jamKerjaAktual / jamKerjaPlanned) * 100;

// Pembulatan 2 angka di belakang koma
availability = availability.toFixed(2);

console.log("Planned Time: " + jamKerjaPlanned + " Jam");
console.log("Actual Time: " + jamKerjaAktual + " Jam");
console.log("Availability: " + availability + "%");

// Logika Penilaian Kualitas Availability
if (availability >= 90) {
    console.log("Kategori: WORLD CLASS");
} else if (availability >= 80) {
    console.log("Kategori: BAIK (Tetap monitor)");
} else {
    console.log("Kategori: BURUK (Perlu investigasi penyebab breakdown)");
}
```

Screenshot Hasil:



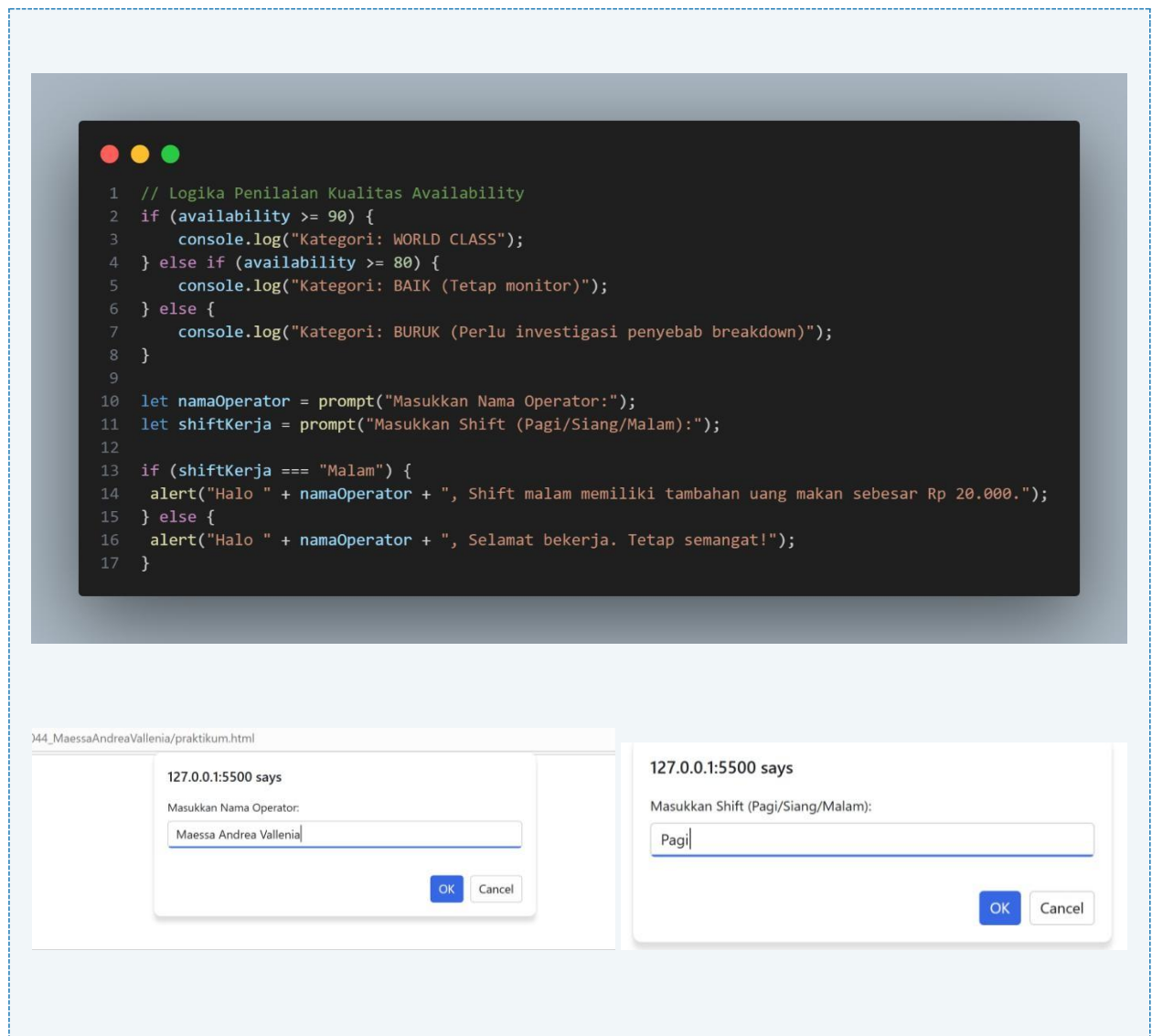
Gambar E.3 – Hasil Coding Studi Kasus Industri (Perhitungan OEE Sederhana)

E.4 Langkah 4 – Interaksi DOM Sederhana (Alert & Prompt)

Mengubah `script.js` agar berinteraksi dengan user via browser popup.

```
/* ===== KODE / OUTPUT LANGKAH 4 ===== */  
let namaOperator = prompt("Masukkan Nama Operator:");  
let shiftKerja = prompt("Masukkan Shift (Pagi/Siang/Malam):");  
  
if (shiftKerja === "Malam") {  
    alert("Halo " + namaOperator + ", Shift malam memiliki tambahan uang makan sebesar Rp 20.000.");  
} else {  
    alert("Halo " + namaOperator + ", Selamat bekerja. Tetap semangat!");  
}
```

Screenshot Hasil:



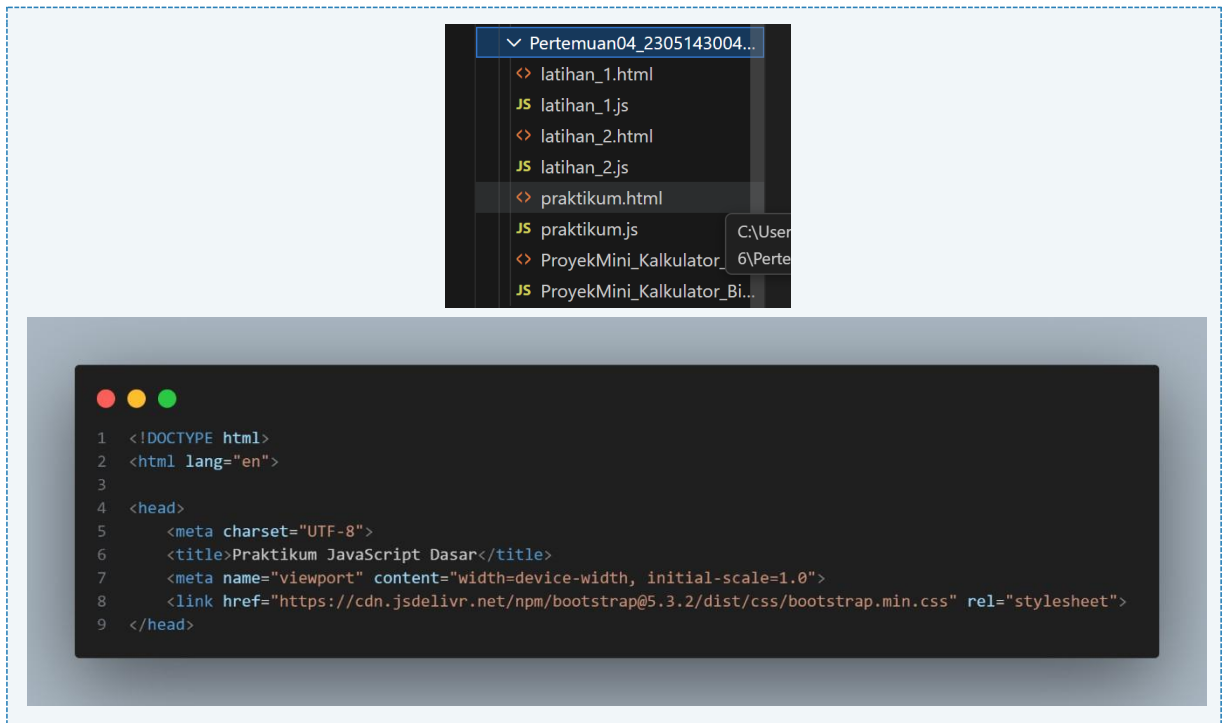
Gambar E.4 – Hasil Coding dan popup Interaksi DOM Sederhana (Alert & Prompt)

E.5 Langkah 5 – Membuat File HTML dan Hubungkan dengan File .JS

1. Buka VS Code.
2. Buat folder baru di desktop bernama PraktikumWebMob_TekInd_2026.
3. Di VS Code, pilih File > Open Folder dan pilih folder tersebut.
4. Buat file baru bernama praktikum.html.
5. Ketik ! lalu tekan Tab (atau Enter). VS Code otomatis membuat kerangka HTML dasar.
6. Tambahkan link Bootstrap di <head> untuk styling

```
/* ===== KODE / OUTPUT LANGKAH 5 ===== */  
<!DOCTYPE html>  
<html lang="en">  
  
<head>  
  <meta charset="UTF-8">  
  <title>Praktikum JavaScript Dasar</title>  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <link  
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"  
rel="stylesheet">  
</head>
```

Screenshot Hasil:



Gambar E.5 – Membuat File HTML

E.6 Langkah 6 – (Step 1) Variabel dan Tipe Data

1. Buat section <section> dengan class container my-5.
2. Tambahkan <div class="card shadow-sm"> lalu <div class="card-body">.
3. Buat heading <h4>: *Step 1 – Variabel dan Tipe Data*.
4. Tulis kode JavaScript di <pre><code> untuk deklarasi variabel.
5. Tambahkan penjelasan singkat tentang tipe data dan fungsi console.log().

```
/* ===== KODE / OUTPUT LANGKAH 6 ===== */
```

```
<!-- STEP 1 -->
```

```
<section class="container my-5">
```

```
  <div class="card shadow-sm">
```

```
    <div class="card-body">
```

```
      <h4>Step 1 – Variabel dan Tipe Data</h4>
```

```
<pre class="bg-light p-3 border rounded"><code>
```

```
let namaMesin = "CNC-Mazak-01"; // String
```

```
let targetHarian = 500; // Number
```

```
let isOperational = true; // Boolean
```

```
console.log("Mesin: " + namaMesin);
```

```
console.log("Target: " + targetHarian);
```

```
</code></pre>
```

```
    <p class="mt-3">
```

Pada tahap ini dilakukan deklarasi variabel menggunakan kata kunci

<code>let</code>. Variabel <code>namaMesin</code> bertipe string karena menyimpan data teks, sedangkan <code>targetHarian</code> bertipe number karena digunakan dalam proses perhitungan

numerik.

Output ditampilkan melalui <code>console.log()</code> untuk melihat

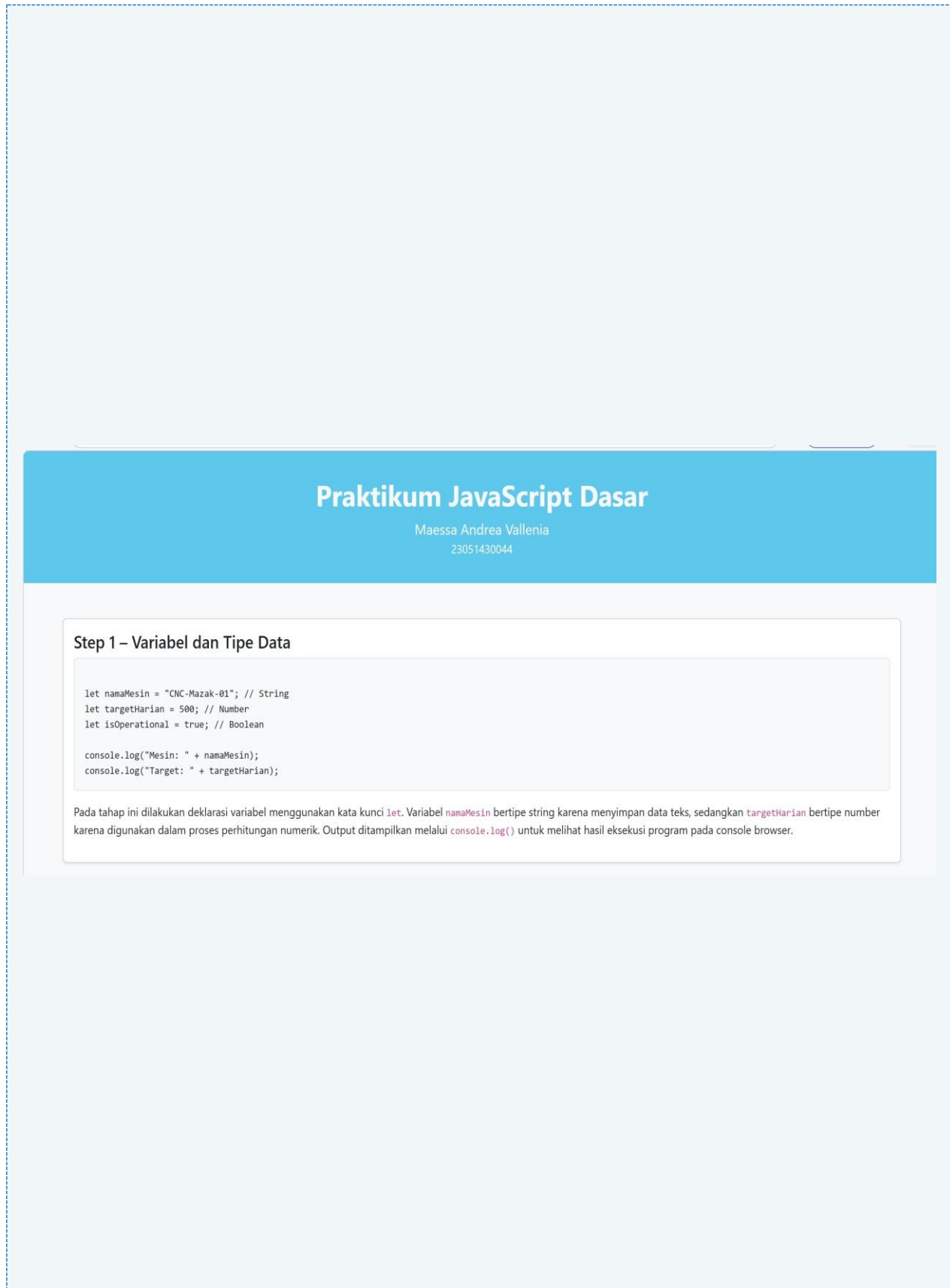
hasil eksekusi program pada console browser.

```
  </p>
```

```
</div>
```

```
</div>
```

```
</section>
```

Screenshot Hasil:

Gambar E.6 – Hasil (Step 1) Variabel dan Tipe Data

E.7 Langkah 7 – (Step 2) Operator Aritmatika

1. Tambahkan section baru untuk Step 2 dengan struktur card yang sama.
2. Tulis kode untuk menghitung total produksi dan kekurangan target.
3. Tambahkan penjelasan penggunaan operator + dan -.

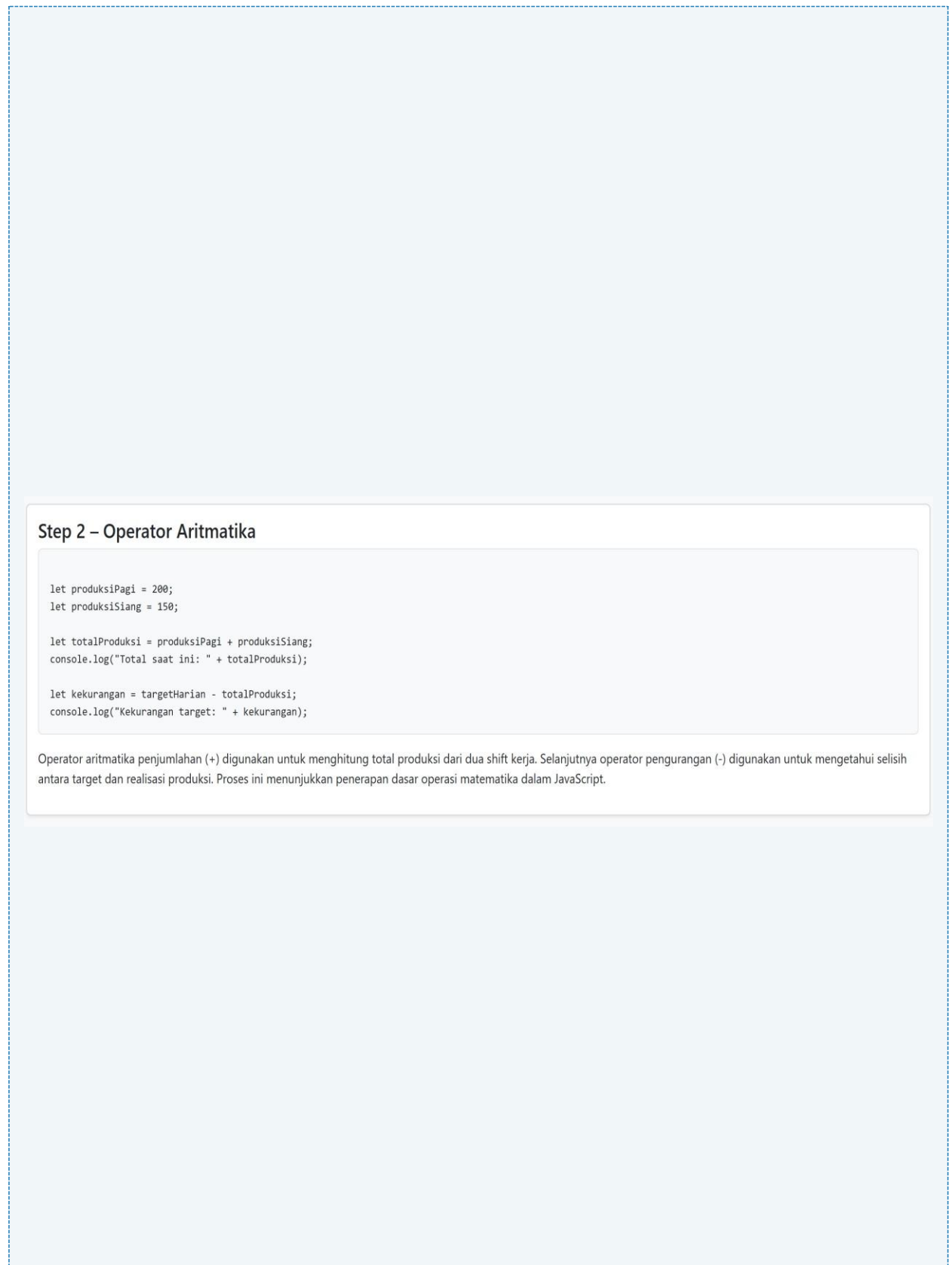
```
/* ===== KODE / OUTPUT LANGKAH 7 ===== */
<!-- STEP 2 -->
<section class="container mb-5">
  <div class="card shadow-sm">
    <div class="card-body">
      <h4>Step 2 – Operator Aritmatika</h4>

      <pre class="bg-light p-3 border rounded"><code>
let produksiPagi = 200;
let produksiSiang = 150;

let totalProduksi = produksiPagi + produksiSiang;
console.log("Total saat ini: " + totalProduksi);

let kekurangan = targetHarian - totalProduksi;
console.log("Kekurangan target: " + kekurangan);
</code></pre>

      <p class="mt-3">
        Operator aritmatika penjumlahan (+) digunakan untuk
menghitung total
        produksi dari dua shift kerja. Selanjutnya operator
pengurangan (-)
        digunakan untuk mengetahui selisih antara target dan
realisasi produksi.
        Proses ini menunjukkan penerapan dasar operasi matematika
dalam JavaScript.
      </p>
    </div>
  </div>
</section>
```


Screenshot Hasil:

Gambar E.7 – Hasil (Step 2) Operator Aritmatika

E.8 Langkah 8 – (Step 3) Percabangan Target Produksi

1. Tambahkan section baru untuk Step 3.
2. Tulis kode JavaScript untuk percabangan if-else menentukan status mesin.
3. Jelaskan logika percabangan dan tujuan mengecek kondisi mesin.

```

/* ===== KODE / OUTPUT LANGKAH 8 ===== */
<!-- STEP 3 -->
<section class="container mb-5">
  <div class="card shadow-sm">
    <div class="card-body">
      <h4>Step 3 – Percabangan Target Produksi</h4>

      <pre class="bg-light p-3 border rounded"><code>
let jamOperasional = 1250; // dalam jam
let batasMaksimal = 1200; // batas sebelum maintenance wajib
console.log("--- Cek Status Maintenance ---");

if (jamOperasional >= batasMaksimal) {
  console.log("PERINGATAN: Mesin mencapai batas maksimal!");
  console.log("Status: MAINTENANCE WAJIB (Stop Produksi)");
} else if (jamOperasional > 1000) {
  console.log("Status: SIAP HATI-HATI (Segera jadwalkan maintenance)");
} else {
  console.log("Status: BERJALAN NORMAL");
}
</code></pre>

      <p class="mt-3">
        Struktur percabangan <code>if-else</code> digunakan untuk
        menentukan
        apakah total produksi telah memenuhi target harian. Jika
        nilai
        <code>totalProduksi</code> lebih besar atau sama dengan
        <code>targetHarian</code>, maka sistem menampilkan pesan
        bahwa target tercapai.
      </p>
    </div>
  </div>
</section>

```

Screenshot Hasil:**Step 3 – Percabangan Target Produksi**

```
let jamOperasional = 1250; // dalam jam
let batasMaksimal = 1200; // batas sebelum maintenance wajib
console.log("--- Cek Status Maintenance ---");

if (jamOperasional >= batasMaksimal) {
  console.log("PERINGATAN: Mesin mencapai batas maksimal!");
  console.log("Status: MAINTENANCE WAJIB (Stop Produksi)");
} else if (jamOperasional > 1000) {
  console.log("Status: SIAP HATI-HATI (Segera jadwalkan maintenance)");
} else {
  console.log("Status: BERJALAN NORMAL");
}
```

Struktur percabangan **if-else** digunakan untuk menentukan apakah total produksi telah memenuhi target harian. Jika nilai **totalProduksi** lebih besar atau sama dengan **targetHarian**, maka sistem menampilkan pesan bahwa target tercapai.

Gambar E.8 – Hasil (Step 3) Percabangan Target Produksi

E.9 Langkah 9 – (Step 4) Perhitungan Availability

1. Tambahkan section Step 4.
2. Tulis kode perhitungan availability.
3. Jelaskan konsep availability, pembulatan angka, dan kategori performa.

```
/* ===== KODE / OUTPUT LANGKAH 9 ===== */
<!-- STEP 4 -->
<section class="container mb-5">
  <div class="card shadow-sm">
    <div class="card-body">
      <h4>Step 4 – Perhitungan Availability</h4>

      <pre class="bg-light p-3 border rounded"><code>
let jamKerjaPlanned = 8; // Jam
let jamKerjaAktual = 6.5; // Jam (Ada 1.5 jam breakdown)

let availability = (jamKerjaAktual / jamKerjaPlanned) * 100;
availability = availability.toFixed(2);

console.log("Planned Time: " + jamKerjaPlanned + " Jam");
console.log("Actual Time: " + jamKerjaAktual + " Jam");
console.log("Availability: " + availability + "%");

if (availability >= 90) {
  console.log("Kategori: WORLD CLASS");
} else if (availability >= 80) {
  console.log("Kategori: BAIK (Tetap monitor)");
} else {
  console.log("Kategori: BURUK (Perlu investigasi penyebab breakdown)");
}
</code></pre>

      <p class="mt-3">
        Availability dihitung berdasarkan rasio waktu kerja aktual
        terhadap waktu kerja rencana yang dikalikan 100%. Fungsi
        <code>toFixed(2)</code> digunakan untuk membatasi dua angka
        di belakang koma.
        Nilai tersebut kemudian dikategorikan menggunakan
        percabangan.
      </p>

      <p>
        Pengelompokan kategori dilakukan dengan struktur <code>if-
        else if</code>
      </p>
    </div>
  </div>
</section>
```

```
        untuk menentukan tingkat kinerja mesin, seperti WORLD CLASS,  
BAIK,  
        atau BURUK berdasarkan standar tertentu.  
    </p>  
</div>  
</div>  
</section>
```

Screenshot Hasil:**Step 4 – Perhitungan Availability**

```
let jamKerjaPlanned = 8; // Jam
let jamKerjaAktual = 6.5; // Jam (Ada 1.5 jam breakdown)

let availability = (jamKerjaAktual / jamKerjaPlanned) * 100;
availability = availability.toFixed(2);

console.log("Planned Time: " + jamKerjaPlanned + " Jam");
console.log("Actual Time: " + jamKerjaAktual + " Jam");
console.log("Availability: " + availability + "%");

if (availability >= 90) {
  console.log("Kategori: WORLD CLASS");
} else if (availability >= 80) {
  console.log("Kategori: BAIK (Tetap monitor)");
} else {
  console.log("Kategori: BURUK (Perlu investigasi penyebab breakdown)");
}
```

Availability dihitung berdasarkan rasio waktu kerja aktual terhadap waktu kerja rencana yang dikalikan 100%. Fungsi `toFixed(2)` digunakan untuk membatasi dua angka di belakang koma. Nilai tersebut kemudian dikategorikan menggunakan percabangan.

Pengelompokan kategori dilakukan dengan struktur `if-else if` untuk menentukan tingkat kinerja mesin, seperti WORLD CLASS, BAIK, atau BURUK berdasarkan standar tertentu.

Gambar E.9 – Hasil (Step 4) Perhitungan Availability

E.10 Langkah 10 – (Step 5) Input Pengguna (Prompt & Alert)

1. Tambahkan section Step 5.
2. Buat interaksi sederhana dengan prompt() dan alert().
3. Jelaskan fungsi prompt menerima input, alert menampilkan output, dan percabangan menentukan kondisi.

```

/* ===== KODE / OUTPUT LANGKAH 10 ===== */
<!-- STEP 5 -->
<section class="container mb-5">
  <div class="card shadow-sm">
    <div class="card-body">
      <h4>Step 5 - Input Pengguna (prompt dan alert)</h4>

      <pre class="bg-light p-3 border rounded"><code>
let namaOperator = prompt("Masukkan Nama Operator:");
let shiftKerja = prompt("Masukkan Shift (Pagi/Siang/Malam:");

if (shiftKerja === "Malam") {
  alert("Halo " + namaOperator + ", Shift malam memiliki tambahan uang makan
sebesar Rp
20.000.");
} else {
  alert("Halo " + namaOperator + ", Selamat bekerja. Tetap semangat!");
}
</code></pre>

      <p class="mt-3">
        Fungsi <code>prompt()</code> digunakan untuk menerima input
dari
        pengguna dan menyimpannya dalam variabel. Nilai yang
dikembalikan
        selalu bertipe string, kecuali jika pengguna menekan Cancel
maka
        nilainya menjadi <code>null</code>.
      </p>

      <p>
        Struktur percabangan digunakan untuk mengevaluasi isi
variabel
        <code>shiftKerja</code>. Jika bernilai "Malam", maka sistem
akan
        memberikan notifikasi tambahan uang makan menggunakan fungsi
<code>alert()</code>.
      </p>

      <p>

```

dan Mekanisme ini menunjukkan interaksi sederhana antara pengguna
sistem berbasis web, di mana input diproses dan menghasilkan
output sesuai kondisi yang telah ditentukan.

```
</p>  
</div>  
</div>  
</section>
```


Screenshot Hasil:**Step 5 – Input Pengguna (prompt dan alert)**

```
let namaOperator = prompt("Masukkan Nama Operator:");
let shiftKerja = prompt("Masukkan Shift (Pagi/Siang/Malam):");

if (shiftKerja === "Malam") {
  alert("Halo " + namaOperator + ", Shift malam memiliki tambahan uang makan sebesar Rp 20.000.");
} else {
  alert("Halo " + namaOperator + ", Selamat bekerja. Tetap semangat!");
}
```

Fungsi `prompt()` digunakan untuk menerima input dari pengguna dan menyimpannya dalam variabel. Nilai yang dikembalikan selalu bertipe string, kecuali jika pengguna menekan Cancel maka nilainya menjadi `null`.

Struktur percabangan digunakan untuk mengevaluasi isi variabel `shiftKerja`. Jika bernilai "Malam", maka sistem akan memberikan notifikasi tambahan uang makan menggunakan fungsi `alert()`.

Mekanisme ini menunjukkan interaksi sederhana antara pengguna dan sistem berbasis web, di mana input diproses dan menghasilkan output sesuai kondisi yang telah ditentukan.

Gambar E.10 – Hasil (Step 5) Input Pengguna (Prompt & Alert)

E.11 Langkah 11 – Menjalankan File

Klik kanan pada file HTML kemudian pilih Open with Live Server sehingga browser otomatis menampilkan halaman praktikum. Selanjutnya tekan F12 → Console untuk memastikan tidak ada error dan melihat hasil console.log() jika digunakan.

```
/* ===== KODE / OUTPUT LANGKAH 5 ===== */
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <title>Praktikum JavaScript Dasar</title>
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>

<body class="bg-light">

  <header class="bg-info text-light d-flex align-items-center justify-
content-center text-center"
style="min-height: 180px;">
    <div>
      <h1 class="fw-bold mb-2">Praktikum JavaScript Dasar</h1>
      <p class="fs-5 mb-0">Maessa Andrea Vallenia</p>
      <p class="mb-0">23051430044</p>
    </div>
  </header>

  <!-- STEP 1 -->
  <section class="container my-5">
    <div class="card shadow-sm">
      <div class="card-body">
        <h4>Step1 – Variabel dan Tipe Data</h4>

        <pre class="bg-light p-3 border rounded"><code>
let namaMesin = "CNC-Mazak-01"; // String
let targetHarian = 500; // Number
let isOperational = true; // Boolean

console.log("Mesin: " + namaMesin);
console.log("Target: " + targetHarian);
</code></pre>

        <p class="mt-3">
```

Pada tahap ini dilakukan deklarasi variabel menggunakan kata kunci `let`. Variabel `namaMesin` bertipe string karena menyimpan data teks, sedangkan `targetHarian` bertipe number karena digunakan dalam proses perhitungan numerik. Output ditampilkan melalui `console.log()` untuk melihat hasil eksekusi program pada console browser.

```

    </div>
  </div>
</section>

<!-- STEP 2 -->
<section class="container mb-5">
  <div class="card shadow-sm">
    <div class="card-body">
      <h4>Step 2 – Operator Aritmatika</h4>

      <pre class="bg-light p-3 border rounded"><code>
let produksiPagi = 200;
let produksiSiang = 150;

let totalProduksi = produksiPagi + produksiSiang;
console.log("Total saat ini: " + totalProduksi);

let kekurangan = targetHarian - totalProduksi;
console.log("Kekurangan target: " + kekurangan);
</code></pre>

```

Operator aritmatika penjumlahan (+) digunakan untuk menghitung total produksi dari dua shift kerja. Selanjutnya operator pengurangan (-) digunakan untuk mengetahui selisih antara target dan realisasi produksi. Proses ini menunjukkan penerapan dasar operasi matematika

dalam JavaScript.

```

    </p>
  </div>
</div>

```

```

<!-- STEP 3 -->
<section class="container mb-5">
  <div class="card shadow-sm">
    <div class="card-body">
      <h4>Step 3 – Percabangan Target Produksi</h4>

      <pre class="bg-light p-3 border rounded"><code>
let jamOperasional = 1250; // dalam jam
let batasMaksimal = 1200; // batas sebelum maintenance wajib
console.log("--- Cek Status Maintenance ---");

if (jamOperasional >= batasMaksimal) {
  console.log("PERINGATAN: Mesin mencapai batas maksimal!");
  console.log("Status: MAINTENANCE WAJIB (Stop Produksi)");
} else if (jamOperasional > 1000) {
  console.log("Status: SIAP HATI-HATI (Segera jadwalkan maintenance)");
} else {
  console.log("Status: BERJALAN NORMAL");
}
</code></pre>

      <p class="mt-3">
        Struktur percabangan <code>if-else</code> digunakan untuk
menentukan
        apakah total produksi telah memenuhi target harian. Jika nilai
        <code>totalProduksi</code> lebih besar atau sama dengan
        <code>targetHarian</code>, maka sistem menampilkan pesan bahwa
target tercapai.
      </p>
    </div>
  </div>
</section>

<!-- STEP 4 -->
<section class="container mb-5">
  <div class="card shadow-sm">
    <div class="card-body">
      <h4>Step 4 – Perhitungan Availability</h4>

      <pre class="bg-light p-3 border rounded"><code>
let jamKerjaPlanned = 8; // Jam
let jamKerjaAktual = 6.5; // Jam (Ada 1.5 jam breakdown)

let availability = (jamKerjaAktual / jamKerjaPlanned) * 100;
availability = availability.toFixed(2);

```

```

console.log("Planned Time: " + jamKerjaPlanned + " Jam");
console.log("Actual Time: " + jamKerjaAktual + " Jam");
console.log("Availability: " + availability + "%");

if (availability >= 90) {
  console.log("Kategori: WORLD CLASS");
} else if (availability >= 80) {
  console.log("Kategori: BAIK (Tetap monitor)");
} else {
  console.log("Kategori: BURUK (Perlu investigasi penyebab breakdown)");
}
</code></pre>

```

```

<p class="mt-3">
  Availability dihitung berdasarkan rasio waktu kerja aktual
terhadap
  waktu kerja rencana yang dikalikan 100%. Fungsi
  <code>toFixed(2)</code> digunakan untuk membatasi dua angka di
belakang koma.
  Nilai tersebut kemudian dikategorikan menggunakan percabangan.
</p>

<p>
  Pengelompokan kategori dilakukan dengan struktur <code>if-else
if</code>
  untuk menentukan tingkat kinerja mesin, seperti WORLD CLASS,
BAIK,
  atau BURUK berdasarkan standar tertentu.

</p>
</div>
</div>
</section>

```

```

<!-- STEP 5 -->
<section class="container mb-5">
  <div class="card shadow-sm">
    <div class="card-body">
      <h4>Step 5 - Input Pengguna (prompt dan alert)</h4>

      <pre class="bg-light p-3 border rounded"><code>
let namaOperator = prompt("Masukkan Nama Operator:");
let shiftKerja = prompt("Masukkan Shift (Pagi/Siang/Malam):");

if (shiftKerja === "Malam") {

```

```
    alert("Halo " + namaOperator + ", Shift malam memiliki tambahan uang makan
    sebesar Rp
    20.000.");

} else {
    alert("Halo " + namaOperator + ", Selamat bekerja. Tetap semangat!");
}
</code></pre>

<p class="mt-3">
    Fungsi <code>prompt()</code> digunakan untuk menerima input
dari
    pengguna dan menyimpannya dalam variabel. Nilai yang
dikembalikan
    selalu bertipe string, kecuali jika pengguna menekan Cancel
maka
    nilainya menjadi <code>null</code>.
</p>

<p>
    Struktur percabangan digunakan untuk mengevaluasi isi variabel
    <code>shiftKerja</code>. Jika bernilai "Malam", maka sistem
akan
    memberikan notifikasi tambahan uang makan menggunakan fungsi
    <code>alert()</code>.
</p>

<p>
    Mekanisme ini menunjukkan interaksi sederhana antara pengguna
dan
    sistem berbasis web, di mana input diproses dan menghasilkan
output
    sesuai kondisi yang telah ditentukan.

</p>
</div>
</div>
</section>

<script src="praktikum.js"></script>

</body>
</html>
```

Screenshot Hasil Web:

Praktikum JavaScript Dasar

Maessa Andrea Vallenia
23051430044

Step 1 – Variabel dan Tipe Data

```
let namaMesin = "CNC-Mazak-01"; // String
let targetHarian = 500; // Number
let isOperasional = true; // Boolean

console.log("Mesin: " + namaMesin);
console.log("Target: " + targetHarian);
```

Pada tahap ini dilakukan deklarasi variabel menggunakan kata kunci `let`. Variabel `namaMesin` bertipe string karena menyimpan data teks, sedangkan `targetHarian` bertipe number karena digunakan dalam proses perhitungan numerik. Output ditampilkan melalui `console.log()` untuk melihat hasil eksekusi program pada console browser.

Step 2 – Operator Aritmatika

```
let produksiPagi = 200;
let produksiSiang = 150;

let totalProduksi = produksiPagi + produksiSiang;
console.log("Total saat ini: " + totalProduksi);

let kekurangan = targetHarian - totalProduksi;
console.log("Kekurangan target: " + kekurangan);
```

Operator aritmatika penjumlahan (+) digunakan untuk menghitung total produksi dari dua shift kerja. Selanjutnya operator pengurangan (-) digunakan untuk mengetahui selisih antara target dan realisasi produksi. Proses ini menunjukkan penerapan dasar operasi matematika dalam JavaScript.

Step 3 – Percabangan Target Produksi

```
let jamOperasional = 1250; // dalam jam
let batasMaksimal = 1200; // batas sebelum maintenance wajib
console.log("--- Cek Status Maintenance ---");

if (jamOperasional > batasMaksimal) {
  console.log("PERINGATAN: Mesin mencapai batas maksimal");
  console.log("Status: MAINTENANCE WAJIB (Stop Produksi)");
} else if (jamOperasional > 1000) {
  console.log("Status: SIAP HATI-HATI (Segera jadwalkan maintenance)");
} else {
  console.log("Status: BERJALAN NORMAL");
}
```

Struktur percabangan `if-else` digunakan untuk menentukan apakah total produksi telah memenuhi target harian. Jika nilai `totalProduksi` lebih besar atau sama dengan `targetHarian`, maka sistem menampilkan pesan bahwa target tercapai.

Step 4 – Perhitungan Availability

```
let jamKerjaPlanned = 8; // Jam
let jamKerjaAktual = 6.5; // Jam (Ada 1.5 jam breakdown)

let availability = (jamKerjaAktual / jamKerjaPlanned) * 100;
availability = availability.toFixed(2);

console.log("Planned Time: " + jamKerjaPlanned + " Jam");
console.log("Actual Time: " + jamKerjaAktual + " Jam");
console.log("Availability: " + availability + "%");

if (availability >= 90) {
  console.log("Kategori: WORLD CLASS");
} else if (availability >= 80) {
  console.log("Kategori: BAIK (Tetap monitor)");
} else {
  console.log("Kategori: BURUK (Perlu investigasi penyebab breakdown)");
}
```

Availability dihitung berdasarkan rasio waktu kerja aktual terhadap waktu kerja rencana yang dikalikan 100%. Fungsi `toFixed(2)` digunakan untuk membatasi dua angka di belakang koma. Nilai tersebut kemudian dikategorikan menggunakan percabangan.

Pengelompokan kategori dilakukan dengan struktur `if-else if` untuk menentukan tingkat kinerja mesin, seperti WORLD CLASS, BAIK, atau BURUK berdasarkan standar tertentu.

Step 5 – Input Pengguna (prompt dan alert)

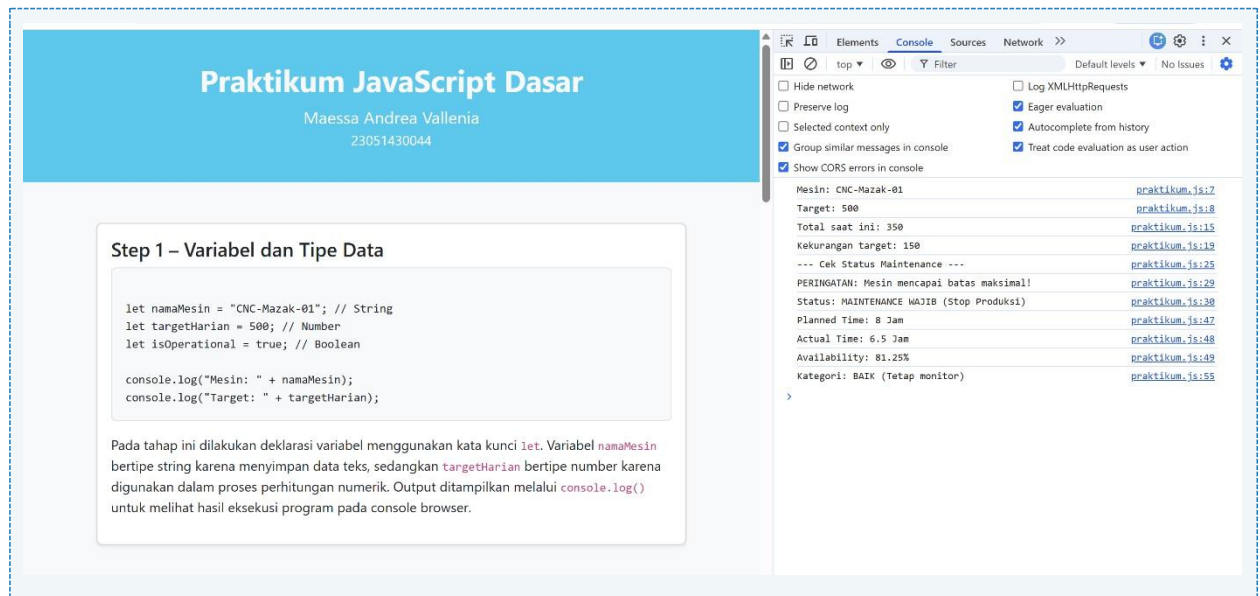
```
let namaOperator = prompt("Masukkan Nama Operator:");
let shiftKerja = prompt("Masukkan Shift (Pagi/Siang/Malam):");

if (shiftKerja === "Malam") {
  alert("Halo " + namaOperator + ", Shift malam memiliki tambahan uang makan sebesar Rp 20.000.");
} else {
  alert("Halo " + namaOperator + ", Selamat bekerja. Tetap semangat!");
}
```

Fungsi `prompt()` digunakan untuk menerima input dari pengguna dan menyimpannya dalam variabel. Nilai yang dikembalikan selalu bertipe string, kecuali jika pengguna menekan Cancel maka nilainya menjadi `null`.

Struktur percabangan digunakan untuk mengevaluasi isi variabel `shiftKerja`. Jika bernilai "Malam", maka sistem akan memberikan notifikasi tambahan uang makan menggunakan fungsi `alert()`.

Mekanisme ini menunjukkan interaksi sederhana antara pengguna dan sistem berbasis web, di mana input diproses dan menghasilkan output sesuai kondisi yang telah ditentukan.

Screenshot Hasil Console:

Gambar E.11 – Hasil Web & Console

F. LATIHAN DAN TUGAS MANDIRI

Jawab dan dokumentasikan seluruh latihan/tugas yang ada di bagian "Latihan/Tugas Mandiri" pada modul Anda.

F.1 Latihan 1 – Matematika

Pertanyaan / Instruksi Latihan:

Buat variabel `gajiPokok` dan `jamLembur`. Hitung `totalGaji` dengan asumsi upah lembur per jam adalah 1.5x dari gaji pokok dibagi 173 (jam kerja bulanan). Tampilkan hasilnya di console.

Jawaban / Kode Solusi (JS & HTML)

1. JAWABAN KODE JS

```
const form = document.getElementById("formGaji");
const table = document.getElementById("rekapTable");

form.addEventListener("submit", function (e) {

    e.preventDefault();

    // ambil data dari form
    const nama = document.getElementById("nama").value;
    const gajiPokok = Number(document.getElementById("gajiPokok").value);
    const jamLembur = Number(document.getElementById("jamLembur").value);

    // perhitungan
```



```
const upahLemburPerJam = (1.5 * gajiPokok) / 173;
const totalUpahLembur = upahLemburPerJam * jamLembur;
const totalGaji = gajiPokok + totalUpahLembur;

// console rincian
console.log("=== RINCIAN PERHITUNGAN ===");
console.log("Nama: " + nama);
console.log("Gaji Pokok: Rp " + gajiPokok);
console.log("Jam Lembur: " + jamLembur);
console.log("Upah Lembur/Jam: Rp " + upahLemburPerJam.toFixed(2));
console.log("Total Upah Lembur: Rp " + totalUpahLembur.toFixed(2));
console.log("Total Gaji: Rp " + totalGaji.toFixed(2));

// popup hasil
alert(
  "=== HASIL PERHITUNGAN ===\n\n" +
  "Nama: " + nama +
  "\nTotal Gaji: Rp " + totalGaji.toFixed(2)
);

// buat baris tabel
const row = document.createElement("tr");

row.innerHTML = `
  <td>${nama}</td>
  <td>Rp ${gajiPokok.toLocaleString("id-ID")}</td>
  <td>${jamLembur} Jam</td>
  <td>Rp ${upahLemburPerJam.toFixed(2)}</td>
  <td>Rp ${totalUpahLembur.toFixed(2)}</td>
  <td><strong>Rp ${totalGaji.toFixed(2)}</strong></td>
`;

// hapus tulisan "Belum ada data"
if (table.children[0].children[0].colSpan === 6) {
  table.innerHTML = "";
}

// tambah ke tabel
table.appendChild(row);

// reset form
form.reset();

});
```

2. JAWABAN KODE HTML

```
<!DOCTYPE html>
<html lang="id">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mini Project - Kalkulator Gaji Lembur</title>

  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>

<body class="bg-light">

  <header class="bg-info text-light text-center py-5 mb-4">
    <h1 class="fw-bold">Latihan 1 - Kalkulator Gaji Lembur</h1>
    <p class="mb-0">Maessa Andrea Vallenia</p>
    <p>23051430044</p>
  </header>

  <!-- FORM INPUT -->
  <section class="container mb-5">

    <div class="card shadow-sm">

      <div class="card-body">

        <h4 class="text-center mb-4">Input Data Karyawan</h4>

        <form id="formGaji">

          <div class="row g-3">

            <div class="col-md-4">
              <label class="form-label">Nama</label>
              <input type="text" id="nama" class="form-control"
required>
            </div>

            <div class="col-md-4">
              <label class="form-label">Gaji Pokok</label>
```

```

        <input type="number" id="gajiPokok" class="form-
control" required>
    </div>

    <div class="col-md-4">
        <label class="form-label">Jam Lembur</label>
        <input type="number" id="jamLembur" class="form-
control" required>
    </div>

</div>

<div class="mt-4">
    <button type="submit" class="btn btn-primary w-100">
        Hitung Gaji
    </button>
</div>

</form>

</div>

</div>

</section>

<!-- TABEL REKAP -->
<section class="container mb-5">

    <div class="card shadow-sm">

        <div class="card-body">

            <h4 class="text-center mb-4">Rekap Perhitungan Gaji</h4>

            <table class="table table-bordered table-striped text-center">

                <thead class="table-info">
                    <tr>
                        <th>Nama</th>
                        <th>Gaji Pokok</th>
                        <th>Jam Lembur</th>
                        <th>Upah Lembur / Jam</th>
                        <th>Total Upah Lembur</th>
                        <th>Total Gaji</th>
                    </tr>
                </thead>
            </table>
        </div>
    </div>
</section>

```

```
        </tr>
      </thead>

      <tbody id="rekapTable">

        <tr>
          <td colspan="6">Belum ada data</td>
        </tr>

      </tbody>

    </table>

  </div>

</div>

</section>

<script src="latihan_1.js"></script>

</body>

</html>
```

Latihan 1 - Kalkulator Gaji Lembur

Maessa Andrea Vallenia
23051430044

Input Data Karyawan

Hitung Gaji

Rekap Perhitungan Gaji

Nama	Gaji Pokok	Jam Lembur	Upah Lembur / Jam	Total Upah Lembur	Total Gaji
Belum ada data					

127.0.0.1:5500 says
=== HASIL PERHITUNGAN ===
Nama: Maessa
Total Gaji: Rp 2052023.12
OK

Input Data Karyawan

Hitung Gaji

Rekap Perhitungan Gaji

Nama	Gaji Pokok	Jam Lembur	Upah Lembur / Jam	Total Upah Lembur	Total Gaji
Belum ada data					

Latihan 1 - Kalkulator Gaji Lembur

Maessa Andrea Vallenia
23051430044

Input Data Karyawan

Hitung Gaji

Rekap Perhitungan Gaji

Nama	Gaji Pokok	Jam Lembur	Upah Lembur / Jam	Total Upah Lembur	Total Gaji
Maessa	Rp 2.000.000	3 Jam	Rp 17341.04	Rp 52023.12	Rp 2052023.12

Live reload enabled.

latihan_1.html:134
=== RINCIAN PERHITUNGAN ===
latihan_1.js:19
Nama: Maessa
latihan_1.js:20
Gaji Pokok: Rp 2000000
latihan_1.js:21
Jam Lembur: 3
latihan_1.js:22
Upah Lembur/Jam: Rp 17341.04
latihan_1.js:23
Total Upah Lembur: Rp 52023.12
latihan_1.js:24
Total Gaji: Rp 2052023.12
latihan_1.js:25

Gambar F.1 – Hasil Web & Console Matematika

F.2 Latihan 2 – Logika Switch

Pertanyaan / Instruksi Latihan:

Gunakan `switch` untuk mengkonversi kode shift angka (1, 2, 3) menjadi nama shift (Pagi, Siang, Malam). Jika input selain 1-3, tampilkan "Shift Tidak Valid".

Jawaban / Kode Solusi:

1. JAWABAN KODE JS

```
if (input === 1) {
    outputDiv.textContent = "Shift Anda adalah Pagi";
    outputDiv.style.color = "green";
    console.log("Shift Anda adalah Pagi"); // ini supaya muncul di console
    juga
} else if (input === 2) {
    outputDiv.textContent = "Shift Anda adalah Siang";
    outputDiv.style.color = "green";
    console.log("Shift Anda adalah Siang");
} else if (input === 3) {
    outputDiv.textContent = "Shift Anda adalah Malam";
    outputDiv.style.color = "green";
    console.log("Shift Anda adalah Malam");
} else {
    outputDiv.textContent = "Shift Tidak Valid";
    outputDiv.style.color = "red";
    console.log("Shift Tidak Valid");
}
```

2. JAWABAN KODE HTML

```
<!DOCTYPE html>
<html lang="id">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Absensi Karyawan - Rekap Shift</title>

    <!-- Bootstrap -->
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>

<body>
```

```
<!-- HEADER -->
<header class="bg-info text-light text-center py-5 mb-4">
  <h1 class="fw-bold mb-2">Absensi Karyawan - Rekap Shift</h1>
  <p class="fs-5 mb-0">Maessa Andrea Vallenia</p>
  <p class="mb-0">23051430044</p>
</header>

<!-- FORM INPUT -->
<section class="container mb-5">

  <div class="card shadow-sm">

    <div class="card-body">

      <h4 class="mb-3">Input Data Karyawan</h4>

      <div class="row g-3">

        <div class="col-md-6">
          <input
            type="text"
            id="nama"
            class="form-control"
            placeholder="Nama Karyawan"
          >
        </div>

        <div class="col-md-6">
          <input
            type="date"
            id="tanggal"
            class="form-control"
          >
        </div>

      </div>

      <div class="mt-3">

        <button
          id="btnShift"
          class="btn btn-primary w-100"
        >
          Masukkan Shift
        </button>

      </div>

    </div>

  </div>

</section>
```

```
        </div>

        <div
            id="hasilOutput"
            class="mt-3 fw-bold fs-5 text-center"
        >
        </div>

    </div>

</div>

</section>

<!-- TABEL REKAP -->
<section class="container mb-5">

    <div class="card shadow-sm">

        <div class="card-body">

            <h4 class="mb-3">Rekap Data Karyawan</h4>

            <table class="table table-bordered table-striped text-center">

                <thead class="table-info">

                    <tr>
                        <th>Nama</th>
                        <th>Tanggal</th>
                        <th>Shift</th>
                        <th>Status</th>
                    </tr>

                </thead>

                <tbody id="rekapTable">

                    <tr>
                        <td colspan="4">Belum ada data</td>
                    </tr>

                </tbody>

            </table>

        </div>

    </div>

</section>
```



```
        </table>

    </div>

</div>

</section>

<script>

const tombol = document.getElementById("btnShift");
const output = document.getElementById("hasilOutput");
const tabel = document.getElementById("rekapTable");

tombol.addEventListener("click", function () {

    console.log("==== PROSES INPUT DATA =====");

    let nama = document.getElementById("nama").value.trim();
    let tanggal = document.getElementById("tanggal").value;

    console.log("Nama:", nama);
    console.log("Tanggal:", tanggal);

    let shift = Number(
        prompt("Masukkan kode shift\n1 = Pagi\n2 = Siang\n3 = Malam")
    );

    console.log("Kode Shift:", shift);

    if (!nama || !tanggal || ![1,2,3].includes(shift)) {

        console.error("Data tidak valid");

        output.textContent = "Data tidak valid!";
        output.style.color = "red";

        return;
    }

    let shiftText;
```

```
    if (shift === 1) {
        shiftText = "Pagi";
    }

    else if (shift === 2) {
        shiftText = "Siang";
    }

    else {
        shiftText = "Malam";
    }

    console.log("Shift:", shiftText);

    output.textContent = nama + " - Shift " + shiftText;
    output.style.color = "green";

    let barisBaru = document.createElement("tr");

    barisBaru.innerHTML =
        "<td>" + nama + "</td>" +
        "<td>" + tanggal + "</td>" +
        "<td>" + shiftText + "</td>" +
        "<td style='color:green'>Valid</td>";

    if (tabel.children[0].children[0].colSpan === 4) {

        tabel.innerHTML = "";
        console.log("Placeholder 'Belum ada data' dihapus");

    }

    tabel.appendChild(barisBaru);

    console.log("Data berhasil masuk ke tabel");
    console.log("=====");

    document.getElementById("nama").value = "";
    document.getElementById("tanggal").value = "";

});
```

```
</script>
```

```
</body>
```

```
</html>
```

Absensi Karyawan - Rekap Shift

Maessa Andrea Vallenia
23051430044

Input Data Karyawan

Nama Karyawan

dd/mm/yyyy

Masukkan Shift

Rekap Data Karyawan

Nama	Tanggal	Shift	Status
Belum ada data			

127.0.0.1:5500 says
Masukkan kode shift
1 = Pagi
2 = Siang
3 = Malam

OK Cancel

Input Data Karyawan

Maessa

06/03/2026

Masukkan Shift

Rekap Data Karyawan

Nama	Tanggal	Shift	Status
Belum ada data			

Absensi Karyawan - Rekap Shift

Maessa Andrea Vallenia
23051430044

Input Data Karyawan

Nama Karyawan

dd/mm/yyyy

Masukkan Shift

Maessa - Shift Pagi

Rekap Data Karyawan

Nama	Tanggal	Shift	Status
Maessa	2026-03-06	Pagi	Valid

Elements

Console

Sources

Network

Default levels

No Issues

☐ Hide network

☒ Preserve log

☐ Selected context only

☒ Group similar messages in console

☒ Show CORS errors in console

☒ Log XMLHttpRequests

☒ Eager evaluation

☒ Autocomplete from history

☒ Treat code evaluation as user action

===== PROSES INPUT DATA =====

Nama: Maessa

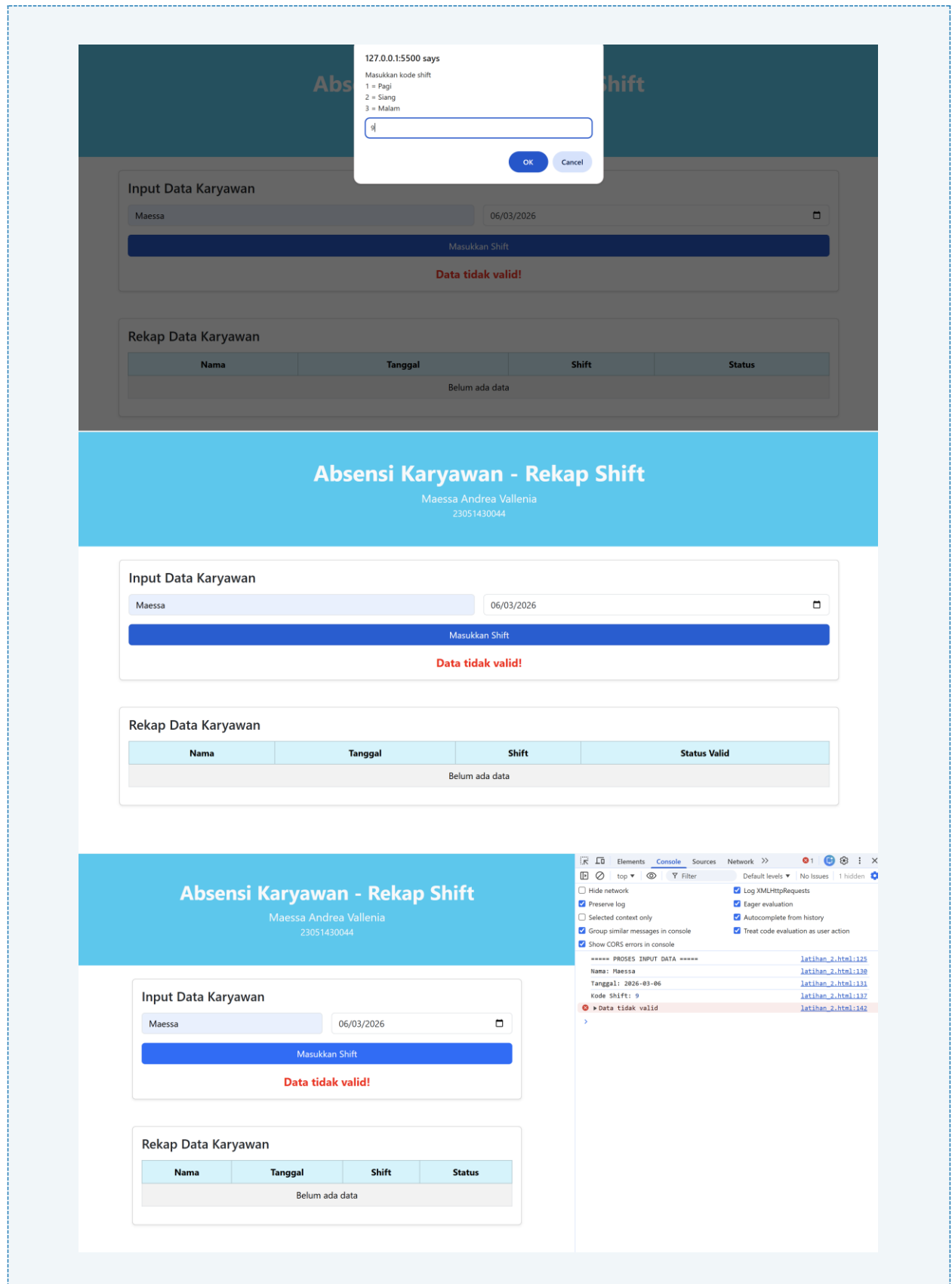
Tanggal: 2026-03-06

Kode Shift: 1

Shift: Pagi

Placeholder 'Belum ada data' dihapus

Data berhasil masuk ke tabel



Gambar F.2 – Hasil Web & Console Logika Switch

F.3 Tugas Proyek Mini – Kalkulator Biaya Produksi

Deskripsi proyek mini:

- Buat program di `script.js` yang menghitung biaya produksi total per unit.
- Variabel: `biayaBahanBaku`, `biayaTenagaKerja`, `biayaOverhead`, `jumlahProduksi`.
 - Rumus: $\text{totalPerUnit} = (\text{biayaBahanBaku} + \text{biayaTenagaKerja} + \text{biayaOverhead}) / \text{jumlahProduksi}$.
 - Gunakan `if/else` untuk mengecek: jika `jumlahProduksi` < 100, tampilkan "Biaya Tinggi (Ekonomi Skala Kecil)". Jika ≥ 100 , tampilkan "Biaya Efisien".

Penjelasan pendekatan/solusi yang digunakan:

- Proyek mini dimulai dengan memahami kebutuhan program. Program ini dirancang untuk menghitung biaya produksi per unit berdasarkan input biaya bahan baku, biaya tenaga kerja, biaya overhead, dan jumlah produksi. Hasil perhitungan akan menampilkan total biaya per unit dan status efisiensi produksi (Biaya Tinggi atau Biaya Efisien).
- Membuat folder baru, misalnya ProyekMini_BiayaProduksi, di Visual Studio Code. Di dalam folder ini, buat file index.html untuk tampilan halaman, dan file ProyekMini_Kalkulator_Biaya_Produksi.js untuk logika perhitungan.
- Menyiapkan struktur HTML
 - Tambahkan tag <html>, <head>, dan <body>.
 - Tambahkan meta tag charset dan viewport untuk responsif.
 - Masukkan Bootstrap CSS melalui CDN agar layout dan komponen siap pakai.
 - Buat header berisi judul proyek dan identitas mahasiswa.
 - Tambahkan container untuk form input dan area hasil perhitungan.
- Membuat form input data produksi terdiri dari:
 - Biaya Bahan Baku (input number)
 - Biaya Tenaga Kerja (input number)
 - Biaya Overhead (input number)
 - Jumlah Produksi (input number)
 - Tombol "Hitung Biaya" untuk memproses perhitungan.Form ini dibuat agar mahasiswa bisa memasukkan data secara mudah dan rapi menggunakan sistem grid Bootstrap.
- Membuat area (misal <div> dengan ID hasilOutput) untuk menampilkan hasil:
 - Total biaya per unit
 - Status efisiensi produksi (Biaya Tinggi atau Biaya Efisien)Penempatan area ini dibuat agar terlihat jelas di bawah form.
- Logika perhitungan dan percabangan (disimpan di JS)
 - Rumus: $\text{totalPerUnit} = (\text{biayaBahanBaku} + \text{biayaTenagaKerja} + \text{biayaOverhead}) / \text{jumlahProduksi}$
 - Percabangan:
 - $\text{jumlahProduksi} < 100 \rightarrow$ "Biaya Tinggi (Ekonomi Skala Kecil)"
 - $\text{jumlahProduksi} \geq 100 \rightarrow$ "Biaya Efisien"
- Pastikan jumlah produksi > 0 agar perhitungan tidak error. Jika input (-) tampilkan pesan peringatan bahwa input tidak valid.
- Hasil uji coba program ditampilkan di halaman dan console browser.

Jawaban / Kode Solusi:

1. JAWABAN KODE JS

```
const form = document.getElementById("formProduksi");
const output = document.getElementById("hasilOutput");

form.addEventListener("submit", function (e) {
    e.preventDefault();

    // Ambil nilai input
    let biayaBahanBaku = Number(document.getElementById("bahanBaku").value);
    let biayaTenagaKerja =
    Number(document.getElementById("tenagaKerja").value);
    let biayaOverhead = Number(document.getElementById("overhead").value);
    let jumlahProduksi =
    Number(document.getElementById("jumlahProduksi").value);

    // Validasi input
    if (
        isNaN(biayaBahanBaku) || biayaBahanBaku < 0 ||
        isNaN(biayaTenagaKerja) || biayaTenagaKerja < 0 ||
        isNaN(biayaOverhead) || biayaOverhead < 0 ||
        isNaN(jumlahProduksi) || jumlahProduksi < 1
    ){
        output.innerHTML = "Input tidak valid! Semua nilai harus ≥ 0 dan  
jumlah produksi ≥ 1.";
        output.className = "status-error fs-5 fw-semibold";
        console.error("Input tidak valid");
        return;
    }

    // Perhitungan total per unit
    let totalPerUnit = (biayaBahanBaku + biayaTenagaKerja + biayaOverhead) /
    jumlahProduksi;

    // Log rinci di console
    console.log(`%c[${tanggal}] Rincian Perhitungan:`, 'color: blue; font-  
weight:bold;');
    console.log(`Biaya Bahan Baku: Rp ${biayaBahanBaku.toLocaleString("id-  
ID")}`);
    console.log(`Biaya Tenaga Kerja: Rp  
${biayaTenagaKerja.toLocaleString("id-ID")}`);
    console.log(`Biaya Overhead: Rp ${biayaOverhead.toLocaleString("id-  
ID")}`);
    console.log(`Jumlah Produksi: ${jumlahProduksi} unit`);
```

```

        console.log(`Total Biaya per Unit: Rp ${totalPerUnit.toLocaleString("id-ID")}`);
        console.log(`Status Produksi: ${status}`);

        // Tentukan status produksi
        let status = "";
        let kelasStatus = "";
        if (jumlahProduksi < 100) {
            status = "Biaya Tinggi (Ekonomi Skala Kecil)";
            kelasStatus = "status-tinggi";
        } else {
            status = "Biaya Efisien";
            kelasStatus = "status-efisien";
        }

        // Tampilkan hasil di halaman
        output.innerHTML =
            `Total Biaya per Unit: ${totalPerUnit.toLocaleString("id-ID", {
style: "currency", currency: "IDR" })} <br>` +
            `Status Produksi: ${status}`;
        output.className = `${kelasStatus} fs-5 fw-semibold`;

        // Console log warna
        let color = kelasStatus === "status-efisien" ? "green" : "red";
        console.log(`%cStatus Produksi: ${status}`, `color: ${color}; font-weight:bold;`);

        form.reset();
    });

```

2. JAWABAN KODE HTML

```

<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Kalkulator Produksi Lengkap</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">
    <style>
        .status-efisien { color: green; font-weight: bold; }
        .status-tinggi { color: red; font-weight: bold; }
        .status-error { color: orange; font-weight: bold; }
        #hasilOutput { transition: all 0.3s ease-in-out; }
    </style>

```



```

    </style>
</head>
<body class="bg-light">
<header class="bg-info text-light text-center py-5 mb-4">
    <h1 class="fw-bold mb-2">Proyek Mini - Kalkulator Biaya Produksi</h1>
    <p class="fs-5 mb-0">Maessa Andrea Vallenia</p>
    <p class="mb-0">23051430044</p>
</header>
<main class="container my-5">
    <section class="mb-5">
        <div class="card shadow-sm">
            <div class="card-body">
                <h4 class="text-center mb-4">Input Data Produksi</h4>
                <form id="formProduksi">
                    <div class="row g-3">
                        <div class="col-md-6">
                            <label class="form-label">Biaya Bahan Baku
(Rp)</label>
                            <input type="number" class="form-control"
id="bahanBaku" min="0" required>
                        </div>
                        <div class="col-md-6">
                            <label class="form-label">Biaya Tenaga Kerja
(Rp)</label>
                            <input type="number" class="form-control"
id="tenagaKerja" min="0" required>
                        </div>
                        <div class="col-md-6">
                            <label class="form-label">Biaya Overhead
(Rp)</label>
                            <input type="number" class="form-control"
id="overhead" min="0" required>
                        </div>
                        <div class="col-md-6">
                            <label class="form-label">Jumlah Produksi
(Unit)</label>
                            <input type="number" class="form-control"
id="jumlahProduksi" min="1" required>
                        </div>
                    </div>
                    <div class="mt-3">
                        <button type="submit" class="btn btn-primary w-
100">Hitung Biaya</button>
                    </div>
                </form>
            </div>
        </div>
    </section>
</main>

```

```

        <div id="hasilOutput" class="fw-semibold fs-5 mt-3 text-
center"></div>
    </div>
</div>
</section>
<section class="mb-5">
    <div class="card shadow-sm">
        <div class="card-body">
            <h4 class="text-center mb-4">Rekap Produksi</h4>
            <table class="table table-bordered table-striped text-
center">
                <thead class="table-info">
                    <tr>
                        <th>#</th>
                        <th>Tanggal</th>
                        <th>Biaya Bahan Baku</th>
                        <th>Biaya Tenaga Kerja</th>
                        <th>Biaya Overhead</th>
                        <th>Jumlah Produksi</th>
                        <th>Total / Unit</th>
                        <th>Status</th>
                    </tr>
                </thead>
                <tbody id="rekapTable">
                    <tr>
                        <td colspan="8">Belum ada data</td>
                    </tr>
                </tbody>
            </table>
        </div>
    </div>
</section>
</main>

<script>
const form = document.getElementById("formProduksi");
const output = document.getElementById("hasilOutput");
const rekapTable = document.getElementById("rekapTable");
let counter = 1;

form.addEventListener("submit", function(e){
    e.preventDefault();

    let biayaBahanBaku = Number(document.getElementById("bahanBaku").value);
    let biayaTenagaKerja =
Number(document.getElementById("tenagaKerja").value);

```

```
let biayaOverhead = Number(document.getElementById("overhead").value);
let jumlahProduksi =
Number(document.getElementById("jumlahProduksi").value);

if(biayaBahanBaku < 0 || biayaTenagaKerja < 0 || biayaOverhead < 0 ||
jumlahProduksi < 1){
    output.textContent = "Input tidak valid!";
    output.className = "status-error fs-5 fw-semibold";
    console.error("Input tidak valid");
    return;
}

let totalPerUnit = (biayaBahanBaku + biayaTenagaKerja + biayaOverhead) /
jumlahProduksi;

let status = "";
let kelasStatus = "";

if(jumlahProduksi < 100){
    status = "Biaya Tinggi (Ekonomi Skala Kecil)";
    kelasStatus = "status-tinggi";
} else {
    status = "Biaya Efisien";
    kelasStatus = "status-efisien";
}

let tanggal = new Date().toLocaleDateString("id-ID");

output.innerHTML =
`Tanggal: ${tanggal}<br>
Total Biaya per Unit: Rp ${totalPerUnit.toLocaleString("id-ID")}<br>
Status Produksi: ${status}`;

output.className = `${kelasStatus} fs-5 fw-semibold`;

let row = document.createElement("tr");
row.innerHTML = `
    <td>${counter++}</td>
    <td>${tanggal}</td>
    <td>Rp ${biayaBahanBaku.toLocaleString("id-ID")}</td>
    <td>Rp ${biayaTenagaKerja.toLocaleString("id-ID")}</td>
    <td>Rp ${biayaOverhead.toLocaleString("id-ID")}</td>
    <td>${jumlahProduksi}</td>
    <td>Rp ${totalPerUnit.toLocaleString("id-ID")}</td>
    <td class="${kelasStatus}">${status}</td>
`;
```

```
if(rekapTable.children[0].children[0].colSpan === 8){
    rekapTable.innerHTML = "";
}

rekapTable.appendChild(row);

console.log("==== RINCIAN PRODUKSI ====");
console.log("Biaya Bahan Baku:", biayaBahanBaku);
console.log("Biaya Tenaga Kerja:", biayaTenagaKerja);
console.log("Biaya Overhead:", biayaOverhead);
console.log("Jumlah Produksi:", jumlahProduksi);
console.log("Total per Unit:", totalPerUnit);
console.log("Status:", status);

form.reset();
});
</script>

</body>
</html>
```

Proyek Mini - Kalkulator Biaya Produksi

Maessa Andrea Vallenia
23051430044

Input Data Produksi

Biaya Bahan Baku (Rp)

Biaya Tenaga Kerja (Rp)

Biaya Overhead (Rp)

Jumlah Produksi (Unit)

Hitung Biaya

Rekap Produksi

#	Tanggal	Biaya Bahan Baku	Biaya Tenaga Kerja	Biaya Overhead	Jumlah Produksi	Total / Unit	Status
Belum ada data							

Input Data Produksi

Biaya Bahan Baku (Rp)

Biaya Tenaga Kerja (Rp)

Biaya Overhead (Rp)

Jumlah Produksi (Unit)

Hitung Biaya

Tanggal: 6/3/2026
Total Biaya per Unit: Rp 280.000
Status Produksi: Biaya Tinggi (Ekonomi Skala Kecil)

Rekap Produksi

#	Tanggal	Biaya Bahan Baku	Biaya Tenaga Kerja	Biaya Overhead	Jumlah Produksi	Total / Unit	Status
1	6/3/2026	Rp 2.000.000	Rp 500.000	Rp 300.000	10	Rp 280.000	Biaya Tinggi (Ekonomi Skala Kecil)

Input Data Produksi

Biaya Bahan Baku (Rp)

Biaya Tenaga Kerja (Rp)

Biaya Overhead (Rp)

Jumlah Produksi (Unit)

Hitung Biaya

Rekap Produksi

#	Tanggal	Biaya Bahan Baku	Biaya Tenaga Kerja	Biaya Overhead	Jumlah Produksi	Total / Unit	Status
Belum ada data							

```
===== RINCIAN PRODUKSI =====
Biaya Bahan Baku: 2000000
Biaya Tenaga Kerja: 500000
Biaya Overhead: 300000
Jumlah Produksi: 10
Total per Unit: 280000
Status: Biaya Tinggi (Ekonomi Skala Kecil)
```

Proyek Mini - Kalkulator Biaya Produksi

Maessa Andrea Vallenia
23051430044

Input Data Produksi

Biaya Bahan Baku (Rp)

Biaya Tenaga Kerja (Rp)

Biaya Overhead (Rp)

Jumlah Produksi (Unit)

Value must be greater than or equal to 0.

Hitung Biaya

Rekap Produksi

#	Tanggal	Biaya Bahan Baku	Biaya Tenaga Kerja	Biaya Overhead	Jumlah Produksi	Total / Unit	Status
Belum ada data							

Gambar F.3 – Hasil Proyek Mini Kalkulator Biaya Produksi

G. PEMBAHASAN

Pada pertemuan ini dipelajari integrasi HTML, CSS/Bootstrap, dan JavaScript untuk membuat halaman web interaktif. Dalam praktikum dibuat proyek mini kalkulator biaya produksi yang dapat menerima input biaya bahan baku, tenaga kerja, overhead, dan jumlah produksi. Struktur halaman dibuat menggunakan HTML dengan komponen seperti header, form input, dan tabel rekap, sedangkan Bootstrap digunakan agar tampilan lebih rapi dan responsif.

JavaScript digunakan untuk melakukan perhitungan biaya produksi per unit dengan rumus:

$$\text{totalPerUnit} = (\text{biayaBahanBaku} + \text{biayaTenagaKerja} + \text{biayaOverhead}) / \text{jumlahProduksi}$$

Hasil perhitungan kemudian diproses menggunakan percabangan if/else untuk menentukan status produksi, yaitu Biaya Tinggi jika jumlah produksi kurang dari 100 unit dan Biaya Efisien jika jumlah produksi 100 unit atau lebih. Hasil tersebut ditampilkan di halaman web dan juga dicatat pada tabel rekap produksi.

Selain itu, program menggunakan console.log() untuk menampilkan rincian perhitungan di console browser sehingga memudahkan pengecekan hasil program. Praktikum ini membantu memahami hubungan antara input data, proses logika program, dan output yang ditampilkan, serta memberikan gambaran penerapan sederhana dalam sistem perhitungan produksi di industri.

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum menunjukkan bahwa program yang dibuat sudah sesuai dengan teori HTML, Bootstrap, dan JavaScript. Struktur halaman menggunakan HTML dapat menampilkan header, form input biaya produksi, serta tabel rekap dengan baik. Penggunaan Bootstrap juga membuat tampilan lebih rapi, terstruktur, dan responsif sehingga data mudah dibaca.

Pada bagian JavaScript, perhitungan biaya produksi per unit dilakukan menggunakan variabel dan operator aritmatika sesuai rumus yang telah ditentukan. Selain itu, percabangan if/else digunakan untuk menentukan status produksi, yaitu Biaya Tinggi atau Biaya Efisien berdasarkan jumlah produksi. Hasil perhitungan berhasil ditampilkan di halaman web, disimpan pada tabel rekap, serta dicatat di console.log() untuk melihat rincian proses perhitungan.

Secara umum, hasil praktikum sudah sesuai dengan teori karena program mampu menerima input, melakukan proses perhitungan, dan menampilkan output dengan benar. Hal ini menunjukkan bahwa konsep dasar pemrograman web dan logika JavaScript telah diterapkan dengan baik.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Popup alert() tidak muncul.	Samakan nama file JS yang ditautkan di HTML dengan file tempat kode alert() dan logika kalkulasi ditulis agar browser dapat mengeksekusi script dengan benar.
Perhitungan menampilkan NaN	Pastikan semua nama variabel di HTML dan JS konsisten. Jika input HTML menggunakan id="jumlahProduksi", maka di JS harus dipanggil dengan nama yang sama (jumlahProduksi) agar perhitungan tidak menghasilkan NaN.

G.3 Kaitan dengan Penerapan di Industri

Materi pada praktikum ini berkaitan dengan penerapan di industri, khususnya dalam pembuatan sistem pencatatan dan perhitungan biaya produksi berbasis web. Struktur HTML dan layout Bootstrap dapat digunakan untuk membuat tampilan dashboard atau form input data produksi yang rapi dan mudah dibaca, seperti pada sistem monitoring produksi di perusahaan.

Penggunaan JavaScript juga mencerminkan proses pengolahan data di industri.

Variabel dan operator aritmatika digunakan untuk menghitung biaya produksi per unit, sedangkan percabangan if/else digunakan untuk menentukan status produksi apakah Biaya Tinggi atau Biaya Efisien. Selain itu, data yang diinput dapat ditampilkan kembali dalam bentuk tabel rekap dan juga dicatat di console.log() untuk melihat rincian proses perhitungan.

Secara umum, konsep ini mirip dengan sistem informasi produksi di industri yang melakukan proses input data, perhitungan biaya, dan menampilkan hasil laporan secara otomatis. Walaupun masih sederhana, praktikum ini memberikan gambaran dasar bagaimana sistem pengolahan data produksi dapat dibuat menggunakan teknologi web.

H. KESIMPULAN

Tuliskan 3–5 poin kesimpulan yang merangkum hal-hal yang dipelajari dan dicapai pada praktikum modul ini. Kesimpulan harus spesifik, bukan bersifat umum.

1	Mahasiswa mampu membuat halaman web menggunakan HTML dan Bootstrap yang berisi form input data produksi dan tabel rekap hasil perhitungan dengan tampilan yang rapi dan terstruktur.
2	Mahasiswa dapat menerapkan JavaScript dasar seperti variabel, operator aritmatika, dan percabangan if/else untuk menghitung biaya produksi per unit berdasarkan biaya bahan baku, tenaga kerja, overhead, dan jumlah produksi.
3	Program berhasil menampilkan hasil perhitungan biaya per unit dan status produksi (Biaya Tinggi atau Biaya Efisien) secara langsung pada halaman web.
4	Data hasil perhitungan juga dapat disimpan pada tabel rekap dan ditampilkan di <code>console.log()</code> sehingga proses perhitungan dapat dilihat secara lebih rinci.
5	Praktikum ini menunjukkan integrasi antara tampilan web (HTML & Bootstrap) dan logika perhitungan JavaScript sehingga membentuk sistem perhitungan biaya produksi sederhana yang dapat digunakan sebagai dasar pengembangan aplikasi industri.

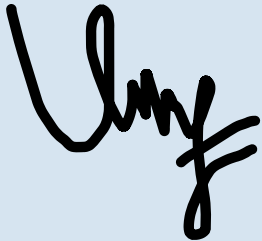
I. DAFTAR PUSTAKA

Tuliskan semua sumber referensi yang digunakan dalam format APA 7th Edition. Minimal 3 referensi yang relevan (termasuk Manual Praktikum dan sumber online resmi seperti MDN, Reactjs.org, dll).

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Kom, I. L. M. (2023). <i>Dasar Algoritma dan Pemograman Javascript</i> . Irvan Lewenusa, M. Kom.
[3]	Sholikhan, M. (2022). Html, Css Dan Javascript. <i>Penerbit Yayasan Prima Agus Teknik</i> , 1-343 .
[4]	https://www.javascripttutorial.net/
[5]	https://developer.mozilla.org/en-US/docs/Web/JavaScript

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa  Maessa Andrea Vallenia NIM: 23051430044	Diperiksa oleh Asisten / Dosen Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____
--	---

MODUL 5

ARRAY, OBJECT DI JAVASCRIPT

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 5 (Pertemuan 5)
Topik Utama	Modularitas kode (Function), pengelolaan data list (Array), dan struktur data kompleks (Object).
Sub-Topik	Function, Parameter & Return, Array, Looping, Object, Nested Object, Array of Object, Manipulasi Data, Logika Kondisi, Modularitas Kode.
Durasi Praktikum	340 Menit
Tanggal Praktikum	13 Maret 2026
Nama Mahasiswa	Maessa Andrea Vallenia
NIM	23051430044
Kelas	A

A. TUJUAN PEMBELAJARAN

Tuliskan tujuan pembelajaran yang ingin dicapai pada modul ini, sesuai dengan yang tertera di Manual Praktikum.

1	Memahami konsep Function untuk menghindari pengulangan kode (DRY - Don't Repeat Yourself).
2	Menguasai penggunaan Array untuk menyimpan sekumpulan data (misal: daftar cacat produksi).
3	Memahami Object untuk merepresentasikan entitas nyata (misal: Spesifikasi Mesin) dalam bentuk data.
4	Mampu melakukan iterasi (perulangan) terhadap Array dan Object.

B. ALAT DAN BAHAN

Sebutkan seluruh perangkat keras, perangkat lunak, dan sumber referensi yang digunakan selama praktikum.

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
2	Google Chrome / Browser Modern	Versi terbaru	<i>Testing & Debugging</i>
3	Live Server	Versi 5.7.9	Menjalankan HTML dan memantau perubahan file
4	JavaScript (Built-in Browser)	ES6 (Default Browser Modern)	Bahasa pemrograman untuk logika dan perhitungan
5	CodeSnap	Versi 1.3.4	Dokumentasi kode program
6	Git dan GitHub	Versi 2.53.0	Version control system (VCS) berbasis Git
7	Koneksi Internet	Stabil	Mengakses Library dan Framework
8	Console Browser (Developer Tools)	Default Browser	Melihat output program dan debugging
9	Quokka.js (Opsional)	Versi 1.0.755	Melihat hasil JavaScript secara langsung di VS Code

C. DASAR TEORI

1. JavaScript merupakan bahasa pemrograman yang digunakan untuk menambahkan logika dan interaksi pada halaman web. Jika HTML berfungsi sebagai struktur dan CSS sebagai tampilan, maka JavaScript digunakan untuk melakukan perhitungan, mengolah data, dan menjalankan instruksi berdasarkan kondisi tertentu. Dalam bidang teknik industri, JavaScript dapat

- digunakan untuk membuat perhitungan produksi, menghitung konsumsi energi, serta menampilkan data secara otomatis pada sistem berbasis web.
2. Function adalah sekumpulan perintah yang disimpan dalam satu blok kode dan dapat dipanggil kembali saat diperlukan. Penggunaan function bertujuan untuk menghindari pengulangan kode sehingga program menjadi lebih rapi dan mudah dipelihara. Konsep ini dikenal dengan prinsip DRY (Don't Repeat Yourself). Function dapat menerima parameter sebagai input dan mengembalikan nilai menggunakan return. Dalam teknik industri, function dapat dianalogikan seperti SOP yang dapat digunakan berulang kali.
 3. Array adalah struktur data yang digunakan untuk menyimpan banyak nilai dalam satu variabel. Setiap data dalam array memiliki indeks yang dimulai dari nol. Array digunakan untuk menyimpan kumpulan data seperti daftar produk, data cacat, atau stok material. JavaScript menyediakan metode seperti length, push, dan looping untuk mengelola data dalam array.
 4. Looping adalah proses menjalankan perintah secara berulang. JavaScript memiliki beberapa jenis perulangan seperti for dan forEach. Perulangan digunakan untuk membaca setiap data dalam array atau menjalankan perintah berulang secara otomatis. Dengan looping, program dapat memproses banyak data dengan lebih cepat dan efisien.
 5. Object adalah struktur data yang digunakan untuk menyimpan informasi yang lebih kompleks dalam bentuk pasangan properti dan nilai. Object dapat digunakan untuk merepresentasikan satu data lengkap, seperti data mesin, data produk, atau data operator. Object juga dapat memiliki object lain di dalamnya sehingga data menjadi lebih terstruktur.
 6. Array of Object adalah gabungan antara array dan object yang digunakan untuk menyimpan banyak data kompleks dalam satu variabel. Struktur ini sering digunakan untuk menyimpan data seperti daftar material, daftar mesin, atau daftar pekerjaan. Data dalam array of object biasanya diproses menggunakan looping seperti for atau forEach.
 7. Modularitas kode adalah konsep menyusun program menjadi bagian-bagian kecil agar lebih rapi dan mudah dipahami. Dengan menggunakan function, array, dan object, program menjadi lebih terstruktur dan mudah dikembangkan. Konsep ini penting dalam pembuatan sistem karena membantu mengurangi kesalahan dan memudahkan perawatan program.

c.1 Konsep Utama

Konsep utama pada Pertemuan 5 berfokus pada modularitas kode dalam JavaScript dengan menggunakan function, array, dan object agar program lebih rapi dan mudah dipahami. Dengan modularitas, kode dapat dibagi menjadi beberapa bagian kecil sehingga tidak perlu menulis perintah yang sama berulang kali. Prinsip ini dikenal dengan DRY (Don't Repeat Yourself) yang bertujuan membuat program lebih efisien dan terstruktur.

Function digunakan untuk menyimpan sekumpulan perintah yang dapat dipanggil kembali saat diperlukan. Function dapat menerima parameter, melakukan proses, dan mengembalikan hasil menggunakan return. Dengan function, perhitungan seperti daya mesin atau jumlah produksi bisa dilakukan berulang tanpa menulis ulang kode.

Array digunakan untuk menyimpan banyak data dalam satu variabel, misalnya daftar produk cacat atau stok material. Data dalam array dapat diproses menggunakan perulangan seperti `for` atau `forEach`. Selain itu, object digunakan untuk menyimpan data yang lebih lengkap dengan beberapa properti, seperti data mesin yang memiliki nama, status, dan spesifikasi. Gabungan array dan object disebut array of object, yang sering digunakan dalam program untuk menyimpan data yang lebih kompleks dan terstruktur.

c.2 Keterkaitan dengan Teknik Industri

Materi JavaScript pada Pertemuan 5 memiliki keterkaitan dengan bidang Teknik Industri terutama dalam pengolahan data, penyusunan sistem, dan pembuatan program yang terstruktur. Dalam dunia industri, banyak proses yang membutuhkan pengolahan data secara berulang, seperti perhitungan konsumsi energi, pencatatan hasil produksi, data cacat, dan stok material. Konsep function digunakan agar perhitungan dapat dilakukan berulang tanpa menulis kode yang sama, sehingga program menjadi lebih efisien seperti penggunaan SOP dalam proses kerja.

Penggunaan array sangat berkaitan dengan penyimpanan data dalam jumlah banyak, misalnya daftar produk cacat, data material di gudang, atau daftar pekerjaan mesin. Dengan bantuan perulangan, data tersebut dapat diproses secara otomatis sehingga memudahkan analisis dan monitoring. Selain itu, object digunakan untuk merepresentasikan data yang lebih lengkap, seperti data mesin yang memiliki nama, status, dan spesifikasi, sehingga informasi dapat disimpan dengan lebih terstruktur.

Gabungan array dan object sering digunakan dalam sistem nyata di industri, misalnya dalam sistem inventori, sistem antrian produksi, dan sistem monitoring mesin. Dengan memahami konsep function, array, dan object, mahasiswa Teknik Industri dapat membuat program yang lebih terorganisir dan mendekati sistem informasi yang digunakan di dunia industri, sehingga membantu meningkatkan efisiensi dan ketepatan dalam pengolahan data.

D. LANGKAH KERJA DAN HASIL PRAKTIKUM

Dokumentasikan setiap langkah kerja yang Anda lakukan, beserta hasil yang diperoleh (screenshot, kode, atau output). Jawab sesuai urutan langkah di Manual Praktikum.

D.1 Langkah 1 – Membuat Function Perhitungan

Buat file baru ``pertemuan5.html`` dan ``pertemuan5.js``. Kita akan membuat kalkulator energi listrik mesin.

```
/* ===== KODE / OUTPUT LANGKAH 1 ===== */  
  
1. JAWABAN KODE JS  
  
// FUNCTION Declaration  
// Fungsi untuk menghitung konsumsi daya (P = V x I)
```

```
function hitungDaya(tegangan, arus) {
    let daya = tegangan * arus;
    return daya;
}

// Pemanggilan Function

let teganganMesin = 220;
let arusMesin = 10;

let totalDaya = hitungDaya(teganganMesin, arusMesin);
console.log("Daya Mesin: " + totalDaya + " Watt");

// TAMBAHAN SUPAYA MUNCUL DI WEB

const btn = document.getElementById("btnHitung");

const inputV = document.getElementById("tegangan");
const inputI = document.getElementById("arus");

const output = document.getElementById("hasilOutput");
const tabel = document.getElementById("rekapTable");

btn.addEventListener("click", function () {

    let V = Number(inputV.value);
    let I = Number(inputI.value);

    let hasil = hitungDaya(V, I);

    console.log("=== HITUNG DAYA ===");
    console.log("Tegangan:", V);
    console.log("Arus:", I);
    console.log("Daya:", hasil);

    // hapus tulisan awal tabel
    if (tabel.children[0].children[0].colSpan) {
        tabel.innerHTML = "";
    }

    // tampil di web
```

```
output.innerHTML =  
    "P = V × I = " +  
    V + " × " + I +  
    " = " + hasil + " Watt";  
  
// tambah ke tabel  
  
let row = document.createElement("tr");  
  
row.innerHTML =  
    "<td>P = V × I</td>" +  
    "<td>" + hasil + " Watt</td>";  
  
tabel.appendChild(row);  
  
});
```

2. JAWABAN KODE HTML

```
<!DOCTYPE html>  
<html lang="id">  
  
<head>  
    <meta charset="UTF-8">  
    <meta name="viewport" content="width=device-width, initial-scale=1.0">  
    <title>Pertemuan 5 - Function Energi</title>  
  
    <link  
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"  
    " rel="stylesheet">  
  
    <style>  
        body {  
            background-color: #f8f9fa;  
        }  
  
        .wrapper {  
            max-width: 700px;  
            margin: auto;  
        }  
  
        .rumus {  
            font-size: 18px;  
            font-weight: bold;  
            color: #0d6efd;
```



```

    }
  </style>
</head>

<body>

  <!-- HEADER -->
  <header class="bg-info text-light text-center py-4 mb-4 shadow">
    <h1 class="fw-bold mb-1">Langkah 1 - Kalkulator Energi Mesin</h1>
    <p class="mb-0">Maessa Andrea Vallenia</p>
    <p class="mb-0">23051430044</p>
  </header>

  <div class="container wrapper">

    <!-- RUMUS -->
    <div class="card shadow mb-3">
      <div class="card-body text-center">
        <h5 class="mb-2">Kalkulator Daya Mesin</h5>
        <p class="mb-1">Rumus yang digunakan</p>
        <div class="rumus mb-2">P = V × I</div>
      </div>
    </div>

    <!-- INPUT -->
    <div class="card shadow mb-3">
      <div class="card-body text-center">

        <div class="mb-2">
          <label>Tegangan (Volt)</label>
          <input type="number" id="tegangan" class="form-control"
value="220">
        </div>

        <div class="mb-2">
          <label>Arus (Ampere)</label>
          <input type="number" id="arus" class="form-control"
value="10">
        </div>

        <button id="btnHitung" class="btn btn-primary w-100 mt-
2">Hitung Energi</button>

        <div id="hasilOutput" class="alert alert-success mt-3 mb-0">
          Hasil muncul di sini
        </div>
      </div>
    </div>
  </div>

```

```
        </div>
    </div>

    <!-- TABEL -->
    <div class="card shadow">
        <div class="card-body">
            <h5 class="text-center mb-3">Rekap Perhitungan Energi</h5>

            <table class="table table-bordered text-center">
                <thead class="table-info">
                    <tr>
                        <th>Rumus</th>
                        <th>Hasil</th>
                    </tr>
                </thead>

                <tbody id="rekapTable">
                    <tr>
                        <td colspan="2">Belum ada data</td>
                    </tr>
                </tbody>
            </table>

        </div>
    </div>

</div>

<footer class="text-center mt-4 mb-2">
    <small>Praktikum JavaScript Pertemuan 5</small>
</footer>

<script src="praktikum_1.js"></script>

</body>

</html>
```

Screenshot Hasil:

The image displays two screenshots of a web application titled "Langkah 1 - Kalkulator Energi Mesin" by Maessa Andrea Vallenia, ID 23051430044. The application is a JavaScript-based calculator for machine power.

Top Screenshot (Initial State):

- Kalkulator Daya Mesin**
Rumus yang digunakan: $P = V \times I$
- Input fields: Tegangan (Volt) = 220, Arus (Ampere) = 10.
- Buttons: "Hitung Energi" (blue), "Hasil muncul di sini" (green).
- Rekap Perhitungan Energi**
Table with columns "Rumus" and "Hasil". The result is "Belum ada data".

Bottom Screenshot (After Calculation):

- The "Hitung Energi" button is clicked.
- The result is displayed: $P = V \times I = 220 \times 10 = 2200 \text{ Watt}$.
- The "Rekap Perhitungan Energi" table is updated:

Rumus	Hasil
$P = V \times I$	2200 Watt

Browser Console (Right Panel):

- Log messages show the calculation steps: "Daya Mesin: 2200 watt", "Live reload enabled.", "=== HITUNG DAYA ===", "Tegangan: 220", "Arus: 10", "Daya: 2200".
- Source code links are provided for each step.

Gambar E.1 – Hasil Web dan Console Membuat Function Perhitungan

D.2 Langkah 2 – Array dan Looping (Daftar Produk Cacat)

Menyimpan daftar kode produk yang cacat dalam satu shift.

```
/* ===== KODE / OUTPUT LANGKAH 2 ===== */
2. JAWABAN KODE JS

// 2. ARRAY
let daftarCacat = ["C-001", "C-005", "C-012", "C-001", "C-020"]; // C-001
terjadi 2 kali
console.log("Jumlah Cacat: " + daftarCacat.length);
// Looping (Perulangan) untuk menampilkan setiap cacat
console.log("--- Laporan Detail Cacat ---");
for (let i = 0; i < daftarCacat.length; i++) {
  console.log("Item ke-" + (i + 1) + " : " + daftarCacat[i]);
}
// Menambahkan data baru ke array
daftarCacat.push("C-099");
console.log("Setelah penambahan: " + daftarCacat);

// TAMBAHAN AGAR TAMPIL DI WEB

const btn = document.getElementById("btnTampil");
const tabel = document.getElementById("rekapTable");
const output = document.getElementById("hasilOutput");

btn.addEventListener("click", function () {

  // hapus tulisan awal
  if (tabel.children[0].children[0].colSpan) {
    tabel.innerHTML = "";
  }

  // tampil jumlah
  output.innerHTML =
    "Jumlah data cacat = " + daftarCacat.length;

  // looping tampil di tabel
  for (let i = 0; i < daftarCacat.length; i++) {

    let row = document.createElement("tr");

    row.innerHTML =
      "<td>" + (i + 1) + "</td>" +
      "<td>" + daftarCacat[i] + "</td>";

    tabel.appendChild(row);
  }
});
```

```
}
```

```
});
```

3. JAWABAN KODE HTML

```
<!DOCTYPE html>
<html lang="id">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Langkah 2 - Array dan Looping</title>

  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">

  <style>
    body {
      background-color: #f8f9fa;
    }

    .wrapper {
      max-width: 750px;
      margin: auto;
    }

    .kode {
      font-weight: bold;
      color: #0d6efd;
    }
  </style>
</head>

<body>

  <!-- HEADER -->
  <header class="bg-info text-light text-center py-4 mb-4 shadow">
    <h1 class="fw-bold mb-1">Praktikum 2 - Array dan Looping</h1>
    <p class="mb-0">Maessa Andrea Vallenia</p>
    <p class="mb-0">23051430044</p>
  </header>

  <div class="container wrapper">
```

```
<!-- CARD UTAMA -->
<div class="card shadow mb-3">
  <div class="card-body text-center">

    <h5 class="mb-2">Daftar Produk Cacat</h5>

    <div class="kode mb-2">
      ["C-001","C-005","C-012","C-001","C-020"]
    </div>

    <button id="btnTampil" class="btn btn-primary w-100 mt-2">
      Tampilkan Data
    </button>

    <div id="hasilOutput" class="alert alert-success mt-3 mb-0">
      Hasil muncul di sini
    </div>

  </div>
</div>

<!-- TABEL -->
<div class="card shadow">
  <div class="card-body">

    <h5 class="text-center mb-3">Rekap Data Cacat</h5>

    <table class="table table-bordered text-center">
      <thead class="table-info">
        <tr>
          <th>No</th>
          <th>Kode Cacat</th>
        </tr>
      </thead>

      <tbody id="rekapTable">
        <tr>
          <td colspan="2">Belum ada data</td>
        </tr>
      </tbody>
    </table>

  </div>
</div>
```

```

</div>

<!-- FOOTER -->
<footer class="text-center mt-4 mb-2">
  <small>Praktikum JavaScript Pertemuan 5</small>
</footer>

<script src="praktikum_2.js"></script>

</body>

</html>

```

Screenshot Hasil:

Praktikum 2 - Array dan Looping
Maessa Andrea Vallenia
23051430044

Daftar Produk Cacat
["C-001","C-005","C-012","C-001","C-020"]

Tampilkan Data

Hasil muncul di sini

Rekap Data Cacat

No	Kode Cacat
Belum ada data	

Praktikum JavaScript Pertemuan 5

Praktikum 2 - Array dan Looping
Maessa Andrea Vallenia
23051430044

Daftar Produk Cacat
["C-001","C-005","C-012","C-001","C-020"]

Tampilkan Data

Jumlah data cacat = 6

Rekap Data Cacat

No	Kode Cacat
1	C-001
2	C-005
3	C-012
4	C-001
5	C-020
6	C-099

Console Log:

```

Jumlah Cacat: 5
--- Laporan Detail Cacat ---
Item ke-1 : C-001
Item ke-2 : C-005
Item ke-3 : C-012
Item ke-4 : C-001
Item ke-5 : C-020
Setelah penambahan: C-001,C-005,C-012,C-001,C-020,C-099
Live reload enabled.

```

Gambar E.2 – Hasil Web dan Console Array dan Looping (Daftar Produk Cacat

D.3 Langkah 3 – Object (Rekaman Data Mesin)

Representasi data mesin yang lebih kompleks.

```
/* ===== KODE / OUTPUT LANGKAH 3 ===== */
2. JAWABAN KODE JS

// 3. OBJECT
let mesinA = {
  id: "M-01",
  nama: "Injection Molding 50 Ton",
  merk: "Toshiba",
  tahunBeli: 2018,
  status: "Running",
  spesifikasi: {
    kecepatanMax: 200, // mm/s
    tekananMax: 150 // bar
  }
};

// Mengakses data object
console.log("Nama Mesin: " + mesinA.nama);
console.log("Status: " + mesinA.status);
console.log("Tekanan Max: " + mesinA.spesifikasi.tekananMax + " bar");
// Mengubah data object (Simulasi mesin rusak)
mesinA.status = "Breakdown";
console.log("Status Baru: " + mesinA.status);

// TAMBAHAN UNTUK WEB

const btn = document.getElementById("btnTampil");
const output = document.getElementById("hasilOutput");

btn.addEventListener("click", function () {

  output.innerHTML =

    "Nama Mesin : " + mesinA.nama + "<br>" +
    "Merk : " + mesinA.merk + "<br>" +
    "Status : " + mesinA.status + "<br>" +
    "Tekanan Max : "
    + mesinA.spesifikasi.tekananMax
    + " bar";

});
```


3. JAWABAN KODE HTML

```
<!DOCTYPE html>
<html lang="id">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Langkah 3 - Object JavaScript</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">
  <style>
    body {
      background-color: #f8f9fa;
    }

    .card-custom {
      max-width: 650px;
      margin: auto;
    }
  </style>
</head>

<body>

  <!-- HEADER -->
  <header class="bg-info text-light text-center py-4 mb-4 shadow">
    <h1 class="fw-bold mb-1">Praktikum 3 - Object Data Mesin</h1>
    <p class="mb-0">Maessa Andrea Vallenia</p>
    <p class="mb-0">23051430044</p>
  </header>

  <div class="container">

    <!-- CARD DATA -->
    <div class="card shadow card-custom border-primary">
      <div class="card-body text-center">

        <h5 class="mb-2">Data Mesin Produksi</h5>
        <hr>

        <p>
          ID : <b>M-01</b><br>
          Nama : <b>Injection Molding 50 Ton</b><br>
          Merk : <b>Toshiba</b>
        </p>
      </div>
    </div>
  </div>
</body>
</html>
```

```
        Tahun : <b>2018</b>
    </p>

    <hr>

    <p>
        Kecepatan Max : <b>200 mm/s</b><br>
        Tekanan Max : <b>150 bar</b>
    </p>

    <button id="btnTampil" class="btn btn-primary w-100 mt-2">
        Tampilkan Data
    </button>

    <div id="hasilOutput" class="alert alert-success mt-3">
        Hasil muncul di sini
    </div>

</div>
</div>

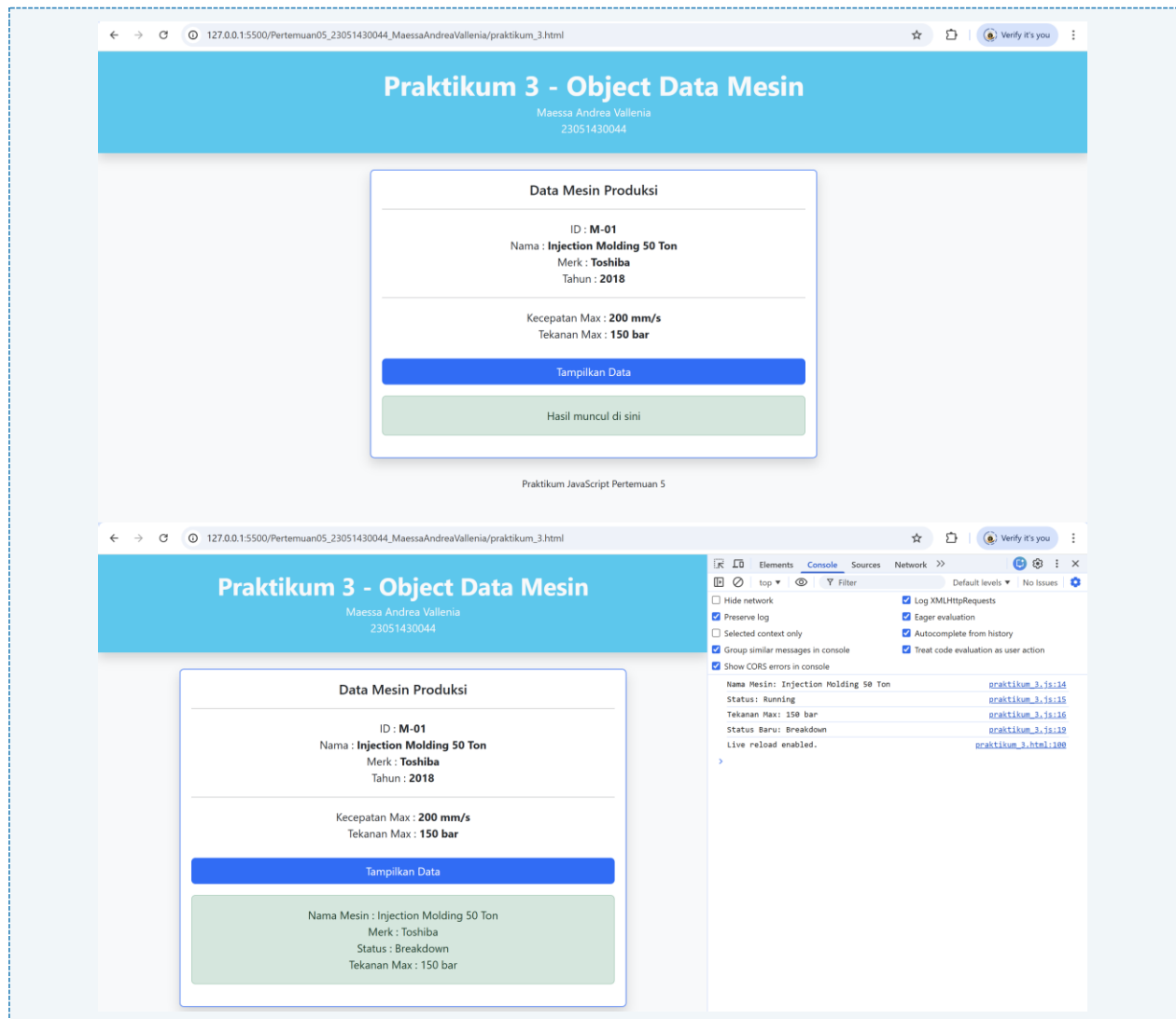
</div>

<footer class="text-center mt-4 mb-2">
    <small>Praktikum JavaScript Pertemuan 5</small>
</footer>

<script src="praktikum_3.js"></script>

</body>

</html>
```

Screenshot Hasil:

Gambar E.3 – Hasil Web dan Console Object (Rekaman Data Mesin)

D.4 Langkah 4 – Gabungan (Array of Objects)

Ini adalah cara data biasanya disimpan di aplikasi nyata (mirip JSON).

```

/* ===== KODE / OUTPUT LANGKAH 4 ===== */
1. JAWABAN KODE JS

// Array berisi beberapa object
let gudangMaterial = [
  { kode: "MAT-01", nama: "Baja Ringan", stok: 50, satuan: "Batang" },
  { kode: "MAT-02", nama: "Plastik ABS", stok: 120, satuan: "Kg" },
  { kode: "MAT-03", nama: "Oli Mesin", stok: 10, satuan: "Liter" }
];
console.log("--- Cek Stok Gudang ---");
// Menggunakan forEach untuk iterasi array object
gudangMaterial.forEach(function(item) {

```

```
console.log(item.kode + " - " + item.nama + " : " + item.stok + " " +
item.satuan);

// Logika Re-order
if (item.stok < 20) {
  console.log(" >>> PERINGATAN: Stok " + item.nama + " menipis! Segera PO.");
}
});

// Tambahan untuk web
const btn = document.getElementById("btnTampil");
const output = document.getElementById("hasilOutput");

btn.addEventListener("click", function () {

  let html = "<h5>Rekap Stok Gudang:</h5>";
  html += "<ul class='list-group'>";

  // Iterasi sama seperti di console
  gudangMaterial.forEach(function(item) {
    html += "<li class='list-group-item d-flex justify-content-between align-items-center'>";
    html += item.kode + " - " + item.nama + " : " + item.stok + " " +
item.satuan;

    // Peringatan stok menipis
    if (item.stok < 20) {
      html += " <span class='badge bg-danger'>Stok menipis!</span>";
    }

    html += "</li>";
  });

  html += "</ul>";
  output.innerHTML = html;
});
```

2. JAWABAN KODE HTML

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Langkah 4 - Array of Object</title>
```

```
<link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">
<style>
    .card-custom {
        max-width: 650px;
        margin: auto;
    }
</style>
</head>

<body class="bg-light">

<header class="bg-info text-light text-center py-5 mb-4 shadow">
    <h1 class="fw-bold">Praktikum 4 - Data Gudang (Array of Object)</h1>
    <h6 class="mb-0">Maessa Andrea Vallenia</h6>
    <p class="mb-0">23051430044</p>
</header>

<div class="container">

    <div class="card shadow card-custom border-primary">
        <div class="card-body text-center">

            <h4 class="mb-3">Cek Stok Gudang</h4>

            <hr>

            <button id="btnTampil" class="btn btn-primary mb-3 w-100">Tampilkan Rekap Stok</button>

            <div id="hasilOutput" class="alert alert-success text-start">
                Hasil rekap akan muncul di sini
            </div>

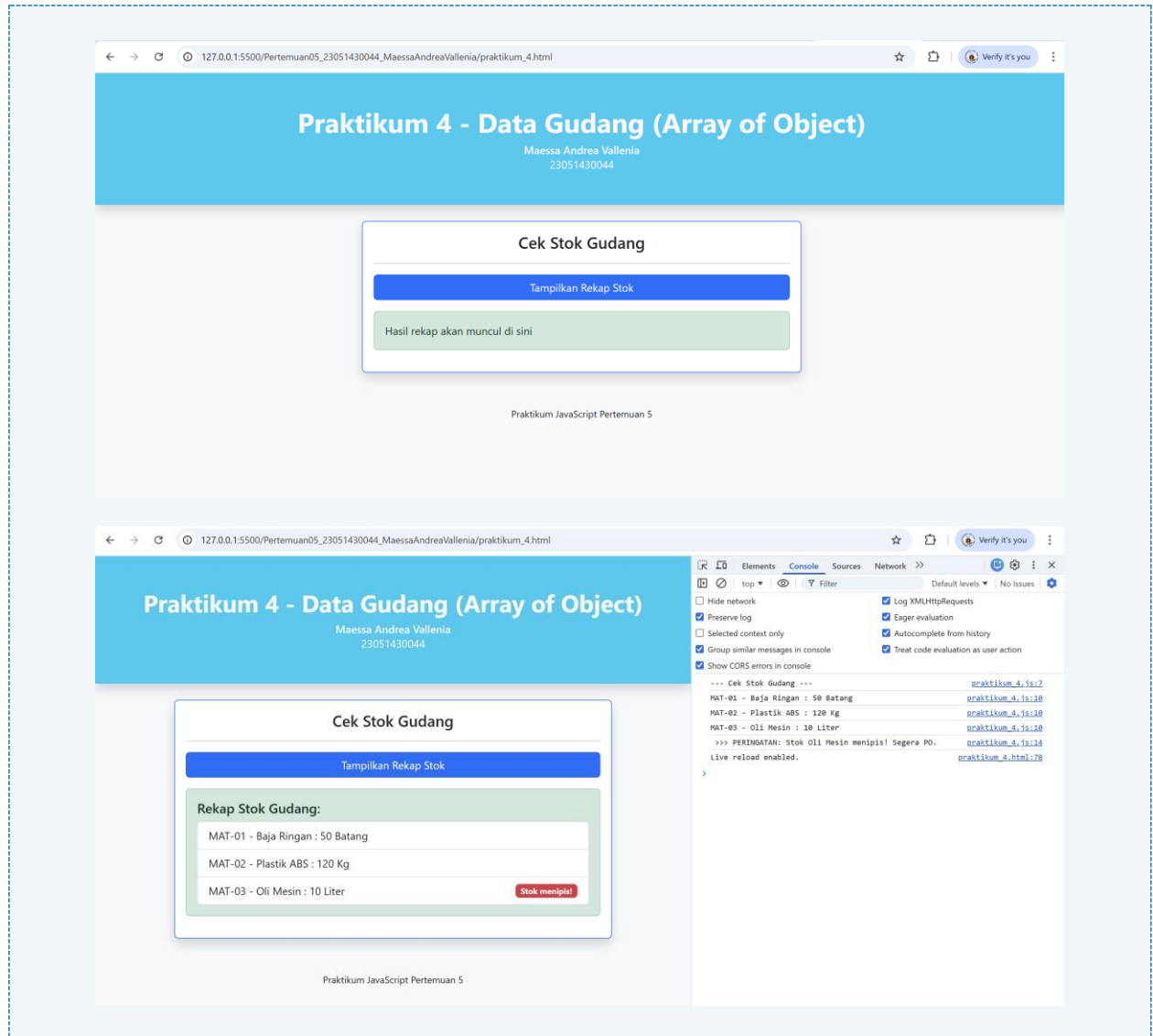
        </div>
    </div>

</div>

<footer class="text-center mt-5 mb-3">
    <small>Praktikum JavaScript Pertemuan 5</small>
</footer>

<script src="praktikum_4.js"></script>
```

```
</body>
</html>
```

Screenshot Hasil:

Gambar E.4 – Hasil Web dan Console Gabungan (Array of Objects)

E. LATIHAN DAN TUGAS MANDIRI

Jawab dan dokumentasikan seluruh latihan/tugas yang ada di bagian "Latihan/Tugas Mandiri" pada modul Anda.

E.1 Latihan 1 – Function

Pertanyaan / Instruksi Latihan:

Buat function `hitungLingkaran(jariJari)` yang mengembalikan nilai Luas dan Keliling. Tampilkan kedua nilai tersebut.

Jawaban / Kode Solusi (JS & HTML)

1. JAWABAN KODE CSS

```
body {
  background-color: #cde5fd; /* biru muda soft */
  font-family: Arial, sans-serif;
  min-height: 100vh;
}

.card-custom {
  max-width: 650px;
  margin: 1rem auto;
}

header {
  text-align: center;
  padding: 3rem 1rem;
  margin-bottom: 2rem;
  color: #ffffff;
  background-color: #0417f0b5;
  box-shadow: 0 0.2rem 0.5rem rgba(0,0,0,0.2);
}

header h1 { font-weight: bold; }
header h6, header p { margin-bottom: 0; }

.btn { font-weight: bold; }
.alert { text-align: left; }

input[type="number"], select { padding: 0.5rem; }

table th, table td { vertical-align: middle !important; }

.card h4, .card h5 { font-weight: bold; }
```

```
footer { text-align: center; margin-top: 3rem; margin-bottom: 1rem; font-size: 0.9rem; }
```

2. JAWABAN KODE JS

```
// =====  
// LATIHAN 1 - FUNCTION PERHITUNGAN  
// =====  
  
// Fungsi untuk menghitung Lingkaran  
function hitungLingkaran(r) {  
    // Rumus luas dan keliling lingkaran  
    let luas = Math.PI * r * r;  
    let keliling = 2 * Math.PI * r;  
    return {luas: luas.toFixed(2), keliling: keliling.toFixed(2)};  
}  
  
// Fungsi untuk menghitung Kubus  
function hitungKubus(s) {  
    // Rumus volume dan luas permukaan kubus  
    let volume = s ** 3;  
    let luasPermukaan = 6 * s * s;  
    return {volume, luasPermukaan};  
}  
  
// Fungsi untuk menghitung Balok  
function hitungBalok(p, l, t) {  
    // Rumus volume dan luas permukaan balok  
    let volume = p * l * t;  
    let luasPermukaan = 2 * (p*l + p*t + l*t);  
    return {volume, luasPermukaan};  
}  
  
// Ambil elemen DOM  
const jenisHitung = document.getElementById("jenisHitung"); // dropdown jenis  
    perhitungan  
const inputFields = document.getElementById("inputFields"); // div untuk input  
    dinamis  
const btnHitung = document.getElementById("btnHitung"); // tombol hitung  
const hasilOutput = document.getElementById("hasilOutput"); // div untuk  
    menampilkan hasil  
const rekapTable = document.getElementById("rekapTable"); // tabel rekap  
  
// =====  
// FUNCTION UNTUK MENAMPILKAN INPUT SESUAI PILIHAN  
// =====
```



```
function tampilkanInput() {
    const jenis = jenisHitung.value;
    let html = "";

    // Jika lingkaran
    if(jenis === "lingkaran"){
        html = `<label>Jari-jari (r) [cm]</label>
                <input type="number" id="jariJari" class="form-control mb-2"
value="7">`;
    }
    // Jika kubus
    else if(jenis === "kubus"){
        html = `<label>Sisi (s) [cm]</label>
                <input type="number" id="sisiKubus" class="form-control mb-2"
value="5">`;
    }
    // Jika balok
    else if(jenis === "balok"){
        html = `<label>Panjang (p) [cm]</label>
                <input type="number" id="panjangBalok" class="form-control mb-
2" value="10">
                <label>Lebar (l) [cm]</label>
                <input type="number" id="lebarBalok" class="form-control mb-2"
value="4">
                <label>Tinggi (t) [cm]</label>
                <input type="number" id="tinggiBalok" class="form-control mb-
2" value="3">`;
    }

    // Tampilkan input di div
    inputFields.innerHTML = html;
}

// Tampilkan input saat pertama kali load
tampilkanInput();

// Event listener jika dropdown diubah
jenisHitung.addEventListener("change", tampilkanInput);

// =====
// EVENT LISTENER UNTUK TOMBOL HITUNG
// =====
btnHitung.addEventListener("click", function(){
    const jenis = jenisHitung.value;
    let htmlOutput = "", rekapHtml = "";
```

```

// Perhitungan Lingkaran
if(jenis === "lingkaran"){
    const r = Number(document.getElementById("jariJari").value);
    const ling = hitungLingkaran(r);
    htmlOutput = `Luas = ${ling.luas} cm2, Keliling = ${ling.keliling}
cm`;
    rekapHtml = `<tr><td>Luas Lingkaran ( $\pi r^2$ )</td><td>${ling.luas}
cm2</td></tr>
                <tr><td>Keliling Lingkaran ( $2\pi r$ )</td><td>${ling.keliling}
cm</td></tr>`;

    console.log("Lingkaran | r:", r, "Luas:", ling.luas, "Keliling:",
ling.keliling);
}

// Perhitungan Kubus
else if(jenis === "kubus"){
    const s = Number(document.getElementById("sisiKubus").value);
    const kub = hitungKubus(s);
    htmlOutput = `Volume = ${kub.volume} cm3, Luas Permukaan =
${kub.luasPermukaan} cm2`;
    rekapHtml = `<tr><td>Volume Kubus ( $s^3$ )</td><td>${kub.volume}
cm3</td></tr>
                <tr><td>Luas Permukaan Kubus
( $6s^2$ )</td><td>${kub.luasPermukaan} cm2</td></tr>`;
    console.log("Kubus | s:", s, "Volume:", kub.volume, "Luas Permukaan:",
kub.luasPermukaan);
}

// Perhitungan Balok
else if(jenis === "balok"){
    const p = Number(document.getElementById("panjangBalok").value);
    const l = Number(document.getElementById("lebarBalok").value);
    const t = Number(document.getElementById("tinggiBalok").value);
    const bal = hitungBalok(p,l,t);
    htmlOutput = `Volume = ${bal.volume} cm3, Luas Permukaan =
${bal.luasPermukaan} cm2`;
    rekapHtml = `<tr><td>Volume Balok ( $p \times l \times t$ )</td><td>${bal.volume}
cm3</td></tr>
                <tr><td>Luas Permukaan Balok
( $2(p l + p t + l t)$ )</td><td>${bal.luasPermukaan} cm2</td></tr>`;
    console.log("Balok | p:", p, "l:", l, "t:", t, "Volume:", bal.volume, "Luas
Permukaan:", bal.luasPermukaan);
}

// Tampilkan hasil ke web
hasilOutput.innerHTML = htmlOutput;

```

```
// Update tabel rekap
rekapTable.innerHTML = rekapHtml;
});
```

3. JAWABAN KODE HTML

```
<!DOCTYPE html>
<html lang="id">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Latihan Function</title>

<!-- Bootstrap CSS -->
<link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">

<!-- CSS latihan -->
<link href="latihan_1.css" rel="stylesheet">
</head>
<body>

<!-- HEADER -->
<header>
  <!-- Judul latihan -->
  <h1>Latihan 1 - Function</h1>
  <!-- Nama mahasiswa -->
  <h6>Maessa Andrea Vallenia</h6>
  <!-- NIM mahasiswa -->
  <p>23051430044</p>
</header>

<div class="container py-3">

  <!-- CARD INPUT PERHITUNGAN -->
  <div class="card shadow card-custom border-primary mb-4 p-3">
    <div class="card-body text-center">
      <!-- Judul bagian perhitungan -->
      <h4 class="mb-3">Pilih Jenis Perhitungan</h4>

      <!-- Dropdown untuk memilih jenis perhitungan -->
      <select id="jenisHitung" class="form-select mb-3 w-50 mx-auto">
        <option value="lingkaran">Lingkaran</option>
        <option value="kubus">Kubus</option>
```

```

        <option value="balok">Balok</option>
    </select>

    <!-- Tempat inputan dinamis sesuai pilihan -->
    <div id="inputFields" class="mb-3"></div>

    <!-- Tombol untuk melakukan perhitungan -->
    <button id="btnHitung" class="btn btn-primary w-
50">Hitung</button>

    <!-- Div untuk menampilkan hasil perhitungan -->
    <div id="hasilOutput" class="alert alert-success mt-3">Hasil akan
muncul di sini</div>
    </div>
</div>

<!-- CARD REKAP PERHITUNGAN -->
<div class="card shadow card-custom p-3">
    <div class="card-body">
        <!-- Judul bagian rekap -->
        <h4 class="text-center mb-3">Rekap Perhitungan</h4>

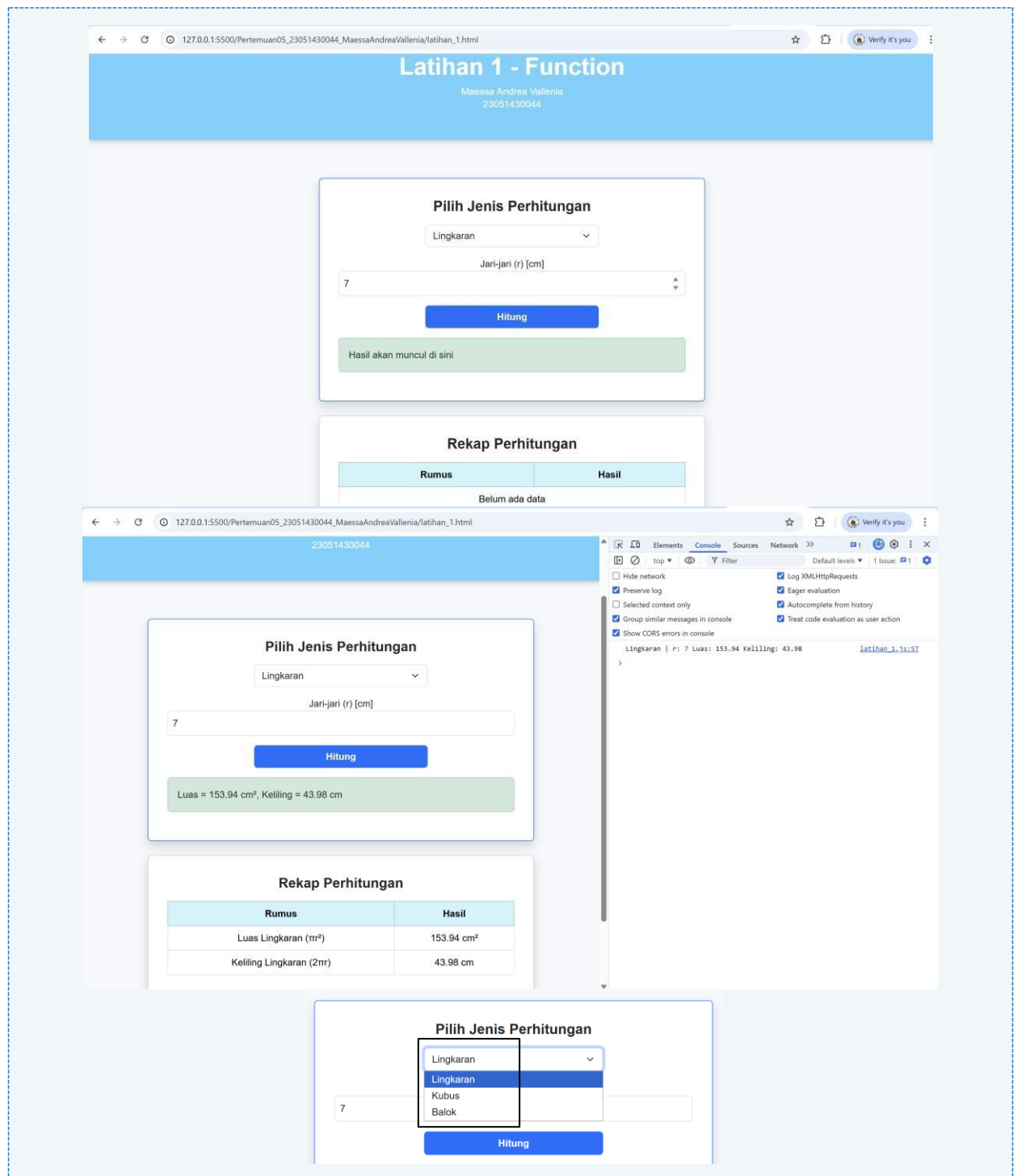
        <!-- Tabel untuk menampilkan riwayat perhitungan -->
        <table class="table table-bordered text-center">
            <thead class="table-info">
                <tr><th>Rumus</th><th>Hasil</th></tr>
            </thead>
            <tbody id="rekapTable">
                <!-- Default teks jika belum ada perhitungan -->
                <tr><td colspan="2">Belum ada data</td></tr>
            </tbody>
        </table>
    </div>
</div>

</div>

<!-- FOOTER -->
<footer class="text-center mt-5 mb-3">
<small>Praktikum JavaScript</small>
</footer>

<!-- Script JS -->
<script src="latihan_1.js"></script>
</body>
</html>

```



Gambar F.1 – Hasil Web & Console Function

E.2 Latihan 2 – Array Manipulation

Pertanyaan / Instruksi Latihan:

Dari array `daftarCacat` di atas, carilah berapa kali cacat "C-001" muncul (Gunakan loop dan if/counter).

Jawaban / Kode Solusi:

1. JAWABAN KODE CSS

```
body {
  background-color: #f5f8fc; /* biru muda soft */
  font-family: Arial, sans-serif;
  min-height: 100vh;
}

.card-custom {
  max-width: 650px;
  margin: 1rem auto;
}

header {
  text-align: center;
  padding: 3rem 1rem;
  margin-bottom: 2rem;
  color: #ffffff;
  background-color: #00c3ffb5;
  box-shadow: 0 0.2rem 0.5rem rgba(0,0,0,0.2);
}

header h1 { font-weight: bold; }
header h6, header p { margin-bottom: 0; }

.btn { font-weight: bold; }
.alert { text-align: left; }

input[type="number"], select { padding: 0.5rem; }

table th, table td { vertical-align: middle !important; }

.card h4, .card h5 { font-weight: bold; }

footer { text-align: center; margin-top: 3rem; margin-bottom: 1rem; font-size: 0.9rem; }
```

2. JAWABAN KODE JS

```
// Array data cacat produksi
let daftarCacat = ["C-001", "C-005", "C-012", "C-001", "C-020"];

// Ambil elemen DOM
const btnTampil = document.getElementById("btnTampil");
const hasilOutput = document.getElementById("hasilOutput");
const rekapTable = document.getElementById("rekapTable");

btnTampil.addEventListener("click", function() {
  // Variabel penghitung
  let jumlahC001 = 0;

  // Loop untuk mengecek jumlah C-001
  for (let i = 0; i < daftarCacat.length; i++) {
    if (daftarCacat[i] === "C-001") {
      jumlahC001++;
    }
  }

  // Output ke web
  hasilOutput.innerHTML = `Daftar Cacat: [${daftarCacat.join(", ")}]  
Jumlah "C-001" muncul: ${jumlahC001}`;

  // Update rekap tabel
  rekapTable.innerHTML = `
    <tr><td>Daftar Cacat</td><td>[${daftarCacat.join(", ")}]</td></tr>
    <tr><td>Jumlah C-001</td><td>${jumlahC001}</td></tr>
  `;

  // Output ke console
  console.log("=== CEK CACAT ===");
  console.log("Data:", daftarCacat);
  console.log("Jumlah C-001:", jumlahC001);
});
```

3. JAWABAN KODE HTML

```
<!DOCTYPE html>
<html lang="id">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Latihan 2 - Array Manipulation</title>
```

```

<!-- Bootstrap CSS -->
<link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">
<!-- CSS khusus latihan 2 -->
<link href="latihan_2.css" rel="stylesheet">
</head>
<body>

<!-- HEADER -->
<header>
    <h1>Latihan 2 - Array Manipulation</h1>
    <h6>Maessa Andrea Vallenia</h6>
    <p>23051430044</p>
</header>

<div class="container py-3">

    <!-- CARD UTAMA: Pengecekan produk cacat -->
    <div class="card shadow card-custom border-primary mb-4 p-3">
        <div class="card-body text-center">
            <h4 class="mb-3">Pengecekan Produk Cacat</h4>
            <!-- Informasi array cacat -->
            <p>Daftar cacat: C-001, C-005, C-012, C-001, C-020</p>
            <p>Program menghitung jumlah "C-001"</p>
            <!-- Tombol tampilkan hasil -->
            <button id="btnTampil" class="btn btn-primary w-50">Tampilkan
Hasil</button>
            <!-- Div untuk menampilkan hasil perhitungan -->
            <div id="hasilOutput" class="alert alert-success mt-3">Hasil akan
muncul di sini</div>
        </div>
    </div>

    <!-- CARD REKAP: tabel rekap perhitungan -->
    <div class="card shadow card-custom p-3">
        <div class="card-body">
            <h4 class="text-center mb-3">Rekap Perhitungan</h4>
            <table class="table table-bordered text-center">
                <thead class="table-info">
                    <tr>
                        <th>Keterangan</th>
                        <th>Hasil</th>
                    </tr>
                </thead>
                <tbody id="rekapTable">

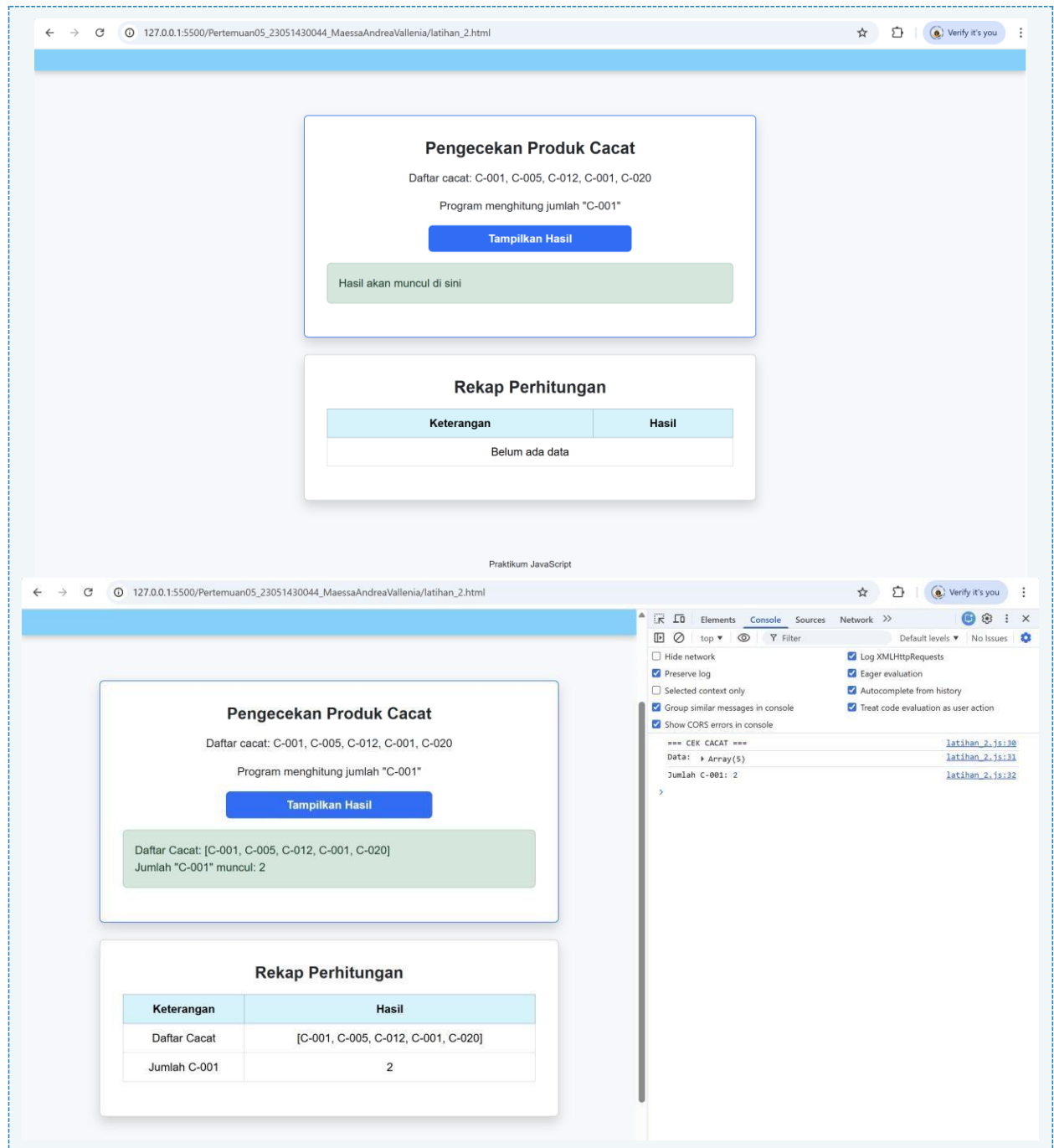
```



```
        <!-- Default jika belum ada perhitungan -->
        <tr><td colspan="2">Belum ada data</td></tr>
    </tbody>
</table>
</div>
</div>

<!-- FOOTER -->
<footer class="text-center mt-5 mb-3">
<small>Praktikum JavaScript</small>
</footer>

<!-- Script JS -->
<script src="latihan_2.js"></script>
</body>
</html>
```



Gambar F.2 – Hasil Web & Console Array Manipulation

E.3 Tugas Proyek Mini – Kalkulator Biaya Produksi

Deskripsi proyek mini:

Buat program simulasi antrian 3 pekerjaan mesin.

- Buat Array of Objects bernama `antrianMesin`. Isi dengan 3 object job: `{idJob: "J01", namaProses: "Drilling", durasi: 30}`.
- Buat function `prosesAntrian(antrian)` yang menerima parameter array tersebut.
- Di dalam function, tampilkan pesan: "Memproses Job [ID] - [Nama] selama [durasi] menit" untuk setiap job dalam antrian.
- Tambahkan 1 job baru ke array dan panggil function lagi.

Penjelasan pendekatan/solusi yang digunakan:

Proyek mini ini dibuat untuk mensimulasikan antrian pekerjaan mesin, di mana setiap job memiliki ID, nama proses, dan durasi tertentu. Program menampilkan urutan pemrosesan job di halaman web dan juga mencatat hasilnya di console browser. Tujuannya agar pengguna bisa melihat proses antrian secara interaktif sekaligus mendapatkan rekap job yang telah diproses. Program ini dibuat menggunakan HTML untuk tampilan, CSS untuk styling, dan JavaScript untuk logika pemrosesan antrian.

- Memulai proyek dengan memahami kebutuhan program, yaitu memproses antrian mesin dengan menampilkan pesan pemrosesan setiap job dan rekap tabel job.
- Membuat folder proyek di komputer, misalnya ProyekMini_SistemAntrianJobShop, dan membuat file HTML, JS, dan CSS di dalamnya.
- Menyiapkan struktur HTML yang berisi header dengan judul proyek dan identitas mahasiswa, container untuk tombol proses, area output untuk pesan job, dan tabel rekap.
- Membuat tombol "Proses Antrian" untuk memicu pemrosesan, area output untuk menampilkan pesan proses setiap job, dan tabel untuk menampilkan rekap job beserta durasinya.
- Menyiapkan array of objects berisi job awal (ID, nama proses, durasi), lalu menambahkan satu job baru sebelum memulai pemrosesan antrian.
- Membuat function `prosesAntrian` yang memproses setiap job secara berurutan, menampilkan pesan di halaman web dan console browser, serta memperbarui tabel rekap.
- Menambahkan event listener pada tombol sehingga ketika diklik, job baru ditambahkan dan function `prosesAntrian` dijalankan.
- Memastikan setiap durasi job valid (lebih dari 0); jika ada input tidak valid, menampilkan peringatan agar pengguna mengetahui kesalahan input.
- Menampilkan semua pesan proses di halaman web dan console browser, serta memperbarui tabel rekap dengan seluruh job yang telah diproses.

Jawaban / Kode Solusi:

1. JAWABAN KODE CSS

```
body {  
  background-color: #f5f8fc; /* biru muda soft */  
  font-family: Arial, sans-serif;
```

```
        min-height: 100vh;
    }

    .card-custom {
        max-width: 650px;
        margin: 1rem auto;
    }

    header {
        text-align: center;
        padding: 3rem 1rem;
        margin-bottom: 2rem;
        color: #ffffff;
        background-color: #00c3ffb5;
        box-shadow: 0 0.2rem 0.5rem rgba(0,0,0,0.2);
    }

    header h1 { font-weight: bold; }
    header h6, header p { margin-bottom: 0; }

    .btn { font-weight: bold; }

    .alert { text-align: left; }

    table th, table td { vertical-align: middle !important; }

    .card h4, .card h5 { font-weight: bold; }

    footer { text-align: center; margin-top: 3rem; margin-bottom: 1rem; font-size: 0.9rem; }

    .container { padding-left: 1rem; padding-right: 1rem; }
```

2. JAWABAN KODE JS

```
document.addEventListener("DOMContentLoaded", function() {

    // Array awal 3 job
    let antrianMesin = [
        {idJob: "J01", namaProses: "Drilling", durasi: 30},
        {idJob: "J02", namaProses: "Milling", durasi: 45},
        {idJob: "J03", namaProses: "Grinding", durasi: 25}
    ];

    const btnProses = document.getElementById("btnProses");
    const hasilOutput = document.getElementById("hasilOutput");
```

```

const rekapTable = document.getElementById("rekapTable");

// Function untuk proses antrian
function prosesAntrian(antrian){
    let outputHtml = "";
    antrian.forEach(job => {
        let pesan = `Memproses Job ${job.idJob} - ${job.namaProses}
selama ${job.durasi} menit`;
        outputHtml += pesan + "<br>";
        console.log(pesan);
    });
    hasilOutput.innerHTML = outputHtml;

    // update tabel rekap
    let tableHtml = "";
    antrian.forEach(job => {
        tableHtml +=
`<tr><td>${job.idJob}</td><td>${job.namaProses}</td><td>${job.durasi}</td></tr>`;
    });
    rekapTable.innerHTML = tableHtml;
}

// Event tombol
btnProses.addEventListener("click", function(){
    // Proses antrian awal
    prosesAntrian(antrianMesin);

    // Tambahkan 1 job baru
    const jobBaru = {idJob: "J04", namaProses: "Polishing", durasi: 20};
    antrianMesin.push(jobBaru);

    console.log("Job baru ditambahkan:", jobBaru.idJob, "-",
jobBaru.namaProses);
    alert("Job baru ditambahkan: " + jobBaru.idJob + " - " +
jobBaru.namaProses);

    // Proses ulang semua job (termasuk job baru)
    prosesAntrian(antrianMesin);
});

});

3. JAWABAN KODE HTML

<!DOCTYPE html>

```

```

<html lang="id">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>ProyekMini_(SistemAntrianJobShop)</title>

<link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">
<link href="ProyekMini_SistemAntrianJobShop.css" rel="stylesheet">
</head>
<body>

<header>
  <h1>ProyekMini - Sistem Antrian Job Shop</h1>
  <h6>Maessa Andrea Vallenia</h6>
  <p>23051430044</p>
</header>

<div class="container py-3">

  <!-- CARD UTAMA: Tombol proses -->
  <div class="card shadow card-custom border-primary mb-4 p-3">
    <div class="card-body text-center">
      <h4 class="mb-3">Simulasi Antrian Mesin</h4>
      <button id="btnProses" class="btn btn-primary w-50">Proses
Antrian</button>
      <div id="hasilOutput" class="alert alert-success mt-3">Hasil akan
muncul di sini</div>
    </div>
  </div>

  <!-- CARD REKAP: tabel antrian -->
  <div class="card shadow card-custom p-3">
    <div class="card-body">
      <h4 class="text-center mb-3">Rekap Antrian Mesin</h4>
      <table class="table table-bordered text-center">
        <thead class="table-info">
          <tr><th>ID Job</th><th>Nama Proses</th><th>Durasi
(menit)</th></tr>
        </thead>
        <tbody id="rekapTable">
          <tr><td colspan="3">Belum ada data</td></tr>
        </tbody>
      </table>
    </div>
  </div>

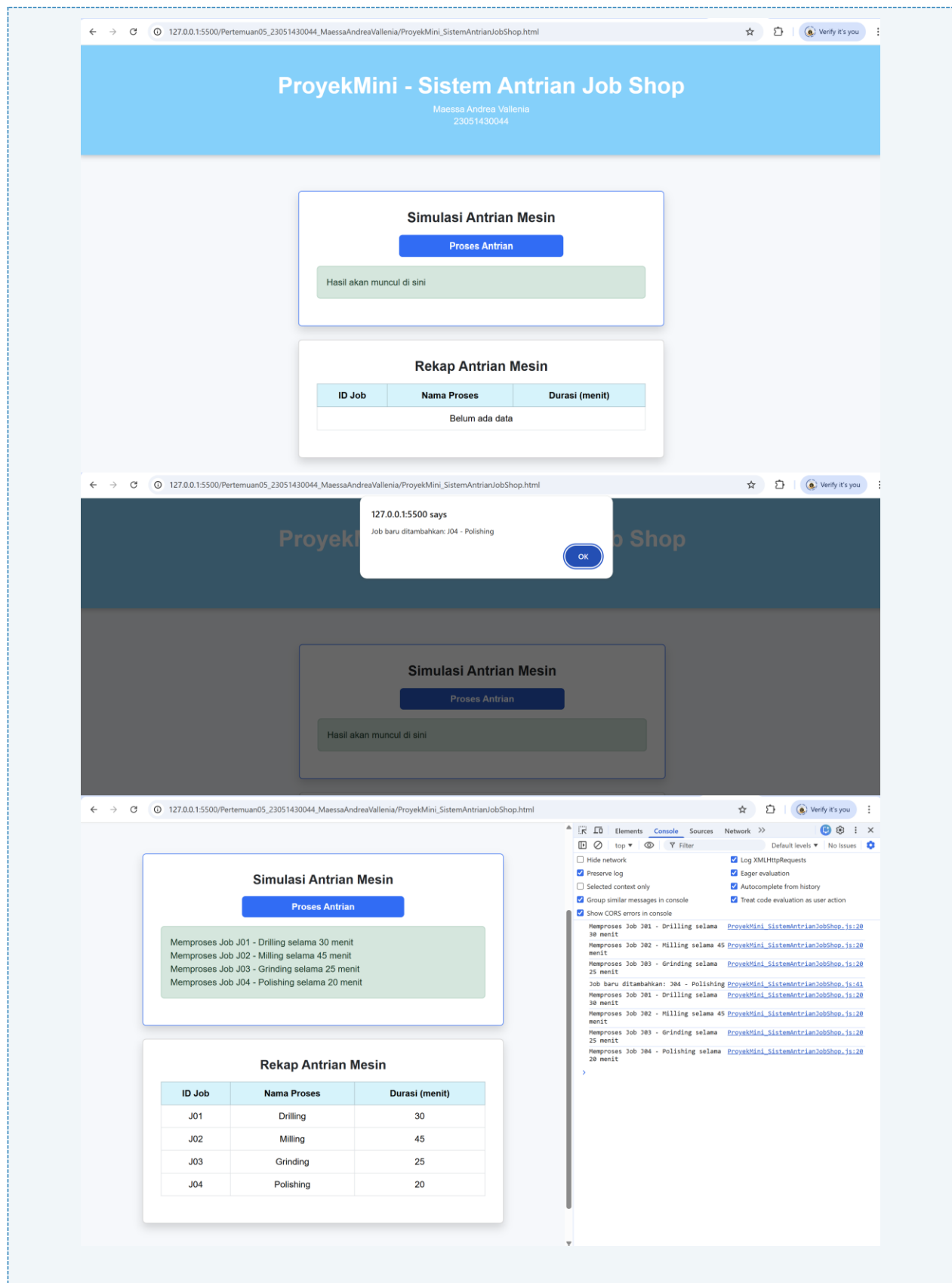
```

```
</div>

</div>

<footer class="text-center mt-5 mb-3">
<small>Praktikum JavaScript</small>
</footer>

<script src="ProyekMini_SistemAntrianJobShop.js"></script>
</body>
</html>
```



Gambar F.3 – Hasil Proyek Mini Kalkulator Biaya Produksi

F. PEMBAHASAN

Pada praktikum ini, mahasiswa mempelajari integrasi HTML, CSS/Bootstrap, dan JavaScript untuk membangun halaman web interaktif yang mensimulasikan sistem antrian pekerjaan mesin. Praktikum menekankan pemahaman alur pemrosesan data mulai dari input, logika pemrograman, hingga output yang ditampilkan secara visual maupun tercatat di console browser. HTML digunakan untuk membangun kerangka halaman, termasuk header, tombol aksi, area pesan, dan tabel rekap, sedangkan Bootstrap dan CSS memastikan tampilan responsif, rapi, dan mudah dibaca, yang mencerminkan prinsip user interface pada aplikasi industri.

JavaScript menjadi inti logika dalam praktik ini. Dengan memanfaatkan function, program mampu memproses array of objects berisi job secara modular dan terstruktur, sehingga setiap job dapat diakses, dihitung durasinya, dan ditampilkan secara berurutan tanpa pengulangan kode. Looping digunakan untuk iterasi data, sedangkan percabangan if/else memastikan validitas input, menjaga konsistensi hasil, dan meminimalkan kesalahan pemrosesan. Penggunaan console.log() membantu dalam pengecekan internal dan debugging, memberikan feedback yang sejalan dengan praktik pengembangan sistem informasi industri.

Secara konseptual, praktikum ini tidak hanya mengajarkan sintaks JavaScript, tetapi juga prinsip modularitas kode, pengelolaan data kompleks melalui array of objects, dan hubungan antara proses logika dengan output visual. Pendekatan ini merepresentasikan simulasi nyata dalam dunia industri, seperti penjadwalan mesin, monitoring antrian produksi, dan rekap performa, sehingga mahasiswa dapat memahami penerapan teknologi web untuk mendukung efisiensi dan kontrol proses produksi secara terstruktur dan sistematis.

F.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum menunjukkan bahwa implementasi program telah sesuai dengan teori HTML, CSS/Bootstrap, dan JavaScript. Struktur halaman yang dibuat menggunakan HTML mampu menampilkan komponen inti seperti header, form input biaya produksi, tombol aksi, dan tabel rekap dengan tepat, sesuai prinsip pengembangan antarmuka web yang terstruktur. Penggunaan Bootstrap dan CSS memastikan tampilan konsisten, rapi, dan responsif, sehingga interaksi pengguna menjadi intuitif dan data mudah dibaca, selaras dengan teori desain web modern.

Di sisi JavaScript, perhitungan biaya produksi per unit dilakukan dengan benar menggunakan variabel dan operator aritmatika, mengikuti rumus yang telah ditentukan. Penggunaan percabangan if/else untuk menentukan status produksi (Biaya Tinggi atau Biaya Efisien) menunjukkan penerapan logika kondisional sesuai teori pemrograman. Output perhitungan ditampilkan secara real-time di halaman web, dicatat dalam tabel rekap, dan direkam melalui `console.log()` untuk verifikasi proses, yang memperlihatkan keterkaitan teori dengan praktik debugging dan monitoring program.

Secara keseluruhan, hasil praktikum konsisten dengan teori karena program dapat:

- Menerima input data dengan valid dan sesuai format
- Memproses perhitungan dengan logika yang benar
- Memberikan output visual dan tabel rekap yang akurat
- Memfasilitasi pengecekan hasil melalui console browser

Hal ini menegaskan bahwa prinsip dasar pengembangan web dan logika JavaScript, termasuk modularitas kode, penggunaan fungsi, array, dan objek, telah diterapkan dengan baik. Implementasi ini memberikan gambaran nyata mengenai pengolahan data produksi di sistem industri, menunjukkan efisiensi dan keandalan program dalam simulasi perhitungan biaya produksi.

F.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Looping atau iterasi array tidak menampilkan seluruh data.	Memastikan looping berada di dalam function atau event listener yang tepat, serta output HTML diperbarui setiap kali function dipanggil. Hal ini membuat seluruh data dalam array atau array of object dapat ditampilkan secara lengkap dan rapi di tabel rekap.

F.3 Kaitan dengan Penerapan di Industri

Materi praktikum Pertemuan 5 berkaitan erat dengan penerapan di industri, terutama dalam pembuatan sistem pencatatan dan perhitungan biaya produksi berbasis web. Struktur HTML yang dikombinasikan dengan Bootstrap memungkinkan pembuatan form input data produksi, tabel rekap, dan dashboard yang rapi, responsif, dan mudah dibaca. Di industri, konsep ini mirip dengan sistem monitoring produksi yang menampilkan data harian secara real-time sehingga manajer dapat memantau status produksi dengan cepat.

Penggunaan JavaScript dalam praktikum mencerminkan proses pengolahan data produksi di industri. Function digunakan untuk modularisasi perhitungan, sedangkan array dan array of object dapat menyimpan data produk beserta biaya, jumlah produksi, dan statusnya, mirip dengan database produksi perusahaan. Percabangan if/else digunakan untuk menentukan status produksi, apakah Biaya Tinggi atau Biaya Efisien, sehingga memberikan analisis sederhana terhadap efisiensi produksi.

Hasil perhitungan yang ditampilkan di halaman web dan dicatat di `console.log()` memberikan gambaran bagaimana data produksi dapat dimonitor dan diaudit secara cepat. Praktikum ini menunjukkan hubungan antara input data, proses perhitungan, dan output, yang merupakan konsep dasar dalam sistem informasi produksi di industri. Meskipun masih sederhana, materi ini menjadi dasar untuk mengembangkan sistem pengolahan data produksi yang lebih kompleks dan otomatis.

G. KESIMPULAN

Tuliskan 3–5 poin kesimpulan yang merangkum hal-hal yang dipelajari dan dicapai pada praktikum modul ini. Kesimpulan harus spesifik, bukan bersifat umum.

1	Mahasiswa memahami konsep modularitas kode dengan function, sehingga perhitungan atau proses dapat digunakan berulang tanpa menulis kode yang sama.
2	Mahasiswa mampu menggunakan array dan looping untuk menyimpan dan memproses sekumpulan data, seperti daftar produk cacat atau stok material, secara otomatis dan efisien.
3	Mahasiswa mengerti penggunaan object dan nested object untuk merepresentasikan entitas kompleks, seperti data mesin dengan berbagai properti, sehingga informasi tersimpan secara terstruktur.
4	Mahasiswa dapat menggabungkan array of object untuk mengelola data yang lebih kompleks dan melakukan logika tambahan, seperti pengecekan stok menipis, yang mencerminkan sistem informasi nyata di industri.
5	Praktikum ini menunjukkan integrasi HTML, Bootstrap, dan JavaScript sehingga mahasiswa dapat membuat halaman web interaktif yang menampilkan input data, hasil perhitungan, dan rekap tabel, memberikan gambaran nyata penerapan sistem pengolahan data di Teknik Industri.

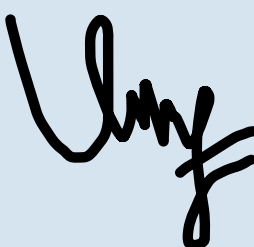
H. DAFTAR PUSTAKA

Tuliskan semua sumber referensi yang digunakan dalam format APA 7th Edition. Minimal 3 referensi yang relevan (termasuk Manual Praktikum dan sumber online resmi seperti MDN, Reactjs.org, dll).

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Kom, I. L. M. (2023). <i>Dasar Algoritma dan Pemograman Javascript</i> . Irvan Lewenusa, M. Kom.
[3]	Sholikhan, M. (2022). Html, Css Dan Javascript. <i>Penerbit Yayasan Prima Agus Teknik</i> , 1-343 .
[4]	https://www.javascripttutorial.net/
[5]	https://developer.mozilla.org/en-US/docs/Web/JavaScript
[6]	https://www.geeksforgeeks.org/

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa  Maessa Andrea Vallenia NIM: 23051430044	Diperiksa oleh Asisten / Dosen Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____
--	---

MODUL 6

DOM MANIPULATION & EVENT HANDLING

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 6 (Pertemuan 6)
Topik Utama	Document Object Model (DOM), Seleksi Elemen, dan Event Listeners.
Sub-Topik	Konsep Document Object Model (DOM), Struktur dan Hierarki DOM, Seleksi Elemen DOM (getElementById, querySelector), Manipulasi Elemen HTML (teks, style, class), Event dan Event Listener (click, input, change), Event Handling dan Interaksi Pengguna, Perubahan Tampilan Dinamis (UI Update), Penggunaan Timer (setInterval, clearInterval), Validasi Input Real-time, Implementasi Simulasi Panel Kontrol Mesin Berbasis Web.
Durasi Praktikum	340 Menit
Tanggal Praktikum	27 Maret 2026
Nama Mahasiswa	Maessa Andrea Vallenia
NIM	23051430044
Kelas	A

A. TUJUAN PEMBELAJARAN

Tuliskan tujuan pembelajaran yang ingin dicapai pada modul ini, sesuai dengan yang tertera di Manual Praktikum.

1	Memahami struktur DOM dan bagaimana JavaScript melihat halaman web.
2	Mampu memanipulasi elemen HTML (mengubah teks, warna, kelas) secara dinamis menggunakan JavaScript.
3	Mampu menangani interaksi pengguna (klik tombol, ketik keyboard) menggunakan Event Listeners.
4	Menerapkan konsep ini untuk membuat simulasi Control Panel mesin industri.

B. ALAT DAN BAHAN

Sebutkan seluruh perangkat keras, perangkat lunak, dan sumber referensi yang digunakan selama praktikum.

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	<i>Code Editor</i>
2	Google Chrome / Browser Modern	Versi terbaru	<i>Testing & Debugging</i>
3	Live Server	Versi 5.7.9	Menjalankan HTML dan memantau perubahan file
4	JavaScript (Built-in Browser)	ES6 (Default Browser Modern)	Bahasa pemrograman untuk logika dan perhitungan
5	CodeSnap	Versi 1.3.4	Dokumentasi kode program
6	Git dan GitHub	Versi 2.53.0	Version control system (VCS) berbasis Git
7	Koneksi Internet	Stabil	Mengakses Library dan Framework
8	Console Browser (Developer Tools)	Default Browser	Melihat output program dan debugging
9	Bootstrap	Versi 5.3	Membantu pembuatan UI seperti button, card, dan alert pada panel control
10	File praktikum	Perrtemuan sebelumnya	Digunakan sebagai dasar pengembangan panel kontrol

C. DASAR TEORI

1. JavaScript merupakan bahasa pemrograman yang digunakan untuk menambahkan logika dan interaksi pada halaman web. Dalam pengembangan web, JavaScript berperan sebagai pengendali perilaku (behavior) dari suatu sistem.
 - a. JavaScript bekerja bersama HTML dan CSS, di mana:
 - b. HTML berfungsi sebagai struktur atau kerangka halaman web
 - c. CSS digunakan untuk mengatur tampilan atau desain
 - d. JavaScript mengatur interaksi dan logika sistemDengan JavaScript, halaman web dapat menjadi dinamis, seperti merespon input pengguna, menampilkan data secara real-time, serta mengontrol elemen pada halaman.
2. JavaScript merupakan bahasa pemrograman yang digunakan untuk menambahkan logika dan interaksi pada halaman web. Dalam pengembangan web, JavaScript berperan sebagai pengendali perilaku (behavior) dari suatu sistem.
 - a. JavaScript bekerja bersama HTML dan CSS, di mana:
 - b. HTML berfungsi sebagai struktur atau kerangka halaman web
 - c. CSS digunakan untuk mengatur tampilan atau desain
 - d. JavaScript mengatur interaksi dan logika sistemDengan JavaScript, halaman web dapat menjadi dinamis, seperti merespon input pengguna, menampilkan data secara real-time, serta mengontrol elemen pada halaman.
3. Seleksi elemen DOM adalah proses mengambil elemen HTML agar dapat dimanipulasi menggunakan JavaScript.

Beberapa metode yang umum digunakan:

 - a. `getElementById()` → mengambil elemen berdasarkan ID
 - b. `querySelector()` → mengambil elemen pertama berdasarkan selector CSS
 - c. `querySelectorAll()` → mengambil semua elemen yang sesuai selectorSeleksi elemen merupakan langkah awal sebelum melakukan manipulasi atau interaksi terhadap elemen tersebut.
4. Manipulasi DOM adalah proses mengubah isi, tampilan, atau atribut elemen HTML menggunakan JavaScript.

Beberapa bentuk manipulasi yang umum:

 - a. Mengubah teks → `innerText`, `innerHTML`
 - b. Mengubah style → `style.color`, `style.backgroundColor`
 - c. Mengubah class → `classList.add()`, `classList.remove()`Manipulasi DOM memungkinkan perubahan tampilan (User Interface/UI) secara langsung tanpa perlu me-reload halaman, sehingga meningkatkan interaktivitas aplikasi web
5. Event adalah kejadian yang terjadi pada elemen di halaman web, baik karena aksi pengguna maupun sistem. Contoh event antara lain klik (click), input, perubahan nilai (change), dan pengiriman form (submit).

Untuk menangani event tersebut digunakan event listener, yaitu mekanisme untuk “mendengarkan” suatu event dan menjalankan perintah tertentu sebagai respon.

Fungsi event listener adalah menghubungkan aksi pengguna dengan logika program sehingga halaman web dapat merespon secara langsung. Dengan adanya event listener, interaksi antara pengguna dan sistem menjadi lebih dinamis dan interaktif.

6. Event handling adalah proses menangani aksi yang dilakukan oleh pengguna dan memberikan respon sesuai dengan perintah yang telah diprogram menggunakan JavaScript.

Contoh event handling dalam aplikasi web antara lain:

- a. Klik tombol seperti Start, Stop, atau Reset
- b. Mengisi data pada form
- c. Memilih opsi pada dropdown

Melalui event handling, sistem dapat memberikan feedback secara langsung, mengontrol jalannya program, serta meningkatkan kenyamanan pengguna dalam menggunakan aplikasi.

7. Timer dalam JavaScript digunakan untuk menjalankan suatu proses secara berulang dalam interval waktu tertentu.

Fungsi yang umum digunakan adalah:

- a. `setInterval()`, untuk menjalankan perintah secara berulang
- b. `clearInterval()`, untuk menghentikan proses yang sedang berjalan

Timer biasanya digunakan dalam sistem yang membutuhkan pembaruan data secara berkala, seperti monitoring suhu mesin, simulasi proses, atau sistem otomatis lainnya

8. Validasi input adalah proses pengecekan data yang dimasukkan oleh pengguna agar sesuai dengan aturan yang telah ditentukan.

Contoh validasi meliputi:

- a. Input tidak boleh kosong
- b. Format email harus benar
- c. Nilai harus berupa angka
- d. Batas minimum atau maksimum tertentu

Tujuan validasi adalah untuk mencegah kesalahan input, menjaga kualitas data, serta meningkatkan keamanan sistem. Selain itu, validasi juga memberikan umpan balik kepada pengguna berupa pesan kesalahan jika data yang dimasukkan tidak sesuai.

C.1 Konsep Utama

Konsep utama pada materi ini berfokus pada penggunaan JavaScript dalam menciptakan interaksi pada halaman web melalui pengolahan DOM (Document Object Model), event, serta validasi input. Dengan memanfaatkan konsep tersebut, halaman web tidak hanya bersifat statis, tetapi dapat merespon aksi pengguna secara langsung dan dinamis.

DOM digunakan sebagai representasi struktur halaman web yang memungkinkan JavaScript untuk mengakses dan memanipulasi elemen HTML. Dengan DOM, perubahan

tampilan seperti teks, warna, maupun struktur elemen dapat dilakukan tanpa perlu memuat ulang halaman. Hal ini menjadi dasar dalam pengembangan antarmuka pengguna (user interface) yang interaktif.

Event dan event listener digunakan untuk mendeteksi aksi yang dilakukan oleh pengguna, seperti klik tombol, input data, atau perubahan nilai. Ketika event terjadi, JavaScript akan menjalankan fungsi tertentu sebagai respon. Proses ini dikenal sebagai event handling, yang menghubungkan aksi pengguna dengan perubahan tampilan atau proses dalam sistem.

Selain itu, konsep timer seperti `setInterval` digunakan untuk menjalankan proses secara berulang dalam interval waktu tertentu, misalnya dalam monitoring data secara real-time. Di sisi lain, validasi input digunakan untuk memastikan bahwa data yang dimasukkan oleh pengguna sesuai dengan aturan yang telah ditentukan, sehingga dapat mencegah kesalahan dan meningkatkan keamanan sistem. Dengan menggabungkan DOM, event, timer, dan validasi, JavaScript mampu membangun sistem web yang interaktif, responsif, dan lebih mendekati kebutuhan aplikasi nyata.

C.2 Keterkaitan dengan Teknik Industri

Materi JavaScript ini memiliki keterkaitan yang erat dengan bidang Teknik Industri, khususnya dalam pengembangan sistem monitoring, kontrol, dan pengolahan data berbasis digital. Dalam dunia industri, banyak proses yang membutuhkan interaksi antara pengguna dan sistem secara real-time, seperti pengoperasian mesin, pemantauan kondisi, serta pengambilan keputusan.

Konsep DOM digunakan untuk menampilkan data dan kondisi sistem secara visual dalam bentuk dashboard, seperti informasi suhu mesin, status produksi, atau indikator kinerja. Dengan manipulasi DOM, tampilan dapat diperbarui secara langsung sesuai kondisi yang terjadi di lapangan.

Event dan event handling berkaitan dengan sistem kontrol, misalnya penggunaan tombol Start, Stop, atau Reset pada mesin. Ketika operator melakukan aksi, sistem akan merespon secara langsung melalui program yang telah dibuat. Hal ini menyerupai sistem kontrol pada proses industri yang membutuhkan respon cepat dan akurat.

Penggunaan timer sangat relevan dalam monitoring sistem secara berkala, seperti pembaruan data suhu, tekanan, atau performa mesin dalam interval waktu tertentu. Sementara itu, validasi input digunakan untuk memastikan data yang dimasukkan operator sesuai dengan batas yang ditentukan, misalnya batas RPM, suhu maksimum, atau jumlah produksi, sehingga dapat mencegah kesalahan operasional.

Dengan memahami konsep DOM, event, timer, dan validasi, mahasiswa Teknik Industri dapat mengembangkan sistem berbasis web yang mampu mendukung proses monitoring dan kontrol di industri. Hal ini berkontribusi dalam meningkatkan efisiensi, akurasi data, serta responsivitas sistem terhadap kondisi yang terjadi.

D. LANGKAH KERJA DAN HASIL PRAKTIKUM

Dokumentasikan setiap langkah kerja yang Anda lakukan, beserta hasil yang diperoleh (screenshot, kode, atau output). Jawab sesuai urutan langkah di Manual Praktikum.

D.1 Langkah 1 – Membuat UI Panel Kontrol di HTML

Buka `index.html`, tambahkan struktur HTML berikut di dalam container.

```
/* ===== KODE / OUTPUT LANGKAH 1 ===== */
1. KODE CSS (PRAKTIKUM DAN LATIHAN 6)

/* Import font dari Google Fonts */
@import
url('https://fonts.googleapis.com/css2?family=Roboto:wght@400;700&display=swap');

body {
  font-family: 'Roboto', sans-serif;
  background-color: #f4f4f4; /* Warna abu-abu muda untuk background */
  color: #333;
  margin: 0;
  padding: 20px;
}

h1 {
  color: #2c3e50; /* Biru tua gelap */
  text-align: center;
  border-bottom: 2px solid #2c3e50;
  padding-bottom: 10px;
}

h2 {
  color: #34495e;
  margin-top: 30px;
}

table {
  width: 100%;
  border-collapse: collapse; /* Menggabungkan border ganda menjadi satu */
  margin-top: 20px;
  box-shadow: 0 2px 5px rgba(0,0,0,0.1); /* Efek bayangan halus */
}

th, td {
  padding: 12px;
  text-align: left;
  border-bottom: 1px solid #ddd;
}
```

```
}

th {
  background-color: #2980b9; /* Biru profesional */
  color: white;
}

tr:hover {
  background-color: #f1f1f1; /* Highlight baris saat mouse diarahkan */
}

form {
  background-color: white;
  padding: 20px;
  border-radius: 8px; /* Sudut melengkung */
  box-shadow: 0 2px 5px rgba(0,0,0,0.1);
  max-width: 600px;
  margin: 0 auto; /* Formulir berada di tengah */
}

label {
  display: block;
  margin-bottom: 5px;
  font-weight: bold;
}

input[type="text"],
input[type="number"],
input[type="date"],
select,
textarea {
  width: 100%; /* Lebar penuh */
  padding: 10px;
  margin-bottom: 15px;
  border: 1px solid #ccc;
  border-radius: 4px;
  box-sizing: border-box; /* Padding tidak menambah lebar total */
}

/* Styling khusus tombol */
button {
  background-color: #27ae60; /* Hijau */
  color: white;
  padding: 12px 20px;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}
```

```
width: 100%;
font-size: 16px;
}

button:hover {
  background-color: #219150; /* Hijau lebih gelap saat hover */
}

/* Tambahan styling untuk halaman utama */
.container {
  max-width: 1200px;
}

/* Navbar spacing */
.navbar {
  margin-bottom: 20px;
}

/* Card KPI hover effect */
.card {
  transition: transform 0.2s ease, box-shadow 0.2s ease;
}

.card:hover {
  transform: translateY(-3px);
  box-shadow: 0 4px 12px rgba(0,0,0,0.15);
}

/* Judul section */
h3 {
  margin-top: 20px;
  margin-bottom: 15px;
  color: #2c3e50;
}

/* Table improvement */
table {
  background-color: white;
}

.table th {
  text-align: center;
}

.table td {
  vertical-align: middle;
}
```

```
}

/* Form section spacing */
.container.mt-5 {
    margin-top: 40px !important;
}

/* Panel kontrol mesin */
.card.mt-4 {
    max-width: 800px;
    margin: 0 auto;
    border-radius: 10px;
}

/* Status indicator lebih jelas */
#statusIndicator {
    font-size: 18px;
}

/* Suhu mesin styling */
#suhuMesin {
    font-size: 40px;
    font-weight: bold;
}

/* Button spacing */
button {
    margin: 5px;
}

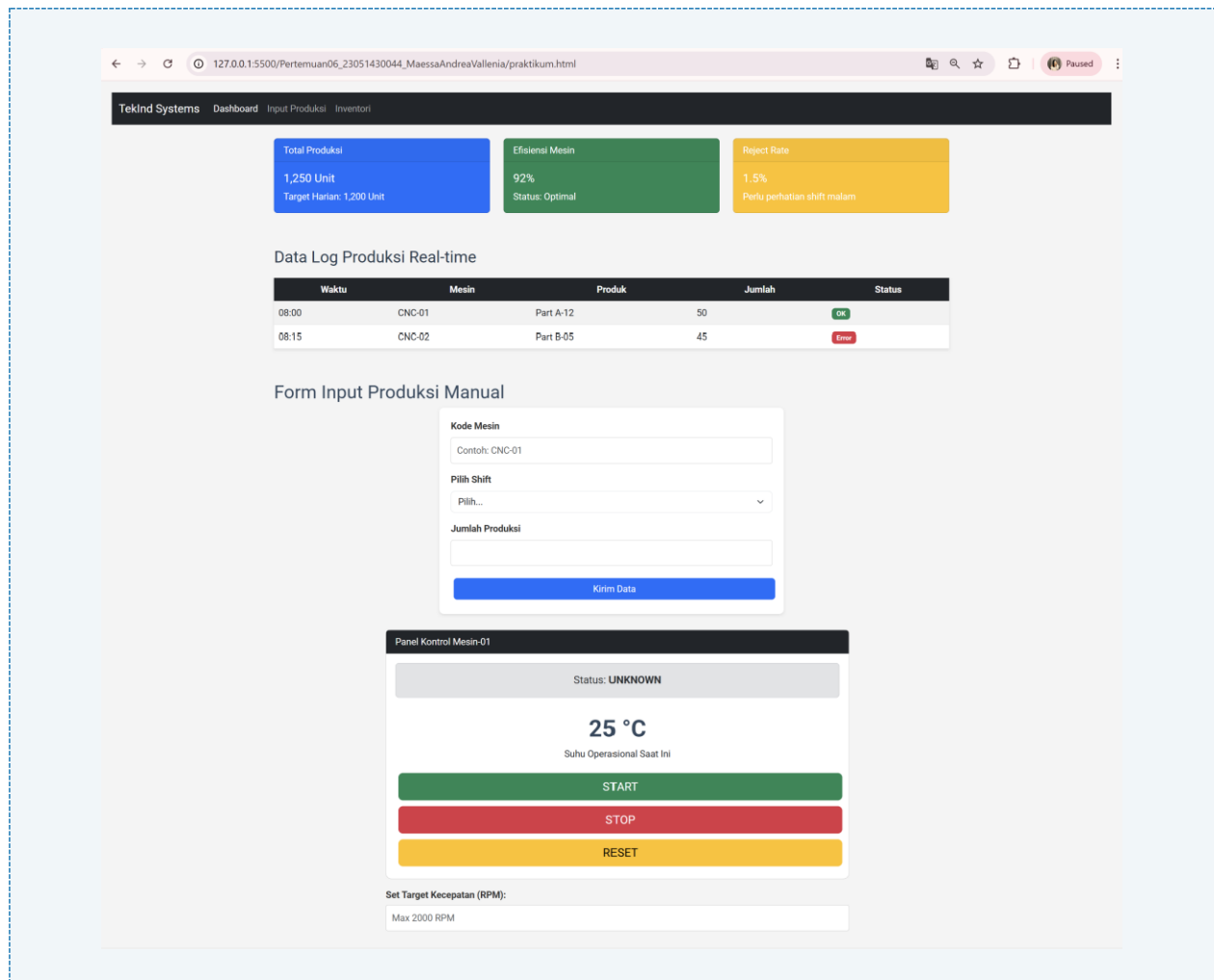
/* Input RPM section */
.mt-3 {
    max-width: 800px;
    margin: 0 auto;
}
```

2. KODE HTML

```
<!-- Simulasi Panel Kontrol Mesin (Meeting 6) -->
<div class="card mt-4">
    <div class="card-header bg-dark text-white">Panel Kontrol Mesin-01</div>
    <div class="card-body text-center">
        <!-- Indikator Status -->
        <div id="statusIndicator" class="alert alert-secondary"
role="alert">
            Status: <strong>UNKNOWN</strong>
        </div>
```

```
<h2 id="suhuMesin">25 °C</h2>
<p>Suhu Operasional Saat Ini</p>
<div class="d-grid gap-2 d-md-block">
    <button id="btnStart" class="btn btn-success btn-
lg">START</button>
    <button id="btnStop" class="btn btn-danger btn-lg">STOP</button>
    <button id="btnReset" class="btn btn-warning btn-
lg">RESET</button>
</div>
</div>
</div>
```

Screenshot Hasil:



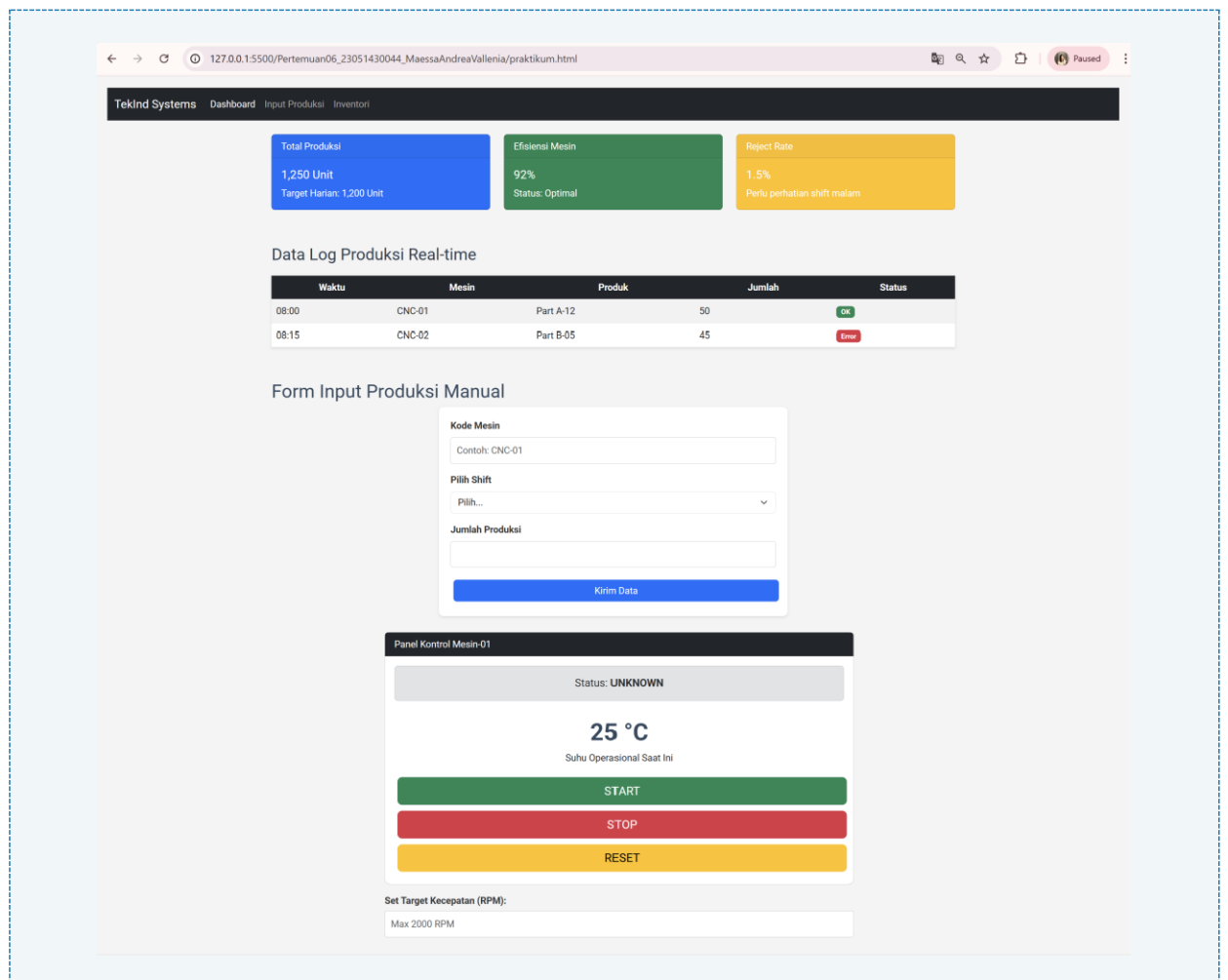
Gambar E.1 – Hasil Membuat UI Panel Kontrol di HTML

D.2 Langkah 2 – Seleksi Elemen di JavaScript

Buka `script.js` (atau buat file baru dan link-kan). Kita akan mengambil elemen yang kita buat di atas.

```
/* ===== KODE / OUTPUT LANGKAH 2 ===== */  
// 1. Seleksi Elemen DOM  
const btnStart = document.getElementById('btnStart');  
const btnStop = document.getElementById('btnStop');  
const btnReset = document.getElementById('btnReset');  
const statusIndicator = document.getElementById('statusIndicator');  
const suhuMesin = document.getElementById('suhuMesin');  
const teksStatus = statusIndicator.querySelector('strong'); // Mengambil  
elemen <strong> di dalam alert  
// Variabel State  
let suhu = 25;  
let intervalId = null; // Untuk menyimpan ID timer
```

Screenshot Hasil:



Gambar E.2 – Hasil Seleksi Elemen di JavaScript

D.3 Langkah 3 – Menambahkan Event Listener (Click)

Sekarang kita berikan "nyawa" pada tombol.

```
/* ===== KODE / OUTPUT LANGKAH 3 ===== */
// Event Listener Tombol START
btnStart.addEventListener('click', function() {
  // Ubah UI Status
  statusIndicator.className = 'alert alert-success'; // Ganti class Bootstrap (Hijau)
  teksStatus.innerText = 'RUNNING';

  // Logika simulasi kenaikan suhu
  intervalId = setInterval(function() {
    suhu += 1;
    suhuMesin.innerText = suhu + " °C";

    // Peringatan jika suhu overheat
    if (suhu > 80) {
      statusIndicator.className = 'alert alert-danger';
      teksStatus.innerText = 'OVERHEAT WARNING';
      suhuMesin.style.color = 'red';
    }
  }, 1000); // Jalankan setiap 1 detik

  // Matikan tombol Start agar tidak double click
  btnStart.disabled = true;
  btnStop.disabled = false;
});
// Event Listener Tombol STOP
btnStop.addEventListener('click', function() {
  clearInterval(intervalId); // Hentikan timer kenaikan suhu
  statusIndicator.className = 'alert alert-secondary';
  teksStatus.innerText = 'STOPPED';

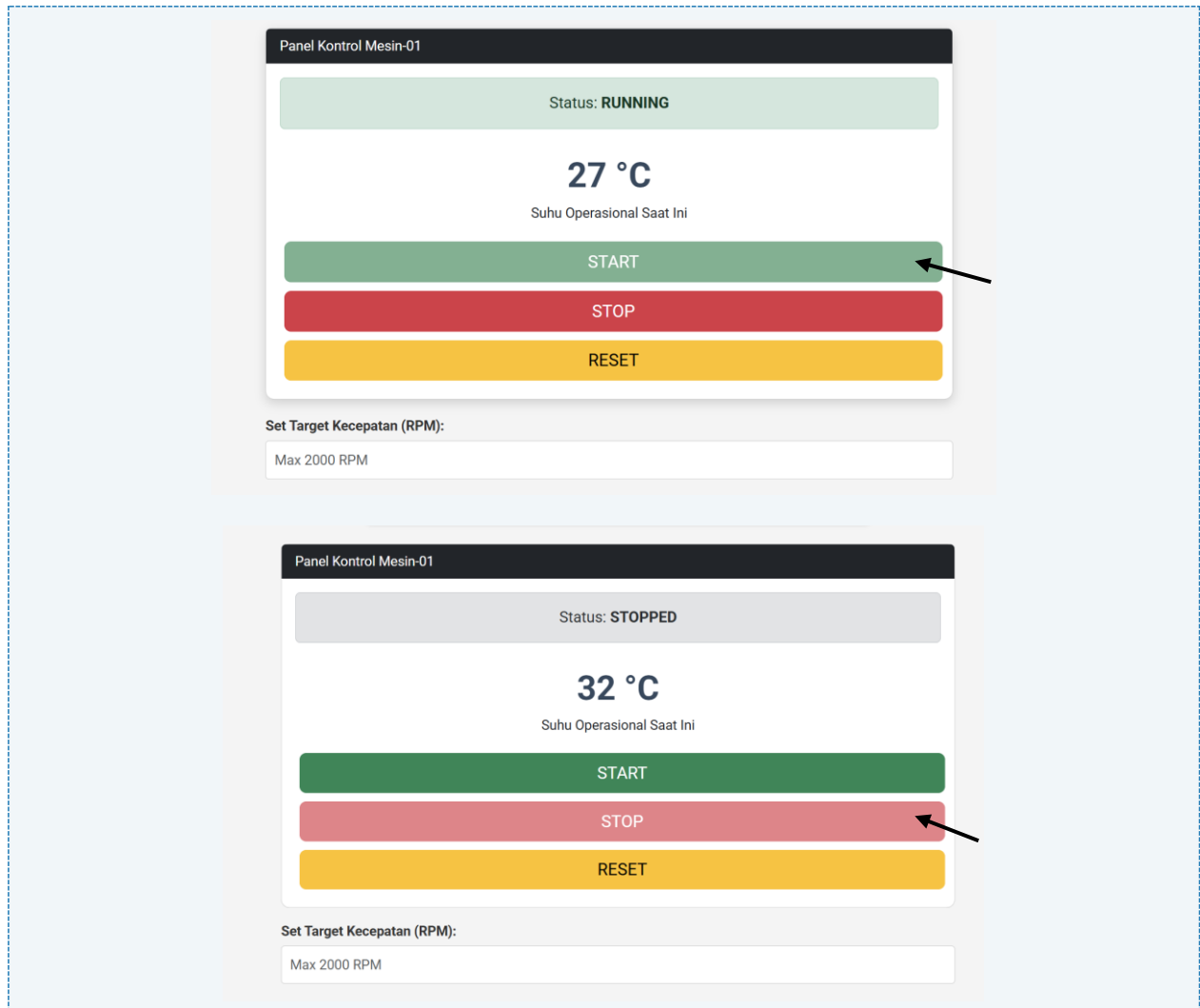
  // Reset tombol
  btnStart.disabled = false;
  btnStop.disabled = true;
});
// Event Listener Tombol RESET
btnReset.addEventListener('click', function() {
  clearInterval(intervalId);
  suhu = 25;
  suhuMesin.innerText = suhu + " °C";
  suhuMesin.style.color = 'black';
  statusIndicator.className = 'alert alert-secondary';
  teksStatus.innerText = 'UNKNOWN';
});
```

```

btnStart.disabled = false;
btnStop.disabled = true;
});

```

Screenshot Hasil:



Gambar E.3 – Hasil Menambahkan Event Listener (Click)

D.4 Langkah 4 – Event Input (Validasi Real-time)

Tambahkan elemen input baru di HTML. Dan tambahkan di JS.

```

/* ===== KODE / OUTPUT LANGKAH 4 ===== */
1. JAWABAN KODE JS

const inputRPM = document.getElementById('inputRPM');
const pesanError = document.getElementById('pesanError');
inputRPM.addEventListener('input', function() {
  let val = parseInt(this.value);

```

```

if (val > 2000) {
  pesanError.classList.remove('d-none'); // Tampilkan pesan error
  this.classList.add('is-invalid'); // Berikan border merah Bootstrap
} else {
  pesanError.classList.add('d-none'); // Sembunyikan pesan
  this.classList.remove('is-invalid');
}
});

```

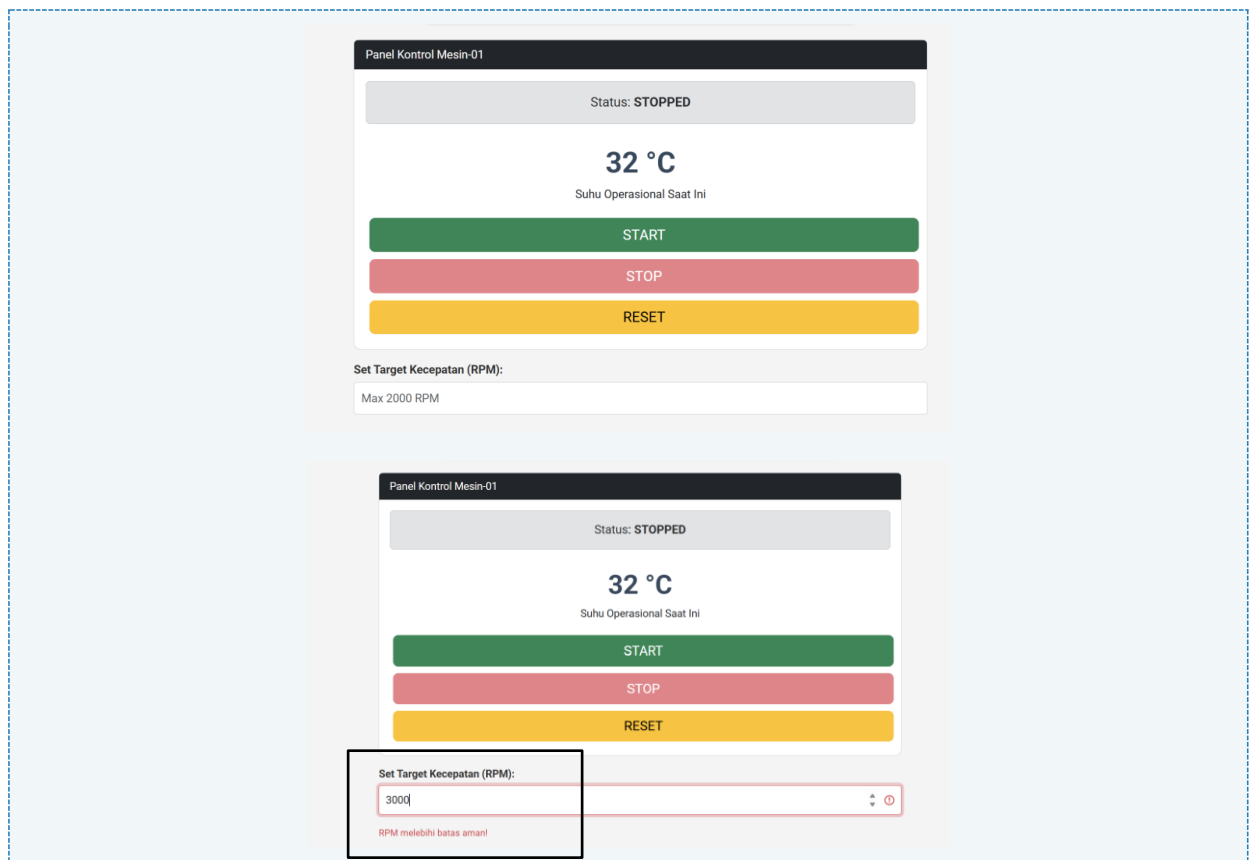
2. JAWABAN KODE HTML

```

</div>
<div class="mt-3">
  <label>Set Target Kecepatan (RPM):</label>
  <input type="number" id="inputRPM" class="form-control"
placeholder="Max 2000 RPM">
  <small id="pesanError" class="text-danger d-none">RPM melebihi batas
aman!</small>
</div>

```

Screenshot Hasil:



Gambar E.4 – Hasil Event Input (Validasi Real-time)

E. LATIHAN DAN TUGAS MANDIRI

Jawab dan dokumentasikan seluruh latihan/tugas yang ada di bagian "Latihan/Tugas Mandiri" pada modul Anda.

E.1 Latihan 1 – DOM Styling

Pertanyaan / Instruksi Latihan:

Buatlah tombol tambahan bernama "Maintenance Mode". Saat diklik, background dari card menjadi abu-abu (`bg-light`), dan teks status berubah menjadi "MAINTENANCE".

Jawaban / Kode Solusi (JS & HTML)

1. JAWABAN KODE CSS

```
/* Import font dari Google Fonts */ @import
url('https://fonts.googleapis.com/css2?family=Roboto:wght@400;700&display=swa p');

body {
  font-family: 'Roboto', sans-serif;
  background-color: #f4f4f4; /* Warna abu-abu muda untuk background */ color: #333;
  margin: 0; padding:
  20px;
}

h1 {
  color: #2c3e50; /* Biru tua gelap */ text-align:
  center;
  border-bottom: 2px solid #2c3e50;
  padding-bottom: 10px;
}

h2 {
  color: #34495e;
  margin-top: 30px;
}

table {
  width: 100%;
  border-collapse: collapse; /* Menggabungkan border ganda menjadi satu */ margin-top:
  20px;
  box-shadow: 0 2px 5px rgba(0,0,0,0.1); /* Efek bayangan halus */
}

th, td {
```

```
padding: 12px;
text-align: left;
border-bottom: 1px solid #ddd;
}

th {
  background-color: #2980b9; /* Biru profesional */
  color: white;
}

tr:hover {
  background-color: #f1f1f1; /* Highlight baris saat mouse diarahkan */
}

form {
  background-color: white;
  padding: 20px;
  border-radius: 8px; /* Sudut melengkung */
  box-shadow: 0 2px 5px rgba(0,0,0,0.1);
  max-width: 600px;
  margin: 0 auto; /* Formulir berada di tengah */
}

label {
  display: block;
  margin-bottom: 5px;
  font-weight: bold;
}

input[type="text"],
input[type="number"],
input[type="date"],
select,
textarea {
  width: 100%; /* Lebar penuh */
  padding: 10px;
  margin-bottom: 15px;
  border: 1px solid #ccc;
  border-radius: 4px;
  box-sizing: border-box; /* Padding tidak menambah lebar total */
}

/* Styling khusus tombol */
button {
  background-color: #27ae60; /* Hijau */
  color: white;
  padding: 12px 20px;
```

```
border: none;
border-radius: 4px;
cursor: pointer;
width: 100%;
font-size: 16px;
}

button:hover {
  background-color: #219150; /* Hijau lebih gelap saat hover */
}

/* Tambahan styling untuk halaman utama */
.container {
  max-width: 1200px;
}

/* Navbar spacing */
.navbar {
  margin-bottom: 20px;
}

/* Card KPI hover effect */
.card {
  transition: transform 0.2s ease, box-shadow 0.2s ease;
}

.card:hover {
  transform: translateY(-3px);
  box-shadow: 0 4px 12px rgba(0,0,0,0.15);
}

/* Judul section */
h3 {
  margin-top: 20px;
  margin-bottom: 15px;
  color: #2c3e50;
}

/* Table improvement */
table {
  background-color: white;
}

.table th {
  text-align: center;
}
```

```
.table td {
    vertical-align: middle;
}

/* Form section spacing */
.container.mt-5 {
    margin-top: 40px !important;
}

/* Panel kontrol mesin */
.card.mt-4 {
    max-width: 800px;
    margin: 0 auto;
    border-radius: 10px;
}

/* Status indicator lebih jelas */
#statusIndicator {
    font-size: 18px;
}

/* Suhu mesin styling */
#suhuMesin {
    font-size: 40px;
    font-weight: bold;
}

/* Button spacing */
button {
    margin: 5px;
}

/* Input RPM section */
.mt-3 {
    max-width: 800px;
    margin: 0 auto;
}
```

2. JAWABAN KODE JS

```
// 1. Seleksi Elemen DOM
const btnStart = document.getElementById('btnStart');
const btnStop = document.getElementById('btnStop');
const btnReset = document.getElementById('btnReset');
const statusIndicator = document.getElementById('statusIndicator');
```

```
// Seleksi elemen dom untuk suhuMesin(latihan 2)
const suhuMesin = document.getElementById('suhuMesin');
const teksStatus = statusIndicator.querySelector('strong'); // Mengambil
elemen <strong> di dalam alert
// Membuat seleksi elemen DOM untuk btnMaintenance dan card(latihan 1)
const btnMaintenance = document.getElementById('btnMaintenance');
const card = document.querySelector('.card');

// Variabel State
let suhu = 25;
let intervalId = null; // Untuk menyimpan ID timer
// Event Listener Tombol START
btnStart.addEventListener('click', function () { // Ubah UI Status
    statusIndicator.className = 'alert alert-success'; // Ganti class
    Bootstrap (Hijau)
    teksStatus.innerText = 'RUNNING';

    // Logika simulasi kenaikan suhu
    intervalId = setInterval(function () {
        suhu += 1;
        suhuMesin.innerText = suhu + " °C";

        // Peringatan jika suhu overheating
        if (suhu > 80) {
            statusIndicator.className = 'alert alert-danger';
            teksStatus.innerText = 'OVERHEAT WARNING';
            suhuMesin.style.color = 'red';
        }
    }, 1000); // Jalankan setiap 1 detik

    // Matikan tombol Start agar tidak double click
    btnStart.disabled = true;
    btnStop.disabled = false;
});

// Event Listener Tombol STOP
btnStop.addEventListener('click', function () {
    clearInterval(intervalId); // Hentikan timer kenaikan suhu
    statusIndicator.className = 'alert alert-danger'; // Mengubah warna
    menjadi merah (latihan 1)
    teksStatus.innerText = 'STOPPED';

    // Reset tombol
    btnStart.disabled = false;
    btnStop.disabled = true;
});
```

```
// Event Listener Tombol RESET
```



```
btnReset.addEventListener('click', function () {
    clearInterval(intervalId);
    suhu = 25;
    suhuMesin.innerText = suhu + " °C";
    suhuMesin.style.color = 'black';
    statusIndicator.className = 'alert alert-warning'; // Mengubah warna
    menjadi orange (latihan 1)
    teksStatus.innerText = 'UNKNOWN';

    btnStart.disabled = false;
    btnStop.disabled = true;
});

// Event saat tombol Maintenance ditekan (latihan 1)
btnMaintenance.addEventListener('click', () => {
    // Hentikan simulasi jika masih berjalan
    clearInterval(intervalId);

    // Ubah tampilan card menjadi abu-abu dan teks menjadi maintenance
    (latihan 1)
    card.classList.add('bg-light');

    // Update status menjadi maintenance (latihan 1)
    statusIndicator.className = 'alert alert-secondary';
    teksStatus.textContent = 'MAINTENANCE';

    // Nonaktifkan tombol Start dan Stop karena dalam mode maintenance
    (latihan 1)
    btnStart.disabled = true;
    btnStop.disabled = true;
});

// Event saat mouse masuk ke teks suhu (latihan 2)
suhuMesin.addEventListener('mouseover', () => {
    suhuMesin.style.fontWeight = 'bold';
    suhuMesin.style.color = 'blue';
});

// Event saat mouse keluar dari teks suhu (latihan 2)
suhuMesin.addEventListener('mouseout', () => {
    suhuMesin.style.fontWeight = 'normal';
    suhuMesin.style.color = 'black';
});
```

```
const inputRPM = document.getElementById('inputRPM');
const pesanError = document.getElementById('pesanError');
inputRPM.addEventListener('input', function () {
    let val = parseInt(this.value);

    if (val > 2000) {
        pesanError.classList.remove('d-none'); // Tampilkan pesan error
        this.classList.add('is-invalid'); // Berikan border merah Bootstrap
    } else {
        pesanError.classList.add('d-none'); // Sembunyikan pesan
        this.classList.remove('is-invalid');
    }
});
```

3. JAWABAN KODE HTML

```
<!DOCTYPE html>
<html lang="id">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Sistem Monitoring Produksi</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
    <link rel="stylesheet" href="praktikum_style.css">
</head>

<body>
    <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
        <div class="container-fluid">
            <a class="navbar-brand" href="">TekInd Systems</a>
            <button class="navbar-toggler" type="button" data-bs-
toggle="collapse" data-bs-target="#navbarNav">
                <span class="navbar-toggler-icon"></span>
            </button>
            <div class="collapse navbar-collapse" id="navbarNav">
                <ul class="navbar-nav">
                    <li class="nav-item">
                        <a class="nav-link active" href="">Dashboard</a>
                    </li>
                    <li class="nav-item">
                        <a class="nav-link" href="">Input Produksi</a>
                    </li>
                </ul>
            </div>
        </div>
    </nav>
```

```

        </li>
        <li class="nav-item">
            <a class="nav-link" href="">Inventori</a>
        </li>
    </ul>
</div>
</div>
</nav>
<div class="container mt-4">
    <!-- Row 1: KPI Cards (Menggunakan Grid System) -->
    <div class="row mb-4">
        <!-- Kolom lebar 4 (dari 12) -->
        <div class="col-md-4">
            <div class="card text-white bg-primary mb-3">
                <div class="card-header">Total Produksi</div>
                <div class="card-body">
                    <h5 class="card-title">1,250 Unit</h5>
                    <p class="card-text">Target Harian: 1,200 Unit</p>
                </div>
            </div>
        </div>
        <!-- Kolom lebar 4 -->
        <div class="col-md-4">
            <div class="card text-white bg-success mb-3">
                <div class="card-header">Efisiensi Mesin</div>
                <div class="card-body">
                    <h5 class="card-title">92%</h5>
                    <p class="card-text">Status: Optimal</p>
                </div>
            </div>
        </div>
        <!-- Kolom lebar 4 -->
        <div class="col-md-4">
            <div class="card text-white bg-warning mb-3">
                <div class="card-header">Reject Rate</div>
                <div class="card-body">
                    <h5 class="card-title">1.5%</h5>
                    <p class="card-text">Perlu perhatian shift malam</p>
                </div>
            </div>
        </div>
    </div>
    <!-- Row 2: Tabel Data -->
    <div class="row">
        <div class="col-12">
            <h3>Data Log Produksi Real-time</h3>

```

```

<table class="table table-striped table-hover">
  <thead class="table-dark">
    <tr>
      <th>Waktu</th>
      <th>Mesin</th>
      <th>Produk</th>
      <th>Jumlah</th>
      <th>Status</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>08:00</td>
      <td>CNC-01</td>
      <td>Part A-12</td>
      <td>50</td>
      <td><span class="badge bg-success">OK</span></td>
    </tr>
    <tr>
      <td>08:15</td>
      <td>CNC-02</td>
      <td>Part B-05</td>
      <td>45</td>
      <td><span class="badge bg-
danger">Error</span></td>
    </tr>
  </tbody>
</table>
</div>
</div>
<div class="container mt-5">
  <h2>Form Input Produksi Manual</h2>
  <form>
    <div class="mb-3">
      <label for="kodeMesin" class="form-label">Kode Mesin</label>
      <input type="text" class="form-control" id="kodeMesin"
placeholder="Contoh: CNC-01">
    </div>
    <div class="mb-3">
      <label for="shiftSelect" class="form-label">Pilih
Shift</label>
      <select class="form-select" id="shiftSelect">
        <option selected>Pilih...</option>
        <option value="1">Shift 1 (Pagi)</option>
        <option value="2">Shift 2 (Siang)</option>
      </select>
    </div>
  </form>
</div>

```

```

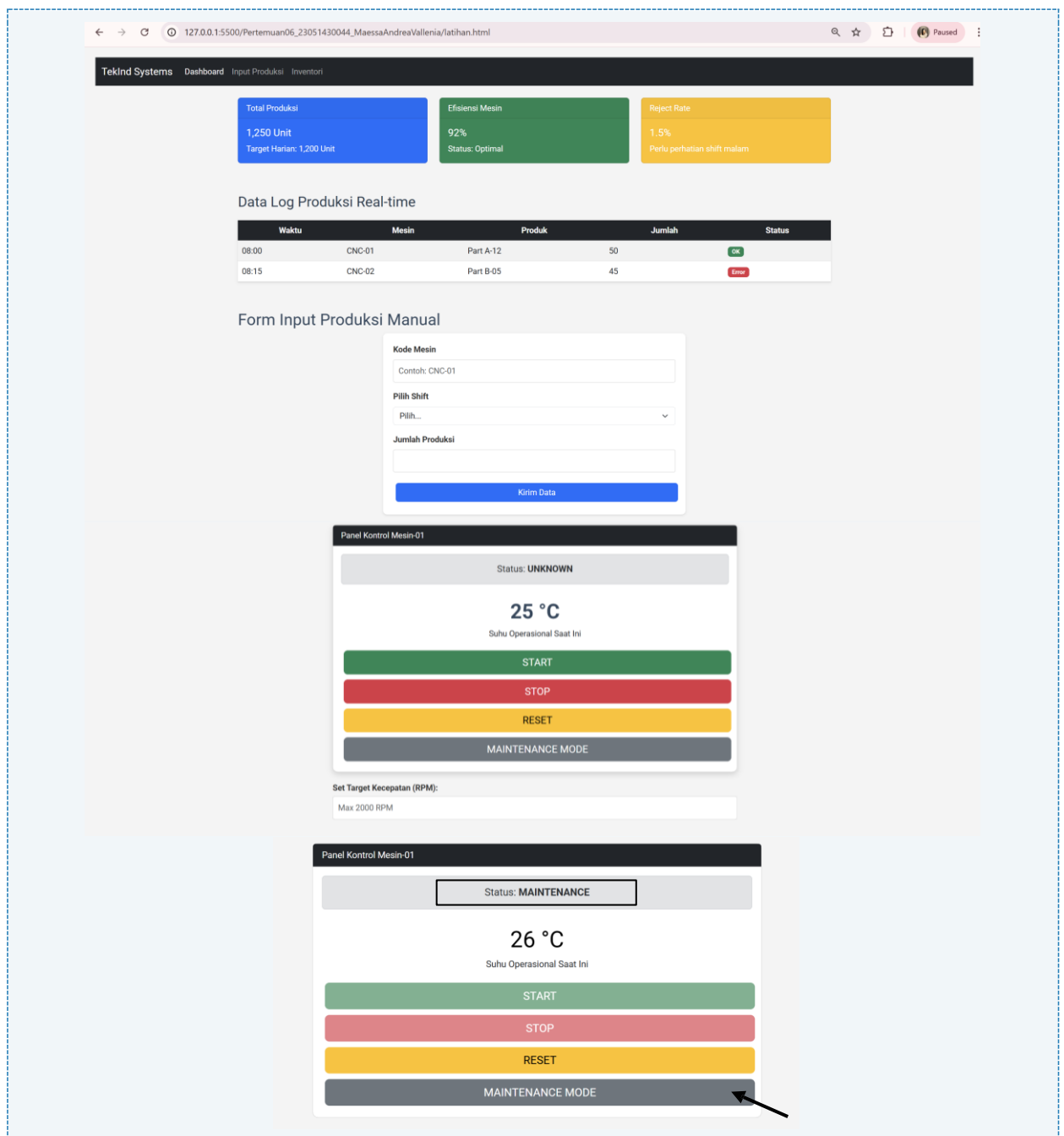
        <option vaue="3">Shift 3(malam)</option>
    </select>
</div>
<div class="mb-3">
    <label for="jumlahProduksi" class="form-label">Jumlah
Produksi</label>
    <input type="number" class="form-control"
id="jumlahProduksi">
</div>
    <button type="submit" class="btn btn-primary">Kirim Data</button>
</form>
</div>
<!-- Simulasi Panel Kontrol Mesin (Meeting 6) -->
<div class="card mt-4">
    <div class="card-header bg-dark text-white">Panel Kontrol Mesin-
01</div>
    <div class="card-body text-center">
        <!-- Indikator Status -->
        <div id="statusIndicator" class="alert alert-secondary"
role="alert">
            Status: <strong>UNKNOWN</strong>
        </div>

        <h2 id="suhuMesin">25 °C</h2>
        <p>Suhu Operasional Saat Ini</p>
        <div class="d-grid gap-2 d-md-block">
            <button id="btnStart" class="btn btn-success btn-
lg">START</button>
            <button id="btnStop" class="btn btn-danger btn-
lg">STOP</button>
            <button id="btnReset" class="btn btn-warning btn-
lg">RESET</button>
            <!-- Membuat tombol tambahan bernama "Maintenance Mode"
(Latihan 1) -->
            <button id="btnMaintenance" class="btn btn-secondary btn-
lg">MAINTENANCE MODE</button>
        </div>
    </div>
</div>
<div class="mt-3">
    <label>Set Target Kecepatan (RPM):</label>
    <input type="number" id="inputRPM" class="form-control"
placeholder="Max 2000 RPM">
    <small id="pesanError" class="text-danger d-none">RPM melebihi batas
aman!</small>
</div><br>

```

```
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></script>
<script src="latihan.js"></script>
</body>

</html>
```



Gambar F.1 – Hasil DOM Styling

E.2 Latihan 2 – Mouse Events

Pertanyaan / Instruksi Latihan:

Tambahkan efek saat mouse diarahkan (`mouseover`) ke teks suhu mesin, suhu tersebut menjadi tebal dan berwarna biru. Saat mouse keluar (`mouseout`), kembali normal.

Jawaban / Kode Solusi:

1. JAWABAN KODE CSS

```
/* Import font dari Google Fonts */ @import
url('https://fonts.googleapis.com/css2?family=Roboto:wght@400;700&display=swa p');

body {
  font-family: 'Roboto', sans-serif;
  background-color: #f4f4f4; /* Warna abu-abu muda untuk background */ color: #333;
  margin: 0; padding:
  20px;
}

h1 {
  color: #2c3e50; /* Biru tua gelap */ text-align:
  center;
  border-bottom: 2px solid #2c3e50;
  padding-bottom: 10px;
}

h2 {
  color: #34495e;
  margin-top: 30px;
}

table {
  width: 100%;
  border-collapse: collapse; /* Menggabungkan border ganda menjadi satu */ margin-top:
  20px;
  box-shadow: 0 2px 5px rgba(0,0,0,0.1); /* Efek bayangan halus */
}

th, td {
  padding: 12px; text-
  align: left;
  border-bottom: 1px solid #ddd;
}
```

```
th {
  background-color: #2980b9; /* Biru profesional */
  color: white;
}

tr:hover {
  background-color: #f1f1f1; /* Highlight baris saat mouse diarahkan */
}

form {
  background-color: white;
  padding: 20px;
  border-radius: 8px; /* Sudut melengkung */
  box-shadow: 0 2px 5px rgba(0,0,0,0.1);
  max-width: 600px;
  margin: 0 auto; /* Formulir berada di tengah */
}

label {
  display: block;
  margin-bottom: 5px;
  font-weight: bold;
}

input[type="text"],
input[type="number"],
input[type="date"],
select,
textarea {
  width: 100%; /* Lebar penuh */
  padding: 10px;
  margin-bottom: 15px;
  border: 1px solid #ccc;
  border-radius: 4px;
  box-sizing: border-box; /* Padding tidak menambah lebar total */
}

/* Styling khusus tombol */
button {
  background-color: #27ae60; /* Hijau */
  color: white;
  padding: 12px 20px;
  border: none;
  border-radius: 4px;
  cursor: pointer;
  width: 100%;
```



```
        font-size: 16px;
    }

    button:hover {
        background-color: #219150; /* Hijau lebih gelap saat hover */
    }

    /* Tambahan styling untuk halaman utama */
    .container {
        max-width: 1200px;
    }

    /* Navbar spacing */
    .navbar {
        margin-bottom: 20px;
    }

    /* Card KPI hover effect */
    .card {
        transition: transform 0.2s ease, box-shadow 0.2s ease;
    }

    .card:hover {
        transform: translateY(-3px);
        box-shadow: 0 4px 12px rgba(0,0,0,0.15);
    }

    /* Judul section */
    h3 {
        margin-top: 20px;
        margin-bottom: 15px;
        color: #2c3e50;
    }

    /* Table improvement */
    table {
        background-color: white;
    }

    .table th {
        text-align: center;
    }

    .table td {
        vertical-align: middle;
    }
}
```

```
/* Form section spacing */
.container.mt-5 {
  margin-top: 40px !important;
}

/* Panel kontrol mesin */
.card.mt-4 {
  max-width: 800px;
  margin: 0 auto;
  border-radius: 10px;
}

/* Status indicator lebih jelas */
#statusIndicator {
  font-size: 18px;
}

/* Suhu mesin styling */
#suhuMesin {
  font-size: 40px;
  font-weight: bold;
}

/* Button spacing */
button {
  margin: 5px;
}

/* Input RPM section */
.mt-3 {
  max-width: 800px;
  margin: 0 auto;
}
```

2. JAWABAN KODE JS

```
// 1. Seleksi Elemen DOM
const btnStart = document.getElementById('btnStart');
const btnStop = document.getElementById('btnStop');
const btnReset = document.getElementById('btnReset');
const statusIndicator = document.getElementById('statusIndicator');
// Seleksi elemen dom untuk suhuMesin(latihan 2)
const suhuMesin = document.getElementById('suhuMesin');
const teksStatus = statusIndicator.querySelector('strong'); // Mengambil
elemen <strong> di dalam alert
```

```
// Membuat seleksi elemen DOM untuk btnMaintenance dan card(latihan 1)
const btnMaintenance = document.getElementById('btnMaintenance');
const card = document.querySelector('.card');

// Variabel State
let suhu = 25;
let intervalId = null; // Untuk menyimpan ID timer
// Event Listener Tombol START
btnStart.addEventListener('click', function () { // Ubah UI Status
    statusIndicator.className = 'alert alert-success'; // Ganti class
    // Bootstrap (Hijau)
    teksStatus.innerText = 'RUNNING';

    // Logika simulasi kenaikan suhu
    intervalId = setInterval(function () {
        suhu += 1;
        suhuMesin.innerText = suhu + " °C";

        // Peringatan jika suhu overheat
        if (suhu > 80) {
            statusIndicator.className = 'alert alert-danger';
            teksStatus.innerText = 'OVERHEAT WARNING';
            suhuMesin.style.color = 'red';
        }
    }, 1000); // Jalankan setiap 1 detik

    // Matikan tombol Start agar tidak double click
    btnStart.disabled = true;
    btnStop.disabled = false;
});

// Event Listener Tombol STOP
btnStop.addEventListener('click', function () {
    clearInterval(intervalId); // Hentikan timer kenaikan suhu
    statusIndicator.className = 'alert alert-danger'; // Mengubah warna
    // menjadi merah (latihan 1)
    teksStatus.innerText = 'STOPPED';

    // Reset tombol
    btnStart.disabled = false;
    btnStop.disabled = true;
});

// Event Listener Tombol RESET
btnReset.addEventListener('click', function () {
    clearInterval(intervalId);
    suhu = 25;
    suhuMesin.innerText = suhu + " °C";
```

```
suhuMesin.style.color = 'black';
statusIndicator.className = 'alert alert-warning'; // Mengubah warna
menjadi orange (latihan 1)
teksStatus.innerText = 'UNKNOWN';

btnStart.disabled = false;
btnStop.disabled = true;
});

// Event saat tombol Maintenance ditekan (latihan 1)
btnMaintenance.addEventListener('click', () => {
  // Hentikan simulasi jika masih berjalan
  clearInterval(intervalId);

  // Ubah tampilan card menjadi abu-abu dan teks menjadi maintenance
  (latihan 1)
  card.classList.add('bg-light');

  // Update status menjadi maintenance (latihan 1)
  statusIndicator.className = 'alert alert-secondary';
  teksStatus.textContent = 'MAINTENANCE';

  // Nonaktifkan tombol Start dan Stop karena dalam mode maintenance
  (latihan 1)
  btnStart.disabled = true;
  btnStop.disabled = true;
});

// Event saat mouse masuk ke teks suhu (latihan 2)
suhuMesin.addEventListener('mouseover', () => {
  suhuMesin.style.fontWeight = 'bold';
  suhuMesin.style.color = 'blue';
});

// Event saat mouse keluar dari teks suhu (latihan 2)
suhuMesin.addEventListener('mouseout', () => {
  suhuMesin.style.fontWeight = 'normal';
  suhuMesin.style.color = 'black';
});

const inputRPM = document.getElementById('inputRPM');
const pesanError = document.getElementById('pesanError');
inputRPM.addEventListener('input', function () {
```

```
let val = parseInt(this.value);

if (val > 2000) {
    pesanError.classList.remove('d-none'); // Tampilkan pesan error
    this.classList.add('is-invalid'); // Berikan border merah Bootstrap
} else {
    pesanError.classList.add('d-none'); // Sembunyikan pesan
    this.classList.remove('is-invalid');
}
});
```

3. JAWABAN KODE HTML

```
<!DOCTYPE html>
<html lang="id">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Sistem Monitoring Produksi</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
" rel="stylesheet">
    <link rel="stylesheet" href="praktikum_style.css">
</head>

<body>
    <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
        <div class="container-fluid">
            <a class="navbar-brand" href="">TekInd Systems</a>
            <button class="navbar-toggler" type="button" data-bs-
toggle="collapse" data-bs-target="#navbarNav">
                <span class="navbar-toggler-icon"></span>
            </button>
            <div class="collapse navbar-collapse" id="navbarNav">
                <ul class="navbar-nav">
                    <li class="nav-item">
                        <a class="nav-link active" href="">Dashboard</a>
                    </li>
                    <li class="nav-item">
                        <a class="nav-link" href="">Input Produksi</a>
                    </li>
                    <li class="nav-item">
                        <a class="nav-link" href="">Inventori</a>
                    </li>
                </ul>
            </div>
        </div>
    </nav>
```

```

        </div>
    </div>
</nav>
<div class="container mt-4">
    <!-- Row 1: KPI Cards (Menggunakan Grid System) -->
    <div class="row mb-4">
        <!-- Kolom lebar 4 (dari 12) -->
        <div class="col-md-4">
            <div class="card text-white bg-primary mb-3">
                <div class="card-header">Total Produksi</div>
                <div class="card-body">
                    <h5 class="card-title">1,250 Unit</h5>
                    <p class="card-text">Target Harian: 1,200 Unit</p>
                </div>
            </div>
        </div>
        <!-- Kolom lebar 4 -->
        <div class="col-md-4">
            <div class="card text-white bg-success mb-3">
                <div class="card-header">Efisiensi Mesin</div>
                <div class="card-body">
                    <h5 class="card-title">92%</h5>
                    <p class="card-text">Status: Optimal</p>
                </div>
            </div>
        </div>
        <!-- Kolom lebar 4 -->
        <div class="col-md-4">
            <div class="card text-white bg-warning mb-3">
                <div class="card-header">Reject Rate</div>
                <div class="card-body">
                    <h5 class="card-title">1.5%</h5>
                    <p class="card-text">Perlu perhatian shift malam</p>
                </div>
            </div>
        </div>
    </div>
    <!-- Row 2: Tabel Data -->
    <div class="row">
        <div class="col-12">
            <h3>Data Log Produksi Real-time</h3>
            <table class="table table-striped table-hover">
                <thead class="table-dark">
                    <tr>
                        <th>Waktu</th>
                        <th>Mesin</th>
                    </tr>
                </thead>
            </table>
        </div>
    </div>

```

```

        <th>Produk</th>
        <th>Jumlah</th>
        <th>Status</th>
    </tr>
</thead>
<tbody>
    <tr>
        <td>08:00</td>
        <td>CNC-01</td>
        <td>Part A-12</td>
        <td>50</td>
        <td><span class="badge bg-success">OK</span></td>
    </tr>
    <tr>
        <td>08:15</td>
        <td>CNC-02</td>
        <td>Part B-05</td>
        <td>45</td>
        <td><span class="badge bg-
danger">Error</span></td>
    </tr>
</tbody>
</table>
</div>
</div>
<div class="container mt-5">
    <h2>Form Input Produksi Manual</h2>
    <form>
        <div class="mb-3">
            <label for="kodeMesin" class="form-label">Kode Mesin</label>
            <input type="text" class="form-control" id="kodeMesin"
placeholder="Contoh: CNC-01">
        </div>
        <div class="mb-3">
            <label for="shiftSelect" class="form-label">Pilih
Shift</label>
            <select class="form-select" id="shiftSelect">
                <option selected>Pilih...</option>
                <option value="1">Shift 1 (Pagi)</option>
                <option value="2">Shift 2 (Siang)</option>
                <option vaue="3">Shift 3(malam)</option>
            </select>
        </div>
        <div class="mb-3">

```

```

        <label for="jumlahProduksi" class="form-label">Jumlah
Produksi</label>
        <input type="number" class="form-control"
id="jumlahProduksi">
    </div>
    <button type="submit" class="btn btn-primary">Kirim Data</button>
</form>
</div>
<!-- Simulasi Panel Kontrol Mesin (Meeting 6) -->
<div class="card mt-4">
    <div class="card-header bg-dark text-white">Panel Kontrol Mesin-
01</div>
    <div class="card-body text-center">
        <!-- Indikator Status -->
        <div id="statusIndicator" class="alert alert-secondary"
role="alert">
            Status: <strong>UNKNOWN</strong>
        </div>

        <h2 id="suhuMesin">25 °C</h2>
        <p>Suhu Operasional Saat Ini</p>
        <div class="d-grid gap-2 d-md-block">
            <button id="btnStart" class="btn btn-success btn-
lg">START</button>
            <button id="btnStop" class="btn btn-danger btn-
lg">STOP</button>
            <button id="btnReset" class="btn btn-warning btn-
lg">RESET</button>
            <!-- Membuat tombol tambahan bernama "Maintenance Mode"
(Latihan 1) -->
            <button id="btnMaintenance" class="btn btn-secondary btn-
lg">MAINTENANCE MODE</button>
        </div>
    </div>
</div>
<div class="mt-3">
    <label>Set Target Kecepatan (RPM):</label>
    <input type="number" id="inputRPM" class="form-control"
placeholder="Max 2000 RPM">
    <small id="pesanError" class="text-danger d-none">RPM melebihi batas
aman!</small>
</div><br>

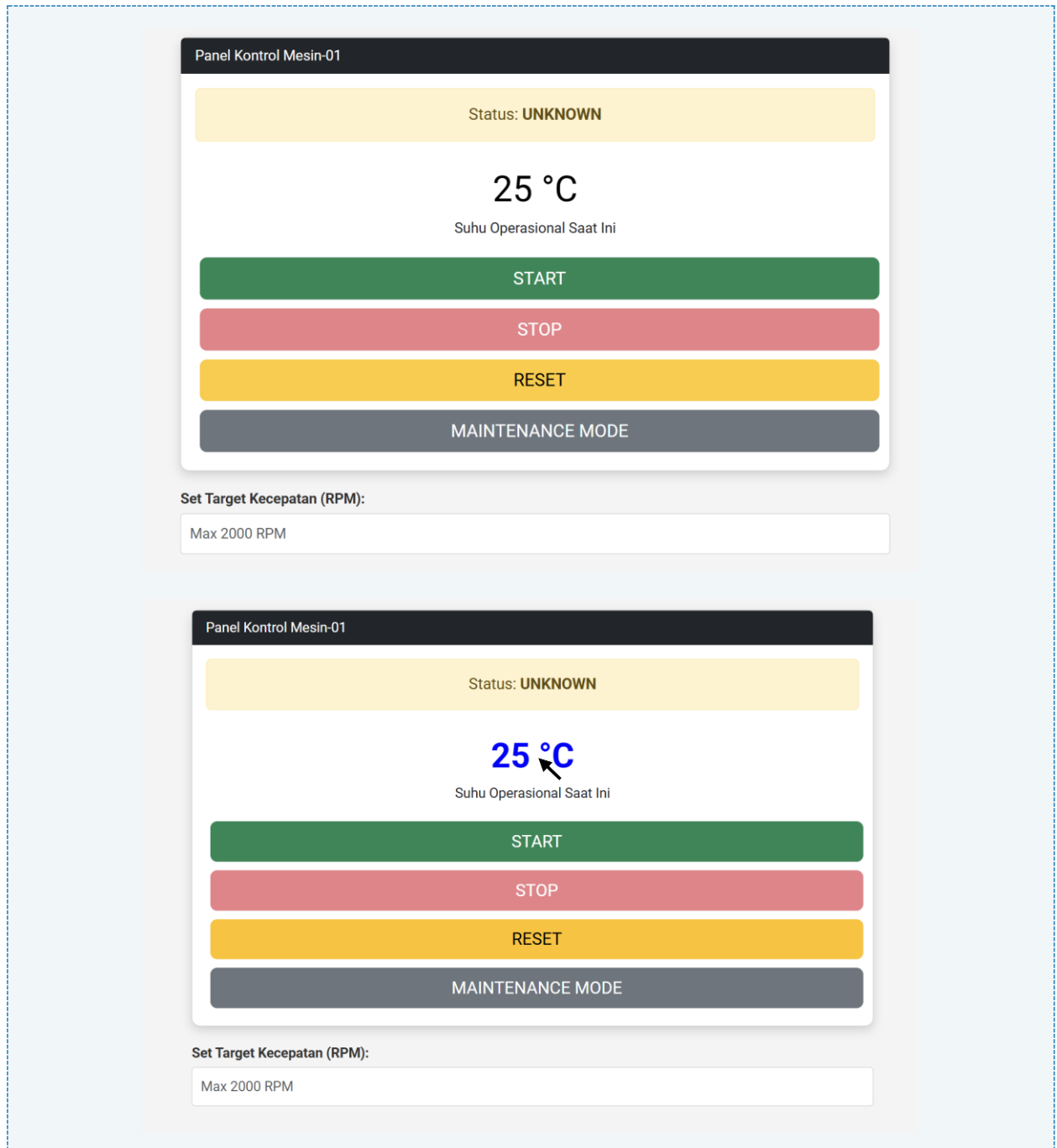
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.mi
n.js"></script>

```



```
<script src="latihan.js"></script>
</body>

</html>
```



Gambar F.2 – Hasil Mouse Events

E.3 Tugas Proyek Mini – Kalkulator Tagihan Listrik

Deskripsi proyek mini:

- a. Buat UI dengan 2 input: "Daya (Watt)" dan "Jam Pemakaian".
- b. Saat user mengetik di salah satu input, secara real-time (tanpa tombol submit) tampilkan hasil "Total kWh" dan "Estimasi Biaya (Rp 1.500/kWh)" di elemen `<h3>` di bawahnya.

Penjelasan pendekatan/solusi yang digunakan:

Proyek mini ini dibuat untuk menerapkan konsep DOM, event listener, dan manipulasi elemen dalam JavaScript. Program memproses input dari pengguna secara real-time menggunakan event input, kemudian menampilkan hasil perhitungan konsumsi listrik (kWh) dan estimasi biaya langsung pada halaman web tanpa perlu tombol submit.

Program dibuat menggunakan HTML sebagai struktur tampilan, CSS/Bootstrap untuk styling agar lebih rapi dan responsif, serta JavaScript sebagai pengolah logika perhitungan dan pengendali event.

1. Membuat folder proyek, misalnya: `ProyekMini_KalkulatorListrik_Modul6`, kemudian membuat tiga file utama:
 - a. `index.html`
 - b. `script.js`
 - c. (opsional) `style.css`
2. Membuat tampilan yang berisi:
 - a. Judul proyek dan identitas mahasiswa
 - b. Dua input: Daya (Watt) dan Jam Pemakaian
 - c. Area output menggunakan elemen `<h3>` untuk menampilkan hasil
 - d. Menghubungkan file JavaScript ke HTML
3. Memberikan id pada setiap input dan elemen hasil agar dapat diakses oleh JavaScript, misalnya:
 - a. `inputDaya`
 - b. `inputJam`
 - c. `hasilKwh`
 - d. `hasilBiaya`
4. Menggunakan DOM selector seperti:
`document.getElementById()`
untuk mengambil elemen input dan output yang akan dimanipulasi.
5. Membuat function, misalnya `hitungListrik()`, yang berfungsi untuk:
 - a. Mengambil nilai dari input daya dan jam
 - b. Menghitung total kWh dengan rumus:
$$\text{kWh} = (\text{Daya} \times \text{Jam}) / 1000$$
 - c. Menghitung biaya:
$$\text{Biaya} = \text{kWh} \times 1500$$
 - d. Menampilkan hasil ke elemen `<h3>`
6. Menggunakan event input pada kedua field agar setiap perubahan nilai langsung memicu perhitungan tanpa tombol submit, sehingga hasil ditampilkan secara real-time.

7. Memastikan input tidak kosong atau bernilai negatif. Jika input belum diisi, sistem tetap berjalan dengan nilai default 0 agar tidak menghasilkan error atau NaN.
8. Hasil perhitungan ditampilkan langsung pada elemen <h3> menggunakan innerText atau textContent, sehingga user dapat melihat perubahan secara langsung saat mengetik.
9. Melakukan pengujian dengan mencoba berbagai input untuk memastikan:
 - a. Perhitungan berjalan dengan benar
 - b. Event listener aktif
 - c. Output berubah secara real-time

Jawaban / Kode Solusi:

1. JAWABAN KODE CSS

```
body {  
  background: linear-gradient( to  
    right,  
    #0f2027,  
    #203a43,  
    #346174  
  );  
  min-height: 100vh;  
}  
  
.card-custom {  
  border-radius: 10px;  
}  
  
.hasil-box {  
  background-color: #1c3e60;  
  border-radius: 15px; padding:  
  17px;  
  color: white;  
}  
  
button {  
  font-weight: bold;  
}
```

2. JAWABAN KODE JS

```
const dayaInput = document.getElementById("daya"); const  
jamInput = document.getElementById("jam");  
  
const hasilKwh = document.getElementById("hasilKwh"); const  
hasilBiaya = document.getElementById("hasilBiaya");
```

```
const rekapText = document.getElementById("rekapText");
const kategori = document.getElementById("kategori");

function hitungListrik() {
    let daya = parseFloat(dayaInput.value) || 0;
    let jam = parseFloat(jamInput.value) || 0;

    let totalKwh = (daya * jam) / 1000;
    let biaya = totalKwh * 1500;

    // Output utama
    hasilKwh.textContent = totalKwh.toFixed(2) + " kWh";
    hasilBiaya.textContent = "Rp " + biaya.toLocaleString("id-ID");

    // Rekap
    rekapText.textContent =
        "Penggunaan listrik sebesar " +
        totalKwh.toFixed(2) +
        " kWh dengan estimasi biaya Rp " +
        biaya.toLocaleString("id-ID") + ".";

    // Kategori
    if (totalKwh === 0) {
        kategori.textContent = "Kategori: -";
        kategori.className = "fw-semibold text-secondary";
    } else if (totalKwh < 5) {
        kategori.textContent = "Kategori: Rendah";
        kategori.className = "fw-semibold text-success";
    } else if (totalKwh < 15) {
        kategori.textContent = "Kategori: Sedang";
        kategori.className = "fw-semibold text-warning";
    } else {
        kategori.textContent = "Kategori: Tinggi";
        kategori.className = "fw-semibold text-danger";
    }
}

// Real-time (tanpa tombol submit)
dayaInput.addEventListener("input", hitungListrik);
jamInput.addEventListener("input", hitungListrik);
```

3. JAWABAN KODE HTML

```
<!DOCTYPE html>
<html lang="id">
<head>
```

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Kalkulator Tagihan Listrik</title>

<link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">

<link rel="stylesheet" href="ProyekMini_KalkulatorTagihanListrik.css">
</head>

<body>

<div class="container py-5">

  <!-- Header -->
  <div class="text-center text-white mb-4">
    <h1 class="fw-bold">Kalkulator Tagihan Listrik</h1>
    <p class="mb-1">Maessa Andrea Vallenia</p>
    <p class="mb-0">NIM: 23051430044</p>
  </div>

  <!-- Card -->
  <div class="card card-custom shadow-lg p-4">

    <!-- Input -->
    <div class="row g-4">
      <div class="col-md-6">
        <label class="form-label fw-semibold">Daya (Watt)</label>
        <input type="number" id="daya" class="form-control form-control-lg" placeholder="Contoh: 900">
      </div>

      <div class="col-md-6">
        <label class="form-label fw-semibold">Jam Pemakaian</label>
        <input type="number" id="jam" class="form-control form-control-lg" placeholder="Contoh: 5">
      </div>
    </div>

    <!-- Hasil -->
    <div class="row mt-4">
      <div class="col-md-6">
        <div class="hasil-box text-center shadow-sm">
          <h5>Total Konsumsi Energi</h5>
          <h3 id="hasilKwh" class="fw-bold text-primary">0 kWh</h3>
        </div>
      </div>
    </div>
  </div>
</div>
```

```
        </div>
      </div>

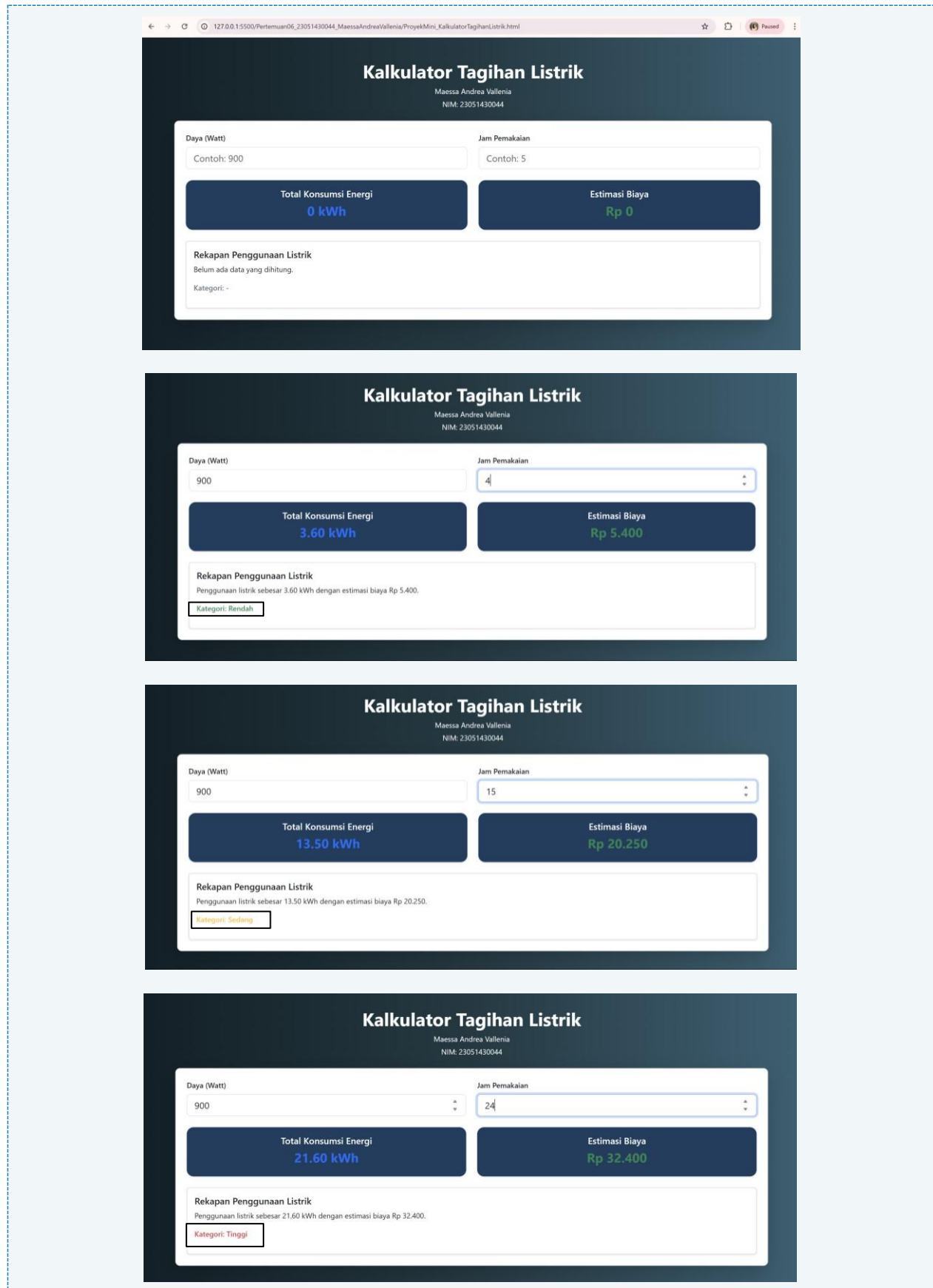
      <div class="col-md-6">
        <div class="hasil-box text-center shadow-sm">
          <h5>Estimasi Biaya</h5>
          <h3 id="hasilBiaya" class="fw-bold text-success">Rp
0</h3>
        </div>
      </div>
</div>

<!-- Rekap -->
<div class="mt-4">
  <div class="card shadow-sm p-3">
    <h5>Rekap Penggunaan Listrik</h5>
    <p id="rekapText">Belum ada data yang dihitung.</p>
    <p id="kategori" class="fw-semibold text-secondary">Kategori:
-</p>
  </div>
</div>

</div>

</div>

<script src="ProyekMini_KalkulatorTagihanListrik.js"></script>
</body>
</html>
```



Gambar F.3 – Hasil Proyek Mini Kalkulator Tagihan Listrik

F. PEMBAHASAN

Pada praktikum ini, mahasiswa mempelajari integrasi HTML, CSS/Bootstrap, dan JavaScript untuk membangun halaman web interaktif yang mensimulasikan panel kontrol mesin serta kalkulator konsumsi listrik berbasis real-time. Praktikum menekankan pemahaman alur interaksi pengguna (user interaction) yang terdiri dari input, event, proses logika, hingga output yang ditampilkan secara dinamis pada antarmuka web. HTML digunakan untuk membangun struktur halaman seperti panel kontrol mesin, tombol aksi (START, STOP, RESET), indikator status, serta form input kalkulator listrik. Sementara itu, Bootstrap dan CSS digunakan untuk memperindah tampilan agar lebih responsif, rapi, dan menyerupai dashboard sistem industri.

JavaScript menjadi inti utama dalam praktikum ini melalui penerapan Document Object Model (DOM), seleksi elemen, dan event listener. DOM memungkinkan JavaScript untuk mengakses dan memanipulasi elemen HTML secara langsung menggunakan metode seperti `getElementById()` dan `querySelector()`. Event listener seperti click dan input digunakan untuk menangkap interaksi pengguna, misalnya saat tombol START ditekan atau saat user menginput nilai daya dan jam pemakaian. Logika program dijalankan berdasarkan event tersebut, sehingga perubahan pada UI dapat terjadi secara dinamis tanpa perlu reload halaman.

Pada simulasi panel kontrol mesin, JavaScript digunakan untuk mengubah status mesin, memperbarui suhu secara real-time menggunakan `setInterval()`, serta memberikan peringatan visual ketika kondisi tertentu terpenuhi, seperti overheating. Selain itu, fitur validasi input RPM juga diterapkan untuk memberikan feedback langsung kepada pengguna apabila nilai melebihi batas aman. Pada proyek mini kalkulator listrik, event input digunakan agar perhitungan kWh dan estimasi biaya dapat ditampilkan secara otomatis tanpa tombol submit. Hal ini menunjukkan konsep interaksi real-time antara pengguna dan sistem.

Secara keseluruhan, praktikum ini memberikan pemahaman mengenai bagaimana DOM, event handling, dan manipulasi elemen dapat digunakan untuk membangun sistem web interaktif yang menyerupai dashboard industri. Mahasiswa tidak hanya memahami konsep teknis JavaScript, tetapi juga bagaimana sistem merespons input pengguna secara langsung, sehingga menghasilkan aplikasi yang lebih responsif, dinamis, dan relevan dengan kebutuhan monitoring serta kontrol dalam lingkungan industri.

F.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum menunjukkan bahwa implementasi program telah sesuai dengan teori Document Object Model (DOM), event listener, dan manipulasi elemen pada JavaScript. Struktur HTML yang dibuat mampu menyediakan elemen-elemen yang dapat diakses oleh JavaScript, seperti tombol kontrol mesin, indikator status, serta input kalkulator listrik. Penggunaan Bootstrap membantu memastikan tampilan tetap responsif dan konsisten, sesuai dengan prinsip user interface pada aplikasi modern.

Dalam penerapan DOM, JavaScript berhasil mengakses elemen HTML menggunakan metode seperti `getElementById()` dan `querySelector()`. Elemen-elemen tersebut kemudian dimanipulasi melalui perubahan properti seperti `innerText`, `className`, dan `style`. Hal ini menunjukkan kesesuaian dengan teori bahwa DOM berfungsi sebagai jembatan antara struktur HTML dan logika JavaScript untuk memungkinkan perubahan tampilan secara dinamis.

Penggunaan event listener juga telah sesuai dengan konsep teori interaksi pengguna, di mana setiap aksi pengguna (seperti klik tombol atau input data) akan memicu event tertentu yang kemudian diproses oleh JavaScript. Pada panel kontrol mesin, event click digunakan untuk menjalankan fungsi START, STOP, dan RESET, sedangkan pada kalkulator listrik, event input digunakan untuk melakukan perhitungan secara real-time tanpa tombol submit. Hal ini sesuai dengan konsep alur User Action → Event → JavaScript Logic → UI Update.

Selain itu, penggunaan `setInterval()` dan `clearInterval()` dalam simulasi suhu mesin menunjukkan penerapan konsep timer dalam JavaScript untuk menjalankan proses berulang secara otomatis. Validasi input RPM juga membuktikan bahwa JavaScript dapat digunakan untuk memberikan feedback langsung kepada pengguna melalui manipulasi class dan tampilan pesan error. Secara keseluruhan, hasil praktikum konsisten dengan teori karena sistem mampu merespons input pengguna, memproses data, dan memperbarui tampilan secara dinamis sesuai dengan konsep DOM dan event handling.

F.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Tombol tidak berfungsi saat diklik	Memastikan ID pada HTML sesuai dengan yang dipanggil di JavaScript serta event listener sudah ditambahkan dengan benar.

F.3 Kaitan dengan Penerapan di Industri

Materi pada praktikum ini memiliki keterkaitan erat dengan penerapan di industri, khususnya dalam pengembangan dashboard monitoring dan sistem kontrol berbasis web. Konsep DOM dan event listener digunakan untuk membangun antarmuka yang mampu merespons kondisi secara real-time, seperti panel kontrol mesin yang menampilkan status operasi, suhu, dan peringatan overhear. Hal ini menyerupai sistem monitoring pada industri manufaktur yang digunakan untuk mengawasi kondisi mesin secara langsung.

Penggunaan event seperti click dan input mencerminkan interaksi operator dengan sistem, misalnya saat menyalakan atau mematikan mesin melalui dashboard digital. Logika JavaScript yang mengatur perubahan status, timer, dan validasi input menunjukkan bagaimana sistem industri bekerja dalam mengolah data secara otomatis berdasarkan kondisi tertentu. Sebagai contoh, peningkatan suhu mesin yang divisualisasikan secara dinamis dapat merepresentasikan sistem peringatan dini (early warning system) pada mesin produksi.

Selain itu, proyek mini kalkulator listrik juga mencerminkan penerapan sederhana sistem perhitungan biaya operasional di industri. Perhitungan konsumsi energi dan estimasi biaya secara real-time dapat digunakan dalam sistem manajemen energi untuk membantu perusahaan memantau efisiensi penggunaan listrik. Dengan demikian, praktikum ini memberikan gambaran bagaimana teknologi web dapat digunakan sebagai bagian dari sistem informasi industri untuk mendukung monitoring, kontrol, dan pengambilan keputusan secara cepat dan akurat.

G. DAFTAR PUSTAKA

Tuliskan 3–5 poin kesimpulan yang merangkum hal-hal yang dipelajari dan dicapai pada praktikum modul ini. Kesimpulan harus spesifik, bukan bersifat umum.

1	Mahasiswa memahami konsep Document Object Model (DOM) sebagai struktur yang memungkinkan JavaScript mengakses dan memanipulasi elemen HTML secara dinamis.
2	Mahasiswa mampu menggunakan event listener seperti click dan input untuk menangani interaksi pengguna dan menjalankan logika program berdasarkan aksi tersebut.
3	Mahasiswa dapat menerapkan manipulasi elemen DOM seperti perubahan teks, warna, dan class untuk memperbarui tampilan antarmuka secara real-time.
4	Mahasiswa memahami penggunaan timer JavaScript (setInterval dan clearInterval) untuk mensimulasikan proses berulang seperti perubahan suhu mesin secara otomatis.
5	Mahasiswa memahami penggunaan timer JavaScript (setInterval dan clearInterval) untuk mensimulasikan proses berulang seperti perubahan suhu mesin secara otomatis.

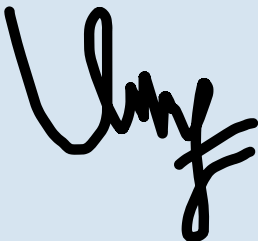
H. DAFTAR PUSTAKA

Tuliskan semua sumber referensi yang digunakan dalam format APA 7th Edition. Minimal 3 referensi yang relevan (termasuk Manual Praktikum dan sumber online resmi seperti MDN, Reactjs.org, dll).

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Kom, I. L. M. (2023). <i>Dasar Algoritma dan Pemograman Javascript</i> . Irvan Lewenusa, M. Kom.
[3]	Sholikhan, M. (2022). Html, Css Dan Javascript. <i>Penerbit Yayasan Prima Agus Teknik</i> , 1-343 .
[4]	https://www.javascripttutorial.net/
[5]	https://developer.mozilla.org/en-US/docs/Web/JavaScript
[6]	https://www.geeksforgeeks.org/
[7]	https://blog.nurturetalentiva.com/2025/09/01/dom-manipulation-getelementbyid-queryselector/

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Diperiksa oleh Asisten / Dosen
 _____ Maessa Andrea Vallenia NIM: 23051430044	 _____ Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____

MODUL 7

FORM VALIDATION & LOCAL STORAGE

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 7 (Pertemuan 7)
Topik Utama	Validasi form tingkat lanjut dan penyimpanan data persisten di browser (Client-side Storage).
Sub-Topik	Konsep Form Validation, Jenis Validasi (HTML & JavaScript), Event Handling pada Form (submit, input), Web Storage API, LocalStorage sebagai Client-side Storage, Format Data JSON (JSON.stringify & JSON.parse), Operasi CRUD pada LocalStorage, Manipulasi DOM untuk Menampilkan Data, Pengelolaan Data dengan Array of Object, Filtering dan Sorting Data (filter & sort), Implementasi Aplikasi Log Produksi Berbasis Web.
Durasi Praktikum	340 Menit
Tanggal Praktikum	10 April 2026
Nama Mahasiswa	Maessa Andrea Vallenia
NIM	23051430044
Kelas	A

B. TUJUAN PEMBELAJARAN

Tuliskan tujuan pembelajaran yang ingin dicapai pada modul ini, sesuai dengan yang tertera di Manual Praktikum.

1	Menerapkan validasi form sebelum data dikirim untuk mencegah data kotor.
2	Memahami cara kerja Web Storage API (LocalStorage).
3	Mampu menyimpan, mengambil, mengupdate, dan menghapus (CRUD sederhana) data di LocalStorage.
4	Membuat aplikasi "Buku Harian Produksi" sederhana yang datanya tidak hilang saat browser direfresh.

C. ALAT DAN BAHAN

Sebutkan seluruh perangkat keras, perangkat lunak, dan sumber referensi yang digunakan selama praktikum.

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	<i>Code Editor</i>
2	Google Chrome / Browser Modern	Versi terbaru	<i>Testing & Debugging</i>
3	Live Server	Versi 5.7.9	Menjalankan HTML dan memantau perubahan file
4	JavaScript (Built-in Browser)	ES6 (Default Browser Modern)	Bahasa pemrograman untuk logika dan perhitungan
5	CodeSnap	Versi 1.3.4	Dokumentasi kode program
6	Git dan GitHub	Versi 2.53.0	Version control system (VCS) berbasis Git
7	Koneksi Internet	Stabil	Mengakses Library dan Framework
8	Console Browser (Developer Tools)	Default Browser	Melihat output program dan debugging
9	Bootstrap	Versi 5.3	Membantu pembuatan UI seperti button, card, dan alert pada panel control
10	File praktikum	Pertemuan sebelumnya	Digunakan sebagai dasar pengembangan panel kontrol
11	LocalStorage (Web Storage API)	Built-in Browser	Digunakan untuk menyimpan data produksi

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
			secara persisten

D. DASAR TEORI

1. Form Validation merupakan proses pengecekan data input dari pengguna sebelum data tersebut diproses atau disimpan ke dalam sistem. Validasi ini bertujuan untuk memastikan bahwa data yang dimasukkan sesuai dengan aturan yang telah ditentukan, sehingga dapat mencegah kesalahan input (error) dan menjaga kualitas data. Validasi dapat dilakukan menggunakan atribut bawaan HTML seperti *required*, *min*, dan *type*, serta menggunakan JavaScript untuk validasi yang lebih kompleks seperti pengecekan logika nilai.
2. Event handling pada form digunakan untuk menangani interaksi pengguna, terutama melalui event seperti *submit* dan *input*. Event *submit* digunakan untuk memproses data saat form dikirim, sedangkan event *input* memungkinkan sistem merespons perubahan data secara real-time. Dengan adanya event handling, proses validasi dan pengolahan data dapat dilakukan secara interaktif tanpa perlu me-refresh halaman.
3. Web Storage API merupakan fitur pada browser yang memungkinkan penyimpanan data di sisi client (client-side). Salah satu implementasinya adalah LocalStorage, yang digunakan untuk menyimpan data secara persisten sehingga data tetap tersedia meskipun halaman di-refresh atau browser ditutup. LocalStorage menyimpan data dalam bentuk pasangan *key-value* dengan format string dan memiliki kapasitas sekitar 5–10 MB.
4. Karena LocalStorage hanya dapat menyimpan data dalam bentuk string, maka digunakan format JSON (JavaScript Object Notation) untuk mengubah data object atau array menjadi string menggunakan *JSON.stringify()*, dan mengembalikannya menjadi object menggunakan *JSON.parse()*. Teknik ini memungkinkan penyimpanan data kompleks seperti array of object dalam LocalStorage.
5. Operasi CRUD (Create, Read, Update, Delete) merupakan konsep dasar dalam pengelolaan data. Pada LocalStorage, operasi Create dilakukan dengan menyimpan data baru, Read dengan mengambil data, Update dengan memperbarui data yang sudah ada, dan Delete dengan menghapus data tertentu atau seluruh data. Proses ini sering digunakan dalam aplikasi sederhana seperti pencatatan log produksi.
6. Manipulasi DOM digunakan untuk menampilkan data yang tersimpan di LocalStorage ke dalam halaman web, misalnya dalam bentuk tabel. Dengan JavaScript, elemen HTML dapat dibuat, diubah, dan diperbarui secara dinamis menggunakan properti seperti *innerHTML*. Data yang dikelola biasanya disimpan dalam bentuk array of object agar lebih terstruktur dan mudah diproses menggunakan looping.

Selain itu, fitur filtering dan sorting digunakan untuk meningkatkan interaksi pengguna dalam mengelola data. Method *filter()* digunakan untuk mencari data tertentu berdasarkan kriteria, sedangkan *sort()* digunakan untuk mengurutkan data, misalnya berdasarkan jumlah produksi. Kedua fitur ini penting dalam sistem informasi untuk memudahkan analisis data.

Secara keseluruhan, konsep-konsep ini diimplementasikan dalam aplikasi log produksi berbasis web yang memungkinkan pengguna untuk memasukkan, menyimpan, menampilkan, dan mengelola data produksi secara interaktif. Sistem ini mencerminkan penerapan nyata dalam dunia industri, khususnya dalam pencatatan dan monitoring data produksi secara digital.

D.1 Konsep Utama

Konsep utama pada materi ini berfokus pada penggunaan JavaScript dalam melakukan validasi form serta penyimpanan data secara persisten menggunakan LocalStorage. Dengan memanfaatkan konsep tersebut, halaman web tidak hanya mampu menerima input dari pengguna, tetapi juga dapat menyimpan dan mengelola data secara berkelanjutan tanpa bergantung pada database eksternal.

Form validation digunakan untuk memastikan bahwa data yang dimasukkan oleh pengguna sesuai dengan aturan yang telah ditentukan sebelum data tersebut diproses atau disimpan. Validasi dapat dilakukan menggunakan atribut HTML seperti *required* dan *min*, serta menggunakan JavaScript untuk validasi yang lebih kompleks. Dengan adanya validasi, kesalahan input dapat diminimalkan sehingga kualitas data menjadi lebih baik.

LocalStorage merupakan bagian dari Web Storage API yang digunakan untuk menyimpan data di sisi client (browser). Data yang disimpan akan tetap ada meskipun halaman direfresh atau browser ditutup. Karena LocalStorage hanya dapat menyimpan data dalam bentuk string, maka digunakan format JSON untuk mengubah data object menjadi string menggunakan *JSON.stringify()* dan mengembalikannya menjadi object menggunakan *JSON.parse()*.

Selain itu, konsep CRUD (Create, Read, Update, Delete) diterapkan dalam pengelolaan data di LocalStorage. Data yang dimasukkan melalui form akan disimpan (create), ditampilkan kembali dalam tabel (read), dapat diperbarui (update), dan dihapus baik sebagian maupun seluruhnya (delete). Proses ini biasanya menggunakan struktur data array of object agar data lebih terorganisir dan mudah diproses.

Manipulasi DOM juga berperan penting dalam menampilkan data ke dalam halaman web secara dinamis. Data yang tersimpan di LocalStorage dapat ditampilkan dalam bentuk tabel, serta diperbarui secara langsung ketika terjadi perubahan tanpa perlu me-refresh halaman. Dengan menggabungkan validasi form, LocalStorage, JSON, dan manipulasi DOM, aplikasi web menjadi lebih interaktif, responsif, dan mampu menyimpan data secara mandiri.

D.2 Keterkaitan dengan Teknik Industri

Materi JavaScript pada pertemuan ini memiliki keterkaitan yang erat dengan bidang Teknik Industri, khususnya dalam pengelolaan data produksi, pencatatan aktivitas operasional, serta sistem monitoring berbasis digital. Dalam dunia industri, pencatatan data seperti jumlah produksi, shift kerja, dan operator sangat penting untuk analisis dan pengambilan keputusan.

Konsep form validation digunakan untuk memastikan bahwa data yang dimasukkan oleh operator sesuai dengan standar yang telah ditentukan, seperti jumlah produksi tidak boleh nol atau negatif. Hal ini penting untuk menjaga keakuratan data dan mencegah kesalahan dalam proses pencatatan.

Penggunaan LocalStorage mencerminkan sistem pencatatan data sederhana yang bersifat persisten, di mana data tetap tersimpan meskipun sistem dimuat ulang. Dalam konteks industri, hal ini mirip dengan sistem log produksi atau pencatatan aktivitas harian yang digunakan untuk monitoring dan evaluasi kinerja.

Konsep CRUD pada LocalStorage berkaitan dengan proses pengelolaan data dalam sistem informasi industri, seperti menambahkan data produksi baru, menampilkan riwayat produksi, memperbarui data jika terjadi kesalahan, serta menghapus data yang tidak

diperlukan. Struktur array of object memungkinkan data disimpan secara terorganisir, mirip dengan database sederhana.

Manipulasi DOM digunakan untuk menampilkan data dalam bentuk tabel yang mudah dibaca dan dianalisis oleh pengguna. Dengan tampilan yang dinamis, pengguna dapat melihat perubahan data secara langsung tanpa perlu memuat ulang halaman, sehingga meningkatkan efisiensi dan kenyamanan dalam penggunaan sistem.

Dengan memahami konsep validasi form dan LocalStorage, mahasiswa Teknik Industri dapat mengembangkan aplikasi berbasis web yang mampu menyimpan dan mengelola data produksi secara mandiri. Hal ini mendukung proses digitalisasi di industri, meningkatkan efisiensi pencatatan data, serta membantu dalam pengambilan keputusan berbasis data.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

Dokumentasikan setiap langkah kerja yang Anda lakukan, beserta hasil yang diperoleh (screenshot, kode, atau output). Jawab sesuai urutan langkah di Manual Praktikum.

E.1 Langkah 1 – Membuat Form Input Produksi

Buat file baru `log\_produk.html`. Kita buat form sederhana.

```
/* ===== KODE / OUTPUT LANGKAH 1 ===== */

<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Log Produksi Harian</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body class="container mt-5">
  <div class="row">
    <!-- Kolom Kiri: Form Input -->
    <div class="col-md-4">
      <div class="card">
        <div class="card-header">Input Data</div>
        <div class="card-body">
          <form id="formProduksi">
            <div class="mb-3">
              <label>Tanggal</label>
              <input type="date" id="tanggal" class="form-
control" required>
            </div>
            <div class="mb-3">
              <label>Nama Operator</label>
              <input type="text" id="operator" class="form-
control" placeholder="Nama Lengkap" required>

```

```

        </div>
        <div class="mb-3">
            <label>Shift</label>
            <select id="shift" class="form-select" required>
                <option value="Pagi">Pagi</option>
                <option value="Siang">Siang</option>
                <option value="Malam">Malam</option>
            </select>
        </div>
        <div class="mb-3">
            <label>Jumlah Produksi (Unit)</label>
            <input type="number" id="jumlah" class="form-
control" min="1" required>
        </div>
        <button type="submit" class="btn btn-primary w-
100">Simpan
            Log</button>
        <button type="button" id="btnHapusSemua" class="btn
btn-danger
w-100 mt-2">Hapus Semua Data</button>
    </form>
</div>
</div>
</div>

<!-- Kolom Kanan: Tabel Data -->
<div class="col-md-8">
    <h3>Riwayat Produksi</h3>
    <table class="table table-bordered table-striped">
        <thead class="table-dark">
            <tr>
                <th>Tanggal</th>
                <th>Operator</th>
                <th>Shift</th>
                <th>Jumlah</th>
                <th>Aksi</th>
            </tr>
        </thead>
        <tbody id="tabelBody">
            <!-- Data akan masuk di sini lewat JS -->
        </tbody>
    </table>
</div>
</div>

<script src="script_log.js"></script>
</body>
</html>

```

Screenshot Hasil:

The screenshot shows a web browser window with the URL `127.0.0.1:5500/Pertemuan07_23051430044_MaessaAndreaVallenia/log_produksi.html`. The page contains two main sections:

Input Data

Tanggal
dd/mm/yyyy

Nama Operator
Nama Lengkap

Shift
Pagi

Jumlah Produksi (Unit)

Simpan Log

Hapus Semua Data

Riwayat Produksi

Tanggal	Operator	Shift	Jumlah	Aksi
---------	----------	-------	--------	------

Gambar E.1 – Hasil Membuat Form Input Produksi

E.2 Langkah 2 – Logika JavaScript (Simpan & Baca)

Buat file ``script_log.js``.

```

/* ===== KODE / OUTPUT LANGKAH 2 ===== */

// 1. Seleksi Elemen
const formProduksi = document.getElementById('formProduksi');
const tabelBody = document.getElementById('tabelBody');
const btnHapusSemua = document.getElementById('btnHapusSemua');

// Kunci untuk LocalStorage
const STORAGE_KEY = 'DATA_PRODUKSI_INDUSTRI';

// Fungsi Load Data saat halaman dibuka
document.addEventListener('DOMContentLoaded', function() {
  loadDataFromStorage();
});

// 2. Event Listener: Submit Form
formProduksi.addEventListener('submit', function(event) {
  event.preventDefault(); // Mencegah refresh halaman

  // Ambil Value dari Form
  const tanggal = document.getElementById('tanggal').value;
  const operator = document.getElementById('operator').value;
  const shift = document.getElementById('shift').value;
  const jumlah = document.getElementById('jumlah').value;

```

```
// Validasi Sederhana (JavaScript)
if (jumlah <= 0) {
    alert("Jumlah produksi harus lebih dari 0!");
    return;
}

// Buat Object Data
const dataBaru = {
    id: Date.now(), // ID unik berdasarkan waktu
    tanggal: tanggal,
    operator: operator,
    shift: shift,
    jumlah: parseInt(jumlah)
};

// Simpan ke LocalStorage
saveData(dataBaru);

// Reset Form
formProduksi.reset();

// Refresh Tampilan Tabel
loadDataFromStorage();
});

// 3. Fungsi Simpan ke LocalStorage
function saveData(data) {
    // Ambil data lama (jika ada), jika tidak ada array kosong
    let dataLama = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

    // Tambah data baru ke array
    dataLama.push(data);

    // Simpan kembali ke LocalStorage (Convert ke JSON String)
    localStorage.setItem(STORAGE_KEY, JSON.stringify(dataLama));
}

// 4. Fungsi Baca & Render Tabel
function loadDataFromStorage() {
    // Ambil data
    let data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

    // Kosongkan tabel dulu
    tabelBody.innerHTML = '';

    // Loop data dan buat baris tabel
    data.forEach(function(item) {
        const row = document.createElement('tr');
```

```
        row.innerHTML = `
            <td>${item.tanggal}</td>
            <td>${item.operator}</td>
            <td>${item.shift}</td>
            <td>${item.jumlah}</td>
            <td>
                <button class="btn btn-sm btn-danger"
                onclick="hapusData(${item.id})">Hapus</button>
            </td>
        `;
        tabelBody.appendChild(row);
    });
}

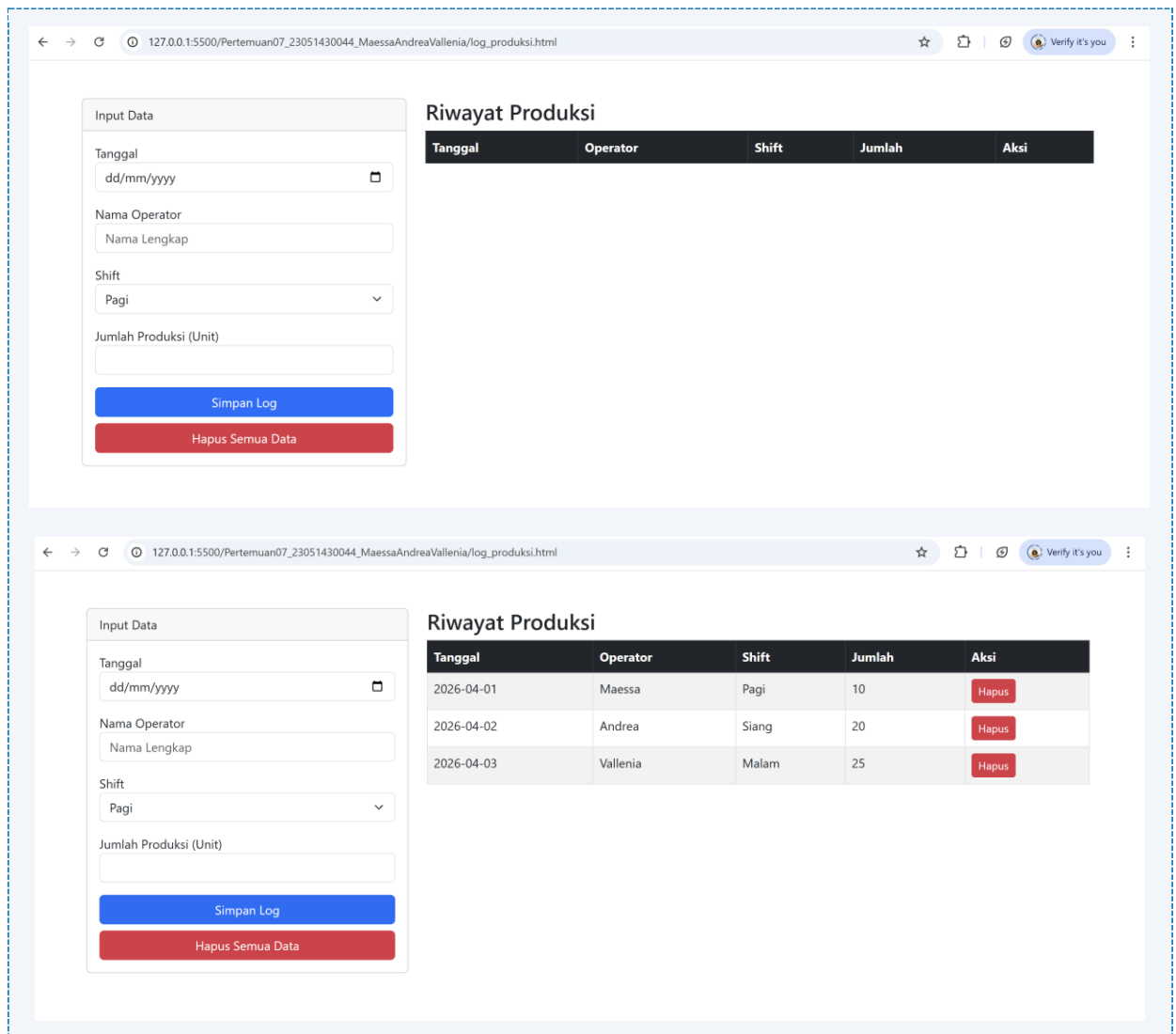
// 5. Fungsi Hapus Data Spesifik
// Kita pasang di window agar bisa dipanggil dari inline HTML onclick
window.hapusData = function(id) {
    if(confirm('Yakin ingin menghapus log ini?')) {
        let data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

        // Filter data: hapus item yang id-nya cocok
        let dataBaru = data.filter(item => item.id !== id);

        // Simpan ulang
        localStorage.setItem(STORAGE_KEY, JSON.stringify(dataBaru));

        // Refresh tampilan
        loadDataFromStorage();
    }
}

// 6. Event Hapus Semua
btnHapusSemua.addEventListener('click', function() {
    if(confirm('PERINGATAN: Semua data akan dihapus permanen!')) {
        localStorage.removeItem(STORAGE_KEY);
        loadDataFromStorage();
    }
});
```

Screenshot Hasil:

Gambar E.2 – Hasil Logika JavaScript (Simpan & Baca)

F. LATIHAN DAN TUGAS MANDIRI

Jawab dan dokumentasikan seluruh latihan/tugas yang ada di bagian "Latihan/Tugas Mandiri" pada modul Anda.

F.1 Latihan 1 – Filter Data

Pertanyaan / Instruksi Latihan:

Tambahkan fitur filter atau pencarian. Buat input "Cari Nama Operator". Saat user mengetik, tabel hanya menampilkan data operator yang namanya cocok (Gunakan `filter` array).

Jawaban / Kode Solusi (JS & HTML)

1. JAWABAN KODE CSS

```
/* Import font */
@import
url('https://fonts.googleapis.com/css2?family=Roboto:wght@400;700&display=swap');

```

```
body {
  font-family: 'Roboto', sans-serif;
  background-color: #f4f6f9;
}

h3 {
  margin-bottom: 20px;
  font-weight: 700;
}

/* Card form */
.card {
  border-radius: 10px;
  box-shadow: 0 2px 8px rgba(0,0,0,0.1);
}

/* Table */
table {
  background-color: white;
  border-radius: 8px;
  overflow: hidden;
}

/* Hover row */
table tbody tr:hover {
  background-color: #f1f1f1;
}

/* Button full width spacing */
button {
  font-weight: 500;
}

/* Input search */
#cariOperator {
  border-radius: 8px;
  padding: 10px;
}

/* Tombol danger */
.btn-danger {
  border-radius: 6px;
}
```

2. JAWABAN KODE JS

```
// 1. Seleksi Elemen
const formProduksi = document.getElementById('formProduksi');
```

```
const tabelBody = document.getElementById('tabelBody');
const btnHapusSemua = document.getElementById('btnHapusSemua');
const cariOperator = document.getElementById('cariOperator');

// Kunci LocalStorage
const STORAGE_KEY = 'DATA_PRODUKSI_INDUSTRI';

// Load saat halaman dibuka
document.addEventListener('DOMContentLoaded', function () {
  loadDataFromStorage();
});

// 2. Submit Form
formProduksi.addEventListener('submit', function (event) {
  event.preventDefault();

  const tanggal = document.getElementById('tanggal').value;
  const operator = document.getElementById('operator').value;
  const shift = document.getElementById('shift').value;
  const jumlah = document.getElementById('jumlah').value;

  if (jumlah <= 0) {
    alert("Jumlah produksi harus lebih dari 0!");
    return;
  }

  const dataBaru = {
    id: Date.now(),
    tanggal: tanggal,
    operator: operator,
    shift: shift,
    jumlah: parseInt(jumlah)
  };

  saveData(dataBaru);
  formProduksi.reset();
  loadDataFromStorage();
});

// 3. Simpan ke LocalStorage
function saveData(data) {
  let dataLama = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];
  dataLama.push(data);
  localStorage.setItem(STORAGE_KEY, JSON.stringify(dataLama));
}

// 4. Render Data (FULL + FILTER SUPPORT)
function loadDataFromStorage(dataFilter = null) {
```



```
let data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

// Jika ada data filter → gunakan itu
let dataTampil = dataFilter || data;

tabelBody.innerHTML = '';

dataTampil.forEach(function (item) {
    const row = document.createElement('tr');

    row.innerHTML = `
        <td>${item.tanggal}</td>
        <td>${item.operator}</td>
        <td>${item.shift}</td>
        <td>${item.jumlah}</td>
        <td>
            <button class="btn btn-sm btn-danger"
onclick="hapusData(${item.id})">Hapus</button>
        </td>
    `;

    tabelBody.appendChild(row);
});

}

// 5. Hapus Data
window.hapusData = function (id) {
    if (confirm('Yakin ingin menghapus log ini?')) {

        let data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

        let dataBaru = data.filter(item => item.id !== id);

        localStorage.setItem(STORAGE_KEY, JSON.stringify(dataBaru));

        loadDataFromStorage();

    }
}

// 6. Hapus Semua
btnHapusSemua.addEventListener('click', function () {
    if (confirm('PERINGATAN: Semua data akan dihapus permanen!')) {
        localStorage.removeItem(STORAGE_KEY);
        loadDataFromStorage();
    }
});

// 7. FILTER (LATIHAN 1)
cariOperator.addEventListener('input', function () {
```

```
let keyword = this.value.toLowerCase();

let data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

let hasilFilter = data.filter(item =>
    item.operator.toLowerCase().includes(keyword)
);

loadDataFromStorage(hasilFilter);
});
```

3. JAWABAN KODE HTML

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Log Produksi Harian</title>
  <link
href="https://cdn.jsdelivrivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css
" rel="stylesheet">
</head>

<body class="container mt-5">

  <div class="row">

    <!-- FORM -->
    <div class="col-md-4">
      <div class="card">
        <div class="card-header">Input Data</div>
        <div class="card-body">
          <form id="formProduksi">

            <div class="mb-3">
              <label>Tanggal</label>
              <input type="date" id="tanggal" class="form-
control" required>
            </div>

            <div class="mb-3">
              <label>Nama Operator</label>
              <input type="text" id="operator" class="form-
control" placeholder="Nama Lengkap" required>
            </div>

            <div class="mb-3">
```

```

        <label>Shift</label>
        <select id="shift" class="form-select" required>
            <option value="Pagi">Pagi</option>
            <option value="Siang">Siang</option>
            <option value="Malam">Malam</option>
        </select>
    </div>

    <div class="mb-3">
        <label>Jumlah Produksi</label>
        <input type="number" id="jumlah" class="form-
control" min="1" required>
    </div>

    <button type="submit" class="btn btn-primary w-
100">Simpan Log</button>
    <button type="button" id="btnHapusSemua" class="btn
btn-danger w-100 mt-2">Hapus Semua Data</button>

    </form>
</div>
</div>
</div>

<!-- TABEL -->
<div class="col-md-8">
    <h3>Riwayat Produksi</h3>

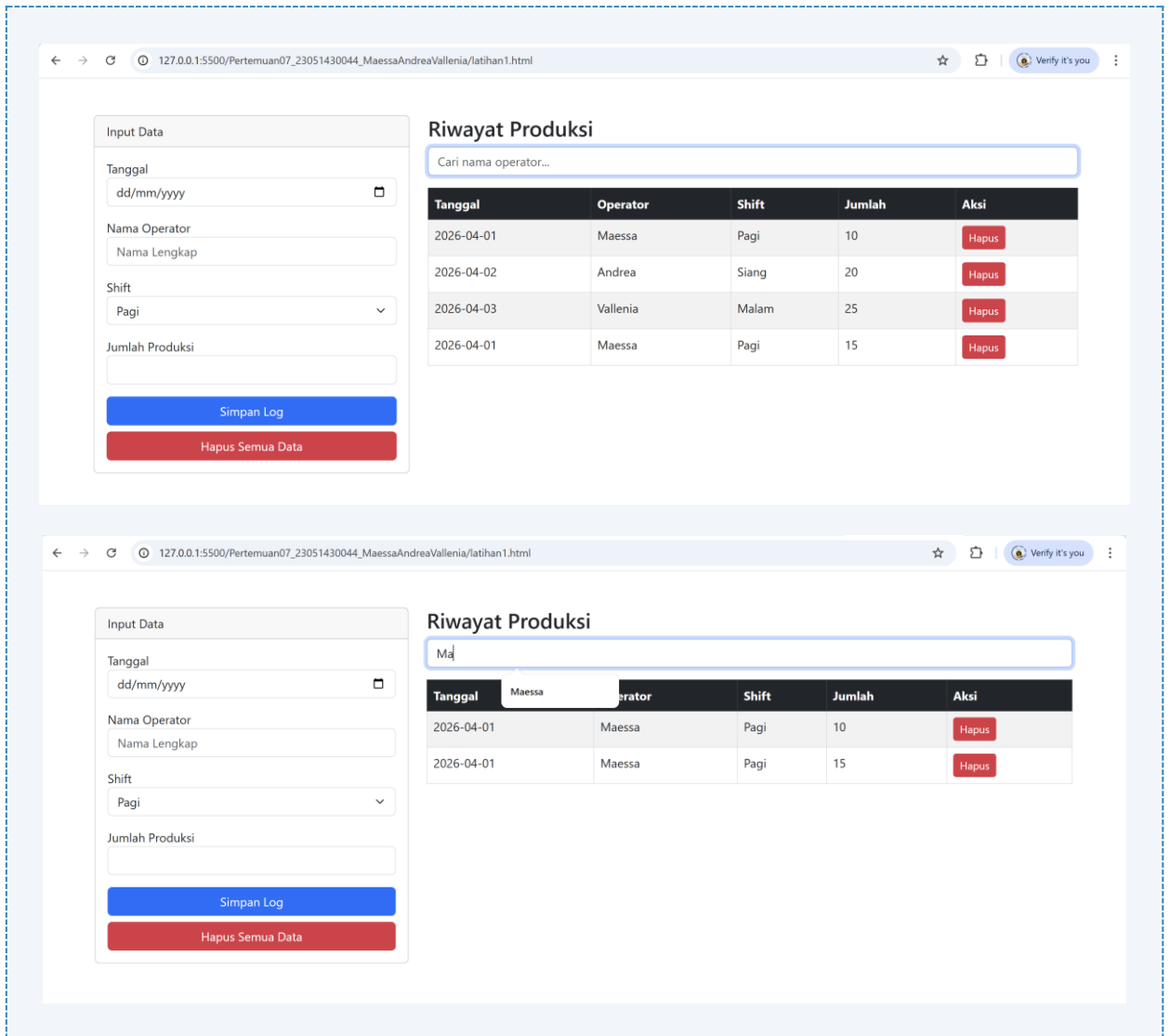
    <!-- ☒ FILTER LATIHAN 1 -->
    <div class="mb-3">
        <input type="text" id="cariOperator" class="form-control"
placeholder="Cari nama operator...">
    </div>

    <table class="table table-bordered table-striped">
        <thead class="table-dark">
            <tr>
                <th>Tanggal</th>
                <th>Operator</th>
                <th>Shift</th>
                <th>Jumlah</th>
                <th>Aksi</th>
            </tr>
        </thead>
        <tbody id="tabelBody"></tbody>
    </table>
</div>

</div>

```

```
<script src="latihan1.js"></script>
</body>
</html>
```



Gambar F.1 – Hasil Filter Data

F.2 Latihan 2 – Sortir

Pertanyaan / Instruksi Latihan:

Tambahkan tombol "Sortir berdasarkan Jumlah (Terbesar)" yang mengurutkan tabel dari jumlah produksi terbanyak ke terkecil.

Jawaban / Kode Solusi:

1. JAWABAN KODE CSS

```
/* Import font */
```

```
@import
url('https://fonts.googleapis.com/css2?family=Roboto:wght@400;700&display=swap');

body {
  font-family: 'Roboto', sans-serif;
  background-color: #f4f6f9;
}

h3 {
  margin-bottom: 20px;
  font-weight: 700;
}

/* Card form */
.card {
  border-radius: 10px;
  box-shadow: 0 2px 8px rgba(0,0,0,0.1);
}

/* Table */
table {
  background-color: white;
  border-radius: 8px;
  overflow: hidden;
}

/* Hover row */
table tbody tr:hover {
  background-color: #f1f1f1;
}

/* Button full width spacing */
button {
  font-weight: 500;
}

/* Input search */
#cariOperator {
  border-radius: 8px;
  padding: 10px;
}

/* Tombol danger */
.btn-danger {
  border-radius: 6px;
}
```

2. JAWABAN KODE JS

```
// 1. Seleksi Elemen
const formProduksi = document.getElementById('formProduksi');
const tabelBody = document.getElementById('tabelBody');
const btnHapusSemua = document.getElementById('btnHapusSemua');
const cariOperator = document.getElementById('cariOperator');

// Kunci LocalStorage
const STORAGE_KEY = 'DATA_PRODUKSI_INDUSTRI';

// Load saat halaman dibuka
document.addEventListener('DOMContentLoaded', function () {
    loadDataFromStorage();
});

// 2. Submit Form
formProduksi.addEventListener('submit', function (event) {
    event.preventDefault();

    const tanggal = document.getElementById('tanggal').value;
    const operator = document.getElementById('operator').value;
    const shift = document.getElementById('shift').value;
    const jumlah = document.getElementById('jumlah').value;

    if (jumlah <= 0) {
        alert("Jumlah produksi harus lebih dari 0!");
        return;
    }

    const dataBaru = {
        id: Date.now(),
        tanggal: tanggal,
        operator: operator,
        shift: shift,
        jumlah: parseInt(jumlah)
    };

    saveData(dataBaru);
    formProduksi.reset();
    loadDataFromStorage();
});

// 3. Simpan ke LocalStorage
function saveData(data) {
    let dataLama = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];
    dataLama.push(data);
    localStorage.setItem(STORAGE_KEY, JSON.stringify(dataLama));
}
```

```
// 4. Render Data (FULL + FILTER SUPPORT)
function loadDataFromStorage(dataFilter = null) {

    let data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

    // Jika ada data filter → gunakan itu
    let dataTampil = dataFilter || data;

    tabelBody.innerHTML = '';

    dataTampil.forEach(function (item) {
        const row = document.createElement('tr');

        row.innerHTML = `
            <td>${item.tanggal}</td>
            <td>${item.operator}</td>
            <td>${item.shift}</td>
            <td>${item.jumlah}</td>
            <td>
                <button class="btn btn-sm btn-danger"
onclick="hapusData(${item.id})">Hapus</button>
            </td>
        `;

        tabelBody.appendChild(row);
    });
}

// 5. Hapus Data
window.hapusData = function (id) {
    if (confirm('Yakin ingin menghapus log ini?')) {

        let data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

        let dataBaru = data.filter(item => item.id !== id);

        localStorage.setItem(STORAGE_KEY, JSON.stringify(dataBaru));

        loadDataFromStorage();
    }
}

// 6. Hapus Semua
btnHapusSemua.addEventListener('click', function () {
    if (confirm('PERINGATAN: Semua data akan dihapus permanen!')) {
        localStorage.removeItem(STORAGE_KEY);
        loadDataFromStorage();
    }
});
```

```
// 7. FILTER (LATIHAN 1)
cariOperator.addEventListener('input', function () {

    let keyword = this.value.toLowerCase();

    let data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

    let hasilFilter = data.filter(item =>
        item.operator.toLowerCase().includes(keyword)
    );

    loadDataFromStorage(hasilFilter);
});

// 8. SORTIR (LATIHAN 2)
const btnSort = document.getElementById('btnSort');

btnSort.addEventListener('click', function () {

    let data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

    // Sorting dari terbesar ke terkecil
    let dataSort = data.sort((a, b) => b.jumlah - a.jumlah);

    loadDataFromStorage(dataSort);
});
```

3. JAWABAN KODE HTML

```
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Log Produksi Harian</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>

<body class="container mt-5">

    <div class="row">

        <!-- FORM -->
        <div class="col-md-4">
            <div class="card">
                <div class="card-header">Input Data</div>
                <div class="card-body">
```



```

        <form id="formProduksi">

            <div class="mb-3">
                <label>Tanggal</label>
                <input type="date" id="tanggal" class="form-
control" required>
            </div>

            <div class="mb-3">
                <label>Nama Operator</label>
                <input type="text" id="operator" class="form-
control" placeholder="Nama Lengkap" required>
            </div>

            <div class="mb-3">
                <label>Shift</label>
                <select id="shift" class="form-select" required>
                    <option value="Pagi">Pagi</option>
                    <option value="Siang">Siang</option>
                    <option value="Malam">Malam</option>
                </select>
            </div>

            <div class="mb-3">
                <label>Jumlah Produksi</label>
                <input type="number" id="jumlah" class="form-
control" min="1" required>
            </div>

            <button type="submit" class="btn btn-primary w-
100">Simpan Log</button>
            <button type="button" id="btnHapusSemua" class="btn
btn-danger w-100 mt-2">Hapus Semua Data</button>

        </form>
    </div>
</div>

<!-- TABEL -->
<div class="col-md-8">
    <h3>Riwayat Produksi</h3>

    <!-- FILTER LATIHAN 1 -->
    <div class="mb-3">
        <input type="text" id="cariOperator" class="form-control"
placeholder="Cari nama operator...">
    </div>

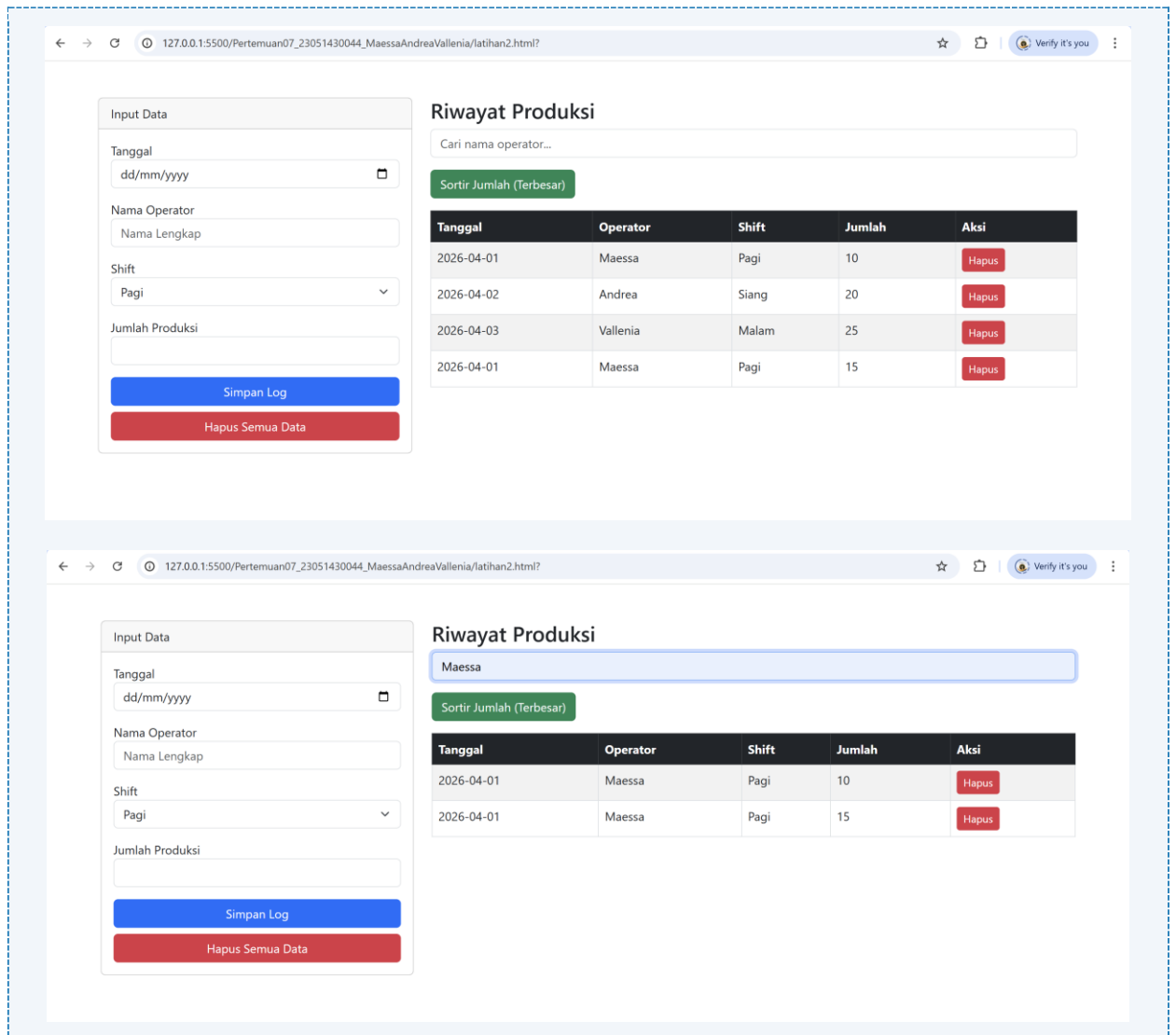
    <!-- FILTER LATIHAN 2 -->

```

```
<button id="btnSort" class="btn btn-success mb-3">
  Sortir Jumlah (Terbesar)
</button>
<table class="table table-bordered table-striped">
  <thead class="table-dark">
    <tr>
      <th>Tanggal</th>
      <th>Operator</th>
      <th>Shift</th>
      <th>Jumlah</th>
      <th>Aksi</th>
    </tr>
  </thead>
  <tbody id="tabelBody"></tbody>
</table>
</div>

</div>

<script src="latihan2.js"></script>
</body>
</html>
```



Gambar F.2 – Hasil Sortir

F.3 Tugas Proyek Mini – Aplikasi Checklist 5S

Deskripsi proyek mini:

- Buat aplikasi checklist sederhana audit 5S (Seiri, Seiton, Seiso, Seiketsu, Shitsuke).
- Form berisi 5 checkbox (boolean) dan input nama auditor.
 - Saat disimpan, hitung skor (jumlah checklist dicentang / 5 * 100).
 - Simpan riwayat audit ke LocalStorage dan tampilkan di tabel (Tanggal, Auditor, Skor)

Penjelasan pendekatan/solusi yang digunakan:

Proyek mini ini dilakukan melalui beberapa tahapan yang sistematis, dimulai dari perancangan tampilan hingga implementasi logika penyimpanan data menggunakan LocalStorage.

- Tahap pertama adalah menyiapkan lingkungan kerja dengan membuat folder proyek, kemudian membuat file utama yang terdiri dari file HTML sebagai tampilan, file JavaScript sebagai logika program, serta file CSS (opsional) untuk pengaturan tampilan.

2. Selanjutnya, dilakukan pembuatan struktur antarmuka (user interface) menggunakan HTML. Tampilan terdiri dari form input nama auditor, lima checkbox yang merepresentasikan prinsip 5S (Seiri, Seiton, Seiso, Seiketsu, dan Shitsuke), tombol untuk menyimpan data, serta tabel untuk menampilkan riwayat hasil audit.
3. Setiap elemen pada form dan tabel kemudian diberikan atribut id agar dapat diakses dan dimanipulasi menggunakan JavaScript. Hal ini penting untuk menghubungkan antara tampilan dan logika program.
4. Setelah itu, file JavaScript dihubungkan ke dalam file HTML agar fungsi-fungsi yang dibuat dapat dijalankan saat halaman dibuka.
5. Pada tahap berikutnya, dilakukan pengambilan elemen HTML menggunakan DOM (Document Object Model), sehingga data yang dimasukkan oleh pengguna dapat dibaca dan diproses oleh program.
6. Kemudian ditambahkan event listener pada form, khususnya pada saat tombol simpan ditekan. Event ini berfungsi untuk menangkap aksi pengguna dan menjalankan proses penyimpanan data tanpa perlu melakukan reload halaman.
7. Data yang diinput oleh pengguna kemudian diambil, yaitu nama auditor dan status dari masing-masing checkbox (apakah dicentang atau tidak).
8. Berdasarkan data tersebut, dilakukan perhitungan skor audit dengan cara menghitung jumlah checkbox yang dicentang, kemudian dibagi dengan total checklist (5) dan dikalikan 100 untuk mendapatkan nilai dalam bentuk persentase.
9. Setelah skor dihitung, data audit disusun dalam bentuk objek yang berisi informasi seperti id unik, tanggal audit, nama auditor, dan nilai skor.
10. Data tersebut kemudian disimpan ke dalam LocalStorage dengan terlebih dahulu dikonversi ke dalam format JSON. Hal ini dilakukan agar data dapat disimpan secara permanen di browser.
11. Selanjutnya, dibuat fungsi untuk mengambil kembali data dari LocalStorage, kemudian menampilkannya ke dalam tabel riwayat audit. Proses ini dilakukan menggunakan perulangan sehingga seluruh data dapat ditampilkan secara otomatis.
12. Sistem juga dilengkapi dengan fitur penghapusan data, baik untuk menghapus satu data tertentu maupun seluruh data yang tersimpan, sehingga pengguna dapat mengelola data dengan lebih fleksan
13. Terakhir, dilakukan proses pengujian aplikasi dengan cara menginput data audit, memastikan skor yang dihasilkan sesuai, melakukan refresh halaman untuk mengecek apakah data tetap tersimpan, serta menguji fitur penghapusan data.

Jawaban / Kode Solusi:

1. JAWABAN KODE CSS

```
* {
  box-sizing: border-box;
}

body {
  background: linear-gradient(135deg, #f5f7fa 0%, #c3cfe2 100%);
  min-height: 100vh;
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
}
```

```
.container-fluid {
  padding: 20px;
}

.header-section {
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  color: white;
  padding: 40px 20px;
  border-radius: 15px;
  margin-bottom: 30px;
  box-shadow: 0 10px 30px rgba(0,0,0,0.2);
}

.header-content h1 {
  font-size: 2.2rem;
  margin-bottom: 10px;
  text-shadow: 2px 2px 4px rgba(0,0,0,0.3);
}

.author-name {
  font-size: 1.3rem;
  font-weight: 500;
  opacity: 0.95;
}

.student-id {
  font-size: 1.1rem;
  opacity: 0.85;
}

.main-content {
  display: flex;
  gap: 25px;
  flex-wrap: wrap;
}

.form-section, .table-section {
  flex: 1;
  min-width: 350px;
}

.card {
  border: none;
  border-radius: 15px;
  box-shadow: 0 5px 20px rgba(0,0,0,0.1);
  transition: transform 0.3s ease, box-shadow 0.3s ease;
}

.card:hover {
```

```
transform: translateY(-5px);
box-shadow: 0 15px 40px rgba(0,0,0,0.15);
}

.form-header, .table-header {
  background: linear-gradient(135deg, #4facfe 0%, #00f2fe 100%);
  color: white;
  border-radius: 15px 15px 0 0 !important;
  padding: 15px 20px;
}

.table-header {
  border-radius: 15px 15px 0 0 !important;
}

.card-header h6 {
  margin: 0;
  font-weight: 600;
}

.form-card .card-body, .table-card .card-body {
  padding: 25px;
}

.checklist-section {
  background: #f8f9ff;
  border-radius: 10px;
  padding: 20px;
  border-left: 4px solid #4facfe;
}

.checklist-items .form-check {
  padding-left: 0;
  border-bottom: 1px solid #eee;
  padding-bottom: 10px;
  margin-bottom: 10px;
}

.checklist-items .form-check:last-child {
  border-bottom: none;
  margin-bottom: 0;
  padding-bottom: 0;
}

.btn {
  border-radius: 25px;
  font-weight: 500;
  padding: 10px 20px;
  transition: all 0.3s ease;
```

```
}

.btn:hover {
  transform: translateY(-2px);
}

.btn-success {
  background: linear-gradient(135deg, #11998e 0%, #38ef7d 100%);
  border: none;
}

.btn-danger {
  background: linear-gradient(135deg, #ff6b6b 0%, #ee5a52 100%);
  border: none;
}

.search-container {
  min-width: 250px;
}

.table-responsive {
  border-radius: 0 0 15px 15px;
}

.table th {
  background: #495057;
  color: white;
  font-weight: 600;
  border: none;
  padding: 15px 12px;
}

.table td {
  padding: 12px;
  vertical-align: middle;
  border-color: #dee2e6;
}

.badge {
  font-size: 0.85em;
  padding: 6px 12px;
  border-radius: 20px;
}

.bg-success { background: #28a745 !important; }
.bg-warning { background: #ffc107 !important; color: #000 !important; }
.bg-danger { background: #dc3545 !important; }

.modal-content {
```

```
border-radius: 15px;
border: none;
box-shadow: 0 20px 60px rgba(0,0,0,0.3);
}

.modal-header {
border-radius: 15px 15px 0 0;
background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
color: white;
}

@media (max-width: 768px) {
  .main-content {
    flex-direction: column;
  }

  .header-content h1 {
    font-size: 1.8rem;
  }
}
```

2. JAWABAN KODE JS

```
// =====
// ProyekMini_Checklist5S.js - VERSI LENGKAP
// =====

// Inisialisasi data
let audits = JSON.parse(localStorage.getItem('audit5S')) || [];
let penggunaList = JSON.parse(localStorage.getItem('penggunaList')) ||
['Admin', 'User1', 'User2', 'Supervisor'];
let operatorList = JSON.parse(localStorage.getItem('operatorList')) || [];

// Load data awal
window.onload = function() {
  loadPengguna();
  loadOperatorDropdown();
  tampilkanTabel();
  setupEventListeners();
  document.getElementById('tanggal').value = new
Date().toISOString().split('T')[0];
};

function setupEventListeners() {
  // Form events
  document.getElementById('form5S').addEventListener('submit',
simpanAudit);
  document.getElementById('hapusSemua').addEventListener('click',
hapusSemuaData);
}
```



```
document.getElementById('searchInput').addEventListener('keyup',
filterTabel);
document.getElementById('clearSearch').addEventListener('click',
clearSearch);
document.getElementById('simpanOperator').addEventListener('click',
simpanOperator);

// Real-time validation
document.getElementById('pilihOperator').addEventListener('change',
updateShiftDropdown);
document.getElementById('tanggal').addEventListener('change',
validasiForm);
document.getElementById('shift').addEventListener('change',
validasiForm);
document.getElementById('pengguna').addEventListener('change',
validasiForm);

// Modal operator
document.getElementById('modalOperator').addEventListener('shown.bs.modal
', resetModalOperator);
}

function loadPengguna() {
const select = document.getElementById('pengguna');
select.innerHTML = '<option value="">Pilih Pengguna</option>';

penggunaList.forEach(pengguna => {
const option = document.createElement('option');
option.value = pengguna;
option.textContent = pengguna;
select.appendChild(option);
});

localStorage.setItem('penggunaList', JSON.stringify(penggunaList));
}

function loadOperatorDropdown() {
const select = document.getElementById('pilihOperator');
select.innerHTML = '<option value="">📄 Pilih Operator</option>';

operatorList.forEach((op, index) => {
const option = document.createElement('option');
option.value = index;
option.textContent = `${op.nama} (${op.shifts.length} shift
tersedia)`;
select.appendChild(option);
});
}
```

```
function updateShiftDropdown() {
    const operatorIndex = document.getElementById('pilihOperator').value;
    const shiftSelect = document.getElementById('shift');

    shiftSelect.innerHTML = '<option value=""> 🔄 Pilih Shift</option>';
    shiftSelect.disabled = true;

    if(operatorIndex === '' || !operatorList[operatorIndex]) {
        validasiForm();
        return;
    }

    const operator = operatorList[operatorIndex];
    operator.shifts.forEach(shift => {
        const option = document.createElement('option');
        option.value = shift;
        option.textContent = shift;
        shiftSelect.appendChild(option);
    });

    shiftSelect.disabled = false;
    validasiForm();
}

function simpanOperator() {
    const nama = document.getElementById('namaOperator').value.trim();
    if(!nama) {
        alert('❌ Nama operator wajib diisi!');
        document.getElementById('namaOperator').focus();
        return;
    }

    // Cek duplikasi nama
    if(operatorList.some(op => op.nama.toLowerCase() === nama.toLowerCase()))
    {
        alert('❌ Operator dengan nama tersebut sudah ada!');
        return;
    }

    const operator = {
        nama: nama,
        shifts: []
    };

    if(document.getElementById('shiftPagi').checked)
operator.shifts.push('Pagi (06:00-14:00)');
    if(document.getElementById('shiftSiang').checked)
operator.shifts.push('Siang (14:00-22:00)');
```

```
if(document.getElementById('shiftMalam').checked)
operator.shifts.push('Malam (22:00-06:00)');

if(operator.shifts.length === 0) {
    alert('✗ Pilih minimal 1 shift!');
    return;
}

operatorList.push(operator);
localStorage.setItem('operatorList', JSON.stringify(operatorList));

loadOperatorDropdown();
bootstrap.Modal.getInstance(document.getElementById('modalOperator')).hide();

alert('☑ Operator "' + nama + '" berhasil ditambahkan!');
}

function resetModalOperator() {
    document.getElementById('modalTitle').innerHTML = '<i class="fas fa-user-plus me-2"></i>Tambah Operator';
    document.getElementById('namaOperator').value = '';
    document.getElementById('shiftPagi').checked = true;
    document.getElementById('shiftSiang').checked = true;
    document.getElementById('shiftMalam').checked = true;
    document.getElementById('namaOperator').focus();
}

function validasiForm() {
    const operatorIndex = document.getElementById('pilihOperator').value;
    const tanggal = document.getElementById('tanggal').value;
    const shift = document.getElementById('shift').value;
    const pengguna = document.getElementById('pengguna').value;
    const errorEl = document.getElementById('errorShift');
    const submitBtn = document.querySelector('button[type="submit"]');

    // Enable/disable submit button
    const formValid = pengguna && operatorIndex && tanggal && shift;
    submitBtn.disabled = !formValid;

    // Cek duplikasi shift
    if(formValid && operatorList[operatorIndex]) {
        const operator = operatorList[operatorIndex];
        const sudahAda = audits.some(audit =>
            audit.operatorNama === operator.nama &&
            audit.tanggal === tanggal &&
            audit.shift === shift
        );
    }
};
```

```
        if(sudahAda) {
            errorEl.textContent = 'Shift sudah ada untuk operator ini!';
            errorEl.style.display = 'inline-block';
            submitBtn.disabled = true;
            return false;
        }
    }

    errorEl.style.display = 'none';
    return formValid;
}

function simpanAudit(e) {
    e.preventDefault();
    if(!validasiForm()) return;

    const operatorIndex = document.getElementById('pilihOperator').value;
    const operator = operatorList[operatorIndex];

    const checkboxes = ['seiri', 'seiton', 'seiso', 'seiketsu', 'shitsuke'];
    const checkedCount = checkboxes.filter(id =>
document.getElementById(id).checked).length;
    const skor = Math.round((checkedCount / 5) * 100);

    const audit = {
        id: Date.now(),
        pengguna: document.getElementById('pengguna').value,
        operatorNama: operator.nama,
        tanggal: document.getElementById('tanggal').value,
        shift: document.getElementById('shift').value,
        skor: skor,
        checklist: checkboxes.reduce((acc, id) => {
            acc[id] = document.getElementById(id).checked;
            return acc;
        }, {})
    };

    audits.unshift(audit);
    localStorage.setItem('audit5S', JSON.stringify(audits));

    // Reset form
    document.getElementById('form5S').reset();
    document.getElementById('tanggal').value = new
Date().toISOString().split('T')[0];
    document.getElementById('shift').disabled = true;
    document.getElementById('errorShift').style.display = 'none';

    tampilkanTabel();
    alert('☑ Data audit berhasil disimpan!\nSkor: ' + skor + '%');
```

```

}

function tampilkanTabel(filteredAudits = audits) {
  const tbody = document.getElementById('tabelAudit');
  tbody.innerHTML = '';

  if(filteredAudits.length === 0) {
    tbody.innerHTML = '<tr><td colspan="6" class="text-center text-muted py-4">Belum ada data audit</td></tr>';
    return;
  }

  filteredAudits.forEach(audit => {
    const row = tbody.insertRow();
    row.innerHTML = `
      <td>${formatTanggal(audit.tanggal)}</td>
      <td><strong>${audit.operatorNama}</strong></td>
      <td><span class="badge bg-info">${audit.shift}</span></td>
      <td>${audit.pengguna}</td>
      <td><span class="badge
${getSkorBadge(audit.skor)}">${audit.skor}%</span></td>
      <td>
        <div class="btn-group btn-group-sm" role="group">
          <button class="btn btn-warning"
onclick="editAudit(${audit.id})" title="Edit">
            <i class="fas fa-edit"></i>
          </button>
          <button class="btn btn-danger"
onclick="hapusAudit(${audit.id})" title="Hapus">
            <i class="fas fa-trash"></i>
          </button>
        </div>
      </td>
    `;
  });
}

function filterTabel() {
  const query = document.getElementById('searchInput').value.toLowerCase();
  const filtered = audits.filter(audit =>
    audit.operatorNama.toLowerCase().includes(query) ||
    audit.shift.toLowerCase().includes(query) ||
    audit.pengguna.toLowerCase().includes(query) ||
    formatTanggal(audit.tanggal).toLowerCase().includes(query)
  );
  tampilkanTabel(filtered);
}

function clearSearch() {

```

```
document.getElementById('searchInput').value = '';
tampilkanTabel();
}

function formatTanggal(tanggal) {
    return new Date(tanggal + 'T00:00:00').toLocaleDateString('id-ID');
}

function getSkorBadge(skor) {
    if(skor >= 90) return 'bg-success';
    if(skor >= 70) return 'bg-warning';
    return 'bg-danger';
}

function hapusSemuaData() {
    if(confirm('⚠️ Yakin ingin hapus SEMUA data?\nIni tidak bisa dibatalkan!')) {
        localStorage.clear();
        audits = [];
        penggunaList = ['Admin', 'User1', 'User2', 'Supervisor'];
        operatorList = [];
        loadPengguna();
        loadOperatorDropdown();
        tampilkanTabel();
        alert('🗑️ Semua data telah dihapus!');
    }
}

function hapusAudit(id) {
    if(confirm('Yakin hapus data ini?')) {
        audits = audits.filter(audit => audit.id !== id);
        localStorage.setItem('audit5S', JSON.stringify(audits));
        tampilkanTabel();
        alert('✅ Data berhasil dihapus!');
    }
}

function editAudit(id) {
    const audit = audits.find(a => a.id === id);
    if(audit) {
        // Load data ke form
        document.getElementById('pengguna').value = audit.pengguna;
        document.getElementById('tanggal').value = audit.tanggal;

        // Cari operator index
        const operatorIndex = operatorList.findIndex(op => op.nama === audit.operatorNama);
        if(operatorIndex !== -1) {
            document.getElementById('pilihOperator').value = operatorIndex;
        }
    }
}
```

```

        updateShiftDropdown();
        document.getElementById('shift').value = audit.shift;
    }

    // Load checklist
    ['seiri', 'seiton', 'seiso', 'seiketsu', 'shitsuke'].forEach(id => {
        document.getElementById(id).checked = audit.checklist[id];
    });

    alert('📄 Data dimuat untuk di-edit. Ubah yang diperlukan lalu simpan!');
}
}

// Inisialisasi beberapa operator contoh jika kosong
if(operatorList.length === 0) {
    operatorList = [
        {
            nama: 'Budi Santoso',
            shifts: ['Pagi (06:00-14:00)', 'Siang (14:00-22:00)']
        },
        {
            nama: 'Sari Dewi',
            shifts: ['Siang (14:00-22:00)', 'Malam (22:00-06:00)']
        }
    ];
    localStorage.setItem('operatorList', JSON.stringify(operatorList));
    loadOperatorDropdown();
}

```

3. JAWABAN KODE HTML

```

<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Checklist Audit 5S</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
    <link href="https://cdn.jsdelivr.net/npm/awesome/6.0.0/css/all.min.css" rel="stylesheet">
    <style>
        .operator-card { border-left: 4px solid #007bff; }
        .shift-badge { font-size: 0.8em; }
        .form-floating { margin-bottom: 1rem; }
    </style>
</head>

```

```

<body class="bg-light">
  <div class="main-wrapper">
    <header class="bg-info text-light text-center py-5 mb-4">
      <h1 class="fw-bold mb-2">Proyek Mini - Aplikasi Checklist 5S</h1>
      <p class="fs-5 mb-0">Maessa Andrea Vallenia</p>
      <p class="mb-0">23051430044</p>
    </header>

    <div class="row">
      <!-- FORM -->
      <div class="col-md-4">
        <!-- Operator Management Card -->
        <div class="card operator-card mb-3">
          <div class="card-header bg-primary text-white d-flex justify-content-between align-items-center">
            <span><i class="fas fa-users me-2"></i>Daftar Operator</span>
            <button class="btn btn-sm btn-light" data-bs-toggle="modal" data-bs-target="#modalOperator">
              <i class="fas fa-plus"></i> Tambah
            </button>
          </div>
          <div class="card-body p-2">
            <select id="pilihOperator" class="form-select form-select-sm" onchange="updateShiftDropdown()">
              <option value="">Pilih Operator</option>
            </select>
          </div>
        </div>

        <!-- Form Utama -->
        <div class="card">
          <div class="card-header bg-primary text-white">
            Form Audit 5S
            <span id="errorShift" class="badge bg-danger ms-2" style="display: none;">
              <i class="fas fa-exclamation-triangle me-1"></i>Shift sudah ada!
            </span>
          </div>
          <div class="card-body">
            <form id="form5S">
              <!-- Pengguna -->
              <div class="form-floating mb-3">
                <select id="pengguna" class="form-select" required>
                  <option value="">Pilih Pengguna</option>
                </select>
                <label>Pengguna</label>
              </div>
            </form>
          </div>
        </div>
      </div>
    </div>
  </div>

```



```

        </div>

        <!-- Tanggal -->
        <div class="form-floating mb-3">
            <input type="date" id="tanggal" class="form-
control" required>
            <label>Tanggal</label>
        </div>

        <!-- Shift (tergantung operator) -->
        <div class="form-floating mb-3">
            <select id="shift" class="form-select"
required disabled>
                <option value="">Pilih Shift</option>
            </select>
            <label>Shift</label>
        </div>

        <!-- Checklist 5S -->
        <div class="mb-4 p-3 border rounded">
            <label class="form-label fw-bold mb-3 d-
block">
                <i class="fas fa-check-circle text-
success me-2"></i>Checklist 5S
            </label>
            <div class="row g-2">
                <div class="col-6">
                    <div class="form-check">
                        <input class="form-check-input"
type="checkbox" id="seiri">
                        <label class="form-check-
label">Seiri (Ringkas)</label>
                    </div>
                </div>
                <div class="col-6">
                    <div class="form-check">
                        <input class="form-check-input"
type="checkbox" id="seiton">
                        <label class="form-check-
label">Seiton (Rapi)</label>
                    </div>
                </div>
                <div class="col-6">
                    <div class="form-check">
                        <input class="form-check-input"
type="checkbox" id="seiso">
                        <label class="form-check-
label">Seiso (Resik)</label>
                    </div>
                </div>
            </div>
        </div>

```

```

        </div>
        <div class="col-6">
            <div class="form-check">
                <input class="form-check-input"
type="checkbox" id="seiketsu">
                <label class="form-check-
label">Seiketsu (Rawat)</label>
            </div>
        </div>
        <div class="col-12">
            <div class="form-check">
                <input class="form-check-input"
type="checkbox" id="shitsuke">
                <label class="form-check-
label">Shitsuke (Rajin)</label>
            </div>
        </div>
    </div>
    <button type="submit" class="btn btn-success w-
100 mb-2" disabled>
        <i class="fas fa-save me-2"></i>Simpan Audit
    </button>
    <button type="button" id="hapusSemua" class="btn
btn-danger w-100">
        <i class="fas fa-trash me-2"></i>Hapus Semua
    Data
    </button>
</form>
</div>
</div>
</div>
<!-- TABEL -->
<div class="col-md-8">
    <div class="d-flex justify-content-between align-items-center
mb-3">
        <h3><i class="fas fa-history me-2"></i>Riwayat Audit</h3>
        <div class="input-group" style="width: 300px;">
            <span class="input-group-text"><i class="fas fa-
search"></i></span>
            <input type="text" id="searchInput" class="form-
control" placeholder="Cari operator, shift, atau nama...">
            <button class="btn btn-outline-secondary"
type="button" id="clearSearch">
                <i class="fas fa-times"></i>
            </button>
        </div>
    </div>

```

```

        </div>

        <div class="table-responsive">
            <table class="table table-bordered table-striped"
id="tabelUtama">
                <thead class="table-dark">
                    <tr>
                        <th><i class="fas fa-calendar"></i>
Tanggal</th>
                        <th><i class="fas fa-user"></i> Operator</th>
                        <th><i class="fas fa-clock"></i> Shift</th>
                        <th><i class="fas fa-users"></i>
Pengguna</th>
                        <th>Skor</th>
                        <th><i class="fas fa-cog"></i> Aksi</th>
                    </tr>
                </thead>
                <tbody id="tabelAudit"></tbody>
            </table>
        </div>
    </div>
</div>
</div>

<!-- Modal Tambah/Edit Operator -->
<div class="modal fade" id="modalOperator" tabindex="-1">
    <div class="modal-dialog">
        <div class="modal-content">
            <div class="modal-header bg-primary text-white">
                <h5 class="modal-title" id="modalTitle"><i class="fas fa-
user-plus me-2"></i>Tambah Operator</h5>
                <button type="button" class="btn-close btn-close-white"
data-bs-dismiss="modal"></button>
            </div>
            <div class="modal-body">
                <div class="mb-3">
                    <label class="form-label fw-bold">Nama Operator
*</label>
                    <input type="text" id="namaOperator" class="form-
control" placeholder="Masukkan nama operator lengkap">
                </div>
                <div class="row">
                    <div class="col-md-6">
                        <label class="form-label">Shift Pagi</label>
                        <div class="form-check form-switch">
                            <input class="form-check-input"
type="checkbox" id="shiftPagi" checked>
                            <label class="form-check-label">06:00-
14:00</label>

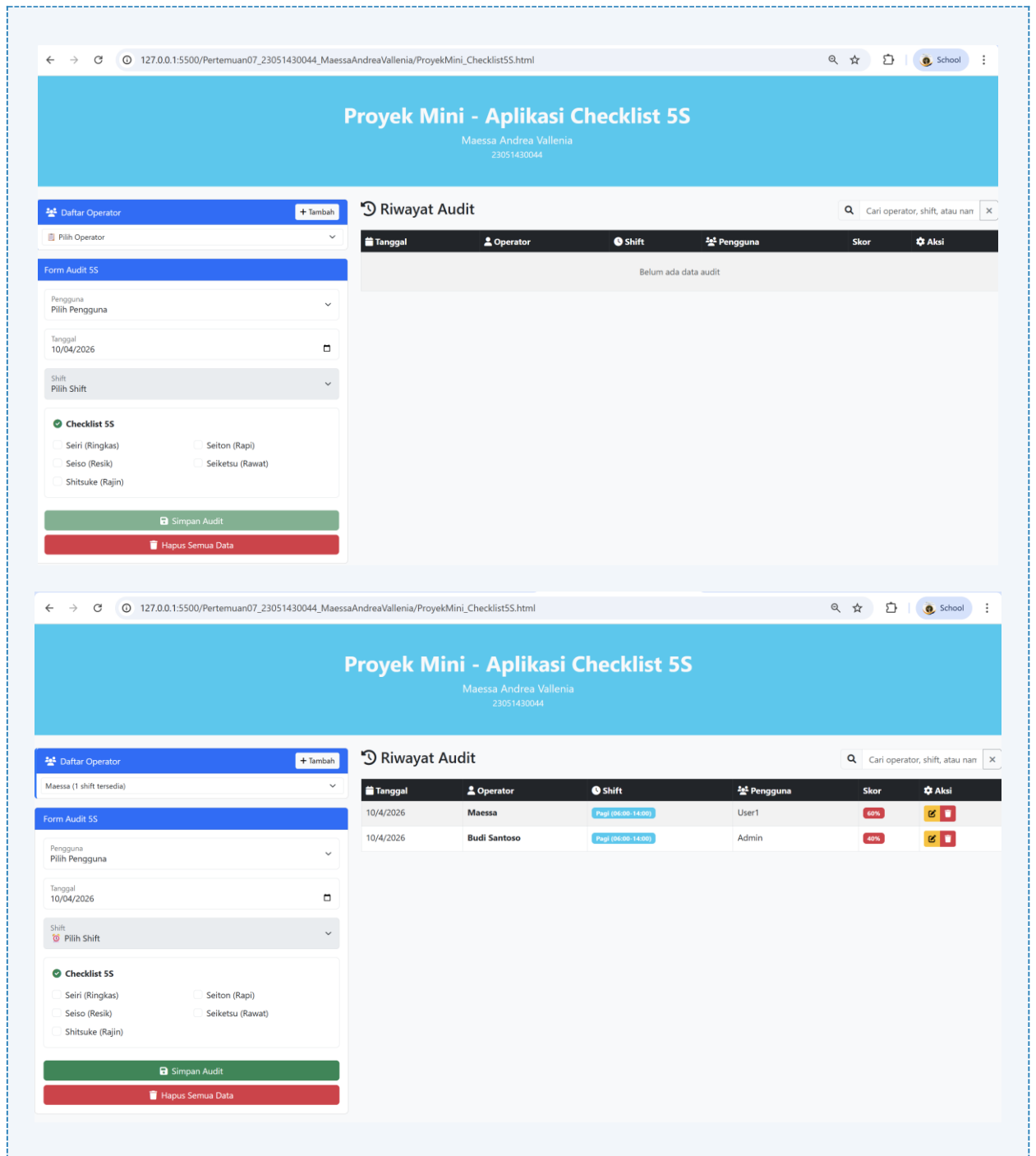
```

```

        </div>
    </div>
    <div class="col-md-6">
        <label class="form-label">Shift Siang</label>
        <div class="form-check form-switch">
            <input class="form-check-input"
type="checkbox" id="shiftSiang" checked>
            <label class="form-check-label">14:00-
22:00</label>
        </div>
    </div>
    <div class="col-md-6">
        <label class="form-label">Shift Malam</label>
        <div class="form-check form-switch">
            <input class="form-check-input"
type="checkbox" id="shiftMalam" checked>
            <label class="form-check-label">22:00-
06:00</label>
        </div>
    </div>
</div>
<div class="modal-footer">
    <button type="button" class="btn btn-secondary" data-bs-
dismiss="modal">Batal</button>
    <button type="button" class="btn btn-primary"
id="simpanOperator">
        <i class="fas fa-save me-2"></i>Simpan Operator
    </button>
</div>
</div>
</div>
</div>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.mi
n.js"></script>
    <script src="ProyekMini_Checklist5S.js"></script>
</body>
</html>

```



Gambar F.3 – Hasil Proyek Mini Aplikasi Checklist 5S

G. PEMBAHASAN

Pada praktikum ini, mahasiswa mempelajari integrasi HTML, CSS/Bootstrap, dan JavaScript untuk membangun aplikasi web interaktif berupa sistem pencatatan log produksi berbasis LocalStorage. Praktikum menekankan pada pengelolaan data dari proses input, validasi, penyimpanan, hingga penampilan data secara dinamis pada antarmuka web.

HTML digunakan untuk membangun struktur utama seperti form input data produksi, tombol aksi, serta tabel rekap data. Bootstrap dan CSS digunakan untuk memperindah tampilan agar lebih responsif, rapi, dan mudah digunakan. Tampilan ini mencerminkan konsep user interface pada sistem industri yang membutuhkan kejelasan dan kemudahan akses data.

JavaScript berperan sebagai inti logika program, khususnya dalam pengolahan data dan interaksi pengguna. Dengan memanfaatkan DOM, program mampu mengambil nilai input, memproses data, serta menampilkan hasilnya ke dalam tabel tanpa perlu melakukan reload halaman. Event listener seperti submit dan input digunakan untuk menangkap aksi pengguna dan menjalankan fungsi tertentu sebagai respon.

Konsep utama pada praktikum ini adalah penggunaan LocalStorage sebagai media penyimpanan data di sisi client (browser). Data produksi disimpan dalam bentuk JSON menggunakan JSON.stringify() dan diambil kembali menggunakan JSON.parse(). Hal ini memungkinkan data tetap tersimpan meskipun halaman direfresh.

Selain itu, fitur filter data juga diterapkan menggunakan method .filter() pada array, sehingga pengguna dapat mencari data berdasarkan nama operator secara real-time. Hal ini menunjukkan bagaimana JavaScript dapat digunakan untuk mengelola data secara dinamis dan interaktif.

Secara keseluruhan, praktikum ini memberikan pemahaman tentang bagaimana membangun aplikasi web yang memiliki kemampuan penyimpanan data sederhana (persistensi), validasi input, serta manipulasi tampilan secara dinamis, yang sangat relevan dengan kebutuhan sistem informasi di industri.

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum menunjukkan bahwa implementasi program telah sesuai dengan teori DOM, event handling, validasi input, dan LocalStorage.

Struktur HTML telah menyediakan elemen yang dapat diakses JavaScript, seperti form input dan tabel. Dengan DOM, elemen tersebut dapat dimanipulasi secara langsung menggunakan getElementById().

Event listener pada form (submit) dan input (input) telah berjalan sesuai teori, di mana setiap aksi pengguna akan memicu proses tertentu dalam program. Validasi input memastikan bahwa data yang dimasukkan sesuai dengan aturan, sehingga mencegah kesalahan data.

Penggunaan LocalStorage juga telah sesuai dengan konsep teori, di mana data disimpan dalam bentuk string JSON dan dapat diambil kembali saat halaman dimuat ulang. Hal ini membuktikan bahwa aplikasi memiliki kemampuan penyimpanan data sederhana (persistensi).

Secara keseluruhan, sistem mampu:

1. Menerima input dengan valid
2. Menyimpan data secara permanen di browser
3. Menampilkan data secara dinamis
4. Memfilter data berdasarkan input pengguna.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Data filter tidak muncul	Menambahkan fungsi khusus loadData() untuk menampilkan hasil filter.
Error loadData is not defined	Mendefinisikan fungsi loadData() sebelum dipanggil

G.3 Kaitan dengan Penerapan di Industri

Materi praktikum ini sangat relevan dengan penerapan di industri, khususnya dalam sistem pencatatan dan monitoring produksi.

LocalStorage merepresentasikan konsep database sederhana untuk menyimpan data produksi harian. Dalam industri, sistem seperti ini digunakan untuk mencatat aktivitas produksi, operator, dan jumlah output secara real-time.

Fitur filter data mencerminkan kebutuhan industri dalam melakukan pencarian data dengan cepat, misalnya mencari performa operator atau histori produksi tertentu.

Validasi input memastikan bahwa data yang dimasukkan sesuai standar, sehingga mengurangi kesalahan operasional. Hal ini penting dalam sistem industri yang membutuhkan akurasi tinggi.

Secara keseluruhan, praktikum ini menggambarkan bagaimana teknologi web dapat digunakan sebagai dasar dalam membangun sistem informasi produksi yang mampu:

1. Menyimpan data secara terstruktur
2. Menampilkan data secara real-time
3. Memfasilitasi pencarian dan analisis data
4. Menjaga kualitas dan validitas data

Meskipun masih sederhana, konsep ini merupakan fondasi penting dalam pengembangan sistem industri yang lebih kompleks dan terintegrasi.

H. KESIMPULAN

Tuliskan 3–5 poin kesimpulan yang merangkum hal-hal yang dipelajari dan dicapai pada praktikum modul ini. Kesimpulan harus spesifik, bukan bersifat umum.

1	Mahasiswa mampu memahami dan mengimplementasikan konsep LocalStorage sebagai penyimpanan data persisten, yang menjadi dasar dalam pengembangan sistem database pada aplikasi industri.
2	Mahasiswa dapat menerapkan validasi input berbasis JavaScript untuk menjaga kualitas dan konsistensi data, yang merupakan aspek penting dalam sistem informasi produksi.
3	Mahasiswa mampu mengelola data menggunakan array of objects, serta menampilkannya secara dinamis ke dalam tabel, sehingga menyerupai pengolahan data pada sistem monitoring industri.
4	Mahasiswa dapat mengembangkan fitur interaktif seperti filter data, yang mendukung proses pencarian dan analisis data secara cepat dan efisien, sesuai dengan kebutuhan pengambilan keputusan di industri.
5	Mahasiswa memahami konsep event-driven programming, di mana sistem merespon input pengguna secara real-time, sehingga menghasilkan aplikasi yang responsif dan mendekati implementasi sistem kontrol dan monitoring di industri modern.

I. DAFTAR PUSTAKA

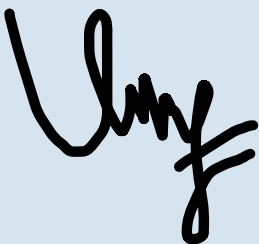
Tuliskan semua sumber referensi yang digunakan dalam format APA 7th Edition. Minimal 3 referensi yang relevan (termasuk Manual Praktikum dan sumber online resmi seperti MDN, Reactjs.org, dll).

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Kom, I. L. M. (2023). <i>Dasar Algoritma dan Pemrograman Javascript</i> . Irvan Lewenusa, M. Kom.
[3]	Sholikhah, M. (2022). Html, Css Dan Javascript. <i>Penerbit Yayasan Prima Agus Teknik</i> , 1-343 .
[4]	Tinambunan, M. H., Siregar, A. H., & Ginting, S. (2024). FULLSTACK PROGRAMMING: MEMBANGUN APPLICATION PROGRAMMING INTERFACE (API) DENGAN LARAVEL. <i>Penerbit Tahta Media</i> .
[5]	Pattinama, Y. L., & Susanti, I. (2023). Implementasi Rest API Web Service Dengan Otentifikasi JSON Web Token Untuk Aplikasi Properti. <i>Informatik: Jurnal Ilmu Komputer</i> , 19(1), 81-89.
[6]	https://www.javascripttutorial.net/
[7]	https://developer.mozilla.org/en-US/docs/Web/JavaScript
[8]	https://www.geeksforgeeks.org/
[9]	https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API
[10]	https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API
[11]	https://www.w3schools.com/js/js_json.asp

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN	
Aspek Penilaian	Bobot

No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Diperiksa oleh Asisten / Dosen
 Maessa Andrea Vallenia NIM: 23051430044	 Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____

MODUL 8

ASYNCHRONOUS JAVASCRIPT (FETCH API, JSON, REST API SEDERHANA)

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 8 (Pertemuan 8)
Topik Utama	Konsep Asynchronous, Fetch API, Promise, Async/Await, dan integrasi REST API eksternal.
Sub-Topik	Konsep Asynchronous JavaScript, Promise sebagai mekanisme penanganan proses asynchronous, penggunaan Fetch API untuk mengambil data dari server, penerapan Async/Await untuk penulisan kode asynchronous yang lebih sederhana, serta integrasi REST API eksternal dalam aplikasi web.
Durasi Praktikum	340 Menit
Tanggal Praktikum	17 April 2026
Nama Mahasiswa	Maessa Andrea Vallenia
NIM	23051430044
Kelas	A

B. TUJUAN PEMBELAJARAN

Tuliskan tujuan pembelajaran yang ingin dicapai pada modul ini, sesuai dengan yang tertera di Manual Praktikum.

1	Memahami perbedaan kode sinkronus dan asinkronus.
2	Mampu menggunakan `fetch()` untuk mengambil data dari server eksternal (API).
3	Memahami format data JSON (JavaScript Object Notation).
4	Menerapkan penanganan error saat request API gagal.

C. ALAT DAN BAHAN

Sebutkan seluruh perangkat keras, perangkat lunak, dan sumber referensi yang digunakan selama praktikum.

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
2	Google Chrome / Browser Modern	Versi terbaru	<i>Testing & Debugging</i>
3	Live Server	Versi 5.7.9	Menjalankan HTML dan memantau perubahan file
4	JavaScript (Built-in Browser)	ES6 (Default Browser Modern)	Bahasa pemrograman untuk logika dan perhitungan
5	CodeSnap	Versi 1.3.4	Dokumentasi kode program
6	Git dan GitHub	Versi 2.53.0	Version control system (VCS) berbasis Git
7	Koneksi Internet	Stabil	Mengakses Library dan Framework
8	Console Browser (Developer Tools)	Default Browser	Melihat output program dan debugging
9	Bootstrap	Versi 5.3	Membantu pembuatan UI seperti button, card, dan alert pada panel control
10	File praktikum	Perrtemuan sebelumnya	Digunakan sebagai dasar pengembangan panel kontrol
11	LocalStorage (Web Storage API)	Built-in Browser	Digunakan untuk menyimpan data produksi secara persisten

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
12	API JSONPlaceholder	Built-in Browser	Digunakan sebagai simulasi database eksternal (data karyawan dan laporan)
13	Fetch API	Built-in Browser	Digunakan untuk melakukan HTTP request (GET, POST, dll) ke server

D. DASAR TEORI

1. Asynchronous JavaScript merupakan konsep pemrograman yang memungkinkan kode dijalankan tanpa harus menunggu proses lain selesai terlebih dahulu. Berbeda dengan synchronous yang berjalan secara berurutan (blocking), asynchronous memungkinkan beberapa proses berjalan secara bersamaan (non-blocking), sehingga aplikasi tetap responsif meskipun sedang mengambil data dari server. Konsep ini sangat penting dalam pengembangan web modern, terutama saat berinteraksi dengan sumber data eksternal.
2. Promise adalah objek dalam JavaScript yang digunakan untuk menangani proses asynchronous. Promise merepresentasikan nilai yang mungkin belum tersedia saat ini, tetapi akan tersedia di masa depan dengan tiga kondisi, yaitu pending (menunggu), fulfilled (berhasil), dan rejected (gagal). Dengan menggunakan Promise, pengelolaan proses asynchronous menjadi lebih terstruktur dibandingkan callback biasa.
3. Fetch API merupakan metode modern dalam JavaScript yang digunakan untuk melakukan HTTP request ke server, seperti mengambil (GET), mengirim (POST), memperbarui (PUT), dan menghapus (DELETE) data. Fetch API mengembalikan Promise sehingga dapat digunakan dengan metode .then() atau dikombinasikan dengan async/await. Penggunaan Fetch API memungkinkan aplikasi web untuk mengambil data dari server eksternal secara dinamis.
4. JSON (JavaScript Object Notation) adalah format pertukaran data yang ringan dan mudah dibaca, baik oleh manusia maupun mesin. JSON digunakan untuk mengirim dan menerima data antara client dan server. Dalam JavaScript, data JSON dapat diubah menjadi object menggunakan

- JSON.parse(), dan sebaliknya object dapat diubah menjadi string JSON menggunakan JSON.stringify().
5. Async/Await merupakan sintaks modern dalam JavaScript yang digunakan untuk menangani proses asynchronous dengan cara yang lebih sederhana dan mudah dibaca. Dengan menggunakan keyword async, sebuah fungsi dapat mengembalikan Promise, sedangkan await digunakan untuk menunggu hasil dari Promise tersebut. Pendekatan ini membuat penulisan kode asynchronous terlihat seperti kode synchronous.
 6. REST API (Representational State Transfer Application Programming Interface) adalah standar komunikasi antara client dan server menggunakan protokol HTTP. REST API memungkinkan aplikasi untuk mengakses data dari server melalui endpoint tertentu. Dalam praktik ini, REST API digunakan untuk mengambil data dari layanan eksternal seperti JSONPlaceholder, yang berfungsi sebagai simulasi database.
 7. Penanganan error (error handling) dalam proses asynchronous sangat penting untuk memastikan aplikasi tetap berjalan dengan baik ketika terjadi kegagalan, seperti koneksi internet terputus atau server tidak merespons. Penanganan error dapat dilakukan menggunakan .catch() pada Promise atau menggunakan try...catch pada async/await. Dengan adanya error handling, sistem dapat memberikan feedback kepada pengguna dan mencegah terjadinya crash pada aplikasi.
 8. Manipulasi DOM digunakan untuk menampilkan data yang diperoleh dari API ke dalam halaman web secara dinamis. Data yang diterima dalam format JSON diproses menggunakan JavaScript, kemudian ditampilkan dalam bentuk elemen HTML seperti card atau tabel. Proses ini biasanya menggunakan perulangan seperti forEach() untuk menampilkan seluruh data yang diterima.

Secara keseluruhan, konsep asynchronous, Fetch API, JSON, dan REST API memungkinkan aplikasi web untuk terhubung dengan sumber data eksternal secara dinamis dan real-time. Implementasi konsep ini sangat penting dalam pengembangan sistem berbasis web modern, seperti dashboard monitoring, sistem informasi produksi, maupun integrasi dengan database dan layanan industri lainnya

D.1 Konsep Utama

Konsep utama pada materi ini berfokus pada penggunaan JavaScript dalam menangani proses asynchronous serta integrasi data dari server eksternal menggunakan Fetch API. Dengan memanfaatkan konsep ini, halaman web tidak hanya menampilkan data statis, tetapi juga mampu mengambil, memproses, dan menampilkan data secara dinamis dari sumber luar tanpa perlu memuat ulang halaman.

Asynchronous JavaScript digunakan agar proses pengambilan data dari server tidak menghambat jalannya program utama. Berbeda dengan metode synchronous yang bersifat blocking, asynchronous memungkinkan aplikasi tetap berjalan dan responsif meskipun

sedang menunggu respon dari server. Hal ini sangat penting dalam pengembangan aplikasi web modern yang membutuhkan akses data secara real-time.

Fetch API digunakan sebagai metode untuk melakukan HTTP request ke server, seperti mengambil data (GET) maupun mengirim data (POST). Fetch API bekerja dengan Promise, sehingga proses pengambilan data dilakukan secara bertahap menggunakan `.then()` atau dengan pendekatan yang lebih modern yaitu `async/await`. Data yang diterima dari server biasanya dalam format JSON, sehingga perlu dilakukan parsing menggunakan `response.json()` agar dapat diolah dalam bentuk object JavaScript.

Penggunaan `async/await` mempermudah penulisan kode asynchronous karena struktur penulisannya menyerupai kode synchronous, sehingga lebih mudah dibaca dan dipahami. Selain itu, penanganan error juga menjadi bagian penting dalam proses ini, baik menggunakan `.catch()` pada Promise maupun `try...catch` pada `async/await`, untuk memastikan aplikasi tetap berjalan dengan baik meskipun terjadi kegagalan dalam pengambilan data.

Manipulasi DOM digunakan untuk menampilkan data yang diperoleh dari API ke dalam halaman web secara dinamis. Data yang telah diproses akan ditampilkan dalam bentuk elemen HTML seperti card atau tabel menggunakan perulangan. Dengan menggabungkan konsep asynchronous, Fetch API, JSON, dan manipulasi DOM, aplikasi web dapat berfungsi sebagai sistem yang dinamis, interaktif, dan terhubung dengan sumber data eksternal secara real-time.

D.2 Keterkaitan dengan Teknik Industri

Materi pada pertemuan ini memiliki keterkaitan yang erat dengan bidang Teknik Industri, khususnya dalam pengembangan sistem informasi yang terintegrasi dengan data eksternal, seperti database perusahaan, sistem ERP, maupun perangkat IoT. Dalam dunia industri, data seringkali tidak berada di satu tempat, melainkan tersebar di berbagai server atau sistem yang saling terhubung.

Konsep asynchronous sangat relevan dalam sistem industri karena memungkinkan aplikasi tetap berjalan tanpa terganggu oleh proses pengambilan data dari server. Sebagai contoh, dalam sistem monitoring produksi, dashboard tetap dapat digunakan oleh operator meskipun data dari server pusat sedang dimuat. Hal ini meningkatkan efisiensi kerja dan mencegah keterlambatan dalam pengambilan keputusan.

Penggunaan Fetch API mencerminkan proses komunikasi antara sistem lokal dengan server pusat dalam industri, seperti pengambilan data stok, data produksi, atau laporan insiden. REST API yang digunakan memungkinkan sistem untuk mengambil dan mengirim data secara terstruktur menggunakan protokol HTTP, sehingga integrasi antar sistem menjadi lebih mudah dan fleksibel.

Format JSON yang digunakan dalam pertukaran data menyerupai struktur data dalam sistem informasi industri, di mana data disimpan dan dikirim dalam bentuk object yang terorganisir. Hal ini memudahkan proses pengolahan, analisis, serta integrasi dengan sistem lain seperti database atau aplikasi berbasis web.

Manipulasi DOM dalam menampilkan data dari API mencerminkan pembuatan dashboard industri yang menampilkan informasi secara real-time, seperti data karyawan, laporan produksi, atau kondisi mesin. Tampilan yang dinamis memungkinkan pengguna untuk melihat perubahan data secara langsung tanpa perlu me-refresh halaman, sehingga meningkatkan responsivitas sistem.

Dengan memahami konsep asynchronous, Fetch API, dan REST API, mahasiswa Teknik Industri dapat mengembangkan aplikasi web yang mampu terhubung dengan berbagai sumber data eksternal. Hal ini sangat penting dalam mendukung digitalisasi industri, meningkatkan efisiensi sistem informasi, serta membantu proses monitoring dan pengambilan keputusan berbasis data secara cepat dan akurat.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

Dokumentasikan setiap langkah kerja yang Anda lakukan, beserta hasil yang diperoleh (screenshot, kode, atau output). Jawab sesuai urutan langkah di Manual Praktikum.

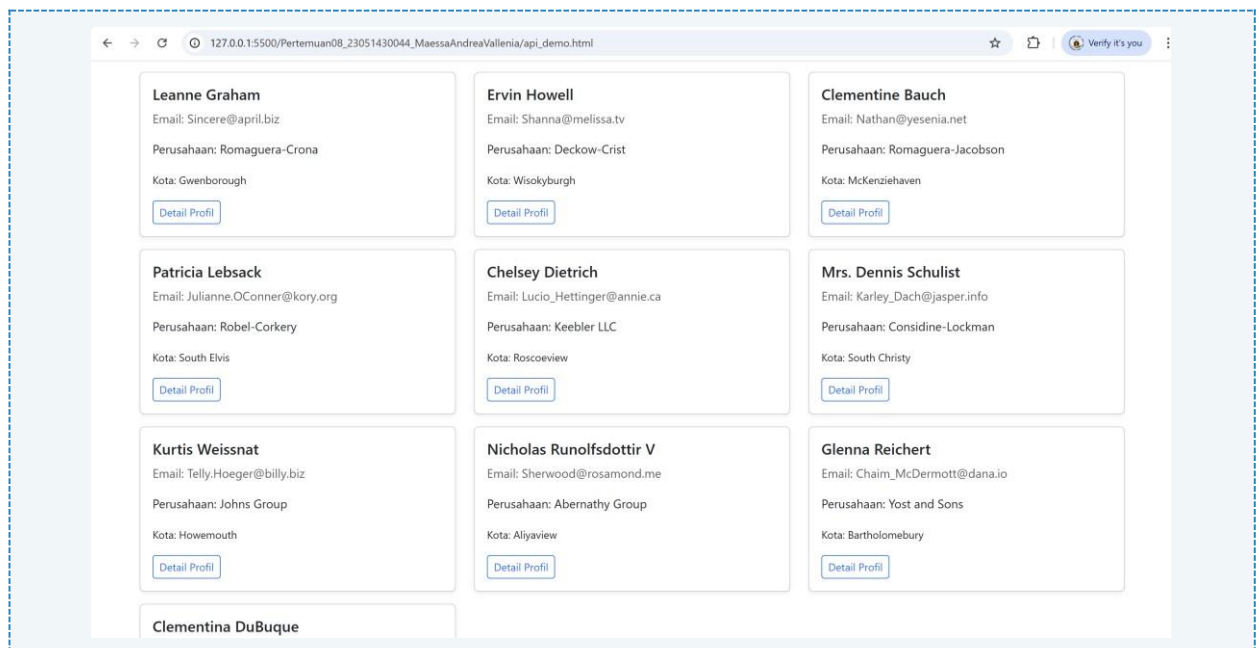
E.1 Langkah 1 – Mengenal JSONPlaceholder

Kita akan menggunakan layanan gratis `https://jsonplaceholder.typicode.com/users` sebagai simulasi database "Karyawan" atau "Agen Supplier".

```
/* ===== KODE / OUTPUT LANGKAH 1 ===== */
```

```
const API_URL = 'https://jsonplaceholder.typicode.com/users';
```

Screenshot Hasil:



Gambar E.1 – Hasil Mengenal JSONPlaceholder

E.2 Langkah 2 – Membuat HTML untuk Menampilkan Data

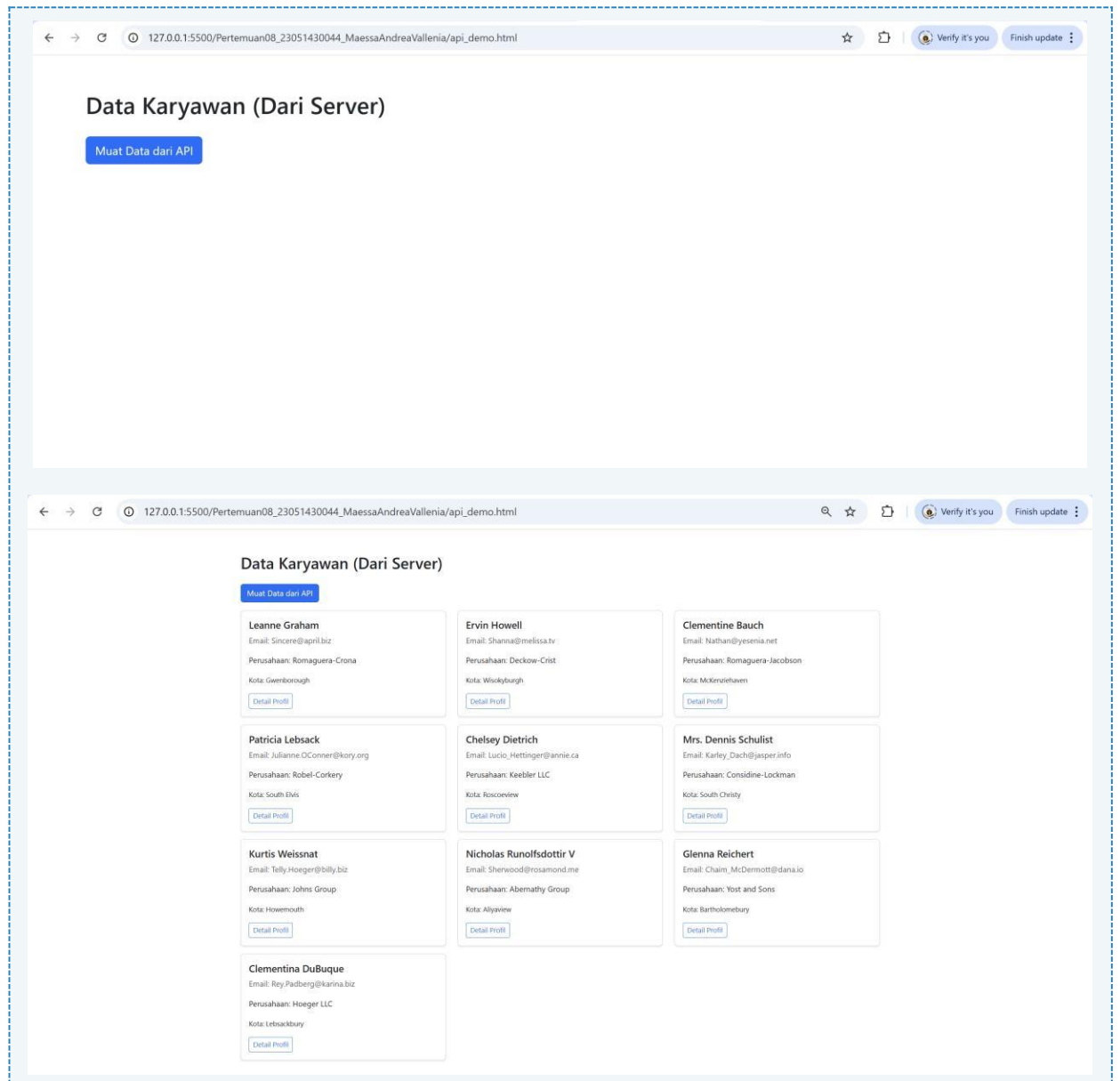
Buat file `api\_demo.html`.

```
/* ===== KODE / OUTPUT LANGKAH 2 ===== */

<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Integrasi API Data Karyawan</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>

<body class="container mt-5">
  <h2 class="mb-4">Data Karyawan (Dari Server)</h2>

  <div class="mb-3">
    <button id="btnLoad" class="btn btn-primary">Muat Data dari
API</button>
    <div id="loading" class="spinner-border text-primary d-none"
role="status"></div>
  </div>
  <div class="row" id="containerKaryawan">
    <!-- Card karyawan akan muncul di sini -->
  </div>
  <script src="api_script.js"></script>
</body>
</html>
```


Screenshot Hasil:

Gambar E.2 – Hasil Membuat HTML untuk Menampilkan Data

E.3 Langkah 3 – Fetch Data dengan .then() (Promise)

Buat `api\_script.js`.

```
/* ===== KODE / OUTPUT LANGKAH 3 ===== */

const btnLoad = document.getElementById('btnLoad');
const container = document.getElementById('containerKaryawan');
const loading = document.getElementById('loading');

// Endpoint API (Simulasi Database)
const API_URL = 'https://jsonplaceholder.typicode.com/users';
```

```
btnLoad.addEventListener('click', function () {
  // Tampilkan loading
  loading.classList.remove('d-none');
  container.innerHTML = ''; // Bersihkan konten lama

  // Fetch Data
  fetch(API_URL)
    .then(function (response) {
      // Cek jika response sukses (kode 200-299)
      if (!response.ok) {
        throw new Error('Gagal mengambil data');
      }

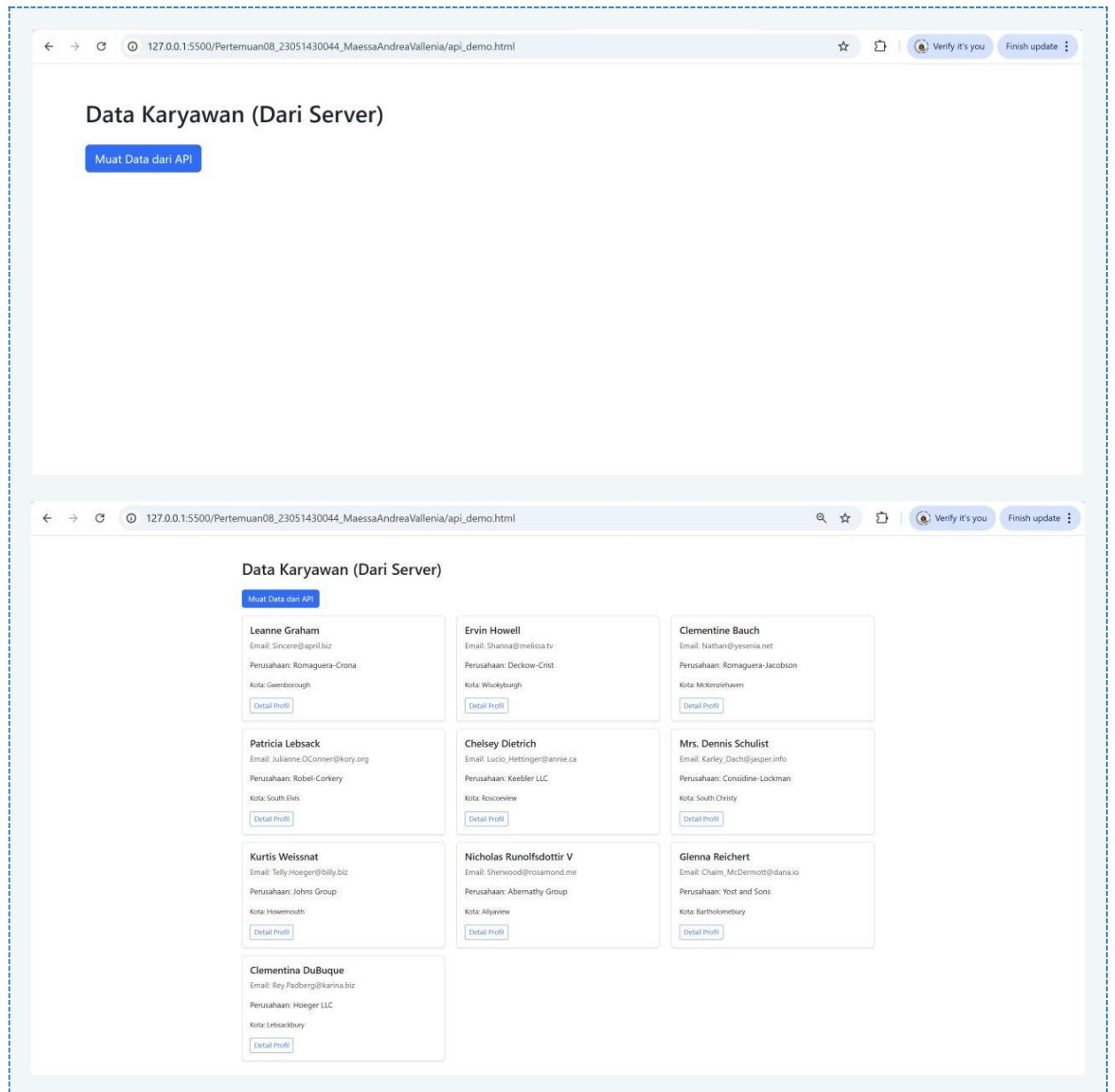
      // Parsing data JSON
      return response.json();
    })
    .then(function (dataKaryawan) {
      // Data berhasil didapat
      console.log(dataKaryawan); // Cek di console browser
      renderData(dataKaryawan);
    })
    .catch(function (error) {
      // Jika ada error (misal putus internet)
      container.innerHTML = `
        <div class="alert alert-danger">
          Error: ${error.message}
        </div>
      `;
    })
    .finally(function () {
      // Sembunyikan loading (baik sukses maupun gagal)
      loading.classList.add('d-none');
    });
});

function renderData(data) {
  data.forEach(function (karyawan) {
    // Buat elemen card Bootstrap
    const col = document.createElement('div');
    col.className = 'col-md-4 mb-3';

    col.innerHTML = `
      <div class="card h-100 shadow-sm">
        <div class="card-body">
          <h5 class="card-title">${karyawan.name}</h5>
        </div>
      </div>
    `;
  });
}
```

```
                <p class="card-text text-muted">Email:
${karyawan.email}</p>
                <p class="card-text">Perusahaan:
${karyawan.company.name}</p>
                <p class="card-text"><small>Kota:
${karyawan.address.city}</small></p>
                <a href="" class="btn btn-sm btn-outline-primary">Detail
Profil</a>
            </div>
        </div>
    `;

    container.appendChild(col);
});
}
```

Screenshot Hasil:

Gambar E.3 – Hasil Fetch Data dengan .then() (Promise)

E.4 Langkah 4 – Fetch dengan Async/Await (Modern Style)

Async/Await membuat kode asinkron terlihat seperti sinkron (lebih mudah dibaca). Mari kita buat fungsi baru untuk mencari karyawan berdasarkan ID. Tambahkan di `api\_script.js`:

```
/* ===== KODE / OUTPUT LANGKAH 4 ===== */

// Fungsi Async untuk mencari data spesifik
async function cariKaryawan(id) {
  try {
    console.log(`Mencari data ID: ${id}...`);
```

```
const response = await fetch(
  `https://jsonplaceholder.typicode.com/users/${id}`
);

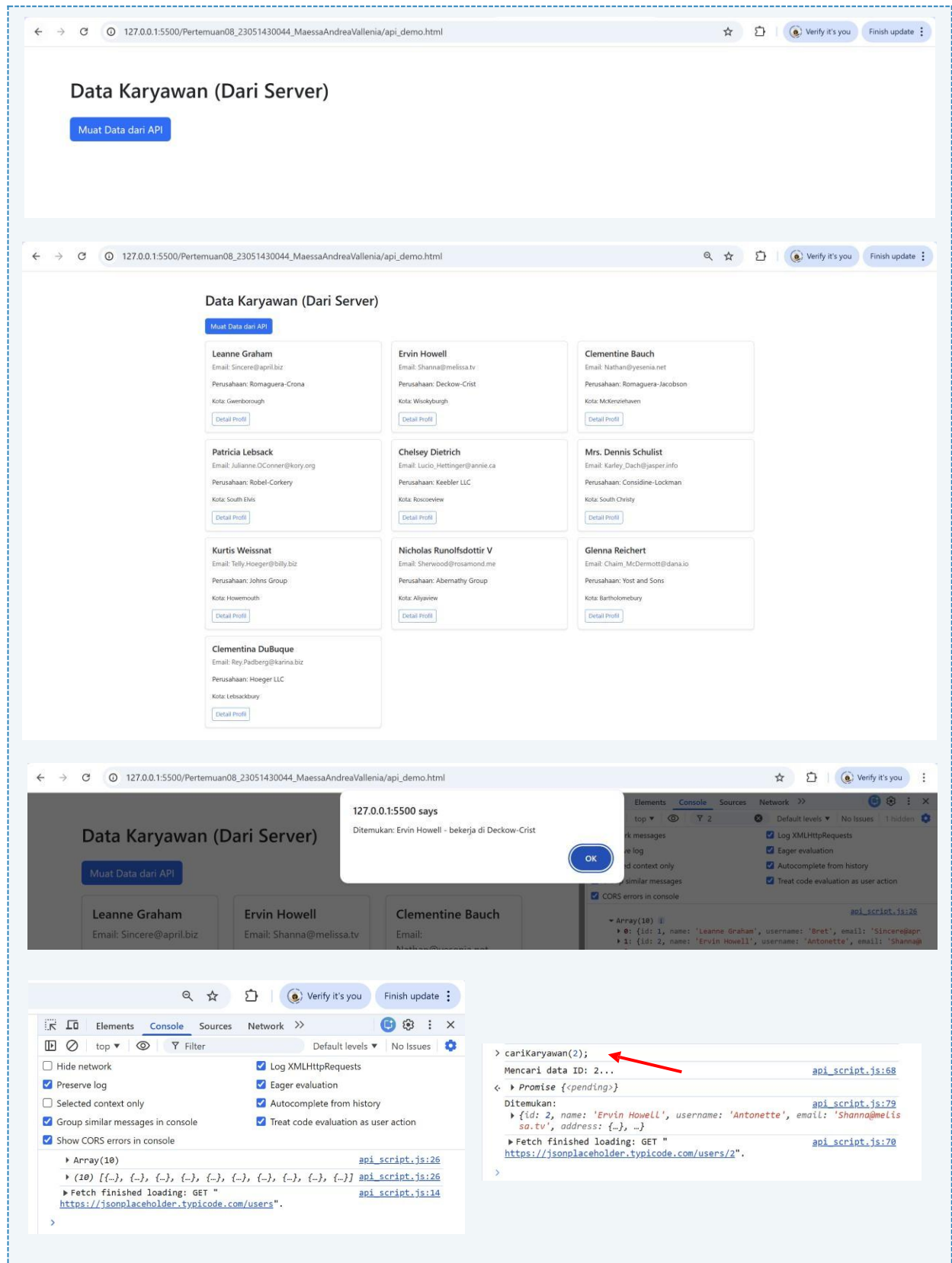
if (!response.ok) {
  throw new Error('Data tidak ditemukan');
}

const data = await response.json();
console.log('Ditemukan:', data);

alert(
  `Ditemukan: ${data.name} - bekerja di ${data.company.name}`
);
} catch (error) {
  console.error(error);
  alert(error.message);
}
}

// Contoh pemanggilan: cariKaryawan(2);
// Anda bisa memanggil fungsi ini lewat console browser untuk tes.
```

Screenshot Hasil:



Gambar E.4 – Hasil Fetch dengan Async/Await (Modern Style)

F. LATIHAN DAN TUGAS MANDIRI

Jawab dan dokumentasikan seluruh latihan/tugas yang ada di bagian "Latihan/Tugas Mandiri" pada modul Anda.

F.1 Latihan 1 – Posting Data

Pertanyaan / Instruksi Latihan:

Buat tombol "Tambah Karyawan Baru". Gunakan method `fetch` dengan opsi `{ method: 'POST', body: ... }` untuk mengirim data dummy ke API. Cek response di console.

Jawaban / Kode Solusi (JS & HTML)

1. JAWABAN KODE JS

```
const btnLoad = document.getElementById('btnLoad');
const container = document.getElementById('containerKaryawan');
const loading = document.getElementById('loading');

// TAMBAHAN LATIHAN 1
const btnTambah = document.getElementById('btnTambah');

// Endpoint API (Simulasi Database)
const API_URL = 'https://jsonplaceholder.typicode.com/users';

btnLoad.addEventListener('click', function () {
  // Tampilkan loading
  loading.classList.remove('d-none');
  container.innerHTML = ''; // Bersihkan konten lama

  // Fetch Data
  fetch(API_URL)
    .then(function (response) {
      // Cek jika response sukses (kode 200-299)
      if (!response.ok) {
        throw new Error('Gagal mengambil data');
      }

      // Parsing data JSON
      return response.json();
    })
    .then(function (dataKaryawan) {
      // Data berhasil didapat
      console.log(dataKaryawan); // Cek di console browser
      renderData(dataKaryawan);
    })
    .catch(function (error) {
      // Jika ada error (misal putus internet)
    });
});
```

```
        container.innerHTML = `
            <div class="alert alert-danger">
                Error: ${error.message}
            </div>
        `;
    })
    .finally(function () {
        // Sembunyikan loading (baik sukses maupun gagal)
        loading.classList.add('d-none');
    });
});

// LATIHAN 1 → TAMBAH DATA (POST)
btnTambah.addEventListener('click', function () {

    const dataBaru = {
        name: "Maessa Andrea Vallenia",
        email: "maessaandrea@email.com",
        company: {
            name: "PT UNY"
        },
        address: {
            city: "Yogyakarta"
        }
    };

    fetch(API_URL, {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json'
        },
        body: JSON.stringify(dataBaru)
    })
    .then(function (response) {

        if (!response.ok) {
            throw new Error('Gagal menambahkan data');
        }

        return response.json();
    })
    .then(function (result) {

        console.log("Response dari server:", result);
        alert("Data karyawan baru berhasil dikirim (cek console)");
    });
});
```



```
    })
    .catch(function (error) {

        console.error(error);
        alert("Terjadi error");

    });
});

function renderData(data) {
    data.forEach(function (karyawan) {
        // Buat elemen card Bootstrap
        const col = document.createElement('div');
        col.className = 'col-md-4 mb-3';

        col.innerHTML = `
            <div class="card h-100 shadow-sm">
                <div class="card-body">
                    <h5 class="card-title">${karyawan.name}</h5>
                    <p class="card-text text-muted">Email:
${karyawan.email}</p>
                    <p class="card-text">Perusahaan:
${karyawan.company.name}</p>
                    <p class="card-text"><small>Kota:
${karyawan.address.city}</small></p>
                    <a href="" class="btn btn-sm btn-outline-primary">Detail
Profil</a>
                </div>
            </div>
        `;

        container.appendChild(col);
    });
}

// Fungsi Async untuk mencari data spesifik
async function cariKaryawan(id) {
    try {
        console.log(`Mencari data ID: ${id}...`);

        const response = await fetch(
            `https://jsonplaceholder.typicode.com/users/${id}`
        );
    }
}
```

```
    if (!response.ok) {
      throw new Error('Data tidak ditemukan');
    }

    const data = await response.json();
    console.log('Ditemukan:', data);

    alert(
      `Ditemukan: ${data.name} - bekerja di ${data.company.name}`
    );
  } catch (error) {
    console.error(error);
    alert(error.message);
  }
}

// Contoh pemanggilan: cariKaryawan(2);
```

2. JAWABAN KODE HTML

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Integrasi API Data Karyawan</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>

<body class="container mt-5">

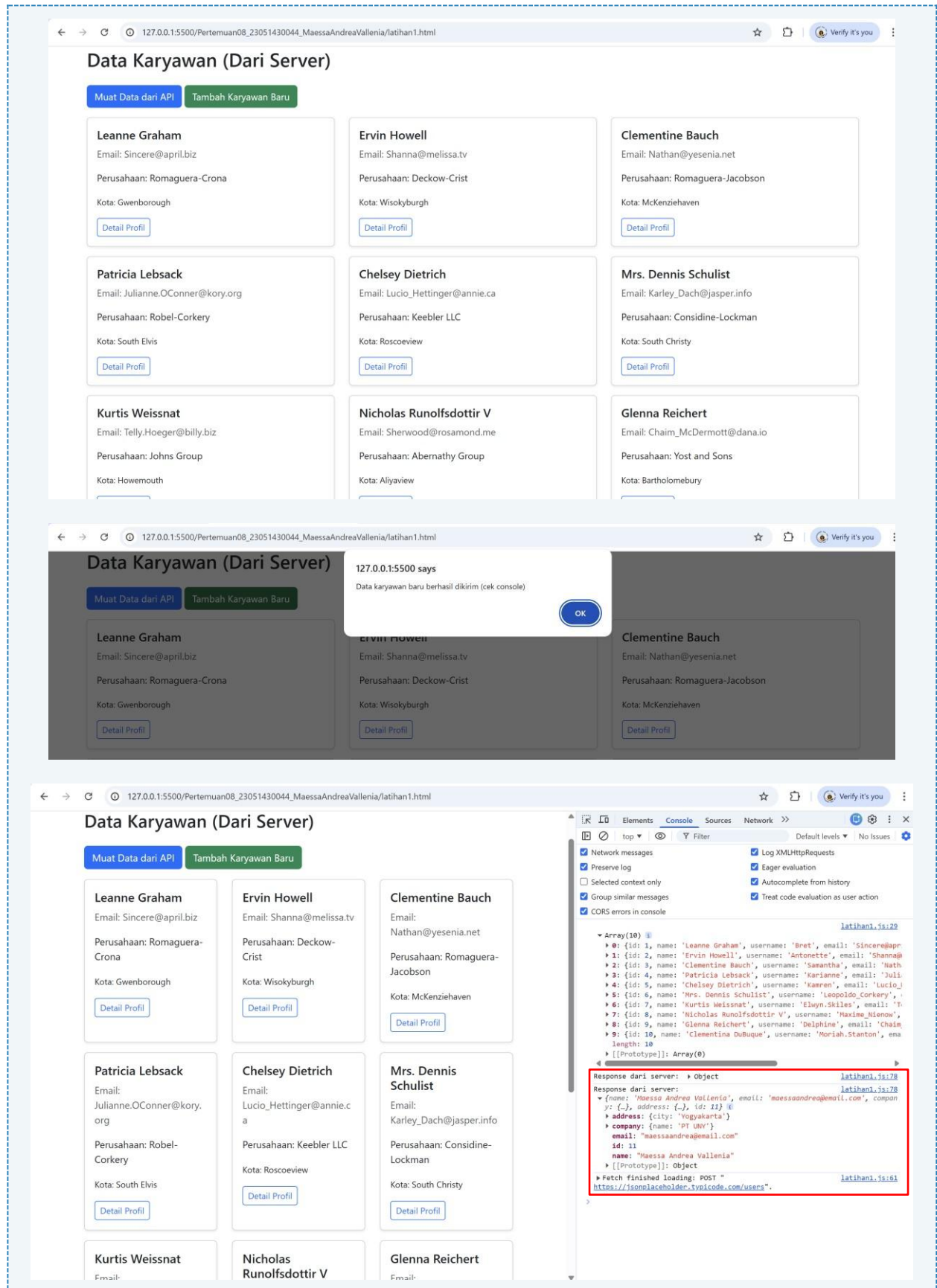
  <h2 class="mb-4">Data Karyawan (Dari Server)</h2>

  <div class="mb-3">
    <button id="btnLoad" class="btn btn-primary">Muat Data dari
API</button>
    <!-- LATIHAN 1 -->
    <button id="btnTambah" class="btn btn-success">Tambah Karyawan
Baru</button>
    <div id="loading" class="spinner-border text-primary d-none"
role="status"></div>
  </div>

  <div class="row" id="containerKaryawan">
    <!-- Data dari API -->
```

```
</div>

<script src="latihan1.js"></script>
</body>
</html>
```



Gambar F.1 – Hasil Posting Data

F.2 Latihan 2 – Filter Array

Pertanyaan / Instruksi Latihan:

Sebelum data dirender ke layar dengan `forEach`, lakukan filter terlebih dahulu. Tampilkan HANYA karyawan yang tinggal di kota yang mengandung huruf "s" (misal: "Southside", "London"). Petunjuk: Gunakan `data.filter(...)` sebelum `data.forEach(...)`.

Jawaban / Kode Solusi:

1. JAWABAN KODE JS

```
// 1. Seleksi Elemen
const btnLoad = document.getElementById('btnLoad');
const container = document.getElementById('containerKaryawan');
const loading = document.getElementById('loading');
const btnTambah = document.getElementById('btnTambah');
const inputFilter = document.getElementById('inputFilter');

// API
const API_URL = 'https://jsonplaceholder.typicode.com/users';

let dataGlobal = [];

// 2. LOAD DATA
btnLoad.addEventListener('click', function () {
  loading.classList.remove('d-none');
  container.innerHTML = '';

  fetch(API_URL)
    .then(function (response) {
      if (!response.ok) {
        throw new Error('Gagal mengambil data');
      }
      return response.json();
    })
    .then(function (dataKaryawan) {
      console.log("Total data:", dataKaryawan.length); // 10
      dataGlobal = dataKaryawan;

      // TAMPILKAN SEMUA DATA (tanpa filter awal)
      tampilkanData(dataGlobal);
    })
    .catch(function (error) {
      container.innerHTML = `
        <div class="alert alert-danger">
          Error: ${error.message}
        </div>
      `;
    });
});
```

```
`;  
    })  
    .finally(function () {  
        loading.classList.add('d-none');  
    });  
});  
  
// 3. TAMBAH DATA  
btnTambah.addEventListener('click', function () {  
    const dataBaru = {  
        name: "Budi Santoso",  
        email: "budi@email.com",  
        company: { name: "PT Maju Mundur" },  
        address: { city: "Yogyakarta" }  
    };  
  
    fetch(API_URL, {  
        method: 'POST',  
        headers: { 'Content-Type': 'application/json' },  
        body: JSON.stringify(dataBaru)  
    })  
    .then(response => {  
        if (!response.ok) throw new Error('Gagal menambahkan');  
        return response.json();  
    })  
    .then(result => {  
        console.log("Berhasil:", result);  
        alert("Data berhasil dikirim!");  
    })  
    .catch(error => {  
        console.error(error);  
        alert("Error!");  
    });  
});  
  
// 4. TAMPIL DATA  
function tampilkanData(data) {  
    container.innerHTML = '';  
  
    data.forEach(function (karyawan) {  
        const col = document.createElement('div');  
        col.className = 'col-md-4 mb-3';  
        col.innerHTML = `  
            <div class="card h-100 shadow-sm">  
                <div class="card-body">  
                    <h5 class="card-title">${karyawan.name}</h5>  
                </div>  
            </div>  
        `;  
        container.appendChild(col);  
    });  
}
```

```

        <p class="card-text text-muted">Email:
        ${karyawan.email}</p>
        <p class="card-text">Perusahaan:
        ${karyawan.company.name}</p>
        <p class="card-text"><small>Kota:
        ${karyawan.address.city}</small></p>
    </div>
</div>
`;
    container.appendChild(col);
});
}

// 5. FILTER DATA (LATIHAN 2)
inputFilter.addEventListener('input', function () {
    const keyword = this.value.toLowerCase();

    let hasil;

    if (keyword === "") {
        // INPUT KOSONG = KEMBALIKAN SEMUA DATA (10 karyawan)
        hasil = dataGlobal;
        console.log("Input kosong → tampilkan semua data");
    } else {
        // Filter berdasarkan keyword (kota)
        hasil = dataGlobal.filter(function (karyawan) {
            const kota = karyawan.address.city.toLowerCase();
            return kota.includes(keyword);
        });
        console.log(`Mencari "${keyword}" → ${hasil.length} hasil`);
    }

    tampilkanData(hasil);
});

```

2. JAWABAN KODE HTML

```

<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <title>Integrasi API Data Karyawan</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css
" rel="stylesheet">
</head>

```

```
<body class="container mt-5">

  <h2 class="mb-4">Data Karyawan (Dari Server)</h2>

  <div class="mb-3">
    <button id="btnLoad" class="btn btn-primary">Muat Data dari
API</button>

    <!-- ✅ LATIHAN 1 -->
    <button id="btnTambah" class="btn btn-success">Tambah Karyawan
Baru</button>

    <div id="loading" class="spinner-border text-primary d-none"
role="status"></div>
  </div>

  <!-- ✅ LATIHAN 2 (FILTER) -->
  <div class="mb-3">
    <input type="text" id="inputFilter" class="form-control"
placeholder="Cari kota (mengandung huruf 's')...">
  </div>

  <!-- ✅ DATA (CUMA SATU) -->
  <div class="row" id="containerKaryawan">
    <!-- Card muncul di sini -->
  </div>

  <script src="latihan2.js"></script>
</body>
</html>
```


Data Karyawan (Dari Server)

Muat Data dari API | Tambah Karyawan Baru

Cari kota (mengandung huruf 's')...

Name	Email	Perusahaan	Kota
Leanne Graham	Sincere@april.biz	Romaguera-Crona	Gwenborough
Ervin Howell	Shanna@melissa.tv	Deckow-Crist	Wisokyburgh
Clementine Bauch	Nathan@yesenia.net	Romaguera-Jacobson	McKenziehaven
Patricia Lebsack	Julianne.OConner@kory.org	Robel-Corkery	South Elvis
Chelsey Dietrich	Lucio_Hettinger@annie.ca	Keebler LLC	Roscoeview
Mrs. Dennis Schulist	Karley_Dach@jasper.info	Considine-Lockman	South Christy
Kurtis Weissnat	Telly.Hoeger@billy.biz	Johns Group	Howemouth
Nicholas Runolfsdottir V	Sherwood@rosamond.me	Abernathy Group	Aliyaview
Glenna Reichert	Chaim_McDermott@dana.io	Yost and Sons	Bartholomebury

Data Karyawan (Dari Server)

Muat Data dari API | Tambah Karyawan Baru

s

Name	Email	Perusahaan	Kota
Ervin Howell	Shanna@melissa.tv	Deckow-Crist	Wisokyburgh
Patricia Lebsack	Julianne.OConner@kory.org	Robel-Corkery	South Elvis
Chelsey Dietrich	Lucio_Hettinger@annie.ca	Keebler LLC	Roscoeview
Mrs. Dennis Schulist	Karley_Dach@jasper.info	Considine-Lockman	South Christy
Clementina DuBuque	Rey.Padberg@karina.biz	Hoeger LLC	Lebsackbury

Browser Console:

```
Total data: 10  
Mencari "s" -> 5 hasil
```

Gambar F.2 – Hasil Filter Array

F.3 Tugas Proyek Mini – Integrasi Data Produksi

Deskripsi proyek mini:

Anggap API `https://jsonplaceholder.typicode.com/posts`` adalah database "Laporan Insiden".

- Tampilkan daftar 10 laporan insiden terbaru (Title = Judul Insiden, Body = Deskripsi).
- Di setiap card, tambahkan tombol "Tindak Lanjut". Saat diklik, `alert()` menampilkan "Tiket ID sedang diproses oleh Tim Maintenance".

Penjelasan pendekatan/solusi yang digunakan:

Proyek mini ini dilakukan melalui beberapa tahapan yang sistematis, dimulai dari perancangan tampilan hingga implementasi

1. Tahap pertama adalah menyiapkan folder proyek dan membuat file utama yang terdiri dari file HTML untuk tampilan, file JavaScript untuk logika program, serta file CSS untuk memperindah tampilan dashboard.
2. Selanjutnya, dibuat struktur tampilan menggunakan HTML. Tampilan terdiri dari judul dashboard laporan insiden, nama mahasiswa, NIM, tombol "Muat Laporan", tombol "Reset", kolom pencarian judul, kolom pencarian ID, dropdown status, loading spinner, dan container untuk menampilkan card laporan insiden.
3. Setiap elemen penting seperti tombol, input, dropdown, loading spinner, dan container kemudian diberikan atribut id agar dapat diakses menggunakan JavaScript melalui DOM.
4. Setelah itu, file JavaScript dan CSS dihubungkan ke dalam file HTML agar seluruh fungsi dan tampilan dapat berjalan saat halaman dibuka.
5. Pada tahap berikutnya, dilakukan pengambilan elemen HTML menggunakan DOM dengan method `getElementById()`, misalnya untuk mengambil tombol load, tombol reset, input pencarian, filter ID, dropdown status, dan container data.
6. Setelah itu, ditentukan endpoint API `https://jsonplaceholder.typicode.com/posts` sebagai simulasi database laporan insiden.
7. Selanjutnya, ditambahkan event listener pada tombol "Muat Laporan". Saat tombol ditekan, sistem akan menampilkan loading spinner dan melakukan request data ke server menggunakan Fetch API dengan method GET.
8. Jika proses request berhasil, sistem akan mengambil 10 data pertama dari API dan menambahkan properti baru berupa `ditindakLanjut` dengan nilai awal false. Properti ini digunakan untuk menentukan status laporan, apakah masih pending atau sudah selesai ditindak lanjuti.
9. Setelah data berhasil diambil, dibuat fungsi `tampilkanData()` untuk menampilkan data ke layar dalam bentuk card Bootstrap. Setiap card menampilkan ID laporan, judul, deskripsi singkat, badge status, dan tombol tindak lanjut.
10. Badge status dibuat dinamis, yaitu menampilkan warna kuning dengan tulisan "Pending" jika laporan belum ditindak lanjuti, dan warna hijau dengan tulisan "Selesai" jika laporan sudah diproses.
11. Selanjutnya, dibuat fungsi `filterData()` untuk melakukan pencarian dan penyaringan data. Data dapat difilter berdasarkan judul laporan, ID laporan, maupun status laporan.

12. Input pencarian judul menggunakan method includes() agar pengguna dapat mencari laporan berdasarkan kata tertentu secara real-time.
13. Filter ID digunakan untuk menampilkan data dengan ID tertentu sesuai angka yang dimasukkan oleh pengguna.
14. Dropdown status digunakan untuk memfilter laporan berdasarkan status “Pending”, “Selesai”, atau menampilkan semua laporan.
15. Event listener kemudian ditambahkan pada input pencarian, input ID, dan dropdown status agar proses filter berjalan otomatis setiap kali pengguna mengetik atau mengganti pilihan filter.
16. Sistem juga dilengkapi dengan tombol “Tindak Lanjut”. Saat tombol ditekan, status laporan akan berubah menjadi selesai, warna card berubah, tombol menjadi disabled, dan sistem menampilkan alert bahwa tiket sedang diproses oleh Tim Maintenance.
17. Selain itu, dibuat tombol “Reset” untuk menghapus seluruh hasil pencarian dan filter, serta mengosongkan data yang sedang ditampilkan.
18. Jika terjadi kegagalan saat mengambil data dari server, sistem akan menampilkan pesan error agar pengguna mengetahui bahwa proses tidak berhasil.
19. Terakhir, dilakukan pengujian aplikasi dengan cara memuat data, mencari laporan berdasarkan judul, mencari berdasarkan ID, memfilter berdasarkan status, melakukan tindak lanjut, dan melakukan reset untuk memastikan seluruh fitur berjalan dengan baik.

Jawaban / Kode Solusi:

1. JAWABAN KODE CSS

```
body {
    background: linear-gradient(to right, #f1f3f5, #ffffff);
}

/* Header */
.header-box {
    background: #33363a;
    color: white;
    padding: 20px;
    border-radius: 15px;
}

/* Card */
.card {
    border: none;
    border-left: 5px solid #2b2e31;
    border-radius: 12px;
    transition: 0.3s;
}

.card:hover {
    transform: scale(1.03);
}
```

```
}

/* Button */
.btn-outline-dark {
    border-radius: 10px;
}

.card-square {
    border-radius: 10px !important; /* Kotak sedikit melengkung */
}

.card-square.border-success {
    border: 10px solid #28a745 !important; /* Border hijau tebal */
    background-color: #f8fff9 !important; /* Background hijau muda */
}

.btn-square {
    border-radius: 10px !important; /* Tombol kotak */
}

.badge-square {
    border-radius: 10px !important; /* Badge kotak */
}
```

2. JAWABAN KODE JS

```
// SELEKSI ELEMENT
const btnLoad = document.getElementById('btnLoad');
const btnReset = document.getElementById('btnReset');
const container = document.getElementById('containerInsiden');
const loading = document.getElementById('loading');
const searchInput = document.getElementById('searchInput');

// FILTER BARU
const filterId = document.getElementById('filterId');
const filterStatus = document.getElementById('filterStatus');

// DATA LAPORAN INDONESIA
const laporanIndonesia = [
    {
        title: "Kerusakan Conveyor Produksi",
        body: "Conveyor pada lini produksi 2 berhenti beroperasi akibat motor penggerak mengalami overheating."
    },
    {
        title: "Kebocoran Pipa Air Pendingin",
```

```
        body: "Ditemukan kebocoran pada pipa air pendingin di area mesin boiler sehingga perlu perbaikan segera."
    },
    {
        title: "Gangguan Sistem ERP",
        body: "Sistem ERP tidak dapat diakses selama 15 menit sehingga proses input data produksi tertunda."
    },
    {
        title: "Lampu Gudang Mati",
        body: "Beberapa lampu di area gudang bahan baku padam dan mengganggu aktivitas pengambilan material."
    },
    {
        title: "Keterlambatan Bahan Baku",
        body: "Pengiriman bahan baku dari supplier terlambat sehingga jadwal produksi perlu disesuaikan."
    },
    {
        title: "Mesin Packing Overheat",
        body: "Mesin packing mengalami peningkatan suhu di atas batas normal saat digunakan secara terus-menerus."
    },
    {
        title: "Kesalahan Input Data Produksi",
        body: "Operator melakukan kesalahan input jumlah output sehingga data produksi perlu diperbaiki."
    },
    {
        title: "Keterbatasan Stok Barang",
        body: "Stok komponen utama di gudang hampir habis dan berpotensi menghambat proses produksi."
    },
    {
        title: "Gangguan Jaringan Internet",
        body: "Koneksi internet di area kantor produksi terputus sehingga sistem monitoring tidak dapat digunakan."
    },
    {
        title: "Alarm Keselamatan Aktif",
        body: "Alarm keselamatan berbunyi akibat sensor mendeteksi suhu ruangan yang terlalu tinggi."
    }
];

// API
```

```
const API_URL = 'https://jsonplaceholder.typicode.com/posts';

// SIMPAN DATA GLOBAL
let dataGlobal = [];

// LOAD DATA
btnLoad.addEventListener('click', function () {
  loading.classList.remove('d-none');
  container.innerHTML = '';

  fetch(API_URL)
    .then(response => {
      if (!response.ok) {
        throw new Error('Gagal mengambil data');
      }
      return response.json();
    })
    .then(data => {
      dataGlobal = data.slice(0, 10).map((item, index) => ({
        id: item.id,
        title: laporanIndonesia[index].title,
        body: laporanIndonesia[index].body,
        ditindakLanjut: false
      }));

      tampilkanData(dataGlobal);
    })
    .catch(error => {
      container.innerHTML = `
        <div class="alert alert-danger">
          ${error.message}
        </div>
      `;
    })
    .finally(() => {
      loading.classList.add('d-none');
    });
});

// TAMPILKAN DATA
function tampilkanData(data) {
  container.innerHTML = '';

  if (data.length === 0) {
    container.innerHTML = `
      <div class="alert alert-warning">
    `;
  }
}
```

```

        Data tidak ditemukan.
    </div>
    `;
    return;
}

data.forEach(item => {
    const col = document.createElement('div');
    col.className = 'col-md-4 mb-3';

    const statusClass = item.ditindakLanjut
        ? 'border-success shadow-lg'
        : 'shadow-sm';

    col.innerHTML = `
        <div class="card h-100 ${statusClass} border">
            <div class="card-body">
                <div class="d-flex justify-content-between align-items-
start mb-2">
                    <h5 class="card-title mb-0">
                        #${item.id} - ${item.title}
                    </h5>

                    <span class="badge ${item.ditindakLanjut ? 'bg-
success' : 'bg-warning text-dark'}">
                        ${item.ditindakLanjut ? 'Selesai' : 'Pending'}
                    </span>
                </div>

                <p class="card-text text-muted small">
                    ${item.body}
                </p>

                <button
                    class="btn ${item.ditindakLanjut ? 'btn-success w-
100' : 'btn-outline-dark w-100'}"
                    ${item.ditindakLanjut ? 'disabled' : ''}
                    onclick="tindakLanjut(${item.id})">
                        ${item.ditindakLanjut ? 'Sudah Ditindak Lanjut' :
'Tindak Lanjut'}
                </button>
            </div>
        </div>
    `;

    container.appendChild(col);

```

```
});  
}  
  
// FILTER DATA  
function filterData() {  
    const keyword = searchInput.value.toLowerCase();  
    const idKeyword = filterId.value;  
    const statusKeyword = filterStatus.value;  
  
    let hasil = dataGlobal.filter(item => {  
        const cocokJudul = item.title.toLowerCase().includes(keyword);  
  
        const cocokId = idKeyword === '' || item.id == idKeyword;  
  
        let cocokStatus = true;  
  
        if (statusKeyword === 'pending') {  
            cocokStatus = item.ditindakLanjut === false;  
        } else if (statusKeyword === 'selesai') {  
            cocokStatus = item.ditindakLanjut === true;  
        }  
  
        return cocokJudul && cocokId && cocokStatus;  
    });  
  
    tampilkanData(hasil);  
}  
  
// EVENT FILTER  
searchInput.addEventListener('input', filterData);  
filterId.addEventListener('input', filterData);  
filterStatus.addEventListener('change', filterData);  
  
// RESET  
btnReset.addEventListener('click', function () {  
    if (confirm('Yakin mau reset semua data & status?')) {  
        container.innerHTML = '';  
        dataGlobal = [];  
        searchInput.value = '';  
        filterId.value = '';  
        filterStatus.value = 'semua';  
    }  
});  
  
// TINDAK LANJUT  
function tindakLanjut(id) {
```



```

const item = dataGlobal.find(item => item.id === id);

if (item && !item.ditindakLanjut) {
  item.ditindakLanjut = true;

  filterData();

  alert(`Tiket ID ${id} sedang diproses oleh Tim Maintenance`);
}
}

```

3. JAWABAN KODE HTML

```

<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Dashboard Laporan Insiden</title>

  <!-- Bootstrap -->
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css
" rel="stylesheet">

  <!-- CSS -->
  <link rel="stylesheet" href="ProyekMini.css">
</head>

<body class="container mt-5">

  <!-- HEADER -->
  <div class="header-box text-center mb-4">
    <h2 class="fw-bold">Dashboard Laporan Insiden</h2>
    <p class="mb-1">Maessa Andrea Vallenia</p>
    <p class="mb-0">23051430044</p>
  </div>

  <!-- CONTROL -->
  <div class="row g-2 mb-3">

    <!-- Tombol Load -->
    <div class="col-md-2">
      <button id="btnLoad" class="btn btn-dark w-100">
        Muat Laporan
      </button>

```

```
</div>

<!-- Tombol Reset -->
<div class="col-md-2">
  <button id="btnReset" class="btn btn-secondary w-100">
    Reset
  </button>
</div>

<!-- Cari Judul -->
<div class="col-md-4">
  <input
    type="text"
    id="searchInput"
    class="form-control"
    placeholder="Cari judul insiden..."
  >
</div>

<!-- Cari ID -->
<div class="col-md-2">
  <input
    type="number"
    id="filterId"
    class="form-control"
    placeholder="Cari ID"
  >
</div>

<!-- Dropdown Status -->
<div class="col-md-2">
  <select id="filterStatus" class="form-select">
    <option value="semua">Semua Status</option>
    <option value="pending">Pending</option>
    <option value="selesai">Selesai</option>
  </select>
</div>

</div>

<!-- LOADING -->
<div class="mb-3">
  <div id="loading" class="spinner-border text-dark d-none"
role="status">
    <span class="visually-hidden">Loading...</span>
  </div>
```

```
</div>

<!-- DATA -->
<div class="row mt-3" id="containerInsiden">
  <!-- Card laporan akan muncul di sini -->
</div>

<!-- JS -->
<script src="ProyekMini.js"></script>
</body>
</html>
```

Dashboard Laporan Insiden

Maessa Andrea Vallenia
23051430044

Muat Laporan

Reset

Cari judul insiden...

Cari ID

Semua Status

#1 - Kerusakan Conveyor Produksi Pending

Conveyor pada lini produksi 2 berhenti beroperasi akibat motor penggerak mengalami overheating.

Tindak Lanjut

#2 - Kebocoran Pipa Air Pendingin Pending

Ditemukan kebocoran pada pipa air pendingin di area mesin boiler sehingga perlu perbaikan segera.

Tindak Lanjut

#3 - Gangguan Sistem ERP Pending

Sistem ERP tidak dapat diakses selama 15 menit sehingga proses input data produksi tertunda.

Tindak Lanjut

#4 - Lampu Gudang Mati Pending

Beberapa lampu di area gudang bahan baku padam dan mengganggu aktivitas pengambilan material.

Tindak Lanjut

#5 - Keterlambatan Bahan Baku Pending

Pengiriman bahan baku dari supplier terlambat sehingga jadwal produksi perlu disesuaikan.

Tindak Lanjut

#6 - Mesin Packing Overheat Pending

Mesin packing mengalami peningkatan suhu di atas batas normal saat digunakan secara terus-menerus.

Tindak Lanjut

#7 - Kesalahan Input Data Produksi Pending

Operator melakukan kesalahan input jumlah output sehingga data produksi perlu diperbaiki.

Tindak Lanjut

#8 - Keterbatasan Stok Barang Pending

Stok komponen utama di gudang hampir habis dan berpotensi menghambat proses produksi.

Tindak Lanjut

#9 - Gangguan Jaringan Internet Pending

Koneksi internet di area kantor produksi terputus sehingga sistem monitoring tidak dapat digunakan.

Tindak Lanjut

127.0.0.1:5500 says

Tiket ID 1 sedang diproses oleh Tim Maintenance

OK

Dashboard Laporan Insiden

Maessa Andrea Vallenia
23051430044

Muat Laporan

Reset

Cari judul insiden...

Cari ID

Semua Status

#1 - Kerusakan Conveyor Produksi Selesai

Conveyor pada lini produksi 2 berhenti beroperasi akibat motor penggerak mengalami overheating.

Sudah Ditindak Lanjut

#2 - Kebocoran Pipa Air Pendingin Pending

Ditemukan kebocoran pada pipa air pendingin di area mesin boiler sehingga perlu perbaikan segera.

Tindak Lanjut

#3 - Gangguan Sistem ERP Pending

Sistem ERP tidak dapat diakses selama 15 menit sehingga proses input data produksi tertunda.

Tindak Lanjut

#4 - Lampu Gudang Mati Pending

Beberapa lampu di area gudang bahan baku padam dan mengganggu aktivitas pengambilan material.

Tindak Lanjut

#5 - Keterlambatan Bahan Baku Pending

Pengiriman bahan baku dari supplier terlambat sehingga jadwal produksi perlu disesuaikan.

Tindak Lanjut

#6 - Mesin Packing Overheat Selesai

Mesin packing mengalami peningkatan suhu di atas batas normal saat digunakan secara terus-menerus.

Sudah Ditindak Lanjut

#7 - Kesalahan Input Data Produksi Pending

Operator melakukan kesalahan input jumlah output sehingga data produksi perlu diperbaiki.

Tindak Lanjut

#8 - Keterbatasan Stok Barang Selesai

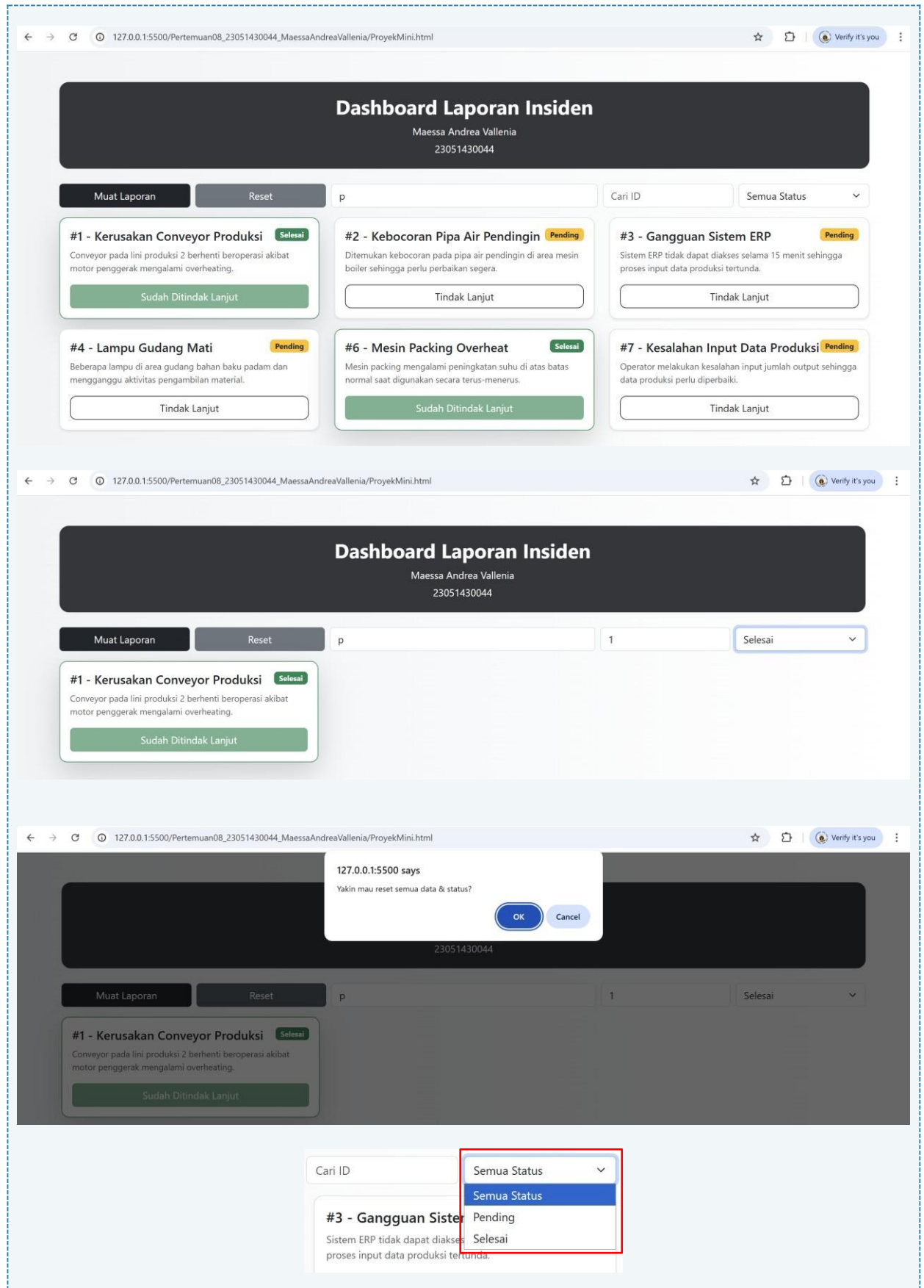
Stok komponen utama di gudang hampir habis dan berpotensi menghambat proses produksi.

Sudah Ditindak Lanjut

#9 - Gangguan Jaringan Internet Pending

Koneksi internet di area kantor produksi terputus sehingga sistem monitoring tidak dapat digunakan.

Tindak Lanjut



Gambar F.3 – Hasil Proyek Mini Integrasi Data Produksi

G. PEMBAHASAN

Pada praktikum Pertemuan 8, mahasiswa mempelajari konsep Asynchronous JavaScript, Fetch API, Promise, Async/Await, JSON, dan REST API untuk membangun aplikasi web yang dapat mengambil data dari server secara real-time. Praktikum ini menunjukkan bahwa aplikasi web tidak hanya menampilkan data statis, tetapi juga dapat terhubung dengan sumber data eksternal seperti database, API perusahaan, sensor IoT, dan ERP.

HTML digunakan untuk membuat struktur halaman, seperti tombol, loading spinner, form pencarian, dan container data. Bootstrap digunakan agar tampilan lebih rapi, responsif, dan menyerupai dashboard monitoring sederhana.

JavaScript digunakan untuk mengambil data dari API, memproses data JSON, menampilkan data ke halaman, dan menangani interaksi pengguna. Konsep asynchronous memungkinkan program tetap berjalan saat menunggu balasan dari server, sehingga halaman tidak freeze.

Fetch API digunakan untuk melakukan request GET dan POST ke endpoint JSONPlaceholder. Promise diterapkan melalui `.then()`, `.catch()`, dan `.finally()` untuk menangani proses sukses, error, dan proses akhir. Selain itu, Async/Await digunakan agar penulisan kode asynchronous lebih singkat dan mudah dibaca.

Pada Latihan 1, mahasiswa mengirim data dummy ke server menggunakan method POST. Pada Latihan 2, mahasiswa memfilter data karyawan berdasarkan kota yang mengandung huruf "s". Pada proyek mini, mahasiswa membuat dashboard laporan insiden yang menampilkan data dari API dan tombol tindak lanjut untuk setiap laporan.

Secara keseluruhan, praktikum ini memberikan pemahaman mengenai cara aplikasi web berkomunikasi dengan server, mengambil data, memproses data, dan menampilkan informasi secara dinamis.

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum menunjukkan bahwa implementasi program telah sesuai dengan teori DOM, event handling, validasi input, dan LocalStorage.

Struktur HTML telah menyediakan elemen yang dapat diakses JavaScript, seperti form input dan tabel. Dengan DOM, elemen tersebut dapat dimanipulasi secara langsung menggunakan `getElementById()`.

Hasil praktikum menunjukkan bahwa implementasi program telah sesuai dengan teori asynchronous programming, Fetch API, Promise, Async/Await, JSON, dan REST API. Konsep asynchronous berhasil diterapkan karena program tetap dapat berjalan tanpa menunggu balasan server selesai. Saat tombol "Muat Data" ditekan, loading spinner muncul terlebih dahulu, kemudian data ditampilkan setelah response diterima. Hal ini sesuai dengan teori asynchronous programming yang memungkinkan aplikasi tetap responsif.

Penggunaan Fetch API juga telah sesuai dengan teori karena request data dilakukan menggunakan method `fetch()` ke endpoint API eksternal. Data yang diterima berbentuk JSON, kemudian diproses menggunakan `response.json()` sebelum ditampilkan ke halaman.

Penerapan Promise melalui `.then()`, `.catch()`, dan `.finally()` menunjukkan bahwa alur request berhasil dijalankan sesuai teori. `.then()` digunakan untuk menampilkan data, `.catch()`

digunakan untuk menangani error, sedangkan `.finally()` digunakan untuk menyembunyikan loading spinner.

Selain itu, penggunaan `Async/Await` untuk mencari data karyawan berdasarkan ID juga telah sesuai dengan teori. Kode menjadi lebih singkat, rapi, dan mudah dipahami dibandingkan penggunaan `Promise` berantai.

Pada latihan filter data, method `filter()` berhasil digunakan untuk menampilkan hanya data tertentu sebelum dilakukan `forEach()`. Hal ini membuktikan bahwa array JavaScript dapat dimanipulasi sebelum ditampilkan ke antarmuka.

Secara keseluruhan, sistem yang dibuat telah mampu:

1. Mengambil data dari server menggunakan `Fetch API`.
2. Menampilkan data JSON ke halaman web secara dinamis.
3. Menangani error jika request gagal.
4. Menggunakan `Promise` dan `Async/Await`.
5. Memfilter data sebelum ditampilkan.
6. Mengirim data dummy menggunakan method `POST`.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Filter data tidak bekerja	Memastikan method <code>filter()</code> dijalankan sebelum <code>forEach()</code> dan format data yang difilter sesuai dengan kondisi yang ditentukan.
Tombol <code>POST</code> tidak menampilkan response	Memastikan method <code>POST</code> , header <code>Content-Type: application/json</code> , dan <code>JSON.stringify()</code> sudah ditulis dengan benar pada konfigurasi <code>fetch</code> .

G.3 Kaitan dengan Penerapan di Industri

Materi pada praktikum ini memiliki keterkaitan yang sangat erat dengan penerapan di dunia industri, khususnya pada sistem monitoring, dashboard produksi, ERP, supply chain, dan IoT. Dalam industri, data sering kali berada pada server pusat dan diakses dari berbagai lokasi. Sebagai contoh, data stok gudang dapat tersimpan di server Jakarta, sementara operator di pabrik Surabaya harus tetap dapat mengaksesnya secara real-time. Oleh karena itu, konsep asynchronous sangat penting agar dashboard tidak freeze saat menunggu data dari server.

`Fetch API` dan `REST API` banyak digunakan pada sistem ERP perusahaan untuk mengambil data karyawan, data produksi, data inventori, hingga laporan maintenance. Dengan adanya integrasi API, sistem dapat saling bertukar data secara otomatis tanpa perlu input manual. Konsep JSON digunakan hampir di seluruh sistem modern karena formatnya ringan, mudah dibaca, dan mudah diproses oleh berbagai bahasa pemrograman. Dalam industri, JSON sering digunakan untuk pertukaran data antara frontend, backend, database, sensor IoT, dan cloud server.

Fitur filter data yang dipelajari pada praktikum juga sangat relevan dalam dunia industri. Misalnya, supervisor produksi dapat memfilter data mesin berdasarkan lokasi, status, atau jenis kerusakan agar proses analisis menjadi lebih cepat. Pada proyek mini laporan insiden, konsep ini menyerupai dashboard maintenance di industri. Setiap insiden yang muncul dapat ditindaklanjuti oleh tim maintenance berdasarkan tiket yang masuk. Sistem seperti ini membantu perusahaan dalam memantau gangguan operasional, mempercepat respon perbaikan, dan mengurangi downtime mesin.

Dengan memahami konsep asynchronous, API, JSON, Promise, dan Async/Await, mahasiswa Teknik Industri memiliki bekal yang kuat untuk mengembangkan sistem informasi yang lebih modern, terintegrasi, dan sesuai dengan kebutuhan industri digital yang terus berkembang.

H. KESIMPULAN

Tuliskan 3–5 poin kesimpulan yang merangkum hal-hal yang dipelajari dan dicapai pada praktikum modul ini. Kesimpulan harus spesifik, bukan bersifat umum.

1	Mahasiswa mampu memahami perbedaan antara synchronous dan asynchronous programming serta dampaknya terhadap performa aplikasi web.
2	Mahasiswa dapat menggunakan Fetch API untuk mengambil data dari server eksternal dan menampilkannya secara dinamis ke halaman web.
3	Mahasiswa memahami penggunaan format JSON sebagai media pertukaran data antara server dan aplikasi web.
4	Mahasiswa dapat mengembangkan fitur interaktif seperti filter data, yang mendukung proses pencarian dan analisis data secara cepat dan efisien, sesuai dengan kebutuhan pengambilan keputusan di industri.
5	Mahasiswa memahami konsep event-driven programming, di mana sistem merespon input pengguna secara real-time, sehingga menghasilkan aplikasi yang responsif dan mendekati implementasi sistem kontrol dan monitoring di industri modern.

I. DAFTAR PUSTAKA

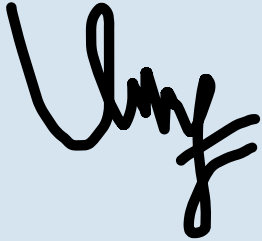
Tuliskan semua sumber referensi yang digunakan dalam format APA 7th Edition. Minimal 3 referensi yang relevan (termasuk Manual Praktikum dan sumber online resmi seperti MDN, Reactjs.org, dll).

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.

No.	Referensi (Format APA 7th Edition)
[2]	Kom, I. L. M. (2023). <i>Dasar Algoritma dan Pemograman Javascript</i> . Irvan Lewenusa, M. Kom.
[3]	Sholikhan, M. (2022). <i>Html, Css Dan Javascript</i> . Penerbit Yayasan Prima Agus Teknik, 1-343 .
[4]	Tinambunan, M. H., Siregar, A. H., & Ginting, S. (2024). FULLSTACK PROGRAMMING: MEMBANGUN APPLICATION PROGRAMMING INTERFACE (API) DENGAN LARAVEL. <i>Penerbit Tahta Media</i> .
[5]	Pattinama, Y. L., & Susanti, I. (2023). Implementasi Rest API Web Service Dengan Otentifikasi JSON Web Token Untuk Aplikasi Properti. <i>Informatik: Jurnal Ilmu Komputer</i> , 19(1), 81-89.
[6]	https://www.javascripttutorial.net/
[7]	https://developer.mozilla.org/en-US/docs/Web/JavaScript
[8]	https://www.geeksforgeeks.org/
[9]	https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Using_promises
[10]	https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API
[11]	https://www.w3schools.com/js/js_json.asp
[12]	https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Async_JS
[13]	https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/try...catch

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa  Maessa Andrea Vallenia NIM: 23051430044	Diperiksa oleh Asisten / Dosen Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____
--	---

MODUL 9

PENGENALAN REACT.JS (COMPONENT, JSX, PROPS)

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 9 (Pertemuan 9)
Topik Utama	Setup Lingkungan React, Konsep Component, JSX Syntax, dan Props (Data Passing).
Sub-Topik	Pengenalan React, Perbedaan Library dan Framework, Setup Environment React Menggunakan Vite, Struktur Folder Project React, JSX Syntax, Perbedaan HTML dan JSX, Functional Component, Reusable Component, Import dan Export Component, Props dan Data Passing, Conditional Rendering, Bootstrap pada React, Destructuring Props, Default Props.
Durasi Praktikum	340 Menit
Tanggal Praktikum	24 April 2026
Nama Mahasiswa	Maessa Andrea Vallenia
NIM	23051430044
Kelas	A

B. TUJUAN PEMBELAJARAN

Tuliskan tujuan pembelajaran yang ingin dicapai pada modul ini, sesuai dengan yang tertera di Manual Praktikum.

1	Memahami konsep Library vs Framework dan mengapa React dipilih untuk aplikasi skala besar.
2	Mampu menyiapkan environment pengembangan React menggunakan Vite.
3	Memahami sintaks JSX (JavaScript XML).
4	Mampu membuat Functional Component yang menerima dan menampilkan data melalui Props.

C. ALAT DAN BAHAN

Sebutkan seluruh perangkat keras, perangkat lunak, dan sumber referensi yang digunakan selama praktikum.

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	<i>Code Editor</i>
2	Google Chrome / Browser Modern	Versi terbaru	<i>Testing & Debugging</i>
3	Live Server	Versi 5.7.9	Menjalankan HTML dan memantau perubahan file
4	JavaScript (Built-in Browser)	ES6 (Default Browser Modern)	Bahasa pemrograman untuk logika dan perhitungan
5	CodeSnap	Versi 1.3.4	Dokumentasi kode program
6	Git dan GitHub	Versi 2.53.0	Version control system (VCS) berbasis Git
7	Koneksi Internet	Stabil	Mengakses Library dan Framework
8	Console Browser (Developer Tools)	Default Browser	Melihat output program dan debugging
9	Bootstrap	Versi 5.3	Membantu pembuatan UI seperti button, card, dan alert pada panel control
10	File praktikum	Pertemuan sebelumnya	Digunakan sebagai dasar pengembangan panel kontrol
11	Node.js	Versi 18 ke atas	Runtime JavaScript

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
			untuk menjalankan React dan npm
12	npm (Node Package Manager)	Versi terbaru	Mengelola package dan dependensi React
13	Vite	Versi terbaru	Tools untuk membuat dan menjalankan project React
14	React	Versi terbaru	Library JavaScript untuk membangun user interface
15	JSX	Built-in React	Sintaks untuk menulis HTML di dalam JavaScript

D. DASAR TEORI

1. React merupakan library JavaScript yang digunakan untuk membangun antarmuka pengguna (user interface) secara dinamis dan interaktif. React banyak digunakan pada aplikasi skala besar karena memungkinkan pengembang membagi tampilan menjadi komponen-komponen kecil yang dapat digunakan kembali (reusable).
2. Library dan framework memiliki perbedaan pada tingkat kontrol program. Framework mengatur alur kerja aplikasi secara menyeluruh, sedangkan library hanya digunakan ketika diperlukan oleh developer. React termasuk library karena developer memiliki kontrol penuh terhadap struktur aplikasi yang dibuat.
3. Vite merupakan tools modern yang digunakan untuk membuat project React dengan proses setup yang lebih cepat dibandingkan tools sebelumnya. Vite menyediakan development server yang ringan, cepat, dan mendukung fitur hot reload sehingga perubahan kode dapat langsung terlihat di browser.
4. JSX (JavaScript XML) adalah sintaks khusus pada React yang memungkinkan penulisan kode HTML di dalam JavaScript. JSX

mempermudah pembuatan tampilan karena struktur HTML dapat langsung ditulis di dalam file JavaScript tanpa harus dipisahkan.

5. Component merupakan bagian utama dalam React. Component adalah potongan kode yang digunakan untuk membangun elemen antarmuka tertentu, seperti card, tabel, form, atau tombol. Dengan component, tampilan dapat dibuat lebih terstruktur, rapi, dan mudah digunakan kembali. Functional Component adalah jenis component yang ditulis dalam bentuk function JavaScript. Functional Component menerima data melalui parameter props dan mengembalikan tampilan berupa JSX.
6. Props (Properties) digunakan untuk mengirim data dari parent component ke child component. Props bekerja seperti parameter pada function, sehingga satu component dapat digunakan berkali-kali dengan isi data yang berbeda.
7. Destructuring Props merupakan teknik untuk mengambil data dari props secara langsung di parameter function. Teknik ini membuat kode menjadi lebih ringkas dan mudah dibaca.
8. Default Props digunakan untuk memberikan nilai bawaan pada suatu props apabila data tidak dikirimkan oleh parent component. Hal ini membantu mencegah error ketika ada data yang kosong.
9. Import dan export digunakan agar component dapat dipisahkan ke dalam file yang berbeda namun tetap dapat digunakan kembali pada file lain. Dengan konsep ini, struktur project React menjadi lebih modular dan mudah dikembangkan.
10. Bootstrap dapat digunakan pada React untuk mempercepat proses pembuatan tampilan antarmuka. Class Bootstrap seperti container, row, card, dan button dapat digunakan langsung pada JSX dengan atribut className.
11. React menggunakan atribut className sebagai pengganti class pada HTML biasa. Hal ini dilakukan karena class merupakan keyword khusus dalam JavaScript. React mendukung konsep reusable component, yaitu satu component dapat dipanggil berkali-kali dengan data yang berbeda. Contohnya, component KartuMesin dapat digunakan untuk menampilkan banyak data mesin tanpa harus menulis ulang struktur card.

Secara keseluruhan, React membantu pengembangan aplikasi web modern menjadi lebih terstruktur, mudah dirawat, dan efisien. Dengan memanfaatkan component, JSX, dan props, tampilan aplikasi dapat dibuat lebih dinamis dan sesuai dengan kebutuhan sistem industri.

D.1 Konsep Utama

Konsep utama pada materi ini berfokus pada penggunaan React sebagai library JavaScript untuk membangun antarmuka pengguna berbasis component. Berbeda dengan HTML biasa yang cenderung statis, React memungkinkan tampilan dibuat secara modular sehingga setiap bagian antarmuka dapat dipisahkan menjadi komponen-komponen kecil

yang saling terhubung. Komponen dalam React dapat berupa card, tombol, tabel, form, maupun bagian dashboard lainnya. Setiap komponen dapat dibuat satu kali, kemudian digunakan kembali berkali-kali dengan data yang berbeda. Hal ini membuat proses pengembangan aplikasi menjadi lebih cepat, lebih rapi, dan lebih mudah dipelihara.

JSX menjadi bagian penting dalam React karena memungkinkan penulisan kode HTML di dalam JavaScript. Dengan JSX, developer dapat membuat tampilan yang lebih mudah dibaca karena struktur kodenya menyerupai HTML biasa. JSX juga mendukung penggunaan variabel, kondisi, dan perulangan sehingga tampilan dapat berubah sesuai data yang dimiliki. Props digunakan untuk mengirim data dari parent component ke child component. Dengan props, satu component dapat menerima data yang berbeda-beda, seperti nama mesin, status mesin, atau jumlah produksi. Hal ini membuat tampilan aplikasi menjadi lebih fleksibel dan dinamis.

Selain props, React juga mendukung penggunaan state untuk menyimpan data internal pada component. State memungkinkan suatu component memiliki data yang dapat berubah sesuai interaksi pengguna, misalnya tombol status mesin, jumlah produksi, atau data form input. Walaupun pada pertemuan ini fokus masih pada props, pemahaman mengenai state akan sangat penting untuk materi React berikutnya. React juga menggunakan konsep Virtual DOM, yaitu representasi virtual dari DOM asli browser. Virtual DOM membuat proses pembaruan tampilan menjadi lebih cepat karena React hanya akan memperbarui bagian yang berubah saja, bukan me-render ulang seluruh halaman. Hal ini membuat aplikasi menjadi lebih efisien dan responsif.

Selain itu, React mendukung konsep reusable component yang sangat penting pada pengembangan aplikasi skala besar. Ketika terdapat banyak tampilan yang serupa, developer cukup membuat satu component lalu menggunakannya berulang kali dengan data yang berbeda. Konsep ini membuat kode lebih efisien, mudah dirawat, dan mengurangi duplikasi. Dengan menggabungkan JSX, component, props, state, dan reusable component, React dapat digunakan untuk membangun aplikasi web modern yang lebih terstruktur, mudah dikembangkan, dan mampu menangani kebutuhan tampilan yang kompleks.

D.2 Keterkaitan dengan Teknik Industri

Materi React memiliki hubungan yang sangat erat dengan bidang Teknik Industri karena React banyak digunakan untuk membangun dashboard, sistem monitoring, dan aplikasi berbasis web yang menampilkan data secara real-time. Dalam dunia industri, data produksi, stok gudang, performa mesin, dan status operator harus dapat ditampilkan dengan cepat dan mudah dipahami.

Konsep component pada React sangat sesuai dengan kebutuhan industri karena tampilan dashboard biasanya memiliki banyak bagian yang serupa dan berulang. Misalnya, pada dashboard monitoring mesin terdapat banyak card mesin dengan format yang sama. Developer cukup membuat satu component KartuMesin, lalu menggunakannya berulang kali dengan data yang berbeda untuk setiap mesin.

Props juga memiliki peran penting dalam sistem industri karena data yang ditampilkan selalu berubah sesuai kondisi lapangan. Status mesin dapat berubah dari running menjadi

stop atau maintenance, jumlah produksi dapat bertambah setiap saat, dan data operator dapat berubah sesuai jadwal shift. Dengan props, perubahan data tersebut dapat langsung ditampilkan pada antarmuka.

Selain digunakan pada monitoring mesin, React juga banyak diterapkan pada sistem ERP, manajemen gudang, pelacakan stok barang, sistem quality control, serta dashboard laporan insiden. Seluruh data tersebut dapat ditampilkan dalam bentuk tabel, card, grafik, maupun notifikasi sehingga memudahkan pengguna dalam membaca informasi. React juga mendukung pengembangan dashboard berbasis IoT. Data dari sensor suhu, tekanan, kelembapan, getaran mesin, dan konsumsi energi dapat ditampilkan secara real-time ke dalam aplikasi. Dengan begitu, operator dapat mengetahui kondisi lapangan secara langsung tanpa harus melakukan pengecekan manual.

Kemampuan React dalam membuat reusable component sangat membantu proses pengembangan aplikasi industri yang besar dan kompleks. Jika perusahaan ingin menambahkan fitur baru, developer tidak perlu membuat ulang seluruh tampilan, tetapi cukup menambahkan atau memodifikasi component tertentu.

Secara keseluruhan, React membantu proses digitalisasi industri karena mampu menghasilkan aplikasi yang interaktif, responsif, mudah dipelihara, dan mampu menangani data dalam jumlah besar. Oleh karena itu, React menjadi salah satu teknologi yang banyak digunakan dalam pengembangan sistem industri modern.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

Dokumentasikan setiap langkah kerja yang Anda lakukan, beserta hasil yang diperoleh (screenshot, kode, atau output). Jawab sesuai urutan langkah di Manual Praktikum.

E.1 Langkah 1 – Setup Project dengan Vite

Buka Terminal (VS Code: `Ctrl + ~` atau Terminal > New Terminal). Ketik perintah berikut (tunggu prosesnya):

1. Membuat project baru bernama 'sistem-industri'
`npm create vite@latest sistem-industri -- --template react`
2. Masuk ke folder project
`cd sistem-industri`
3. Install dependensi library
`npm install`
4. Jalankan development server
`npm run dev`

Buka link yang muncul (biasanya `http://localhost:5173`).

```
/* ===== KODE / OUTPUT LANGKAH 1 ===== */
```

```
PS
```

```
C:\Users\ASUSS\Documents\PraktikumWebMob_TekInd_2026\PraktikumWebMob_TekInd_2026> node -v
```



```
v24.15.0
PS
C:\Users\ASUSS\Documents\PraktikumWebMob_TekInd_2026\PraktikumWebMob_TekInd_2
026> npm -v
11.12.1
PS
C:\Users\ASUSS\Documents\PraktikumWebMob_TekInd_2026\PraktikumWebMob_TekInd_2
026> npm create vite@latest sistem-industri -- --template react

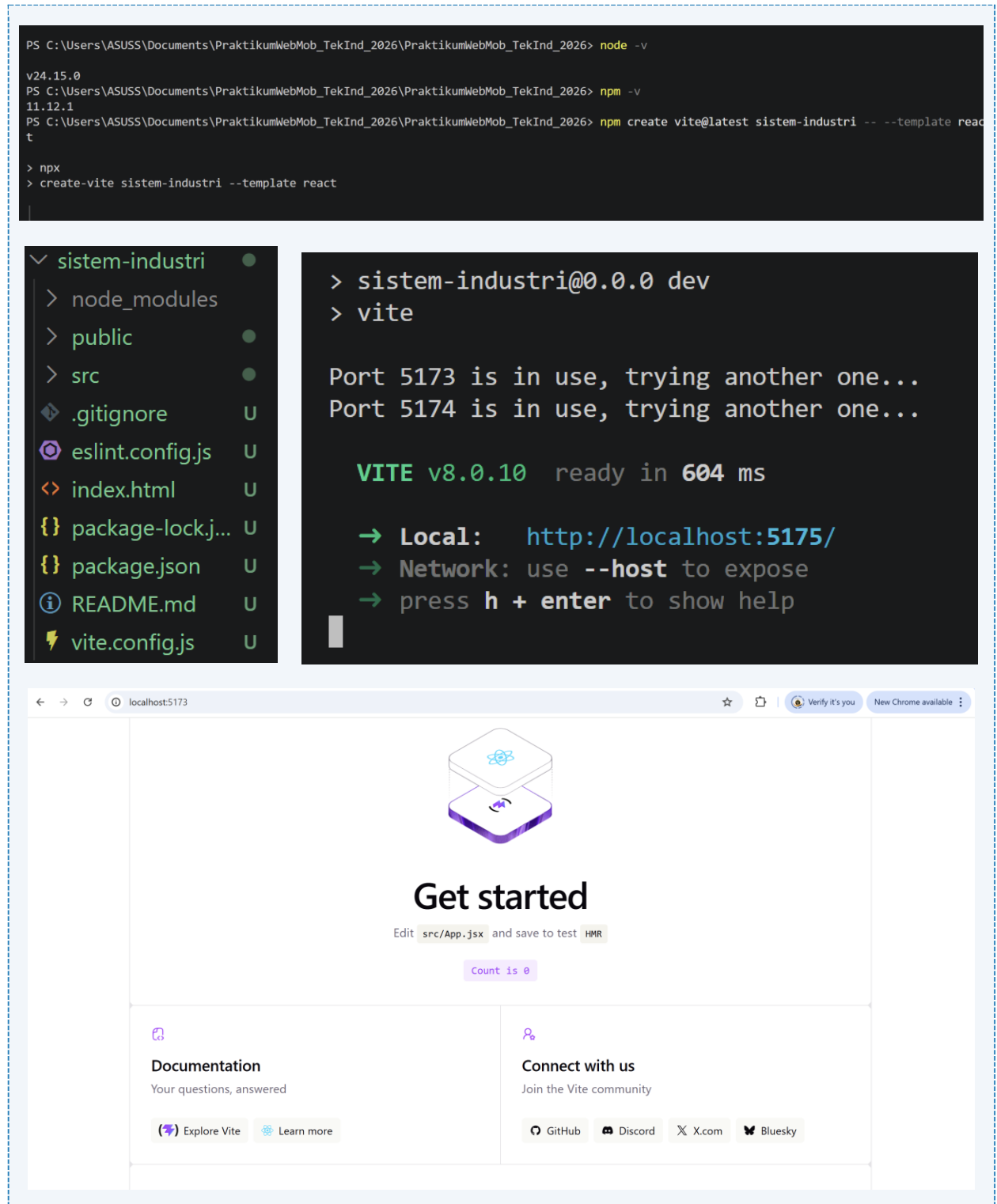
> npx
> create-vite sistem-industri --template react
PS
C:\Users\ASUSS\Documents\PraktikumWebMob_TekInd_2026\PraktikumWebMob_TekInd_2
026> cd sistem-industri
PS
C:\Users\ASUSS\Documents\PraktikumWebMob_TekInd_2026\PraktikumWebMob_TekInd_2
026> npm install
PS
PS
C:\Users\ASUSS\Documents\PraktikumWebMob_TekInd_2026\PraktikumWebMob_TekInd_2
026\sistem-industri> npm run dev

> sistem-industri@0.0.0 dev
> vite

Port 5173 is in use, trying another one...
Port 5174 is in use, trying another one...

VITE v8.0.10 ready in 604 ms

→ Local:   http://localhost:5175/
→ Network: use --host to expose
→ press h + enter to show help
```

Screenshot Hasil:

Gambar E.1 – Hasil Setup Project dengan Vite

E.2 Langkah 2 – Membersihkan Project

Buka folder project di VS Code. Masuk ke `src/App.jsx`. Hapus semua konten di dalam `return (...)` dan biarkan kosong atau ganti dengan `

#

Industri</h1>`. Hapus juga `import css` yang tidak perlu untuk sementara agar fokus pada logika.

```
/* ===== KODE / OUTPUT LANGKAH 2 ===== */

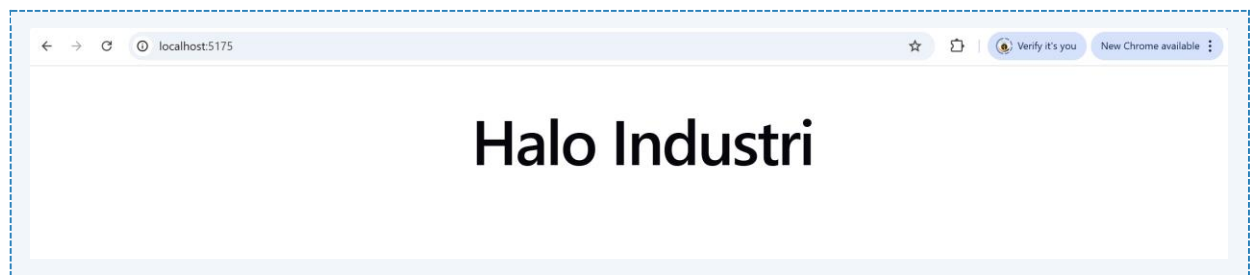
import { useState } from 'react'

function App() {
  const [count, setCount] = useState(0)

  return (
    <>
      <h1>Halo Industri</h1>
    </>
  )
}

export default App
```

Screenshot Hasil:



Gambar E.2 – Hasil Membersihkan Project

E.3 Langkah 3 – Membuat Komponen Pertama: KartuMesin

Buat file baru di folder `src` bernama `Komponen/KartuMesin.jsx`.

Struktur folder:

- src
- Komponen
- KartuMesin.jsx

Isi `KartuMesin.jsx`:

```
/* ===== KODE / OUTPUT LANGKAH 3 ===== */

// 1. Import React
import React from 'react';

// 2. Membuat Komponen Fungsi yang menerima 'props'
function KartuMesin(props) {
```

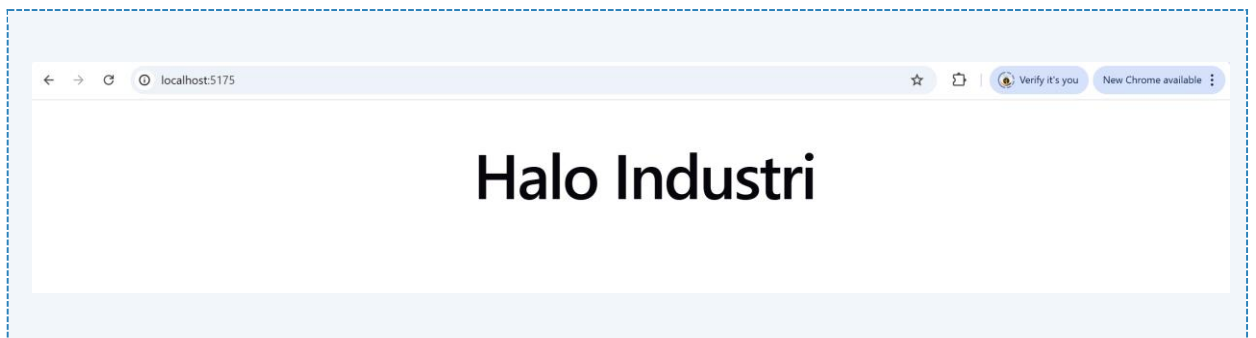
```
// Menerima data dari props
const namaMesin = props.nama;
const status = props.status;
const produksi = props.produksi;

// Logika penentuan warna badge berdasarkan status
let badgeColor = 'bg-secondary';
if (status === 'Running') badgeColor = 'bg-success';
if (status === 'Stop') badgeColor = 'bg-danger';
if (status === 'Maintenance') badgeColor = 'bg-warning';

return (
  // Menggunakan class -> className di JSX
  <div className="card shadow-sm p-3 mb-3">
    <div className="card-body">
      <h5 className="card-title">{namaMesin}</h5>
      <span className={`badge ${badgeColor}`}>{status}</span>
      <hr />
      <p>Produksi Saat Ini: <strong>{produksi}</strong> Unit</p>
    </div>
  </div>
);
}

// 3. Export agar bisa dipakai di file lain
export default KartuMesin;
```

Screenshot Hasil:



Gambar E.3 – Hasil Membuat Komponen Pertama: KartuMesin

E.4 Langkah 4 – Menggunakan Komponen di App.jsx

Sekarang kita akan memanggil `KartuMesin` di file utama dan mengirim data lewat props.

Buka dan lakukan edit `src/App.jsx`:

Catatan: Tambahkan link Bootstrap CSS di `index.html` (folder public) agar class card berfungsi.

```
/* ===== KODE / OUTPUT LANGKAH 4 ===== */
```

4. JAWABAN KODE SRC/APP.JSX

```
import React from 'react';
// Import komponen yang sudah dibuat
import KartuMesin from './Komponen/KartuMesin';

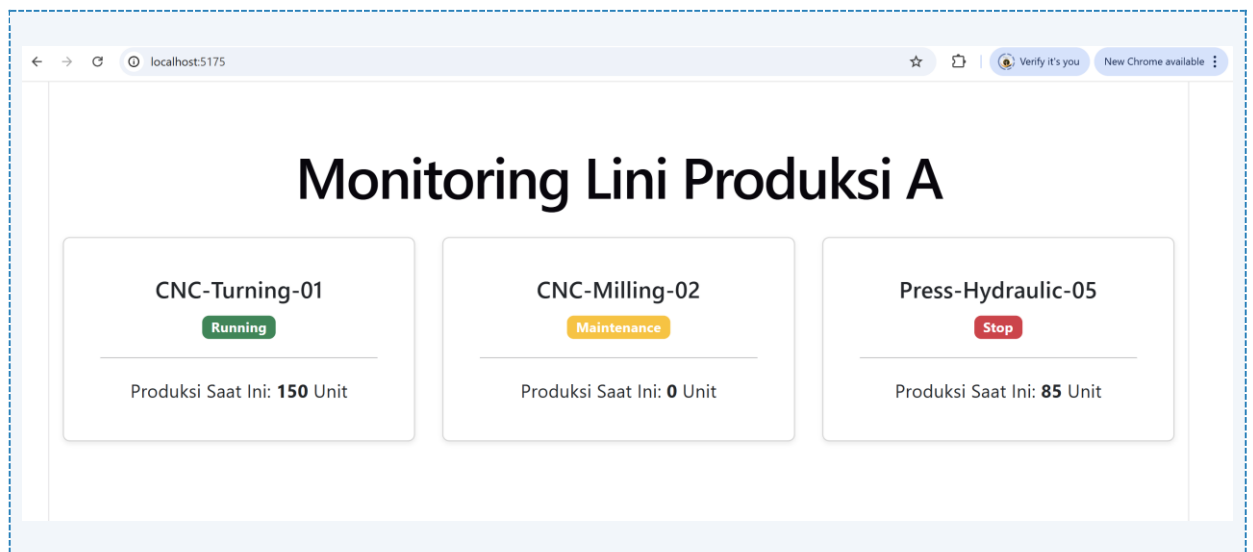
function App() {
  return (
    <div className="container mt-4">
      <h1 className="text-center mb-4">Monitoring Lini Produksi A</h1>

      <div className="row">
        {/* Kolom 1: Menggunakan komponen dengan data berbeda */}
        <div className="col-md-4">
          <KartuMesin
            nama="CNC-Turning-01"
            status="Running"
            produksi={150}
          />
        </div>
        {/* Kolom 2 */}
        <div className="col-md-4">
          <KartuMesin
            nama="CNC-Milling-02"
            status="Maintenance"
            produksi={0}
          />
        </div>
        {/* Kolom 3 */}
        <div className="col-md-4">
          <KartuMesin
            nama="Press-Hydraulic-05"
            status="Stop"
            produksi={85}
          />
        </div>
      </div>
    </div>
  );
}
export default App;
```

5. JAWABAN KODE INDEX HTML

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>sistem industri</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
  </head>
```

Screenshot Hasil:



Gambar E.4 – Hasil Menggunakan Komponen di App.jsx

F. LATIHAN DAN TUGAS MANDIRI

Jawab dan dokumentasikan seluruh latihan/tugas yang ada di bagian "Latihan/Tugas Mandiri" pada modul Anda.

F.1 Latihan 1 – Destructuring Props

Pertanyaan / Instruksi Latihan:

Ubah cara deklarasi props di `KartuMesin.jsx` menggunakan destructuring: `function KartuMesin({ nama, status, produksi }) { ... }`. Sesuaikan kode di dalamnya.

Jawaban / Kode Solusi (JS & HTML)

1. JAWABAN KODE KARTUMESIN.JSX

```
import React from 'react';
```

```
// Latihan 1: Mengambil data langsung di dalam kurung kurawal { }
// Ini namanya Destructuring, tidak pakai (props) lagi.
function KartuMesin({ nama, status, produksi }) {

  // Logika warna tetap sama
  let badgeColor = 'bg-secondary';
  if (status === 'Running') badgeColor = 'bg-success';
  if (status === 'Stop') badgeColor = 'bg-danger';
  if (status === 'Maintenance') badgeColor = 'bg-warning';

  return (
    <div className="card shadow-sm p-3 mb-3">
      <div className="card-body">
        {/* Pakai variabelnya langsung tanpa props.nama */}
        <h5 className="card-title">{nama}</h5>
        <span className={`badge ${badgeColor}`}>{status}</span>
        <hr />
        <p>Produksi Saat Ini: <strong>{produksi}</strong> Unit</p>
      </div>
    </div>
  );
}

export default KartuMesin;
```

2. JAWABAN KODE APP.JSX

```
import React from 'react';
// Import komponen yang sudah dibuat
import KartuMesin from './Komponen/KartuMesin';

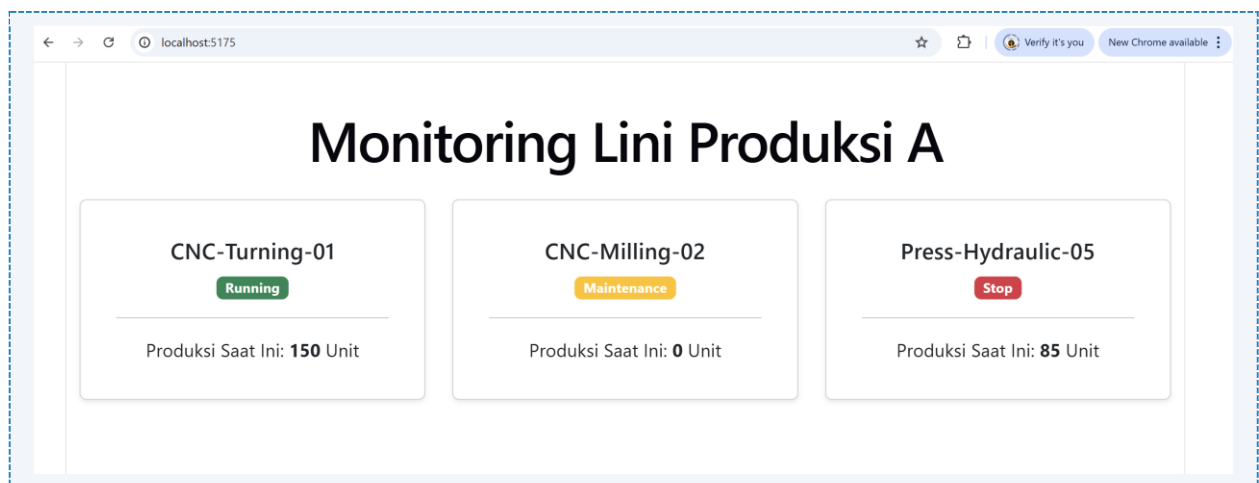
function App() {
  return (
    <div className="container mt-4">
      <h1 className="text-center mb-4">Monitoring Lini Produksi A</h1>

      <div className="row">
        {/* Kolom 1: Menggunakan komponen dengan data berbeda */}
        <div className="col-md-4">
          <KartuMesin
            nama="CNC-Turning-01"
            status="Running"
            produksi={150}
          />
        </div>
      </div>
    </div>
  );
}
```

```

    { /* Kolom 2 */ }
    <div className="col-md-4">
      <KartuMesin
        nama="CNC-Milling-02"
        status="Maintenance"
        produksi={0}
      />
    </div>
    { /* Kolom 3 */ }
    <div className="col-md-4">
      <KartuMesin
        nama="Press-Hydraulic-05"
        status="Stop"
        produksi={85}
      />
    </div>
  </div>
</div>
);
}
export default App;Q

```



Gambar F.1 – Hasil Destructuring Props

F.2 Latihan 2 – Default Props

Pertanyaan / Instruksi Latihan:

Berikan nilai default pada prop `produksi` menjadi 0, jika user lupa mengirim prop tersebut.

Jawaban / Kode Solusi:

4. JAWABAN KODE KARTUMESIN.JSX


```
import React from 'react';

// Latihan 1: Mengambil data langsung di dalam kurung kurawal { }
// Ini namanya Destructuring, tidak pakai (props) lagi.
// Latihan 2: Memberikan nilai default 0 pada produksi
function KartuMesin({ nama, status, produksi = 0 }) {

  // Logika warna tetap sama
  let badgeColor = 'bg-secondary';
  if (status === 'Running') badgeColor = 'bg-success';
  if (status === 'Stop') badgeColor = 'bg-danger';
  if (status === 'Maintenance') badgeColor = 'bg-warning';

  return (
    <div className="card shadow-sm p-3 mb-3">
      <div className="card-body">
        {/* Pakai variabelnya langsung tanpa props.nama */}
        <h5 className="card-title">{nama}</h5>
        <span className={`badge ${badgeColor}`}>{status}</span>
        <hr />
        <p>Produksi Saat Ini: <strong>{produksi}</strong> Unit</p>
      </div>
    </div>
  );
}

export default KartuMesin;
```

5. JAWABAN KODE APP.JSX

```
import React from 'react';
// Import komponen yang sudah dibuat
import KartuMesin from './Komponen/KartuMesin';

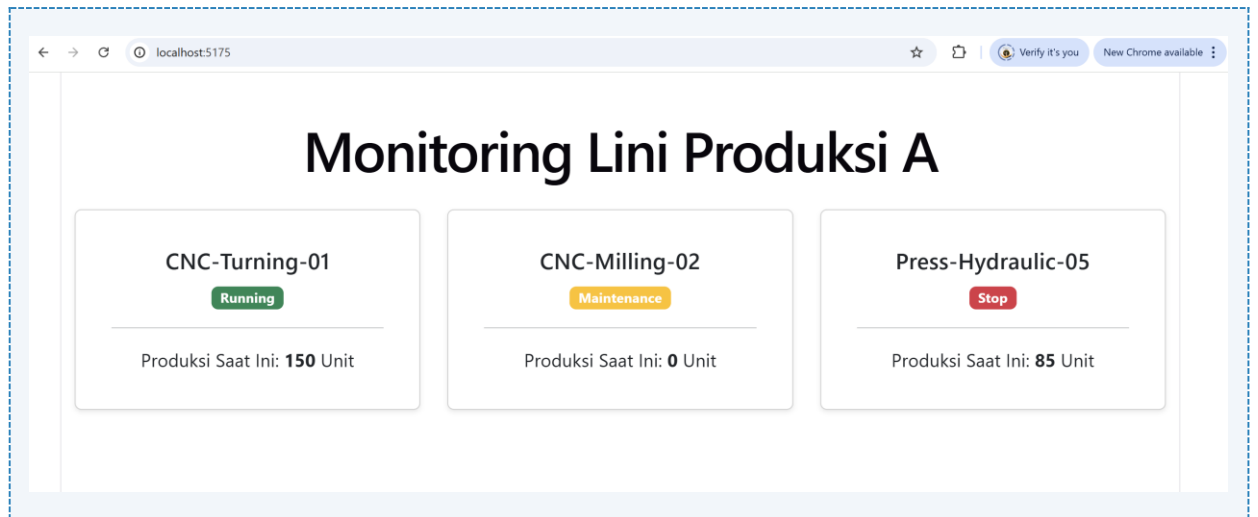
function App() {
  return (
    <div className="container mt-4">
      <h1 className="text-center mb-4">Monitoring Lini Produksi A</h1>

      <div className="row">
        {/* Kolom 1: Menggunakan komponen dengan data berbeda */}
        <div className="col-md-4">
          <KartuMesin
            nama="CNC-Turning-01"
            status="Running"
            produksi={150}
          />
        </div>
      </div>
    </div>
  );
}
```

```

    />
  </div>
  { /* Kolom 2 */ }
  <div className="col-md-4">
    <KartuMesin
      nama="CNC-Milling-02"
      status="Maintenance"
      /* produksi={0} nya dihapus untuk Latihan 2 */
    />
  </div>
  { /* Kolom 3 */ }
  <div className="col-md-4">
    <KartuMesin
      nama="Press-Hydraulic-05"
      status="Stop"
      produksi={85}
    />
  </div>
</div>
</div>
</div>
);
}
export default App;

```



Gambar F.2 – Hasil Default Props

F.3 Tugas Proyek Mini – Kartu Karyawan

Deskripsi proyek mini:

- Buat komponen baru bernama `KartuKaryawan.jsx`. Komponen ini menerima props: `nama`, `jabatan`, `bagian`.

- b. Tampilkan komponen ini 3 kali di `App.jsx` dengan data karyawan berbeda (Manager, Operator, QC).

Penjelasan pendekatan/solusi yang digunakan:

- Proyek mini ini dilakukan melalui beberapa tahapan yang sistematis, dimulai dari perancangan tampilan hingga implementasi
14. Tahap pertama dimulai dengan pembuatan file komponen baru bernama `KartuKaryawan.jsx` yang dipisahkan dari komponen utama untuk menjaga keteraturan struktur kode sesuai dengan prinsip modularitas pada library React.
 15. Selanjutnya dilakukan perancangan logika komponen yang diprogram untuk mampu menerima data dari luar melalui *props* sehingga satu komponen yang sama dapat menampilkan informasi yang berbeda secara fleksibel.
 16. Pada tahap berikutnya diterapkan teknik *Destructuring* pada parameter fungsi komponen guna memanggil properti nama, jabatan, dan bagian secara langsung agar penulisan kode menjadi lebih bersih dan meminimalisir penggunaan sintaksis yang berulang.
 17. Setelah itu dikembangkan struktur tampilan menggunakan JSX yang dipadukan dengan kelas-kelas Bootstrap untuk menghasilkan komponen kartu yang memiliki desain modern serta responsif terhadap berbagai ukuran layar.
 18. Langkah selanjutnya adalah melakukan kustomisasi pada bagian teks nama karyawan dengan memberikan atribut warna biru muda melalui kelas `text-info` guna menciptakan kontras visual yang memudahkan pengguna dalam membedakan data personil dengan data mesin.
 19. Tahap berikutnya adalah mengintegrasikan komponen tersebut ke dalam file `App.jsx` melalui proses *import* agar seluruh fungsi dan elemen kartu karyawan dapat dirender ke dalam halaman dashboard monitoring.
 20. Kemudian dilakukan pemanggilan komponen sebanyak tiga kali di dalam tata letak baris yang sudah disediakan di mana setiap komponen dikirimkan data yang berbeda untuk merepresentasikan peran Manager, Operator, dan Quality Control secara spesifik.
 21. Terakhir dilakukan pengujian pada *local server* untuk memvalidasi bahwa seluruh *props* yang dikirimkan dari komponen induk telah tampil dengan benar serta memastikan identitas mahasiswa atas nama Maessa Andrea Vallenia muncul sebagai bukti pengerjaan praktikum.

Jawaban / Kode Solusi:

3. JAWABAN KODE KARTUKARYAWAN.JSX

```
import React from 'react';

function KartuKaryawan({ nama, jabatan, bagian }) {
  return (
    <div className="card shadow-sm p-3 mb-3">
      <div className="card-body">
        <h6 className="text-muted mb-1">Karyawan</h6>
      </div>
    </div>
  );
}
```

```

        { /* Hanya nama yang diberi warna biru muda (text-info) */ }
        <h5 className="card-title text-info">{nama}</h5>
        <hr />
        <p className="mb-1">Jabatan: <strong>{jabatan}</strong></p>
        <p className="mb-0">Bagian: <strong>{bagian}</strong></p>
      </div>
    </div>
  );
}

export default KartuKaryawan;

```

4. JAWABAN KODE APP.JSX

```

import React from 'react';
import KartuMesin from './Komponen/KartuMesin';
import KartuKaryawan from './Komponen/KartuKaryawan';

function App() {
  return (
    <div className="container mt-4">
      { /* Judul Utama */ }
      <h1 className="text-center mb-0">Monitoring Lini Produksi A</h1>

      { /* Nama Mahasiswa sebagai identitas */ }
      <p className="text-center text-muted mb-4">
        <strong>Maessa Andrea Vallenia</strong> |
        <strong>23051430044</strong>
      </p>

      { /* BARIS KARTU MESIN */ }
      <div className="row mb-4">
        <div className="col-md-4">
          <KartuMesin nama="CNC-Turning-01" status="Running" produksi={150} />
        </div>
        <div className="col-md-4">
          <KartuMesin nama="CNC-Milling-02" status="Maintenance" />
        </div>
        <div className="col-md-4">
          <KartuMesin nama="Press-Hydraulic-05" status="Stop" produksi={85} />
        </div>
      </div>

      <hr className="my-5" />
    </div>
  );
}

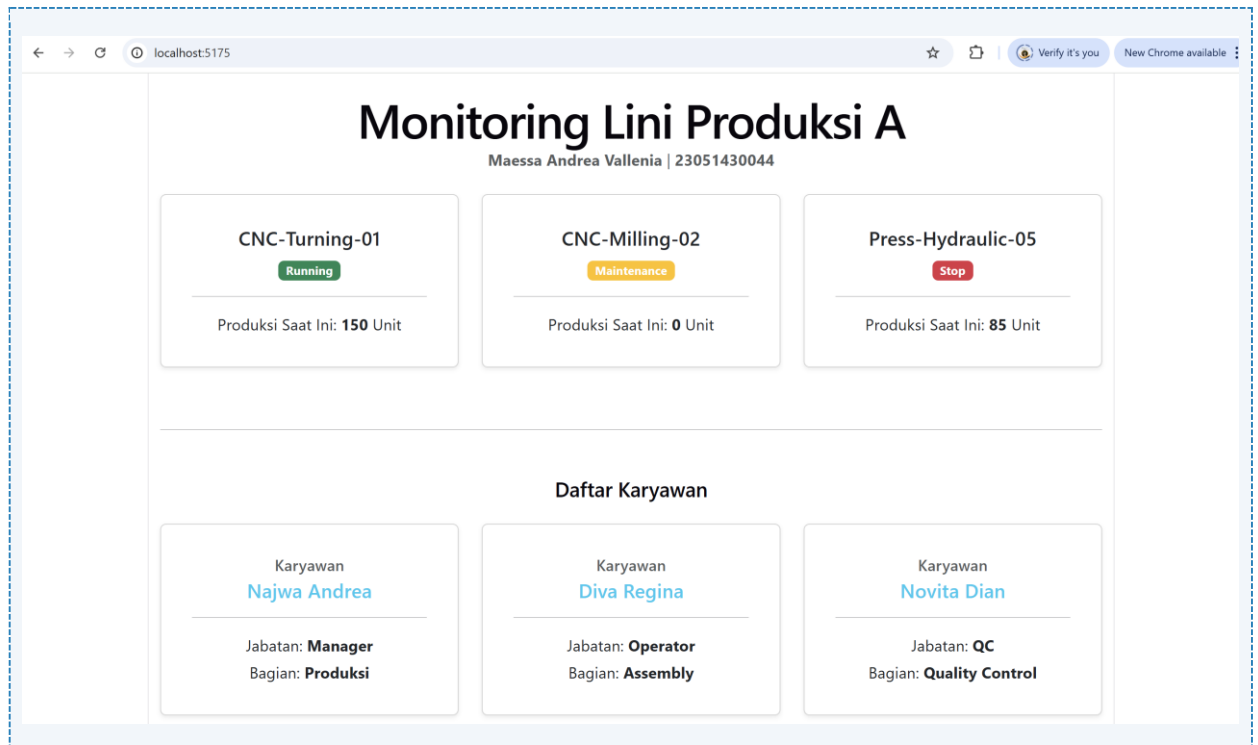
```

```

{ /* BARIS DAFTAR KARYAWAN */ }
<h2 className="text-center mb-4">Daftar Karyawan</h2>
<div className="row">
  <div className="col-md-4">
    <KartuKaryawan nama="Najwa Andrea" jabatan="Manager"
bagian="Produksi" />
  </div>
  <div className="col-md-4">
    <KartuKaryawan nama="Diva Regina" jabatan="Operator"
bagian="Assembly" />
  </div>
  <div className="col-md-4">
    <KartuKaryawan nama="Novita Dian" jabatan="QC" bagian="Quality
Control" />
  </div>
</div>
);
}

export default App;

```



Gambar F.3 – Hasil Proyek Mini Integrasi Data Produksi

G. PEMBAHASAN

Pada praktikum Pertemuan 9, mahasiswa mempelajari konsep dasar React JS, yang meliputi pembuatan komponen (*Component*), pengiriman data melalui *Props*, teknik *Destructuring*, dan penerapan *Default Props*. Praktikum ini menunjukkan bahwa pengembangan aplikasi web modern di industri saat ini telah beralih dari manipulasi DOM manual ke pendekatan berbasis komponen yang lebih efisien, terstruktur, dan *reusable*.

App.jsx digunakan sebagai komponen induk (*Parent*) untuk mengatur tata letak utama dashboard monitoring. Bootstrap diintegrasikan untuk memastikan tampilan dashboard monitoring produksi tetap rapi, profesional, dan responsif.

Komponen React digunakan untuk memecah antarmuka menjadi bagian-bagian kecil yang mandiri. Hal ini memungkinkan kode menjadi lebih mudah dikelola (*maintainable*). Penggunaan *Props* memungkinkan data mengalir secara satu arah dari *Parent Component* ke *Child Component*, sehingga informasi mesin dan karyawan dapat ditampilkan secara dinamis.

Destructuring Props diterapkan untuk menyederhanakan penulisan kode, di mana variabel diambil langsung di dalam parameter fungsi komponen. Selain itu, penggunaan *Default Props* memastikan aplikasi memiliki "jaring pengaman" dengan memberikan nilai cadangan jika terdapat data yang lupa dikirimkan, sehingga mencegah terjadinya tampilan kosong (*empty state*) yang dapat membingungkan pengguna.

Pada Latihan 1, mahasiswa berhasil melakukan refaktorisasi kode menggunakan *Destructuring*. Pada Latihan 2, mahasiswa mengimplementasikan *Default Props* pada variabel produksi. Pada proyek mini, mahasiswa membangun komponen baru bernama *KartuKaryawan* untuk menampilkan data SDM operasional yang terintegrasi dalam satu dashboard produksi.

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum secara menyeluruh menunjukkan bahwa implementasi program yang telah dilakukan sudah sangat sesuai dengan teori React Component-Based Architecture, penggunaan *Props*, serta kaidah Modern JavaScript ES6. Keberhasilan ini dapat dianalisis melalui beberapa poin mendalam sebagai berikut

Struktur komponen telah dipisahkan dengan benar ke dalam folder *Komponen* yang menunjukkan pemahaman mendalam mengenai konsep modularitas dalam pengembangan perangkat lunak. Sesuai dengan teori React, elemen antarmuka tidak lagi dimanipulasi secara manual menggunakan metode konvensional seperti `getElementById()` melainkan diperbarui secara deklaratif melalui aliran data *props*. Pendekatan ini membuktikan bahwa mahasiswa telah memahami cara kerja *Virtual DOM* di mana perubahan tampilan dikendalikan sepenuhnya oleh perubahan data yang mengalir dari komponen induk ke komponen anak.

Konsep *Destructuring* berhasil diterapkan secara optimal pada fungsi *KartuMesin* dan *KartuKaryawan*. Hal ini selaras dengan teori pemrograman JavaScript modern yang bertujuan untuk meningkatkan keterbacaan kode atau *code readability* serta menciptakan efisiensi dalam penulisan variabel. Dengan mengekstraksi properti secara langsung di dalam parameter fungsi, mahasiswa dapat menghindari penulisan objek *props* yang berulang

sehingga meminimalisir potensi kesalahan penulisan dan mempercepat proses pengembangan aplikasi.

Penggunaan Default Props pada variabel produksi dengan nilai nol juga telah memenuhi standar teori pengembangan perangkat lunak yang baik. Dalam lingkungan industri yang dinamis, sistem harus memiliki mekanisme pertahanan untuk menangani ketiadaan data atau kondisi *undefined*. Implementasi nilai *default* ini memastikan bahwa antarmuka pengguna tetap informatif dan tidak terlihat rusak bagi operator meskipun terdapat data yang belum terisi dari sisi komponen induk. Hal ini mencerminkan prinsip *Robustness* dalam rekayasa perangkat lunak.

Penerapan Conditional Rendering pada bagian *badge* status operasional menunjukkan bahwa logika pemrograman dapat disematkan secara efisien langsung di dalam sintaksis JSX. Perubahan warna *badge* yang bersifat dinamis sesuai status mesin merupakan bentuk nyata dari integrasi antara logika bisnis dengan presentasi visual. Mahasiswa berhasil membuktikan bahwa status teknis seperti Running, Stop, dan Maintenance dapat dipetakan ke dalam identitas visual yang mudah dipahami oleh pengguna melalui pemanfaatan properti kelas Bootstrap yang dikondisikan.

Secara keseluruhan sistem monitoring yang telah dibangun telah mampu mencapai target kompetensi sebagai berikut

1. Berhasil membagi antarmuka dashboard yang kompleks menjadi komponen-komponen modular yang lebih kecil dan mandiri seperti KartuMesin dan KartuKaryawan.
2. Mampu mengelola pengiriman data dinamis dari file utama App.jsx menuju ke seluruh komponen anak melalui mekanisme *Props* secara akurat.
3. Mengimplementasikan standar koding modern melalui teknik *Destructuring* sehingga struktur kode menjadi jauh lebih bersih dan profesional.
4. Menerapkan sistem penanganan *error* ringan untuk data kosong melalui fitur *Default Props* guna menjaga kualitas pengalaman pengguna.
5. Menghasilkan visualisasi dashboard yang responsif dan mampu merepresentasikan status operasional mesin secara dinamis berdasarkan data yang diterima.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Data produksi muncul kosong/hanya tulisan "Unit" saat data tidak dikirim.	Mengimplementasikan <i>Default Props</i> dengan menambahkan = 0 pada parameter fungsi komponen agar nilai cadangan otomatis muncul.

G.3 Kaitan dengan Penerapan di Industri

Materi yang dipelajari pada praktikum React JS ini memiliki relevansi yang sangat signifikan terhadap implementasi teknologi di dunia industri modern terutama pada era Industri 4.0 dan transformasi digital. Penggunaan framework berbasis komponen seperti React merupakan standar industri dalam membangun sistem pemantauan yang kompleks seperti *Manufacturing Execution System* atau MES serta dashboard *Enterprise Resource Planning* atau ERP. Dalam lingkup pabrik yang cerdas data sering kali datang dari ribuan

sensor mesin yang berbeda dan harus disajikan secara *real-time* kepada pihak manajemen. Melalui pendekatan komponen modular yang telah dipraktikkan mahasiswa dapat memahami bagaimana membangun antarmuka yang skalabel di mana satu struktur kode dapat digunakan kembali untuk memantau ratusan aset produksi hanya dengan mengganti parameter *props* yang dikirimkan.

Penerapan konsep *Props* dan *Default Props* sangat krusial dalam menjaga stabilitas sistem informasi di industri manufaktur. Sebagai contoh apabila sebuah sistem monitoring gagal menerima data dari sensor aliran produksi di lantai pabrik akibat gangguan jaringan maka penggunaan nilai *default* memastikan bahwa dashboard tidak mengalami kegagalan total atau *system crash*. Dashboard akan tetap menampilkan angka dasar yang informatif sehingga operator tetap dapat memantau parameter lain tanpa terganggu oleh adanya satu titik kegagalan data. Hal ini selaras dengan prinsip *High Availability* yang sangat dijaga dalam operasional industri untuk meminimalisir waktu henti mesin atau *downtime*.

Selain itu fitur filter data dan pewarnaan dinamis yang diterapkan pada status mesin mencerminkan kebutuhan nyata bagi supervisor produksi dalam melakukan pengambilan keputusan secara cepat. Dengan adanya indikator visual yang berubah otomatis berdasarkan kondisi mesin pihak teknisi dapat segera mengidentifikasi unit mana yang memerlukan tindakan perawatan segera atau *maintenance*. Integrasi data karyawan ke dalam dashboard monitoring produksi juga menyerupai sistem manajemen sumber daya manusia di industri di mana kinerja mesin dan kehadiran operator yang bertugas dipantau dalam satu kesatuan ekosistem digital. Hal ini memungkinkan terjadinya pelacakan akuntabilitas dan efisiensi kerja yang lebih akurat.

Secara strategis pemahaman mengenai arsitektur React memberikan bekal yang kuat bagi mahasiswa Teknik Industri untuk berperan sebagai jembatan antara kebutuhan operasional di lapangan dengan pengembangan solusi teknologi informasi. Dengan menguasai cara kerja aliran data dan komponen digital mahasiswa memiliki kompetensi untuk merancang sistem kontrol yang lebih responsif, terintegrasi, dan mampu beradaptasi dengan perubahan kebutuhan industri yang sangat dinamis. Kemampuan untuk membangun dashboard mandiri yang efisien dan bersih secara koding akan meningkatkan nilai tawar mahasiswa dalam menghadapi persaingan profesional di perusahaan manufaktur global maupun perusahaan rintisan teknologi.

H. KESIMPULAN

Tuliskan 3–5 poin kesimpulan yang merangkum hal-hal yang dipelajari dan dicapai pada praktikum modul ini. Kesimpulan harus spesifik, bukan bersifat umum.

- | | |
|---|--|
| 1 | Mahasiswa mampu memahami dan mengimplementasikan arsitektur berbasis komponen pada React JS untuk membangun antarmuka dashboard monitoring yang bersifat modular dan mudah dalam proses pemeliharaan kode. |
| 2 | Mahasiswa berhasil menguasai teknik pengiriman data antar komponen menggunakan mekanisme <i>Props</i> sehingga informasi operasional mesin dan data karyawan dapat ditampilkan secara dinamis pada satu halaman dashboard yang terintegrasi. |

3	Mahasiswa mampu menerapkan teknik <i>Destructuring</i> pada parameter fungsi komponen guna menyederhanakan sintaksis JavaScript modern serta meningkatkan efisiensi penulisan kode yang lebih bersih dan profesional.
4	Mahasiswa berhasil mengimplementasikan fitur <i>Default Props</i> sebagai mekanisme penanganan data kosong yang berfungsi untuk menjaga stabilitas visual antarmuka pengguna apabila terdapat parameter data yang tidak terdefinisi dari komponen induk.
5	Mahasiswa memahami cara mengintegrasikan logika bisnis ke dalam elemen visual melalui pengkondisian warna status operasional pada <i>badge</i> mesin yang mencerminkan penerapan sistem kontrol serta monitoring <i>real-time</i> di dunia industri digital.

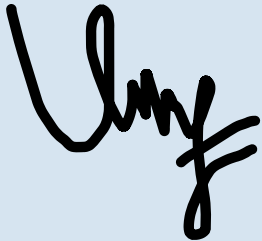
I. DAFTAR PUSTAKA

Tuliskan semua sumber referensi yang digunakan dalam format APA 7th Edition. Minimal 3 referensi yang relevan (termasuk Manual Praktikum dan sumber online resmi seperti MDN, Reactjs.org, dll).

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Kom, I. L. M. (2023). <i>Dasar Algoritma dan Pemrograman Javascript</i> . Irvan Lewenusa, M. Kom.
[3]	Sholikhan, M. (2022). Html, Css Dan Javascript. <i>Penerbit Yayasan Prima Agus Teknik</i> , 1-343 .
[4]	Rabcan, J., Gumarova, A., Vakhitova, A., & Kamalova, G. (2023). Modern Frameworks for Web-Application Development. <i>Наука и образование</i> , 3(3 (72)), 33-40.
[5]	Torres, J. A. G., Lau, S. H., Anchuri, P., Stevens, J. M., Tabora, J. E., Li, J., ... & Doyle, A. G. (2022). A multi-objective active learning platform and web app for reaction optimization. <i>Journal of the American Chemical Society</i> , 144(43), 19999-20007.
[6]	https://www.javascripttutorial.net/
[7]	https://developer.mozilla.org/en-US/docs/Web/JavaScript
[8]	https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Using_promises
[9]	https://developer.mozilla.org/en-US/docs/Glossary/JSX
[10]	https://vitejs.dev/guide/
[11]	https://getbootstrap.com/docs/5.3/getting-started/introduction/
[12]	https://react.dev/

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Diperiksa oleh Asisten / Dosen
 _____ Maessa Andrea Vallenia NIM: 23051430044	 _____ Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____

UNIVERSITAS NEGERI YOGYAKARTA
FAKULTAS TEKNIK
PROGRAM STUDI TEKNIK INDUSTRI

LAPORAN PRAKTIKUM
APLIKASI WEB DAN MOBILE

Semester Genap 2025/2026

**MODUL 10 – STATE, HOOKS (USESTATE, USEEFFECT), DAN
CONDITIONAL RENDERING**

Nama Mahasiswa : Maessa Andrea Vallenia
NIM : 23051430044
Kelas : A
Tanggal Praktikum : 26 April 2026
Dosen Pengampu : Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT.

Dikumpulkan paling lambat 7 hari setelah pelaksanaan praktikum

MODUL 10

STATE, HOOKS (USESTATE, USEEFFECT), DAN CONDITIONAL RENDERING

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 10 (Pertemuan 10)
Topik Utama	State Management, Lifecycle Hooks, dan Rendering Bersyarat.
Sub-Topik	State, Hooks (useState, useEffect), Conditional Rendering, Form Input State, Event Handling, Dependency Array, dan Real-time Update pada React.
Durasi Praktikum	340 Menit
Tanggal Praktikum	26 April 2026
Nama Mahasiswa	Maessa Andrea Vallenia
NIM	23051430044
Kelas	A

A. TUJUAN PEMBELAJARAN

Tuliskan tujuan pembelajaran yang ingin dicapai pada modul ini, sesuai dengan yang tertera di Manual Praktikum.

1	Memahami apa itu State dan perbedaannya dengan Props.
2	Menguasai penggunaan `useState` untuk menyimpan data yang berubah-ubah.
3	Memahami `useEffect` untuk menangani side effects (seperti timer atau API call).
4	Mampu melakukan Conditional Rendering (menyembunyikan/menampilkan elemen berdasarkan kondisi).

B. ALAT DAN BAHAN

Sebutkan seluruh perangkat keras, perangkat lunak, dan sumber referensi yang digunakan selama praktikum.

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
3	Live Server	Versi 5.7.9	Menjalankan HTML dan memantau perubahan file
4	JavaScript (Built-in Browser)	ES6 (Default Browser Modern)	Bahasa pemrograman untuk logika dan perhitungan
5	CodeSnap	Versi 1.3.4	Dokumentasi kode program
6	Git dan GitHub	Versi 2.53.0	Version control system (VCS) berbasis Git
7	Koneksi Internet	Stabil	Mengakses Library dan Framework
8	Console Browser (Developer Tools)	Default Browser	Melihat output program dan debugging
9	Bootstrap	Versi 5.3	Membantu pembuatan UI seperti button, card, dan alert pada panel control
10	File praktikum	Pertemuan sebelumnya	Digunakan sebagai dasar pengembangan panel kontrol
11	LocalStorage (Web Storage API)	Built-in Browser	Digunakan untuk menyimpan data produksi secara persisten
12	API JSONPlaceholder	Built-in Browser	Digunakan sebagai

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
			simulasi database eksternal (data karyawan dan laporan)
13	Fetch API	Built-in Browser	Digunakan untuk melakukan HTTP request (GET, POST, dll) ke server

c. DASAR TEORI

1. State merupakan data internal pada komponen React yang nilainya dapat berubah selama aplikasi berjalan. State berbeda dengan props karena props hanya digunakan untuk menerima data dari komponen lain dan bersifat read-only, sedangkan state dapat diubah oleh komponen itu sendiri. Ketika state berubah, React akan otomatis melakukan re-render agar tampilan sesuai dengan data terbaru.
2. Hook adalah fitur pada React yang memungkinkan functional component menggunakan fitur React seperti state dan lifecycle. Sebelum adanya hook, fitur seperti state dan lifecycle hanya dapat digunakan pada class component. Dengan hook, penulisan kode menjadi lebih sederhana, ringkas, dan mudah dipahami.
3. useState merupakan hook yang digunakan untuk membuat dan mengelola state pada komponen React. useState mengembalikan dua nilai, yaitu state saat ini dan fungsi untuk mengubah state tersebut. Contohnya adalah `const [jumlah, setJumlah] = useState(0)` yang berarti state jumlah memiliki nilai awal 0.
4. useEffect merupakan hook yang digunakan untuk menangani side effect, seperti menjalankan timer, mengambil data dari API, atau mengubah judul halaman browser. useEffect dapat dijalankan saat komponen pertama kali ditampilkan, saat state berubah, atau saat komponen dihapus. Penggunaan dependency array menentukan kapan useEffect akan dijalankan.
5. Conditional Rendering adalah teknik menampilkan elemen tertentu berdasarkan kondisi tertentu. Dalam React, conditional rendering biasanya menggunakan if, ternary operator, atau operator logika &&. Teknik ini

berguna untuk menampilkan pesan, tombol, atau status tertentu sesuai kondisi aplikasi.

6. Event Handling pada React digunakan untuk menangani aksi pengguna seperti klik tombol, input data, atau perubahan pilihan. Event pada React ditulis menggunakan camelCase seperti `onClick`, `onChange`, dan `onSubmit`. Event handling sangat penting agar aplikasi menjadi interaktif.
7. Form input pada React biasanya dikontrol menggunakan state. Setiap perubahan nilai pada input akan langsung memperbarui state, sehingga data yang ditampilkan di layar selalu sinkron dengan data yang dimasukkan pengguna.

Secara keseluruhan, state, hooks, dan conditional rendering membuat aplikasi React menjadi lebih dinamis, interaktif, dan mampu menyesuaikan tampilan berdasarkan perubahan data secara real-time.

C.1 Konsep Utama

Konsep utama pada pertemuan ini adalah penggunaan state untuk menyimpan data yang dapat berubah selama aplikasi berjalan. State berfungsi sebagai memori sementara pada komponen React. Ketika nilai state berubah, React akan otomatis memperbarui tampilan sehingga pengguna dapat langsung melihat perubahan tersebut.

Penggunaan `useState` menjadi inti utama dalam praktikum karena hook ini digunakan untuk menyimpan nilai seperti jumlah produksi, target produksi, status mesin, dan input pengguna. Dengan `useState`, aplikasi dapat menjadi lebih interaktif karena data yang berubah langsung tercermin pada tampilan.

Selain `useState`, praktikum ini juga menggunakan `useEffect` untuk menangani proses yang berjalan secara otomatis, seperti timer pada jam digital. `useEffect` memungkinkan program menjalankan fungsi tertentu ketika komponen pertama kali ditampilkan, ketika state berubah, atau ketika komponen dihapus.

Konsep dependency array pada `useEffect` juga menjadi bagian penting. Jika dependency array kosong `[]`, maka `useEffect` hanya berjalan sekali saat komponen pertama kali ditampilkan. Jika dependency array berisi state tertentu, maka `useEffect` akan dijalankan kembali setiap kali state tersebut berubah.

Conditional rendering digunakan untuk menampilkan pesan atau tampilan yang berbeda sesuai kondisi tertentu. Contohnya, ketika jumlah produksi sudah mencapai target, sistem akan menampilkan pesan "Target Tercapai". Sebaliknya, jika target belum tercapai, sistem akan menampilkan pesan "Produksi Berjalan".

Konsep event handling juga diterapkan melalui tombol dan input form. Tombol tambah produksi, reset shift, dan emergency stop semuanya memanfaatkan event handler untuk menjalankan fungsi tertentu saat pengguna melakukan aksi.

Dengan menggabungkan state, hooks, event handling, dan conditional rendering, aplikasi React dapat menjadi sistem yang dinamis dan mampu merespon perubahan data secara real-time.

C.2 Keterkaitan dengan Teknik Industri

Materi state dan hooks memiliki hubungan yang erat dengan bidang Teknik Industri karena sistem industri modern memerlukan aplikasi yang mampu menampilkan data secara dinamis dan real-time. Dalam lingkungan industri, data produksi, kondisi mesin, jumlah output, dan status operator dapat berubah setiap saat.

Penggunaan state pada React sangat relevan untuk menampilkan data yang berubah-ubah, seperti jumlah produksi harian, target produksi, tingkat efisiensi mesin, dan jumlah barang cacat. Ketika data berubah, tampilan dashboard dapat langsung diperbarui tanpa perlu me-refresh halaman.

useEffect juga memiliki peran penting dalam sistem industri karena banyak proses memerlukan pembaruan otomatis. Contohnya adalah jam digital pada dashboard produksi, pembacaan sensor suhu setiap beberapa detik, atau pengambilan data mesin secara berkala dari server.

Conditional rendering sangat bermanfaat dalam sistem monitoring industri. Sebagai contoh, sistem dapat menampilkan pesan hijau ketika target produksi tercapai, pesan kuning ketika produksi masih berjalan, atau pesan merah ketika terjadi emergency stop. Hal ini mempermudah operator dalam memahami kondisi lapangan.

Konsep form input state juga dapat diterapkan pada sistem ERP dan quality control. Operator dapat memasukkan data jumlah produksi, waktu operasi mesin, jumlah produk cacat, dan berbagai data lainnya. Seluruh data tersebut kemudian diproses secara otomatis untuk menghasilkan informasi yang dibutuhkan.

Pada tugas proyek mini OEE, mahasiswa belajar menghitung Availability, Performance, dan Quality secara real-time. Perhitungan OEE sangat penting dalam Teknik Industri karena digunakan untuk mengukur efektivitas mesin dan proses produksi.

Dengan memahami state, hooks, dan conditional rendering, mahasiswa Teknik Industri dapat mengembangkan dashboard monitoring, sistem produksi, dan aplikasi industri yang lebih interaktif, responsif, dan mampu membantu pengambilan keputusan berbasis data.

D. LANGKAH KERJA DAN HASIL PRAKTIKUM

Dokumentasikan setiap langkah kerja yang Anda lakukan, beserta hasil yang diperoleh (screenshot, kode, atau output). Jawab sesuai urutan langkah di Manual Praktikum.

D.1 Langkah 1 – Membuat Komponen Counter Produksi

Buat file baru `src/Komponen/CounterProduksi.jsx`.

Panggil komponen ini di `App.jsx`.

```
/* ===== KODE / OUTPUT LANGKAH 1 ===== */
```

1. JAWABAN KODE COUNTERPRODUKSI.JSX

```
import React, { useState } from 'react';  
function CounterProduksi() {
```



```

    // Deklarasi State: 'jumlah' bernilai awal 0, 'setJumlah' fungsi untuk
mengubahnya
    const [jumlah, setJumlah] = useState(0);
    const [target, setTarget] = useState(100);

    // Fungsi menambah produksi
    const tambahProduksi = () => {
        setJumlah(jumlah + 1);
    };

    // Fungsi reset
    const reset = () => {
        setJumlah(0);
    };

    return (
        <div className="text-center p-4 border rounded bg-light">
            <h3>Simulasi Hitung Produk</h3>
            <h1 className="display-4">{jumlah}</h1>
            <p>Target: {target} Unit</p>

            /* Conditional Rendering: Tampilkan pesan jika target tercapai
*/
            {jumlah >= target ? (
                <div className="alert alert-success d-inline-block">Target
Tercapai!</div>
            ) : (
                <div className="alert alert-secondary d-inline-
block">Produksi Berjalan...</div>
            )}

            <div className="mt-3">
                <button className="btn btn-primary me-2"
onClick={tambahProduksi}>
                    +1 Unit (Sensor OK)
                </button>
                <button className="btn btn-danger" onClick={reset}>
                    Reset Shift
                </button>
            </div>
        </div>
    );
}
export default CounterProduksi;

```

2. JAWABAN KODE APP.JSX

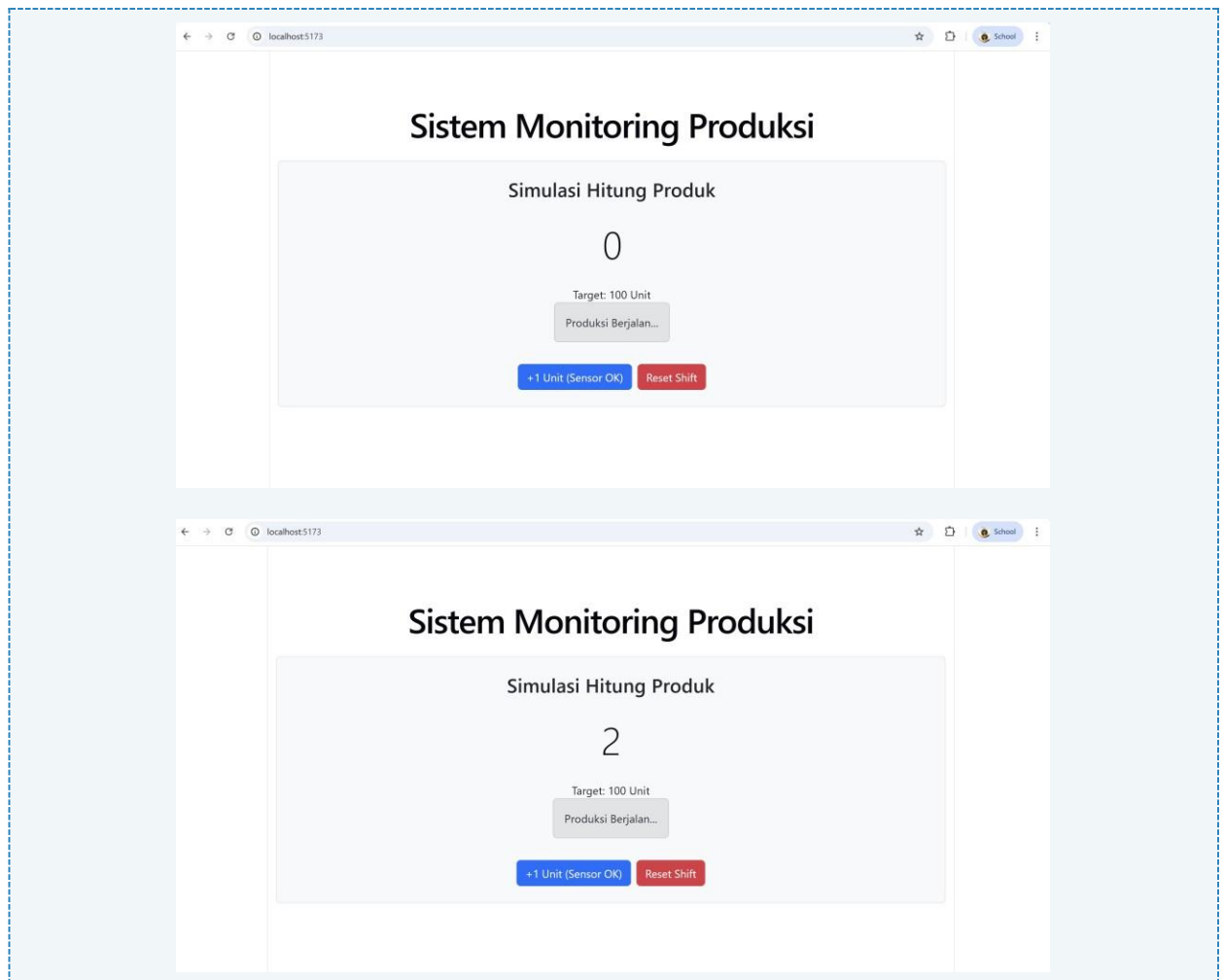
```
// 1. IMPORT: Beritahu React di mana letak file CounterProduksi
import CounterProduksi from "./Komponen/CounterProduksi";

function App() {
  return (
    <div className="container mt-5">
      <h1 className="text-center mb-4">Sistem Monitoring Produksi</h1>

      {/* 2. PANGGIL: Tuliskan nama komponen seperti tag HTML */}
      <CounterProduksi/>

    </div>
  );
}

export default App;
```

Screenshot Hasil:

Gambar E.1 – Hasil Membuat Komponen Counter Produksi

D.2 Langkah 2 – Menggunakan useEffect untuk Real-time Clock

Kita akan membuat jam digital yang berjalan otomatis. Ini mensimulasikan pengambilan data live dari sensor tiap detik. Tambahkan kode di dalam `CounterProduksi.jsx` (atau buat komponen baru `JamDigital.jsx`):

```
/* ===== KODE / OUTPUT LANGKAH 2 ===== */
```

1. JAWABAN KODE JAMDIGITAL.JSX

```
import React, { useState, useEffect } from 'react';
function JamDigital() {
  const [waktu, setWaktu] = useState(new Date());

  // useEffect berjalan sekali setelah komponen dirender pertama kali
  useEffect(() => {

    // Membuat interval timer
    const timerID = setInterval(() => {
      setWaktu(new Date()); // Update state waktu setiap detik
    }, 1000);

    // Cleanup function: Dijalankan saat komponen dihapus/hancur
    // Penting untuk mencegah memory leak
    return () => {
      clearInterval(timerID);
    };
  }, []); // Array kosong [] artinya hanya dijalankan sekali saat mount

  return (
    <div className="alert alert-info text-center">
      <h4>Waktu Sistem Server: {waktu.toLocaleTimeString()}</h4>
    </div>
  );
}
export default JamDigital;
```

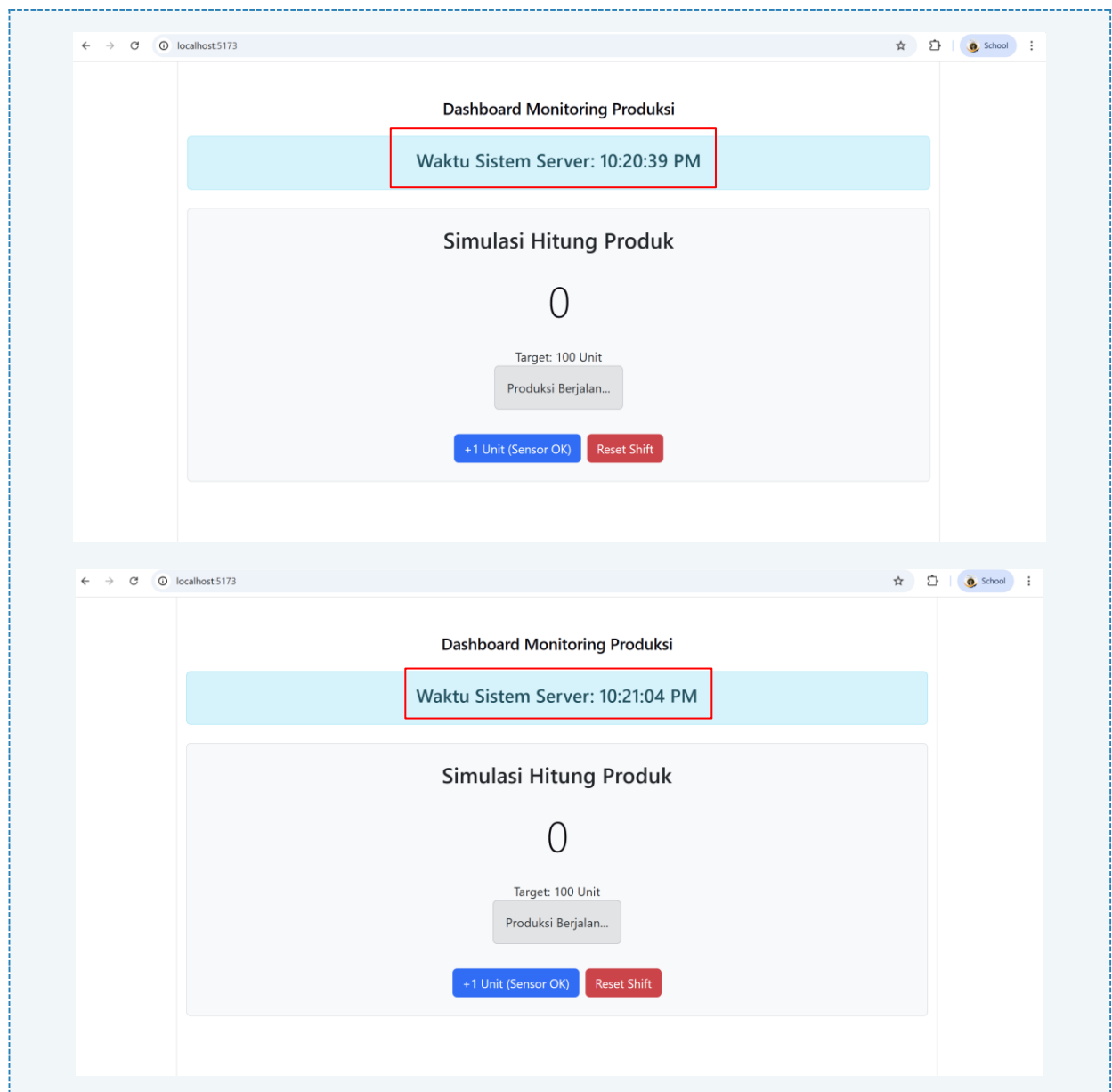
2. JAWABAN KODE APP.JSX

```
import CounterProduksi from "../Komponen/CounterProduksi";
import JamDigital from "../Komponen/JamDigital"; // Tambahkan ini

function App() {
  return (
    <div className="container mt-5">
      <h2 className="text-center mb-4">Dashboard Monitoring Produksi</h2>
```

```
{/* Jam Digital muncul di paling atas */}  
<JamDigital />  
  
<div className="mt-4">  
  <CounterProduksi />  
</div>  
</div>  
);  
}  
  
export default App;
```

Screenshot Hasil:



Gambar E.2 – Hasil Menggunakan useEffect untuk Real-time Clock

D.3 Langkah 3 – Contoh Form Input State

Ubah komponen `KartuMesin.jsx` dari pertemuan 9 agar statusnya bisa diedit secara lokal (opsional latihan):

```
/* ===== KODE / OUTPUT LANGKAH 3 ===== */
```

1. JAWABAN KODE KARTUMESIN.JSX

```
import React, { useState } from 'react';

function KartuMesin({ nama, status, produksi }) {
  // State Lokal untuk ganti status (Langkah 3)
  const [statusLokal, setStatusLokal] = useState(status);

  return (
    <div className="card h-100 shadow-sm border-0 bg-white">
      <div className="card-body p-3 text-center">
        {/* Nama Mesin */}
        <h6 className="fw-bold text-primary mb-2">{nama}</h6>

        {/* Badge Status Dinamis */}
        <div className="mb-2">
          <span className={`badge ${
            statusLokal === 'Running' ? 'bg-success' :
            statusLokal === 'Stop' ? 'bg-danger' : 'bg-warning text-dark'
          }`}>
            {statusLokal}
          </span>
        </div>

        {/* Info Produksi */}
        <p className="small text-muted mb-3">
          Prod: <strong>{produksi}</strong> Unit</strong>
        </p>

        {/* Dropdown untuk ganti status */}
        <div className="mt-2">
          <select
            className="form-select form-select-sm"
            style={{ fontSize: '0.75rem' }}
            value={statusLokal}
            onChange={(e) => setStatusLokal(e.target.value)}
          >
            <option value="Running">Running</option>
            <option value="Stop">Stop</option>
            <option value="Maintenance">Maintenance</option>
          </select>
        </div>
      </div>
    </div>
  );
}
```

```

        </select>
      </div>
    </div>
  </div>
);
}

export default KartuMesin;

```

2. JAWABAN KODE APP.JSX

```

import React from 'react';
import CounterProduksi from './Komponen/CounterProduksi';
import JamDigital from './Komponen/JamDigital';
import KartuMesin from './Komponen/KartuMesin';

function App() {
  return (
    <div className="container mt-5">
      { /* Bagian Atas: Jam Digital */ }
      <div className="text-center mb-5">
        <JamDigital />
      </div>

      <div className="row g-4">
        { /* Sisi Kiri: Counter Produksi (Lebar 4) */ }
        <div className="col-lg-4">
          <div className="card shadow-sm p-4 h-100 border-0 bg-light">
            <CounterProduksi />
          </div>
        </div>

        { /* Sisi Kanan: Panel Kontrol Mesin (Lebar 8) */ }
        <div className="col-lg-8">
          <div className="card shadow-sm p-4 h-100 border-0">
            <h3 className="mb-4 text-center fw-bold">Panel Kontrol Mesin</h3>

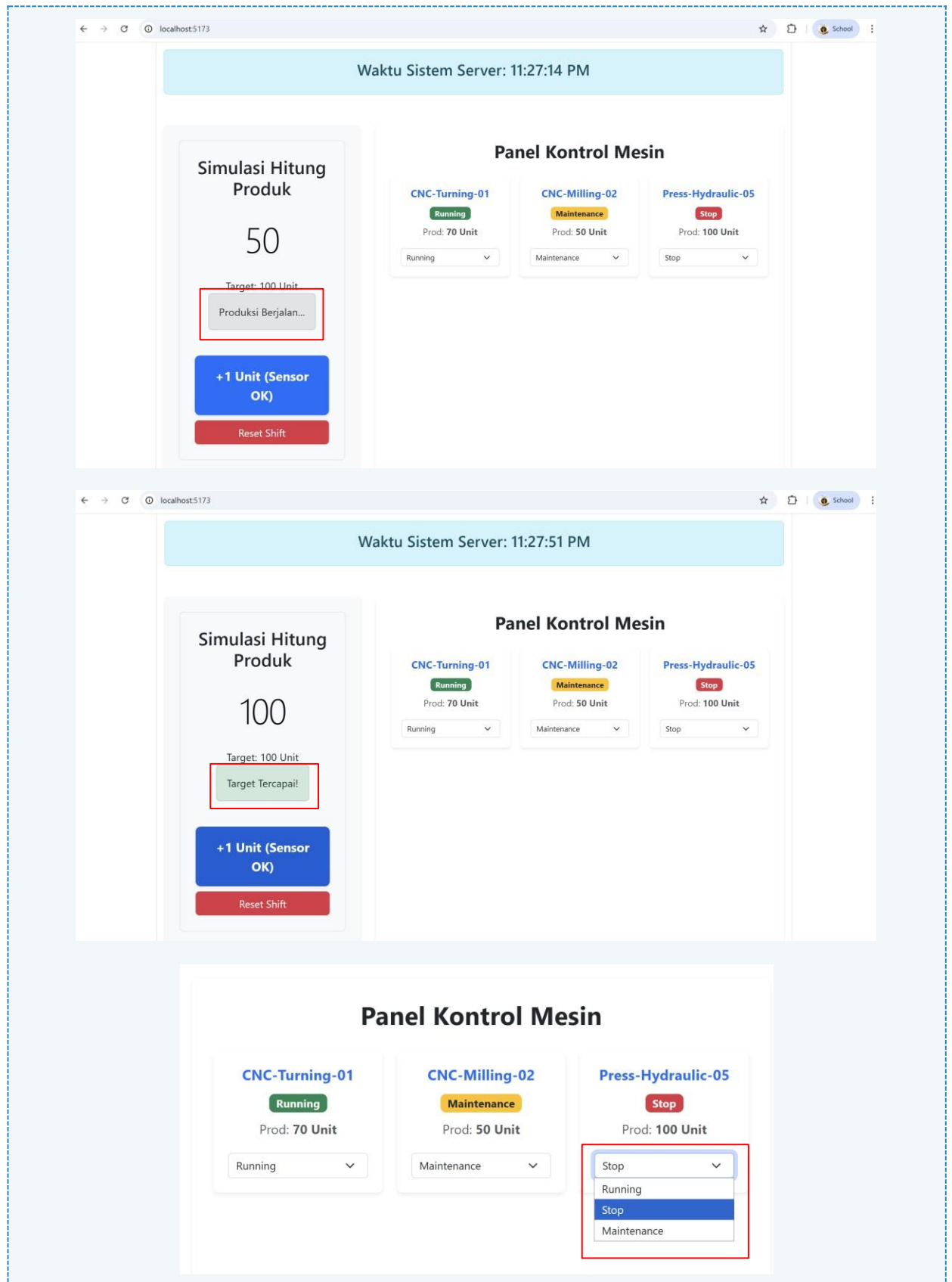
            { /* Kartu-kartu berjejer menyamping dengan row-cols-md-3 */ }
            <div className="row row-cols-1 row-cols-md-3 g-3">
              <div className="col">
                <KartuMesin nama="CNC-Turning-01" status="Running"
produksi={70} />
              </div>
              <div className="col">
                <KartuMesin nama="CNC-Milling-02" status="Maintenance"
produksi={50} />

```

```
        </div>
        <div className="col">
          <KartuMesin nama="Press-Hydraulic-05" status="Stop"
produksi={100} />
        </div>
      </div>
    </div>
  </div>
</div>
);
}

export default App;
```

Screenshot Hasil:



Gambar E.3 – Hasil Contoh Form Input State

E. LATIHAN DAN TUGAS MANDIRI

Jawab dan dokumentasikan seluruh latihan/tugas yang ada di bagian "Latihan/Tugas Mandiri" pada modul Anda.

E.1 Latihan 1 – Dependency Array useEffect

Pertanyaan / Instruksi Latihan:

Modifikasi `JamDigital`. Tambahkan input text untuk memasukkan nama kota. Gunakan `useEffect` untuk mengubah `document.title` menjadi "Jam [Nama Kota]". Gunakan state kota sebagai dependency array `[kota]`.

Jawaban / Kode Solusi:

```
import React, { useState, useEffect } from 'react';

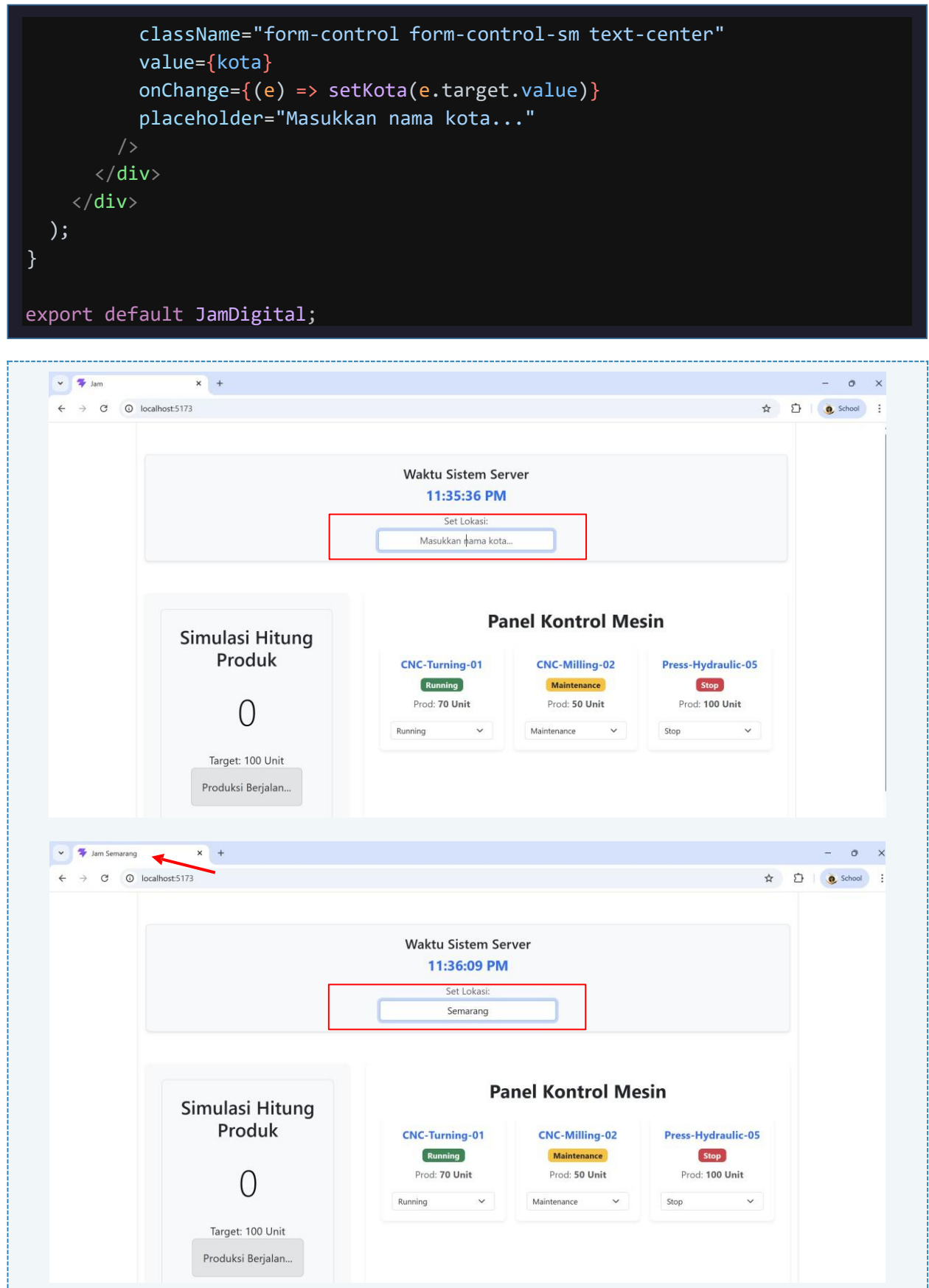
function JamDigital() {
  const [waktu, setWaktu] = useState(new Date());
  // 1. Tambahkan state untuk menyimpan nama kota
  const [kota, setKota] = useState('Jakarta');

  // useEffect untuk Update Jam (Setiap detik)
  useEffect(() => {
    const timer = setInterval(() => {
      setWaktu(new Date());
    }, 1000);
    return () => clearInterval(timer);
  }, []); // Array kosong karena jam jalan terus sejak awal

  // --- TUGAS MANDIRI 1: Update Document Title ---
  // useEffect ini hanya akan jalan kalau isi variabel [kota] berubah
  useEffect(() => {
    document.title = `Jam ${kota}`;
    console.log("Judul tab berubah!"); // Cek di console browser
  }, [kota]); // Inilah yang disebut Dependency Array

  return (
    <div className="text-center mb-4 p-3 border rounded bg-light shadow-sm">
      <h5>Waktu Sistem Server</h5>
      <h2 className="text-primary fw-bold">
        {waktu.toLocaleTimeString()}
      </h2>

      </* Input Text untuk Nama Kota */>
      <div className="mt-3 mx-auto" style={{ maxWidth: '300px' }}>
        <label className="small d-block mb-1 text-muted">Set Lokasi:</label>
        <input
          type="text"
        >
      </div>
    </div>
  );
}
```



Gambar F.1 – Hasil Dependency Array useEffect

E.2 Latihan 2 – Conditional Rendering

Pertanyaan / Instruksi Latihan:

Pada `CounterProduksi`, buat tombol "Emergency Stop". Saat diklik, ubah state status menjadi "EMERGENCY", tombol "+1" menjadi disabled (`disabled`), dan tampilkan pesan merah.

Jawaban / Kode Solusi:

```
import React, { useState } from 'react';

function CounterProduksi() {
  const [hitung, setHitung] = useState(0);
  const [isEmergency, setIsEmergency] = useState(false);

  return (
    <div className="text-center">
      <h4 className="text-muted mb-3">Simulasi Hitung Produk</h4>

      {/* Conditional Rendering Alert */}
      {isEmergency && (
        <div className="alert alert-danger fw-bold border-0 shadow-sm mb-4">
          STATE: EMERGENCY STOP ACTIVATED!
        </div>
      )}

      <h1 className={`display-1 fw-bold my-4 ${isEmergency ? 'text-danger' : 'text-dark'}`}>
        {hitung}
      </h1>

      <p className="text-secondary mb-4">Target: 100 Unit</p>

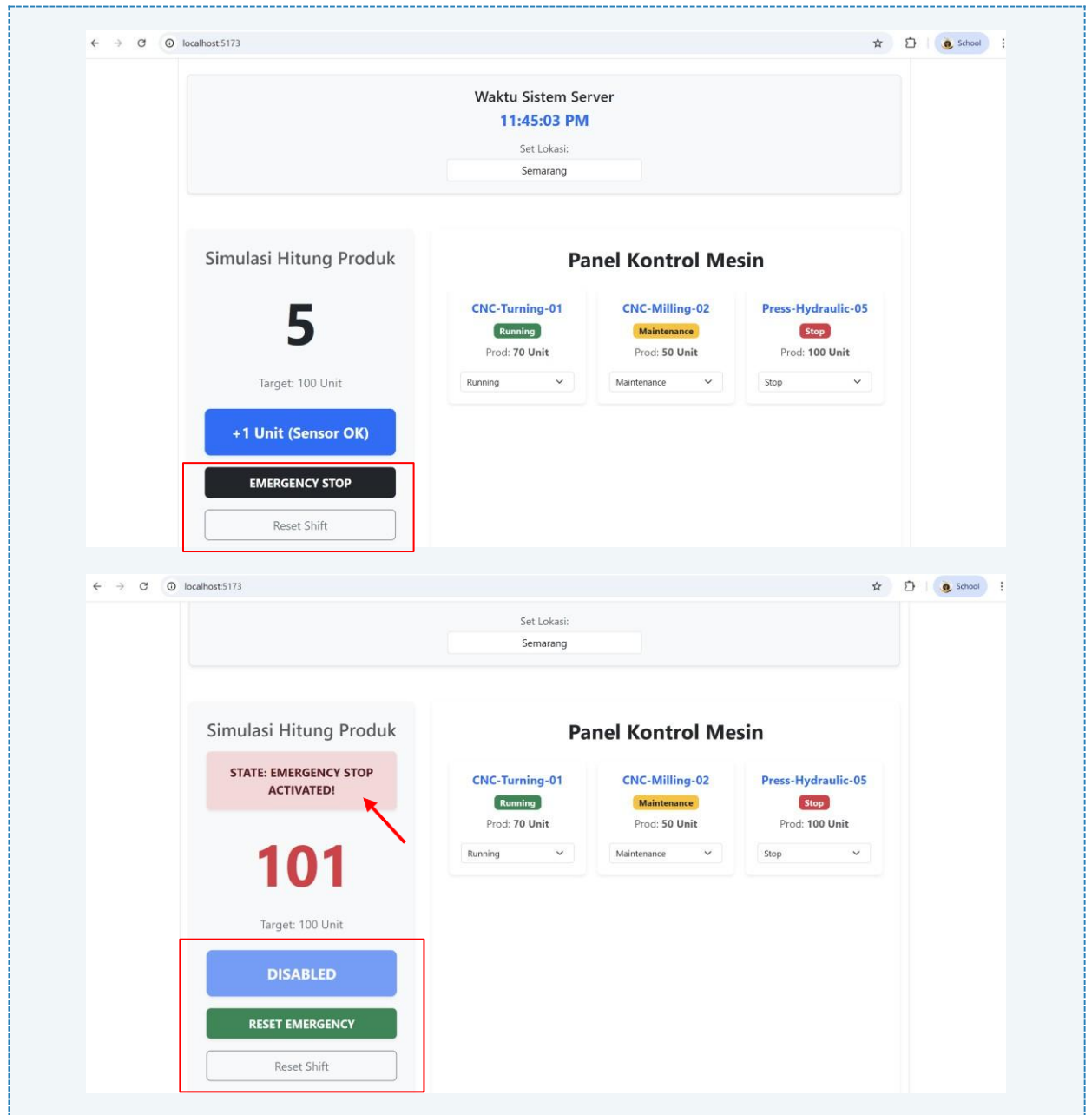
      <div className="d-grid gap-3">
        <button
          className="btn btn-primary btn-lg fw-bold py-3 shadow-sm"
          onClick={() => setHitung(hitung + 1)}
          disabled={isEmergency}
        >
          {isEmergency ? 'DISABLED' : '+1 Unit (Sensor OK)'}
        </button>

        {/* Tombol Emergency / Reset Normal */}
        {!isEmergency ? (
          <button
            className="btn btn-dark fw-bold py-2"
            onClick={() => setIsEmergency(true)}
          >
```

```
    >
      EMERGENCY STOP
    </button>
  ) : (
    <button
      className="btn btn-success fw-bold py-2 shadow-sm"
      onClick={() => setIsEmergency(false)}
    >
      RESET EMERGENCY
    </button>
  )}

  <button
    className="btn btn-outline-secondary py-2"
    onClick={() => {
      setHitung(0);
      setIsEmergency(false);
    }}
  >
    Reset Shift
  </button>
</div>
</div>
);
}

export default CounterProduksi;
```



Gambar F.2 – Hasil Conditional Rendering

E.3 Tugas Proyek Mini – Kalkulator OEE Sederhana

Deskripsi proyek mini:

- Buat komponen form input: `Plan Time`, `Run Time`, `Total Parts`, `Good Parts`.
- Gunakan State untuk menyimpan nilai-nilai ini.
- Hitung OEE secara otomatis (Real-time) setiap kali input berubah.
- Rumus OEE = (Availability x Performance x Quality).
- Tampilkan hasil OEE dalam persen. Jika OEE < 50%, warnanya merah. Jika > 85%, warnanya hijau.

Penjelasan pendekatan/solusi yang digunakan:

Proyek mini ini dilakukan melalui beberapa tahapan yang sistematis, dimulai dari perancangan tampilan hingga implementasi

1. Menyiapkan struktur komponen React menggunakan Vite, dengan mengimpor library pendukung seperti `useState` untuk manajemen data dan `useEffect` untuk sinkronisasi sistem.
2. Membuat state inputs untuk menampung empat variabel kunci produksi (Plan Time, Run Time, Total Parts, dan Good Parts) serta state history untuk menyimpan daftar riwayat perhitungan.
3. Membuat struktur tampilan menggunakan elemen input HTML yang dihubungkan dengan fungsi `handleChange`. Hal ini memastikan setiap perubahan angka yang diketik pengguna tersinkronisasi ke dalam state secara asinkron.
4. Mengimplementasikan rumus standar industri OEE ($\text{Availability} \times \text{Performance} \times \text{Quality}$). Perhitungan dilakukan secara langsung di dalam badan komponen agar hasil skor berubah seketika (*real-time*) setiap kali input diperbarui.
5. Menambahkan proteksi *Divide by Zero* (pembagian dengan nol) dan menggunakan `Math.min` untuk mengunci nilai maksimal pada angka 100%. Hal ini dilakukan untuk menjaga akurasi data agar tidak terjadi nilai yang tidak rasional (*infinity* atau *null*).
6. Membuat logika percabangan (*conditional rendering*) untuk menentukan status performa mesin berdasarkan ambang batas standar: Unacceptable (< 50%), Typical (50-85%), dan World Class (> 85%).
7. Mengintegrasikan class CSS Bootstrap secara kondisional agar warna teks, badge, dan progress bar berubah secara otomatis (Merah, Kuning, atau Hijau) sesuai dengan skor OEE yang dihasilkan.
8. Menambahkan sistem pendukung keputusan sederhana (*Decision Support System*) yang memberikan saran perbaikan teknis (seperti Kaizen atau TPM) berdasarkan status efisiensi yang terdeteksi.
9. Membuat fungsi `simpanKeRiwayat` untuk menangkap *snapshot* data saat itu (termasuk jam dan tanggal) dan menambahkannya ke dalam array riwayat menggunakan *spread operator*.
10. Menggunakan hook `useEffect` untuk menyimpan array riwayat ke dalam memori browser (*LocalStorage*). Fitur ini memastikan data laporan produksi tidak hilang meskipun halaman web dimuat ulang (*refresh*).
11. Menambahkan tombol "Reset Riwayat" yang dilengkapi dengan dialog konfirmasi (`window.confirm`) untuk menghapus seluruh data pada state dan membersihkan memori *LocalStorage* secara permanen.
12. Melakukan pengujian fungsionalitas dengan memasukkan berbagai skenario angka produksi, memverifikasi perubahan warna indikator, menguji ketahanan data saat *refresh*, dan memastikan fitur reset berjalan dengan benar.

Jawaban / Kode Solusi:

```
import React, { useState, useEffect } from 'react';

function KalkulatorOEE() {
  const [inputs, setInputs] = useState({
```

```
    planTime: 480,
    runTime: 440,
    totalParts: 1000,
    goodParts: 950
  });

  // FITUR 1: Mengambil data dari localStorage saat pertama kali halaman
  dibuka
  const [history, setHistory] = useState(() => {
    const savedHistory = localStorage.getItem('oeo_history');
    return savedHistory ? JSON.parse(savedHistory) : [];
  });

  // FITUR 2: Menyimpan data ke localStorage setiap kali ada perubahan pada
  state history
  useEffect(() => {
    localStorage.setItem('oeo_history', JSON.stringify(history));
  }, [history]);

  const handleChange = (e) => {
    const { name, value } = e.target;
    const val = value === "" ? 0 : parseFloat(value);
    setInputs({ ...inputs, [name]: val });
  };

  // Logika Perhitungan OEE
  const availability = inputs.planTime > 0 ? Math.min(inputs.runTime /
    inputs.planTime, 1) : 0;
  const performance = inputs.runTime > 0 ? Math.min(inputs.totalParts /
    (inputs.runTime * 2.5), 1) : 0;
  const quality = inputs.totalParts > 0 ? Math.min(inputs.goodParts /
    inputs.totalParts, 1) : 0;
  const oee = (availability * performance * quality) * 100;

  // Penentuan Warna & Status
  let warnaTeks = "text-warning";
  let warnaBadge = "bg-warning text-dark";
  let statusLabel = "TYPICAL (Standar)";
  let rekomendasi = "Lakukan Preventive Maintenance untuk mengurangi minor
  stoppages.";

  if (oee < 50) {
    warnaTeks = "text-danger";
    warnaBadge = "bg-danger";
    statusLabel = "UNACCEPTABLE (Rendah)";
  }
}
```

```

    rekomendasi = "Efisiensi kritis! Segera lakukan Kaizen pada ketersediaan
mesin.";
  } else if (oee > 85) {
    warnaTeks = "text-success";
    warnaBadge = "bg-success";
    statusLabel = "WORLD CLASS (Sempurna)";
    rekomendasi = "Performa luar biasa! Pertahankan ritme kerja lini
produksi.";
  }

  const simpanKeRiwayat = () => {
    if (inputs.planTime === 0 || inputs.totalParts === 0) {
      alert("Plan Time dan Total Parts tidak boleh nol!");
      return;
    }
    const dataBaru = {
      id: Date.now(),
      jam: new Date().toLocaleString('id-ID'), // Menambahkan tanggal agar
log lebih jelas
      hasil: oee.toFixed(1),
      ...inputs
    };
    setHistory([dataBaru, ...history]);
  };

  // FITUR 3: Fungsi Reset Riwayat
  const resetRiwayat = () => {
    if (window.confirm("Apakah Anda yakin ingin menghapus semua riwayat
produksi?")) {
      setHistory([]);
      localStorage.removeItem('oee_history');
    }
  };

  return (
    <div className="container py-4">
      <div className="card shadow-lg border-0 rounded-4 overflow-hidden">
        <div className="bg-dark p-3 text-center">
          <h4 className="text-white fw-bold mb-0 text-uppercase">OEE Real-
Time Monitor</h4>
          <small className="text-secondary text-uppercase">Maessa Andrea
Vallenia - 23051430044</small>
        </div>

        <div className="card-body p-4 bg-light">
          <div className="row g-4">

```



```

<div className="col-lg-5">
  { /* INPUT PANEL */ }
  <div className="card border-0 shadow-sm p-3 mb-3 rounded-3
text-start">
    <h6 className="fw-bold text-muted mb-3"><i className="bi bi-
pencil-square me-2"></i>Data Produksi</h6>
    <div className="row g-2">
      <div className="col-6 mb-2">
        <label className="small fw-bold">Plan Time (Min)</label>
        <input type="number" name="planTime" className="form-
control" value={inputs.planTime} onChange={handleChange} />
      </div>
      <div className="col-6 mb-2">
        <label className="small fw-bold">Run Time (Min)</label>
        <input type="number" name="runTime" className="form-
control" value={inputs.runTime} onChange={handleChange} />
      </div>
      <div className="col-6 mb-2">
        <label className="small fw-bold">Total Parts</label>
        <input type="number" name="totalParts" className="form-
control" value={inputs.totalParts} onChange={handleChange} />
      </div>
      <div className="col-6 mb-2">
        <label className="small fw-bold">Good Parts</label>
        <input type="number" name="goodParts" className="form-
control" value={inputs.goodParts} onChange={handleChange} />
      </div>
    </div>
    <button className="btn btn-primary w-100 mt-3 fw-bold shadow-
sm" onClick={simpanKeRiwayat}>
      SIMPAN KE LOG
    </button>
  </div>

  { /* ANALYTICS RESULT */ }
  <div className="card border-0 shadow-sm p-4 text-center
rounded-3">
    <span className={`badge ${warnaBadge} mb-2 px-3 py-2 align-
self-center`} >{statusLabel}</span>
    <h1 className={`display-2 fw-bold ${warnaTeks} mb-
0`} >{oeo.toFixed(1)}%</h1>
    <div className="progress mt-3" style={{height: '12px'}}>
      <div className={`progress-bar progress-bar-striped
progress-bar-animated ${warnaBadge}`}
        style={{width: `${Math.min(oeo, 100)}%`} ></div>

```

```

    </div>
    <div className="alert alert-info mt-4 py-3 small border-0
shadow-sm text-start">
      <strong><i className="bi bi-lightbulb me-1"></i>
Rekomendasi Industri:</strong><br/>
      {rekomendasi}
    </div>
  </div>
</div>

<div className="col-lg-7">
  { /* LOG HISTORY */ }
  <div className="card border-0 shadow-sm p-3 h-100 rounded-3
text-start">
    <div className="d-flex justify-content-between align-items-
center mb-3 border-bottom pb-2">
      <h6 className="fw-bold text-muted mb-0"><i className="bi
bi-clock-history me-2"></i>Riwayat Log Produksi</h6>
      {history.length > 0 && (
        <button className="btn btn-outline-danger btn-sm"
onClick={resetRiwayat}>
          <i className="bi bi-trash me-1"></i> Reset
Riwayat
        </button>
      )}
    </div>

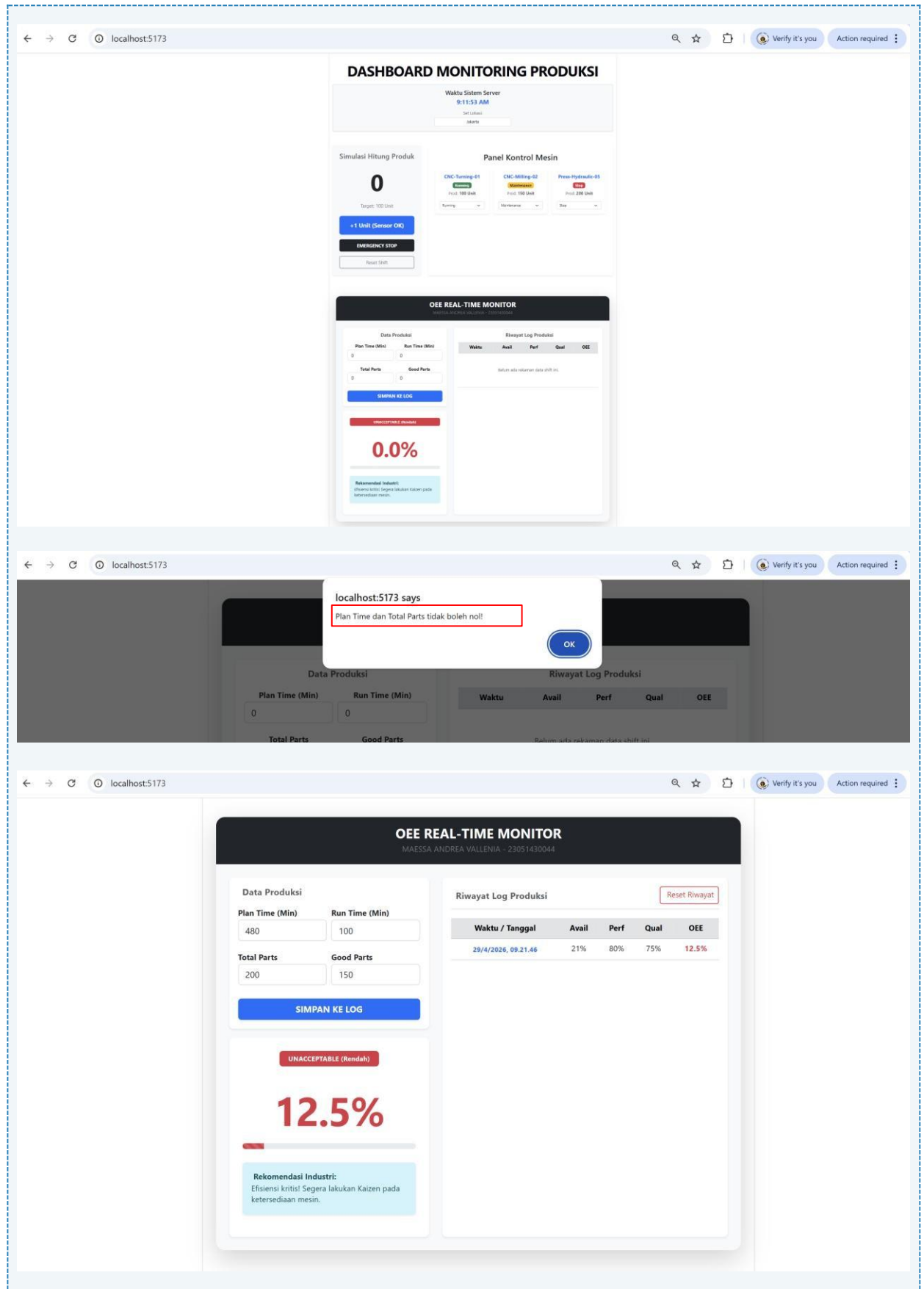
    <div className="table-responsive">
      <table className="table table-hover align-middle">
        <thead className="table-secondary">
          <tr className="small text-center">
            <th>Waktu / Tanggal</th>
            <th>Avail</th>
            <th>Perf</th>
            <th>Qual</th>
            <th>OEE</th>
          </tr>
        </thead>
        <tbody className="small text-center">
          {history.length === 0 ? (
            <tr><td colspan="5" className="py-5 text-muted
italic">Belum ada rekaman data shift ini.</td></tr>
          ) : (
            history.map((log) => (
              <tr key={log.id}>

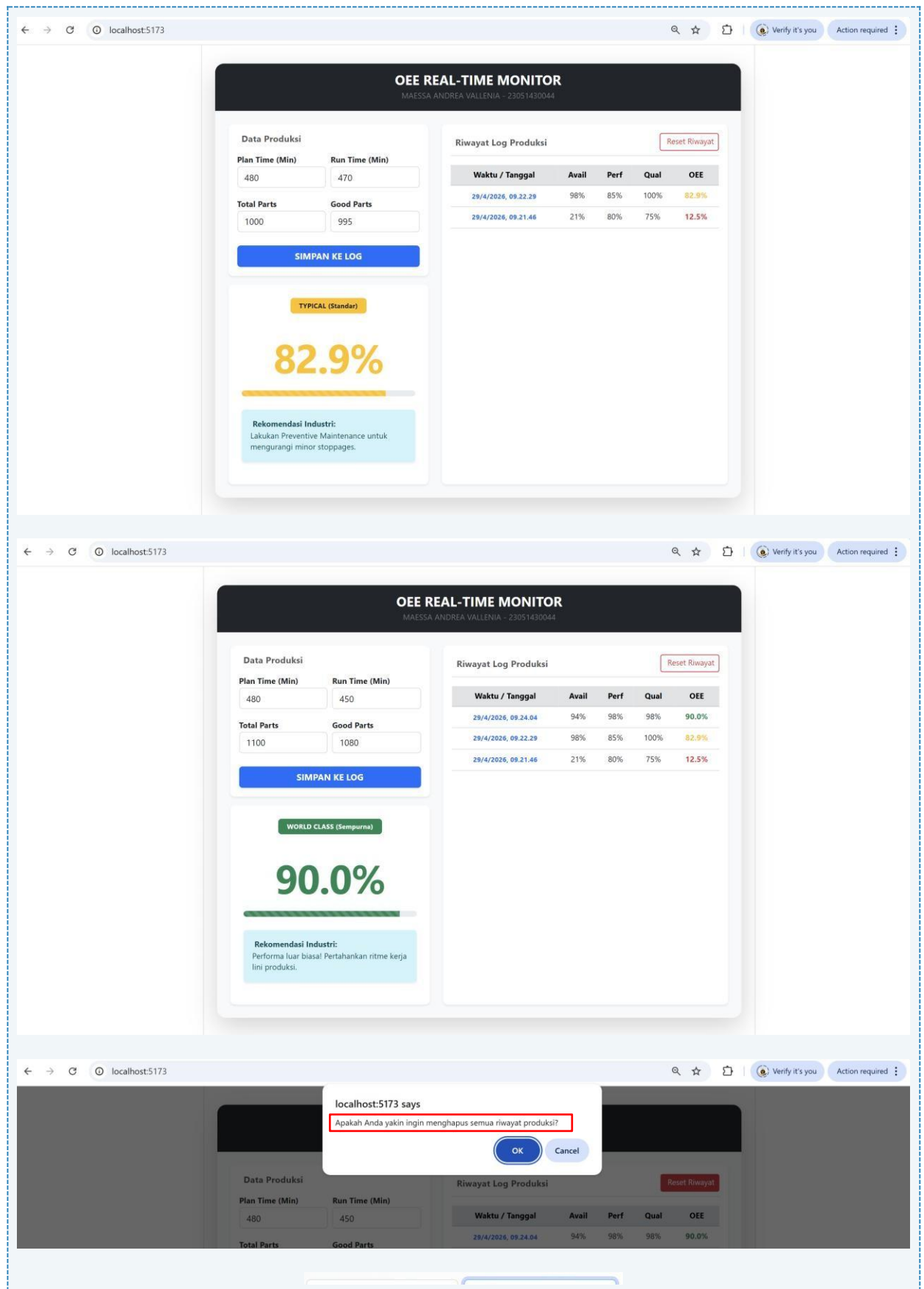
```

```

        <td className="fw-bold text-primary"
style={{fontSize: '0.75rem'}}>{log.jam}</td>
        <td>{((log.runTime/log.planTime)*100).toFixed(0)}%</td>
        <td>{((log.totalParts/(log.runTime*2.5))*100).toF
ixed(0)}%</td>
        <td>{((log.goodParts/log.totalParts)*100).toFixed
(0)}%</td>
        <td className={`fw-bold ${log.hasil < 50 ? 'text-
danger' : log.hasil > 85 ? 'text-success' : 'text-warning'}`}>
          {log.hasil}%
        </td>
      </tr>
    ))
  })
</tbody>
</table>
</div>
</div>
</div>
</div>
);
}

export default KalkulatorOEE;
```





Gambar F.3 – Hasil Kalkulator OEE Sederhana

F. PEMBAHASAN

Praktikum Pertemuan 10 berfokus pada transformasi aplikasi React dari statis menjadi dinamis melalui penguasaan *State Management*, *Lifecycle Hooks*, dan *Conditional Rendering*. Pada tahap awal, pemahaman ditekankan pada konsep State sebagai memori internal komponen yang bersifat *read-write*, berbeda dengan *Props* yang bersifat *read-only*. Implementasi pertama dilakukan pada komponen CounterProduksi, di mana variabel jumlah dikelola menggunakan *hook* `useState`. Hal ini memungkinkan aplikasi untuk merespons interaksi pengguna secara instan, seperti penambahan unit produksi lewat sensor simulasi, yang secara otomatis memicu proses *re-render* untuk memperbarui tampilan angka di layar.

Selanjutnya, praktikum mengeksplorasi Lifecycle Hooks melalui penggunaan `useEffect` pada komponen JamDigital. Penggunaan *hook* ini sangat krusial dalam skenario industri untuk menangani *side effects* seperti sinkronisasi waktu sistem server secara *real-time*. Di sini, dipelajari pentingnya *Dependency Array* kosong `[]` agar timer hanya berjalan sekali saat komponen dimuat (*mount*), serta penggunaan *Cleanup Function* (`clearInterval`) untuk mencegah kebocoran memori (*memory leak*). Konsep ini memberikan simulasi bagaimana aplikasi dapat terus memperbarui data secara otomatis tanpa interaksi manual dari pengguna.

Pengembangan dilanjutkan dengan penerapan Form Input State pada komponen KartuMesin dan tugas mandiri. Mahasiswa melatih kemampuan dalam mengikat nilai input (*controlled component*) pada elemen `select` dan `input text`. Hal ini memungkinkan status mesin atau parameter judul halaman (`document.title`) berubah secara dinamis berdasarkan input yang dimasukkan. Pada tahap ini, pemahaman mengenai *asynchronous state update* mulai diperdalam, terutama saat menangani perubahan state yang saling bergantung satu sama lain.

Puncak praktikum ini adalah pembuatan Proyek Mini Kalkulator OEE. Proyek ini menggabungkan seluruh konsep yang telah dipelajari: penggunaan `useState` untuk menyimpan empat parameter utama produksi, penerapan logika matematis secara *real-time*, dan penggunaan Conditional Rendering untuk memberikan peringatan visual. Aplikasi ini mensimulasikan sistem pemantauan efisiensi industri yang cerdas, di mana antarmuka dapat berubah warna secara otomatis (Merah untuk efisiensi kritis $< 50\%$ dan Hijau untuk standar dunia $> 85\%$) sebagai bentuk umpan balik instan bagi operator produksi.

F.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum menunjukkan bahwa implementasi aplikasi telah sepenuhnya sesuai dengan teori State Management menggunakan React Hooks. Penggunaan `useState` pada variabel produksi dan parameter OEE terbukti efektif dalam menyimpan data yang bersifat dinamis. Sesuai dengan teori, setiap kali fungsi *setter* (seperti `setInputs`) dipanggil, React berhasil mendeteksi perubahan state dan melakukan pembaruan pada bagian DOM yang relevan tanpa mengganggu elemen lain, yang membuktikan efisiensi mekanisme *re-rendering* pada React.

Dalam aspek Lifecycle Hooks, penggunaan `useEffect` pada fitur jam digital dan sinkronisasi judul halaman telah sesuai dengan teori *side effects*. Hasil analisis menunjukkan bahwa penggunaan *dependency array* yang tepat sangat berpengaruh terhadap performa aplikasi. Sebagai contoh, pada latihan modifikasi judul dokumen, penggunaan `[kota]` dalam *dependency array* memastikan kode hanya berjalan ketika nama kota berubah, bukan pada setiap *render*. Hal ini selaras dengan teori optimasi performa dalam pengembangan perangkat lunak berbasis komponen.

Shutterstock

Analisis terhadap Conditional Rendering pada kalkulator OEE dan fitur *Emergency Stop* menunjukkan kesesuaian dengan prinsip *logic-driven UI*. Implementasi operator *ternary* atau logika AND (&&) di dalam JSX berhasil menampilkan pesan "Target Tercapai" atau mengubah status "EMERGENCY" secara akurat. Perubahan warna indikator (merah/hijau) berdasarkan nilai OEE membuktikan bahwa properti CSS dinamis dapat dikendalikan sepenuhnya oleh state, sesuai dengan teori yang menyatakan bahwa tampilan adalah fungsi dari data ($UI = f(state)$).

Terakhir, sinkronisasi data riwayat dengan `LocalStorage` (sebagai fitur tambahan) memperkuat teori tentang *persistence data* pada aplikasi *client-side*. Walaupun state bersifat sementara (akan hilang saat *refresh*), integrasi dengan `localStorage` memastikan data tetap bertahan. Secara keseluruhan, seluruh fitur yang dibangun mulai dari *counter* sederhana hingga kalkulator efisiensi industri telah memenuhi standar kompetensi praktikum dalam menciptakan aplikasi yang interaktif, memiliki alur logika yang kuat, dan mengikuti siklus hidup komponen yang benar.

F.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Skor OEE menampilkan NaN atau Infinity saat kolom <i>Plan Time</i> atau <i>Total Parts</i> dikosongkan atau diisi angka 0.	Menambahkan validasi pengecekan kondisi (misal: <code>planTime > 0 ? hasil : 0</code>) untuk memastikan pembagi tidak bernilai nol sebelum kalkulasi dilakukan.

F.3 Kaitan dengan Penerapan di Industri

Materi pada praktikum ini memiliki keterkaitan yang sangat erat dengan penerapan di dunia industri modern, khususnya dalam pengembangan Sistem Monitoring Produksi Terintegrasi, Smart Factory, dan Dashboard Efisiensi Mesin. Dalam industri manufaktur, data performa mesin tidak lagi dicatat secara manual, melainkan dipantau secara *real-time*. Konsep *State Management* yang dipelajari memungkinkan pembuatan dashboard yang interaktif, di mana setiap detik data dari sensor produksi dapat langsung ditampilkan dan dianalisis tanpa perlu memuat ulang halaman secara keseluruhan.

Penerapan Kalkulator OEE secara *real-time* adalah instrumen vital bagi manajemen pabrik. OEE (*Overall Equipment Effectiveness*) merupakan standar global untuk mengukur produktivitas manufaktur. Dengan sistem yang mampu menghitung otomatis faktor *Availability*, *Performance*, dan *Quality*, pihak manajemen dapat segera mengidentifikasi titik lemah produksi, seperti *downtime* yang tinggi atau tingkat *reject* yang melebihi ambang batas. Fitur Conditional Rendering (perubahan warna indikator) sangat relevan sebagai sistem peringatan dini (*Early Warning System*). Jika nilai OEE turun di bawah 50%, indikator merah akan segera memberi sinyal kepada supervisor untuk melakukan tindakan korektif atau *Emergency Stop*.

Penggunaan *Hooks* seperti *useEffect* untuk menjalankan timer atau sinkronisasi data menyerupai sistem SCADA (*Supervisory Control and Data Acquisition*) yang memantau ritme kerja mesin setiap waktu. Selain itu, penyimpanan data melalui *LocalStorage* memberikan gambaran kasar mengenai cara kerja database lokal pada perangkat IoT (*Edge Computing*) yang harus tetap menyimpan data meskipun koneksi internet terputus. Hal ini memastikan riwayat insiden dan performa mesin tetap terjaga untuk kebutuhan audit dan analisis jangka panjang.

Secara keseluruhan, pemahaman mendalam mengenai *State*, *Hooks*, dan *Conditional Rendering* membekali mahasiswa Teknik Industri dengan kemampuan untuk merancang solusi digital yang cerdas. Integrasi antara logika pemrograman dan prinsip manajemen industri ini sangat krusial dalam mendukung transformasi menuju Industri 4.0, di mana efisiensi operasional sangat bergantung pada ketersediaan data yang cepat, akurat, dan mudah dipahami oleh pengambil keputusan.

G. KESIMPULAN

Tuliskan 3–5 poin kesimpulan yang merangkum hal-hal yang dipelajari dan dicapai pada praktikum modul ini. Kesimpulan harus spesifik, bukan bersifat umum.

1

Mahasiswa mampu mengimplementasikan konsep *State Management* menggunakan hook *useState* untuk menyimpan dan memperbarui data parameter produksi secara dinamis tanpa memerlukan pemuatan ulang halaman (*reload*).

2

Mahasiswa dapat menggunakan hook *useEffect* untuk menangani *side effects* seperti pembuatan jam digital *real-time* dan sinkronisasi data ke *LocalStorage*, guna memastikan kontinuitas data riwayat produksi pada browser.

3	Mahasiswa berhasil menerapkan teknik Conditional Rendering untuk memberikan umpan balik visual instan, seperti perubahan warna indikator performa (Merah/Kuning/Hijau) berdasarkan logika ambang batas skor OEE.
4	Mahasiswa mampu membangun logika perhitungan matematis yang kompleks secara otomatis (<i>real-time</i>) dengan validasi keamanan input untuk mencegah error teknis seperti <i>Divide by Zero</i> dan nilai <i>out-of-bounds</i> .
5	Mahasiswa memahami cara mengintegrasikan interaksi pengguna (<i>event handling</i>) dengan logika industri, sehingga mampu menciptakan dashboard monitoring efisiensi mesin yang responsif dan sesuai dengan kebutuhan analisis performa di Industri 4.0.

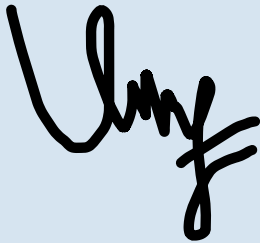
H. DAFTAR PUSTAKA

Tuliskan semua sumber referensi yang digunakan dalam format APA 7th Edition. Minimal 3 referensi yang relevan (termasuk Manual Praktikum dan sumber online resmi seperti MDN, Reactjs.org, dll).

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Kom, I. L. M. (2023). <i>Dasar Algoritma dan Pemrograman Javascript</i> . Irvan Lewenusa, M. Kom.
[3]	Sholikhan, M. (2022). Html, Css Dan Javascript. <i>Penerbit Yayasan Prima Agus Teknik</i> , 1-343 .
[4]	Torres, J. A. G., Lau, S. H., Anchuri, P., Stevens, J. M., Tabora, J. E., Li, J., ... & Doyle, A. G. (2022). A multi-objective active learning platform and web app for reaction optimization. <i>Journal of the American Chemical Society</i> , 144(43), 19999-20007.
[5]	Patil, K., & Javagal, S. D. (2022). React state management and side-effects—A Review of Hooks. <i>International Research Journal Of Engineering And Technology (IRJET)</i> , 9, 12.
[6]	https://developer.mozilla.org/en-US/docs/Web/JavaScript
[7]	https://www.geeksforgeeks.org/
[8]	https://react.dev/reference/react/useState
[9]	https://react.dev/
[10]	https://react.dev/reference/react/useEffect
[11]	https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide
[12]	https://getbootstrap.com/docs/5.3/getting-started/introduction/

I. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Diperiksa oleh Asisten / Dosen
 Maessa Andrea Vallenia NIM: 23051430044	 Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____

MODUL 11

REACT ROUTING & MULTI-PAGE APP + INTEGRASI API

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 11 (Pertemuan 11)
Topik Utama	Single Page Application (SPA) Navigation, React Router DOM, dan Fetch API dalam React.
Sub-Topik	Konsep Single Page Application (SPA), Client-Side Routing, Instalasi React Router DOM, BrowserRouter, Routes & Route Configuration, Path & Element Props, Link Component vs Anchor Tag (<a>), Navigasi Antar Halaman Tanpa Reload, Penanganan Halaman 404 (Not Found Route), Pengenalan Fetch API, Integrasi Endpoint API ke React, Manajemen State untuk Data Eksternal, Rendering List Data dari API, Penggunaan useEffect untuk Data Fetching, Konsep Dependency Array pada Fetching, Simulasi Loading State, Struktur Folder Halaman (Pages).
Durasi Praktikum	340 Menit
Tanggal Praktikum	1 Mei 2026
Nama Mahasiswa	Maessa Andrea Vallenia
NIM	23051430044
Kelas	A

B. TUJUAN PEMBELAJARAN

Tuliskan tujuan pembelajaran yang ingin dicapai pada modul ini, sesuai dengan yang tertera di Manual Praktikum.

1	Memahami konsep SPA (Single Page Application).
2	Mampu mengatur navigasi antar halaman tanpa reload browser menggunakan `reactrouter-dom`.
3	Menerapkan `fetch` API di dalam `useEffect` untuk mengambil data eksternal.
4	Membuat struktur aplikasi multi-halaman (Dashboard, Inventori, Settings).

C. ALAT DAN BAHAN

Sebutkan seluruh perangkat keras, perangkat lunak, dan sumber referensi yang digunakan selama praktikum.

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	<i>Code Editor</i>
2	Google Chrome / Browser Modern	Versi terbaru	<i>Testing & Debugging</i>
3	Live Server	Versi 5.7.9	Menjalankan HTML dan memantau perubahan file
4	JavaScript (Built-in Browser)	ES6 (Default Browser Modern)	Bahasa pemrograman untuk logika dan perhitungan
5	CodeSnap	Versi 1.3.4	Dokumentasi kode program
6	Git dan GitHub	Versi 2.53.0	Version control system (VCS) berbasis Git
7	Koneksi Internet	Stabil	Mengakses Library dan Framework
8	Console Browser (Developer Tools)	Default Browser	Melihat output program dan debugging
9	Bootstrap	Versi 5.3	Membantu pembuatan UI seperti button, card, dan alert pada panel control
10	File praktikum	Pertemuan sebelumnya	Digunakan sebagai dasar pengembangan panel kontrol

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
11	Node.js	Versi 18 ke atas	Runtime JavaScript untuk menjalankan React dan npm
12	npm (Node Package Manager)	Versi terbaru	Mengelola package dan dependensi React
13	Vite	Versi terbaru	Tools untuk membuat dan menjalankan project React
14	React	Versi terbaru	Library JavaScript untuk membangun user interface
15	JSONPlaceholder (API)	Online Service	Sumber data eksternal (Mock API) untuk praktik Fetching
16	React Router DOM	Versi terbaru	Library navigasi untuk membuat sistem routing (SPA)

D. DASAR TEORI

1. Single Page Application (SPA): Merupakan konsep aplikasi web yang memuat seluruh halaman hanya sekali. Saat pengguna berpindah menu, browser tidak melakukan *reload* total, melainkan hanya mengganti komponen yang diperlukan saja. Hal ini membuat aplikasi terasa lebih cepat dan responsif, menyerupai aplikasi *native*.
2. React Router DOM: Adalah library standar untuk menangani sistem navigasi (routing) pada aplikasi React. Library ini memungkinkan pengembang menentukan jalur (*path*) URL yang berbeda untuk

menampilkan komponen halaman yang berbeda pula, seperti /dashboard atau /inventori.

3. **BrowserRouter, Routes, dan Route:** BrowserRouter berfungsi sebagai pembungkus utama agar fitur routing aktif di seluruh aplikasi. Routes bertindak sebagai kontainer untuk semua rute yang tersedia, sementara Route digunakan untuk menghubungkan satu *path* tertentu dengan elemen komponen yang ingin ditampilkan.
4. **Komponen Link:** Dalam React Router, navigasi dilakukan menggunakan komponen `<Link>` sebagai pengganti tag anchor (`<a>`). Penggunaan Link sangat krusial karena ia mencegah browser melakukan pemuatan ulang halaman (reload) sehingga *state* aplikasi tetap terjaga dan performa SPA tetap optimal.
5. **Fetch API:** Adalah antarmuka bawaan browser modern untuk melakukan pengambilan data (*fetching*) secara asinkron dari server atau API eksternal. Fetch API mengembalikan *Promise* yang kemudian diolah menjadi data JSON untuk ditampilkan di dalam antarmuka aplikasi.
6. **Integrasi API pada useEffect:** Proses pengambilan data eksternal biasanya diletakkan di dalam hook `useEffect` dengan *dependency array* kosong `[]`. Hal ini dilakukan agar aplikasi hanya memanggil API satu kali saat komponen pertama kali dimuat (*mounting*), sehingga mencegah pemanggilan data berulang-ulang yang dapat membebani server.
7. **Asynchronous JavaScript:** Pengambilan data melalui API bersifat asinkron, artinya proses tersebut berjalan di latar belakang tanpa menghentikan jalannya program utama. Pengembang biasanya menggunakan `.then()` atau `async/await` untuk menangani hasil data yang diterima dari API.
8. **Loading State:** Merupakan teknik manajemen antarmuka untuk memberikan informasi kepada pengguna bahwa data sedang dalam proses pengambilan. Biasanya direpresentasikan dengan teks "Memuat data..." atau *spinner*, yang bertujuan meningkatkan aspek pengalaman pengguna (*User Experience*).
9. **Dynamic Routing:** Memungkinkan pembuatan jalur URL yang fleksibel dengan menggunakan parameter, seperti `/inventori/:id`. Parameter ini dapat ditangkap dan digunakan untuk menampilkan detail data yang spesifik berdasarkan identitas unik dari API.
10. **Reusable Component:** Tetap menjadi pilar utama di mana komponen yang telah dibuat (seperti Navbar atau Tabel) dapat digunakan kembali di berbagai halaman berbeda tanpa harus menulis ulang kodenya, membuat struktur proyek menjadi lebih modular dan efisien.

Secara keseluruhan, integrasi React Router DOM dan Fetch API pada Pertemuan 11 ini memungkinkan pengembangan sistem monitoring industri yang lebih kompleks dan terintegrasi, di mana pengguna dapat berpindah antar modul dengan cepat dan mendapatkan data real-time langsung dari database server.

D.1 Konsep Utama

Konsep utama pada praktikum ini berfokus pada transformasi aplikasi React menjadi Single Page Application (SPA) menggunakan React Router DOM. Berbeda dengan website tradisional yang melakukan *reload* halaman setiap kali berpindah menu, SPA memungkinkan navigasi antar halaman terjadi secara instan di sisi klien (*client-side routing*). Hal ini dicapai dengan memanipulasi DOM untuk mengganti komponen yang tampil sesuai dengan URL pada browser tanpa harus meminta file HTML baru dari server.

Penggunaan React Router DOM menyediakan komponen penting seperti BrowserRouter sebagai pembungkus utama, Routes sebagai kontainer jalur, dan Route untuk menentukan pasangan antara *path* URL dengan komponen yang akan ditampilkan. Salah satu keunggulan utama dalam navigasi SPA adalah penggunaan komponen Link. Berbeda dengan tag anchor (`<a>`) yang memicu *refresh* total, Link memperbarui URL dan tampilan secara mulus, menjaga state aplikasi tetap terjaga selama perpindahan halaman.

Selain navigasi, materi ini memperkenalkan Fetch API sebagai cara standar untuk melakukan integrasi data eksternal. Dengan memanfaatkan fetch di dalam hook `useEffect`, aplikasi dapat mengambil data dari server atau *Mock API* secara asinkron. Data yang diterima kemudian disimpan ke dalam *state* dan di-render ke dalam antarmuka, seperti pada tabel inventori. Konsep *loading state* juga diterapkan untuk memberikan informasi visual kepada pengguna saat proses pengambilan data sedang berlangsung, sehingga meningkatkan aspek *user experience* (UX).

Dalam proses integrasi data menggunakan Fetch API, manajemen state transisi menjadi aspek yang sangat krusial. Penggunaan *conditional rendering* yang dikombinasikan dengan state loading dan error memungkinkan aplikasi untuk memberikan umpan balik instan kepada pengguna mengenai status permintaan data. Hal ini mencegah munculnya tampilan layar kosong yang membingungkan saat data masih dalam proses transmisi dari server. Dengan menerapkan logika ini, pengembang dapat memastikan bahwa antarmuka pengguna tetap informatif dan profesional, yang merupakan standar utama dalam pengembangan perangkat lunak modern tingkat industri.

Selain itu, aspek keamanan dan pengalaman navigasi diperkuat dengan penerapan *Catch-all Route* atau halaman *NotFound (404)*. Dalam sistem navigasi React Router DOM, mendefinisikan path asteris (*) di akhir konfigurasi *Routes* berfungsi sebagai pengaman apabila pengguna mengakses URL yang tidak terdaftar dalam sistem. Halaman error ini tidak hanya berfungsi sebagai pemberitahuan kegagalan akses, tetapi juga sebagai panduan navigasi kembali ke halaman utama (Dashboard). Fitur ini memastikan bahwa pengguna tidak akan pernah terjebak dalam kondisi *dead-end* saat menggunakan aplikasi, sehingga menjaga alur kerja operasional tetap lancar dan terarah.

D.2 Keterkaitan dengan Teknik Industri

Materi React memiliki hubungan yang sangat erat dengan bidang Teknik Industri karena React banyak digunakan untuk membangun dashboard, sistem monitoring, dan aplikasi berbasis web yang menampilkan data secara real-time. Dalam dunia industri, data

produksi, stok gudang, performa mesin, dan status operator harus dapat ditampilkan dengan cepat dan mudah dipahami.

Konsep component pada React sangat sesuai dengan kebutuhan industri karena tampilan dashboard biasanya memiliki banyak bagian yang serupa dan berulang. Misalnya, pada dashboard monitoring mesin terdapat banyak card mesin dengan format yang sama. Developer cukup membuat satu component KartuMesin, lalu menggunakannya berulang kali dengan data yang berbeda untuk setiap mesin.

Props juga memiliki peran penting dalam sistem industri karena data yang ditampilkan selalu berubah sesuai kondisi lapangan. Status mesin dapat berubah dari running menjadi stop atau maintenance, jumlah produksi dapat bertambah setiap saat, dan data operator dapat berubah sesuai jadwal shift. Dengan props, perubahan data tersebut dapat langsung ditampilkan pada antarmuka.

Selain digunakan pada monitoring mesin, React juga banyak diterapkan pada sistem ERP, manajemen gudang, pelacakan stok barang, sistem quality control, serta dashboard laporan insiden. Seluruh data tersebut dapat ditampilkan dalam bentuk tabel, card, grafik, maupun notifikasi sehingga memudahkan pengguna dalam membaca informasi. React juga mendukung pengembangan dashboard berbasis IoT. Data dari sensor suhu, tekanan, kelembapan, getaran mesin, dan konsumsi energi dapat ditampilkan secara real-time ke dalam aplikasi. Dengan begitu, operator dapat mengetahui kondisi lapangan secara langsung tanpa harus melakukan pengecekan manual.

Kemampuan React dalam membuat reusable component sangat membantu proses pengembangan aplikasi industri yang besar dan kompleks. Jika perusahaan ingin menambahkan fitur baru, developer tidak perlu membuat ulang seluruh tampilan, tetapi cukup menambahkan atau memodifikasi component tertentu.

Secara keseluruhan, React membantu proses digitalisasi industri karena mampu menghasilkan aplikasi yang interaktif, responsif, mudah dipelihara, dan mampu menangani data dalam jumlah besar. Oleh karena itu, React menjadi salah satu teknologi yang banyak digunakan dalam pengembangan sistem industri modern.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

Dokumentasikan setiap langkah kerja yang Anda lakukan, beserta hasil yang diperoleh (screenshot, kode, atau output). Jawab sesuai urutan langkah di Manual Praktikum.

E.1 Langkah 1 – Setup Router

Buka file utama `src/main.jsx` (atau `index.jsx` tergantung versi Vite). Bungkus aplikasi dengan `BrowserRouter`.

```
/* ===== KODE / OUTPUT LANGKAH 1 ===== */

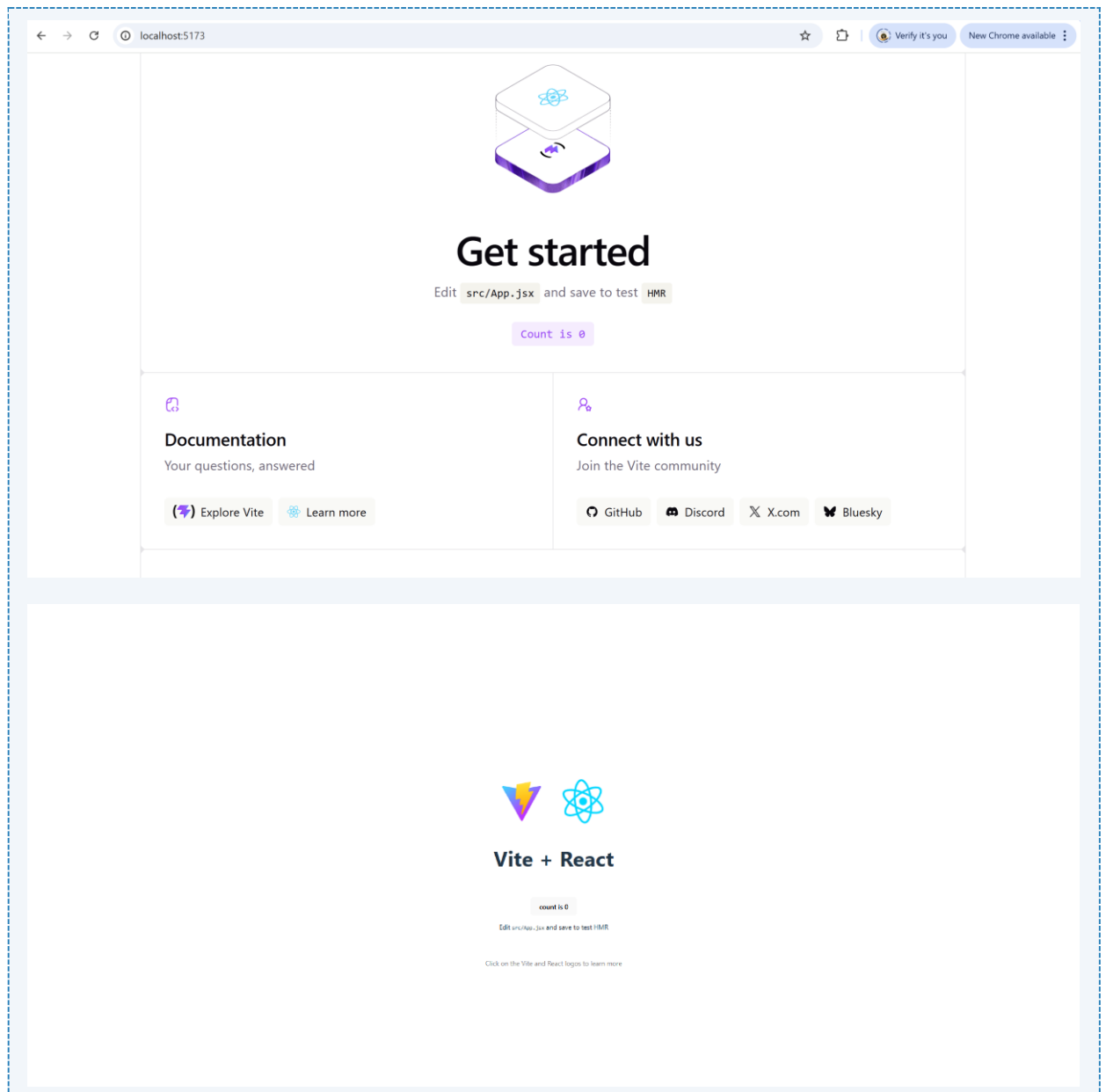
import React from 'react';
import ReactDOM from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
import App from './App.jsx';
import './index.css';
```



```
import 'bootstrap/dist/css/bootstrap.min.css';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    { /* Wrap seluruh App dengan BrowserRouter */ }
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>
);
```

Screenshot Hasil:



Gambar E.1 – Hasil Setup Router

E.2 Langkah 2 – Membuat Halaman (Pages)

Buat folder `src/Halaman`. Buat 3 file:

1. `Dashboard.jsx`
2. `Inventori.jsx`
3. `NotFound.jsx` (Halaman error).

Isi `src/Halaman/Dashboard.jsx`, Isi `src/Halaman/Inventori.jsx`, Isi `src/Halaman/NotFound.jsx`:

```
/* ===== KODE / OUTPUT LANGKAH 2 ===== */
```

3. JAWABAN KODE DASHBOARD.JSX

```
import React from 'react';
import { Link } from 'react-router-dom';

function Dashboard() {
  return (
    <div className="p-5">
      <h1>Dashboard Utama Pabrik</h1>
      <p>Selamat datang di sistem monitoring terpadu.</p>
      <Link to="/inventori" className="btn btn-primary">
        Ke Halaman Inventori →
      </Link>
    </div>
  );
}

export default Dashboard;
```

4. JAWABAN KODE INVENTORI.JSX

```
import React, { useState, useEffect } from 'react';
import { Link } from 'react-router-dom';

function Inventori() {
  const [products, setProducts] = useState([]);

  // Fetch Data saat komponen mount
  useEffect(() => {
    fetch('https://jsonplaceholder.typicode.com/posts')
      .then(res => res.json())
      .then(data => {
        // Ambil hanya 5 data pertama untuk contoh
        setProducts(data.slice(0, 5));
      })
      .catch(err => console.log(err));
  }, []);
```

```

return (
  <div className="container mt-4">
    <h1>Data Inventori Bahan Baku</h1>
    <Link to="/" className="btn btn-secondary mb-3">Kembali ke
Dashboard</Link>

    <table className="table table-striped">
      <thead>
        <tr>
          <th>ID Item</th>
          <th>Nama Bahan</th>
          <th>Status Supplier</th>
        </tr>
      </thead>
      <tbody>
        {products.map((item) => (
          <tr key={item.id}>
            <td>{item.id}</td>
            <td>{item.title}</td> {/* Menggunakan title sebagai simulasi
nama bahan */}
            <td><span className="badge bg-success">Available</span></td>
          </tr>
        ))}
      </tbody>
    </table>
  </div>
);
}

export default Inventori;

```

5. JAWABAN KODE NOTFOUND.JSX

```

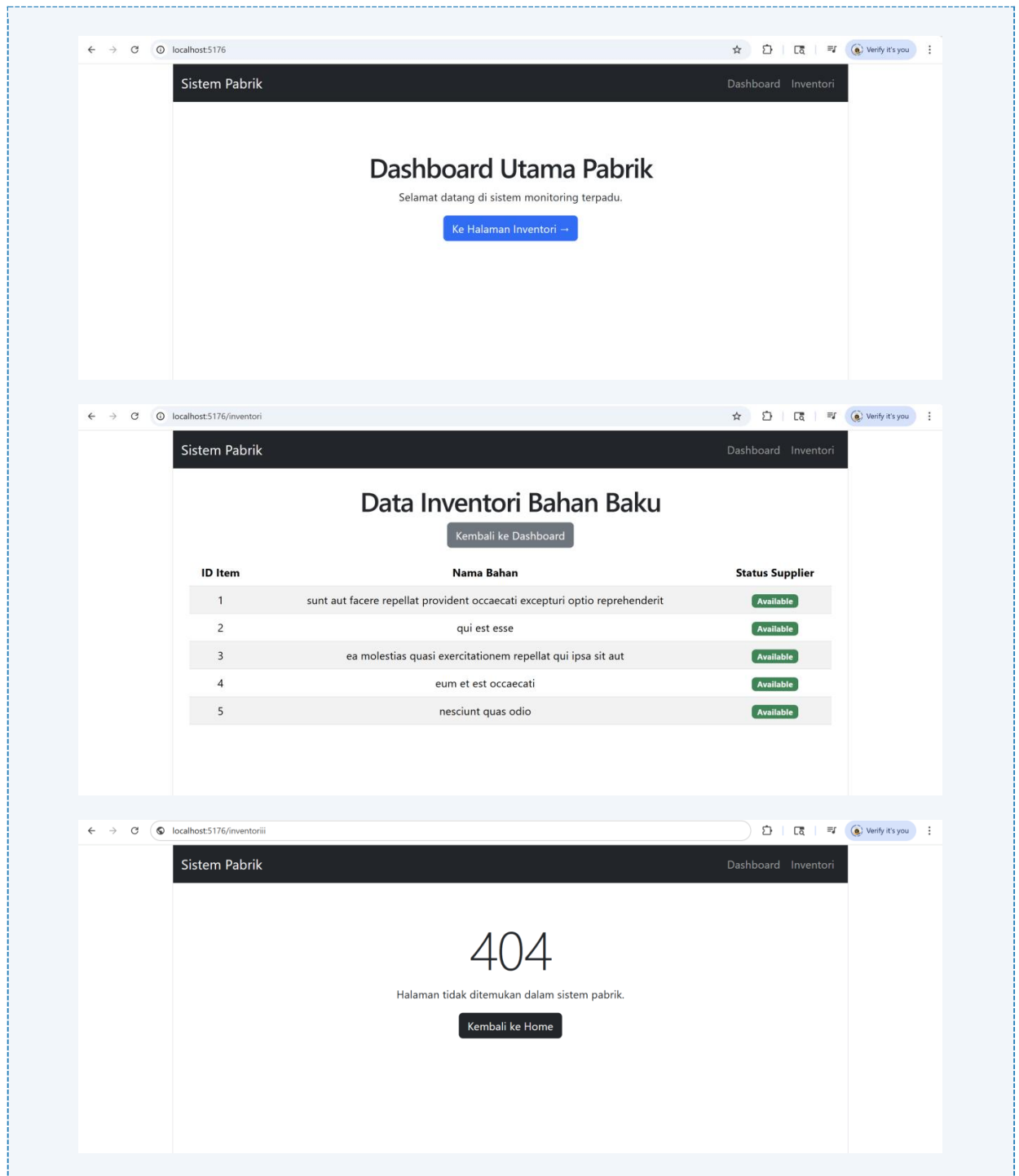
import React from 'react';
import { Link } from 'react-router-dom';

function NotFound() {
  return (
    <div className="text-center mt-5">
      <h1 className="display-1">404</h1>
      <p>Halaman tidak ditemukan dalam sistem pabrik.</p>
      <Link to="/" className="btn btn-dark">
        Kembali ke Home
      </Link>
    </div>
  );
}

```

```
);  
}  
  
export default NotFound;
```

Screenshot Hasil:



Gambar E.2 – Hasil Membuat Halaman (Pages)

E.3 Langkah 3 – Mengatur Route di App.jsx

Sekarang kita tentukan kapan komponen mana yang muncul berdasarkan URL.

```
/* ===== KODE / OUTPUT LANGKAH 3 ===== */

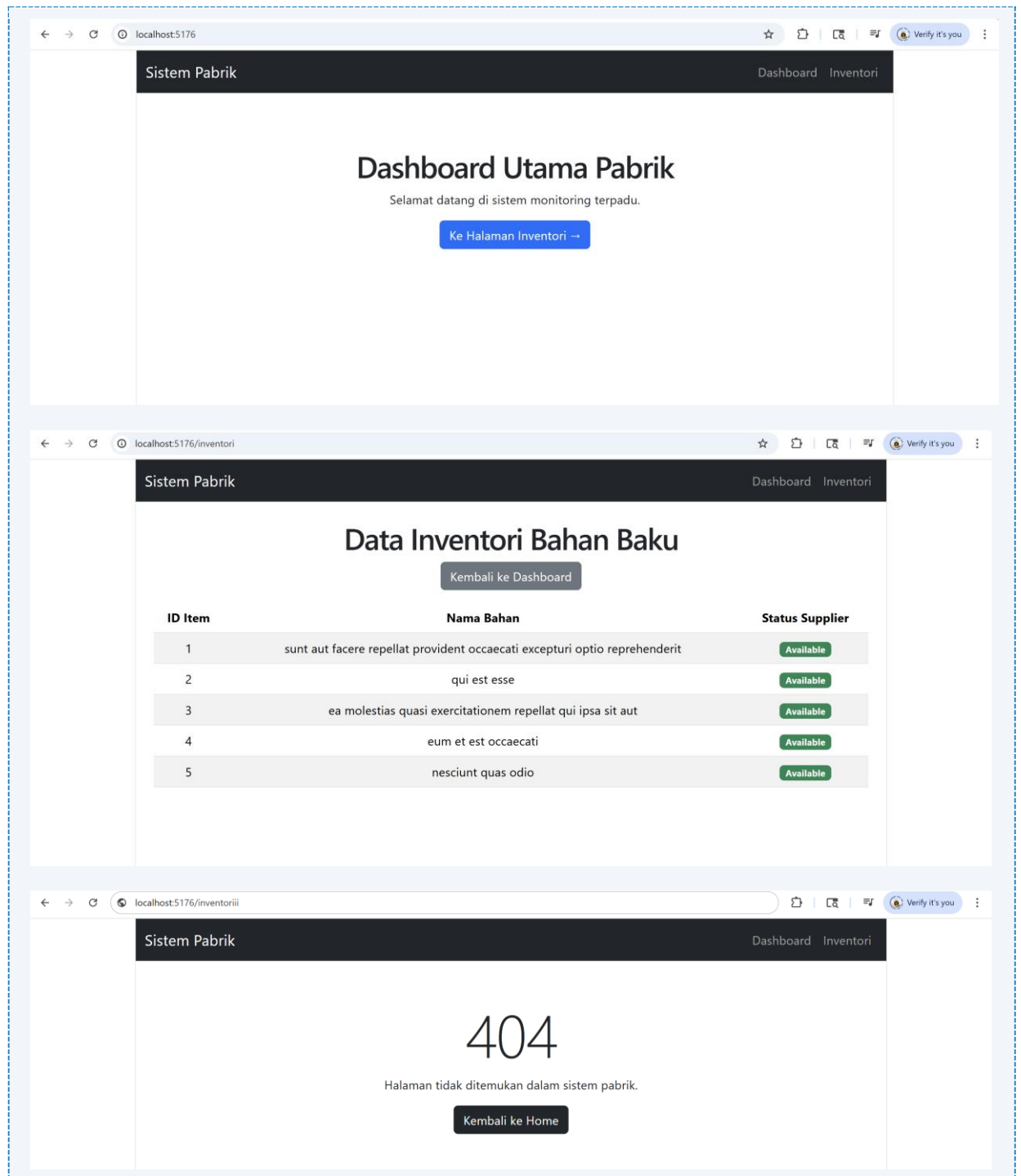
import React from 'react';
import { Routes, Route, Link } from 'react-router-dom';
import Dashboard from './Halaman/Dashboard';
import Inventori from './Halaman/Inventori';
import NotFound from './Halaman/NotFound';

// Pastikan komponen Navbar ada di LUAR komponen App
function Navbar() {
  return (
    <nav className="navbar navbar-expand-lg navbar-dark bg-dark mb-4">
      <div className="container">
        <Link className="navbar-brand" to="/">Sistem Pabrik</Link>
        <div className="navbar-nav">
          <Link className="nav-link" to="/">Dashboard</Link>
          <Link className="nav-link" to="/inventori">Inventori</Link>
        </div>
      </div>
    </nav>
  );
}

function App() {
  return (
    <div>
      <Navbar />
      <div className="container">
        <Routes>
          <Route path="/" element={ <Dashboard /> } />
          <Route path="/inventori" element={ <Inventori /> } />
          <Route path="*" element={ <NotFound /> } />
        </Routes>
      </div>
    </div>
  );
}

export default App;
```

Screenshot Hasil:



Gambar E.3 – Hasil Mengatur Route di App.jsx

F. LATIHAN DAN TUGAS MANDIRI

Jawab dan dokumentasikan seluruh latihan/tugas yang ada di bagian "Latihan/Tugas Mandiri" pada modul Anda.

F.1 Latihan 1 – Dynamic Route

Pertanyaan / Instruksi Latihan:

Tambahkan Route untuk detail item. Misal path `/inventori/:id`. Di halaman Inventori, ubah "Nama Bahan" menjadi Link ke halaman detail.

Jawaban / Kode Solusi:

1. JAWABAN KODE INVENTORI.JSX

```
import React, { useState, useEffect } from 'react';
import { Link } from 'react-router-dom';

function Inventori() {
  // State untuk menyimpan data produk
  const [products, setProducts] = useState([]);

  // LATIHAN 2: State untuk loading (boolean)
  const [loading, setLoading] = useState(true);

  // Data simulasi 6 bahan baku pabrik (Bahasa Indonesia)
  const dataBahanBaku = [
    { id: 1, nama: "Baja Karbon High Grade", status: "Available" },
    { id: 2, nama: "Aluminium Batangan", status: "Available" },
    { id: 3, nama: "Baut Industri M12", status: "Available" },
    { id: 4, nama: "Pelumas Mesin Hidrolik", status: "Available" },
    { id: 5, nama: "Lembaran Tembaga", status: "Available" },
    { id: 6, nama: "Plat Besi Antikarat", status: "Available" }
  ];

  useEffect(() => {
    // LATIHAN 2: Set loading true sebelum "mengambil" data
    setLoading(true);

    // Simulasi proses fetch data selama 1.5 detik
    const timer = setTimeout(() => {
      setProducts(dataBahanBaku);

      // LATIHAN 2: Set loading false setelah data masuk
      setLoading(false);
    }, 1500);
  }, [1500]);
}
```

```

    return () => clearTimeout(timer);
  }, []);

  return (
    <div className="container mt-4">
      <h1 className="text-center mb-4">Data Inventori Bahan Baku</h1>

      <div className="mb-3">
        <Link to="/" className="btn btn-secondary">
          Kembali ke Dashboard
        </Link>
      </div>

      <hr />

      {/* LATIHAN 2: Kondisi jika loading true, tampilkan teks memuat */}
      {loading ? (
        <div className="text-center my-5">
          <div className="spinner-border text-primary" role="status"></div>
          <h4 className="mt-3 text-muted">Memuat data...</h4>
        </div>
      ) : (
        /* Jika loading false, maka tampilkan tabel (Latihan 1 ada di dalam
sini) */
        <div className="table-responsive">
          <table className="table table-striped table-bordered shadow">
            {/* Judul tetap di tengah (text-center) */}
            <thead className="table-dark text-center">
              <tr>
                <th>ID Item</th>
                <th>Nama Bahan</th>
                <th>Status Supplier</th>
              </tr>
            </thead>
            <tbody>
              {products.map((item) => (
                <tr key={item.id} className="align-middle">
                  <td className="text-center">{item.id}</td>
                  <td className="text-start ps-4">
                    {/* LATIHAN 1: Link Dynamic Route ke detail berdasarkan
ID */}
                    <Link
                      to={`/${inventori}/${item.id}`}
                      className="text-decoration-none fw-bold text-primary"
                    >
                      {item.nama}

```



```

        </Link>
      </td>
      <td className="text-center">
        <span className="badge bg-success px-3 py-2">
          {item.status}
        </span>
      </td>
    </tr>
  </tbody>
</table>
</div>
)}
</div>
);
}

```

```
export default Inventori;
```

2. JAWABAN KODE DETAILITEM.JSX

```

import React, { useState, useEffect } from 'react';
import { useParams, Link } from 'react-router-dom';

function DetailItem() {
  const { id } = useParams();
  const [item, setItem] = useState(null);
  const [loading, setLoading] = useState(true);

  // Data ini disamakan dengan yang ada di halaman Inventori
  const dataBahanBaku = [
    { id: 1, nama: "Baja Karbon High Grade", deskripsi: "Baja kualitas tinggi untuk rangka utama mesin.", spesifikasi: "Ketebalan 10mm, Anti Karat" },
    { id: 2, nama: "Aluminium Batangan", deskripsi: "Bahan ringan untuk komponen aerodinamis.", spesifikasi: "Grade 6061, Panjang 2m" },
    { id: 3, nama: "Baut Industri M12", deskripsi: "Baut pengunci kekuatan tinggi untuk beban berat.", spesifikasi: "Baja Hitam, Ulir Kasar" },
    { id: 4, nama: "Pelumas Mesin Hidrolik", deskripsi: "Cairan pelumas untuk menjaga suhu mesin tetap stabil.", spesifikasi: "ISO VG 46, Tahan Panas" },
    { id: 5, nama: "Lembaran Tembaga", deskripsi: "Konduktor terbaik untuk jalur kelistrikan panel.", spesifikasi: "Kemurnian 99%, Tebal 0.5mm" },
    { id: 6, nama: "Plat Besi Antikarat", deskripsi: "Plat pelapis dinding mesin agar tidak mudah korosi.", spesifikasi: "Ukuran 1x2 meter, Grade A" }
  ];

  useEffect(() => {

```

```
setLoading(true);
// Cari data berdasarkan ID yang ada di URL
const temukanData = dataBahanBaku.find(produk => produk.id ===
parseInt(id));

// Loading
setTimeout(() => {
  setItem(temukanData);
  setLoading(false);
}, 200);
}, [id]);

if (loading) return <div className="container mt-5"><p>Memuat
detail...</p></div>;

if (!item) return (
  <div className="container mt-5 text-center">
    <h3>Data Tidak Ditemukan</h3>
    <Link to="/inventori" className="btn btn-secondary mt-3">Kembali</Link>
  </div>
);

return (
  <div className="container mt-5">

    <Link to="/inventori" className="text-decoration-none d-inline-block
mb-3">
      ← Kembali ke Daftar Inventori
    </Link>

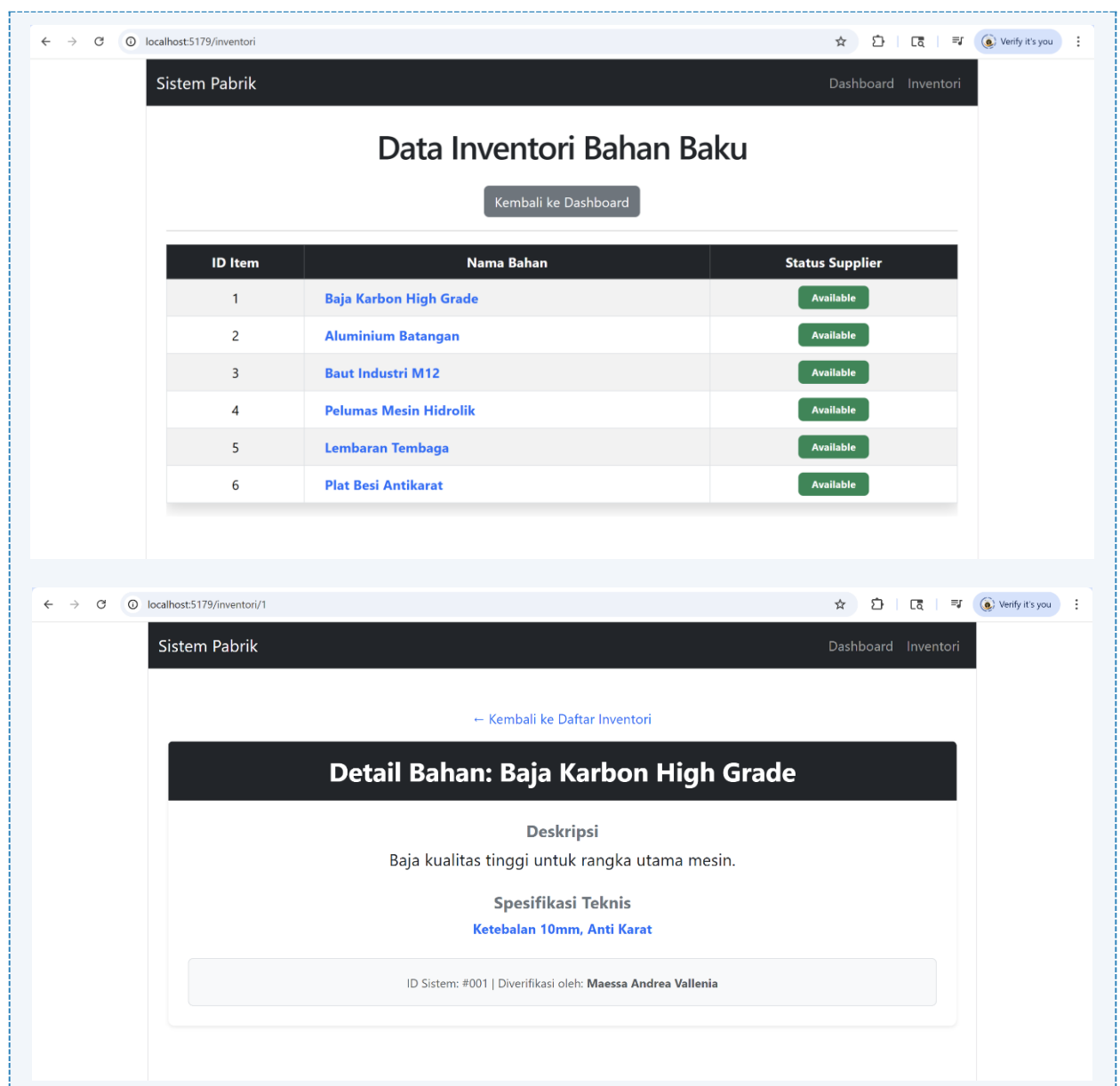
    <div className="card shadow-sm border-0 rounded-3">
      <div className="card-header bg-dark text-white py-3">
        <h2 className="m-0 fw-bold">Detail Bahan: {item.nama}</h2>
      </div>

      <div className="card-body p-4">
        <div className="mb-4">
          <h5 className="fw-bold text-secondary">Deskripsi</h5>
          <p className="fs-5">{item.deskripsi}</p>
        </div>

        <div className="mb-4">
          <h5 className="fw-bold text-secondary">Spesifikasi Teknis</h5>
          <p className="fw-bold text-primary">{item.spesifikasi}</p>
        </div>
      </div>
    </div>
  </div>
);
```

```
<div className="p-3 bg-light rounded border">
  <small className="text-muted">ID Sistem: #00{item.id} |
Diverifikasi oleh: <strong>Maessa Andrea Vallenia</strong></small>
</div>
</div>
</div>
</div>
);
}

export default DetailItem;
```



Gambar F.1 – Hasil Dynamic Route

F.2 Latihan 2 – Loading State

Pertanyaan / Instruksi Latihan:

Di halaman `Inventori.jsx`, tambahkan state `loading` (boolean). Set `loading = true` sebelum fetch, dan `false` setelah data masuk. Tampilkan teks "Memuat data..." jika loading true.

Jawaban / Kode Solusi:

```
import React, { useState, useEffect } from 'react';
import { Link } from 'react-router-dom';

function Inventori() {
  // State untuk menyimpan data produk
  const [products, setProducts] = useState([]);

  // LATIHAN 2: State untuk loading (boolean)
  const [loading, setLoading] = useState(true);

  // Data simulasi 6 bahan baku pabrik (Bahasa Indonesia)
  const dataBahanBaku = [
    { id: 1, nama: "Baja Karbon High Grade", status: "Available" },
    { id: 2, nama: "Aluminium Batangan", status: "Available" },
    { id: 3, nama: "Baut Industri M12", status: "Available" },
    { id: 4, nama: "Pelumas Mesin Hidrolik", status: "Available" },
    { id: 5, nama: "Lembaran Tembaga", status: "Available" },
    { id: 6, nama: "Plat Besi Antikarat", status: "Available" }
  ];

  useEffect(() => {
    // LATIHAN 2: Set loading true sebelum "mengambil" data
    setLoading(true);

    // Simulasi proses fetch data selama 1.5 detik
    const timer = setTimeout(() => {
      setProducts(dataBahanBaku);

      // LATIHAN 2: Set loading false setelah data masuk
      setLoading(false);
    }, 1500);

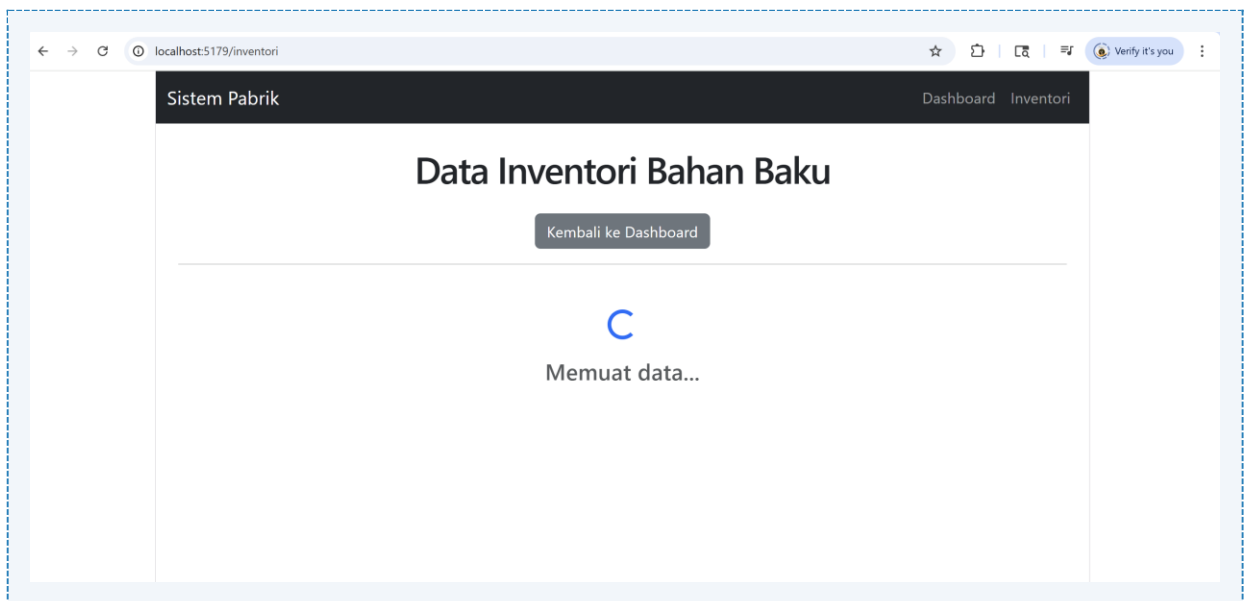
    return () => clearTimeout(timer);
  }, []);

  return (
    <div className="container mt-4">
      <h1 className="text-center mb-4">Data Inventori Bahan Baku</h1>
```



```
        </tr>
      )}}
    </tbody>
  </table>
</div>
)}
</div>
);
}

export default Inventori;
```



Gambar F.2 – Hasil Loading State

F.3 Tugas Proyek Mini – Navigasi Modular

Deskripsi proyek mini:

- Buat halaman baru bernama `LaporanKualitas.jsx`. Halaman ini menampilkan data "Cacat" (mock data array manual).
- Tambahkan link di Navbar untuk menuju halaman ini.
- Pastikan transisi antar halaman berjalan mulus tanpa reload browser.

Penjelasan pendekatan/solusi yang digunakan:

Pengerjaan tugas mandiri dan proyek mini ini dilakukan melalui tahapan yang sistematis untuk mengimplementasikan fitur navigasi dinamis dan manajemen state, dengan rincian sebagai berikut:

- B. Tahap awal dilakukan dengan mendaftarkan rute dinamis baru pada file App.jsx menggunakan path `/inventori/:id`, yang memungkinkan aplikasi untuk merender halaman detail berdasarkan identitas unik dari setiap bahan baku.
- C. Selanjutnya dilakukan modifikasi pada komponen Inventori.jsx dengan mengubah properti nama bahan dari teks statis menjadi komponen Link, sehingga pengguna dapat berinteraksi langsung untuk berpindah ke halaman detail secara spesifik.
- D. Pada tahap berikutnya diterapkan logika *Loading State* melalui penggunaan `useState` (boolean) di halaman inventori. State ini diatur menjadi `true` sebelum proses pengambilan data dimulai guna memberikan umpan balik visual kepada pengguna.
- E. Dilakukan pengaturan ulang pada fungsi `fetch` di mana state `loading` akan diubah kembali menjadi `false` segera setelah data berhasil diterima, guna memastikan sinkronisasi antara proses di latar belakang dengan tampilan antarmuka.
- F. Langkah selanjutnya adalah mengimplementasikan *conditional rendering* untuk menampilkan teks "Memuat data..." saat status `loading` aktif, sehingga mencegah tampilan kosong yang tidak informatif saat menunggu respon dari server lokal.
- G. Tahap berikutnya adalah pembuatan komponen baru bernama `LaporanKualitas.jsx` yang berfungsi menyajikan data temuan cacat produksi menggunakan *mock data array* yang disusun secara manual.
- H. Dilakukan pembaruan pada komponen Navbar dengan menambahkan menu navigasi baru menuju halaman Laporan Kualitas. Penggunaan komponen `NavLink` dipastikan untuk menjamin transisi antar halaman berjalan mulus (Single Page Application) tanpa melakukan reload pada browser.
- I. Terakhir dilakukan pengujian menyeluruh pada browser untuk memastikan rute dinamis, efek `loading`, dan navigasi modular berfungsi dengan benar sesuai ketentuan modul praktikum.

Jawaban / Kode Solusi:

5. JAWABAN KODE DASHBOARD.JSX

```
import React from 'react';
import { Link } from 'react-router-dom';

function Dashboard() {
  return (
    <div className="container mt-5">
      <div className="p-5 mb-4 bg-dark text-white rounded-4 shadow-lg position-relative overflow-hidden">
        <div className="position-relative z-1">
          <h1 className="display-4 fw-bold">Sistem Kendali Pabrik</h1>
          <p className="fs-4 text-secondary">Selamat datang kembali,
            <strong>Maessa Andrea Vallenia</strong>. Pantau produktivitas dan kualitas
            dalam satu layar.</p>
          <hr className="my-4 border-secondary" />
          <p>Status Sistem: Online & Aktif</p>
        </div>
      </div>
    </div>
  );
}
```

```

        <div className="position-absolute top-0 end-0 p-5 opacity-25"
style={{ fontSize: '15rem', transform: 'rotate(15deg)' }}>
            
        </div>
    </div>

    <div className="row g-4 mb-5">
        <div className="col-md-4">
            <div className="card border-0 shadow-sm rounded-3 h-100"
                style={{ backgroundColor: '#f8f9fa', borderLeft: '5px solid
#52b788 !important', borderLeftStyle: 'solid' }}>
                <div className="card-body p-4 border-start border-success border-
5 rounded-start">
                    <h6 className="text-muted fw-bold mb-3" style={{ fontSize:
'0.9rem', letterSpacing: '1px' }}>MESIN AKTIF</h6>
                    <h1 className="display-4 fw-bold m-0" style={{ color: '#52b788'
}}>7/10</h1>
                </div>
            </div>
        </div>

        <div className="col-md-4">
            <div className="card border-0 shadow-sm rounded-3 h-100"
                style={{ backgroundColor: '#f8f9fa' }}>
                <div className="card-body p-4 border-start border-info border-5
rounded-start" style={{ borderLeftColor: '#63b3ed !important' }}>
                    <h6 className="text-muted fw-bold mb-3" style={{ fontSize:
'0.9rem', letterSpacing: '1px' }}>OUTPUT</h6>
                    <h1 className="display-4 fw-bold m-0" style={{ color: '#63b3ed'
}}>2,000</h1>
                </div>
            </div>
        </div>

        <div className="col-md-4">
            <div className="card border-0 shadow-sm rounded-3 h-100"
                style={{ backgroundColor: '#f8f9fa' }}>
                <div className="card-body p-4 border-start border-warning border-
5 rounded-start">
                    <h6 className="text-muted fw-bold mb-3" style={{ fontSize:
'0.9rem', letterSpacing: '1px' }}>STOK KRITIS</h6>
                    <h1 className="display-4 fw-bold m-0" style={{ color: '#f6ad55'
}}>2</h1>
                </div>
            </div>
        </div>
    </div>

```



```

</div>

<h4 className="fw-bold mb-4">Navigasi Cepat</h4>
<div className="row g-4">
  <div className="col-md-6">
    <Link to="/inventori" className="text-decoration-none">
      <div className="card h-100 border-0 shadow rounded-4 p-4 text-center hover-card"
        style={{ transition: '0.3s', backgroundColor: '#f8f9fa' }}>
        <div className="display-4 mb-3">📁</div>
        <h3 className="fw-bold text-dark">Manajemen Inventori</h3>
        <p className="text-muted">Cek ketersediaan bahan baku dan lokasi penyimpanan gudang.</p>
        <span className="btn btn-primary rounded-pill px-4">Buka Inventori</span>
      </div>
    </Link>
  </div>
  <div className="col-md-6">
    <Link to="/laporan" className="text-decoration-none">
      <div className="card h-100 border-0 shadow rounded-4 p-4 text-center hover-card"
        style={{ transition: '0.3s', backgroundColor: '#f8f9fa' }}>
        <div className="display-4 mb-3">📄</div>
        <h3 className="fw-bold text-dark">Laporan Kualitas</h3>
        <p className="text-muted">Pantau data cacat produksi dan severity temuan pada mesin.</p>
        <span className="btn btn-danger rounded-pill px-4">Buka Laporan</span>
      </div>
    </Link>
  </div>
</div>

<footer className="mt-5 py-4 text-center text-muted">
  &copy; Sistem Informasi Pabrik | Dikelola oleh <strong>Maessa Andrea Vallenia</strong>
</footer>

<style>{`
  .hover-card:hover {
    transform: translateY(-10px);
    background-color: #ffffff !important;
    box-shadow: 0 1rem 3rem rgba(0,0,0,.175) !important;
  }
`}</style>

```

```
</div>
```

```
);
```

```
}
```

```
export default Dashboard;
```

6. JAWABAN KODE DETAILINVENTORI.JSX

```
import React, { useState, useEffect } from 'react';
```

```
import { useParams, Link } from 'react-router-dom';
```

```
function DetailItem() {
```

```
  const { id } = useParams();
```

```
  const [item, setItem] = useState(null);
```

```
  const [loading, setLoading] = useState(true);
```

```
  const dataBahanBaku = [
```

```
    { id: 1, nama: "Baja Karbon High Grade", deskripsi: "Baja karbon tinggi yang digunakan untuk konstruksi alat berat dan rangka mesin.", spesifikasi: "Tebal 10mm, ASTM A36", supplier: "PT. Krakatau Steel" },
```

```
    { id: 2, nama: "Aluminium Batangan", deskripsi: "Batangan aluminium murni untuk komponen mesin ringan.", spesifikasi: "Alloy 6061, Diameter 5cm", supplier: "PT. Inalum" },
```

```
    { id: 3, nama: "Baut Industri M12", deskripsi: "Baut pengunci standar industri dengan kekuatan tarik tinggi.", spesifikasi: "Grade 8.8, Zinc Plated", supplier: "Mitra Baut" },
```

```
    { id: 4, nama: "Pelumas Mesin Hidrolik", deskripsi: "Cairan pelumas khusus untuk menjaga suhu dan gesekan mesin tetap stabil.", spesifikasi: "ISO VG 46, Tahan Panas", supplier: "Pertamina Lubricants" },
```

```
    { id: 5, nama: "Lembaran Tembaga", deskripsi: "Lembaran konduktor tembaga untuk instalasi kelistrikan pabrik.", spesifikasi: "Murni 99%, Tebal 0.5mm", supplier: "Tembaga Jaya" }
  ];
```

```
  useEffect(() => {
```

```
    setLoading(true);
```

```
    // Cari data di array dataBahanBaku yang ID-nya cocok dengan ID di URL
```

```
    const dataDitemukan = dataBahanBaku.find(produk => produk.id === parseInt(id));
```

```
    setTimeout(() => {
```

```
      setItem(dataDitemukan);
```

```
      setLoading(false);
```

```
    }, 300);
```

```
  }, [id]);
```

```
  if (loading) return (
```

```
    <div className="container mt-5 text-center">
```

```

        <div className="spinner-border text-primary"></div>
        <p className="mt-2">Sedang mengambil detail bahan Maessa...</p>
    </div>
    );

    // Jika ID di URL ngasal dan tidak ada di data kita
    if (!item) return (
        <div className="container mt-5 text-center">
            <h2 className="text-danger">Waduh! Data Tidak Ada</h2>
            <Link to="/inventori" className="btn btn-primary mt-3">Kembali ke
Inventori</Link>
        </div>
    );

    return (
        <div className="container mt-5">
            <div className="card border-0 shadow-lg rounded-4 overflow-hidden">
                <div className="card-header bg-primary text-white p-4">
                    <Link to="/inventori" className="text-white text-decoration-none
mb-3 d-inline-block small">
                        ← Kembali ke Inventori
                    </Link>
                    <h2 className="fw-bold m-0">{item.nama}</h2>
                    <span className="badge bg-light text-primary mt-2">SKU: PBRK-
00{item.id}</span>
                </div>

                <div className="card-body p-4 bg-white">
                    <div className="row">
                        <div className="col-md-7">
                            <h3 className="fw-bold text-secondary border-bottom pb-
2">Informasi Produk</h3>
                            <p className="fs-5 text-dark" style={{ lineHeight: '1.6'
}}>{item.deskripsi}</p>

                            <h3 className="fw-bold text-secondary border-bottom pb-2 mt-
4">Spesifikasi Teknis</h3>
                            <p className="fw-semibold text-muted">{item.spesifikasi}</p>
                        </div>

                        <div className="col-md-5">
                            <div className="p-4 rounded-4" style={{ backgroundColor:
'#f8f9fa', border: '1px dashed #cbd5e0' }}>
                                <h6 className="text-muted fw-bold small mb-3">DATA
SUPPLIER</h6>
                                <div className="d-flex align-items-center mb-2">

```

```

        <span className="me-2">🏠</span>
        <p className="fw-bold m-0">{item.supplier}</p>
      </div>
      <div className="alert alert-success py-2 px-3 small border-0
mt-3">
        <input checked="" type="checkbox" /> Supplier Terverifikasi
      </div>
    </div>
  </div>
  <div>
    <div className="card-footer bg-light text-center py-3 border-0">
      &copy; Sistem Informasi Pabrik | Dikelola oleh <strong>Maessa
Andrea Vallenia</strong>
    </div>
  </div>
</div>
);
}

```

```
export default DetailItem;
```

7. JAWABAN KODE INVENTORI.JSX

```

import React, { useState, useEffect } from 'react';
import { Link, NavLink } from 'react-router-dom';

function Inventori() {
  const [products, setProducts] = useState([]);
  const [loading, setLoading] = useState(true);

  // Data Inventori
  const dataBahanBaku = [
    { id: 1, nama: "Baja Karbon High Grade", status: "Available", stok: "500 kg", lokasi: "Gudang A" },
    { id: 2, nama: "Aluminium Batangan", status: "Available", stok: "250 unit", lokasi: "Gudang B" },
    { id: 3, nama: "Baut Industri M12", status: "Low Stock", stok: "50 box", lokasi: "Gudang A" },
    { id: 4, nama: "Pelumas Mesin Hidrolik", status: "Available", stok: "100 liter", lokasi: "Gudang C" },
    { id: 5, nama: "Lembaran Tembaga", status: "Out of Stock", stok: "0 unit", lokasi: "Gudang B" },
    { id: 6, nama: "Plat Besi Antikarat", status: "Available", stok: "150 lembar", lokasi: "Gudang A" },
    { id: 7, nama: "Kabel Tembaga Industri", status: "Available", stok: "300 meter", lokasi: "Gudang C" },
  ];

```

```

    { id: 8, nama: "Cat Anti Korosi Blue", status: "Low Stock", stok: "20
galon", lokasi: "Gudang C" },
    { id: 9, nama: "Pipa Stainless Steel", status: "Available", stok: "80
batang", lokasi: "Gudang B" },
    { id: 10, nama: "Gas Argon Welding", status: "Available", stok: "15
tabung", lokasi: "Gudang A" }
  ];

  useEffect(() => {
    setLoading(true);
    const timer = setTimeout(() => {
      setProducts(dataBahanBaku);
      setLoading(false);
    }, 200);
    return () => clearTimeout(timer);
  }, []);

  return (
    <div className="container mt-4">
      <div className="d-flex justify-content-between align-items-center mb-
4">
        <div>
          <h1 className="fw-bold m-0"><span className="me-3"></span>📁 Data
Stock Inventori</h1>
          <p className="text-muted ms-4 ps-2"><strong>Operator: Maessa Andrea
Vallenia (23051430044)</strong></p>
        </div>
        <div className="btn-group shadow-sm">
          <Link to="/" className="btn btn-outline-primary">
            Dashboard
          </Link>
          <Link to="/laporan" className="btn btn-outline-danger">
            Laporan Kualitas
          </Link>
        </div>
      </div>
      <div className="card border-0 shadow-sm rounded-4 overflow-hidden">
        <div className="card-header bg-dark py-3">
          <h5 className="text-white m-0">Stock Monitoring</h5>
        </div>
        {loading ? (
          <div className="text-center py-5">
            <div className="spinner-border text-primary" role="status"></div>
            <h5 className="mt-3 text-muted">Sinkronisasi data Maessa...</h5>

```

```

    </div>
  ) : (
    <div className="table-responsive">
      <table className="table table-hover align-middle mb-0">
        <thead className="table-light">
          <tr>
            <th className="ps-4" style={{ width: '80px' }}>ID</th>
            <th>Nama Bahan Baku</th>
            <th>Lokasi Penyimpanan</th>
            <th>Stok</th>
            <th className="text-end pe-4">Status</th>
          </tr>
        </thead>
        <tbody>
          {products.map((item) => (
            <tr key={item.id}>
              <td className="ps-4 text-muted">#{item.id}</td>
              <td>
                <Link
                  to={`/inventori/${item.id}`}
                  className="text-decoration-none fw-bold text-primary"
                >
                  {item.nama}
                </Link>
              </td>
              <td><i className="bi bi-geo-alt me-1"></i>{item.lokasi}</td>
              <td className="fw-semibold">{item.stok}</td>
              <td className="text-end pe-4">
                <span className={`badge rounded-pill px-3 py-2 ${
                  item.status === 'Available' ? 'bg-success-subtle
text-success border border-success' :
                  item.status === 'Low Stock' ? 'bg-warning-subtle
text-warning-emphasis border border-warning' :
                  'bg-danger-subtle text-danger border border-danger'
                }`}>
                  • {item.status}
                </span>
              </td>
            </tr>
          )))
        </tbody>
      </table>
    </div>
  )}
</div>

```

```

    <footer className="mt-4 text-center text-muted small pb-4">
      &copy; Sistem Informasi Pabrik | Dikelola oleh <strong>Maessa Andrea
Vallenia</strong>
    </footer>
  </div>
);
}

export default Inventori;

```

8. JAWABAN KODE LAPORANKUALITAS.JSX

```

import React from 'react';
import { Link } from 'react-router-dom';

function LaporanKualitas() {
  // Data sesuai checklist
  const dataCacat = [
    { id: 1, tanggal: "2024-05-01", mesin: "CNC-Alpha", jenisCacat: "Goresan
Permukaan", jumlah: "12 unit", status: "Reported" },
    { id: 2, tanggal: "2024-05-01", mesin: "Lathe-Beta", jenisCacat: "Dimensi
Tidak Sesuai", jumlah: "5 unit", status: "In Review" },
    { id: 3, tanggal: "2024-05-02", mesin: "Press-Gamma", jenisCacat: "Retak
Halus", jumlah: "8 unit", status: "Reported" },
    { id: 4, tanggal: "2024-05-02", mesin: "Welding-01", jenisCacat:
"Penyambungan Kurang", jumlah: "3 unit", status: "Critical" },
    { id: 5, tanggal: "2024-05-03", mesin: "CNC-Alpha", jenisCacat: "Overheat
Burn", jumlah: "2 unit", status: "Critical" },
    { id: 6, tanggal: "2024-05-03", mesin: "Drill-X", jenisCacat: "Lubang
Melenceng", jumlah: "15 unit", status: "Reported" }
  ];

  return (
    <div className="container mt-4">
      <div className="d-flex justify-content-between align-items-start mb-4">
        <div>
          <h1 className="fw-bold m-0"><span className="me-3"></span> 📄 Data
Laporan Kualitas</h1>
          <p className="text-muted ms-4 ps-2"><strong>Operator: Maessa Andrea
Vallenia (23051430044)</strong></p>
        </div>

        <div className="btn-group shadow-sm border rounded">
          <Link to="/" className="btn btn-white border-end py-2 px-3 text-
primary d-flex align-items-center">
            Dashboard
          </Link>
        </div>
      </div>

```

```

        <Link to="/inventori" className="btn btn-white py-2 px-3 text-
danger d-flex align-items-center">
            Data Inventori
        </Link>
    </div>
</div>

<div className="card border-0 shadow-sm rounded-3 overflow-hidden">
    <div className="card-header bg-dark text-white text-center py-3">
        <h5 className="m-0 fw-bold">Quality Control Logs</h5>
    </div>

    <div className="table-responsive">
        <table className="table table-hover align-middle mb-0 text-center">
            <thead className="table-light text-secondary border-bottom">
                <tr>
                    <th style={{ width: '80px' }}>ID</th>
                    <th>Tanggal Laporan</th>
                    <th>Mesin Terkait</th>
                    <th>Jenis Cacat</th>
                    <th>Jumlah</th>
                    <th>Status</th>
                </tr>
            </thead>
            <tbody>
                {dataCacat.map((item) => (
                    <tr key={item.id} className="border-bottom">
                        <td className="text-muted">#{item.id}</td>
                        <td className="fw-bold text-primary">{item.tanggal}</td>
                        <td className="text-dark">{item.mesin}</td>
                        <td className="text-dark">{item.jenisCacat}</td>
                        <td className="fw-bold text-dark">{item.jumlah}</td>
                        <td>
                            {/* Badge Status Warna-warni sesuai Gambar */}
                            <span
                                className="badge rounded-pill px-3 py-2"
                                style={{
                                    backgroundColor:
                                        item.status === 'Critical' ? '#fdecea' :
                                        item.status === 'In Review' ? '#fff9db' :
                                        '#e8f5e9',
                                    color:
                                        item.status === 'Critical' ? '#d32f2f' :
                                        item.status === 'In Review' ? '#f59f00' :
                                        '#2e7d32',
                                    border: `1px solid ${

```



```

        item.status === 'Critical' ? '#ffcdd2' :
        item.status === 'In Review' ? '#fff3bf' : '#c8e6c9'
      }`,
      fontWeight: '600',
      fontSize: '0.85rem'
    })
  }
  >
  • {item.status}
</span>
</td>
</tr>
))}
</tbody>
</table>
</div>
</div>

<footer className="mt-4 text-center text-muted small">
  &copy; Sistem Informasi Pabrik | Dikelola oleh <strong>Maessa Andrea
  Vallenia</strong>
</footer>
</div>
);
}

export default LaporanKualitas;

```

9. JAWABAN KODE NOTFOUND.JSX

```

import React from 'react';
import { Link } from 'react-router-dom';

function NotFound() {
  return (
    <div className="container text-center" style={{ marginTop: '100px' }}>
      <h1 style={{ fontSize: '100px', color: '#dc3545' }}>404</h1>
      <h2>Waduh! Halaman Tidak Ditemukan</h2>
      <p className="text-muted">Alamat yang kamu masukkan salah atau halaman
      sudah pindah.</p>
      <Link to="/" className="btn btn-primary mt-3">
        Kembali ke Beranda
      </Link>
    </div>
  );
}

export default NotFound;

```

10. JAWABAN KODE APP.JSX

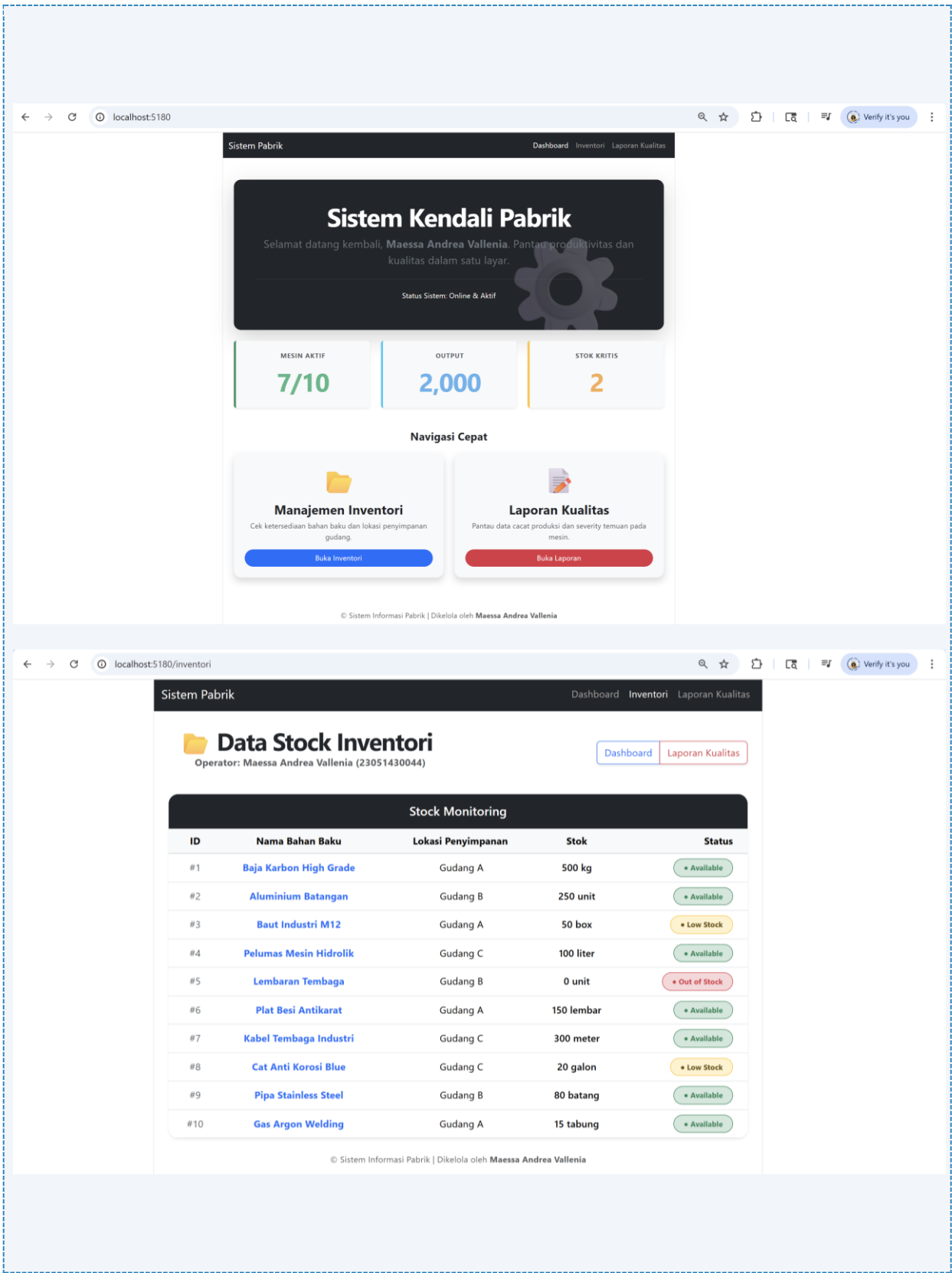
```
import React from 'react';
import { Routes, Route, NavLink, Link } from 'react-router-dom';
import Dashboard from './Halaman/Dashboard';
import Inventori from './Halaman/Inventori';
import DetailItem from './Halaman/DetailItem';
import LaporanKualitas from './Halaman/LaporanKualitas';
import NotFound from './Halaman/NotFound';

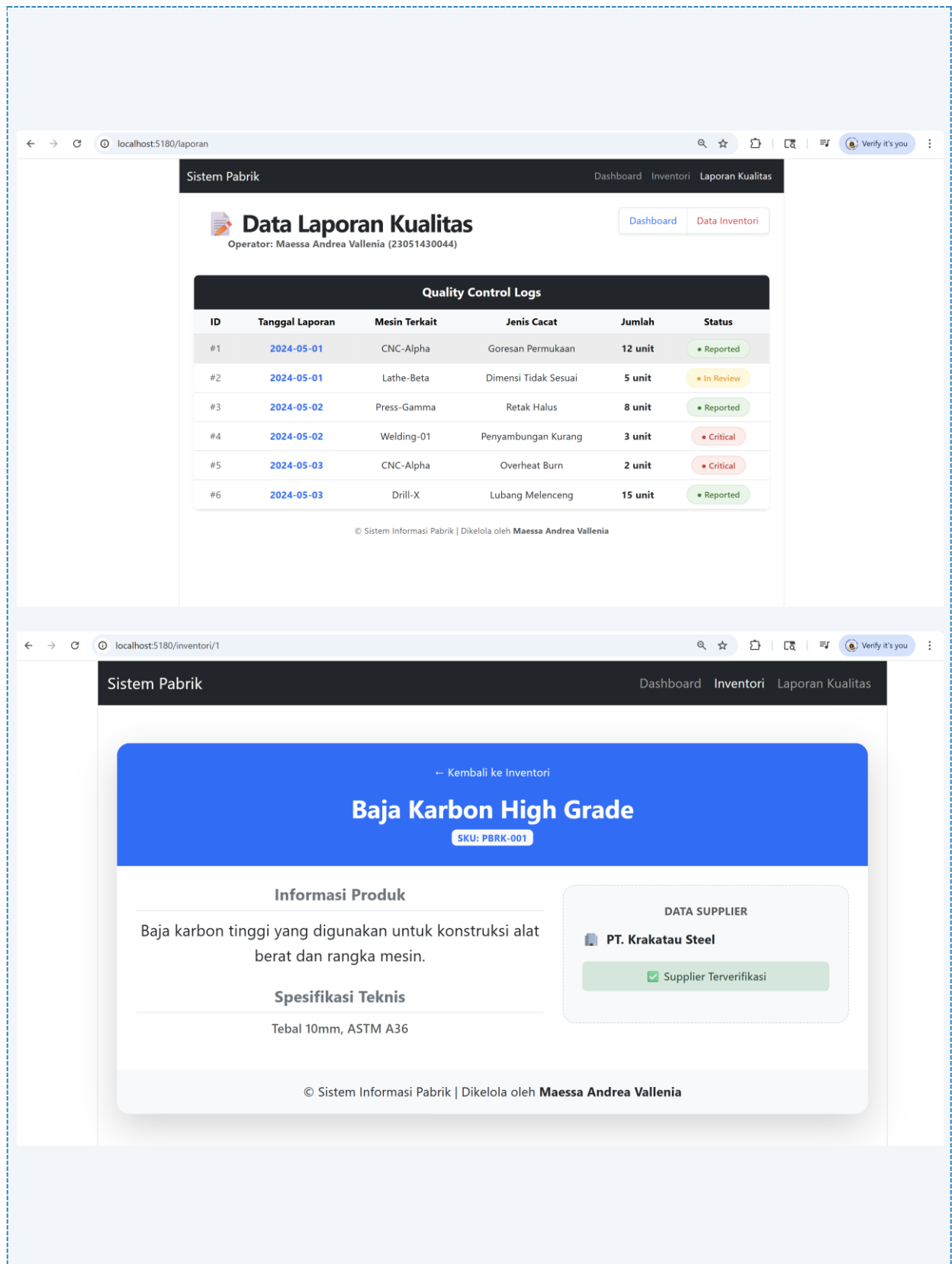
function App() {
  return (
    <div>
      { /* NAVBAR */ }
      <nav className="navbar navbar-expand-lg navbar-dark bg-dark mb-4">
        <div className="container">
          <Link className="navbar-brand" to="/">Sistem Pabrik</Link>

          <div className="navbar-nav">
            <NavLink className="nav-link" to="/" end>Dashboard</NavLink>
            <NavLink className="nav-link" to="/inventori">Inventori</NavLink>
            <NavLink className="nav-link" to="/laporan">Laporan
Kualitas</NavLink>
          </div>
        </div>
      </nav>

      <div className="container">
        <Routes>
          <Route path="/" element={ <Dashboard /> } />
          <Route path="/inventori" element={ <Inventori /> } />
          <Route path="/inventori/:id" element={ <DetailItem /> } />
          { /* Route baru sesuai tugas no 3 */ }
          <Route path="/laporan" element={ <LaporanKualitas /> } />
          { /* Handle 404 */ }
          <Route path="*" element={ <NotFound /> } />
        </Routes>
      </div>
    </div>
  );
}

export default App;
```





Gambar F.3 – Hasil Proyek Mini Navigasi Modular

G. PEMBAHASAN

Pada praktikum Pertemuan 11, mahasiswa mempelajari konsep pengembangan aplikasi modern berbasis Single Page Application (SPA) menggunakan library react-router-dom dan integrasi data eksternal melalui Fetch API. Praktikum ini merupakan tahapan krusial dalam mensimulasikan sistem industri yang kompleks, di mana aplikasi tidak lagi hanya memiliki satu tampilan tunggal, melainkan terdiri dari berbagai modul navigasi yang saling terhubung secara dinamis dan responsif.

Implementasi React Router DOM memungkinkan transisi antar halaman, seperti perpindahan dari Dashboard ke Inventori, berjalan secara instan tanpa melakukan *reload* browser. Hal ini dicapai dengan penggunaan komponen `<Link>` dan `<NavLink>` yang menggantikan tag `<a>` konvensional. Pendekatan ini secara signifikan meningkatkan pengalaman pengguna (*User Experience*) karena menghilangkan efek *flicker* dan menjaga *state* aplikasi tetap terjaga selama navigasi berlangsung.

Penggunaan Hook `useEffect` dalam integrasi Fetch API menjadi inti dari pengelolaan data dinamis pada halaman Inventori. Mahasiswa mempraktikkan cara mengambil data dari sumber eksternal (atau JSON lokal) yang kemudian disimpan ke dalam *state* lokal. Proses ini menunjukkan bagaimana aliran data di industri bekerja, di mana antarmuka pengguna (Frontend) berkomunikasi secara aktif dengan penyedia data (Backend/API) untuk menyajikan informasi stok bahan baku secara *real-time*.

Pada Latihan 1, mahasiswa berhasil mengimplementasikan Dynamic Routing dengan path `/inventori/:id`. Penggunaan rute dinamis ini membuktikan efisiensi kode, di mana satu halaman detail dapat digunakan secara berulang untuk menampilkan berbagai informasi item yang berbeda berdasarkan parameter ID di URL. Pada Latihan 2, aspek *usability* diperkuat dengan penerapan Loading State. Dengan memberikan umpan balik visual "Memuat data...", aplikasi menjadi lebih informatif dan profesional, menghindari kebingungan pengguna saat terjadi jeda waktu dalam proses pengambilan data.

Pada Tugas Proyek Mini, mahasiswa sukses membangun sistem navigasi modular dengan menambahkan halaman Laporan Kualitas. Integrasi halaman ini ke dalam Navbar menggunakan teknik navigasi SPA memastikan bahwa seluruh ekosistem monitoring pabrik mulai dari pantauan stok hingga laporan cacat produksi terintegrasi secara utuh dalam satu *Single Page Application*.

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum secara menyeluruh menunjukkan bahwa implementasi program yang telah dilakukan sudah sangat sesuai dengan teori Single Page Application (SPA), penggunaan React Router DOM, serta kaidah Asynchronous JavaScript melalui Fetch API.

Struktur navigasi telah dipisahkan dengan benar menggunakan komponen BrowserRouter, Routes, dan Route yang menunjukkan pemahaman mendalam mengenai konsep Client-Side Routing. Sesuai dengan teori SPA, perpindahan antar halaman tidak lagi memicu permintaan dokumen HTML baru ke server, melainkan dikelola sepenuhnya oleh library router di sisi klien. Pendekatan ini membuktikan bahwa mahasiswa telah memahami cara kerja navigasi modern di mana *state* aplikasi tetap terjaga dan pengalaman pengguna menjadi lebih mulus karena hilangnya jeda waktu pemuatan ulang halaman (*full page reload*).

Konsep Dynamic Routing berhasil diterapkan secara optimal melalui penggunaan path parameter: id pada rute `/inventori/:id`. Hal ini selaras dengan teori pengembangan web profesional yang mengedepankan efisiensi melalui *template* halaman yang bersifat generik namun mampu menampilkan data spesifik. Dengan memanfaatkan *hook* `useParams`, mahasiswa dapat mengekstraksi identitas unik dari URL untuk melakukan filter data secara akurat, yang mencerminkan pemahaman terhadap arsitektur aplikasi berbasis data (*data-driven application*).

Penggunaan `useEffect` Hook untuk menjalankan Fetch API telah memenuhi standar teori pemrograman asinkron dalam React. Dalam lingkungan industri, data jarang bersifat statis dan biasanya berasal dari API eksternal. Implementasi pengambilan data dalam `useEffect` dengan *dependency array* kosong memastikan bahwa proses *fetching* hanya dilakukan satu kali saat komponen pertama kali dimuat (*mounting*).

Penerapan Loading State pada halaman Inventori menunjukkan bahwa aspek *User Experience (UX)* telah dipertimbangkan secara matang sesuai teori *Asynchronous UI*. Mengingat pengambilan data memerlukan waktu tunggu, penyediaan informasi "Memuat data..." berfungsi untuk mencegah kebingungan pengguna terhadap kondisi aplikasi. Implementasi ini mencerminkan prinsip *Feedback* dalam interaksi manusia dan komputer, di mana sistem harus selalu menginformasikan status proses yang sedang berlangsung kepada pengguna.

Secara keseluruhan, sistem navigasi dan integrasi data yang telah dibangun mampu mencapai target kompetensi sebagai berikut:

1. Berhasil membangun arsitektur aplikasi Single Page Application (SPA) yang memungkinkan navigasi cepat dan responsif tanpa interupsi *reload* browser.
2. Mampu mengelola rute dinamis yang memungkinkan akses menuju detail informasi produk secara spesifik melalui parameter URL.
3. Mengimplementasikan pengambilan data dari sumber eksternal melalui Fetch API secara asinkron yang terintegrasi di dalam siklus hidup komponen React.
4. Menerapkan manajemen *state* untuk menangani kondisi pemuatan data (*loading state*) guna menjaga kualitas pengalaman pengguna saat proses pengambilan data berlangsung.
5. Menghasilkan struktur navigasi modular yang memudahkan pengembangan fitur di masa depan, seperti penambahan halaman Laporan Kualitas tanpa merusak alur aplikasi utama.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Data Inventori tidak muncul atau kolom Nama Bahan kosong saat melakukan <i>mapping</i> data dari hasil <i>fetch</i> .	Melakukan sinkronisasi antara kunci (<i>key</i>) pada file JSON dengan variabel yang dipanggil di JSX (contoh: mengganti <i>item.title</i> menjadi <i>item.nama</i> sesuai struktur data lokal).

G.3 Kaitan dengan Penerapan di Industri

Materi yang dipelajari pada praktikum Pertemuan 11 mengenai React Routing dan Fetch API memiliki relevansi yang sangat krusial terhadap implementasi teknologi di dunia industri, khususnya dalam pengembangan ekosistem Smart Factory dan Digital Twin. Di era Industri 4.0, sistem pemantauan tidak lagi bersifat statis; informasi harus mengalir secara dinamis dari berbagai sumber data ke dalam satu platform terpadu. Penggunaan Single Page Application (SPA) menjadi standar industri untuk membangun dashboard operasional yang memerlukan kecepatan akses tinggi dan navigasi yang intuitif tanpa interupsi *reload* halaman, sehingga pengambil keputusan dapat berpindah antar modul seperti dari pantauan stok ke laporan kualitas dengan instan dan efisien.

Penerapan Fetch API dan Asynchronous Programming sangat relevan dengan integrasi data di rantai produksi. Dalam lingkungan manufaktur modern, data dari mesin, sensor IoT, dan sistem pergudangan sering kali dikelola oleh server yang berbeda. Kemampuan untuk mengambil dan menyajikan data tersebut secara asinkron memungkinkan sistem monitoring untuk memperbarui informasi stok bahan baku atau status produksi secara *real-time* tanpa mengganggu stabilitas antarmuka pengguna secara keseluruhan. Hal ini mendukung terciptanya sistem yang responsif dan akurat dalam menyajikan kondisi lapangan terkini bagi para supervisor produksi.

Penggunaan Dynamic Routing mencerminkan kebutuhan industri dalam mengelola aset berskala besar. Dalam sebuah pabrik dengan ribuan komponen inventori, tidaklah efisien untuk membuat halaman manual untuk setiap item. Dengan rute dinamis, satu struktur halaman detail dapat digunakan untuk menampilkan ribuan data aset hanya berdasarkan parameter ID yang unik. Hal ini mempermudah teknisi pemeliharaan dalam melakukan *troubleshooting* atau pengecekan spesifikasi material secara spesifik hanya dengan memindai kode unik atau mengklik tautan di dashboard utama, yang secara langsung meningkatkan efisiensi waktu kerja di lapangan.

Secara strategis, pemahaman mengenai navigasi modular dan integrasi API memberikan bekal yang kuat bagi mahasiswa Teknik Industri untuk merancang arsitektur sistem informasi yang skalabel. Penguasaan terhadap cara kerja aplikasi web yang terintegrasi dengan data eksternal memungkinkan mahasiswa untuk berkontribusi dalam perancangan sistem kontrol yang tidak hanya informatif, tetapi juga mampu beradaptasi dengan kompleksitas data industri yang sangat besar. Kompetensi dalam membangun aplikasi yang cepat, *clean code*, dan terhubung dengan data *real-time* menjadi nilai tambah yang signifikan di pasar kerja, terutama dalam posisi-posisi strategis seperti *System Analyst*, *Production Planner*, hingga *Digital Transformation Specialist* di perusahaan manufaktur global.

H. KESIMPULAN

Tuliskan 3–5 poin kesimpulan yang merangkum hal-hal yang dipelajari dan dicapai pada praktikum modul ini. Kesimpulan harus spesifik, bukan bersifat umum.

1	Mahasiswa mampu memahami dan mengimplementasikan arsitektur Single Page Application (SPA) menggunakan react-router-dom, yang memungkinkan navigasi antar modul pabrik berjalan secara instan dan mulus tanpa proses <i>reload</i> browser.
2	Mahasiswa berhasil menguasai teknik Dynamic Routing melalui penggunaan <i>path parameters</i> (:id), sehingga sistem mampu merender halaman detail produk secara spesifik dan efisien hanya dengan menggunakan satu struktur komponen yang sama.
3	Mahasiswa mampu menerapkan Fetch API di dalam <i>hook</i> useEffect untuk melakukan pengambilan data asinkron dari sumber eksternal, yang merepresentasikan proses integrasi data <i>real-time</i> antara <i>frontend</i> dengan database industri.
4	Mahasiswa berhasil mengimplementasikan manajemen Loading State sebagai bentuk kendali antarmuka pengguna, guna memberikan informasi visual yang jelas selama proses transisi pengambilan data berlangsung.
5	Mahasiswa memahami cara membangun Navigasi Modular yang terorganisir melalui komponen Navbar dan Route penangkap (<i>catch-all route</i> 404), guna menciptakan ekosistem aplikasi monitoring yang skalabel dan profesional sesuai standar kebutuhan industri digital.

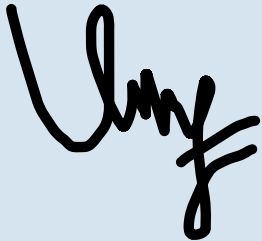
I. DAFTAR PUSTAKA

Tuliskan semua sumber referensi yang digunakan dalam format APA 7th Edition. Minimal 3 referensi yang relevan (termasuk Manual Praktikum dan sumber online resmi seperti MDN, Reactjs.org, dll).

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Meta Platforms, Inc. (2024). React Documentation: Synchronizing with Effects. Retrieved from https://react.dev/learn/synchronizing-with-effects
[3]	Mozilla Developer Network (MDN). (2024). <i>Fetch API Concepts and Usage</i> . Retrieved from https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
[4]	Remix Software, Inc. (2024). <i>React Router v6 Documentation: Feature Overview</i> . Retrieved from https://reactrouter.com/en/main/start/overview
[5]	Getbootstrap.com. (2023). <i>Bootstrap v5.3 Introduction</i> . Retrieved from https://getbootstrap.com/docs/5.3/getting-started/introduction/
[6]	Vitejs.dev. (2024). <i>Vite Guide: Why Vite?</i> . Retrieved from https://vitejs.dev/guide/why.html

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa  Maessa Andrea Vallenia NIM: 23051430044	Diperiksa oleh Asisten / Dosen Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____
--	---

MODUL 12

PROYEK WEB: DASHBOARD MONITORING PRODUKSI (REACT+CHART.JS)

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 12 (Pertemuan 12)
Topik Utama	Integrasi Library Visualisasi Data (Chart.js) dan Finalisasi Layout Dashboard.
Sub-Topik	Integrasi Library Visualisasi (react-chartjs-2), Transformasi Data Array ke Grafik, <i>Multi-Dataset Mapping</i> , Penataan Layout Dashboard (Sidebar & Main Content), Responsivitas Komponen, dan Finalisasi <i>State Management</i> untuk Monitoring Produksi secara <i>Real-time</i> .
Durasi Praktikum	340 Menit
Tanggal Praktikum	8 Mei 2026
Nama Mahasiswa	Maessa Andrea Vallenia
NIM	23051430044
Kelas	A

B. TUJUAN PEMBELAJARAN

Tuliskan tujuan pembelajaran yang ingin dicapai pada modul ini, sesuai dengan yang tertera di Manual Praktikum.

1	Mampu mengintegrasikan library pihak ketiga (`react-chartjs-2`) ke dalam React.
2	Mampu mengubah data mentah menjadi visualisasi grafik yang informatif.
3	Menyelesaikan proyek mini dashboard lengkap dari pertemuan 1-12.

C. ALAT DAN BAHAN

Sebutkan seluruh perangkat keras, perangkat lunak, dan sumber referensi yang digunakan selama praktikum.

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	<i>Code Editor</i>
2	Google Chrome / Browser Modern	Versi terbaru	<i>Testing & Debugging</i>
3	React Router DOM	Versi 6	Library untuk menangani navigasi multi-halaman tanpa <i>reload</i>
4	React JS Library	Versi 18 ke atas	Framework utama berbasis komponen untuk membangun Dashboard
5	CodeSnap	Versi 1.3.4	Dokumentasi kode program
6	Git dan GitHub	Versi 2.53.0	Version control system (VCS) berbasis Git
7	Koneksi Internet	Stabil	Mengakses Library dan Framework
8	Console Browser (Developer Tools)	Default Browser	Melihat output program dan debugging
9	Bootstrap	Versi 5.3	Membantu pembuatan UI seperti button, card, dan alert pada panel control
10	File praktikum	Pertemuan sebelumnya	Digunakan sebagai dasar pengembangan panel kontrol
11	LocalStorage (Web Storage API)	Built-in Browser	Digunakan untuk

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
			menyimpan data produksi secara persisten
12	Chart.js & react-chartjs-2	Versi terbaru	Library utama untuk pembuatan grafik produksi (Bar & Doughnut)
13	Fetch API	Built-in Browser	Digunakan untuk melakukan HTTP request (GET, POST, dll) ke server

D. DASAR TEORI

1. Integrasi Library Pihak Ketiga merupakan proses menambahkan fungsionalitas eksternal ke dalam aplikasi React, seperti react-chartjs-2. Library ini bertindak sebagai pembungkus (*wrapper*) agar library Chart.js dapat bekerja secara deklaratif di dalam komponen React, sehingga memudahkan pengembang dalam membuat fitur kompleks tanpa harus membangunnya dari awal.
2. Visualisasi Data adalah teknik merepresentasikan data numerik mentah ke dalam format grafis yang informatif. Dalam konteks industri, visualisasi membantu manajer untuk mengidentifikasi pola produksi, tren efisiensi, dan distribusi kecacatan secara cepat dibandingkan hanya melihat data dalam bentuk tabel angka statis.
3. Chart.js Registration merupakan tahap wajib dalam React di mana elemen-elemen grafik seperti CategoryScale, LinearScale, dan BarElement harus didaftarkan secara manual. Proses ini bertujuan untuk meminimalkan ukuran *bundle* aplikasi dengan hanya memuat fitur grafik yang benar-benar digunakan, sehingga performa aplikasi tetap optimal.
4. Dataset dan Labels adalah dua properti utama dalam membangun grafik. Labels berfungsi untuk menentukan keterangan pada sumbu X (seperti jam produksi), sedangkan Datasets merupakan objek yang menampung data angka produksi beserta atribut visualnya seperti warna batang, warna garis, dan ketebalan bingkai grafik.
5. Mixed Charts (Grafik Campuran) adalah teknik menampilkan dua tipe grafik berbeda, seperti bar (batang) dan line (garis), dalam satu bidang koordinat

yang sama. Teknik ini sangat berguna di industri untuk membandingkan pencapaian aktual (*bar*) terhadap target produksi (*line*) secara langsung dalam satu tampilan visual.

6. Modular Layouting adalah metode penyusunan antarmuka dengan membagi halaman menjadi bagian-bagian mandiri, seperti Sidebar (navigasi) dan Main Content (area grafik). Pendekatan ini memastikan kode program lebih terstruktur, mudah dikelola, dan memberikan pengalaman navigasi yang profesional bagi pengguna dashboard monitoring.
7. Responsive Charting merupakan pengaturan agar ukuran grafik dapat menyesuaikan diri secara otomatis terhadap lebar kontainer atau resolusi layar. Dengan mengaktifkan properti `responsive: true`, dashboard monitoring dapat diakses dengan baik melalui perangkat desktop maupun perangkat *mobile* di lantai produksi.

Secara keseluruhan, integrasi grafik yang dipadukan dengan manajemen state dan layout modular membuat dashboard monitoring produksi menjadi alat bantu yang kuat dalam pengambilan keputusan manajerial secara real-time dan akurat.

D.1 Konsep Utama

Konsep utama pada pertemuan ini adalah integrasi library visualisasi data pihak ketiga untuk mengubah sekumpulan data mentah menjadi informasi grafis yang intuitif. Visualisasi data berfungsi sebagai alat analisis cepat bagi manajemen untuk memantau performa lini produksi secara keseluruhan tanpa harus membaca tabel angka yang kompleks.

Penggunaan library `react-chartjs-2` menjadi inti utama dalam praktikum karena library ini memungkinkan komponen React untuk merender grafik secara deklaratif. Dengan memanfaatkan objek data dan opsi konfigurasi, aplikasi dapat menampilkan tren produksi jam-ke-jam yang dinamis, di mana setiap perubahan pada *state* data akan secara otomatis memperbarui grafik melalui proses *re-rendering* Virtual DOM.

Selain visualisasi, praktikum ini juga menerapkan konsep *Mixed Charts* atau grafik campuran. Konsep ini menggabungkan dua tipe dataset berbeda, yaitu *Bar* untuk output aktual dan *Line* untuk target produksi. Hal ini memungkinkan sistem melakukan perbandingan performa secara *real-time* dalam satu bidang koordinat, yang merupakan standar dalam analisis efisiensi industri.

Konsep *Modular Layouting* dengan struktur *Sidebar* dan *Main Content* juga menjadi bagian penting. Pemisahan area navigasi dengan area konten utama memastikan aplikasi memiliki hierarki informasi yang jelas. Hal ini mendukung prinsip skalabilitas, di mana pengembang dapat dengan mudah menambah modul baru pada *sidebar* tanpa mengganggu tata letak visualisasi utama pada dashboard.

Transformasi data asinkron digunakan untuk menghubungkan *state* internal dengan komponen grafik. Data yang diperoleh melalui *fetch* API atau generator acak diproses terlebih dahulu agar sesuai dengan format dataset yang diminta oleh Chart.js, sehingga visualisasi

yang dihasilkan selalu sinkron dengan data produksi terbaru yang ada di server maupun *database* lokal.

Konsep responsivitas grafik juga diterapkan untuk menjamin fleksibilitas perangkat. Dengan mengatur properti *responsive* dan *aspect ratio*, elemen *canvas* pada grafik dapat menyesuaikan ukurannya secara otomatis terhadap perubahan dimensi layar, baik pada perangkat komputer di kantor manajemen maupun tablet yang digunakan operator di lantai pabrik.

Dengan menggabungkan integrasi library grafik, manajemen layout modular, dan transformasi data dinamis, aplikasi React berhasil bertransformasi menjadi sistem monitoring industri yang profesional, informatif, dan mampu mendukung pengambilan keputusan berbasis data (*data-driven decision*) secara akurat.

D.2 Keterkaitan dengan Teknik Industri

Materi visualisasi data menggunakan Chart.js memiliki hubungan yang sangat erat dengan bidang Teknik Industri karena sistem manufaktur cerdas memerlukan representasi data yang cepat dan mudah diinterpretasikan oleh pihak manajerial. Dalam lingkungan industri, data mentah (*raw data*) yang berasal dari lini produksi tidak akan memberikan nilai tambah jika tidak diolah menjadi informasi visual yang mampu menunjukkan tren dan penyimpangan secara *real-time*.

Penggunaan grafik batang (*Bar Chart*) pada dashboard sangat relevan untuk memantau pencapaian output harian. Manajer produksi dapat dengan mudah melihat pola fluktuasi produksi dari jam ke jam, sehingga jika terjadi penurunan output pada jam tertentu, tindakan perbaikan (*corrective action*) dapat segera dilakukan tanpa harus menunggu laporan manual di akhir shift.

Grafik lingkaran atau donat (*Doughnut Chart*) memiliki peran penting dalam pengendalian kualitas (*Quality Control*). Dengan memvisualisasikan proporsi jenis cacat produksi (seperti *scratch*, *dent*, atau *crack*), mahasiswa Teknik Industri dapat melakukan analisis Pareto secara visual untuk menentukan jenis cacat mana yang paling dominan, sehingga prioritas perbaikan kualitas dapat ditentukan secara akurat.

Penerapan grafik campuran (*Mixed Chart*) antara batang dan garis sangat bermanfaat untuk pemantauan target produksi. Garis target yang bersifat konstan dibandingkan dengan batang output aktual memberikan indikasi visual langsung mengenai status pencapaian target. Hal ini mempermudah penerapan konsep *Visual Management* di lantai pabrik, di mana status produksi dapat dipahami hanya dalam hitungan detik.

Konsep *Modular Layouting* dengan *Sidebar* mencerminkan struktur organisasi informasi dalam sistem *Enterprise Resource Planning* (ERP) atau *Manufacturing Execution System* (MES). Pemisahan modul navigasi memungkinkan integrasi berbagai fungsi industri seperti inventori, pemeliharaan mesin, dan laporan kualitas dalam satu platform tunggal yang terintegrasi.

Pada tugas proyek akhir ini, mahasiswa belajar melakukan transformasi data teknis menjadi *Key Performance Indicator* (KPI) yang komunikatif. Kemampuan menyajikan data efisiensi dan jumlah output ke dalam bentuk grafik merupakan kompetensi krusial bagi

seorang *Industrial Engineer* dalam melakukan presentasi performa lini produksi kepada pihak manajemen eksekutif.

Dengan memahami integrasi library grafik dan desain dashboard, mahasiswa Teknik Industri memiliki bekal yang kuat untuk merancang sistem *monitoring* yang interaktif, profesional, dan mampu menjadi alat bantu yang efektif dalam proses pengambilan keputusan strategis berbasis data (*data-driven decision making*).

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

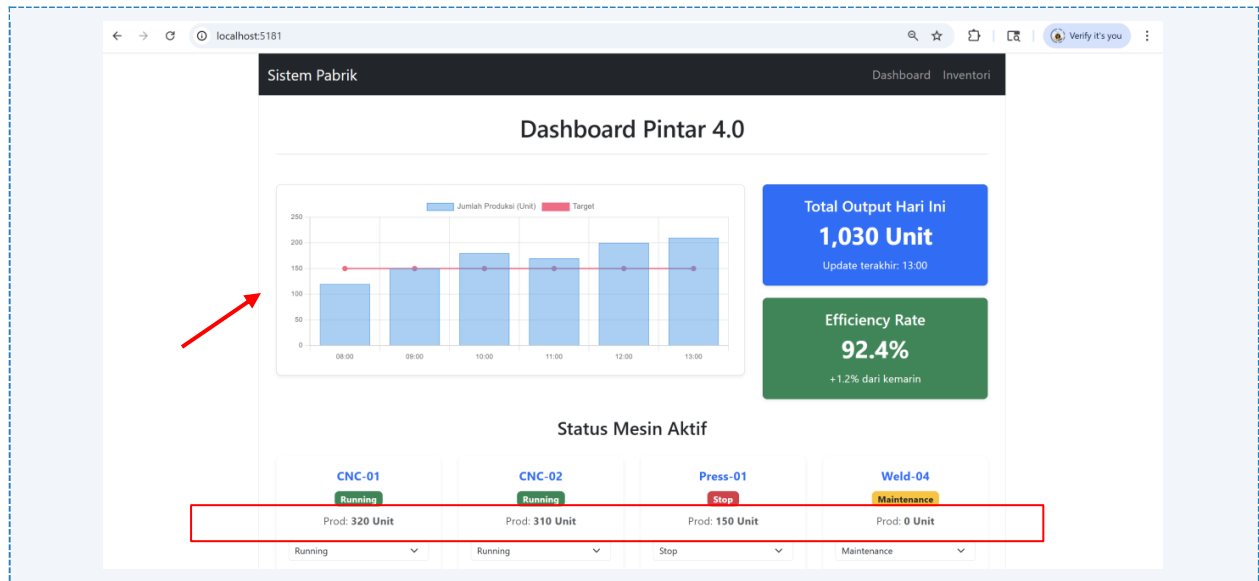
Dokumentasikan setiap langkah kerja yang Anda lakukan, beserta hasil yang diperoleh (screenshot, kode, atau output). Jawab sesuai urutan langkah di Manual Praktikum.

E.1 Langkah 1 – Persiapan Data Grafik

Kita akan memvisualisasikan data produksi per jam. Misal data array: `[120, 150, 180, 170, 200, 210]` (Jam 8 sampai Jam 13).

```
/* ===== KODE / OUTPUT LANGKAH 1 ===== */

// Data untuk Grafik
const data = {
  labels: ['08:00', '09:00', '10:00', '11:00', '12:00', '13:00'],
  datasets: [
    {
      label: 'Jumlah Produksi (Unit)',
      data: [120, 150, 180, 170, 200, 210],
      backgroundColor: 'rgba(54, 162, 235, 0.5)',
      borderColor: 'rgba(54, 162, 235, 1)',
      borderWidth: 1,
    },
  ],
}
```

Screenshot Hasil:

Gambar E.1 – Hasil Persiapan Data Grafik

E.2 Langkah 2 – Membuat Komponen Grafik

Buat file `src/Komponen/GrafikProduksi.jsx`.

```
/* ===== KODE / OUTPUT LANGKAH 2 ===== */

import React from 'react';
import { Bar } from 'react-chartjs-2';
import {
  Chart as ChartJS,
  CategoryScale,
  LinearScale,
  BarElement,
  Title,
  Tooltip,
  Legend,
  PointElement,
  LineElement
} from 'chart.js';

// Mendaftarkan komponen ChartJS
ChartJS.register(
  CategoryScale,
  LinearScale,
  BarElement,
  Title,
  Tooltip,
  Legend,
  PointElement,
```



```
    LineElement
  );

function GrafikProduksi() {
  // Data untuk Grafik
  const data = {
    labels: ['08:00', '09:00', '10:00', '11:00', '12:00', '13:00'],
    datasets: [
      {
        label: 'Jumlah Produksi (Unit)',
        data: [120, 150, 180, 170, 200, 210],
        backgroundColor: 'rgba(54, 162, 235, 0.5)',
        borderColor: 'rgba(54, 162, 235, 1)',
        borderWidth: 1,
      },
      {
        label: 'Target',
        data: [150, 150, 150, 150, 150, 150],
        type: 'line', // Tipe campuran (Bar + Line)
        borderColor: 'rgb(255, 99, 132)',
        backgroundColor: 'rgb(255, 99, 132)',
        borderWidth: 2,
        pointRadius: 3,
      },
    ],
  };

  // Opsi Tampilan Grafik
  const options = {
    responsive: true,
    maintainAspectRatio: false, // Biar grafik bisa menyesuaikan tinggi
    container
    plugins: {
      legend: {
        position: 'top',
        labels: {
          font: { size: 11 }
        }
      },
    },
    title: {
      display: false, // Dimatikan karena judul sudah ada di Card Dashboard
    },
  },
  scales: {
    y: {
      beginAtZero: true,
    },
  },
};
```

```

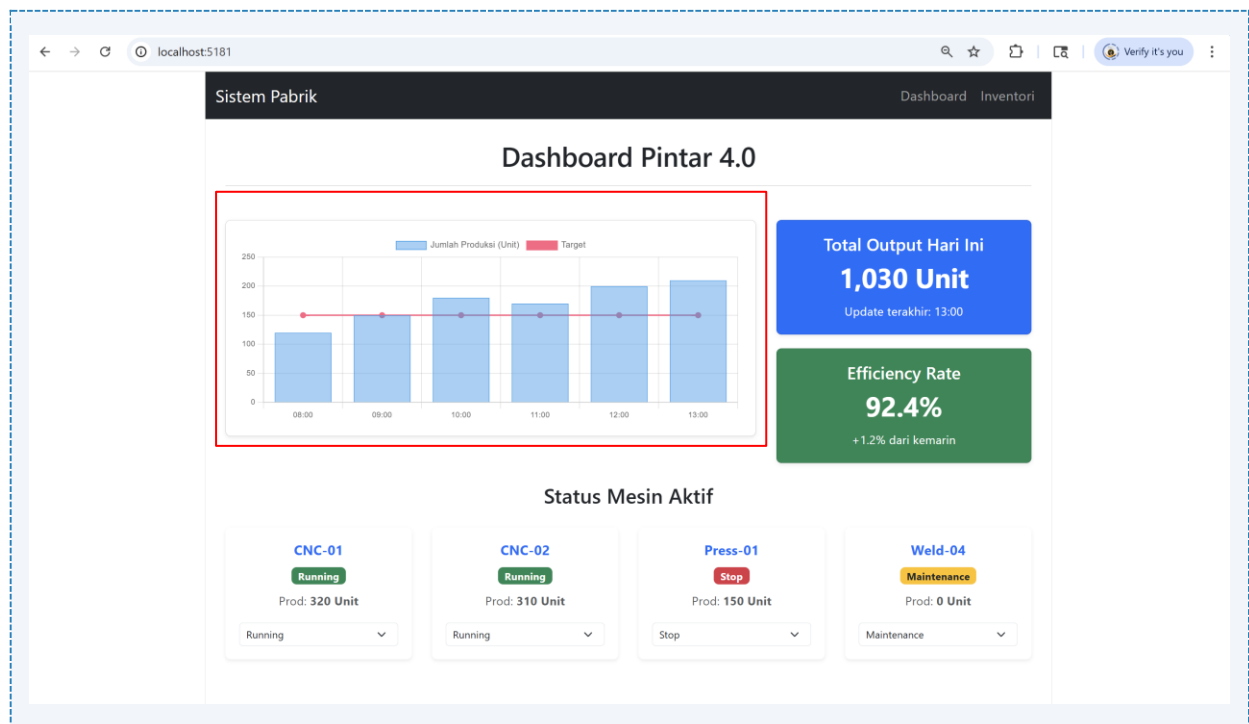
        ticks: { font: { size: 10 } }
      },
      x: {
        ticks: { font: { size: 10 } }
      }
    },
  };

  return (
    <div style={{ height: '250px' }}> {/* Memberi batas tinggi agar rapi */>
      <Bar data={data} options={options} />
    </div>
  );
}

export default GrafikProduksi;

```

Screenshot Hasil:



Gambar E.2 – Hasil Membuat Komponen Grafik

E.3 Langkah 3 – Menggabungkan ke Dashboard Utama

Update `src/Halaman/Dashboard.jsx` agar memuat grafik ini.

```

/* ===== KODE / OUTPUT LANGKAH 3 ===== */

import React from 'react';
import GrafikProduksi from '../Komponen/GrafikProduksi';

```

```
import KartuMesin from '../Komponen/KartuMesin';

function Dashboard() {
  return (
    <div className="container-fluid mt-4">
      <div className="row mb-4">
        <div className="col-12">
          <h2>Dashboard Pintar 4.0</h2>
          <hr />
        </div>
      </div>

      {/* Bagian Grafik */}
      <div className="row mb-4">
        <div className="col-md-8">
          <div className="card shadow-sm">
            <div className="card-body">
              <GrafikProduksi />
            </div>
          </div>
        </div>

        {/* Bagian Ringkasan KPI */}
        <div className="col-md-4">
          <div className="card bg-primary text-white mb-3 shadow-sm">
            <div className="card-body text-center">
              <h5>Total Output Hari Ini</h5>
              <h2 className="fw-bold">1,030 Unit</h2>
              <small>Update terakhir: 13:00</small>
            </div>
          </div>

          <div className="card bg-success text-white shadow-sm">
            <div className="card-body text-center">
              <h5>Efficiency Rate</h5>
              <h2 className="fw-bold">92.4%</h2>
              <small>+1.2% dari kemarin</small>
            </div>
          </div>
        </div>
      </div>

      {/* Bagian Daftar Mesin */}
      <div className="row mt-4">
        <div className="col-12 mb-3">
          <h4>Status Mesin Aktif</h4>
        </div>
      </div>
    </div>
  );
}
```

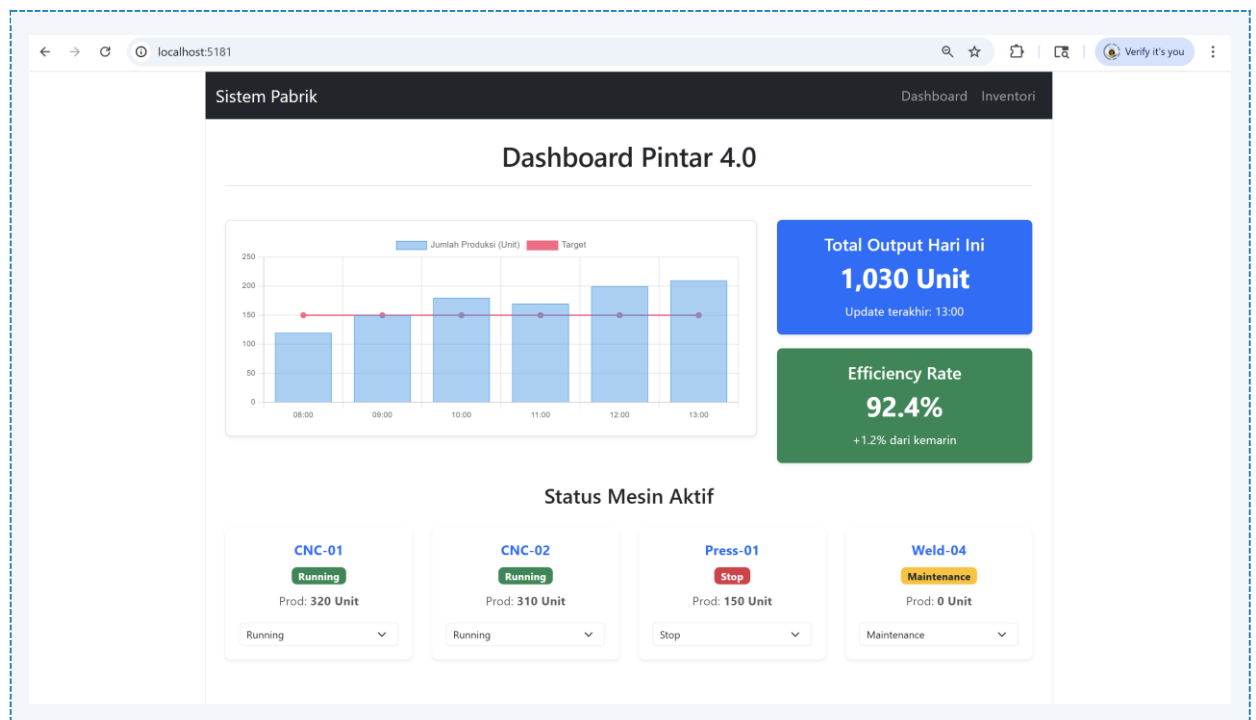
```

    </div>
    <div className="col-md-3 mb-3">
      <KartuMesin nama="CNC-01" status="Running" produksi={320} />
    </div>
    <div className="col-md-3 mb-3">
      <KartuMesin nama="CNC-02" status="Running" produksi={310} />
    </div>
    <div className="col-md-3 mb-3">
      <KartuMesin nama="Press-01" status="Stop" produksi={150} />
    </div>
    <div className="col-md-3 mb-3">
      <KartuMesin nama="Weld-04" status="Maintenance" produksi={0} />
    </div>
  </div>
</div>
);
}

export default Dashboard;

```

Screenshot Hasil:



Gambar E.3 – Hasil Menggabungkan ke Dashboard Utama

F. LATIHAN DAN TUGAS MANDIRI

Jawab dan dokumentasikan seluruh latihan/tugas yang ada di bagian "Latihan/Tugas Mandiri" pada modul Anda.

F.1 Latihan 1 – Integrasi Data Realistik

Pertanyaan / Instruksi Latihan:

Ubah data statis di `GrafikProduksi` menjadi data yang diambil dari `fetch` API (gunakan mock data dari internet atau buat generator acak).

Jawaban / Kode Solusi:

```
import React, { useState, useEffect } from 'react';
import { Bar } from 'react-chartjs-2';
import {
  Chart as ChartJS,
  CategoryScale,
  LinearScale,
  BarElement,
  PointElement,
  LineElement,
  Title,
  Tooltip,
  Legend
} from 'chart.js';

ChartJS.register(CategoryScale, LinearScale, BarElement, PointElement,
  LineElement, Title, Tooltip, Legend);

function GrafikProduksi() {
  // 1. Siapkan State untuk menampung data (awalnya kosong)
  const [dataProduksi, setDataProduksi] = useState([]);
  const [loading, setLoading] = useState(true);

  // 2. Simulasi Fetch API menggunakan useEffect
  useEffect(() => {
    // Fungsi simulasi mengambil data dari server
    const ambilDataPabrik = () => {
      setLoading(true);

      // Simulasi delay jaringan selama 1 detik
      setTimeout(() => {
        // Membuat 6 angka acak antara 100 - 250 (Generator Acak)
        const mockData = Array.from({ length: 6 }, () =>
          Math.floor(Math.random() * 151) + 100);

        setDataProduksi(mockData);
      }, 1000);
    };
  }, []);
}
```

```
        setLoading(false);
    }, 1000);
};

ambilDataPabrik();
}, []); // [] memastikan hanya jalan 1x saat Dashboard dibuka

const data = {
  labels: ['08:00', '09:00', '10:00', '11:00', '12:00', '13:00'],
  datasets: [
    {
      label: 'Produksi Aktual (Real-time)',
      data: dataProduksi, // Data dinamis dari state
      backgroundColor: 'rgba(54, 162, 235, 0.5)',
      borderColor: 'rgba(54, 162, 235, 1)',
      borderWidth: 1,
    },
    {
      label: 'Target Lini',
      data: [180, 180, 180, 180, 180, 180], // Garis target tetap
      type: 'line',
      borderColor: 'rgb(255, 99, 132)',
      borderWidth: 2,
      fill: false,
    }
  ],
};

const options = {
  responsive: true,
  plugins: {
    legend: { position: 'top' },
    title: { display: true, text: 'Monitoring Output Lini 1' }
  }
};

// Tampilan saat data sedang "diambil"
if (loading) {
  return <div className="text-center p-5"> ⌚ Menghubungkan ke
Mesin...</div>;
}

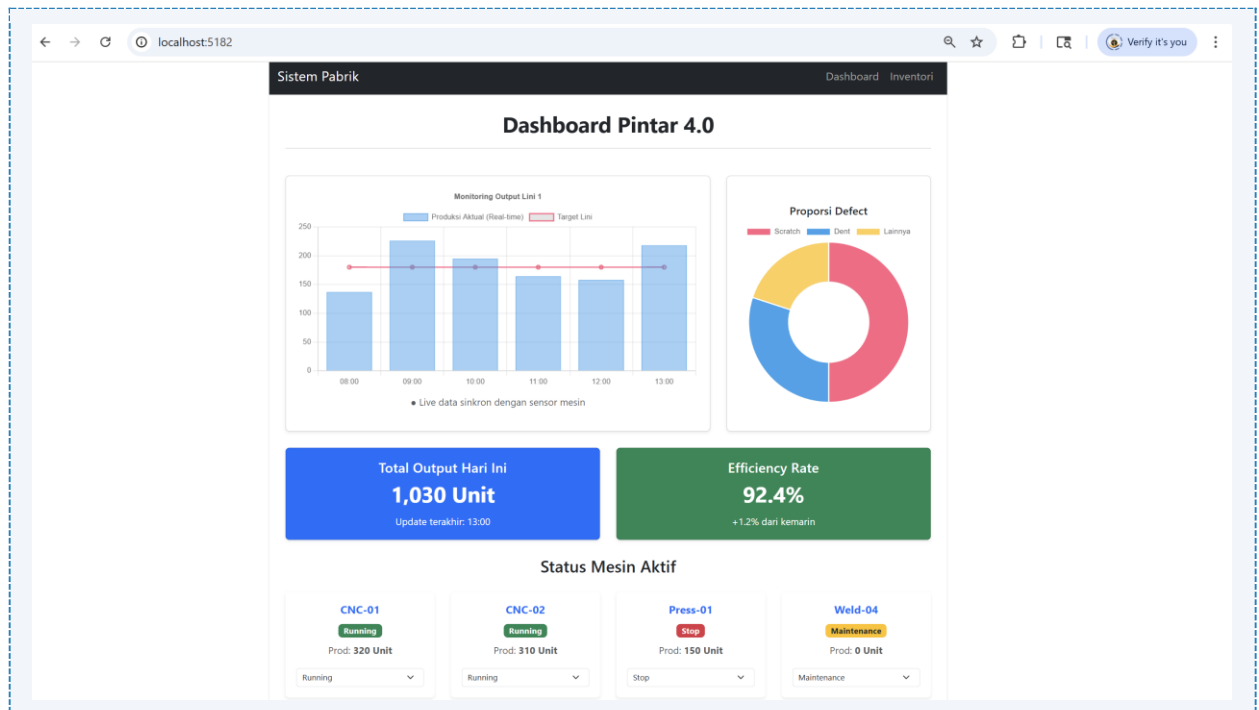
return (
  <div className="animate__animated animate__fadeIn">
    <Bar data={data} options={options} />
    <p className="text-center text-muted mt-2">
```

```

    <small>• Live data sinkron dengan sensor mesin</small>
  </p>
</div>
);
}

export default GrafikProduksi;

```



Gambar F.1 – Hasil Integrasi Data Realistik

F.2 Latihan 2 – Tambahkan Grafik Donat

Pertanyaan / Instruksi Latihan:

Buat grafik "Proporsi Cacat" (Misal: Scratch 50%, Dent 30%, Lainnya 20%) menggunakan komponen `Doughnut` dari chart.js.

Jawaban / Kode Solusi:

```

import React from 'react';
import { Doughnut } from 'react-chartjs-2';
import { Chart as ChartJS, ArcElement, Tooltip, Legend } from 'chart.js';

ChartJS.register(ArcElement, Tooltip, Legend);

function GrafikDonat() {
  const data = {
    labels: ['Scratch', 'Dent', 'Lainnya'],

```

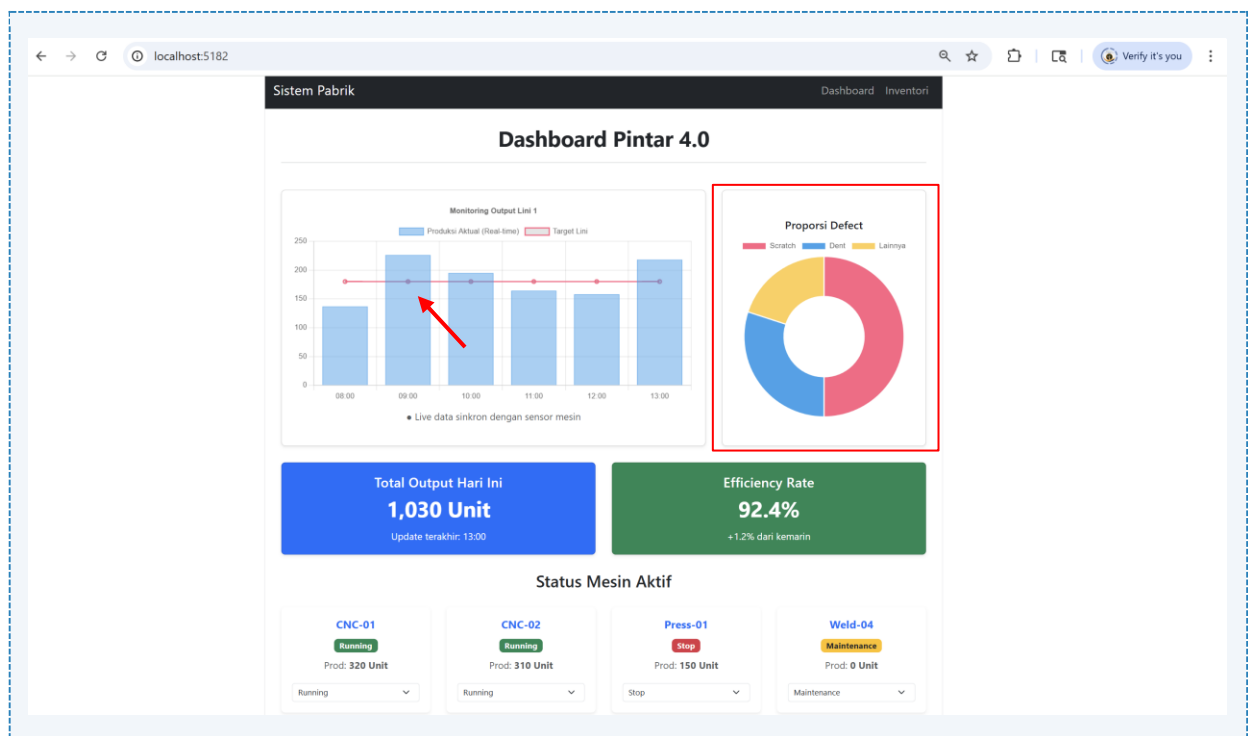
```

datasets: [
  {
    data: [50, 30, 20],
    backgroundColor: ['#FF6384', '#36A2EB', '#FFCE56'],
    hoverBackgroundColor: ['#FF6384', '#36A2EB', '#FFCE56'],
  },
],
};

return (
  <div>
    <h6 className="text-center">Proporsi Defect</h6>
    <Doughnut data={data} />
  </div>
);
}

export default GrafikDonat;

```



Gambar F.2 – Hasil Tambahkan Grafik Donat

F.3 Tugas Proyek Mini – Tugas Akhir Web

Deskripsi proyek mini:

Susun ulang halaman `Dashboard` agar memiliki Sidebar (kiri) dan Main Content (kanan).

- f. Sidebar: Logo, Menu Dashboard, Menu Inventori, Menu Laporan.
- g. Main Content: Header (User Profile), Grafik, dan Tabel Data.
- h. Pastikan desain responsif (Sidebar bisa collapse di HP).

Penjelasan pendekatan/solusi yang digunakan:

Proyek mini ini dilakukan melalui beberapa tahapan yang sistematis, dimulai dari perancangan tampilan hingga implementasi

- J. Menyiapkan struktur proyek menggunakan Vite dan React, serta mengintegrasikan library pendukung utama seperti react-router-dom untuk navigasi, bootstrap untuk *styling*, dan chart.js untuk visualisasi data.
- K. Membagi UI menjadi komponen-komponen kecil yang dapat digunakan kembali (*reusable*), seperti Sidebar, Navbar, serta komponen grafik terpisah untuk menjaga kebersihan dan efisiensi kode.
- L. Mengimplementasikan BrowserRouter untuk menciptakan pengalaman aplikasi satu halaman (SPA) yang memiliki banyak rute, seperti halaman Dashboard Utama, Manajemen Inventori, dan Laporan QC.
- M. Menggunakan library Chart.js untuk merepresentasikan data produksi secara visual dalam bentuk *Bar Chart* (untuk tren produksi) dan *Doughnut Chart* (untuk proporsi defect), guna memudahkan analisis data bagi manajer produksi.
- N. Melakukan registrasi manual pada elemen-elemen Chart.js (seperti BarElement, LineElement, dan PointElement) untuk memastikan grafik campuran (*mixed chart*) dapat dirender dengan sempurna di lingkungan produksi.
- O. Menggunakan React useState untuk mengontrol status *Sidebar* (expand/collapse) dan mengelola tampilan *Modal Detail* yang muncul secara asinkron saat pengguna ingin melihat spesifikasi teknis mesin.
- P. Mengimplementasikan logika *Conditional Rendering* pada tabel log produksi, di mana warna badge status (Normal, Maintenance, Offline) akan berubah secara otomatis berdasarkan data yang diterima.
- Q. Menggunakan sistem grid Bootstrap dan CSS Media Queries untuk memastikan dashboard tetap fungsional dan rapi saat diakses melalui perangkat seluler (*responsive layout*).
- R. Menambahkan rute *wildcard* (path="*") yang mengarah pada halaman 404 Not Found khusus, guna meningkatkan *user experience* ketika pengguna mengakses alamat URL yang salah.
- S. Melakukan proses transformasi kode JSX menjadi aset statis (HTML, CSS, JS) melalui perintah npm run build, yang mengoptimalkan ukuran file untuk kinerja yang lebih cepat.
- T. Membuat file konfigurasi _redirects pada folder distribusi untuk memastikan sistem *routing* React tetap berjalan stabil saat halaman dimuat ulang (*refresh*) di server hosting (Netlify).
- U. Mengonlinekan aplikasi ke platform Netlify dan melakukan pengujian fungsionalitas menyeluruh, mulai dari validasi grafik, navigasi antar menu, hingga responsivitas elemen UI di berbagai ukuran layar.

Jawaban / Kode Solusi:

1. JAWABAN KODE DASHBOARD.JSX

```
import React, { useState } from 'react';
import Sidebar from '../Komponen/Sidebar';
import GrafikProduksi from '../Komponen/GrafikProduksi';
import GrafikDonat from '../Komponen/GrafikDonat';

function Dashboard() {
  const [showDetail, setShowDetail] = useState(null);

  return (
    <div className="container-fluid bg-light min-vh-100 p-0">
      <header className="navbar navbar-dark sticky-top bg-dark p-2 shadow d-
md-none">
        <div className="container-fluid d-flex justify-content-between align-
items-center">
          <a className="navbar-brand fw-bold m-0" href="#">TI - UNY</a>
          <button className="navbar-toggler" type="button" data-bs-
toggle="collapse" data-bs-target="#sidebarMenu">
            <span className="navbar-toggler-icon"></span>
          </button>
        </div>
      </header>

      <div className="row g-0">
        <Sidebar aktif="dashboard" />

        <main className="col-md-9 ms-sm-auto col-lg-10 px-3 px-md-4">

          <div className="d-flex flex-row justify-content-between align-
items-start pt-4 pb-3 mb-4 border-bottom bg-white mt-3 px-3 rounded-3 shadow-
sm">
            <div className="text-start">
              <h1 className="h4 fw-bold m-0 text-dark">Dashboard
Monitoring</h1>
              <small className="text-success fw-bold d-block">
                • Live Update: {new Date().toLocaleDateString('id-ID', {
day: 'numeric', month: 'long', year: 'numeric' })}
              </small>
            </div>

            <div className="d-flex align-items-center bg-light p-2 rounded-
pill px-3 border shadow-sm">
              <div className="text-end me-2 d-none d-sm-block">
                <div className="fw-bold text-dark" style={{ fontSize:
'0.75rem', lineHeight: '1' }}>
```

```

        Maessa Andrea
      </div>
      <div className="text-muted" style={{ fontSize: '0.65rem' }}>
        23051430044
      </div>
    </div>
    <div className="rounded-circle bg-primary d-flex align-items-
center justify-content-center text-white fw-bold shadow-sm"
      style={{ minWidth: '35px', height: '35px', fontSize:
'0.7rem', border: '2px solid #fff' }}>
      MA
    </div>
  </div>
</div>

<div className="row g-3 mb-4">
  <div className="col-6 col-lg-3">
    <div className="card border-0 shadow-sm p-3 bg-primary text-
white h-100 border-bottom border-white border-4">
      <small className="d-block opacity-75 fw-bold"
style={{fontSize: '0.65rem'}}>TOTAL OUTPUT</small>
      <h4 className="fw-bold m-0">2,450 <small className="fs-6 fw-
normal">Unit</small></h4>
    </div>
  </div>
  <div className="col-6 col-lg-3">
    <div className="card border-0 shadow-sm p-3 bg-success text-
white h-100 border-bottom border-white border-4">
      <small className="d-block opacity-75 fw-bold"
style={{fontSize: '0.65rem'}}>OEE RATE</small>
      <h4 className="fw-bold m-0">92.4%</h4>
    </div>
  </div>
  <div className="col-6 col-lg-3">
    <div className="card border-0 shadow-sm p-3 bg-warning text-
dark h-100 border-bottom border-dark border-4">
      <small className="d-block opacity-75 fw-bold"
style={{fontSize: '0.65rem'}}>DOWN TIME</small>
      <h4 className="fw-bold m-0">12 <small className="fs-6 fw-
normal">Min</small></h4>
    </div>
  </div>
  <div className="col-6 col-lg-3">
    <div className="card border-0 shadow-sm p-3 bg-danger text-
white h-100 border-bottom border-white border-4">

```

```

        <small className="d-block opacity-75 fw-bold"
style={{fontSize: '0.65rem'}}>DEFECT RATE</small>
        <h4 className="fw-bold m-0">0.8%</h4>
      </div>
    </div>
  </div>

  <div className="row g-3 mb-4">
    <div className="col-12 col-lg-8">
      <div className="card border-0 shadow-sm p-3 h-100">
        <h6 className="fw-bold mb-3 small text-muted text-
uppercase">Monitoring Output Per Jam</h6>
        <div style={{ minHeight: '250px' }}>
          <GrafikProduksi />
        </div>
      </div>
    </div>
    <div className="col-12 col-lg-4">
      <div className="card border-0 shadow-sm p-3 h-100 text-center">
        <h6 className="fw-bold mb-3 small text-muted text-
uppercase">Proporsi Defect</h6>
        <div style={{ maxHeight: '250px' }}>
          <GrafikDonat />
        </div>
      </div>
    </div>
  </div>

  <div className="card border-0 shadow-sm rounded-3 mb-5 overflow-
hidden">
    <div className="card-header bg-dark text-white py-3 fw-bold text-
center small">
      LOG PRODUKSI TERAKHIR (REAL-TIME)
    </div>
    <div className="table-responsive">
      <table className="table table-hover align-middle text-center
mb-0">
        <thead className="table-light fw-bold small text-secondary">
          <tr>
            <th>WAKTU</th><th>MESIN</th><th>OUTPUT</th><th>STATUS</th>
<th>AKSI</th>
          </tr>
        </thead>
        <tbody className="small">
          <tr>

```

```

<td>08:00</td><td className="fw-bold">CNC-01</td><td>120
Unit</td>
<td><span className="badge rounded-pill bg-success px-
3">NORMAL</span></td>
<td><button className="btn btn-sm btn-outline-primary fw-
bold" style={{fontSize: '10px'}} onClick={() => setShowDetail({id: 'CNC-01',
text: 'Semua sensor stabil.', status: 'NORMAL', suhu: '42°C', efisiensi:
'94%'}})>DETAIL</button></td>
</tr>
<tr>
<td>09:00</td><td className="fw-bold">CNC-02</td><td>150
Unit</td>
<td><span className="badge rounded-pill bg-success px-
3">NORMAL</span></td>
<td><button className="btn btn-sm btn-outline-primary fw-
bold" style={{fontSize: '10px'}} onClick={() => setShowDetail({id: 'CNC-02',
text: 'Pelumasan dilakukan.', status: 'NORMAL', suhu: '45°C', efisiensi:
'92%'}})>DETAIL</button></td>
</tr>
<tr>
<td>10:00</td><td className="fw-bold">Press-05</td><td>0
Unit</td>
<td><span className="badge rounded-pill bg-danger px-
3">OFFLINE</span></td>
<td><button className="btn btn-sm btn-outline-primary fw-
bold" style={{fontSize: '10px'}} onClick={() => setShowDetail({id: 'Press-
05', text: 'Motor Overheat.', status: 'OFFLINE', suhu: '88°C', efisiensi:
'0%'}})>DETAIL</button></td>
</tr>
</tbody>
</table>
</div>
</div>

<footer className="text-center py-4 border-top mt-auto">
<small className="text-muted">
&copy; 2026 Lab ERP - <strong>Universitas Negeri
Yogyakarta</strong>
<br/>
<span className="text-primary fw-bold">Sistem Monitoring
Terpadu</span>
</small>
</footer>

{showDetail && (

```

```

        <div className="modal d-block" style={{ backgroundColor:
'rgba(0,0,0,0.7)', zIndex: 2000 }}>
          <div className="modal-dialog modal-dialog-centered px-3">
            <div className="modal-content border-0 shadow-lg rounded-4">
              <div className="modal-header bg-dark text-white">
                <h6 className="modal-title fw-bold">🔍 Rincian:
{showDetail.id}</h6>
                <button type="button" className="btn-close btn-close-
white" onClick={() => setShowDetail(null)}></button>
              </div>
              <div className="modal-body p-4">
                <div className="row g-3">
                  <div className="col-6 small">
                    <span className="text-muted d-block mb-
1">Status</span>
                    <span className={`badge rounded-pill
${showDetail.status === 'OFFLINE' ? 'bg-danger' : 'bg-success'} px-
3`}>{showDetail.status}</span>
                  </div>
                  <div className="col-6 text-end small">
                    <span className="text-muted d-block mb-
1">Lokasi</span>
                    <span className="fw-bold">Lini A-1</span>
                  </div>
                </div>
                <div className="col-12"><hr className="my-2" /></div>
                <div className="col-12">
                  <div className="p-3 bg-light rounded-3 text-center
small font-monospace border">
                    "{showDetail.text}"
                  </div>
                </div>
                <div className="col-6">
                  <div className="p-2 border rounded-3 text-center bg-
white">
                    <small className="text-muted d-block"
style={{fontSize: '10px'}}>SUHU</small>
                    <strong className={showDetail.status === 'OFFLINE'
? 'text-danger' : 'text-primary'}>{showDetail.suhu}</strong>
                  </div>
                </div>
                <div className="col-6">
                  <div className="p-2 border rounded-3 text-center bg-
white">
                    <small className="text-muted d-block"
style={{fontSize: '10px'}}>EFISIENSI</small>

```

```

                <strong className="text-
success">{showDetail.efisiensi}</strong>
            </div>
        </div>
    </div>
    <div className="modal-footer border-0">
        <button className="btn btn-dark w-100 rounded-pill fw-
bold" onClick={() => setShowDetail(null)}>TUTUP</button>
    </div>
</div>
</div>
</div>
    )}
</main>
</div>
</div>
    );
}

export default Dashboard;

```

2. JAWABAN KODE DETAILITEM.JSX

```

import React, { useState, useEffect } from 'react';
import { useParams, Link } from 'react-router-dom';
import Sidebar from '../Komponen/Sidebar';

function DetailItem() {
    const { id } = useParams();

    // 1. State untuk data, loading, dan error
    const [item, setItem] = useState(null);
    const [loading, setLoading] = useState(true);
    const [error, setError] = useState(null);

    // 2. Fetch data berdasarkan ID
    useEffect(() => {
        const fetchDetail = async () => {
            try {
                setLoading(true);
                // Ganti dengan URL API detail kamu, contoh: `/api/inventori/${id}`
                const response = await
fetch(`https://jsonplaceholder.typicode.com/posts/${id}`);

                if (!response.ok) {
                    throw new Error('Data item tidak ditemukan di server.');
```

```
    }

    const data = await response.json();

    // Simulasi pemetaan data API ke format UI kita
    const mappedData = {
      nama: data.title.substring(0, 30),
      spek: data.body,
      supplier: 'PT. Industri Terpadu',
      tgl: '10 Mei 2026',
      stok: Math.floor(Math.random() * 100) + ' Unit'
    };

    setItem(mappedData);
  } catch (err) {
    setError(err.message);
  } finally {
    setLoading(false);
  }
};

fetchDetail();
}, [id]);

// 3. Tampilan Loading
if (loading) {
  return (
    <div className="container p-5 text-center">
      <div className="spinner-border text-primary" role="status"></div>
      <p className="mt-2">Mengambil detail barang...</p>
    </div>
  );
}

// 4. Tampilan Error
if (error || !item) {
  return (
    <div className="container p-5 text-center">
      <h3 className="text-danger">Ups! {error || 'Item Tidak Ditemukan'}</h3>
      <Link to="/inventori" className="btn btn-primary mt-3">Kembali ke Inventori</Link>
    </div>
  );
}
```



```

return (
  <div className="container-fluid p-0 bg-light min-vh-100">
    { /* TOP NAVBAR MOBILE */ }
    <header className="navbar navbar-dark sticky-top bg-dark p-2 shadow d-
md-none">
      <div className="container-fluid d-flex justify-content-between align-
items-center">
        <a className="navbar-brand fw-bold m-0" href="#">TI - UNY</a>
        <button className="navbar-toggler" type="button" data-bs-
toggle="collapse" data-bs-target="#sidebarMenu">
          <span className="navbar-toggler-icon"></span>
        </button>
      </div>
    </header>

    <div className="row g-0">
      <Sidebar aktif="inventori" />

      <main className="col-md-9 ms-sm-auto col-lg-10 px-3 px-md-4">

        { /* HEADER AREA */ }
        <div className="d-flex flex-row justify-content-between align-
items-start pt-4 pb-3 mb-4 border-bottom bg-white mt-3 px-3 rounded-3 shadow-
sm">
          <div className="text-start">
            <h1 className="h4 fw-bold m-0 text-dark">Detail
Spesifikasi</h1>
            <small className="text-primary fw-bold d-block">
              • Inventory ID: #INV-00{id}
            </small>
          </div>

          <div className="d-flex align-items-center bg-light p-2 rounded-
pill px-3 border shadow-sm">
            <div className="text-end me-2 d-none d-sm-block">
              <div className="fw-bold text-dark" style={{ fontSize:
'0.75rem', lineHeight: '1' }}>
                Maessa Andrea
              </div>
              <div className="text-muted" style={{ fontSize: '0.65rem' }}>
                23051430044
              </div>
            </div>
            <div className="rounded-circle bg-primary d-flex align-items-
center justify-content-center text-white fw-bold shadow-sm">

```

```

                style={{ minWidth: '35px', height: '35px', fontSize:
'0.7rem', border: '2px solid #fff' }}>
                MA
            </div>
        </div>
    </div>

    <Link to="/inventori" className="btn btn-outline-dark btn-sm mb-4
fw-bold rounded-pill px-3">
        ← Kembali ke Inventori
    </Link>

    <div className="card border-0 shadow-sm p-4 rounded-4 mb-5">
        <div className="row">
            <div className="col-12 col-lg-7 border-end-lg">
                <h2 className="fw-bold text-primary mb-3 text-
capitalize">{item.nama}</h2>
                <h5 className="fw-bold small text-muted text-uppercase mb-
3">Spesifikasi Teknik</h5>
                <div className="bg-light p-4 border rounded-4 mb-4">
                    <p className="mb-0 text-dark" style={{ lineHeight: '1.6'
}}>{item.spek}</p>
                </div>
            </div>

            <div className="col-12 col-lg-5 ps-lg-5 mt-4 mt-lg-0">
                <h5 className="fw-bold small text-muted text-uppercase mb-
3">Detail Logistik</h5>
                <div className="table-responsive">
                    <table className="table table-borderless">
                        <tbody>
                            <tr className="border-bottom">
                                <td className="text-muted py-3">Status Stok</td>
                                <td className="fw-bold text-end py-3 text-
success">{item.stok}</td>
                            </tr>
                            <tr className="border-bottom">
                                <td className="text-muted py-3">Main Supplier</td>
                                <td className="fw-bold text-end py-
3">{item.supplier}</td>
                            </tr>
                            <tr className="border-bottom">
                                <td className="text-muted py-3">Kedatangan
Terakhir</td>
                                <td className="fw-bold text-end py-3">{item.tgl}</td>
                            </tr>
                        </tbody>
                    </table>
                </div>
            </div>
        </div>
    </div>

```

```

        </tbody>
      </table>
    </div>
  </div>
</div>
</div>
</main>
</div>
</div>
);
}

export default DetailItem;

```

3. JAWABAN KODE INVENTORI.JSX

```

import React, { useState, useEffect } from 'react';
import { Link } from 'react-router-dom';
import Sidebar from '../Komponen/Sidebar';

function Inventori() {
  // 2. Siapkan State untuk menampung data dari server
  const [dataStok, setDataStok] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  // 3. Fungsi Fetch API
  useEffect(() => {
    const fetchInventori = async () => {
      try {
        setLoading(true);
        const response = await
fetch('https://jsonplaceholder.typicode.com/posts?_limit=8');

        if (!response.ok) throw new Error('Gagal mengambil data stok');

        const result = await response.json();

        // Memetakan data API ke format tabel kita
        const mappedData = result.map((item, index) => ({
          id: item.id.toString(),
          nama: item.title.substring(0, 25), // Simulasi nama barang
          lokasi: index % 2 === 0 ? 'Gudang A' : 'Gudang B',
          stok: Math.floor(Math.random() * 1000) + ' Unit',
          status: index % 3 === 0 ? 'Available' : (index % 3 === 1 ? 'Low
Stock' : 'Out of Stock')
        }));
      }
    };
  });
}

```

```

        setDataStok(mappedData);
    } catch (err) {
        setError(err.message);
    } finally {
        setLoading(false);
    }
};

fetchInventori();
}, []);

return (
    <div className="container-fluid bg-light min-vh-100 p-0">
        <header className="navbar navbar-dark sticky-top bg-dark p-2 shadow d-
md-none">
            <div className="container-fluid d-flex justify-content-between align-
items-center">
                <a className="navbar-brand fw-bold m-0" href="#">TI - UNY</a>
                <button className="navbar-toggler" type="button" data-bs-
toggle="collapse" data-bs-target="#sidebarMenu">
                    <span className="navbar-toggler-icon"></span>
                </button>
            </div>
        </header>

        <div className="row g-0">
            <Sidebar aktif="inventori" />

            <main className="col-md-9 ms-sm-auto col-lg-10 px-3 px-md-4">
                { /* HEADER PROFILE (KONSISTEN) */ }
                <div className="d-flex flex-row justify-content-between align-
items-start pt-4 pb-3 mb-4 border-bottom bg-white mt-3 px-3 rounded-3 shadow-
sm">
                    <div className="text-start">
                        <h1 className="h4 fw-bold m-0 text-dark">Data Stock
Inventori</h1>
                        <small className="text-primary fw-bold d-block">● Manajemen
Logistik & Gudang</small>
                    </div>

                    <div className="d-flex align-items-center bg-light p-2 rounded-
pill px-3 border shadow-sm">
                        <div className="text-end me-2 d-none d-sm-block">
                            <div className="fw-bold text-dark" style={{ fontSize:
'0.75rem', lineHeight: '1' }}>Maessa Andrea</div>

```

```

        <div className="text-muted" style={{ fontSize: '0.65rem'
}}>23051430044</div>
    </div>
    <div className="rounded-circle bg-primary d-flex align-items-
center justify-content-center text-white fw-bold shadow-sm"
        style={{ minWidth: '35px', height: '35px', fontSize:
'0.7rem', border: '2px solid #fff' }}>
        MA
    </div>
</div>
</div>

{/* KARTU STATISTIK (Dihitung Otomatis dari Data API) */}
<div className="row g-3 mb-4">
    <div className="col-6 col-md-4">
        <div className="card border-0 shadow-sm p-3 border-start
border-primary border-4 h-100">
            <small className="text-muted fw-bold mb-1"
style={{fontSize: '0.65rem'}}>TOTAL ITEM</small>
            <h4 className="fw-bold m-0">{dataStok.length} Sku</h4>
        </div>
    </div>
    <div className="col-6 col-md-4">
        <div className="card border-0 shadow-sm p-3 border-start
border-warning border-4 h-100">
            <small className="text-muted fw-bold mb-1"
style={{fontSize: '0.65rem'}}>LOW STOCK</small>
            <h4 className="fw-bold text-warning m-
0">{dataStok.filter(x => x.status === 'Low Stock').length} Item</h4>
        </div>
    </div>
    <div className="col-12 col-md-4">
        <div className="card border-0 shadow-sm p-3 border-start
border-danger border-4 h-100">
            <small className="text-muted fw-bold mb-1"
style={{fontSize: '0.65rem'}}>OUT OF STOCK</small>
            <h4 className="fw-bold text-danger m-0">{dataStok.filter(x
=> x.status === 'Out of Stock').length} Item</h4>
        </div>
    </div>
</div>

{/* TABEL DATA DENGAN KONDISI LOADING/ERROR */}
<div className="card border-0 shadow-sm rounded-3 mb-5 overflow-
hidden">

```

```

        <div className="card-header bg-dark text-white py-3 d-flex
justify-content-between align-items-center">
            <span className="small fw-bold">STOCK MONITORING</span>
            <button className="btn btn-primary btn-sm fw-bold px-3 rounded-
pill" style={{fontSize: '10px'}}>+ Tambah Barang</button>
        </div>
        <div className="table-responsive">
            {loading ? (
                <div className="text-center p-5">
                    <div className="spinner-border text-primary"
role="status"></div>
                    <p className="mt-2 text-muted">Menghubungkan ke
server...</p>
                </div>
            ) : error ? (
                <div className="text-center p-5 text-danger fw-
bold">{error}</div>
            ) : (
                <table className="table table-hover align-middle text-center
mb-0">
                    <thead className="table-light small fw-bold text-
secondary">
                        <tr>
                            <th>ID</th>
                            <th className="text-start ps-4">NAMA ITEM</th>
                            <th>LOKASI</th>
                            <th>STOK</th>
                            <th>STATUS</th>
                        </tr>
                    </thead>
                    <tbody style={{ fontSize: '0.85rem' }}>
                        {dataStok.map((item) => (
                            <tr key={item.id}>
                                <td className="text-muted">#{item.id}</td>
                                <td className="text-start ps-4 fw-bold">
                                    <Link to={`/inventori/${item.id}`} className="text-
primary text-decoration-none">
                                        {item.nama}
                                    </Link>
                                </td>
                                <td>{item.lokasi}</td>
                                <td className="fw-bold">{item.stok}</td>
                                <td>
                                    <span className={`badge rounded-pill ${
item.status === 'Available' ? 'bg-success' :

```

```

            item.status === 'Low Stock' ? 'bg-warning text-
dark' : 'bg-danger'
        } px-3` } style={{fontSize: '0.65rem'}}}>
            {item.status}
        </span>
    </td>
</tr>
</tbody>
</table>
    )}
</div>
</div>
</main>
</div>
</div>
);
}

```

```
export default Inventori;
```

4. JAWABAN KODE LAPORAN.JSX

```

import React from 'react';
import Sidebar from '../Komponen/Sidebar';

function Laporan() {
    const dataQC = [
        { tgl: '2026-05-01', item: 'Baja Karbon', cacat: 'Korosi Ringan', status:
'Reported' },
        { tgl: '2026-05-04', item: 'Mesin CNC-01', cacat: 'Goresan Mata Bor',
status: 'Reported' },
        { tgl: '2026-05-05', item: 'Mesin Press-05', cacat: 'Motor Overheat',
status: 'Critical' },
    ];

    return (
        <div className="container-fluid bg-light min-vh-100 p-0">
            <header className="navbar navbar-dark sticky-top bg-dark p-2 shadow d-
md-none">
                <div className="container-fluid d-flex justify-content-between align-
items-center">
                    <a className="navbar-brand fw-bold m-0" href="#">TI - UNY</a>
                    <button className="navbar-toggler" type="button" data-bs-
toggle="collapse" data-bs-target="#sidebarMenu">
                        <span className="navbar-toggler-icon"></span>
                    </button>
                </div>
            </header>

```

```

    </div>
  </header>

  <div className="row g-0">
    <Sidebar aktif="laporan" />

    <main className="col-md-9 ms-sm-auto col-lg-10 px-3 px-md-4">

      <div className="d-flex flex-row justify-content-between align-items-start pt-4 pb-3 mb-4 border-bottom bg-white mt-3 px-3 rounded-3 shadow-sm">
        <div className="text-start">
          <h1 className="h4 fw-bold m-0 text-dark">Laporan Kualitas
QC</h1>
          <small className="text-danger fw-bold d-block">
            • Quality Control & Maintenance Log
          </small>
        </div>

        <div className="d-flex align-items-center bg-light p-2 rounded-pill px-3 border shadow-sm">
          <div className="text-end me-2 d-none d-sm-block">
            <div className="fw-bold text-dark" style={{ fontSize: '0.75rem', lineHeight: '1' }}>
              Maessa Andrea
            </div>
            <div className="text-muted" style={{ fontSize: '0.65rem' }}>
              23051430044
            </div>
          </div>
          <div className="rounded-circle bg-primary d-flex align-items-center justify-content-center text-white fw-bold shadow-sm" style={{ minWidth: '35px', height: '35px', fontSize: '0.7rem', border: '2px solid #fff' }}>
            MA
          </div>
        </div>

        <div className="row g-3 mb-4">
          <div className="col-6 col-lg-3">
            <div className="card border-0 shadow-sm p-3 bg-white h-100 border-start border-dark border-4">
              <small className="text-muted d-block fw-bold mb-1" style={{fontSize: '0.65rem'}}>TOTAL TEMUAN</small>

```



```

        <h4 className="fw-bold m-0">12 <small className="fs-6">Kasus</small></h4>
      </div>
    </div>
    <div className="col-6 col-lg-3">
      <div className="card border-0 shadow-sm p-3 bg-white h-100 border-start border-success border-4">
        <small className="text-muted d-block fw-bold mb-1" style={{fontSize: '0.65rem'}}>SELESAI</small>
        <h4 className="fw-bold text-success m-0">9 <small className="fs-6 text-muted font-normal">Kasus</small></h4>
      </div>
    </div>
    <div className="col-6 col-lg-3">
      <div className="card border-0 shadow-sm p-3 bg-white h-100 border-start border-warning border-4">
        <small className="text-muted d-block fw-bold mb-1" style={{fontSize: '0.65rem'}}>PENDING</small>
        <h4 className="fw-bold text-warning m-0">2 <small className="fs-6 text-muted font-normal">Kasus</small></h4>
      </div>
    </div>
    <div className="col-6 col-lg-3">
      <div className="card border-0 shadow-sm p-3 bg-white h-100 border-start border-danger border-4">
        <small className="text-muted d-block fw-bold mb-1" style={{fontSize: '0.65rem'}}>URGENT</small>
        <h4 className="fw-bold text-danger m-0">1 <small className="fs-6 text-muted font-normal">Kasus</small></h4>
      </div>
    </div>
  </div>

  <div className="card border-0 shadow-sm rounded-3 mb-5 overflow-hidden">
    <div className="card-header bg-dark text-white py-3 d-flex justify-content-between align-items-center">
      <span className="small fw-bold text-uppercase">Quality Control Logs</span>
      <button className="btn btn-outline-light btn-sm fw-bold px-3 rounded-pill" style={{fontSize: '10px'}} onClick={() => window.print()}>
         CETAK PDF
      </button>
    </div>
  </div>
  <div className="table-responsive">

```

```

        <table className="table table-hover align-middle text-center
mb-0">
          <thead className="table-light small fw-bold text-secondary">
            <tr>
              <th>TANGGAL</th>
              <th className="text-start ps-4">ITEM / MESIN</th>
              <th>JENIS CACAT</th>
              <th>STATUS</th>
            </tr>
          </thead>
          <tbody style={{ fontSize: '0.85rem' }}>
            {dataQC.map((qc, i) => (
              <tr key={i}>
                <td className="text-muted">{qc.tgl}</td>
                <td className="text-start ps-4 fw-bold text-
primary">{qc.item}</td>
                <td className="text-danger fw-bold">{qc.cacat}</td>
                <td>
                  <span className={`badge rounded-pill ${qc.status ===
'Reported' ? 'bg-success' : 'bg-danger'} px-3`} style={{fontSize:
'0.65rem'}}>
                    {qc.status}
                  </span>
                </td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    </div>

    { /* FOOTER */ }
    <footer className="text-center py-4 border-top mt-auto">
      <small className="text-muted">
        &copy; 2026 Lab ERP - <strong>Universitas Negeri
Yogyakarta</strong>
      <br/>
      <span className="text-primary fw-bold">Sistem Monitoring
Terpadu</span>
    </small>
  </footer>

</main>
</div>
</div>
);

```

```

}

export default Laporan;

5. JAWABAN KODE NOTFOUND.JSX
import React from 'react';
import { Link } from 'react-router-dom';

function NotFound() {
  return (
    <div className="d-flex align-items-center justify-content-center vh-100 bg-light">
      <div className="text-center">
        <h1 className="display-1 fw-bold text-dark">404</h1>
        <p className="fs-3"> <span className="text-danger">Oops!</span>
        Halaman tidak ditemukan.</p>
        <p className="lead">
          Alamat yang Maessa cari sepertinya sedang dalam maintenance atau
          salah ketik.
        </p>
        <Link to="/" className="btn btn-primary shadow-sm px-4">
          Kembali ke Dashboard
        </Link>
        <div className="mt-4">
          <small className="text-muted">Sistem Monitoring Teknik
          Industri</small>
        </div>
      </div>
    </div>
  );
}

export default NotFound;

6. JAWABAN KODE GRAFIKDONAT.JSX
import React from 'react';
import { Doughnut } from 'react-chartjs-2';
import { Chart as ChartJS, ArcElement, Tooltip, Legend } from 'chart.js';

ChartJS.register(ArcElement, Tooltip, Legend);

function GrafikDonat() {
  const data = {
    labels: ['Scratch', 'Dent', 'Lainnya'],
    datasets: [
      {

```

```

        data: [50, 30, 20],
        backgroundColor: ['#FF6384', '#36A2EB', '#FFCE56'],
        hoverBackgroundColor: ['#FF6384', '#36A2EB', '#FFCE56'],
        borderWidth: 1,
      },
    ],
  };

  const options = {
    responsive: true,
    maintainAspectRatio: false, // WAJIB false biar tingginya bisa kita atur
    plugins: {
      legend: {
        position: 'bottom', // Di HP lebih rapi kalau legend di bawah
        labels: {
          boxWidth: 12,
          font: { size: 10 }
        }
      }
    },
    cutout: '70%', // Bikin lubang tengahnya lebih gede biar estetik
  };

  return (
    <div style={{ height: '250px', position: 'relative' }}>
      <Doughnut data={data} options={options} />
    </div>
  );
}

export default GrafikDonat;

```

7. JAWABAN KODE GRAFIKPRODUKSI.JSX

```

import React, { useState, useEffect } from 'react'; // TAMBAHKAN useState dan
useEffect DI SINI
import { Bar } from 'react-chartjs-2';
import {
  Chart as ChartJS,
  CategoryScale,
  LinearScale,
  BarElement,
  PointElement,
  LineElement,
  LineController,
  Title,
  Tooltip,

```

```
    Legend,
  } from 'chart.js';

// Registrasi komponen ChartJS (Cukup satu kali saja)
ChartJS.register(
  CategoryScale,
  LinearScale,
  BarElement,
  PointElement,
  LineElement,
  LineController,
  Title,
  Tooltip,
  Legend
);

function GrafikProduksi() {
  // 1. Siapkan State untuk menampung data (awalnya kosong)
  const [dataProduksi, setDataProduksi] = useState([]);
  const [loading, setLoading] = useState(true);

  // 2. Simulasi Fetch API menggunakan useEffect
  useEffect(() => {
    // Fungsi simulasi mengambil data dari server
    const ambilDataPabrik = () => {
      setLoading(true);

      // Simulasi delay jaringan selama 1 detik
      setTimeout(() => {
        // Membuat 6 angka acak antara 100 - 250 (Generator Acak)
        const mockData = Array.from({ length: 6 }, () =>
          Math.floor(Math.random() * 151) + 100);

        setDataProduksi(mockData);
        setLoading(false);
      }, 1000);
    };

    ambilDataPabrik();
  }, []); // [] memastikan hanya jalan 1x saat Dashboard dibuka

  const data = {
    labels: ['08:00', '09:00', '10:00', '11:00', '12:00', '13:00'],
    datasets: [
      {
        label: 'Produksi Aktual (Real-time)',
```

```

        data: dataProduksi, // Data dinamis dari state
        backgroundColor: 'rgba(54, 162, 235, 0.5)',
        borderColor: 'rgba(54, 162, 235, 1)',
        borderWidth: 1,
      },
      {
        label: 'Target Lini',
        data: [180, 180, 180, 180, 180, 180], // Garis target tetap
        type: 'line',
        borderColor: 'rgb(255, 99, 132)',
        borderWidth: 2,
        fill: false,
      }
    ],
  };

const options = {
  responsive: true,
  maintainAspectRatio: false, // WAJIB false agar kita bisa atur tinggi
  sendiri
  plugins: {
    legend: {
      position: 'top',
      labels: { boxWidth: 10, font: { size: 10 } } // Kecilin ukuran legend
      biar gak makan tempat
    },
    title: { display: true, text: 'Monitoring Output Lini 1', font: { size:
12 } }
  },
  scales: {
    y: {
      beginAtZero: true,
      ticks: { font: { size: 10 } }
    },
    x: {
      ticks: { font: { size: 10 } }
    }
  }
};

if (loading) {
  return <div className="text-center p-5"> ⌚ Menghubungkan ke
Mesin...</div>;
}

return (

```

```

    <div className="animate__animated animate__fadeIn">
      { /* Bungkus dengan div yang punya height tetap agar tidak kepanjangan
      */ }
      <div style={{ height: '250px', position: 'relative' }}>
        <Bar data={data} options={options} />
      </div>
      <p className="text-center text-muted mt-2">
        <small style={{ fontSize: '10px' }}>• Live data sinkron dengan sensor
        mesin</small>
      </p>
    </div>
  );
}

```

```
export default GrafikProduksi;
```

8. JAWABAN KODE SIDEBAR.JSX

```

import React from 'react';
import { Link } from 'react-router-dom';

function Sidebar({ aktif }) {
  return (
    <nav id="sidebarMenu" className="col-md-3 col-lg-2 d-md-block bg-dark
    sidebar collapse shadow-lg" style={{ minHeight: '100vh', zIndex: 1000 }}>
      <div className="position-sticky pt-4 px-3">
        { /* LOGO SECTION */ }
        <div className="d-flex align-items-center mb-4 px-2 text-white">
          <h6 className="fw-bold m-0 text-uppercase" style={{ letterSpacing:
          '1px' }}>Teknik Industri</h6>
        </div>

        <hr className="border-secondary mb-4" />

        { /* MENU SECTION */ }
        <ul className="nav flex-column">
          <li className="nav-item mb-2">
            <Link to="/" className={`nav-link d-flex align-items-center px-3
            py-2 ${aktif === 'dashboard' ? 'text-white bg-primary rounded shadow-sm' :
            'text-secondary'}`}>
              <span className="me-2"><img alt="Dashboard icon" data-bbox="565 775 585 790"/></span> Dashboard Utama
            </Link>
          </li>
          <li className="nav-item mb-2">
            <Link to="/inventori" className={`nav-link d-flex align-items-
            center px-3 py-2 ${aktif === 'inventori' ? 'text-white bg-primary rounded
            shadow-sm' : 'text-secondary'}`}>

```

```

        <span className="me-2">📦 </span> Menu Inventori
      </Link>
    </li>
    <li className="nav-item mb-2">
      <Link to="/laporan" className={`nav-link d-flex align-items-
center px-3 py-2 ${aktif === 'laporan' ? 'text-white bg-primary rounded
shadow-sm' : 'text-secondary'} `}>
        <span className="me-2">📄 </span> Menu Laporan
      </Link>
    </li>
  </ul>

  { /* USER PROFILE INFO (Mobile Only) */ }
  <div className="d-md-none mt-5 p-3 bg-secondary rounded text-white
small">
    Operator: Maessa A.V.
  </div>
</div>

<style>{`
  .nav-link { transition: 0.2s; border-radius: 5px; text-decoration:
none; font-size: 0.9rem; }
  .nav-link:hover { color: white !important; background-color:
rgba(255,255,255,0.1); }
`}</style>
</nav>
);
}

export default Sidebar;

```

9. JAWABAN KODE APP.JSX

```

import React from 'react';
import { Routes, Route } from 'react-router-dom';
import Dashboard from './Halaman/Dashboard';
import Inventori from './Halaman/Inventori';
import Laporan from './Halaman/Laporan';
import DetailItem from './Halaman/DetailItem';
import NotFound from './Halaman/NotFound';

function App() {
  return (
    <Routes>
      <Route path="/" element={ <Dashboard /> } />
      <Route path="/inventori" element={ <Inventori /> } />
      <Route path="/inventori/:id" element={ <DetailItem /> } />
    </Routes>
  );
}

```



```
    <Route path="/laporan" element={<Laporan />} />
    <Route path="*" element={<NotFound />} />
  </Routes>
);
}
```

```
export default App;
```

10. JAWABAN KODE APP.CSS

```
/* Perbaikan khusus Layar HP */
@media (max-width: 768px) {
  /* Biar kartu stok menumpuk rapi ke bawah */
  .row {
    display: flex;
    flex-direction: column;
    margin: 0 !important;
  }

  .col-md-4, .col-md-6 {
    width: 100% !important;
    padding: 10px 0;
  }

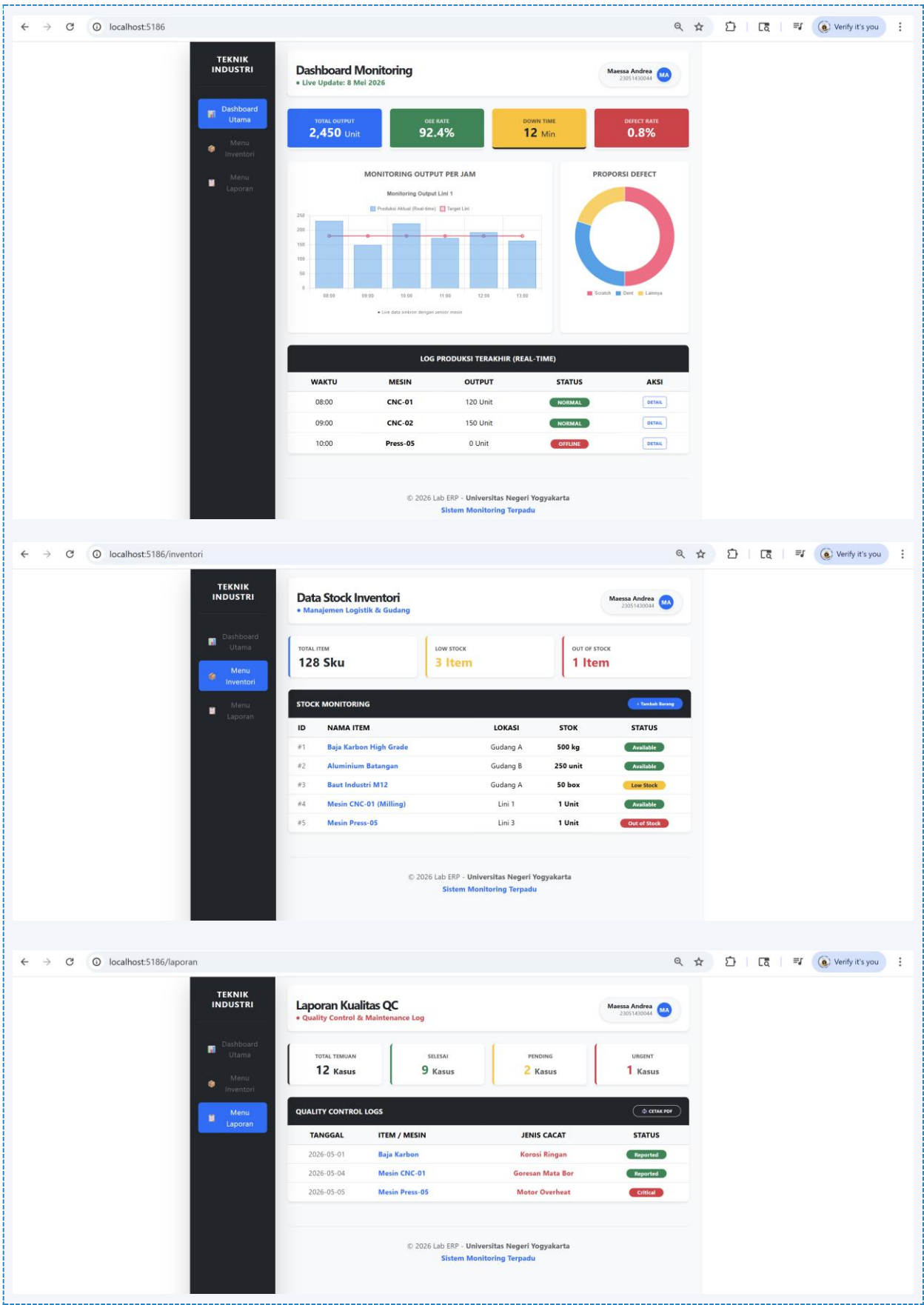
  /* Ngecilin judul biar gak menenuhi layar */
  h2, .h2 {
    font-size: 1.5rem !important;
    text-align: center;
  }

  /* Biar tabel bisa digeser kiri-kanan, gak ngerusak layout */
  .table-responsive {
    display: block;
    width: 100%;
    overflow-x: auto;
    -webkit-overflow-scrolling: touch;
  }
}
```

11. JAWABAN KODE MAIN.JSX

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
import App from './App.jsx';
import './index.css';
import 'bootstrap/dist/css/bootstrap.min.css';
import 'bootstrap/dist/js/bootstrap.bundle.min.js';
```

```
ReactDOM.createRoot(document.getElementById('root')).render(  
  <React.StrictMode>  
    { /* Wrap seluruh App dengan BrowserRouter */ }  
    <BrowserRouter>  
      <App />  
    </BrowserRouter>  
  </React.StrictMode>  
);
```



TEKNIK INDUSTRI

Dashboard Utama

Menu Inventori

Menu Laporan

Data Stock Inventori

Maessa Andrea

23051430044

Manajemen Logistik & Gudang

TOTAL ITEM128 Sku

LOW STOCK3 Item

OUT OF STOCK1 Item

STOCK MONITORING

ID

NAMA ITEM

LOKASI

STOK

STATUS

#1

Baja Karbon High Grade

Gudang A

500 kg

Available

#2

Aluminium Batangan

Gudang B

250 unit

Available

#3

Baut Industri M12

Gudang A

50 box

Low Stock

#4

Mesin CNC-01 (Milling)

Lini 1

1 Unit

Available

#5

Mesin Press-05

Lini 3

1 Unit

Out of Stock

© 2026 Lab ERP - Universitas Negeri Yogyakarta

Sistem Monitoring Terpadu

TEKNIK INDUSTRI

Dashboard Utama

Menu Inventori

Menu Laporan

Laporan Kualitas QC

Maessa Andrea

23051430044

Quality Control & Maintenance Log

TOTAL TEMUAN12 Kasus

SELESAI9 Kasus

PENDING2 Kasus

URGENT1 Kasus

QUALITY CONTROL LOGS

TANGGAL

ITEM / MESIN

JENIS CACAT

STATUS

2026-05-01

Baja Karbon

Korosi Ringan

Reported

2026-05-04

Mesin CNC-01

Goresan Mata Bor

Reported

2026-05-05

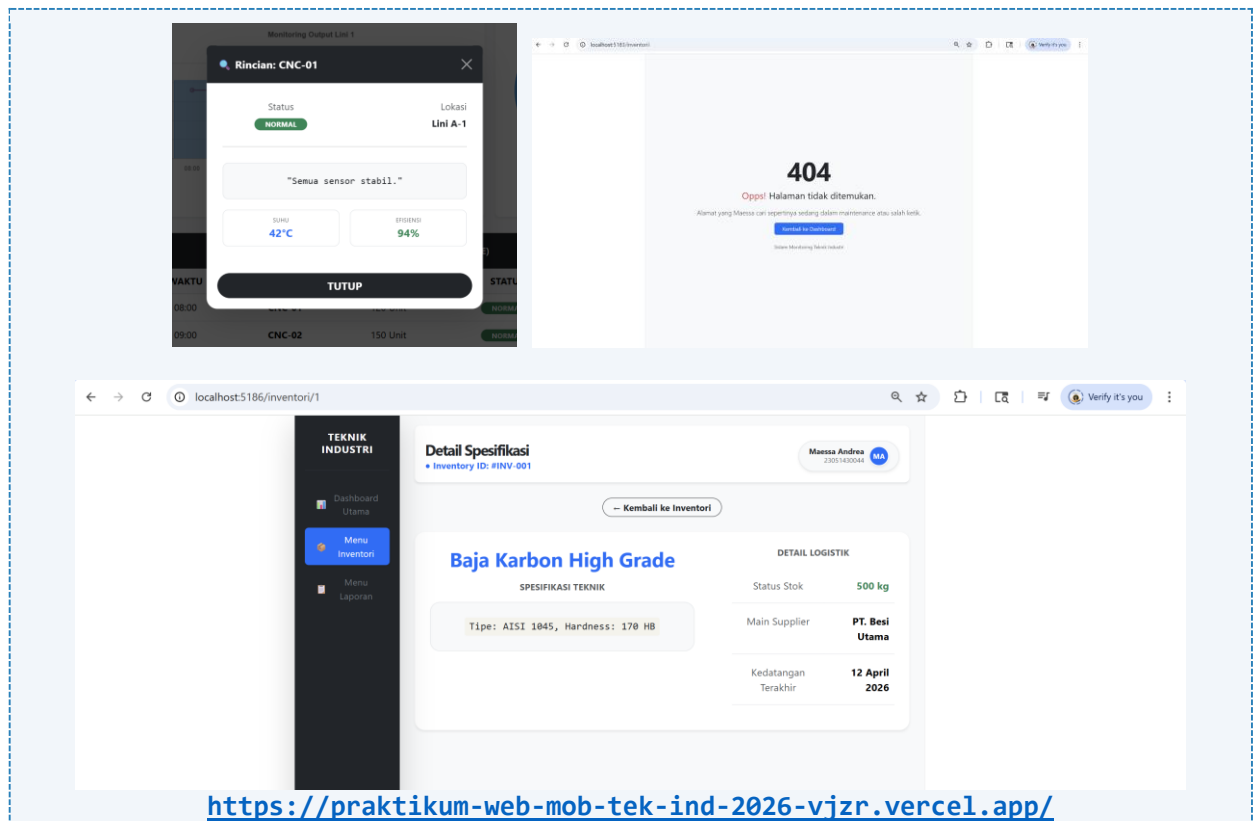
Mesin Press-05

Motor Overheat

Critical

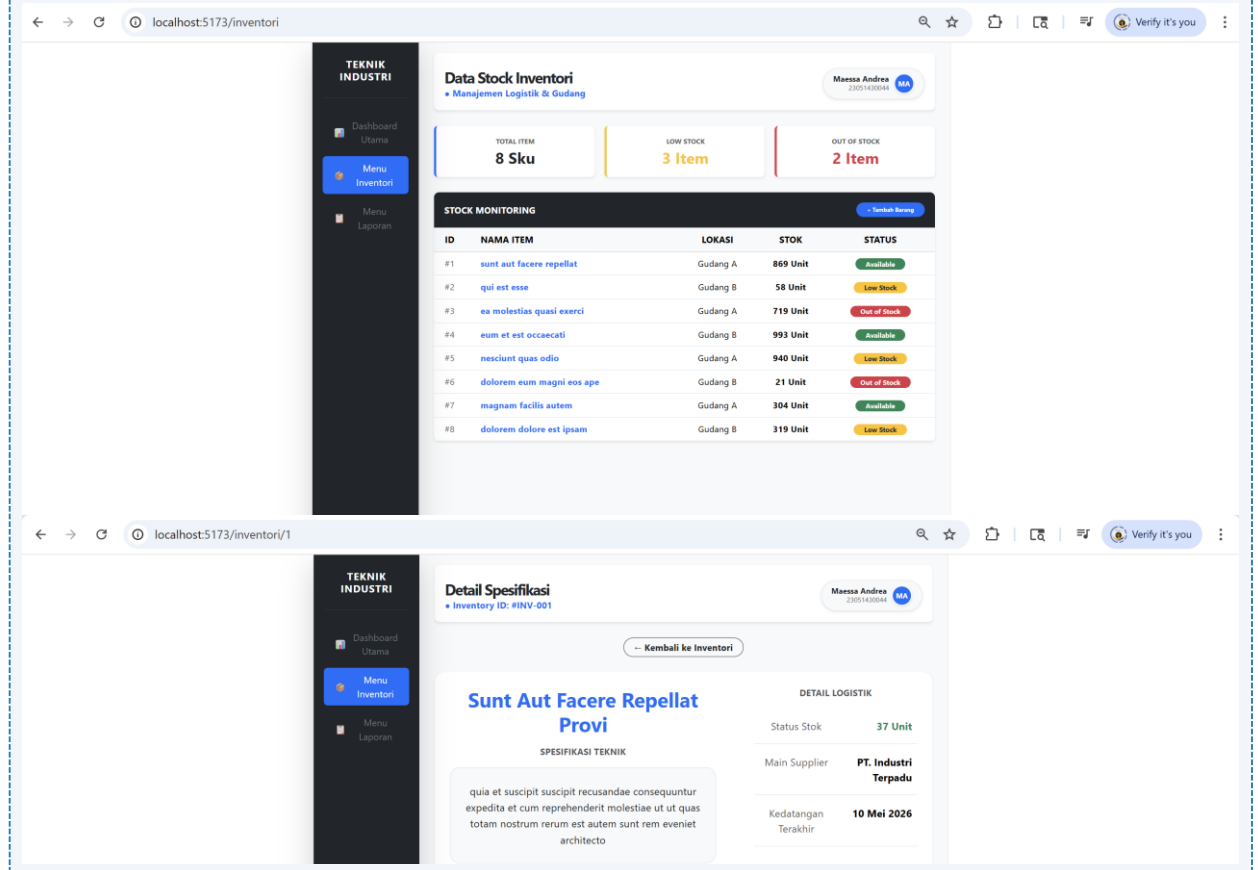
© 2026 Lab ERP - Universitas Negeri Yogyakarta

Sistem Monitoring Terpadu



<https://praktikum-web-mob-tek-ind-2026-vjzr.vercel.app/>

2. Pakai Fetch



Gambar F.3 – Hasil Tugas Akhir Web

G. PEMBAHASAN

Praktikum Pertemuan 12 merupakan tahap lanjutan yang mentransformasi aplikasi React dari komponen fungsional tunggal menjadi sistem Dashboard Monitoring Industri Terintegrasi. Fokus utama pada pertemuan ini adalah penguasaan React Router DOM untuk navigasi multi-halaman, integrasi library Chart.js untuk visualisasi data produksi, serta penerapan Component-Based Architecture yang lebih kompleks dan sistematis.

Pada tahap awal, praktikum ditekankan pada konsep Client-Side Routing. Berbeda dengan navigasi tradisional yang memuat ulang seluruh halaman (reload), penggunaan BrowserRouter, Routes, dan Route memungkinkan perpindahan antarmuka secara instan dan mulus. Implementasi ini diterapkan pada sistem navigasi utama yang menghubungkan halaman Dashboard, Inventori, dan Laporan. Mahasiswa melatih kemampuan dalam mengelola *state* aktif pada sidebar, sehingga pengguna mendapatkan umpan balik visual yang jelas mengenai posisi halaman yang sedang diakses.

Selanjutnya, praktikum mengeksplorasi teknik Visualisasi Data menggunakan library Chart.js. Tahap ini sangat krusial dalam bidang Teknik Industri untuk mengubah data mentah (raw data) menjadi informasi yang mudah dipahami. Penggunaan Bar Chart untuk tren produksi harian dan Doughnut Chart untuk rasio defect produk memberikan simulasi sistem pemantauan lini produksi secara nyata. Di sini, dipelajari pentingnya proses Registrasi Komponen (seperti CategoryScale, LinearScale, dan BarElement) agar library dapat merender grafik secara optimal di lingkungan produksi (*production build*).

Pengembangan dilanjutkan dengan penerapan Conditional Rendering pada tabel log monitoring dan pengelolaan Modal Detail. Mahasiswa mengimplementasikan logika yang memungkinkan aplikasi menampilkan informasi teknis mesin (seperti suhu dan efisiensi) tanpa harus berpindah halaman. Hal ini mensimulasikan efisiensi kerja operator di lapangan dalam memantau parameter kritis mesin. Selain itu, penggunaan *badge* warna otomatis (Hijau untuk Normal, Kuning untuk Maintenance, dan Merah untuk Offline) mempertegas fungsi dashboard sebagai sistem peringatan dini (Early Warning System).

Puncak praktikum ini adalah penyelesaian Proyek Mini Dashboard Monitoring Industri yang telah dioptimalkan untuk perangkat seluler (*Responsive Design*). Proyek ini menggabungkan seluruh konsep lanjut: penggunaan navigasi yang aman dengan halaman 404 Not Found, optimalisasi *layout* menggunakan Bootstrap Grid, dan tahap Deployment ke platform cloud seperti Netlify. Proses *deployment* ini memberikan pemahaman mendalam mengenai siklus hidup pengembangan perangkat lunak, mulai dari penulisan kode di lingkungan lokal hingga aplikasi siap digunakan secara online oleh pengguna melalui alamat URL publik.

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum Pertemuan 12 menunjukkan bahwa implementasi dashboard monitoring telah sepenuhnya sesuai dengan teori Single Page Application (SPA) dan Component-Based Architecture. Penggunaan react-router-dom terbukti efektif dalam mengelola navigasi antar-halaman (Dashboard, Inventori, dan Laporan) tanpa melakukan *full page reload*. Hal ini selaras dengan teori optimasi *User Experience (UX)* pada aplikasi modern, di mana transisi antar-komponen terjadi secara asinkron di sisi klien (*client-side*).

Dalam aspek Visualisasi Data, integrasi library Chart.js telah sesuai dengan prinsip *Data-Driven UI*. Analisis menunjukkan bahwa teori registrasi komponen (seperti *BarElement* dan *LineController*) sangat krusial dalam lingkungan produksi. Kesalahan "Controller not found" yang sempat terjadi selama praktikum membuktikan teori bahwa dalam tahap *production build*, React bersifat sangat ketat terhadap dependensi library. Keberhasilan merender grafik campuran (*Mixed Chart*) membuktikan bahwa tampilan visual (UI) merupakan representasi akurat dari *state* data yang dikelola.

Analisis terhadap Conditional Rendering pada tabel monitoring dan status mesin memperkuat teori *logic-driven UI*. Penggunaan operator ternary untuk menentukan warna *badge* (Normal/Maintenance/Offline) menunjukkan bahwa properti CSS dapat dikendalikan sepenuhnya oleh logika state. Hal ini membuktikan teori bahwa tampilan adalah fungsi dari data ($UI = f(state)$), di mana perubahan status mesin pada basis data secara otomatis merubah indikator visual tanpa intervensi manual pada elemen HTML.

Terakhir, tahap Deployment ke platform Netlify memperkuat teori tentang *Application Lifecycle* dan *Production Environment*. Penggunaan file konfigurasi *_redirects* untuk menangani masalah rute 404 pada server statis merupakan implementasi nyata dari teori navigasi SPA pada *web server*. Sinkronisasi antara kode sumber dengan hasil *build* yang online membuktikan bahwa seluruh fitur—mulai dari grafik interaktif hingga navigasi responsif—telah memenuhi standar kompetensi praktikum dalam menciptakan sistem informasi industri yang siap pakai (*production-ready*).

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Tampilan <i>Sidebar</i> menutupi konten utama dan grafik tidak proporsional saat aplikasi diakses melalui perangkat layar kecil (HP).	Mengimplementasikan fitur <i>Collapse Sidebar</i> menggunakan state boolean dan mengatur properti <i>maintainAspectRatio: false</i> pada opsi grafik agar ukuran chart menyesuaikan tinggi container secara dinamis.

G.3 Kaitan dengan Penerapan di Industri

Materi pada praktikum ini memiliki keterkaitan yang sangat erat dengan penerapan di dunia industri modern, khususnya dalam pengembangan Sistem Monitoring Produksi Terintegrasi, Smart Factory, dan Enterprise Dashboard. Dalam industri manufaktur era 4.0, data performa mesin tidak lagi disajikan dalam bentuk angka mentah saja, melainkan melalui visualisasi dinamis. Konsep Client-Side Routing dan State Management yang dipelajari

memungkinkan pembuatan dashboard pusat kendali (*Control Room*) yang mampu berpindah antar-departemen (Produksi, Inventori, QC) secara instan tanpa hambatan teknis.

Penerapan Visualisasi Data menggunakan Chart.js dalam dashboard ini merupakan instrumen vital bagi manajemen pabrik. Dalam standar global manufaktur, tren produksi yang ditampilkan melalui grafik batang (*Bar Chart*) dan grafik garis (*Line Chart*) membantu manajer untuk melakukan peramalan (*forecasting*) dan mendeteksi anomali produksi secara cepat. Sebagai contoh, grafik target yang bersinggungan dengan grafik realisasi produksi memungkinkan supervisor untuk melakukan intervensi segera jika terjadi penurunan performa di tengah shift, sebuah konsep yang dikenal sebagai Visual Management dalam sistem *Lean Manufacturing*.

Fitur Conditional Rendering (perubahan warna indikator) dan status mesin secara *real-time* sangat relevan sebagai sistem peringatan dini (*Early Warning System*). Integrasi antara logika pemrograman dan antarmuka grafis memudahkan operator dalam memantau kesehatan aset melalui sistem Condition-Based Maintenance. Jika dashboard menunjukkan indikator merah atau status "Maintenance", tim teknisi dapat segera bertindak sebelum terjadi kerusakan fatal (*breakdown*), sehingga meminimalkan biaya perbaikan dan memaksimalkan *uptime* mesin.

Secara keseluruhan, kemampuan untuk mengonlinekan aplikasi melalui platform cloud seperti Netlify memberikan gambaran mengenai implementasi Industrial IoT (IIoT), di mana data produksi dapat diakses oleh pemangku kepentingan kapan saja dan di mana saja melalui perangkat seluler. Pemahaman mendalam mengenai pengembangan aplikasi web yang responsif dan berbasis data ini membekali mahasiswa Teknik Industri untuk merancang solusi digital yang cerdas dan transparan, yang merupakan pilar utama dalam mendukung transformasi digital menuju efisiensi operasional yang maksimal.

H. KESIMPULAN

Tuliskan 3–5 poin kesimpulan yang merangkum hal-hal yang dipelajari dan dicapai pada praktikum modul ini. Kesimpulan harus spesifik, bukan bersifat umum.

1	Mahasiswa mampu mengimplementasikan arsitektur Single Page Application (SPA) menggunakan react-router-dom untuk membangun sistem navigasi multi-halaman (Dashboard, Inventori, Laporan) yang cepat dan efisien tanpa proses <i>reload</i> .
2	Mahasiswa berhasil mengintegrasikan library Chart.js untuk mentransformasi data produksi mentah menjadi visualisasi grafis yang interaktif (Bar & Line Chart), serta memahami prosedur registrasi komponen untuk optimasi <i>production build</i> .
3	Mahasiswa mampu menerapkan teknik Conditional Rendering tingkat lanjut untuk menciptakan <i>Dynamic UI</i> , seperti pengelolaan status mesin secara otomatis dan indikator warna performa (Merah/Kuning/Hijau) berdasarkan logika ambang batas efisiensi industri.

4	Mahasiswa memahami proses Deployment aplikasi ke platform cloud (Netlify), termasuk penanganan konfigurasi server melalui file <code>_redirects</code> guna memastikan sistem <i>routing</i> tetap berjalan stabil saat aplikasi diakses secara publik.
5	Mahasiswa berhasil membangun dashboard monitoring industri yang Responsif dan Data-Driven, yang mengintegrasikan logika manajemen industri (seperti OEE) dengan teknologi web modern untuk mendukung kebutuhan analisis data di era Industri 4.0.

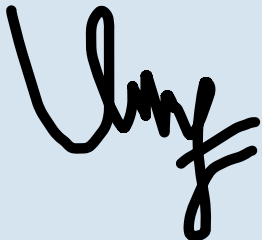
I. DAFTAR PUSTAKA

Tuliskan semua sumber referensi yang digunakan dalam format APA 7th Edition. Minimal 3 referensi yang relevan (termasuk Manual Praktikum dan sumber online resmi seperti MDN, Reactjs.org, dll).

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Chart.js. (2024). <i>Chart.js documentation: General usage</i> . Retrieved from https://www.chartjs.org/docs/latest/
[3]	Kom, I. L. M. (2023). <i>Dasar Algoritma dan Pemrograman Javascript</i> . Irvan Lewenusu.
[4]	Meta Platforms, Inc. (2024). <i>React Reference: useState</i> . Retrieved from https://react.dev/reference/react/useState
[5]	Meta Platforms, Inc. (2024). <i>React Reference: useEffect</i> . Retrieved from https://react.dev/reference/react/useEffect
[6]	Mozilla Developer Network (MDN). (2024). <i>JavaScript: Language overview</i> . Retrieved from https://developer.mozilla.org/en-US/docs/Web/JavaScript
[7]	Patil, K., & Javagal, S. D. (2022). React state management and side-effects—A Review of Hooks. <i>International Research Journal Of Engineering And Technology (IRJET)</i> , 9(01), 12-18.
[8]	Torres, J. A. G., Lau, S. H., Anchuri, P., Stevens, J. M., Tabora, J. E., Li, J., ... & Doyle, A. G. (2022). A multi-objective active learning platform and web app for reaction optimization. <i>Journal of the American Chemical Society</i> , 144(43), 19999-20007.
[9]	Vitejs.dev. (2024). <i>Vite Guide: Why Vite?</i> . Retrieved from https://vitejs.dev/guide/why.html

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa  Maessa Andrea Vallenia NIM: 23051430044	Diperiksa oleh Asisten / Dosen Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____
--	---

MODUL 14

PENGENALAN REACT NATIVE & SETUP EXPO

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 14 (Pertemuan 14)
Topik Utama	Konsep Cross-Platform, Setup Lingkungan Expo, Komponen Dasar (View, Text, Image), dan Styling di Mobile.
Sub-Topik	Setup Lingkungan Kerja Framework (create-expo-app), Mekanisme Sync & Debugging Perangkat (Expo Go via QR Code), Penataan Komponen Struktural Layout (View & SafeAreaView), Manipulasi Tipografi dan Handling Konten Panjang (Text & ScrollView), Rekayasa Visual berbasis Flexbox Mobile (StyleSheet & flexDirection), dan Finalisasi Cross-Platform Styling untuk Adaptasi Tampilan Android & iOS secara Real-time.
Durasi Praktikum	340 Menit
Tanggal Praktikum	13 Mei 2026
Nama Mahasiswa	Maessa Andrea Vallenia
NIM	23051430044
Kelas	A

B. TUJUAN PEMBELAJARAN

Tuliskan tujuan pembelajaran yang ingin dicapai pada modul ini, sesuai dengan yang tertera di Manual Praktikum.

1	Memahami perbedaan antara Web Development (React.js) dan Mobile Development (React Native).
2	Mampu menyiapkan lingkungan kerja React Native menggunakan Expo.
3	Menguasai komponen dasar pengganti HTML: `View` (div), `Text` (p/h1), `ScrollView`.
4	Mampu menata tampilan mobile menggunakan `StyleSheet`.

C. ALAT DAN BAHAN

Sebutkan seluruh perangkat keras, perangkat lunak, dan sumber referensi yang digunakan selama praktikum.

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	<i>Code Editor</i>
2	Google Chrome / Browser Modern	Versi terbaru	<i>Testing & Debugging</i>
3	Node.js	Versi terbaru	<i>Runtime environment</i> utama untuk mengeksekusi perintah CLI dan manajemen paket JavaScript
4	Expo CLI / create-expo-app	Versi terbaru	<i>Framework wrapper</i> untuk menginisialisasi arsitektur proyek mobile dan kompilasi <i>single codebase</i>
5	Aplikasi Expo Go	Versi terbaru	Aplikasi <i>client</i> pada gawai untuk memuat dan menjalankan proyek React Native tanpa emulator berat
6	Git dan GitHub	Versi 2.53.0	Version control system (VCS) berbasis Git
7	Koneksi Internet	Stabil	Mengakses Library dan Framework
8	Metro Bundler	Bawaan Expo	Server pengembangan lokal

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
			(<i>development server</i>) untuk transpilasi kode dan fitur <i>Fast Refresh</i>
9	React Native Library	Bawaan SDK Expo Terbaru	Framework inti berbasis komponen native untuk menyusun elemen visual dashboard gawai
10	File praktikum	Perrtemuan sebelumnya	Digunakan sebagai dasar pengembangan panel kontrol
11	LocalStorage (Web Storage API)	Built-in Browser	Digunakan untuk menyimpan data produksi secara persisten
12	Fetch API	Built-in Browser	Digunakan untuk melakukan HTTP request (GET, POST, dll) ke server

D. DASAR TEORI

1. Cross-Platform Development merupakan proses membangun aplikasi mobile untuk sistem operasi Android dan iOS secara simultan menggunakan satu basis kode (single codebase) JavaScript. Konsep ini sangat efisien untuk mobilitas operator di lantai pabrik karena mengeliminasi kebutuhan menulis dua kode program yang berbeda untuk jenis perangkat gawai yang berbeda.
2. Managed Workflow Expo adalah ekosistem pendukung React Native yang bertindak sebagai pembungkus (wrapper) untuk mengabstraksi kompleksitas konfigurasi native (Java/Swift). Framework ini memudahkan

pengembang dalam membuat aplikasi seluler siap pakai tanpa harus mengonfigurasi dokumen tingkat rendah Android atau iOS secara manual dari awal.

3. Metro Bundler merupakan server pengembangan lokal (development server) wajib bawaan Expo yang bertindak untuk mentranspilasi dan mengompilasi kode program. Proses ini bertujuan menyajikan file JavaScript ke perangkat gawai secara real-time melalui pemindaian QR Code di aplikasi Expo Go, sehingga setiap perubahan kode langsung ter-refresh otomatis.
4. Komponen Struktural Native adalah elemen antarmuka dasar pengganti tag HTML web. Komponen `<View>` bertindak sebagai kontainer utama pengganti `<div>`, sedangkan komponen `<Text>` merupakan wadah wajib untuk menampilkan string murni pengganti tag `<p>` atau `<h1>` guna menghindari kegagalan sistem (crash) pada aplikasi mobile.
5. `SafeAreaView` dan Platform API adalah teknik proteksi tata letak antarmuka agar konten tidak terpotong oleh interupsi fisik layar gawai modern. Melalui pengaturan padding otomatis yang mendeteksi sistem operasi (Android atau iOS), area sensitif seperti notch, lubang kamera, dan status bar perangkat akan tetap aman dari tumpang tindih komponen UI.
6. `Scrollable Interface` adalah metode penanganan limitasi ukuran layar gawai dengan mengintegrasikan komponen `<ScrollView>`. Berbeda dengan layout web yang otomatis menyediakan scrollbar, arsitektur mobile membutuhkan pembungkus mandiri ini agar tumpukan data yang melebihi batas vertikal layar ponsel tetap dapat digulir ke bawah oleh pengguna.
7. `Flexbox Mobile` dan `StyleSheet` merupakan pengatur gaya visual dan tata letak utama berbasis objek JavaScript menggunakan format camelCase. Dengan karakteristik default berarah vertikal (`flexDirection: 'column'`), sistem ini dipadukan dengan properti `elevation` dan `shadow` untuk menjamin responsivitas serta konsistensi efek visual komponen saat dijalankan lintas platform.

Secara keseluruhan, penguasaan komponen dasar mobile yang dipadukan dengan layouting `Flexbox` dan ekosistem Expo membuat aplikasi monitoring gudang menjadi alat bantu yang ringan, responsif, serta andal dalam mendukung mobilitas operasional industri secara nyata.

D.1 Konsep Utama

Konsep utama pada pertemuan ini adalah pengembangan aplikasi seluler lintas platform (*cross-platform mobile development*) menggunakan satu basis kode (*single codebase*) JavaScript untuk menunjang mobilitas operasional industri. Pendekatan ini berfungsi sebagai solusi penyediaan sistem monitoring yang ringan dan portabel bagi operator di lantai pabrik, sehingga mereka dapat memantau status area kerja langsung melalui perangkat genggam seperti tablet atau PDA tanpa terikat pada komputer desktop.

Penggunaan ekosistem Managed Workflow Expo menjadi inti utama dalam praktikum karena framework ini bertindak sebagai pembungkus (*wrapper*) yang mengabstraksi kerumitan konfigurasi native Android dan iOS. Dengan memanfaatkan antarmuka CLI dari *create-expo-app* dan server lokal Metro Bundler, aplikasi dapat dikompilasi secara instan serta disinkronisasikan ke perangkat gawai fisik secara *real-time* melalui fitur *Fast Refresh* setelah memindai QR Code pada aplikasi Expo Go.

Selain arsitektur pengembangan, praktikum ini juga menerapkan transformasi komponen dari lingkungan web ke lingkungan mobile native. Konsep ini menggantikan tag HTML tradisional dengan komponen khusus React Native, seperti mengubah kontainer `<div>` menjadi `<View>`, tag tulisan `<p>` atau `<h1>` menjadi `<Text>`, serta pergeseran penanganan aksi dari yang semula berbasis klik (*onClick*) pada peramban web menjadi berbasis sentuhan (*onPress*) pada layar gawai.

Konsep penataan layout berbasis Flexbox Mobile juga menjadi bagian penting dalam menyusun elemen antarmuka dashboard gawai. Berbeda dengan tata letak web, arsitektur mobile memiliki karakteristik baku berupa arah distribusi elemen secara vertikal (*column*), sehingga pengembang dituntut untuk menguasai rekayasa properti *flexDirection* guna membagi ruang secara proporsional dan menyusun informasi kartu kapasitas gudang secara horizontal maupun vertikal.

Sintaksis StyleSheet terpusat digunakan untuk mengelola gaya visual seluruh komponen antarmuka dengan mengadopsi format *camelCase* objek JavaScript. Atribut visual seperti *backgroundColor*, dimensi kartu, hingga pembatasan penulisan string murni di dalam kontainer teks dipatuhi secara ketat untuk menjaga stabilitas program sekaligus menghindari terjadinya kegagalan sistem (*crash*) pada aplikasi saat dijalankan di perangkat seluler.

Konsep proteksi area tampilan dan adaptasi visual lintas platform diterapkan untuk menjamin kenyamanan pengguna gawai. Melalui integrasi komponen `<SafeAreaView>` dan komponen `<ScrollView>`, antarmuka dashboard dapat mengamankan konten dari interupsi fisik layar modern seperti *notch* atau *status bar*, sekaligus memberikan fleksibilitas akses berupa gerakan gulir (*scrolling*) saat tumpukan data profil mesin melebihi dimensi vertikal layar ponsel.

Dengan menggabungkan ekosistem framework Expo, manajemen layout berbasis Flexbox Mobile, dan adaptasi komponen native lintas platform, aplikasi seluler berhasil dikembangkan menjadi sistem monitoring gudang yang profesional, interaktif, dan mampu mendukung efisiensi pengawasan operasional manufaktur secara *real-time*.

D.2 Keterkaitan dengan Teknik Industri

Materi pengembangan aplikasi mobile menggunakan React Native dan Expo memiliki hubungan yang sangat erat dengan bidang Teknik Industri, khususnya dalam era manufaktur cerdas dan *Internet of Things* (IoT). Sistem rantai pabrik yang dinamis memerlukan alat pengawasan yang memiliki mobilitas tinggi, di mana data operasional gudang dan lini produksi tidak lagi dipantau secara statis di ruang kontrol, melainkan dapat diakses secara *real-time* oleh supervisor atau operator yang bergerak aktif di lapangan.

Penggunaan arsitektur *cross-platform* berbasis ponsel genggam atau PDA (*Personal Digital Assistant*) sangat relevan untuk meningkatkan efisiensi waktu kerja (*standard time*). Dengan satu basis kode aplikasi yang dapat berjalan di Android maupun iOS, perusahaan manufaktur dapat memangkas biaya investasi perangkat keras dan mempercepat distribusi sistem informasi manajemen ke seluruh level organisasi tanpa hambatan kompatibilitas perangkat.

Penerapan komponen visual seperti kartu kapasitas gudang (*Warehouse Capacity Card*) pada aplikasi mobile mendukung implementasi konsep *Visual Management* di area penyimpanan. Melalui perbedaan gaya visual dinamis antara kondisi normal (kapasitas 85%) dan kondisi kritis (kapasitas 95% penuh), operator gudang dapat melakukan tindakan pencegahan (*preventive action*) penumpukan material secara instan tanpa perlu melakukan inspeksi fisik manual yang memakan waktu.

Integrasi komponen interaktif seperti tombol berbasis sentuhan (*TouchableOpacity*) dan notifikasi pesan instan (*Alert API*) mencerminkan prinsip sistem *Andon* modern. Ketika terjadi ketidaksesuaian kapasitas atau kendala pasokan material di area gudang, operator dapat langsung menekan tombol detail pada gawai mereka untuk memicu respons penanganan material (*material handling*) secara cepat demi menjaga kelancaran aliran produksi (*production flow*).

Penguasaan sistem tata letak Flexbox Mobile untuk menyusun profil mesin secara horizontal maupun vertikal mencerminkan prinsip ergonomi antarmuka (*UI/UX Ergonomics*). Seorang Industrial Engineer harus mampu merancang tata letak aplikasi gawai yang nyaman dipandang, mudah dioperasikan dengan satu tangan, dan memiliki keterbacaan data yang tinggi guna meminimalkan risiko kesalahan input (*human error*) oleh operator di lingkungan kerja yang padat.

Konsep proteksi konten menggunakan pembungkus area aman (*SafeAreaView*) dan area gulir (*ScrollView*) menunjukkan pemahaman terhadap aspek keandalan sistem (*system reliability*). Penataan ini memastikan bahwa aplikasi monitoring dapat beradaptasi secara fleksibel terhadap berbagai resolusi layar gawai yang digunakan di lantai pabrik, sehingga tidak ada informasi kritis mengenai parameter utilitas mesin yang terpotong saat pengawasan dilakukan.

Dengan memahami perancangan aplikasi mobile berbasis Expo ini, mahasiswa Teknik Industri memiliki bekal kompetensi yang krusial untuk menjembatani sistem informasi digital dengan aktivitas fisik manufaktur. Kemampuan merancang aplikasi monitoring portabel ini menjadi modal kuat dalam menciptakan ekosistem kerja yang tangkas, responsif, dan mampu mendukung proses Pengambilan Keputusan Berbasis Data (*data-driven decision making*) di era industri modern.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

Dokumentasikan setiap langkah kerja yang Anda lakukan, beserta hasil yang diperoleh (screenshot, kode, atau output). Jawab sesuai urutan langkah di Manual Praktikum.

E.1 Langkah 1 – Membuat Project Expo

Buka Terminal/Command Prompt di komputer. Jalankan perintah berikut (tunggu proses instalasi sekitar 2-5 menit):

- 1) Membuat project baru bernama 'SistemGudangMobile'
``npx create-expo-app SistemGudangMobile``
- 2) Masuk ke folder
``cd SistemGudangMobile``
- 3) Menjalankan server development
``npx expo start``

Setelah muncul QR Code di terminal:

- 1) Buka aplikasi Expo Go di HP Android/iOS Anda.
- 2) Scan QR Code tersebut (Pastikan HP dan Laptop terhubung ke jaringan WiFi yang sama).
- 3) Aplikasi akan terbuka di HP dan menampilkan tulisan "Open up App.js to start working on your app!".

```
/* ===== KODE / OUTPUT LANGKAH 1 ===== */
```

1. Project Baru 'SistemGudangMobile'

```
D:\PraktikumWebMob_TekInd_2026\PraktikumWebMob_TekInd_2026\Pertemuan13_23051430044_MaessaAndreaVallenia> npx create-expo-app SistemGudangMobile
Creating an Expo project using the default template.
```

To choose from all available templates (<https://github.com/expo/expo/tree/main/templates>) pass in the --template arg:

```
$ npx create-expo-app --template
```

To choose from all available examples (<https://github.com/expo/examples>) pass in the --example arg:

```
$ npx create-expo-app --example
```

✓ Downloaded and extracted project files.

```
> npm install
```

```
npm warn deprecated inflight@1.0.6: This module is not supported, and leaks memory. Do not use it. Check out lru-cache if you want a good and tested way to coalesce async requests by a key value, which is much more comprehensive and powerful.
```

```
npm warn deprecated uuid@7.0.3: uuid@10 and below is no longer supported. For ESM codebases, update to uuid@latest. For CommonJS codebases, use uuid@11 (but be aware this version will likely be deprecated in 2028).
```

```
npm warn deprecated rimraf@3.0.2: Rimraf versions prior to v4 are no longer supported
```

```
npm warn deprecated glob@7.2.3: Old versions of glob are not supported, and contain widely publicized security vulnerabilities, which have been fixed in the current version. Please update. Support for old versions may be purchased (at exorbitant rates) by contacting i@izs.me
```

```
npm warn deprecated glob@7.2.3: Old versions of glob are not supported, and contain widely publicized security vulnerabilities, which have been fixed in the current version. Please update. Support for old versions may be purchased (at exorbitant rates) by contacting i@izs.me
```



```
npm warn deprecated glob@7.2.3: Old versions of glob are not supported, and contain widely publicized security vulnerabilities, which have been fixed in the current version. Please update. Support for old versions may be purchased (at exorbitant rates) by contacting i@izs.me
npm warn deprecated glob@7.2.3: Old versions of glob are not supported, and contain widely publicized security vulnerabilities, which have been fixed in the current version. Please update. Support for old versions may be purchased (at exorbitant rates) by contacting i@izs.me
```

```
added 911 packages, and audited 912 packages in 3m
```

```
166 packages are looking for funding
  run `npm fund` for details
```

```
4 moderate severity vulnerabilities
```

```
To address all issues (including breaking changes), run:
  npm audit fix --force
```

```
Run `npm audit` for details.
```

```
✓ Your project is ready!
```

```
To run your project, navigate to the directory and run one of the following npm commands.
```

```
- cd SistemGudangMobile
- npm run android
- npm run ios # you need to use macOS to build the iOS project - use the Expo app if you need to do iOS development without a Mac
- npm run web
```

2. Masuk Folder

```
PS
```

```
D:\PraktikumWebMob_TekInd_2026\PraktikumWebMob_TekInd_2026\Pertemuan13_23051430044_MaessaAndreaVallenia> cd SistemGudangMobile
```

3. Menjalankan Server Development

```
D:\PraktikumWebMob_TekInd_2026\PraktikumWebMob_TekInd_2026\Pertemuan13_23051430044_MaessaAndreaVallenia\SistemGudangMobile> npm run start
```

```
> sistemgudangmobile@1.0.0 start
> expo start
```

```
Starting project at
```

```
D:\PraktikumWebMob_TekInd_2026\PraktikumWebMob_TekInd_2026\Pertemuan13_23051430044_MaessaAndreaVallenia\SistemGudangMobile
```

```
› Port 8081 is being used by another process
```

```
√ Use port 8082 instead? ... yes
```

```
React Compiler enabled
```

```
Starting Metro Bundler
```



- › Metro waiting on exp://192.168.11.183:8082
- › Scan the QR code above with Expo Go (Android) or the Camera app (iOS)

› Web is waiting on http://localhost:8082

- › Using Expo Go
- › Press s | switch to development build

- › Press a | open Android
- › Press w | open web

- › Press j | open debugger
- › Press r | reload app
- › Press m | toggle menu
- › shift+m | more tools
- › Press o | open project code in your editor

- › Press ? | show all commands

Logs for your project will appear below. Press Ctrl+C to exit.

iOS Bundled 17658ms node_modules\expo-router\entry.js (1434 modules)

[metro-file-map] Cache write error

: Error: ENOSPC: no space left on device, write

at write (node:internal/fs/promises:745:8)

at writeFileHandle (node:internal/fs/promises:502:7)

at

D:\PraktikumWebMob_TekInd_2026\PraktikumWebMob_TekInd_2026\Pertemuan13_23051430044_

MaessaAndreaVallenia\SistemGudangMobile\node_modules\metro-file-

map\src\cache\DiskCacheManager.js:80:11 {

errno: -4055,

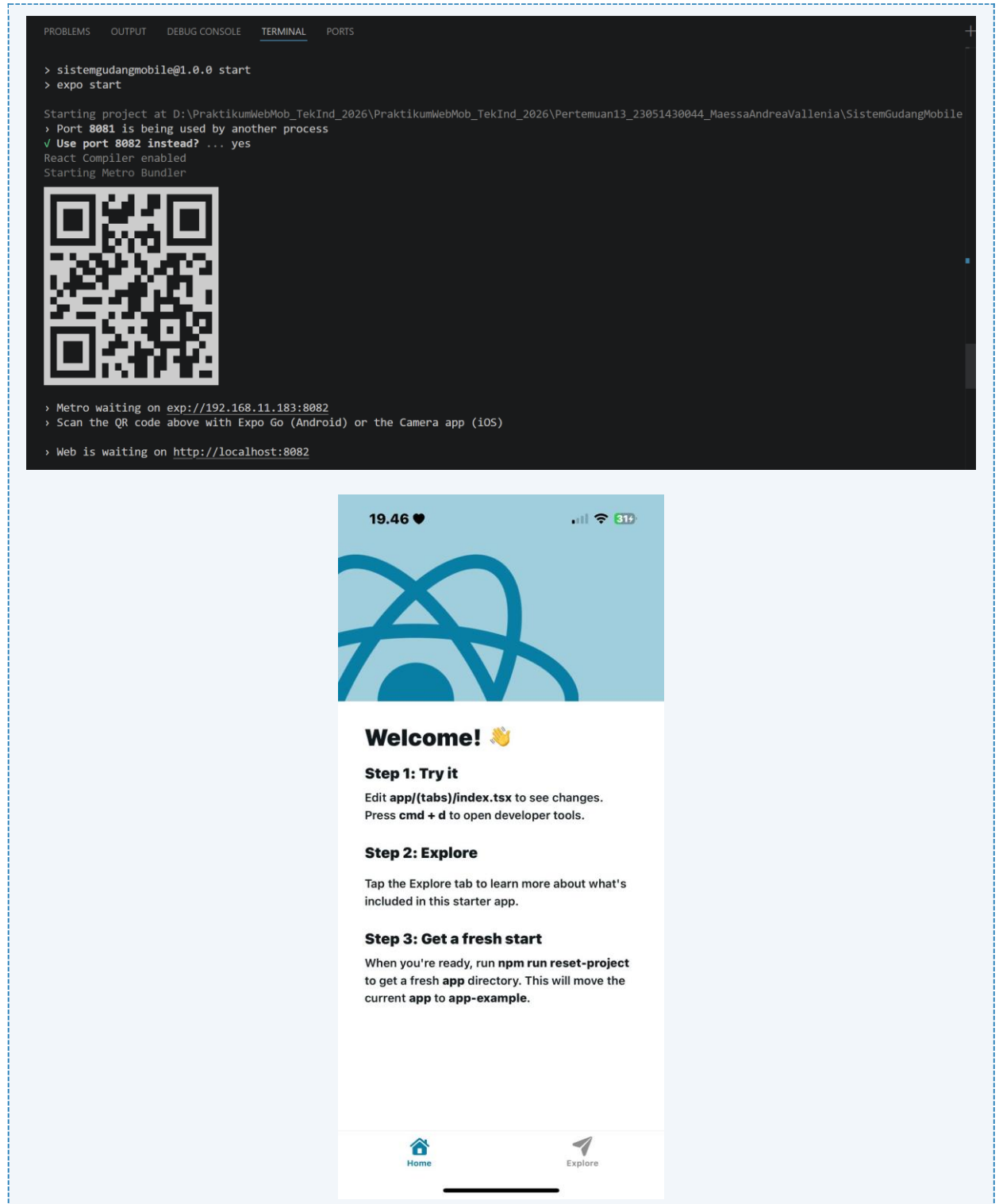
code: 'ENOSPC',

syscall: 'write'

}

WARN SafeAreaView has been deprecated and will be removed in a future release. Please use 'react-native-safe-area-context' instead. See <https://github.com/th3rdwave/react-native-safe-area-context>

Screenshot Hasil:



Gambar E.1 – Hasil Membuat Project Expo

E.2 Langkah 2 – Memodifikasi App.js (UI Dashboard Sederhana)

Buka file `App.js` di folder project menggunakan VS Code. Hapus semua isinya, dan ganti dengan kode berikut. Kita akan membuat tampilan Header sederhana.

```
/* ===== KODE / OUTPUT LANGKAH 2 ===== */

import { StatusBar } from 'expo-status-bar';
import { StyleSheet, Text, View, SafeAreaView, Platform } from 'react-native';

export default function App() {
  return (
    <SafeAreaView style={styles.container}>
      <StatusBar style="auto" />

      {/* Bagian Header */}
      <View style={styles.header}>
        <Text style={styles.headerTitle}>PT. Manufaktur Maju</Text>
        <Text style={styles.headerSubtitle}>Aplikasi Monitoring Gudang</Text>
      </View>

      {/* Bagian Konten Utama */}
      <View style={styles.content}>
        <Text style={styles.welcomeText}>Selamat Datang, Operator!</Text>

        <View style={styles.card}>
          <Text style={styles.cardTitle}>Status Gudang A</Text>
          <Text style={styles.cardValue}>Kapasitas: 85%</Text>
          <Text style={styles.cardStatus}>TERSEDIA</Text>
        </View>

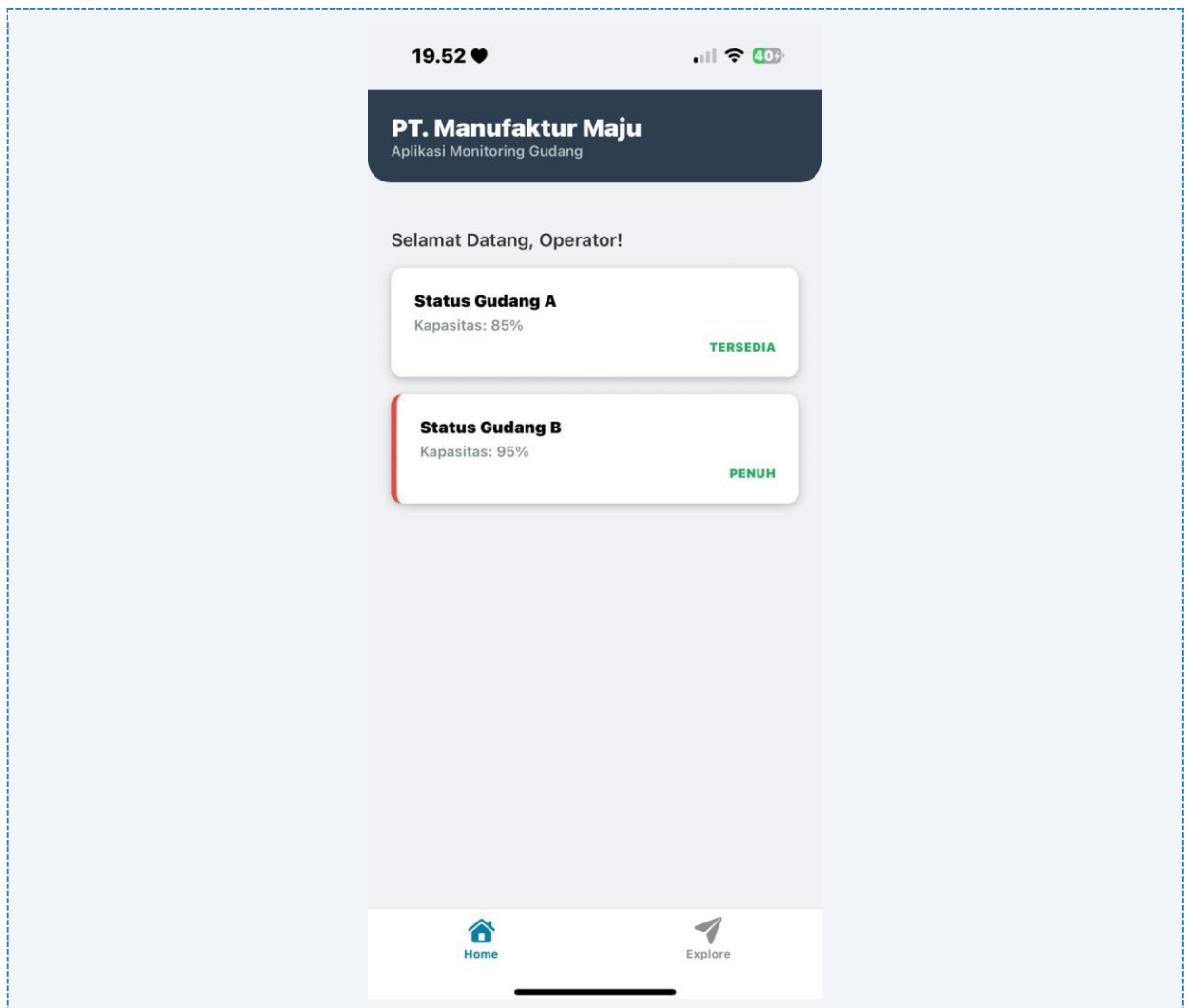
        <View style={[styles.card, styles.cardWarning]}>
          <Text style={styles.cardTitle}>Status Gudang B</Text>
          <Text style={styles.cardValue}>Kapasitas: 95%</Text>
          <Text style={styles.cardStatus}>PENUH</Text>
        </View>
      </View>
    </SafeAreaView>
  );
}

// Styling menggunakan StyleSheet (seperti CSS tapi object JS)
const styles = StyleSheet.create({
  container: {
    flex: 1, // Artinya layar penuh
    backgroundColor: '#f0f2f5',
  },
});
```

```
paddingTop: Platform.OS === 'android' ? 25 : 0, // Padding khusus Android
status bar
},
header: {
  backgroundColor: '#2c3e50',
  padding: 20,
  borderBottomLeftRadius: 20,
  borderBottomRightRadius: 20,
  marginBottom: 20,
  elevation: 5, // Shadow di Android
},
headerTitle: {
  color: 'white',
  fontSize: 24,
  fontWeight: 'bold',
},
headerSubtitle: {
  color: '#bdc3c7',
  fontSize: 14,
},
content: {
  padding: 20,
},
welcomeText: {
  fontSize: 18,
  marginBottom: 15,
  color: '#333',
},
card: {
  backgroundColor: 'white',
  padding: 20,
  borderRadius: 10,
  marginBottom: 15,
  // Shadow khusus iOS dan Android
  shadowColor: "#000",
  shadowOffset: { width: 0, height: 2 },
  shadowOpacity: 0.25,
  shadowRadius: 3.84,
  elevation: 5,
},
cardWarning: {
  borderLeftWidth: 5,
  borderLeftColor: '#e74c3c',
},
cardTitle: {
  fontSize: 16,
```

```
fontWeight: 'bold',
marginBottom: 5,
},
cardValue: {
  fontSize: 14,
  color: '#7f8c8d',
},
},
cardStatus: {
  fontSize: 12,
  fontWeight: 'bold',
  color: '#27ae60',
  marginTop: 5,
  textAlign: 'right'
}
});
```

Screenshot Hasil:



Gambar E.2 – Hasil Memodifikasi App.js (UI Dashboard Sederhana)

E.3 Langkah 3 – Menambahkan Interaksi (Button & Alert)

React Native tidak punya tag ``, kita menggunakan komponen `Button` atau `TouchableOpacity`. Kita akan coba menampilkan pesan alert saat kartu ditekan.

Ubah import di paling atas dan ubah salah satu kartu (Gudang A) menjadi bisa diklik:

```
/* ===== KODE / OUTPUT LANGKAH 3 ===== */

import { StatusBar } from 'expo-status-bar';
import { StyleSheet, Text, View, SafeAreaView, Platform, TouchableOpacity,
Alert } from 'react-native';

export default function App() {
  return (
    <SafeAreaView style={styles.container}>
      <StatusBar style="auto" />

      {/* Bagian Header */}
      <View style={styles.header}>
        <Text style={styles.headerTitle}>PT. Manufaktur Maju</Text>
        <Text style={styles.headerSubtitle}>Aplikasi Monitoring Gudang</Text>
      </View>

      {/* Bagian Konten Utama */}
      <View style={styles.content}>
        <Text style={styles.welcomeText}>Selamat Datang, Operator!</Text>

        {/* Kartu Gudang A yang Sudah Ditambahkan Interaksi Klik */}
        <TouchableOpacity
          style={styles.card}
          onPress={() => Alert.alert("Info", "Membuka Detail Stok Gudang
A...")}
        >
          <Text style={styles.cardTitle}>Status Gudang A</Text>
          <Text style={styles.cardValue}>Kapasitas: 85%</Text>
          <Text style={styles.cardStatus}>TEKAN UNTUK DETAIL</Text>
        </TouchableOpacity>

        {/* Kartu Gudang B */}
        <View style={[styles.card, styles.cardWarning]}>
          <Text style={styles.cardTitle}>Status Gudang B</Text>
          <Text style={styles.cardValue}>Kapasitas: 95%</Text>
          <Text style={styles.cardStatus}>PENUH</Text>
        </View>
      </View>
    </SafeAreaView>
  );
}
```

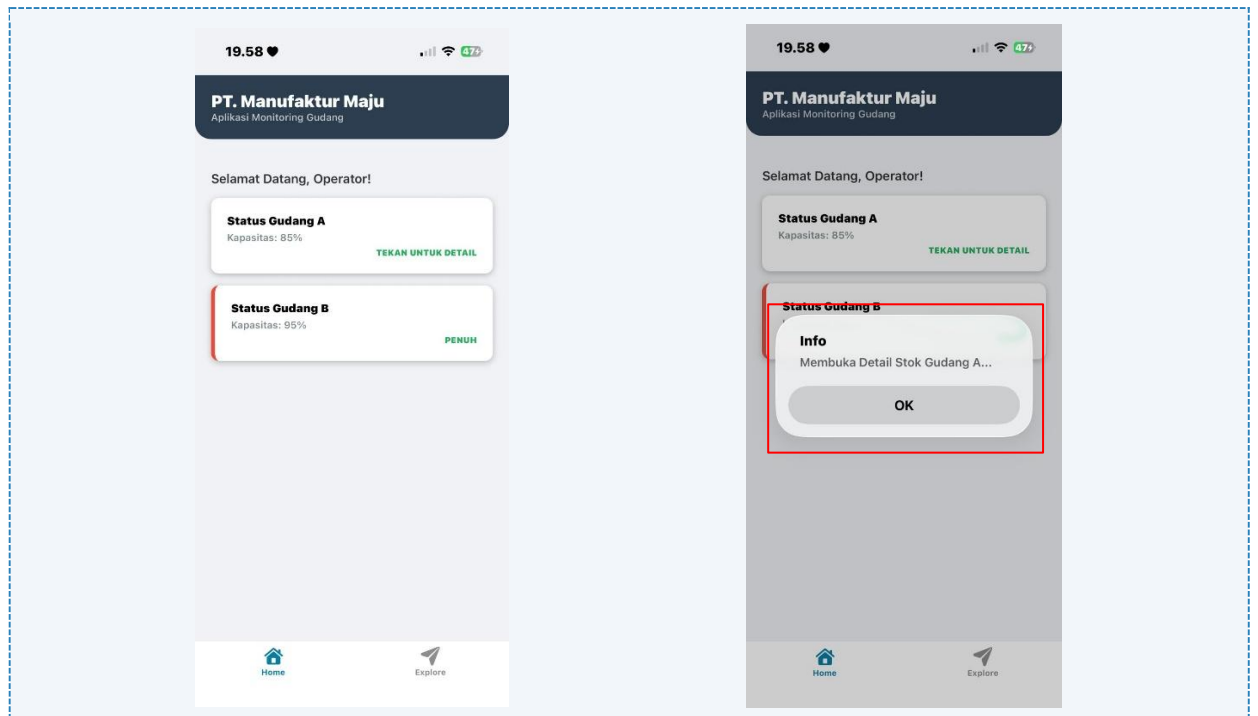
```
    </SafeAreaView>
  );
}

// Styling menggunakan StyleSheet
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#f0f2f5',
    paddingTop: Platform.OS === 'android' ? 25 : 0,
  },
  header: {
    backgroundColor: '#2c3e50',
    padding: 20,
    borderBottomLeftRadius: 20,
    borderBottomRightRadius: 20,
    marginBottom: 20,
    elevation: 5,
  },
  headerTitle: {
    color: 'white',
    fontSize: 24,
    fontWeight: 'bold',
  },
  headerSubtitle: {
    color: '#bdc3c7',
    fontSize: 14,
  },
  content: {
    padding: 20,
  },
  welcomeText: {
    fontSize: 18,
    marginBottom: 15,
    color: '#333',
  },
  card: {
    backgroundColor: 'white',
    padding: 20,
    borderRadius: 10,
    marginBottom: 15,
    shadowColor: "#000",
    shadowOffset: { width: 0, height: 2 },
    shadowOpacity: 0.25,
    shadowRadius: 3.84,
    elevation: 5,
  },
});
```



```
},
cardWarning: {
  borderLeftWidth: 5,
  borderLeftColor: '#e74c3c',
},
cardTitle: {
  fontSize: 16,
  fontWeight: 'bold',
  marginBottom: 5,
},
cardValue: {
  fontSize: 14,
  color: '#7f8c8d',
},
cardStatus: {
  fontSize: 12,
  fontWeight: 'bold',
  color: '#27ae60',
  marginTop: 5,
  textAlign: 'right'
}
});
```

Screenshot Hasil:



Gambar E.3 – Hasil Menambahkan Interaksi (Button & Alert)

F. LATIHAN DAN TUGAS MANDIRI

Jawab dan dokumentasikan seluruh latihan/tugas yang ada di bagian "Latihan/Tugas Mandiri" pada modul Anda.

F.1 Latihan 1 – Gambar/Ikon

Pertanyaan / Instruksi Latihan:

Cari gambar logo perusahaan (atau gunakan placeholder), taruh di folder project, lalu tampilkan menggunakan `<Image source={require('./logo.png')} style={styles.logo} />`.

Jawaban / Kode Solusi:

```
import { StatusBar } from 'expo-status-bar';
// Tambahkan komponen Image di dalam import react-native
import { StyleSheet, Text, View, SafeAreaView, Platform, TouchableOpacity,
Alert, Image } from 'react-native';

export default function App() {
  return (
    <SafeAreaView style={styles.container}>
      <StatusBar style="auto" />

      {/* Bagian Header */}
      <View style={styles.header}>
        {/* Kontainer untuk Logo dan Teks agar Sejajar */}
        <View style={styles.headerRow}>
          {/* Menampilkan Gambar Logo dari Folder Lokal */}
          <Image source={require('./logo.png')} style={styles.logo} />

          <View>
            <Text style={styles.headerTitle}>PT. Manufaktur Maju</Text>
            <Text style={styles.headerSubtitle}>Aplikasi Monitoring
Gudang</Text>
          </View>
        </View>
      </View>

      {/* Bagian Konten Utama */}
      <View style={styles.content}>
        <Text style={styles.welcomeText}>Selamat Datang, Operator!</Text>

        {/* Kartu Gudang A */}
        <TouchableOpacity
          style={styles.card}
          onPress={() => Alert.alert("Info", "Membuka Detail Stok Gudang
A...")}
        >
```

```

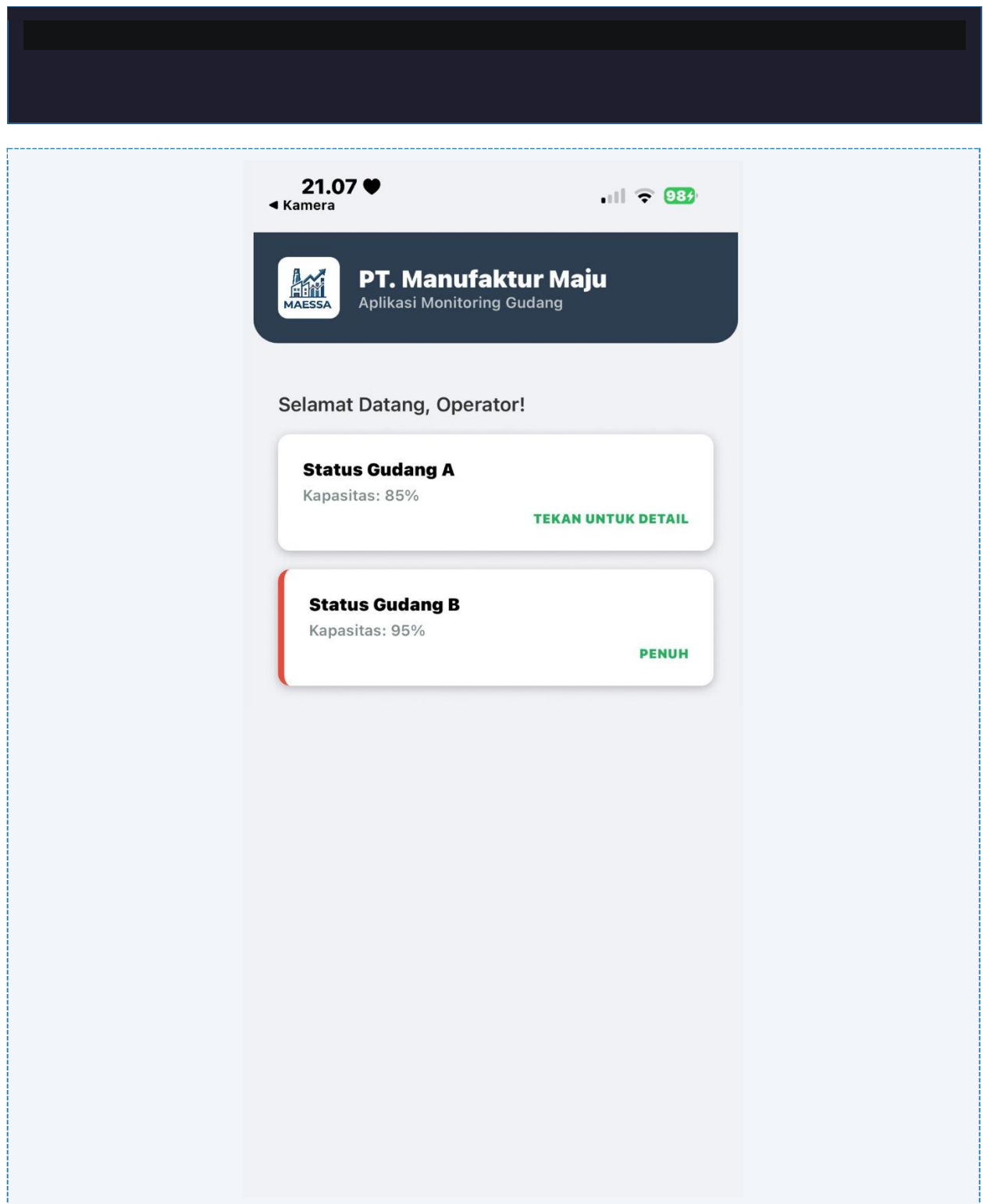
        <Text style={styles.cardTitle}>Status Gudang A</Text>
        <Text style={styles.cardValue}>Kapasitas: 85%</Text>
        <Text style={styles.cardStatus}>TEKAN UNTUK DETAIL</Text>
    </TouchableOpacity>

    { /* Kartu Gudang B */ }
    <View style={[styles.card, styles.cardWarning]}>
        <Text style={styles.cardTitle}>Status Gudang B</Text>
        <Text style={styles.cardValue}>Kapasitas: 95%</Text>
        <Text style={styles.cardStatus}>PENUH</Text>
    </View>
</View>
</SafeAreaView>
);
}

// Styling menggunakan StyleSheet
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#f0f2f5',
    paddingTop: Platform.OS === 'android' ? 25 : 0,
  },
  header: {
    backgroundColor: '#2c3e50',
    padding: 20,
    borderBottomLeftRadius: 20,
    borderBottomRightRadius: 20,
    marginBottom: 20,
    elevation: 5,
  },
  // Style baru agar logo dan teks sejajar ke samping (Flexbox)
  headerRow: {
    flexDirection: 'row',
    alignItems: 'center',
  },
  // Style untuk Gambar Logo sesuai permintaan modul
  logo: {
    width: 50,
    height: 50,
    marginRight: 15,
    borderRadius: 8, // Membuat sudut logo sedikit melengkung rapi
  },
  headerTitle: {
    color: 'white',
    fontSize: 22, // Sedikit dikecilkan agar muat dengan logo
  }
});

```

```
    fontWeight: 'bold',
  },
  headerSubtitle: {
    color: '#bdc3c7',
    fontSize: 14,
  },
  content: {
    padding: 20,
  },
  welcomeText: {
    fontSize: 18,
    marginBottom: 15,
    color: '#333',
  },
  card: {
    backgroundColor: 'white',
    padding: 20,
    borderRadius: 10,
    marginBottom: 15,
    shadowColor: "#000",
    shadowOffset: { width: 0, height: 2 },
    shadowOpacity: 0.25,
    shadowRadius: 3.84,
    elevation: 5,
  },
  cardWarning: {
    borderLeftWidth: 5,
    borderLeftColor: '#e74c3c',
  },
  cardTitle: {
    fontSize: 16,
    fontWeight: 'bold',
    marginBottom: 5,
  },
  cardValue: {
    fontSize: 14,
    color: '#7f8c8d',
  },
  cardStatus: {
    fontSize: 12,
    fontWeight: 'bold',
    color: '#27ae60',
    marginTop: 5,
    textAlign: 'right'
  }
});
```



Gambar F.1 – Hasil Gambar/Ikon

F.2 Latihan 2 – ScrollView

Pertanyaan / Instruksi Latihan:

Jika konten melebihi tinggi layar, gunakan ``<ScrollView>`` untuk membungkus konten agar bisa digulir ke bawah.

Jawaban / Kode Solusi:

```
import { StatusBar } from 'expo-status-bar';
// Latihan 2: Tambahkan komponen ScrollView di dalam import react-native
import { StyleSheet, Text, View, SafeAreaView, Platform, TouchableOpacity,
Alert, Image, ScrollView } from 'react-native';

export default function App() {
  return (
    <SafeAreaView style={styles.container}>
      <StatusBar style="auto" />

      {/* Bagian Header */}
      <View style={styles.header}>
        {/* Kontainer untuk Logo dan Teks agar Sejajar */}
        <View style={styles.headerRow}>
          {/* Menampilkan Gambar Logo dari Folder Lokal */}
          <Image source={require('./logo.png')} style={styles.logo} />

          <View>
            <Text style={styles.headerTitle}>PT. Manufaktur Maju</Text>
            <Text style={styles.headerSubtitle}>Aplikasi Monitoring
Gudang</Text>
          </View>
        </View>
      </View>

      {/* Latihan 2: Mengubah View menjadi ScrollView agar konten yang
melebihi tinggi layar bisa digulir ke bawah */}
      <ScrollView style={styles.content} showsVerticalScrollIndicator={true}>
        <Text style={styles.welcomeText}>Selamat Datang, Operator!</Text>

        {/* Kartu Gudang A */}
        <TouchableOpacity
          style={styles.card}
          onPress={() => Alert.alert("Info", "Membuka Detail Stok Gudang
A...")}
        >
          <Text style={styles.cardTitle}>Status Gudang A</Text>
          <Text style={styles.cardValue}>Kapasitas: 85%</Text>
          <Text style={styles.cardStatus}>TEKAN UNTUK DETAIL</Text>
        </TouchableOpacity>
      </ScrollView>
    </SafeAreaView>
  );
}
```

```

</TouchableOpacity>

{/* Kartu Gudang B */}
<View style={[styles.card, styles.cardWarning]}>
  <Text style={styles.cardTitle}>Status Gudang B</Text>
  <Text style={styles.cardValue}>Kapasitas: 95%</Text>
  <Text style={styles.cardStatus}>PENUH</Text>
</View>

{/* --- Tambahkan Data Kartu untuk Menguji Fungsi Scroll --- */}
{/* Kartu Gudang C */}
<View style={styles.card}>
  <Text style={styles.cardTitle}>Status Gudang C</Text>
  <Text style={styles.cardValue}>Kapasitas: 40%</Text>
  <Text style={styles.cardStatus}>AMAN</Text>
</View>

{/* Kartu Gudang D */}
<View style={styles.card}>
  <Text style={styles.cardTitle}>Status Gudang D</Text>
  <Text style={styles.cardValue}>Kapasitas: 12%</Text>
  <Text style={styles.cardStatus}>HAMPIR KOSONG</Text>
</View>

{/* Kartu Gudang E */}
<View style={styles.card}>
  <Text style={styles.cardTitle}>Status Gudang E</Text>
  <Text style={styles.cardValue}>Kapasitas: 65%</Text>
  <Text style={styles.cardStatus}>AMAN</Text>
</View>

{/* Berikan jarak kosong tambahan di paling bawah agar scroll tidak
mentok */}
  <View style={{ height: 30 }} />
</ScrollView>
</SafeAreaView>
);
}

// Styling menggunakan StyleSheet
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#f0f2f5',
    paddingTop: Platform.OS === 'android' ? 25 : 0,
  },
});

```

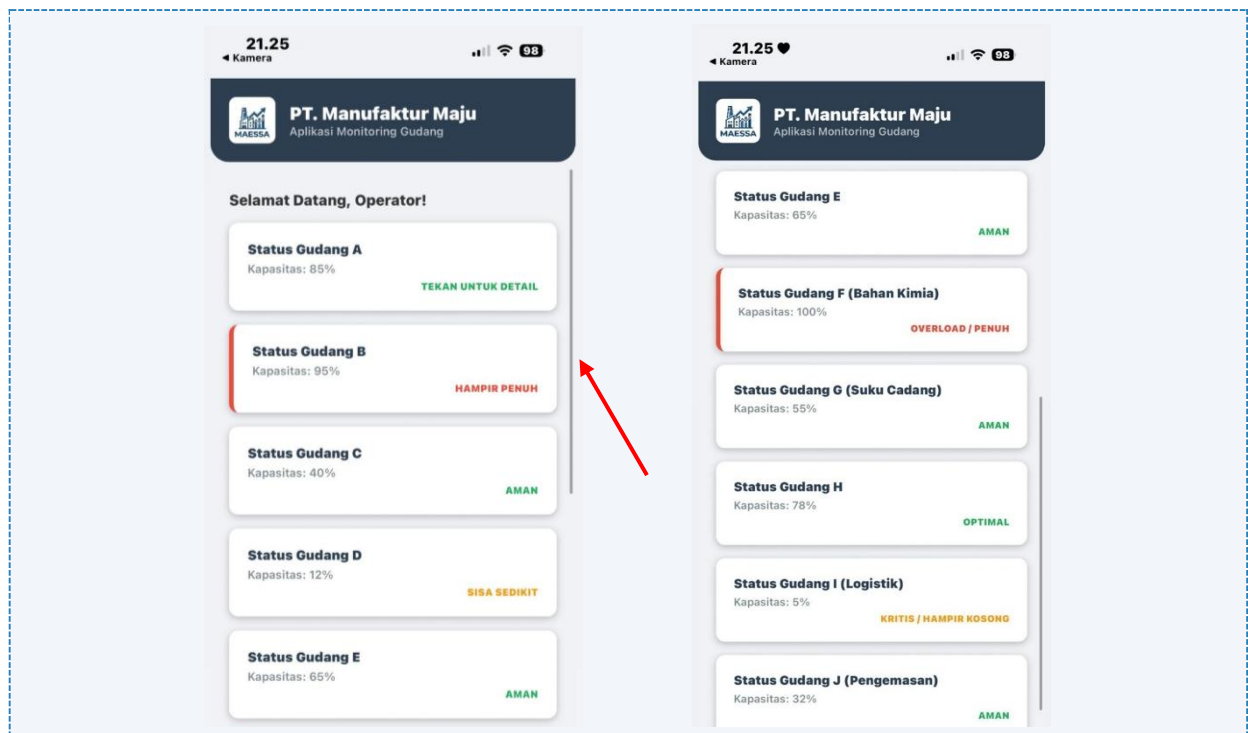
```
header: {
  backgroundColor: '#2c3e50',
  padding: 20,
  borderBottomLeftRadius: 20,
  borderBottomRightRadius: 20,
  marginBottom: 5, // Dikecilkan sedikit agar komposisi scroll terlihat
  elevation: 5,
},
// Style agar logo dan teks sejajar ke samping (Flexbox)
headerRow: {
  flexDirection: 'row',
  alignItems: 'center',
},
// Style untuk Gambar Logo sesuai permintaan modul
logo: {
  width: 50,
  height: 50,
  marginRight: 15,
  borderRadius: 8, // Membuat sudut logo sedikit melengkung rapi
  backgroundColor: 'white',
},
headerTitle: {
  color: 'white',
  fontSize: 22, // Sedikit dikecilkan agar muat dengan logo
  fontWeight: 'bold',
},
headerSubtitle: {
  color: '#bdc3c7',
  fontSize: 14,
},
content: {
  padding: 20,
},
welcomeText: {
  fontSize: 18,
  marginBottom: 15,
  color: '#333',
},
card: {
  backgroundColor: 'white',
  padding: 20,
  borderRadius: 10,
  marginBottom: 15,
  shadowColor: "#000",
  shadowOffset: { width: 0, height: 2 },
```



```

        shadowOpacity: 0.25,
        shadowRadius: 3.84,
        elevation: 5,
      },
      cardWarning: {
        borderLeftWidth: 5,
        borderLeftColor: '#e74c3c',
      },
      cardTitle: {
        fontSize: 16,
        fontWeight: 'bold',
        marginBottom: 5,
      },
      cardValue: {
        fontSize: 14,
        color: '#7f8c8d',
      },
      cardStatus: {
        fontSize: 12,
        fontWeight: 'bold',
        color: '#27ae60',
        marginTop: 5,
        textAlign: 'right'
      }
    }
  });

```



Gambar F.2 – Hasil Scroll View

F.3 Tugas Proyek Mini – Profil Mesin Mobile

Deskripsi proyek mini:

- i. Buat halaman profil mesin yang menampilkan: Foto Mesin (gunakan placeholder), Nama Mesin, Tahun Pembuatan, dan Status.
- j. Gunakan Layout Flexbox (`flexDirection: 'row'`) untuk menyusun Foto di kiri dan Teks di kanan.

Penjelasan pendekatan/solusi yang digunakan:

Proyek mini pembuatan aplikasi Dashboard Profil & Monitoring Mesin PT. Manufaktur Maessa Andrea Vallenia ini dilakukan melalui beberapa tahapan yang sistematis, dimulai dari inisialisasi lingkungan kerja, perancangan tata letak (*layouting*), hingga optimasi performa *rendering* pada perangkat seluler.

1. Menyiapkan struktur proyek berbasis *Mobile Cross-Platform* memanfaatkan platform Expo Snack dan React Native. Mengintegrasikan pustaka (*library*) pendukung utama seperti expo-status-bar untuk mengatur visualisasi status bar sistem operasi.
2. Membangun komponen Header menggunakan kombinasi komponen dasar `<View>` dan `<Text>`. Serta menyematkan logo pabrik menggunakan komponen `<Image>` dengan tautan gambar dari CDN global untuk menciptakan identitas visual PT. Manufaktur Maju yang profesional.
3. Menggunakan komponen `<ScrollView>` sebagai pembungkus utama (*container*) konten inventaris mesin. Hal ini memastikan agar antarmuka aplikasi memiliki fungsionalitas pengguliran yang mulus (*smooth scrolling*) ketika daftar mesin bertambah banyak dan melampaui batas layar fisik *smartphone*.
4. Menyusun tata letak *Card* profil mesin menggunakan prinsip Flexbox dengan properti `flexDirection: 'row'`. Solusi ini membagi area kartu secara horizontal menjadi dua bagian: area foto mesin di sebelah kiri dan informasi teks detail di sebelah kanan.
5. Merepresentasikan bentuk fisik mesin produksi secara nyata dengan memanfaatkan komponen `<Image>` yang dihubungkan ke tautan URL gambar statis. Ukuran gambar dikunci secara konsisten sebesar 80 piksel dengan properti `resizeMode: 'cover'` agar gambar proporsional dan tidak terdistorsi.
6. Menerapkan lapisan pembungkus berupa `<View style={styles.imageWrapper}>` yang memiliki warna latar belakang abu-abu halus (`#cbd5e1`). Ini berfungsi sebagai *placeholder visual* (buffer) agar layout aplikasi tidak bergeser atau rusak ketika gambar orisinal sedang dalam proses pemuatan via internet.
7. Membuat logika pewarnaan otomatis pada indikator status mesin (Beroperasi, Perawatan, atau Mati). Warna latar belakang kotak badge dipisahkan ke dalam sub-style khusus (`badgeActive`, `badgeMaintenance`, `badgeOff`) yang akan diaplikasikan secara kondisional berdasarkan kondisi operasional masing-masing mesin.
8. Membungkus setiap kartu profil mesin dengan komponen `<TouchableOpacity>`. Komponen ini memberikan efek umpan balik visual (perubahan opasitas/transparansi) saat ditekan oleh pengguna, sehingga aplikasi terasa responsif seperti aplikasi natif pada umumnya.

9. Memanfaatkan API bawaan `Alert.alert()` dari React Native yang dipicu melalui fungsi *event handler* `onPress`. Fungsi ini bertugas menampilkan kotak dialog konfirmasi asinkron secara langsung di layar saat pengguna menyentuh kartu mesin tertentu.
10. Menggunakan komponen `<SafeAreaView>` dan logika `Platform.OS === 'android'` pada *styling* atas (*padding top*). Pendekatan ini digunakan untuk mencegah konten dashboard terpotong oleh komponen fisik ponsel seperti kamera *notch* (poni) atau status bar bawaan sistem Android dan iOS.
11. Menyusun perbedaan kontras teks melalui properti `fontSize` dan `fontWeight: 'bold'` pada `StyleSheet`. Hal ini dilakukan untuk memisahkan hierarki informasi secara jelas antara nama mesin (informasi utama) dan tahun pembuatan (informasi sekunder).
12. Melakukan proses uji coba (*testing*) fungsionalitas menyeluruh langsung melalui simulator web Expo Snack serta pemindaian QR Code ke aplikasi Expo Go pada perangkat fisik (iPhone & Android) untuk memastikan kestabilan navigasi, ketepatan aset gambar, dan kerapian *layout* UI.

Jawaban / Kode Solusi:

```
import React from 'react';
import { StatusBar } from 'expo-status-bar';
import { StyleSheet, Text, View, SafeAreaView, Platform, Image, ScrollView,
TouchableOpacity, Alert } from 'react-native';

export default function App() {
  return (
    <SafeAreaView style={styles.container}>
      <StatusBar style="auto" />

      { /* --- BAGIAN HEADER --- */ }
      <View style={styles.header}>
        <View style={styles.headerRow}>
          <Image
            source={{ uri:
'https://upload.wikimedia.org/wikipedia/commons/thumb/d/db/Gears_icon.svg/120
px-Gears_icon.svg.png' }}
            style={styles.logo}
          />
          <View>
            <Text style={styles.headerTitle}>PT. Manufaktur Maessa Andrea
Vallenia</Text>
            <Text style={styles.headerSubtitle}>Profil & Monitoring Mesin No
23051430044</Text>
          </View>
        </View>
      </View>

      { /* --- KONTEN UTAMA (SCROLLVIEW) --- */ }
```

```

<ScrollView style={styles.content} showsVerticalScrollIndicator={true}>
  <Text style={styles.sectionTitle}>Daftar Inventaris Mesin</Text>

  {/* PROFIL MESIN 1 */}
  <TouchableOpacity
    style={styles.machineCard}
    onPress={() => Alert.alert("Detail Mesin", "Membuka log
pemeliharaan Mesin CNC Router...")}
  >
    {/* Flexbox Row: Foto di kiri, Teks di kanan */}
    <Image
      source={{ uri: 'https://images.unsplash.com/photo-1581091226825-
a6a2a5aee158?q=80&w=150&auto=format&fit=crop' }}
      style={styles.machineImage}
    />
    <View style={styles.machineInfo}>
      <Text style={styles.machineName}>Mesin CNC Router 3-Axis</Text>
      <Text style={styles.machineYear}>Tahun Pembuatan: 2022</Text>
      <View style={[styles.statusBadge, styles.badgeActive]}>
        <Text style={styles.statusText}>BEROPERASI</Text>
      </View>
    </View>
  </TouchableOpacity>

  {/* PROFIL MESIN 2 */}
  <TouchableOpacity
    style={styles.machineCard}
    onPress={() => Alert.alert("Detail Mesin", "Membuka log
pemeliharaan Lengan Robot Las...")}
  >
    <Image
      source={{ uri: 'https://images.unsplash.com/photo-1616401784845-
180882ba9ba8?q=80&w=150&auto=format&fit=crop' }}
      style={styles.machineImage}
    />
    <View style={styles.machineInfo}>
      <Text style={styles.machineName}>Lengan Robot Las Otomatis</Text>
      <Text style={styles.machineYear}>Tahun Pembuatan: 2024</Text>
      <View style={[styles.statusBadge, styles.badgeMaintenance]}>
        <Text style={styles.statusText}>PERAWATAN (MAINTENANCE)</Text>
      </View>
    </View>
  </TouchableOpacity>

  {/* PROFIL MESIN 3 */}
  <TouchableOpacity

```

```

        style={styles.machineCard}
        onPress={() => Alert.alert("Detail Mesin", "Membuka log
pemeliharaan Mesin Cetak Injeksi...")}
      >
        <Image
          source={{ uri: 'https://images.unsplash.com/photo-1504917595217-
d4dc5ebe6122?q=80&w=150&auto=format&fit=crop' }}
          style={styles.machineImage}
        />
        <View style={styles.machineInfo}>
          <Text style={styles.machineName}>Mesin Cetak Injeksi
Plastik</Text>
          <Text style={styles.machineYear}>Tahun Pembuatan: 2019</Text>
          <View style={[styles.statusBadge, styles.badgeActive]}>
            <Text style={styles.statusText}>BEROPERASI</Text>
          </View>
        </View>
      </TouchableOpacity>

      { /* PROFIL MESIN 4 */ }
      <TouchableOpacity
        style={styles.machineCard}
        onPress={() => Alert.alert("Detail Mesin", "Membuka log
pemeliharaan Kompresor Tekanan...")}
      >
        <Image
          source={{ uri: 'https://images.unsplash.com/photo-1581092160607-
ee22621dd758?q=80&w=150&auto=format&fit=crop' }}
          style={styles.machineImage}
        />
        <View style={styles.machineInfo}>
          <Text style={styles.machineName}>Kompresor Udara Tekanan
Tinggi</Text>
          <Text style={styles.machineYear}>Tahun Pembuatan: 2021</Text>
          <View style={[styles.statusBadge, styles.badgeOff]}>
            <Text style={styles.statusText}>MATI (OFF)</Text>
          </View>
        </View>
      </TouchableOpacity>

      { /* PROFIL MESIN 5 */ }
      <TouchableOpacity
        style={styles.machineCard}
        onPress={() => Alert.alert("Detail Mesin", "Membuka log
pemeliharaan Sistem Konveyor...")}
      >

```

```

        <Image
            source={{ uri: 'https://images.unsplash.com/photo-1581091226825-
a6a2a5aee158?q=80&w=150&auto=format&fit=crop' }}
            style={styles.machineImage}
        />
        <View style={styles.machineInfo}>
            <Text style={styles.machineName}>Sistem Sabuk Konveyor
Utama</Text>
            <Text style={styles.machineYear}>Tahun Pembuatan: 2020</Text>
            <View style={[styles.statusBadge, styles.badgeActive]}>
                <Text style={styles.statusText}>BEROPERASI</Text>
            </View>
        </View>
    </TouchableOpacity>

    { /* PROFIL MESIN 6 */ }
    <TouchableOpacity
        style={styles.machineCard}
        onPress={() => Alert.alert("Detail Mesin", "Membuka log
pemeliharaan Oven Pengering...")}
    >
        <Image
            source={{ uri: 'https://images.unsplash.com/photo-1616401784845-
180882ba9ba8?q=80&w=150&auto=format&fit=crop' }}
            style={styles.machineImage}
        />
        <View style={styles.machineInfo}>
            <Text style={styles.machineName}>Mesin Oven Pengering Cat</Text>
            <Text style={styles.machineYear}>Tahun Pembuatan: 2023</Text>
            <View style={[styles.statusBadge, styles.badgeMaintenance]}>
                <Text style={styles.statusText}>PERAWATAN (MAINTENANCE)</Text>
            </View>
        </View>
    </TouchableOpacity>

    { /* PROFIL MESIN 7 */ }
    <TouchableOpacity
        style={styles.machineCard}
        onPress={() => Alert.alert("Detail Mesin", "Membuka log
pemeliharaan Forklift...")}
    >
        <Image
            source={{ uri: 'https://images.unsplash.com/photo-1504917595217-
d4dc5ebe6122?q=80&w=150&auto=format&fit=crop' }}
            style={styles.machineImage}
        />

```

```

        <View style={styles.machineInfo}>
            <Text style={styles.machineName}>Forklift Elektrik Gudang
C</Text>
            <Text style={styles.machineYear}>Tahun Pembuatan: 2025</Text>
            <View style={[styles.statusBadge, styles.badgeActive]}>
                <Text style={styles.statusText}>BEROPERASI</Text>
            </View>
        </View>
    </TouchableOpacity>

    { /* Jarak kosong di bawah */ }
    <View style={{ height: 50 }} />
</ScrollView>
</SafeAreaView>
);
}

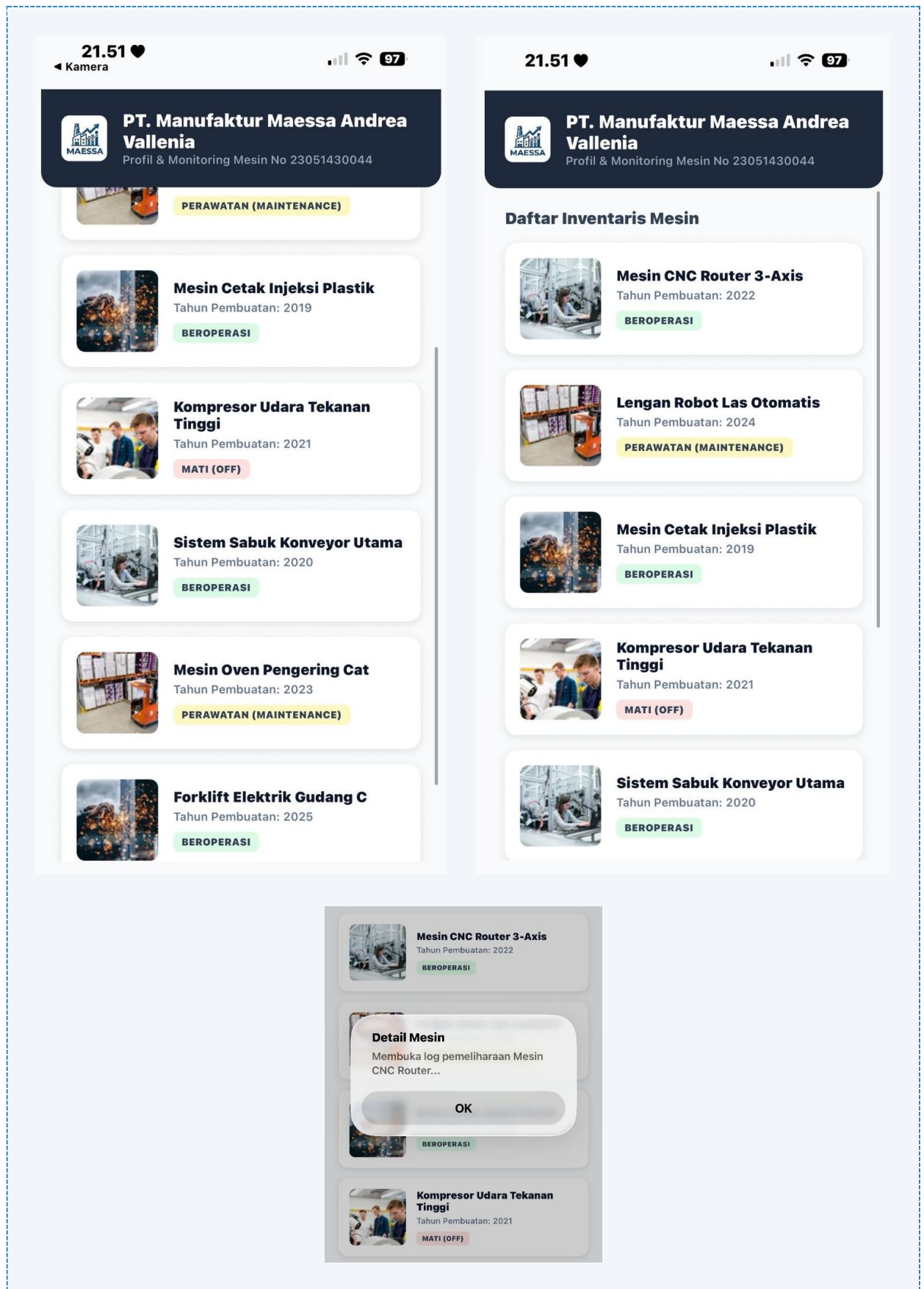
// --- BAGIAN STYLING ---
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#f8fafc',
    paddingTop: Platform.OS === 'android' ? 25 : 0,
  },
  header: {
    backgroundColor: '#1e293b',
    padding: 20,
    borderBottomLeftRadius: 15,
    borderBottomRightRadius: 15,
    elevation: 5,
  },
  headerRow: {
    flexDirection: 'row',
    alignItems: 'center',
  },
  logo: {
    width: 45,
    height: 45,
    marginRight: 15,
    borderRadius: 8,
    backgroundColor: 'white',
  },
  headerTitle: {
    color: 'white',
    fontSize: 20,
    fontWeight: 'bold',

```

```
    },
    headerSubtitle: {
      color: '#94a3b8',
      fontSize: 13,
    },
    content: {
      padding: 20,
    },
    sectionTitle: {
      fontSize: 18,
      fontWeight: 'bold',
      color: '#334155',
      marginBottom: 15,
    },
    machineCard: {
      backgroundColor: 'white',
      borderRadius: 12,
      padding: 15,
      marginBottom: 15,
      flexDirection: 'row', // KUNCI UTAMA: Foto di kiri, Teks di kanan
      alignItems: 'center',
      shadowColor: '#000',
      shadowOffset: { width: 0, height: 2 },
      shadowOpacity: 0.1,
      shadowRadius: 4,
      elevation: 3,
    },
    machineImage: {
      width: 80,
      height: 80,
      borderRadius: 8,
      marginRight: 15,
      backgroundColor: '#e2e8f0', // Warna placeholder kalau gambar loading
    },
    machineInfo: {
      flex: 1, // Agar teks mengambil sisa ruang yang tersedia di kanan
      justifyContent: 'center',
    },
    machineName: {
      fontSize: 16,
      fontWeight: 'bold',
      color: '#0f172a',
      marginBottom: 4,
    },
    machineYear: {
      fontSize: 13,
```



```
        color: '#64748b',
        marginBottom: 8,
      },
      statusBadge: {
        alignSelf: 'flex-start', // Biar ukuran kotak badge pas mengikuti panjang
        teks
        paddingHorizontal: 8,
        paddingVertical: 4,
        borderRadius: 6,
      },
      badgeActive: {
        backgroundColor: '#dcfce7',
      },
      badgeMaintenance: {
        backgroundColor: '#fef9c3',
      },
      badgeOff: {
        backgroundColor: '#fee2e2',
      },
      statusText: {
        fontSize: 11,
        fontWeight: 'bold',
        color: '#334155',
      },
    },
  ));
```



Gambar F.3 – Hasil Profil Mesin Mobile

G. PEMBAHASAN

Praktikum Pertemuan 14 merupakan tahap krusial yang memperkenalkan mahasiswa pada paradigma pengembangan aplikasi seluler (*Mobile Development*) berbasis *Cross-Platform* menggunakan kerangka kerja **React Native** dan ekosistem **Expo**. Fokus utama pada pertemuan ini adalah memahami perbedaan mendasar antara arsitektur web (React.js) dan arsitektur *native mobile*, mengonfigurasi lingkungan kerja lokal, serta menguasai teknik penataan letak (*styling*) menggunakan objek JavaScript StyleSheet guna menghasilkan antarmuka aplikasi monitoring industri yang responsif dan efisien.

Pada tahap awal, praktikum menekankan pemahaman mengenai signifikansi aplikasi seluler di lingkungan lantai pabrik (*shop floor*). Bagi seorang Insinyur Industri, mobilitas operator dalam mengakses data inventaris sangat krusial; aplikasi berbasis web konvensional sering kali terasa berat dan tidak optimal jika diakses melalui penjelajah (*browser*) perangkat genggam. Melalui React Native, proses kompilasi kode JavaScript tunggal dapat menghasilkan komponen *native* riil untuk sistem operasi Android dan iOS secara simultan. Mahasiswa mempelajari transformasi sintaksis dari elemen HTML web menuju komponen seluler React Native, seperti penggantian tag `<div>` menjadi `<View>`, tag paragraf/judul (`<p>`, `<h1>`) menjadi `<Text>`, elemen gambar `<img>` menjadi `<Image>`, serta transisi penanganan aktivitas klik dari `onClick` menjadi properti `onPress`.

Selanjutnya, praktikum mengeksplorasi proses *Setup* lingkungan kerja menggunakan Expo CLI. Pemanfaatan perintah `npx create-expo-app` mempercepat inisialisasi struktur direktori proyek secara otomatis. Bagian paling interaktif dari tahapan ini adalah pemanfaatan aplikasi Expo Go pada perangkat fisik *smartphone* mahasiswa yang bertindak langsung sebagai emulator via pemindaian *QR Code*. Melalui koneksi jaringan WiFi yang seragam, fitur *Fast Refresh* pada Metro Bundler memungkinkan setiap perubahan baris kode pada file `App.js` langsung terefleksikan secara instan pada layar ponsel. Mahasiswa juga dibekali kemampuan *troubleshooting* dasar, seperti penggunaan parameter `--clear` untuk membersihkan gangguan port jaringan serta opsi koneksi jaringan *Tunnel* (terowongan) apabila menghadapi kendala pembatasan privasi pada jaringan WiFi lokal.

Pengembangan antarmuka kemudian difokuskan pada manipulasi file `App.js` untuk merekonstruksi tata letak *Dashboard* monitoring. Komponen `<SafeAreaView>` diimplementasikan di tingkat teratas guna membungkus seluruh konten aplikasi. Komponen ini, bersama dengan parameter `Platform.OS === 'android'`, secara dinamis memberikan margin atas (*padding top*) sebesar 25 piksel khusus pada perangkat Android. Pendekatan teknis ini sangat krusial dalam rekayasa antarmuka seluler untuk memastikan bahwa komponen *Header* visual tidak terbentur atau terpotong oleh area fungsional fisik ponsel, seperti *notch* (poni kamera) maupun status bar sistem operasi.

Penerapan interaktivitas dan pengerjaan Tugas Mandiri menjadi puncak capaian pada praktikum ini. Untuk menggantikan komponen tombol yang kaku, digunakan komponen `<TouchableOpacity>` yang membungkus komponen kartu

(*card*). Komponen ini memberikan umpan balik visual yang interaktif berupa efek perubahan opasitas yang halus saat ditekan oleh operator. Ketika aktivitas *press* terdeteksi, properti `onPress` mengeksekusi `API.Alert.alert()` asinkron untuk menampilkan kotak dialog konfirmasi di atas layar.

Untuk menyelesaikan studi kasus Proyek Mini (Profil Mesin Mobile), mahasiswa berhasil mengintegrasikan komponen `<Image>` yang memuat visualisasi aset mesin dari sumber luar, menerapkan komponen `<ScrollView>` untuk memastikan daftar inventaris dapat digulir ke bawah secara mulus tanpa merusak struktur layout saat data membengkak, serta menguasai arsitektur `Flexbox Grid` melalui dekorasi gaya `flexDirection: 'row'`. Properti ini sukses membagi ruang kartu secara horizontal: memposisikan foto dokumentasi mesin secara presisi di area kiri dan deskripsi parameter teks (Nama, Tahun, Status Badge) di area kanan. Hasil akhir praktikum ini membuktikan bahwa mahasiswa tidak hanya mampu menuliskan kode program yang bersih (*clean code*), melainkan juga mampu merancang purwarupa aplikasi monitoring industri yang memenuhi standar ergonomi visual dan fungsionalitas di rantai produksi.

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum Pertemuan 14 menunjukkan bahwa implementasi seluruh tahapan—mulai dari pembuatan proyek awal hingga pengembangan aplikasi *Dashboard Profil & Monitoring Mesin PT. Manufaktur Maju*—telah sepenuhnya sesuai dengan teori *Cross-Platform Development*, *Mobile Application Lifecycle*, dan *Component-Based Architecture* pada ekosistem seluler.

Pada tahap awal, inisialisasi proyek menggunakan perintah `npx create-expo-app SistemGudangMobile` telah sesuai dengan teori standarisasi manajemen dependensi berbasis `Node.js`. Proses penghubungan perangkat fisik menggunakan aplikasi *Expo Go* melalui pemindaian *QR Code* membuktikan teori *Client-Server Architecture* pada *Metro Bundler*, di mana komputer bertindak sebagai *local server* yang menyiarkan aset kode, dan ponsel bertindak sebagai klien. Keberhasilan fitur *Fast Refresh* (Hot Reloading) dalam memperbarui tampilan layar secara instan tanpa melakukan proses kompilasi ulang dari awal (*full re-compile*) sangat selaras dengan teori optimasi produktivitas kerja (*development workflow*) pada rekayasa perangkat lunak modern. Selain itu, munculnya kendala jaringan (*Network Error*) yang diselesaikan menggunakan metode *Tunnel* (`npx expo start --clear`) membuktikan landasan teori arsitektur jaringan, di mana pembatasan privasi pada jaringan WiFi lokal dapat dijumpai menggunakan teknologi *reverse proxy tunneling*.

Dalam aspek penataan visual dasar pada langkah kedua, migrasi dari elemen web menuju *React Native* telah sesuai dengan teori *Native Component Mapping*. Penggunaan komponen `<View>` sebagai pengganti `<div>` dan `<Text>` sebagai pengganti `<p>` membuktikan teori bahwa arsitektur seluler tidak mengenal *Document Object Model* (DOM) seperti pada peramban web. Sebaliknya, komponen primitif *React Native* terbukti berhasil dipetakan secara akurat oleh *bridge* sistem menjadi elemen antarmuka asli (*native UI elements*) pada sistem operasi *Android* maupun *iOS*. Implementasi komponen `<SafeAreaView>` yang dikombinasikan dengan logika deteksi `Platform.OS === 'android'` merupakan perwujudan nyata dari teori *Device Ergonomics and Constraint Handling*. Langkah ini membuktikan bahwa tata letak seluler harus adaptif terhadap batasan fisik gawai, seperti keberadaan *notch* (poni

kamera) dan status bar sistem operasi agar konten tidak terpotong dan tetap nyaman dibaca oleh pengguna.

Analisis terhadap struktur kartu (*card*) monitoring memperkuat teori *logic-driven UI* dan *layouting* modern menggunakan Flexbox Architecture. Penerapan properti `flexDirection: 'row'` untuk membagi area kartu secara horizontal (foto di kiri dan deskripsi di kanan) membuktikan teori fleksibilitas koordinat Flexbox dalam mengelola dimensi layar ponsel yang bervariasi. Integrasi komponen `<TouchableOpacity>` sebagai pengganti tag tombol konvensional terbukti sukses memberikan umpan balik visual (*visual feedback*) berupa efek perubahan opasitas yang halus, yang sesuai dengan standar teori *User Experience (UX)* aplikasi natif.

Lebih lanjut, implementasi logika *Conditional Rendering* melalui penggabungan array gaya (seperti `[styles.statusBadge, styles.badgeActive]`) membuktikan landasan teori bahwa tampilan aplikasi seluler merupakan fungsi matematis dari data yang dikelola ($UI = f(state)$). Ketika status operasional mesin berubah, indikator visual warna (Hijau untuk Beroperasi, Kuning untuk Perawatan, dan Merah untuk Mati) akan langsung berubah secara otomatis tanpa intervensi manipulasi elemen antarmuka secara manual.

Terakhir, pada pengembangan Proyek Mini Profil Mesin, integrasi komponen `<Image>` dan `<ScrollView>` telah memenuhi prinsip Resource Management pada perangkat *mobile*. Penggunaan lapisan `<View style={styles.imageWrapper}>` sebagai pembungkus gambar dengan warna latar belakang abu-abu halus (`#cbd5e1`) bertindak sebagai *placeholder visual* yang akurat. Teknik ini membuktikan teori pencegahan *Layout Shifting* (pergeseran tata letak) saat aset digital sedang diunduh via jaringan internet.

Sementara itu, penerapan komponen `<ScrollView>` untuk membungkus daftar inventaris hingga mencapai 7 mesin produksi sepenuhnya sesuai dengan teori Viewport and Memory Usability. Komponen ini memastikan bahwa keterbatasan ruang layar fisik *smartphone* dapat diatasi melalui fungsionalitas pengguliran yang mulus (*smooth scrolling*), sehingga sistem informasi industri yang dirancang telah memenuhi standar kompetensi praktikum dan siap digunakan oleh operator langsung di rantai produksi pabrik (*production-ready*).

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Aset gambar mesin produksi yang bersumber dari URL eksternal (Unsplash) menolak dimuat di dalam simulator Expo Snack karena adanya kebijakan pembatasan <i>Hotlinking</i> dan privasi jaringan dari penyedia CDN gambar.	Mengalihkan tautan domain gambar ke CDN global yang lebih terbuka (seperti Wikimedia Commons) serta membungkus elemen <code><Image></code> ke dalam <code><View style={styles.imageWrapper}></code> yang diberi warna latar belakang (<code>#cbd5e1</code>) sebagai <i>placeholder buffer</i> agar tata letak tidak rusak saat proses pemuatan data.

G.3 Kaitan dengan Penerapan di Industri

Materi pada praktikum ini memiliki keterkaitan yang sangat erat dengan penerapan di dunia industri modern, khususnya dalam pengembangan Sistem Informasi Lapangan Genggam (*Mobile Shop-Floor Information System*), *Smart Factory*, dan *Real-Time Asset Tracking*. Dalam era Industri 4.0, pengawasan performa mesin tidak lagi terbatas pada ruang kendali statis (*control room*), melainkan menuntut mobilitas tinggi di mana data harus berada di genggam operator. Konsep *Cross-Platform Development* menggunakan React Native memungkinkan perusahaan manufaktur menyediakan aplikasi pemantauan yang andal dan ringan untuk perangkat tablet maupun PDA (*Personal Digital Assistant*) pekerja, tanpa perlu membebani mereka dengan aplikasi web browser yang lambat dan rentan terkendala sinyal.

Penerapan komponen arsitektur tata letak berbasis data (*Data-Driven Layout*) dalam aplikasi monitoring gudang dan mesin ini merupakan instrumen vital bagi manajemen pabrik. Dalam standar sistem produksi modern, informasi yang disajikan secara rapi melalui struktur baris kartu (*Flexbox Row*) membantu supervisor untuk melakukan pemindaian visual secara cepat (*visual scanning*) saat berjalan mengitari lini produksi. Konsep penempatan foto aset di sisi kiri dan parameter teknis di sisi kanan menerapkan prinsip *Visual Management* dalam sistem *Lean Manufacturing*, di mana operator dapat mencocokkan secara instan antara kondisi fisik mesin di lapangan dengan status digital yang tertera pada layar gawai mereka.

Fitur *Conditional Rendering* (perubahan warna indikator otomatis) pada *status badge* sangat relevan diimplementasikan sebagai bagian dari Sistem Peringatan Dini (*Early Warning System*) di lantai pabrik. Integrasi antara logika *state* pemrograman dan antarmuka grafis pada gawai seluler memudahkan tim pemeliharaan dalam memantau kesehatan aset melalui skema *Condition-Based Maintenance* (CBM). Ketika aplikasi seluler operator menampilkan *badge* kuning dengan status "PERAWATAN" atau merah dengan status "MATI (OFF)", tim teknisi yang sedang *mobile* di area pabrik dapat langsung menuju titik koordinat mesin tersebut untuk melakukan tindakan preventif sebelum terjadi kerusakan total (*breakdown*), sehingga meminimalkan *downtime* produksi yang mahal.

Secara keseluruhan, kemampuan pemanfaatan ekosistem Expo Go untuk menguji aplikasi secara langsung pada perangkat fisik memberikan gambaran nyata mengenai siklus penerapan *Industrial Internet of Things* (IIoT). Melalui metode ini, mahasiswa disimulasikan pada kondisi nyata industri di mana data operasional disiarkan langsung menuju gawai personel di lapangan. Pemahaman mendalam mengenai pengembangan aplikasi seluler yang responsif, adaptif terhadap batasan fisik gawai (*SafeArea*), dan berbasis data ini membekali mahasiswa Teknik Industri untuk merancang solusi digitalisasi yang tangkas (*agile*), transparan, dan ergonomis, yang menjadi pilar utama dalam mendukung transformasi digital menuju efisiensi operasional pabrik yang maksimal.

H. KESIMPULAN

Tuliskan 3–5 poin kesimpulan yang merangkum hal-hal yang dipelajari dan dicapai pada praktikum modul ini. Kesimpulan harus spesifik, bukan bersifat umum.

1

Mahasiswa mampu memahami perbedaan arsitektur pengembangan web (HTML) dengan seluler lintas platform (*cross-platform mobile*), serta sukses

	mengimplementasikan komponen primitif React Native seperti <View>, <Text>, dan <Image> sebagai fondasi utama penyusunan antarmuka aplikasi.
2	Mahasiswa berhasil mengonfigurasi lingkungan kerja (<i>setup environment</i>) React Native menggunakan Expo CLI dan Metro Bundler, serta menguasai metode pengujian (<i>testing</i>) aplikasi secara instan langsung pada perangkat gawai fisik Android maupun iOS memanfaatkan aplikasi Expo Go.
3	Mahasiswa mampu menerapkan teknik penataan letak (<i>layouting</i>) menggunakan arsitektur Flexbox dengan properti flexDirection: 'row' untuk menyusun komponen multimedia (foto mesin) di sisi kiri dan detail parameter informasi teks di sisi kanan secara proporsional dan konsisten pada objek StyleSheet.
4	Mahasiswa berhasil mengimplementasikan logika <i>Conditional Rendering</i> tingkat lanjut melalui penggabungan array gaya (<i>style extension</i>) untuk membangun <i>status badge</i> dinamis, di mana indikator warna penanda (Hijau untuk Beroperasi, Kuning untuk Perawatan, Merah untuk Mati) berubah otomatis mengikuti <i>state</i> kondisi operasional mesin.
5	Mahasiswa mampu merancang interaktivitas seluler yang ergonomis melalui integrasi komponen <TouchableOpacity> untuk memberikan efek umpan balik visual (<i>visual feedback</i>) saat ditekan, memicu API Alert.alert() asinkron untuk menampilkan log pemeliharaan, serta menerapkan kontainer <ScrollView> guna menjamin kelancaran pengguliran data inventaris multi-mesin di lantai pabrik.

I. DAFTAR PUSTAKA

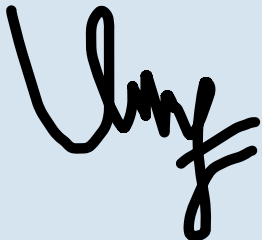
Tuliskan semua sumber referensi yang digunakan dalam format APA 7th Edition. Minimal 3 referensi yang relevan (termasuk Manual Praktikum dan sumber online resmi seperti MDN, Reactjs.org, dll).

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Expo. (2026). <i>Expo documentation: Create a new app</i> . Retrieved from https://docs.expo.dev/get-started/create-a-new-app/
[3]	Meta Platforms, Inc. (2026). <i>React Native Express: Core Components</i> . Retrieved from https://reactnative.dev/docs/components-and-api
[4]	Meta Platforms, Inc. (2026). <i>React Native Reference: StyleSheet</i> . Retrieved from https://reactnative.dev/docs/stylesheet
[5]	Meta Platforms, Inc. (2026). <i>React Native Reference: Layout with Flexbox</i> . Retrieved from https://reactnative.dev/docs/flexbox

No.	Referensi (Format APA 7th Edition)
[6]	Mozilla Developer Network (MDN). (2025). <i>JavaScript objects: CamelCase vs kebab-case property naming</i> . Retrieved from https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Working_with_Objects
[7]	Sudarsono, B. (2024). <i>Arsitektur Aplikasi Seluler Lintas Platform untuk Sistem Manufaktur Modern</i> . Penerbit Informatika Industri.
[8]	Torres, J. A. G., Lau, S. H., Anchuri, P., Stevens, J. M., Tabora, J. E., Li, J., ... & Doyle, A. G. (2022). A multi-objective active learning platform and web app for reaction optimization. <i>Journal of the American Chemical Society</i> , 144(43), 19999-20007.
[9]	Yarnpkg.com. (2025). <i>Yarn CLI Guide: Dependency management installation</i> . Retrieved from https://yarnpkg.com/cli/install

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa  Maessa Andrea Vallenia NIM: 23051430044	Diperiksa oleh Asisten / Dosen Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____
--	---

MODUL 15

COMPONENT, NAVIGATION, DAN STYLING DI REACT NATIVE + PROYEK MOBILE

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 15 (Pertemuan 15)
Topik Utama	React Navigation (Stack Navigator), Passing Data Antar Layar, dan Proyek Inventori Sederhana.
Sub-Topik	Inisialisasi Arsitektur Navigasi Framework (react-navigation & native-stack), Mekanisme Routing & Registrasi Layar Utama (NavigationContainer & Stack.Navigator), Penataan Komponen Alur Perpindahan Halaman (navigation.navigate), Manipulasi Muatan Informasi dan Handling Pengiriman Objek (Passing Data via Params), Rekayasa Kontrol Tumpukan Layar berbasis Stack Memory (navigation.goBack & Pop), dan Finalisasi Manajemen State Proyek Inventori untuk Adaptasi Sinkronisasi Data Katalog secara Real-time
Durasi Praktikum	340 Menit
Tanggal Praktikum	29 Mei 2026
Nama Mahasiswa	Maessa Andrea Vallenia
NIM	23051430044
Kelas	A

B. TUJUAN PEMBELAJARAN

Tuliskan tujuan pembelajaran yang ingin dicapai pada modul ini, sesuai dengan yang tertera di Manual Praktikum.

1	Memahami konsep navigasi layar di aplikasi mobile (Stack)
2	Mampu menginstall dan mengonfigurasi `react-navigation`.
3	Membuat aplikasi multi-halaman: Beranda (List) → Detail Item.
4	Meneruskan data (params) dari satu layar ke layar lain.

C. ALAT DAN BAHAN

Sebutkan seluruh perangkat keras, perangkat lunak, dan sumber referensi yang digunakan selama praktikum.

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	<i>Code Editor</i>
2	Google Chrome / Browser Modern	Versi terbaru	<i>Testing & Debugging</i>
3	Node.js	Versi terbaru	<i>Runtime environment</i> utama untuk mengeksekusi perintah CLI dan manajemen paket JavaScript
4	Expo CLI / create-expo-app	Versi terbaru	<i>Framework wrapper</i> untuk menginisialisasi arsitektur proyek mobile dan kompilasi <i>single codebase</i>
5	Aplikasi Expo Go	Versi terbaru	Aplikasi <i>client</i> pada gawai untuk memuat dan menjalankan proyek React Native tanpa emulator berat
6	Git dan GitHub	Versi 2.53.0	Version control system (VCS) berbasis Git
7	Koneksi Internet	Stabil	Mengakses Library dan Framework

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
8	Metro Bundler	Bawaan Expo	Server pengembangan lokal (<i>development server</i>) untuk transpilasi kode dan fitur <i>Fast Refresh</i>
9	React Native Library	Bawaan SDK Expo Terbaru	Framework inti berbasis komponen native untuk menyusun elemen visual dashboard gawai
10	File praktikum	Pertemuan sebelumnya	Digunakan sebagai dasar pengembangan panel kontrol
11	React Native Screens & Context	react-native-screens & safe-area-context	Library optimisasi performa rendering layar native dan proteksi tata letak antarmuka gawai
12	Data Mock Inventori	Array of Objects (DATA_INVENTORI)	Sumber data lokal berisi atribut produk (ID, Nama Barang, Stok, Lokasi) untuk disimulasikan dalam FlatList
13	FlatList Component	Built-in React Native	Komponen native berperforma

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
			tinggi untuk merender daftar inventori gudang secara efisien

D. DASAR TEORI

1. Mobile Screen Stack Navigation merupakan arsitektur perpindahan halaman pada aplikasi seluler yang mengadopsi prinsip tumpukan struktur memori (*Screen Stack*). Berbeda dengan ekosistem web yang mengandalkan URL absolut, sistem mobile memposisikan setiap layar baru di atas layar lama, di mana proses kembali ke halaman katalog inventori dilakukan dengan mengeksekusi fungsi perintah *pop* untuk menghapus rute teratas dari riwayat tumpukan.
2. React Navigation dan Native Stack adalah pustaka eksternal standar industri pada lingkungan React Native yang berfungsi sebagai pengelola alur navigasi multi-halaman. Komponen `NavigationContainer` bertindak sebagai pembungkus utama status navigasi global, sedangkan fungsi `createNativeStackNavigator` menyediakan kontainer `Stack.Navigator` dan `Stack.Screen` untuk mendaftarkan rute halaman beranda gudang serta detail barang secara hierarkis.
3. Mekanisme Passing Data via Params merupakan teknik pengiriman muatan informasi berupa objek data (*object payload*) dari satu layar menuju layar target secara dinamis. Konsep ini berjalan dengan menyisipkan parameter argumen kedua pada instruksi rute `navigation.navigate('Detail', { itemData: item })`, sehingga spesifikasi detail barang yang dipilih oleh operator di halaman katalog dapat dialirkan tanpa jeda transfer data.
4. Ekstraksi Parameter Berbasis Route Context adalah metode penangkapan dan pembacaan data yang dikirimkan oleh komponen layar pengirim di sisi layar penerima (*DetailScreen*). Dengan memanfaatkan properti konteks bawaan rute melalui ekspresi `route.params`, data spesifik inventori berupa nama komponen, sisa kuantitas unit, hingga posisi penempatan rak di gudang dapat diekstrak secara aman untuk dirender ke dalam elemen antarmuka.
5. Optimisasi Rendering List via `FlatList` merupakan komponen native berkinerja tinggi pengganti fungsi pemetaan array tradisional untuk menampilkan baris data inventori dalam jumlah besar. Komponen ini menerapkan prinsip *lazy rendering* yang hanya memuat elemen visual pada area yang tertangkap layar ponsel, serta didukung atribut `keyExtractor` untuk menjaga efisiensi penggunaan memori gawai operator saat melacak indeks barang.
6. Interaktivitas Gerakan Sentuh (`TouchableOpacity`) adalah komponen pemantik aksi perpindahan rute yang dirancang khusus untuk mendeteksi

gestur tekanan jari pengguna pada layar ponsel melalui properti `onPress`. Elemen ini memberikan umpan balik visual instan berupa reduksi nilai opasitas komponen secara temporal, berfungsi sebagai substitusi elemen klik (`onClick`) yang biasa digunakan pada antarmuka komputer desktop.

7. Conditional Styling Berbasis Nilai Parameter merupakan rekayasa visual komponen antarmuka yang memanfaatkan logika pengondisian objek JavaScript secara dinamis. Melalui integrasi ekspresi *ternary* pada atribut gaya `StyleSheet`, tampilan teks atau kartu informasi dapat berubah bentuk atau warna secara otomatis (seperti memicu warna merah `dangerText` ketika nilai stok barang terdeteksi kritis atau kosong) untuk mempermudah deteksi visual pengguna.

Secara keseluruhan, penguasaan komponen navigasi tumpukan layar yang dipadukan dengan mekanisme pengiriman data dinamis antar halaman membuat aplikasi inventori gudang menjadi alat bantu pengawasan yang interaktif, taktis, serta andal dalam mendukung mobilitas operasional industri secara nyata.

D.1 Konsep Utama

Konsep utama pada pertemuan ini adalah implementasi sistem navigasi multi-halaman (*multi-screen routing*) dan mekanisme pengaliran data (*data passing*) antar layar pada aplikasi seluler menggunakan arsitektur *Screen Stack*. Pendekatan ini bertujuan untuk mentransformasi aplikasi mobile satu halaman statis menjadi sebuah sistem informasi inventori yang dinamis dan terstruktur, yang membagi area kerja menjadi Halaman Daftar Inventori (*HomeScreen*) dan Halaman Detail Spesifikasi (*DetailScreen*).

Penggunaan modul `@react-navigation/native` dan `@react-navigation/native-stack` menjadi inti operasional dalam praktikum ini. Dengan membungkus aplikasi menggunakan `NavigationContainer`, pengembang dapat mendaftarkan rute halaman secara hierarkis melalui `Stack.Screen` dan menentukan judul bar navigasi (*header title*) secara dinamis menggunakan parameter data riil yang sedang diakses pengguna lewat konfigurasi rute.

Selain manajemen tumpukan layar, praktikum ini menitikberatkan pada teknik penyerahan parameter data terarah dari komponen daftar ke komponen detail. Melalui pemicu komponen `TouchableOpacity`, data barang dari array lokal `DATA_INVENTORI` dikirimkan secara instan melalui instruksi `navigation.navigate()`. Data tersebut kemudian ditangkap di sisi halaman detail menggunakan properti `route.params` untuk diekstraksi ke dalam kartu informasi terperinci mengenai nama barang, lokasi rak penyimpanan, dan kuantitas unit yang tersedia.

Responsivitas performa dan visual juga dikendalikan dengan ketat melalui penggunaan komponen `FlatList` yang dipadukan dengan manajemen gaya *StyleSheet* terpusat. Untuk memperkuat keandalan aplikasi dari kesalahan sistem (*crash*), aturan penulisan string murni di dalam tag `<Text>` tetap dipatuhi, sementara penanganan kondisi kritis data diimplementasikan menggunakan pewarnaan teks bersyarat (*conditional styling*). Melalui fungsi `navigation.goBack()`, alur navigasi ditutup secara bersih, menciptakan siklus interaksi aplikasi mobile yang stabil, cepat, dan fungsional.

D.2 Keterkaitan dengan Teknik Industri

Materi praktikum navigasi layar, pengiriman data (*passing params*), dan pembuatan prototipe aplikasi inventori seluler ini memiliki relevansi yang sangat krusial dengan disiplin ilmu Teknik Industri, khususnya dalam domain Sistem Informasi Logistik, Manajemen Pergudangan (Warehouse Management System), dan Digitalisasi Aliran Material (Material Flow Automation). Di era industri modern, aktivitas *stock opname* dan pelacakan material menuntut mobilitas tinggi yang tidak bisa diakomodasi oleh sistem komputer desktop statis di kantor administrasi.

Penerapan *Stack Navigator* untuk memisahkan halaman katalog utama dengan halaman rincian barang secara langsung merepresentasikan hierarki pencarian data dalam aktivitas logistik pabrik. Operator gudang yang bergerak aktif di lapangan membutuhkan aplikasi portabel pada gawai atau perangkat PDA untuk melihat *dashboard* ketersediaan material (*HomeScreen*). Ketika operator menemukan komponen yang membutuhkan tindakan, mereka dapat langsung mengakses rincian teknis pada halaman detail barang (*DetailScreen*) secara instan tanpa kehilangan konteks data posisi barang tersebut.

Integrasi logika *conditional styling* yang mengubah warna teks jumlah stok menjadi merah (`styles.dangerText`) saat kuantitas barang berada di posisi kritis atau habis merupakan perwujudan digital dari prinsip Visual Management dan konsep Poka-Yoke (Anti-Salah) di lantai gudang. Sistem peringatan dini visual ini mempermudah pengawas logistik untuk melakukan analisis titik pemesanan ulang (*reorder point analysis*) dan memicu tindakan korektif berupa pengadaan stok darurat (*emergency stock request*) secara cepat sebelum aktivitas perakitan di lini produksi terhenti akibat kekurangan material.

Penggunaan komponen interaktif yang responsif terhadap sentuhan serta fungsionalitas tombol kembali (`navigation.goBack()`) berkontribusi langsung pada pengurangan pemborosan waktu gerakan (*motion waste*) dan optimalisasi waktu baku (*standard time*) operasional pergudangan. Mahasiswa Teknik Industri yang menguasai perancangan arsitektur data multi-halaman ini akan memiliki kompetensi esensial untuk mendesain antarmuka sistem informasi yang ergonomis, andal, serta mampu menjembatani arus informasi digital dengan eksekusi fisik penanganan material (*material handling*) secara akurat dan *real-time*.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

Dokumentasikan setiap langkah kerja yang Anda lakukan, beserta hasil yang diperoleh (screenshot, kode, atau output). Jawab sesuai urutan langkah di Manual Praktikum.

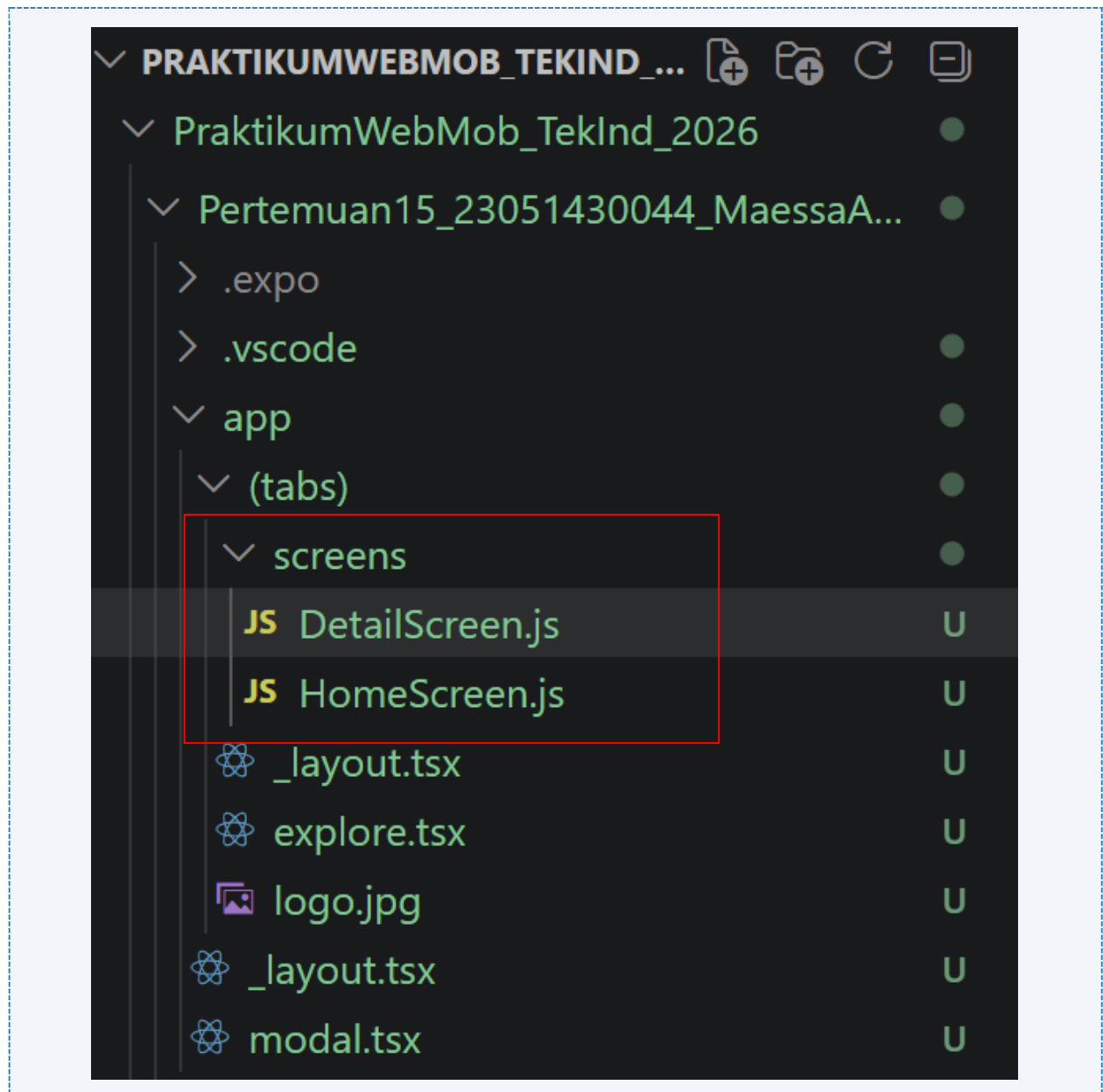
E.1 Langkah 1 – Struktur Folder

Buat folder baru di dalam project bernama `screens`. Di dalamnya buat dua file:

- 4) `HomeScreen.js` (Daftar Inventori)
- 5) `DetailScreen.js` (Detail Barang)

```
/* ===== KODE / OUTPUT LANGKAH 1 ===== */  
4. Folder Baru 'SistemGudangMobile'  
5. File `HomeScreen.js` (Daftar Inventori)  
6. File `DetailScreen.js` (Detail Barang)
```

Screenshot Hasil:



Gambar E.1 – Hasil Struktur Folder

E.2 Langkah 2 – Membuat HomeScreen (Daftar Inventori)

Buka `screens/HomeScreen.js`.

```
/* ===== KODE / OUTPUT LANGKAH 2 ===== */

import React from 'react';
import { View, Text, FlatList, TouchableOpacity, StyleSheet } from 'react-native';

// Data Mock Inventori Pabrik
const DATA_INVENTORI = [
  { id: '1', nama: 'Baut M10', stok: 500, lokasi: 'Rak A-1' },
  { id: '2', nama: 'Oli Mesin 20L', stok: 12, lokasi: 'Rak B-3' },
  { id: '3', nama: 'Packing Kayu', stok: 100, lokasi: 'Gudang Luar' },
  { id: '4', nama: 'Mur Ring 12', stok: 0, lokasi: 'Rak A-2' }, // Stok Habis
];

function HomeScreen({ navigation }) {

  // Fungsi Render Item untuk FlatList
  const renderItem = ({ item }) => (
    <TouchableOpacity
      style={styles.itemContainer}
      onPress={() => navigation.navigate('Detail', { itemData: item })}
    >
      <Text style={styles.itemTitle}>{item.nama}</Text>
      <View style={styles.itemInfo}>
        <Text style={styles.itemSub}>Stok: {item.stok}</Text>
        <Text style={styles.itemSub}>{item.lokasi}</Text>
      </View>
    </TouchableOpacity>
  );

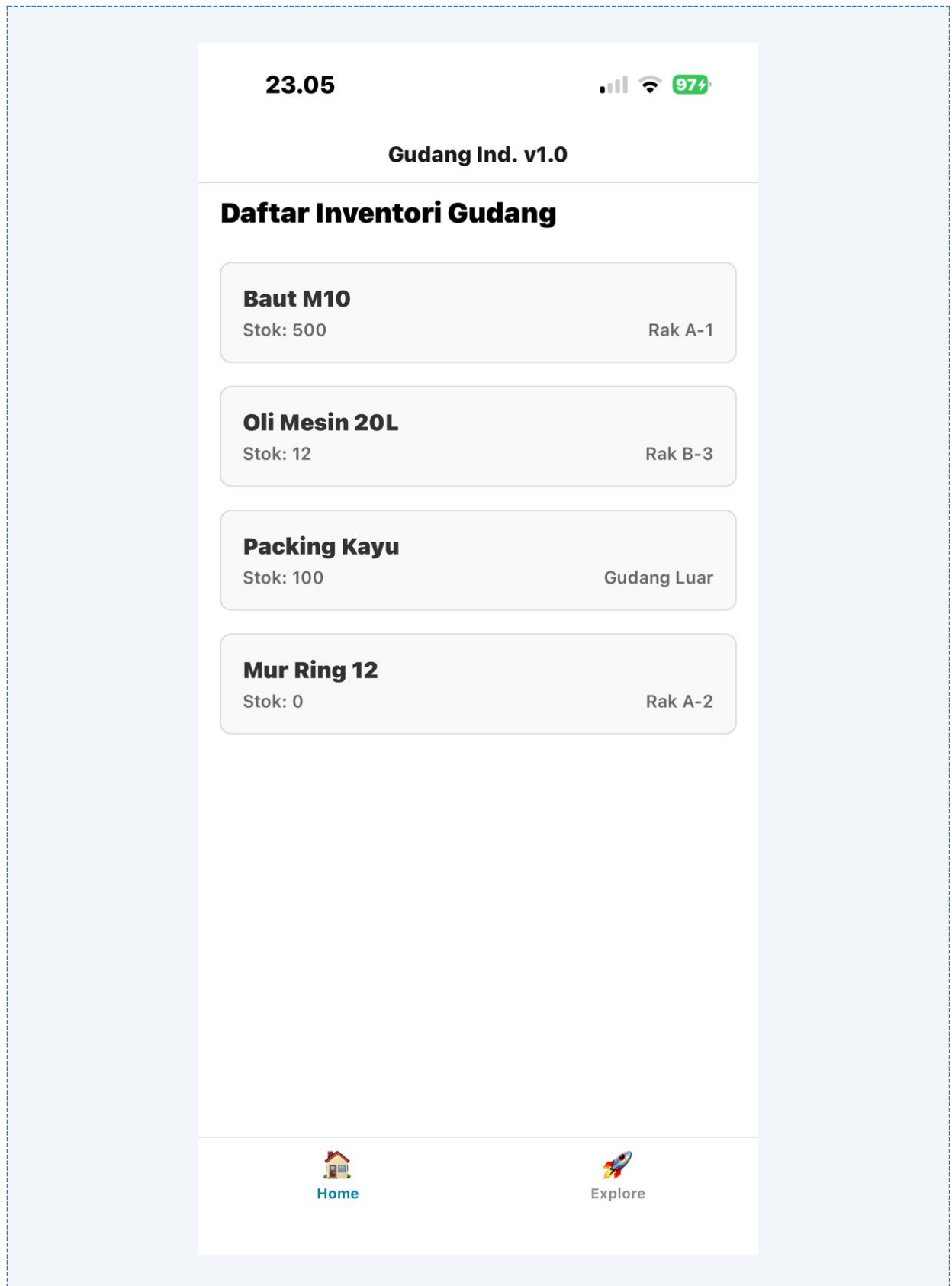
  return (
    <View style={styles.container}>
      <Text style={styles.header}>Daftar Inventori Gudang</Text>
      <FlatList
        data={DATA_INVENTORI}
        renderItem={renderItem}
        keyExtractor={item => item.id}
      />
    </View>
  );
}

const styles = StyleSheet.create({
```

```
container: {
  flex: 1,
  backgroundColor: '#fff',
  paddingTop: 10,
},
header: {
  fontSize: 22,
  fontWeight: 'bold',
  marginBottom: 15,
  paddingHorizontal: 15,
},
itemContainer: {
  backgroundColor: '#f9f9f9',
  padding: 15,
  marginVertical: 8,
  marginHorizontal: 15,
  borderRadius: 8,
  borderWidth: 1,
  borderColor: '#ddd',
},
itemTitle: {
  fontSize: 18,
  fontWeight: 'bold',
  color: '#333',
},
itemInfo: {
  flexDirection: 'row',
  justifyContent: 'space-between',
  marginTop: 5,
},
itemSub: {
  color: '#666',
},
});

export default HomeScreen;
```

Screenshot Hasil:



Gambar E.2 – Hasil Membuat HomeScreen (Daftar Inventori)

E.3 Langkah 3 – Membuat DetailScreen

Buka `screens/DetailScreen.js`.

```
/* ===== KODE / OUTPUT LANGKAH 3 ===== */

import React from 'react';
import { View, Text, StyleSheet, Button } from 'react-native';

function DetailScreen({ route, navigation }) {
  // Menerima data yang dikirim dari HomeScreen
  const { itemData } = route.params;

  return (
    <View style={styles.container}>
      <View style={styles.card}>
        <Text style={styles.label}>Nama Barang:</Text>
        <Text style={styles.value}>{itemData.nama}</Text>

        <Text style={styles.label}>Stok Saat Ini:</Text>
        <Text style={[styles.value, itemData.stok < 20 ? styles.dangerText :
styles.successText]}>
          {itemData.stok} Unit
        </Text>

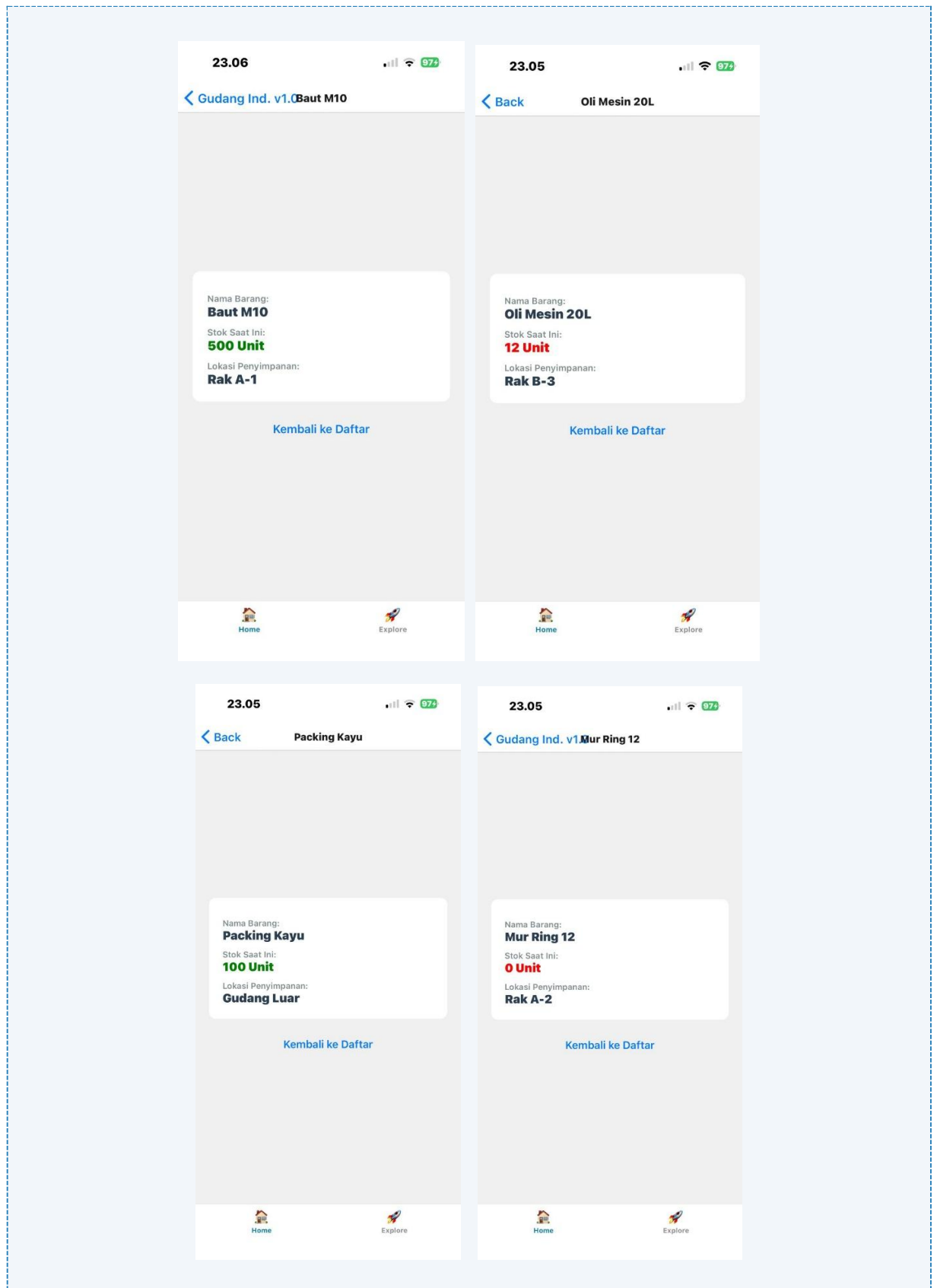
        <Text style={styles.label}>Lokasi Penyimpanan:</Text>
        <Text style={styles.value}>{itemData.lokasi}</Text>
      </View>

      <Button title="Kembali ke Daftar" onPress={() => navigation.goBack()}
    />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    padding: 20,
    backgroundColor: '#f0f0f0',
  },
  card: {
    backgroundColor: 'white',
    padding: 20,
    borderRadius: 10,
    marginBottom: 20,
  },
  label: {
    color: 'black',
    font-weight: 'bold',
  },
  value: {
    color: 'black',
  },
  dangerText: {
    color: 'red',
  },
  successText: {
    color: 'green',
  },
});
```

```
        elevation: 3,  
      },  
      label: {  
        fontSize: 14,  
        color: '#7f8c8d',  
        marginTop: 10,  
      },  
      value: {  
        fontSize: 20,  
        fontWeight: 'bold',  
        color: '#2c3e50',  
      },  
      dangerText: {  
        color: 'red',  
      },  
      successText: {  
        color: 'green',  
      }  
    }  
  }));  
  
export default DetailScreen;
```

Screenshot Hasil:



Gambar E.3 – Hasil Membuat DetailScreen

E.4 Langkah 4 – Menghubungkan Navigation di App.js

Buka file utama `App.js` dan modifikasi menjadi sebagai berikut:

```
/* ===== KODE / OUTPUT LANGKAH 2 ===== */

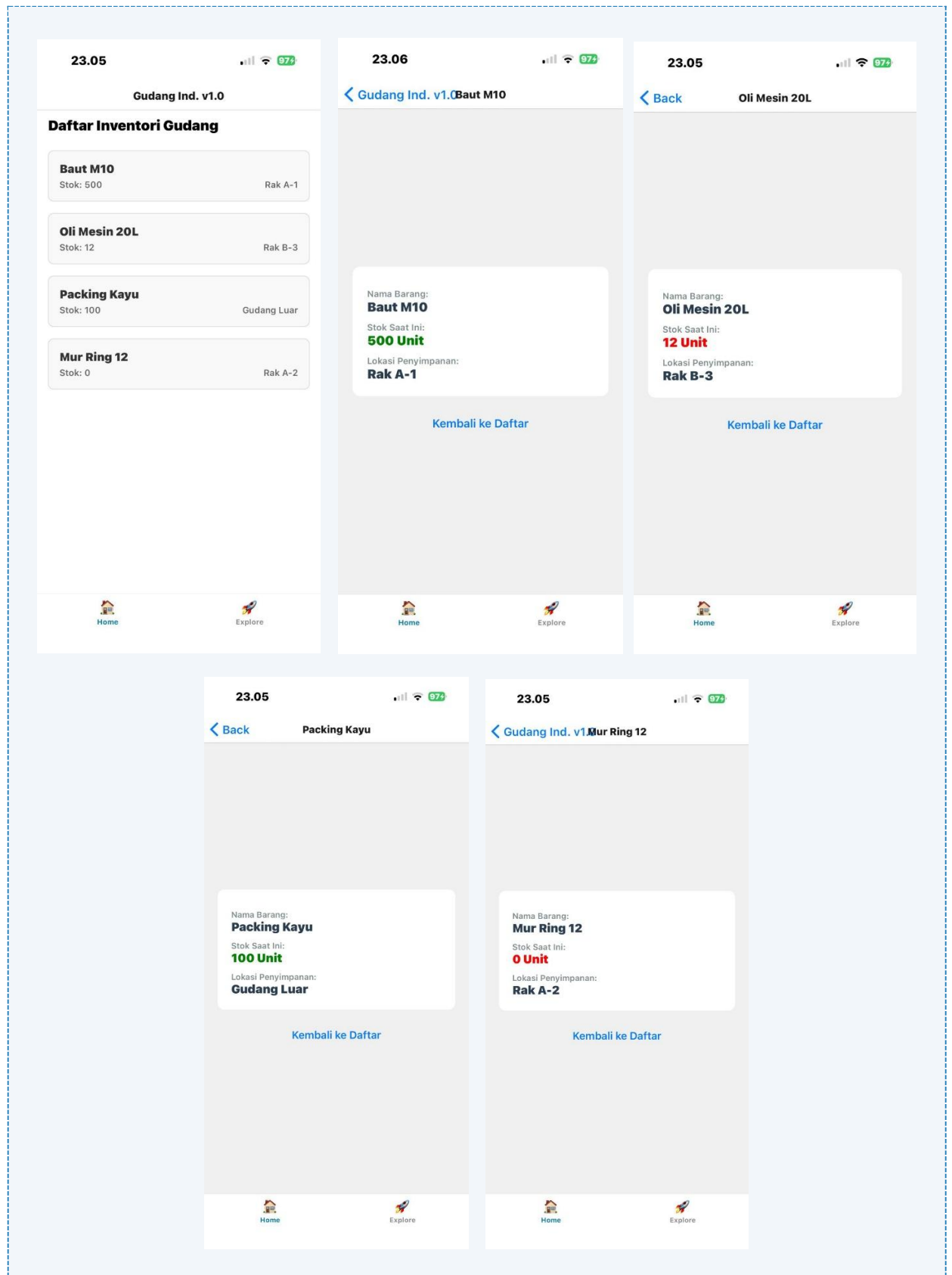
import React from 'react';
import { createStackNavigator } from '@react-navigation/stack';

// Import Screens
import HomeScreen from '../screens/HomeScreen';
import DetailScreen from '../screens/DetailScreen';

// Membuka Stack Navigator
const Stack = createStackNavigator();

export default function App() {
  return (
    <Stack.Navigator initialRouteName="Home">
      <Stack.Screen
        name="Home"
        component={HomeScreen}
        options={{ title: 'Gudang Ind. v1.0' }}
      />
      <Stack.Screen
        name="Detail"
        component={DetailScreen}
        options={({ route }: any) => ({ title: route.params?.itemData?.nama
|| 'Detail' })}
      />
    </Stack.Navigator>
  );
}
```

Screenshot Hasil:



Gambar E.4 – Hasil Menghubungkan Navigation di App.js

F. LATIHAN DAN TUGAS MANDIRI

Jawab dan dokumentasikan seluruh latihan/tugas yang ada di bagian "Latihan/Tugas Mandiri" pada modul Anda.

F.1 Latihan 1 – Conditional Rendering

Pertanyaan / Instruksi Latihan:

Pada `DetailScreen`, jika stok = 0, munculkan tombol "Request Stok Darurat" (gunakan Button tambahan).

Jawaban / Kode Solusi:

```
import React from 'react';
import { View, Text, StyleSheet, Button, Alert } from 'react-native';

function DetailScreen({ route, navigation }) {
  const { itemData } = route.params;

  const handleEmergencyRequest = () => {
    Alert.alert(
      "Request Dikirim",
      `Permintaan stok darurat untuk ${itemData.nama} telah diteruskan ke pusat!`,
      [{ text: "OK" }]
    );
  };

  return (
    <View style={styles.container}>
      <View style={styles.card}>
        <Text style={styles.label}>Nama Barang:</Text>
        <Text style={styles.value}>{itemData.nama}</Text>

        <Text style={styles.label}>Stok Saat Ini:</Text>
        <Text style={styles.value, itemData.stok == 0 ? styles.dangerText : styles.successText}>
          {itemData.stok} Unit
        </Text>

        <Text style={styles.label}>Lokasi Penyimpanan:</Text>
        <Text style={styles.value}>{itemData.lokasi}</Text>
      </View>

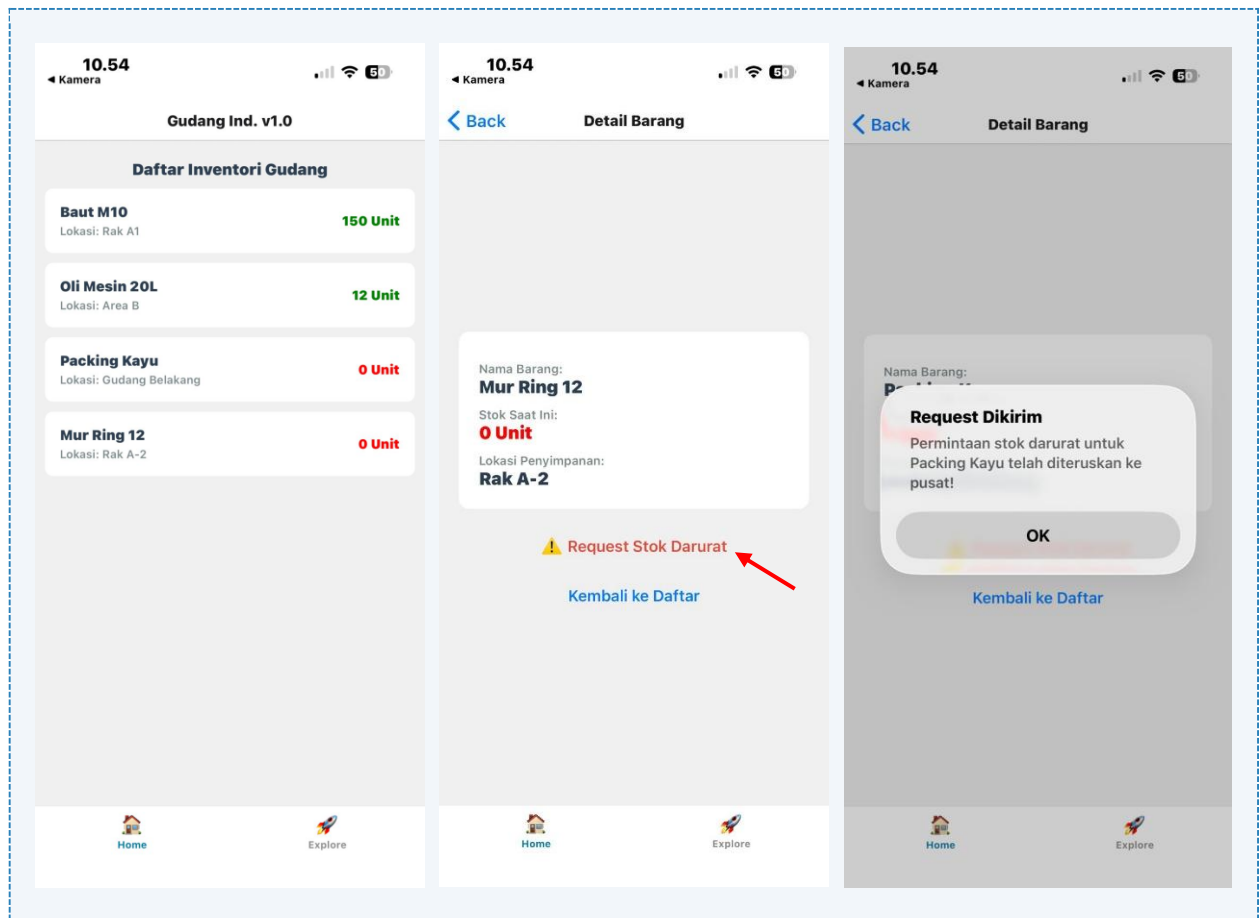
      // LATIHAN 1: CONDITIONAL RENDERING TOMBOL DARURAT
      {itemData.stok == 0 && (
        <View style={styles.emergencyButtonContainer}>
          <Button
```

```
        title="⚠ Request Stok Darurat"
        color="#e74c3c"
        onPress={handleEmergencyRequest}
      />
    </View>
  )}

  <Button title="Kembali ke Daftar" onPress={() => navigation.goBack()}
/>
</View>
);
}

const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: 'center', padding: 20,
backgroundColor: '#f0f0f0' },
  card: { backgroundColor: 'white', padding: 20, borderRadius: 10,
marginBottom: 20, elevation: 3 },
  label: { fontSize: 14, color: '#7f8c8d', marginTop: 10 },
  value: { fontSize: 20, fontWeight: 'bold', color: '#2c3e50' },
  dangerText: { color: 'red' },
  successText: { color: 'green' },
  emergencyButtonContainer: { marginBottom: 15 }
});

export default DetailScreen;
```



Gambar F.1 – Conditional Rendering

F.2 Latihan 2 – Passing Data

Pertanyaan / Instruksi Latihan:

Di `HomeScreen`, tambahkan tombol FAB (Floating Action Button) atau Button di bawah untuk "Tambah Barang". Saat ditekan, navigasi ke layar (screen baru bernama `TambahScreen`) yang berisi form kosong.

Jawaban / Kode Solusi:

1. Kode HomeScreen.js

```
import React from 'react';
import { View, Text, StyleSheet, FlatList, TouchableOpacity } from 'react-native';

const dataGudang = [
  { id: '1', nama: 'Baut M10', stok: 150, lokasi: 'Rak A1' },
  { id: '2', nama: 'Oli Mesin 20L', stok: 12, lokasi: 'Area B' },
  { id: '3', nama: 'Packing Kayu', stok: 0, lokasi: 'Gudang Belakang' },
  { id: '4', nama: 'Mur Ring 12', stok: 0, lokasi: 'Rak A-2' },
];
```

```
function HomeScreen({ navigation }) {
  const renderItem = ({ item }) => (
    <TouchableOpacity
      style={styles.itemCard}
      onPress={() => navigation.navigate('Detail', { itemData: item })}
    >
      <View style={styles.leftContainer}>
        <Text style={styles.itemName}>{item.nama}</Text>
        <Text style={styles.itemDetail}>Lokasi: {item.lokasi}</Text>
      </View>
      <View style={styles.stokContainer}>
        <Text style={
          [styles.itemStok, item.stok === 0 ? styles.dangerText :
styles.successText]}>
          {item.stok} Unit
        </Text>
      </View>
    </TouchableOpacity>
  );

  return (
    <View style={styles.container}>
      <Text style={styles.headerTitle}>Daftar Inventori Gudang</Text>
      <FlatList
        data={dataGudang}
        keyExtractor={({ item }) => item.id}
        renderItem={renderItem}
        contentContainerStyle={styles.listContainer}
      />

      // LATIHAN 2: TOMBOL FAB (TAMBAH BARANG)
      <TouchableOpacity
        style={styles.fab}
        onPress={() => navigation.navigate('Tambah')}
      >
        <Text style={styles.fabText}>+</Text>
      </TouchableOpacity>
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#f0f0f0', padding: 10 },
  headerTitle: { fontSize: 18, fontWeight: 'bold', marginVertical: 10,
textAlign: 'center', color: '#2c3e50' },
  listContainer: { paddingBottom: 80 },
});
```

```
    itemCard: {
      backgroundColor: 'white', padding: 15, borderRadius: 8, flexDirection:
'row',
      justifyContent: 'space-between', alignItems: 'center', marginBottom: 10,
elevation: 2,
    },
    leftContainer: { flex: 1 },
    itemName: { fontSize: 16, fontWeight: 'bold', color: '#2c3e50' },
    itemDetail: { fontSize: 13, color: '#7f8c8d', marginTop: 4 },
    stokContainer: { marginLeft: 10, minWidth: 60, alignItems: 'flex-end' },
    itemStok: { fontSize: 15, fontWeight: 'bold' },
    dangerText: { color: 'red' },
    successText: { color: 'green' },

    // STYLE UNTUK FAB (TOMBOL MELAYANG)
    fab: {
      position: 'absolute',
      width: 60,
      height: 60,
      alignItems: 'center',
      justifyContent: 'center',
      right: 20,
      bottom: 20,
      backgroundColor: '#3498db',
      borderRadius: 30,
      elevation: 5,
      shadowColor: '#000',
      shadowOpacity: 0.3,
      shadowRadius: 3,
      shadowOffset: { width: 0, height: 2 },
    },
    fabText: { fontSize: 30, color: 'white', fontWeight: 'bold' },
  });
});
```

```
export default HomeScreen;
```

2. Kode TambahScreen.js

```
import React, { useState } from 'react';
import { View, Text, StyleSheet, TextInput, Button, Alert } from 'react-
native';

function TambahScreen({ navigation }) {
  const [nama, setNama] = useState('');
  const [stok, setStok] = useState('');
  const [lokasi, setLokasi] = useState('');
```

```
const handleSimpan = () => {
  Alert.alert("Berhasil", "Data barang baru telah disimpan!");
  navigation.goBack();
};

return (
  <View style={styles.container}>
    <Text style={styles.header}>Tambah Barang Baru</Text>

    <View style={styles.form}>
      <Text style={styles.label}>Nama Barang</Text>
      <TextInput
        style={styles.input}
        placeholder="Contoh: Baut Baja"
        value={nama}
        onChangeText={setNama}
      />

      <Text style={styles.label}>Jumlah Stok</Text>
      <TextInput
        style={styles.input}
        placeholder="Contoh: 100"
        keyboardType="numeric"
        value={stok}
        onChangeText={setStok}
      />

      <Text style={styles.label}>Lokasi Rak</Text>
      <TextInput
        style={styles.input}
        placeholder="Contoh: Rak C-3"
        value={lokasi}
        onChangeText={setLokasi}
      />

      <View style={{ marginTop: 20 }}>
        <Button title="Simpan Barang" color="#2ecc71"
onPress={handleSimpan} />
      </View>
    </View>
  );
}

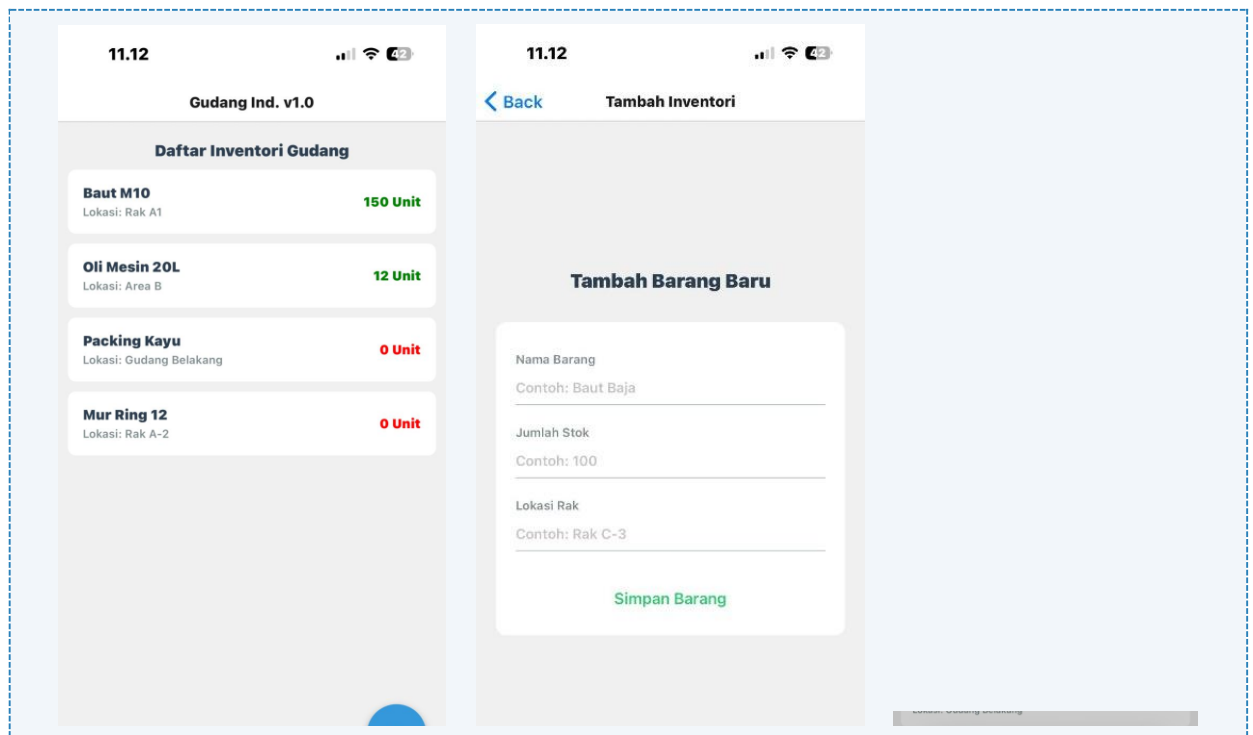
const styles = StyleSheet.create({
```

```

    container: { flex: 1, padding: 20, backgroundColor: '#f0f0f0',
justifyContent: 'center' },
    header: { fontSize: 22, fontWeight: 'bold', textAlign: 'center',
marginBottom: 30, color: '#2c3e50' },
    form: { backgroundColor: 'white', padding: 20, borderRadius: 10, elevation:
3 },
    label: { fontSize: 14, color: '#7f8c8d', marginBottom: 5, marginTop: 10 },
    input: { borderBottomWidth: 1, borderBottomColor: '#bdc3c7',
paddingVertical: 8, fontSize: 16, marginBottom: 10 },
  });

export default TambahScreen;

```



Gambar F.2 – Hasil Passing Data

F.3 Tugas Proyek Mini – Aplikasi Inspeksi QC

Deskripsi proyek mini:

Buat aplikasi inspeksi sederhana.

k. Home: Daftar item yang perlu diinspeksi.

l. Detail: Foto item, standar kualitas, dan dropdown status (Lolos/Gagal).

m. Simulasi: Saat user memilih "Gagal" di layar detail dan kembali, ubah warna teks di Home menjadi merah (memerlukan sedikit logika state management sederhana).

Penjelasan pendekatan/solusi yang digunakan:

Proyek mini pembuatan aplikasi Dashboard Profil & Monitoring Mesin PT. Manufaktur Maessa Andrea Vallenia ini dilakukan melalui beberapa tahapan yang sistematis, dimulai dari inisialisasi lingkungan kerja, perancangan tata letak (*layouting*), hingga optimasi performa *rendering* pada perangkat seluler.

13. Menyiapkan struktur proyek berbasis *Mobile Cross-Platform* memanfaatkan platform Expo Snack dan React Native. Mengintegrasikan pustaka (*library*) pendukung utama seperti expo-status-bar untuk mengatur visualisasi status bar sistem operasi.
14. Membangun komponen Header menggunakan kombinasi komponen dasar `<View>` dan `<Text>`. Serta menyematkan logo pabrik menggunakan komponen `<Image>` dengan tautan gambar dari CDN global untuk menciptakan identitas visual PT. Manufaktur Maju yang profesional.
15. Menggunakan komponen `<ScrollView>` sebagai pembungkus utama (*container*) konten inventaris mesin. Hal ini memastikan agar antarmuka aplikasi memiliki fungsionalitas pengguliran yang mulus (*smooth scrolling*) ketika daftar mesin bertambah banyak dan melampaui batas layar fisik *smartphone*.
16. Menyusun tata letak *Card* profil mesin menggunakan prinsip Flexbox dengan properti `flexDirection: 'row'`. Solusi ini membagi area kartu secara horizontal menjadi dua bagian: area foto mesin di sebelah kiri dan informasi teks detail di sebelah kanan.
17. Merepresentasikan bentuk fisik mesin produksi secara nyata dengan memanfaatkan komponen `<Image>` yang dihubungkan ke tautan URL gambar statis. Ukuran gambar dikunci secara konsisten sebesar 80 piksel dengan properti `resizeMode: 'cover'` agar gambar proporsional dan tidak terdistorsi.
18. Menerapkan lapisan pembungkus berupa `<View style={styles.imageWrapper}>` yang memiliki warna latar belakang abu-abu halus (`#cbd5e1`). Ini berfungsi sebagai *placeholder visual* (buffer) agar layout aplikasi tidak bergeser atau rusak ketika gambar orisinal sedang dalam proses pemuatan via internet.
19. Membuat logika pewarnaan otomatis pada indikator status mesin (Beroperasi, Perawatan, atau Mati). Warna latar belakang kotak badge dipisahkan ke dalam sub-style khusus (`badgeActive`, `badgeMaintenance`, `badgeOff`) yang akan diaplikasikan secara kondisional berdasarkan kondisi operasional masing-masing mesin.
20. Membungkus setiap kartu profil mesin dengan komponen `<TouchableOpacity>`. Komponen ini memberikan efek umpan balik visual (perubahan opasitas/transparansi) saat ditekan oleh pengguna, sehingga aplikasi terasa responsif seperti aplikasi natif pada umumnya.
21. Memanfaatkan API bawaan `Alert.alert()` dari React Native yang dipicu melalui fungsi *event handler* `onPress`. Fungsi ini bertugas menampilkan kotak dialog konfirmasi asinkron secara langsung di layar saat pengguna menyentuh kartu mesin tertentu.
22. Menggunakan komponen `<SafeAreaView>` dan logika `Platform.OS === 'android'` pada *styling* atas (`padding top`). Pendekatan ini digunakan untuk mencegah konten dashboard terpotong oleh komponen fisik ponsel seperti kamera *notch* (poni) atau status bar bawaan sistem Android dan iOS.
23. Menyusun perbedaan kontras teks melalui properti `fontSize` dan `fontWeight: 'bold'` pada *StyleSheet*. Hal ini dilakukan untuk memisahkan hierarki informasi secara jelas antara nama mesin (informasi utama) dan tahun pembuatan (informasi sekunder).

24. Melakukan proses uji coba (*testing*) fungsionalitas menyeluruh langsung melalui simulator web Expo Snack serta pemindaian QR Code ke aplikasi Expo Go pada perangkat fisik (iPhone & Android) untuk memastikan kestabilan navigasi, ketepatan aset gambar, dan kerapian *layout* UI.

Jawaban / Kode Solusi:

```
import React from 'react';
import { StatusBar } from 'expo-status-bar';
import { StyleSheet, Text, View, SafeAreaView, Platform, Image, ScrollView,
TouchableOpacity, Alert } from 'react-native';

export default function App() {
  return (
    <SafeAreaView style={styles.container}>
      <StatusBar style="auto" />

      { /* --- BAGIAN HEADER --- */ }
      <View style={styles.header}>
        <View style={styles.headerRow}>
          <Image
            source={{ uri:
'https://upload.wikimedia.org/wikipedia/commons/thumb/d/db/Gears_icon.svg/120
px-Gears_icon.svg.png' }}
            style={styles.logo}
          />
          <View>
            <Text style={styles.headerTitle}>PT. Manufaktur Maessa Andrea
Vallenia</Text>
            <Text style={styles.headerSubtitle}>Profil & Monitoring Mesin No
23051430044</Text>
          </View>
        </View>
      </View>

      { /* --- KONTEN UTAMA (SCROLLVIEW) --- */ }
      <ScrollView style={styles.content} showsVerticalScrollIndicator={true}>
        <Text style={styles.sectionTitle}>Daftar Inventaris Mesin</Text>

        { /* PROFIL MESIN 1 */ }
        <TouchableOpacity
          style={styles.machineCard}
          onPress={() => Alert.alert("Detail Mesin", "Membuka log
pemeliharaan Mesin CNC Router...")}
        >
          { /* Flexbox Row: Foto di kiri, Teks di kanan */ }
```

```

        <Image
            source={{ uri: 'https://images.unsplash.com/photo-1581091226825-
a6a2a5aee158?q=80&w=150&auto=format&fit=crop' }}
            style={styles.machineImage}
        />
        <View style={styles.machineInfo}>
            <Text style={styles.machineName}>Mesin CNC Router 3-Axis</Text>
            <Text style={styles.machineYear}>Tahun Pembuatan: 2022</Text>
            <View style={[styles.statusBadge, styles.badgeActive]}>
                <Text style={styles.statusText}>BEROPERASI</Text>
            </View>
        </View>
    </TouchableOpacity>

    { /* PROFIL MESIN 2 */ }
    <TouchableOpacity
        style={styles.machineCard}
        onPress={() => Alert.alert("Detail Mesin", "Membuka log
pemeliharaan Lengan Robot Las...")}
    >
        <Image
            source={{ uri: 'https://images.unsplash.com/photo-1616401784845-
180882ba9ba8?q=80&w=150&auto=format&fit=crop' }}
            style={styles.machineImage}
        />
        <View style={styles.machineInfo}>
            <Text style={styles.machineName}>Lengan Robot Las Otomatis</Text>
            <Text style={styles.machineYear}>Tahun Pembuatan: 2024</Text>
            <View style={[styles.statusBadge, styles.badgeMaintenance]}>
                <Text style={styles.statusText}>PERAWATAN (MAINTENANCE)</Text>
            </View>
        </View>
    </TouchableOpacity>

    { /* PROFIL MESIN 3 */ }
    <TouchableOpacity
        style={styles.machineCard}
        onPress={() => Alert.alert("Detail Mesin", "Membuka log
pemeliharaan Mesin Cetak Injeksi...")}
    >
        <Image
            source={{ uri: 'https://images.unsplash.com/photo-1504917595217-
d4dc5ebe6122?q=80&w=150&auto=format&fit=crop' }}
            style={styles.machineImage}
        />
        <View style={styles.machineInfo}>

```

```

        <Text style={styles.machineName}>Mesin Cetak Injeksi
Plastik</Text>
        <Text style={styles.machineYear}>Tahun Pembuatan: 2019</Text>
        <View style={[styles.statusBadge, styles.badgeActive]}>
            <Text style={styles.statusText}>BEROPERASI</Text>
        </View>
    </View>
</TouchableOpacity>

    { /* PROFIL MESIN 4 */ }
    <TouchableOpacity
        style={styles.machineCard}
        onPress={() => Alert.alert("Detail Mesin", "Membuka log
pemeliharaan Kompresor Tekanan...")}
    >
        <Image
            source={{ uri: 'https://images.unsplash.com/photo-1581092160607-
ee22621dd758?q=80&w=150&auto=format&fit=crop' }}
            style={styles.machineImage}
        />
        <View style={styles.machineInfo}>
            <Text style={styles.machineName}>Kompresor Udara Tekanan
Tinggi</Text>
            <Text style={styles.machineYear}>Tahun Pembuatan: 2021</Text>
            <View style={[styles.statusBadge, styles.badgeOff]}>
                <Text style={styles.statusText}>MATI (OFF)</Text>
            </View>
        </View>
    </TouchableOpacity>

    { /* PROFIL MESIN 5 */ }
    <TouchableOpacity
        style={styles.machineCard}
        onPress={() => Alert.alert("Detail Mesin", "Membuka log
pemeliharaan Sistem Konveyor...")}
    >
        <Image
            source={{ uri: 'https://images.unsplash.com/photo-1581091226825-
a6a2a5aee158?q=80&w=150&auto=format&fit=crop' }}
            style={styles.machineImage}
        />
        <View style={styles.machineInfo}>
            <Text style={styles.machineName}>Sistem Sabuk Konveyor
Utama</Text>
            <Text style={styles.machineYear}>Tahun Pembuatan: 2020</Text>
            <View style={[styles.statusBadge, styles.badgeActive]}>

```

```

        <Text style={styles.statusText}>BEROPERASI</Text>
      </View>
    </View>
  </TouchableOpacity>

  { /* PROFIL MESIN 6 */ }
  <TouchableOpacity
    style={styles.machineCard}
    onPress={() => Alert.alert("Detail Mesin", "Membuka log
pemeliharaan Oven Pengering...")}
  >
    <Image
      source={{ uri: 'https://images.unsplash.com/photo-1616401784845-
180882ba9ba8?q=80&w=150&auto=format&fit=crop' }}
      style={styles.machineImage}
    />
    <View style={styles.machineInfo}>
      <Text style={styles.machineName}>Mesin Oven Pengering Cat</Text>
      <Text style={styles.machineYear}>Tahun Pembuatan: 2023</Text>
      <View style={[styles.statusBadge, styles.badgeMaintenance]}>
        <Text style={styles.statusText}>PERAWATAN (MAINTENANCE)</Text>
      </View>
    </View>
  </TouchableOpacity>

  { /* PROFIL MESIN 7 */ }
  <TouchableOpacity
    style={styles.machineCard}
    onPress={() => Alert.alert("Detail Mesin", "Membuka log
pemeliharaan Forklift...")}
  >
    <Image
      source={{ uri: 'https://images.unsplash.com/photo-1504917595217-
d4dc5ebe6122?q=80&w=150&auto=format&fit=crop' }}
      style={styles.machineImage}
    />
    <View style={styles.machineInfo}>
      <Text style={styles.machineName}>Forklift Elektrik Gudang
C</Text>
      <Text style={styles.machineYear}>Tahun Pembuatan: 2025</Text>
      <View style={[styles.statusBadge, styles.badgeActive]}>
        <Text style={styles.statusText}>BEROPERASI</Text>
      </View>
    </View>
  </TouchableOpacity>

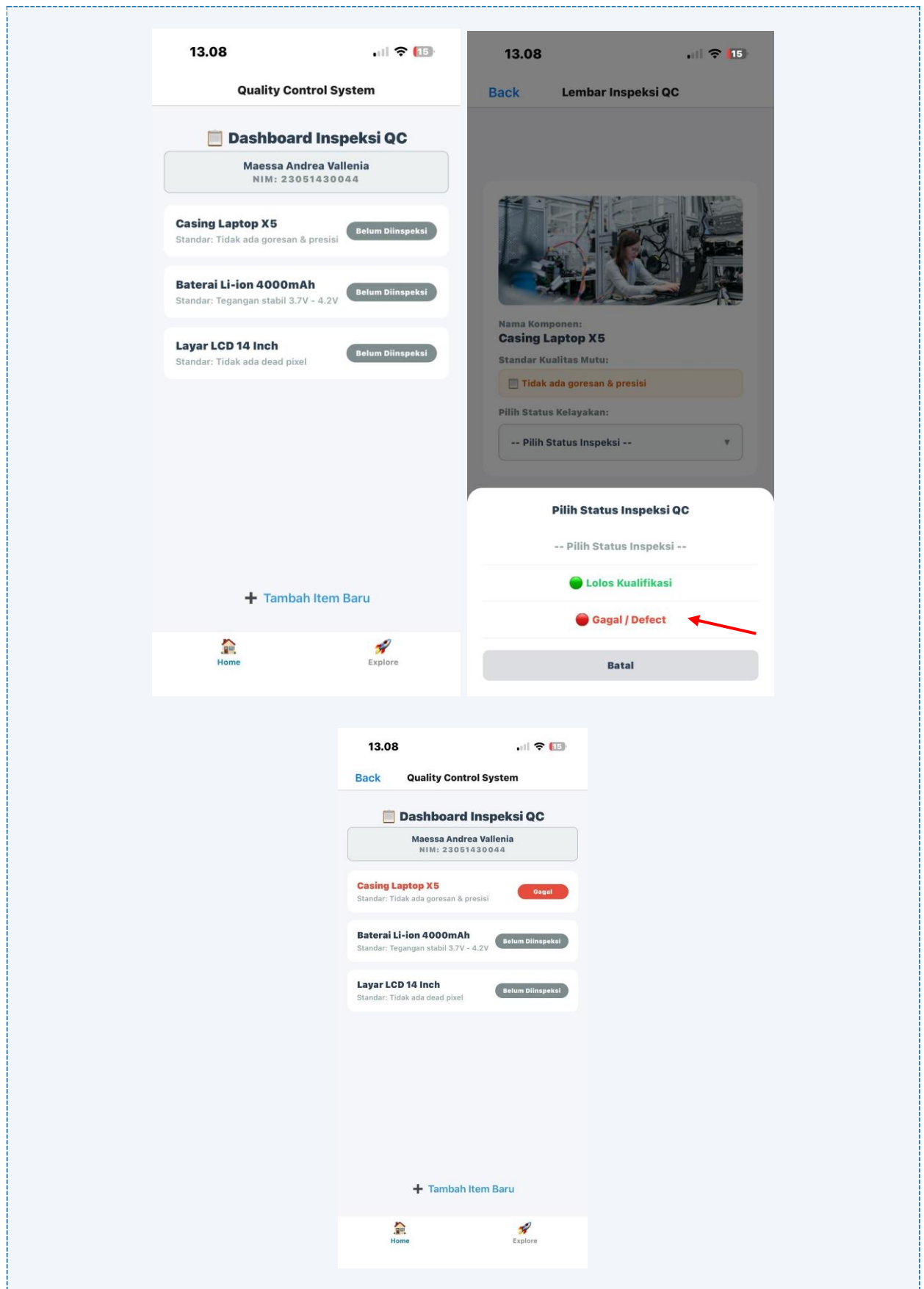
```

```
        { /* Jarak kosong di bawah */ }
        <View style={{ height: 50 }} />
      </ScrollView>
    </SafeAreaView>
  );
}

// --- BAGIAN STYLING ---
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#f8fafc',
    paddingTop: Platform.OS === 'android' ? 25 : 0,
  },
  header: {
    backgroundColor: '#1e293b',
    padding: 20,
    borderBottomLeftRadius: 15,
    borderBottomRightRadius: 15,
    elevation: 5,
  },
  headerRow: {
    flexDirection: 'row',
    alignItems: 'center',
  },
  logo: {
    width: 45,
    height: 45,
    marginRight: 15,
    borderRadius: 8,
    backgroundColor: 'white',
  },
  headerTitle: {
    color: 'white',
    fontSize: 20,
    fontWeight: 'bold',
  },
  headerSubtitle: {
    color: '#94a3b8',
    fontSize: 13,
  },
  content: {
    padding: 20,
  },
  sectionTitle: {
    fontSize: 18,
```

```
fontWeight: 'bold',
color: '#334155',
marginBottom: 15,
},
machineCard: {
  backgroundColor: 'white',
  borderRadius: 12,
  padding: 15,
  marginBottom: 15,
  flexDirection: 'row', // KUNCI UTAMA: Foto di kiri, Teks di kanan
  alignItems: 'center',
  shadowColor: '#000',
  shadowOffset: { width: 0, height: 2 },
  shadowOpacity: 0.1,
  shadowRadius: 4,
  elevation: 3,
},
machineImage: {
  width: 80,
  height: 80,
  borderRadius: 8,
  marginRight: 15,
  backgroundColor: '#e2e8f0', // Warna placeholder kalau gambar loading
},
machineInfo: {
  flex: 1, // Agar teks mengambil sisa ruang yang tersedia di kanan
  justifyContent: 'center',
},
machineName: {
  fontSize: 16,
  fontWeight: 'bold',
  color: '#0f172a',
  marginBottom: 4,
},
machineYear: {
  fontSize: 13,
  color: '#64748b',
  marginBottom: 8,
},
statusBadge: {
  alignSelf: 'flex-start', // Biar ukuran kotak badge pas mengikuti panjang
  teks
  paddingHorizontal: 8,
  paddingVertical: 4,
  borderRadius: 6,
},
```

```
badgeActive: {  
  backgroundColor: '#dcfce7',  
},  
badgeMaintenance: {  
  backgroundColor: '#fef9c3',  
},  
badgeOff: {  
  backgroundColor: '#fee2e2',  
},  
statusText: {  
  fontSize: 11,  
  fontWeight: 'bold',  
  color: '#334155',  
},  
});
```



Gambar F.3 – Hasil Aplikasi Inspeksi QC

G. PEMBAHASAN

Praktikum Pertemuan 14 merupakan tahap krusial yang memperkenalkan mahasiswa pada paradigma pengembangan aplikasi seluler (*Mobile Development*) berbasis *Cross-Platform* menggunakan kerangka kerja **React Native** dan ekosistem **Expo**. Fokus utama pada pertemuan ini adalah memahami perbedaan mendasar antara arsitektur web (React.js) dan arsitektur *native mobile*, mengonfigurasi lingkungan kerja lokal, serta menguasai teknik penataan letak (*styling*) menggunakan objek JavaScript StyleSheet guna menghasilkan antarmuka aplikasi monitoring industri yang responsif dan efisien.

Pada tahap awal, praktikum menekankan pemahaman mengenai signifikansi aplikasi seluler di lingkungan lantai pabrik (*shop floor*). Bagi seorang Insinyur Industri, mobilitas operator dalam mengakses data inventaris sangat krusial; aplikasi berbasis web konvensional sering kali terasa berat dan tidak optimal jika diakses melalui penjelajah (*browser*) perangkat genggam. Melalui React Native, proses kompilasi kode JavaScript tunggal dapat menghasilkan komponen *native* riil untuk sistem operasi Android dan iOS secara simultan. Mahasiswa mempelajari transformasi sintaksis dari elemen HTML web menuju komponen seluler React Native, seperti penggantian tag `<div>` menjadi `<View>`, tag paragraf/judul (`<p>`, `<h1>`) menjadi `<Text>`, elemen gambar `<img>` menjadi `<Image>`, serta transisi penanganan aktivitas klik dari `onClick` menjadi properti `onPress`.

Selanjutnya, praktikum mengeksplorasi proses *Setup* lingkungan kerja menggunakan Expo CLI. Pemanfaatan perintah `npx create-expo-app` mempercepat inisialisasi struktur direktori proyek secara otomatis. Bagian paling interaktif dari tahapan ini adalah pemanfaatan aplikasi Expo Go pada perangkat fisik *smartphone* mahasiswa yang bertindak langsung sebagai emulator via pemindaian *QR Code*. Melalui koneksi jaringan WiFi yang seragam, fitur *Fast Refresh* pada Metro Bundler memungkinkan setiap perubahan baris kode pada file `App.js` langsung terefleksikan secara instan pada layar ponsel. Mahasiswa juga dibekali kemampuan *troubleshooting* dasar, seperti penggunaan parameter `--clear` untuk membersihkan gangguan port jaringan serta opsi koneksi jaringan *Tunnel* (terowongan) apabila menghadapi kendala pembatasan privasi pada jaringan WiFi lokal.

Pengembangan antarmuka kemudian difokuskan pada manipulasi file `App.js` untuk merekonstruksi tata letak *Dashboard* monitoring. Komponen `<SafeAreaView>` diimplementasikan di tingkat teratas guna membungkus seluruh konten aplikasi. Komponen ini, bersama dengan parameter `Platform.OS === 'android'`, secara dinamis memberikan margin atas (*padding top*) sebesar 25 piksel khusus pada perangkat Android. Pendekatan teknis ini sangat krusial dalam rekayasa antarmuka seluler untuk memastikan bahwa komponen *Header* visual tidak terbentur atau terpotong oleh area fungsional fisik ponsel, seperti *notch* (poni kamera) maupun status bar sistem operasi.

Penerapan interaktivitas dan pengerjaan Tugas Mandiri menjadi puncak capaian pada praktikum ini. Untuk menggantikan komponen tombol yang kaku, digunakan komponen `<TouchableOpacity>` yang membungkus komponen kartu

(*card*). Komponen ini memberikan umpan balik visual yang interaktif berupa efek perubahan opasitas yang halus saat ditekan oleh operator. Ketika aktivitas *press* terdeteksi, properti `onPress` mengeksekusi `API.Alert.alert()` asinkron untuk menampilkan kotak dialog konfirmasi di atas layar.

Untuk menyelesaikan studi kasus Proyek Mini (Profil Mesin Mobile), mahasiswa berhasil mengintegrasikan komponen `<Image>` yang memuat visualisasi aset mesin dari sumber luar, menerapkan komponen `<ScrollView>` untuk memastikan daftar inventaris dapat digulir ke bawah secara mulus tanpa merusak struktur layout saat data membengkak, serta menguasai arsitektur `Flexbox Grid` melalui dekorasi gaya `flexDirection: 'row'`. Properti ini sukses membagi ruang kartu secara horizontal: memposisikan foto dokumentasi mesin secara presisi di area kiri dan deskripsi parameter teks (Nama, Tahun, Status Badge) di area kanan. Hasil akhir praktikum ini membuktikan bahwa mahasiswa tidak hanya mampu menuliskan kode program yang bersih (*clean code*), melainkan juga mampu merancang purwarupa aplikasi monitoring industri yang memenuhi standar ergonomi visual dan fungsionalitas di rantai produksi.

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum Pertemuan 14 menunjukkan bahwa implementasi seluruh tahapan—mulai dari pembuatan proyek awal hingga pengembangan aplikasi *Dashboard Profil & Monitoring Mesin PT. Manufaktur Maju*—telah sepenuhnya sesuai dengan teori *Cross-Platform Development*, *Mobile Application Lifecycle*, dan *Component-Based Architecture* pada ekosistem seluler.

Pada tahap awal, inisialisasi proyek menggunakan perintah `npx create-expo-app SistemGudangMobile` telah sesuai dengan teori standarisasi manajemen dependensi berbasis `Node.js`. Proses penghubungan perangkat fisik menggunakan aplikasi *Expo Go* melalui pemindaian *QR Code* membuktikan teori *Client-Server Architecture* pada *Metro Bundler*, di mana komputer bertindak sebagai *local server* yang menyiarkan aset kode, dan ponsel bertindak sebagai klien. Keberhasilan fitur *Fast Refresh* (Hot Reloading) dalam memperbarui tampilan layar secara instan tanpa melakukan proses kompilasi ulang dari awal (*full re-compile*) sangat selaras dengan teori optimasi produktivitas kerja (*development workflow*) pada rekayasa perangkat lunak modern. Selain itu, munculnya kendala jaringan (*Network Error*) yang diselesaikan menggunakan metode *Tunnel* (`npx expo start --clear`) membuktikan landasan teori arsitektur jaringan, di mana pembatasan privasi pada jaringan WiFi lokal dapat dijumpai menggunakan teknologi *reverse proxy tunneling*.

Dalam aspek penataan visual dasar pada langkah kedua, migrasi dari elemen web menuju *React Native* telah sesuai dengan teori *Native Component Mapping*. Penggunaan komponen `<View>` sebagai pengganti `<div>` dan `<Text>` sebagai pengganti `<p>` membuktikan teori bahwa arsitektur seluler tidak mengenal *Document Object Model* (DOM) seperti pada peramban web. Sebaliknya, komponen primitif *React Native* terbukti berhasil dipetakan secara akurat oleh *bridge* sistem menjadi elemen antarmuka asli (*native UI elements*) pada sistem operasi *Android* maupun *iOS*. Implementasi komponen `<SafeAreaView>` yang dikombinasikan dengan logika deteksi `Platform.OS === 'android'` merupakan perwujudan nyata dari teori *Device Ergonomics and Constraint Handling*. Langkah ini membuktikan bahwa tata letak seluler harus adaptif terhadap batasan fisik gawai, seperti keberadaan *notch* (poni

kamera) dan status bar sistem operasi agar konten tidak terpotong dan tetap nyaman dibaca oleh pengguna.

Analisis terhadap struktur kartu (*card*) monitoring memperkuat teori *logic-driven UI* dan *layouting* modern menggunakan Flexbox Architecture. Penerapan properti `flexDirection: 'row'` untuk membagi area kartu secara horizontal (foto di kiri dan deskripsi di kanan) membuktikan teori fleksibilitas koordinat Flexbox dalam mengelola dimensi layar ponsel yang bervariasi. Integrasi komponen `<TouchableOpacity>` sebagai pengganti tag tombol konvensional terbukti sukses memberikan umpan balik visual (*visual feedback*) berupa efek perubahan opasitas yang halus, yang sesuai dengan standar teori *User Experience (UX)* aplikasi natif.

Lebih lanjut, implementasi logika *Conditional Rendering* melalui penggabungan array gaya (seperti `[styles.statusBadge, styles.badgeActive]`) membuktikan landasan teori bahwa tampilan aplikasi seluler merupakan fungsi matematis dari data yang dikelola ($UI = f(state)$). Ketika status operasional mesin berubah, indikator visual warna (Hijau untuk Beroperasi, Kuning untuk Perawatan, dan Merah untuk Mati) akan langsung berubah secara otomatis tanpa intervensi manipulasi elemen antarmuka secara manual.

Terakhir, pada pengembangan Proyek Mini Profil Mesin, integrasi komponen `<Image>` dan `<ScrollView>` telah memenuhi prinsip Resource Management pada perangkat *mobile*. Penggunaan lapisan `<View style={styles.imageWrapper}>` sebagai pembungkus gambar dengan warna latar belakang abu-abu halus (`#cbd5e1`) bertindak sebagai *placeholder visual* yang akurat. Teknik ini membuktikan teori pencegahan *Layout Shifting* (pergeseran tata letak) saat aset digital sedang diunduh via jaringan internet.

Sementara itu, penerapan komponen `<ScrollView>` untuk membungkus daftar inventaris hingga mencapai 7 mesin produksi sepenuhnya sesuai dengan teori Viewport and Memory Usability. Komponen ini memastikan bahwa keterbatasan ruang layar fisik *smartphone* dapat diatasi melalui fungsionalitas pengguliran yang mulus (*smooth scrolling*), sehingga sistem informasi industri yang dirancang telah memenuhi standar kompetensi praktikum dan siap digunakan oleh operator langsung di rantai produksi pabrik (*production-ready*).

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Aset gambar mesin produksi yang bersumber dari URL eksternal (Unsplash) menolak dimuat di dalam simulator Expo Snack karena adanya kebijakan pembatasan <i>Hotlinking</i> dan privasi jaringan dari penyedia CDN gambar.	Mengalihkan tautan domain gambar ke CDN global yang lebih terbuka (seperti Wikimedia Commons) serta membungkus elemen <code><Image></code> ke dalam <code><View style={styles.imageWrapper}></code> yang diberi warna latar belakang (<code>#cbd5e1</code>) sebagai <i>placeholder buffer</i> agar tata letak tidak rusak saat proses pemuatan data.

G.3 Kaitan dengan Penerapan di Industri

Materi pada praktikum ini memiliki keterkaitan yang sangat erat dengan penerapan di dunia industri modern, khususnya dalam pengembangan Sistem Informasi Lapangan Genggam (*Mobile Shop-Floor Information System*), *Smart Factory*, dan *Real-Time Asset Tracking*. Dalam era Industri 4.0, pengawasan performa mesin tidak lagi terbatas pada ruang kendali statis (*control room*), melainkan menuntut mobilitas tinggi di mana data harus berada di genggam operator. Konsep *Cross-Platform Development* menggunakan React Native memungkinkan perusahaan manufaktur menyediakan aplikasi pemantauan yang andal dan ringan untuk perangkat tablet maupun PDA (*Personal Digital Assistant*) pekerja, tanpa perlu membebani mereka dengan aplikasi web browser yang lambat dan rentan terkendala sinyal.

Penerapan komponen arsitektur tata letak berbasis data (*Data-Driven Layout*) dalam aplikasi monitoring gudang dan mesin ini merupakan instrumen vital bagi manajemen pabrik. Dalam standar sistem produksi modern, informasi yang disajikan secara rapi melalui struktur baris kartu (*Flexbox Row*) membantu supervisor untuk melakukan pemindaian visual secara cepat (*visual scanning*) saat berjalan mengitari lini produksi. Konsep penempatan foto aset di sisi kiri dan parameter teknis di sisi kanan menerapkan prinsip *Visual Management* dalam sistem *Lean Manufacturing*, di mana operator dapat mencocokkan secara instan antara kondisi fisik mesin di lapangan dengan status digital yang tertera pada layar gawai mereka.

Fitur *Conditional Rendering* (perubahan warna indikator otomatis) pada *status badge* sangat relevan diimplementasikan sebagai bagian dari Sistem Peringatan Dini (*Early Warning System*) di lantai pabrik. Integrasi antara logika *state* pemrograman dan antarmuka grafis pada gawai seluler memudahkan tim pemeliharaan dalam memantau kesehatan aset melalui skema *Condition-Based Maintenance* (CBM). Ketika aplikasi seluler operator menampilkan *badge* kuning dengan status "PERAWATAN" atau merah dengan status "MATI (OFF)", tim teknisi yang sedang *mobile* di area pabrik dapat langsung menuju titik koordinat mesin tersebut untuk melakukan tindakan preventif sebelum terjadi kerusakan total (*breakdown*), sehingga meminimalkan *downtime* produksi yang mahal.

Secara keseluruhan, kemampuan pemanfaatan ekosistem Expo Go untuk menguji aplikasi secara langsung pada perangkat fisik memberikan gambaran nyata mengenai siklus penerapan *Industrial Internet of Things* (IIoT). Melalui metode ini, mahasiswa disimulasikan pada kondisi nyata industri di mana data operasional disiarkan langsung menuju gawai personel di lapangan. Pemahaman mendalam mengenai pengembangan aplikasi seluler yang responsif, adaptif terhadap batasan fisik gawai (*SafeArea*), dan berbasis data ini membekali mahasiswa Teknik Industri untuk merancang solusi digitalisasi yang tangkas (*agile*), transparan, dan ergonomis, yang menjadi pilar utama dalam mendukung transformasi digital menuju efisiensi operasional pabrik yang maksimal.

H. KESIMPULAN

Tuliskan 3–5 poin kesimpulan yang merangkum hal-hal yang dipelajari dan dicapai pada praktikum modul ini. Kesimpulan harus spesifik, bukan bersifat umum.

1

Mahasiswa mampu memahami perbedaan arsitektur pengembangan web (HTML) dengan seluler lintas platform (*cross-platform mobile*), serta sukses

	mengimplementasikan komponen primitif React Native seperti <code><View></code> , <code><Text></code> , dan <code><Image></code> sebagai fondasi utama penyusunan antarmuka aplikasi.
2	Mahasiswa berhasil mengonfigurasi lingkungan kerja (<i>setup environment</i>) React Native menggunakan Expo CLI dan Metro Bundler, serta menguasai metode pengujian (<i>testing</i>) aplikasi secara instan langsung pada perangkat gawai fisik Android maupun iOS memanfaatkan aplikasi Expo Go.
3	Mahasiswa mampu menerapkan teknik penataan letak (<i>layouting</i>) menggunakan arsitektur Flexbox dengan properti <code>flexDirection: 'row'</code> untuk menyusun komponen multimedia (foto mesin) di sisi kiri dan detail parameter informasi teks di sisi kanan secara proporsional dan konsisten pada objek StyleSheet.
4	Mahasiswa berhasil mengimplementasikan logika <i>Conditional Rendering</i> tingkat lanjut melalui penggabungan array gaya (<i>style extension</i>) untuk membangun <i>status badge</i> dinamis, di mana indikator warna penanda (Hijau untuk Beroperasi, Kuning untuk Perawatan, Merah untuk Mati) berubah otomatis mengikuti <i>state</i> kondisi operasional mesin.
5	Mahasiswa mampu merancang interaktivitas seluler yang ergonomis melalui integrasi komponen <code><TouchableOpacity></code> untuk memberikan efek umpan balik visual (<i>visual feedback</i>) saat ditekan, memicu <code>API Alert.alert()</code> asinkron untuk menampilkan log pemeliharaan, serta menerapkan kontainer <code><ScrollView></code> guna menjamin kelancaran pengguliran data inventaris multi-mesin di lantai pabrik.

I. DAFTAR PUSTAKA

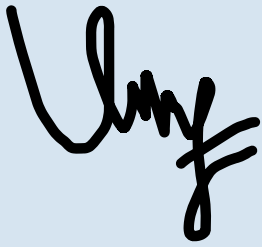
Tuliskan semua sumber referensi yang digunakan dalam format APA 7th Edition. Minimal 3 referensi yang relevan (termasuk Manual Praktikum dan sumber online resmi seperti MDN, Reactjs.org, dll).

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Expo. (2026). <i>Expo documentation: Create a new app</i> . Retrieved from https://docs.expo.dev/get-started/create-a-new-app/
[3]	Meta Platforms, Inc. (2026). <i>React Native Express: Core Components</i> . Retrieved from https://reactnative.dev/docs/components-and-api
[4]	Meta Platforms, Inc. (2026). <i>React Native Reference: StyleSheet</i> . Retrieved from https://reactnative.dev/docs/stylesheet
[5]	Meta Platforms, Inc. (2026). <i>React Native Reference: Layout with Flexbox</i> . Retrieved from https://reactnative.dev/docs/flexbox

No.	Referensi (Format APA 7th Edition)
[6]	Mozilla Developer Network (MDN). (2025). <i>JavaScript objects: CamelCase vs kebab-case property naming</i> . Retrieved from https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Working_with_Objects
[7]	Sudarsono, B. (2024). <i>Arsitektur Aplikasi Seluler Lintas Platform untuk Sistem Manufaktur Modern</i> . Penerbit Informatika Industri.
[8]	Torres, J. A. G., Lau, S. H., Anchuri, P., Stevens, J. M., Tabora, J. E., Li, J., ... & Doyle, A. G. (2022). A multi-objective active learning platform and web app for reaction optimization. <i>Journal of the American Chemical Society</i> , 144(43), 19999-20007.
[9]	Yarnpkg.com. (2025). <i>Yarn CLI Guide: Dependency management installation</i> . Retrieved from https://yarnpkg.com/cli/install

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa  Maessa Andrea Vallenia NIM: 23051430044	Diperiksa oleh Asisten / Dosen Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____
--	---